STUDY GUIDE

Java Foundations to Algorithm Mastery

Complete Beginner-to-Advanced Study Guide

83 lessons in six modules — from installing Java to Manacher's algorithm and the Knight's tour — each following the same format: objective, intuition, syntax, worked example, dry run, complete code with walkthrough, complexity, common mistakes, practice, quiz, video and a revision card.

374 Java programs (242 exercises, 363 quiz questions, 83 step-by-step dry-run traces), all compiled for Java 17+ and run against their expected output.

Built from your course archive: all 38 slide decks are covered; where a slide disagreed with a verified result, the correction is explained.

Generated 10 October 2026 from the same lesson data that powers the website.
Every Java program in this guide was compiled with --release 17 and run against its expected output.

Contents

How to use this guide 5
Study plans 7

Part 0 · Programming & Java Setup 12
0.1 How Java Works: Source, Bytecode, JVM, JRE, JDK 13
0.2 The Java Landscape: History, Features, Editions, JVM Memory 19
0.3 Installing Java on macOS and Choosing an Editor 25
0.4 Your First Program: class, main, statements, comments 30
0.5 Compile-time, Runtime and Logic Errors 37
0.6 What Is an Algorithm? 43
Part 0 — Solutions to the exercises 49
Part 0 — Quiz answer key 53

Part 1 · Java Syntax & Fundamentals 55
1.1 Variables, Constants and Primitive Types 56
1.2 Type Conversion, Casting and Overflow 62
1.3 Arithmetic, Assignment and Increment Operators 68
1.4 Relational, Logical and Ternary Operators; Precedence 73
1.5 Bitwise Operators and Shifts 79
1.6 Output: print, println, printf and String Concatenation 85
1.7 Input: Scanner, BufferedReader and Command-Line Arguments 91
1.8 Decision Making: if, else, switch 98
1.9 Loops: for, while, do-while, break and continue 105
1.10 Methods: Parameters, Return Values, Scope and Overloading 112
1.11 Tracing and Debugging Programs 119
Part 1 — Solutions to the exercises 126
Part 1 — Quiz answer key 138

Part 2 · Core Programming Skills 141
2.1 One-Dimensional Arrays 142
2.2 Two-Dimensional Arrays (Matrices) 149
2.3 Strings, StringBuilder and Characters 156
2.4 Number Systems, Binary and Two's Complement 163
2.5 Divisibility, GCD/LCM, Primes and Modular Arithmetic 170
2.6 Recursion: Base Case, Recursive Case and the Call Stack 176
2.7 References, Pass-by-Value and Memory 183
2.8 Boundary Conditions, Validation and Common Mistakes 189
Part 2 — Solutions to the exercises 196
Part 2 — Quiz answer key 208

Part 3 · Object-Oriented Programming 210
3.1 Classes, Objects, Fields, Constructors and this 211
3.2 Encapsulation and Access Modifiers 218
3.3 Inheritance, Overriding and Polymorphism 224
3.4 Abstract Classes and Interfaces 231
3.5 static, final and Initialisation Order 237
3.6 Packages and Imports 244

3.7 Exceptions: try, catch, finally, throw and throws 249
3.8 Generics and Lists: ArrayList and LinkedList 255
3.9 Sets and Maps: HashSet, TreeSet, HashMap, TreeMap 262
3.10 Queues, Deques, Stacks and PriorityQueue 268
3.11 Sorting with Comparable and Comparator 275
3.12 equals(), hashCode() and Immutability 281
Part 3 — Solutions to the exercises 287
Part 3 — Quiz answer key 308

Part 4 · Data Structures & Algorithm Foundations 311
4.1 Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables 312
4.2 Trees, Graphs and Heaps 320
4.3 Searching: Linear and Binary Search 328
4.4 Elementary Sorting: Bubble, Insertion, Selection; Stability 334
4.5 Merge Sort and Divide and Conquer 340
4.6 Iteration versus Recursion 347
4.7 Time Complexity and Big-O Notation 354
4.8 Algorithm Design Paradigms 361
4.9 Analysing Loops: Counting Iterations 368
4.10 Analysing Recursion: Recurrence Relations 376
4.11 Space Complexity and the Call Stack 383
4.12 Best, Average and Worst Case 390
Part 4 — Solutions to the exercises 397
Part 4 — Quiz answer key 419

Part 5 · Archive Algorithm Mastery 422
5.1 Euclid's Algorithm (GCD) 424
5.2 Euler's Phi (Totient) Function 433
5.3 Chinese Remainder Theorem 441
5.4 Combinations (nCr) 448
5.5 Karatsuba's Multiplication 457
5.6 Booth's Algorithm (signed binary multiplication) 465
5.7 Simple Sieve of Eratosthenes 473
5.8 Segmented & Incremental Sieve 480
5.9 Binary Palindrome 488
5.10 Swap Two Nibbles in a Byte 495
5.11 Longest Sequence of 1s (and after flipping a bit) 501
5.12 Josephus Problem 509
5.13 Block Swap Algorithm (Array Rotation) 517
5.14 Leaders in an Array 527
5.15 Majority Element (Boyer–Moore Voting) 534
5.16 Maximum Product Subarray 543
5.17 Maximum Equilibrium Sum 551
5.18 Maximum Sum of an Hourglass in a Matrix 558
5.19 Toggle the Switch (100 Doors) 564
5.20 Alice's Apple Tree (Minimum Garden Perimeter) 571

5.21 Weighted Substring (Smallest String of a Given Weight) . . . 577
5.22 Move Hyphen to the Beginning . . . 584
5.23 Strobogrammatic Number . . . 591
5.24 Lexicographically First Palindromic Rearrangement . . . 598
5.25 Manacher's Algorithm (Longest Palindromic Substring) . . . 604
5.26 Natural Sort Order . . . 616
5.27 Maneuvering (Counting Grid Paths) . . . 622
5.28 Sorted Distinct Permutations . . . 631
5.29 N-Queens (Backtracking) . . . 639
5.30 Maze Solving (Rat in a Maze) . . . 648
5.31 Hamiltonian Cycle . . . 657
5.32 Knight's Tour & Warnsdorff's Rule . . . 663
5.33 Selection Sort . . . 673
5.34 Quick Sort . . . 679
Part 5 — Solutions to the exercises . . . 688
Part 5 — Quiz answer key . . . 753

Appendices . . . 761
Appendix A — Cheat sheets . . . 762
Appendix B — Core facts to remember . . . 766
Appendix C — Video and reading library . . . 767
Appendix D — Archive coverage checklist . . . 775
Appendix E — Where the slides were corrected . . . 777

How to use this guide

This is a self-contained path from "I have never written Java" to "I can design, analyse and implement every algorithm in my course archive". It is organised in six parts that you can read in order, or enter at the level you need.

Part	Module	Lessons	Reading time	Essential
0	Programming & Java Setup — How Java runs, installing it, your first program	6	2.9 h	5/6
1	Java Syntax & Fundamentals — Variables, operators, input/output, control flow, methods	11	6.3 h	11/11
2	Core Programming Skills — Arrays, strings, binary numbers, math, recursion, memory	8	5.4 h	8/8
3	Object-Oriented Programming — Classes, interfaces, exceptions, collections, comparators	12	7.1 h	4/12
4	Data Structures & Algorithm Foundations — Structures, searching, sorting, paradigms, Big-O	12	8.4 h	8/12
5	Archive Algorithm Mastery — Every algorithm and problem from your course slides	34	25.8 h	19/34

The four kinds of knowledge

- **Essential Java syntax**
- **Core programming & data structures**
- **Archive algorithm knowledge**
- **Optional enrichment**

Every lesson header names its kind, so you can decide what to skip when time is short: **Essential Java syntax** and **Core programming & data structures** are the base; **Archive algorithm knowledge** is what your course slides test; **Optional enrichment** goes beyond the slides.

How each lesson is laid out

- **You will be able to…** — measurable objectives, then the prerequisites (clickable).
- **The idea / Syntax / Tiny example / Guided example** — intuition first, then notation, then a worked case.
- **Dry run** — a step-by-step execution trace of the algorithm with variable values.
- **Complete code** — every program is real, runnable Java; numbered badges in the margin point to the **line-by-line walkthrough** table.
- **Complexity, Common mistakes, Practice, Quick check** — exercises have hidden solutions (at the end of each part); the quiz answer key is there too.
- **Watch & read** — links to a video or a clearly-labelled YouTube search; **Revision card** — 30-second recap.

Running the code

Terminal (macOS)

```
javac Hello.java # compile (needs a JDK, version 17 or newer)
java Hello # run

# or, since Java 11, compile and run one file in one step:
java Hello.java
```

Sample input for programs that use `Scanner` is shown next to the code; pipe it in with `java Name < input.txt`.

> **AN HONEST PROMISE**
> This guide gives you a complete, verified map and enough practice to learn the material quickly — but nobody can promise mastery in a fixed number of days. How fast you progress depends on how much you code yourself. Use the quizzes and exercises to find out what you actually know, and revisit weak topics.

Study plans

Plans pack lessons in curriculum order (so prerequisites always come first), revise yesterday's and the previous days' material, and finish with a weak-topic review and a mixed assessment. The website rebuilds the plan from your actual results.

Minimum-viable path: 55 lessons (about 36 hours of reading plus practice) covering every essential syntax and core idea. **Complete path:** all 83 lessons (about 56 hours of reading).

14-day plan — minimum-viable path

Day	What to do	Minutes
1	Learn: 0.1 How Java Works: Source, Bytecode, JVM, JRE, JDK · Practise: 0.1 How Java Works: Source, Bytecode, JVM, JRE, JDK · Learn: 0.3 Installing Java on macOS and Choosing an Editor · Practise: 0.3 Installing Java on macOS and Choosing an Editor · Learn: 0.4 Your First Program: class, main, statements, comments · Practise: 0.4 Your First Program: class, main, statements, comments · Learn: 0.5 Compile-time, Runtime and Logic Errors · Practise: 0.5 Compile-time, Runtime and Logic Errors · Learn: 0.6 What Is an Algorithm? · Practise: 0.6 What Is an Algorithm? · Learn: 1.1 Variables, Constants and Primitive Types · Practise: 1.1 Variables, Constants and Primitive Types	289
2	Revise yesterday: How Java Works: Source, Bytecode, JVM, JRE, JDK, Installing Java on macOS and Choosing an Editor, Your First Program: class, main, statements, comments… · Learn: 1.2 Type Conversion, Casting and Overflow · Practise: 1.2 Type Conversion, Casting and Overflow · Learn: 1.3 Arithmetic, Assignment and Increment Operators · Practise: 1.3 Arithmetic, Assignment and Increment Operators · Learn: 1.4 Relational, Logical and Ternary Operators; Precedence · Practise: 1.4 Relational, Logical and Ternary Operators; Precedence · Learn: 1.5 Bitwise Operators and Shifts · Practise: 1.5 Bitwise Operators and Shifts	241
3	Revise yesterday: Type Conversion, Casting and Overflow, Arithmetic, Assignment and Increment Operators, Relational, Logical and Ternary Operators; Precedence… · Learn: 1.6 Output: print, println, printf and String Concatenation · Practise: 1.6 Output: print, println, printf and String Concatenation · Learn: 1.7 Input: Scanner, BufferedReader and Command-Line Arguments · Practise: 1.7 Input: Scanner, BufferedReader and Command-Line Arguments · Learn: 1.8 Decision Making: if, else, switch · Practise: 1.8 Decision Making: if, else, switch · Learn: 1.9 Loops: for, while, do-while, break and continue · Practise: 1.9 Loops: for, while, do-while, break and continue · Learn: 1.10 Methods: Parameters, Return Values, Scope and Overloading · Practise: 1.10 Methods: Parameters, Return Values, Scope and Overloading	310
4	Revise yesterday: Output: print, println, printf and String Concatenation, Input: Scanner, BufferedReader and Command-Line Arguments, Decision Making: if, else, switch… · Spaced review (3 days ago): How Java Works: Source, Bytecode, JVM, JRE, JDK, Installing Java on macOS and Choosing an Editor · Learn: 1.11 Tracing and Debugging Programs · Practise: 1.11 Tracing and Debugging Programs · Learn: 2.1 One-Dimensional Arrays · Practise: 2.1 One-Dimensional Arrays · Learn: 2.2 Two-Dimensional Arrays (Matrices) · Practise: 2.2 Two-Dimensional Arrays (Matrices) · Learn: 2.3 Strings, StringBuilder and Characters · Practise: 2.3 Strings, StringBuilder and Characters	269
5	Revise yesterday: Tracing and Debugging Programs, One-Dimensional Arrays, Two-Dimensional Arrays (Matrices)… · Spaced review (3 days ago): Type Conversion, Casting and Overflow, Arithmetic, Assignment and Increment Operators · Learn: 2.4 Number Systems, Binary and Two's Complement · Practise: 2.4 Number Systems, Binary and Two's Complement · Learn: 2.5 Divisibility, GCD/LCM, Primes and Modular Arithmetic · Practise: 2.5 Divisibility, GCD/LCM, Primes and Modular Arithmetic · Learn: 2.6 Recursion: Base Case, Recursive Case and the Call Stack · Practise: 2.6 Recursion: Base Case, Recursive Case and the Call Stack · Learn: 2.7 References, Pass-by-Value and Memory · Practise: 2.7 References, Pass-by-Value and Memory	284

Day	What to do	Minutes
6	Revise yesterday: Number Systems, Binary and Two's Complement, Divisibility, GCD/LCM, Primes and Modular Arithmetic, Recursion: Base Case, Recursive Case and the Call Stack... · Spaced review (3 days ago): Output: print, println, printf and String Concatenation, Input: Scanner, BufferedReader and Command-Line Arguments · Learn: 2.8 Boundary Conditions, Validation and Common Mistakes · Practise: 2.8 Boundary Conditions, Validation and Common Mistakes · Learn: 3.8 Generics and Lists: ArrayList and LinkedList · Practise: 3.8 Generics and Lists: ArrayList and LinkedList · Learn: 3.9 Sets and Maps: HashSet, TreeSet, HashMap, TreeMap · Practise: 3.9 Sets and Maps: HashSet, TreeSet, HashMap, TreeMap · Learn: 3.10 Queues, Deques, Stacks and PriorityQueue · Practise: 3.10 Queues, Deques, Stacks and PriorityQueue	259
7	Revise yesterday: Boundary Conditions, Validation and Common Mistakes, Generics and Lists: ArrayList and LinkedList, Sets and Maps: HashSet, TreeSet, HashMap, TreeMap... · Spaced review (3 days ago): Tracing and Debugging Programs, One-Dimensional Arrays · Learn: 3.11 Sorting with Comparable and Comparator · Practise: 3.11 Sorting with Comparable and Comparator · Learn: 4.1 Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables · Practise: 4.1 Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables · Learn: 4.3 Searching: Linear and Binary Search · Practise: 4.3 Searching: Linear and Binary Search · Learn: 4.4 Elementary Sorting: Bubble, Insertion, Selection; Stability · Practise: 4.4 Elementary Sorting: Bubble, Insertion, Selection; Stability	279
8	Revise yesterday: Sorting with Comparable and Comparator, Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables, Searching: Linear and Binary Search... · Spaced review (3 days ago): Number Systems, Binary and Two's Complement, Divisibility, GCD/LCM, Primes and Modular Arithmetic · Learn: 4.5 Merge Sort and Divide and Conquer · Practise: 4.5 Merge Sort and Divide and Conquer · Learn: 4.7 Time Complexity and Big-O Notation · Practise: 4.7 Time Complexity and Big-O Notation · Learn: 4.8 Algorithm Design Paradigms · Practise: 4.8 Algorithm Design Paradigms · Learn: 4.9 Analysing Loops: Counting Iterations · Practise: 4.9 Analysing Loops: Counting Iterations	304
9	Revise yesterday: Merge Sort and Divide and Conquer, Time Complexity and Big-O Notation, Algorithm Design Paradigms... · Spaced review (3 days ago): Boundary Conditions, Validation and Common Mistakes, Generics and Lists: ArrayList and LinkedList · Learn: 4.11 Space Complexity and the Call Stack · Practise: 4.11 Space Complexity and the Call Stack · Learn: 5.1 Euclid's Algorithm (GCD) · Practise: 5.1 Euclid's Algorithm (GCD) · Learn: 5.2 Euler's Phi (Totient) Function · Practise: 5.2 Euler's Phi (Totient) Function · Learn: 5.7 Simple Sieve of Eratosthenes · Practise: 5.7 Simple Sieve of Eratosthenes	269
10	Revise yesterday: Space Complexity and the Call Stack, Euclid's Algorithm (GCD), Euler's Phi (Totient) Function... · Spaced review (3 days ago): Sorting with Comparable and Comparator, Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables · Learn: 5.9 Binary Palindrome · Practise: 5.9 Binary Palindrome · Learn: 5.12 Josephus Problem · Practise: 5.12 Josephus Problem · Learn: 5.13 Block Swap Algorithm (Array Rotation) · Practise: 5.13 Block Swap Algorithm (Array Rotation) · Learn: 5.14 Leaders in an Array · Practise: 5.14 Leaders in an Array	279
11	Revise yesterday: Binary Palindrome, Josephus Problem, Block Swap Algorithm (Array Rotation)... · Spaced review (3 days ago): Merge Sort and Divide and Conquer, Time Complexity and Big-O Notation · Learn: 5.15 Majority Element (Boyer–Moore Voting) · Practise: 5.15 Majority Element (Boyer–Moore Voting) · Learn: 5.16 Maximum Product Subarray · Practise: 5.16 Maximum Product Subarray · Learn: 5.17 Maximum Equilibrium Sum · Practise: 5.17 Maximum Equilibrium Sum · Learn: 5.18 Maximum Sum of an Hourglass in a Matrix · Practise: 5.18 Maximum Sum of an Hourglass in a Matrix	279
12	Revise yesterday: Majority Element (Boyer–Moore Voting), Maximum Product Subarray, Maximum Equilibrium Sum... · Spaced review (3 days ago): Space Complexity and the Call Stack, Euclid's Algorithm (GCD) · Learn: 5.19 Toggle the Switch (100 Doors) · Practise: 5.19 Toggle the Switch (100 Doors) · Learn: 5.22 Move Hyphen to the Beginning · Practise: 5.22 Move Hyphen to the Beginning · Learn: 5.24 Lexicographically First Palindromic Rearrangement · Practise: 5.24 Lexicographically First Palindromic Rearrangement · Learn: 5.27 Maneuvering (Counting Grid Paths) · Practise: 5.27 Maneuvering (Counting Grid Paths)	269
13	Revise yesterday: Toggle the Switch (100 Doors), Move Hyphen to the Beginning, Lexicographically First Palindromic Rearrangement... · Spaced review (3 days ago): Binary Palindrome, Josephus Problem · Learn: 5.29 N-Queens (Backtracking) · Practise: 5.29 N-Queens (Backtracking) · Learn: 5.30 Maze Solving (Rat in a Maze) · Practise: 5.30 Maze Solving (Rat in a Maze) · Learn: 5.33 Selection Sort · Practise: 5.33 Selection Sort · Learn: 5.34 Quick Sort · Practise: 5.34 Quick Sort	319
14	Weak-topic review (from your quiz and practice results) · Final mixed assessment (archive topics)	75

14-day plan — complete path

Day	What to do	Minutes
1	Learn: 0.1 How Java Works: Source, Bytecode, JVM, JRE, JDK · Practise: 0.1 How Java Works: Source, Bytecode, JVM, JRE, JDK · Learn: 0.2 The Java Landscape: History, Features, Editions, JVM Memory · Practise: 0.2 The Java Landscape: History, Features, Editions, JVM Memory · Learn: 0.3 Installing Java on macOS and Choosing an Editor · Practise: 0.3 Installing Java on macOS and Choosing an Editor · Learn: 0.4 Your First Program: class, main, statements, comments · Practise: 0.4 Your First Program: class, main, statements, comments · Learn: 0.5 Compile-time, Runtime and Logic Errors · Practise: 0.5 Compile-time, Runtime and Logic Errors · Learn: 0.6 What Is an Algorithm? · Practise: 0.6 What Is an Algorithm? · Learn: 1.1 Variables, Constants and Primitive Types · Practise: 1.1 Variables, Constants and Primitive Types · Learn: 1.2 Type Conversion, Casting and Overflow · Practise: 1.2 Type Conversion, Casting and Overflow	381
2	Revise yesterday: How Java Works: Source, Bytecode, JVM, JRE, JDK, The Java Landscape: History, Features, Editions, JVM Memory, Installing Java on macOS and Choosing an Editor… · Learn: 1.3 Arithmetic, Assignment and Increment Operators · Practise: 1.3 Arithmetic, Assignment and Increment Operators · Learn: 1.4 Relational, Logical and Ternary Operators; Precedence · Practise: 1.4 Relational, Logical and Ternary Operators; Precedence · Learn: 1.5 Bitwise Operators and Shifts · Practise: 1.5 Bitwise Operators and Shifts · Learn: 1.6 Output: print, println, printf and String Concatenation · Practise: 1.6 Output: print, println, printf and String Concatenation · Learn: 1.7 Input: Scanner, BufferedReader and Command-Line Arguments · Practise: 1.7 Input: Scanner, BufferedReader and Command-Line Arguments · Learn: 1.8 Decision Making: if, else, switch · Practise: 1.8 Decision Making: if, else, switch · Learn: 1.9 Loops: for, while, do-while, break and continue · Practise: 1.9 Loops: for, while, do-while, break and continue	418
3	Revise yesterday: Arithmetic, Assignment and Increment Operators, Relational, Logical and Ternary Operators; Precedence, Bitwise Operators and Shifts… · Learn: 1.10 Methods: Parameters, Return Values, Scope and Overloading · Practise: 1.10 Methods: Parameters, Return Values, Scope and Overloading · Learn: 1.11 Tracing and Debugging Programs · Practise: 1.11 Tracing and Debugging Programs · Learn: 2.1 One-Dimensional Arrays · Practise: 2.1 One-Dimensional Arrays · Learn: 2.2 Two-Dimensional Arrays (Matrices) · Practise: 2.2 Two-Dimensional Arrays (Matrices) · Learn: 2.3 Strings, StringBuilder and Characters · Practise: 2.3 Strings, StringBuilder and Characters · Learn: 2.4 Number Systems, Binary and Two's Complement · Practise: 2.4 Number Systems, Binary and Two's Complement · Learn: 2.5 Divisibility, GCD/LCM, Primes and Modular Arithmetic · Practise: 2.5 Divisibility, GCD/LCM, Primes and Modular Arithmetic	458
4	Revise yesterday: Methods: Parameters, Return Values, Scope and Overloading, Tracing and Debugging Programs, One-Dimensional Arrays… · Spaced review (3 days ago): How Java Works: Source, Bytecode, JVM, JRE, JDK, The Java Landscape: History, Features, Editions, JVM Memory · Learn: 2.6 Recursion: Base Case, Recursive Case and the Call Stack · Practise: 2.6 Recursion: Base Case, Recursive Case and the Call Stack · Learn: 2.7 References, Pass-by-Value and Memory · Practise: 2.7 References, Pass-by-Value and Memory · Learn: 2.8 Boundary Conditions, Validation and Common Mistakes · Practise: 2.8 Boundary Conditions, Validation and Common Mistakes · Learn: 3.1 Classes, Objects, Fields, Constructors and this · Practise: 3.1 Classes, Objects, Fields, Constructors and this · Learn: 3.2 Encapsulation and Access Modifiers · Practise: 3.2 Encapsulation and Access Modifiers · Learn: 3.3 Inheritance, Overriding and Polymorphism · Practise: 3.3 Inheritance, Overriding and Polymorphism	392
5	Revise yesterday: Recursion: Base Case, Recursive Case and the Call Stack, References, Pass-by-Value and Memory, Boundary Conditions, Validation and Common Mistakes… · Spaced review (3 days ago): Arithmetic, Assignment and Increment Operators, Relational, Logical and Ternary Operators; Precedence · Learn: 3.4 Abstract Classes and Interfaces · Practise: 3.4 Abstract Classes and Interfaces · Learn: 3.5 static, final and Initialisation Order · Practise: 3.5 static, final and Initialisation Order · Learn: 3.6 Packages and Imports · Practise: 3.6 Packages and Imports · Learn: 3.7 Exceptions: try, catch, finally, throw and throws · Practise: 3.7 Exceptions: try, catch, finally, throw and throws · Learn: 3.8 Generics and Lists: ArrayList and LinkedList · Practise: 3.8 Generics and Lists: ArrayList and LinkedList · Learn: 3.9 Sets and Maps: HashSet, TreeSet, HashMap, TreeMap · Practise: 3.9 Sets and Maps: HashSet, TreeSet, HashMap, TreeMap · Learn: 3.10 Queues, Deques, Stacks and PriorityQueue · Practise: 3.10 Queues, Deques, Stacks and PriorityQueue	426

Day	What to do	Minutes
6	Revise yesterday: Abstract Classes and Interfaces, static, final and Initialisation Order, Packages and Imports… · Spaced review (3 days ago): Methods: Parameters, Return Values, Scope and Overloading, Tracing and Debugging Programs · Learn: 3.11 Sorting with Comparable and Comparator · Practise: 3.11 Sorting with Comparable and Comparator · Learn: 3.12 equals(), hashCode() and Immutability · Practise: 3.12 equals(), hashCode() and Immutability · Learn: 4.1 Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables · Practise: 4.1 Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables · Learn: 4.2 Trees, Graphs and Heaps · Practise: 4.2 Trees, Graphs and Heaps · Learn: 4.3 Searching: Linear and Binary Search · Practise: 4.3 Searching: Linear and Binary Search · Learn: 4.4 Elementary Sorting: Bubble, Insertion, Selection; Stability · Practise: 4.4 Elementary Sorting: Bubble, Insertion, Selection; Stability	412
7	Revise yesterday: Sorting with Comparable and Comparator, equals(), hashCode() and Immutability, Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables… · Spaced review (3 days ago): Recursion: Base Case, Recursive Case and the Call Stack, References, Pass-by-Value and Memory · Learn: 4.5 Merge Sort and Divide and Conquer · Practise: 4.5 Merge Sort and Divide and Conquer · Learn: 4.6 Iteration versus Recursion · Practise: 4.6 Iteration versus Recursion · Learn: 4.7 Time Complexity and Big-O Notation · Practise: 4.7 Time Complexity and Big-O Notation · Learn: 4.8 Algorithm Design Paradigms · Practise: 4.8 Algorithm Design Paradigms · Learn: 4.9 Analysing Loops: Counting Iterations · Practise: 4.9 Analysing Loops: Counting Iterations · Learn: 4.10 Analysing Recursion: Recurrence Relations · Practise: 4.10 Analysing Recursion: Recurrence Relations · Learn: 4.11 Space Complexity and the Call Stack · Practise: 4.11 Space Complexity and the Call Stack	481
8	Revise yesterday: Merge Sort and Divide and Conquer, Iteration versus Recursion, Time Complexity and Big-O Notation… · Spaced review (3 days ago): Abstract Classes and Interfaces, static, final and Initialisation Order · Learn: 4.12 Best, Average and Worst Case · Practise: 4.12 Best, Average and Worst Case · Learn: 5.1 Euclid's Algorithm (GCD) · Practise: 5.1 Euclid's Algorithm (GCD) · Learn: 5.2 Euler's Phi (Totient) Function · Practise: 5.2 Euler's Phi (Totient) Function · Learn: 5.3 Chinese Remainder Theorem · Practise: 5.3 Chinese Remainder Theorem · Learn: 5.4 Combinations (nCr) · Practise: 5.4 Combinations (nCr) · Learn: 5.5 Karatsuba's Multiplication · Practise: 5.5 Karatsuba's Multiplication	417
9	Revise yesterday: Best, Average and Worst Case, Euclid's Algorithm (GCD), Euler's Phi (Totient) Function… · Spaced review (3 days ago): Sorting with Comparable and Comparator, equals(), hashCode() and Immutability · Learn: 5.6 Booth's Algorithm (signed binary multiplication) · Practise: 5.6 Booth's Algorithm (signed binary multiplication) · Learn: 5.7 Simple Sieve of Eratosthenes · Practise: 5.7 Simple Sieve of Eratosthenes · Learn: 5.8 Segmented & Incremental Sieve · Practise: 5.8 Segmented & Incremental Sieve · Learn: 5.9 Binary Palindrome · Practise: 5.9 Binary Palindrome · Learn: 5.10 Swap Two Nibbles in a Byte · Practise: 5.10 Swap Two Nibbles in a Byte · Learn: 5.11 Longest Sequence of 1s (and after flipping a bit) · Practise: 5.11 Longest Sequence of 1s (and after flipping a bit)	427
10	Revise yesterday: Booth's Algorithm (signed binary multiplication), Simple Sieve of Eratosthenes, Segmented & Incremental Sieve… · Spaced review (3 days ago): Merge Sort and Divide and Conquer, Iteration versus Recursion · Learn: 5.12 Josephus Problem · Practise: 5.12 Josephus Problem · Learn: 5.13 Block Swap Algorithm (Array Rotation) · Practise: 5.13 Block Swap Algorithm (Array Rotation) · Learn: 5.14 Leaders in an Array · Practise: 5.14 Leaders in an Array · Learn: 5.15 Majority Element (Boyer–Moore Voting) · Practise: 5.15 Majority Element (Boyer–Moore Voting) · Learn: 5.16 Maximum Product Subarray · Practise: 5.16 Maximum Product Subarray · Learn: 5.17 Maximum Equilibrium Sum · Practise: 5.17 Maximum Equilibrium Sum	427
11	Revise yesterday: Josephus Problem, Block Swap Algorithm (Array Rotation), Leaders in an Array… · Spaced review (3 days ago): Best, Average and Worst Case, Euclid's Algorithm (GCD) · Learn: 5.18 Maximum Sum of an Hourglass in a Matrix · Practise: 5.18 Maximum Sum of an Hourglass in a Matrix · Learn: 5.19 Toggle the Switch (100 Doors) · Practise: 5.19 Toggle the Switch (100 Doors) · Learn: 5.20 Alice's Apple Tree (Minimum Garden Perimeter) · Practise: 5.20 Alice's Apple Tree (Minimum Garden Perimeter) · Learn: 5.21 Weighted Substring (Smallest String of a Given Weight) · Practise: 5.21 Weighted Substring (Smallest String of a Given Weight) · Learn: 5.22 Move Hyphen to the Beginning · Practise: 5.22 Move Hyphen to the Beginning · Learn: 5.23 Strobogrammatic Number · Practise: 5.23 Strobogrammatic Number	397

Day	What to do	Minutes
12	Revise yesterday: Maximum Sum of an Hourglass in a Matrix, Toggle the Switch (100 Doors), Alice's Apple Tree (Minimum Garden Perimeter)... · Spaced review (3 days ago): Booth's Algorithm (signed binary multiplication), Simple Sieve of Eratosthenes · Learn: 5.24 Lexicographically First Palindromic Rearrangement · Practise: 5.24 Lexicographically First Palindromic Rearrangement · Learn: 5.25 Manacher's Algorithm (Longest Palindromic Substring) · Practise: 5.25 Manacher's Algorithm (Longest Palindromic Substring) · Learn: 5.26 Natural Sort Order · Practise: 5.26 Natural Sort Order · Learn: 5.27 Maneuvering (Counting Grid Paths) · Practise: 5.27 Maneuvering (Counting Grid Paths) · Learn: 5.28 Sorted Distinct Permutations · Practise: 5.28 Sorted Distinct Permutations · Learn: 5.29 N-Queens (Backtracking) · Practise: 5.29 N-Queens (Backtracking)	462
13	Revise yesterday: Lexicographically First Palindromic Rearrangement, Manacher's Algorithm (Longest Palindromic Substring), Natural Sort Order... · Spaced review (3 days ago): Josephus Problem, Block Swap Algorithm (Array Rotation) · Learn: 5.30 Maze Solving (Rat in a Maze) · Practise: 5.30 Maze Solving (Rat in a Maze) · Learn: 5.31 Hamiltonian Cycle · Practise: 5.31 Hamiltonian Cycle · Learn: 5.32 Knight's Tour & Warnsdorff's Rule · Practise: 5.32 Knight's Tour & Warnsdorff's Rule · Learn: 5.33 Selection Sort · Practise: 5.33 Selection Sort · Learn: 5.34 Quick Sort · Practise: 5.34 Quick Sort	393
14	Weak-topic review (from your quiz and practice results) · Final mixed assessment (archive topics)	75

AN HONEST PROMISE

This guide gives you a complete, verified map and enough practice to learn the material quickly — but nobody can promise mastery in a fixed number of days. How fast you progress depends on how much you code yourself. Use the quizzes and exercises to find out what you actually know, and revisit weak topics.

Part 0 · Programming & Java Setup

What programming is, how source code becomes bytecode and runs on the JVM, installing the JDK on macOS, running from an IDE and the terminal, and the three kinds of errors.

	Lesson	Level	Min	Core
0.1	How Java Works: Source, Bytecode, JVM, JRE, JDK	beginner	25	★
0.2	The Java Landscape: History, Features, Editions, JVM Memory	beginner	25	
0.3	Installing Java on macOS and Choosing an Editor	beginner	30	★
0.4	Your First Program: class, main, statements, comments	beginner	35	★
0.5	Compile-time, Runtime and Logic Errors	beginner	30	★
0.6	What Is an Algorithm?	beginner	30	★

0.1 How Java Works: Source, Bytecode, JVM, JRE, JDK

Programming & Java Setup · Beginner · Essential Java syntax · about 25 min · ★ on the minimum-viable path

What actually happens between typing a .java file and seeing output — and what JDK, JRE and JVM mean.

YOU WILL BE ABLE TO

- Explain what programming is and why a program must be translated before a computer can run it.
- Trace the path: source code (.java) → compiler (javac) → bytecode (.class) → JVM → machine code.
- Tell JDK, JRE and JVM apart.
- Explain "write once, run anywhere" and why Java is called both compiled and interpreted.

The idea

A **program** is a list of precise instructions for a computer. Humans write instructions in a *programming language* like Java because it is readable; a CPU only understands very low-level machine code. Something must translate.

Java uses a two-step trick. First the **compiler** (`javac`) translates your readable **source code** (`Hello.java`) into **bytecode** (`Hello.class`): a compact instruction set for an imaginary computer. Then the **Java Virtual Machine (JVM)** — a real program installed on your computer — runs that bytecode, translating it to the actual machine's instructions on the fly.

```
Hello.java ──javac──► Hello.class ──java──► JVM ──► your CPU
(you write) compile (bytecode) run interprets + JIT compiles
```

WHY THIS WORKS

Because the bytecode is the same on every computer, you compile **once** and the file runs on any operating system that has a JVM — Windows, macOS, Linux. Only the JVM is different for each platform. That is Java's slogan: **write once, run anywhere**.

term	what it is	you need it to…
JVM	Java Virtual Machine: runs bytecode	run any Java program
JRE	Java Runtime Environment = JVM + standard libraries	run Java programs (end users)
JDK	Java Development Kit = JRE + compiler (javac) + tools	write and compile Java (developers: you)

"Compiled **and** interpreted": the compiler turns source into bytecode (compiled); the JVM then executes the bytecode (interpreted), and compiles the *hot* (frequently used) parts into native machine code at run time with a **JIT** (Just-In-Time) compiler to make them fast.

Syntax

The commands you will use all course (macOS / Linux / Windows terminal)

```
javac Hello.java # compile: Hello.java -> Hello.class
java Hello # run it: the CLASS name (no .java, no .class)
java Hello.java # Java 11+: compile in memory and run one source file
java -version # which runtime is first on the PATH?
javac -version # which compiler?
```

- The file is named after its **public class**: `public class Hello` must be in `Hello.java` (upper/lower case matters).
- Source files are plain text ending in `.java`; the compiler produces `.class` files containing bytecode.
- Execution starts at `public static void main(String[] args)`.

A tiny example

```
// Hello.java -- the file you write
public class Hello {
 public static void main(String[] args) {
 System.out.println("Hi");
 }
}
// $ javac Hello.java -> creates Hello.class (bytecode)
// $ java Hello -> the JVM runs it and prints: Hi
```

Guided example

This program asks the JVM that is running it a few questions, proving that the JVM — not your operating system — executes your bytecode:

ShowJavaVersion.java — Which JVM am I running on? (demonstration)

```
1  public class ShowJavaVersion {
2   public static void main(String[] args) {
3   // These values come from the JVM that is running this very program.
4   System.out.println("Java version : " + System.getProperty("java.version"));  (1)
5   System.out.println("JVM name : " + System.getProperty("java.vm.name"));  (2)
6   System.out.println("Operating sys: " + System.getProperty("os.name"));  (3)
7   System.out.println("This program was compiled to bytecode and is now being
        interpreted/JIT-compiled by the JVM.");
8   }
9  }
```

System.getProperty reads values the JVM exposes about itself and the machine.

Line-by-line walkthrough

#	Where	What happens
1	line 4	A system property: a named setting provided by the JVM. This one is the Java version.
2	line 5	The name of the virtual machine implementation (HotSpot on most computers).
3	line 6	The operating system. The same bytecode prints a different value on a different OS.

SameBytecodeEverywhere.java — Same bytecode, different platform (demonstration)

```
1  public class SameBytecodeEverywhere {
2   // The same Hello.class runs on macOS, Windows and Linux: only the JVM is different per
        platform.
3   public static void main(String[] args) {
4   String os = System.getProperty("os.name");                                          (1)
5   System.out.println("Compiled once, running on: " + os);
6   System.out.println("File separator on this OS: '" + java.io.File.separator + "'");   (2)
7   }
8  }
```

The program text never changes; only the operating-system values it reads do.

Line-by-line walkthrough

#	Where	What happens
1	line 4	Read the operating system name at run time — it differs on macOS, Windows and Linux although the .class file is identical.
2	line 6	The path separator also comes from the platform: "/" on macOS and Linux, "\" on Windows.

Dry run: watch it execute

From Hello.java to the text on your screen: compile, load, run.

Algorithm: From source file to output. **Input:** cls = `Hello`, msg = `Hi`.

Pseudocode

```
write Name.java
javac Name.java → Name.class (bytecode)
java Name → start a JVM
class loader + bytecode verifier
find public static void main(String[] args)
execute: interpret hot code → JIT-compile it
JVM exits with status 0
```

Step 1 of 7. You write the source file Hello.java: plain text a human can read.

file = Hello.java · kind = source code

files in the folder `Hello.java`
0

Step 2 of 7. javac Hello.java: the COMPILER checks syntax and types and translates the file into bytecode. If it finds an error it stops here and no .class file is created.

command = javac Hello.java · created = Hello.class

files in the folder `Hello.jHello.class`
0 1

Step 3 of 7. java Hello: the operating system starts a Java Virtual Machine (JVM). Note: you give the class name, not the file name.

command = java Hello · process started = JVM

Step 4 of 7. The class loader reads Hello.class; the bytecode verifier checks it cannot corrupt memory or break the type rules.

loaded = Hello · verified = true

Step 5 of 7. The JVM looks for the entry point public static void main(String[] args) in Hello and calls it.

entry = Hello.main(String[])

Step 6 of 7. The bytecode runs (interpreted first; hot loops are compiled to machine code by the JIT). System.out.println prints "Hi".

output = Hi

Step 7 of 7. main returns, the JVM exits and the operating system receives status 0 (success).

exit status = 0

> **RESULT**
> console shows: Hi

Complexity

Not applicable for this concept lesson. (Cost model: the JVM starts, loads classes, verifies bytecode, then interprets — the first run is a little slower than later iterations of a hot loop because of JIT compilation.)

Common mistakes

Installing only a JRE

Problem

```
$ javac Hello.java
zsh: command not found: javac
```

Why: A JRE can run programs but does not contain the compiler. You need a JDK.

Fix: Install a JDK (Temurin / OpenJDK 17 or newer).

Running the source file name with .class

Problem

```
$ java Hello.class
```

Why: `java` takes a **class name**, not a file name.

Fix: $ java Hello

Thinking the .class file is machine code

Problem

```
// "I compiled on a Mac, so it will not run on Windows"
```

Why: Bytecode is platform independent; it only needs a JVM for that platform.

Fix: Copy Hello.class to the other machine and run java Hello there.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Print your own message — *Beginner*

Write a program that prints exactly: `Write once, run anywhere`

Example input	Expected output
(no input)	Write once, run anywhere

Hint 1: System.out.println("...");

Quick check

Answers are in the answer key at the end of this part.

Q1. What does the `javac` command produce?

- **A.** machine code
- **B.** bytecode (.class files)
- **C.** a .exe file
- **D.** a PDF

Q2. Which package must you install to COMPILE Java programs?

- **A.** JVM
- **B.** JRE
- **C.** JDK
- **D.** JIT

Q3. Why can the same .class file run on Windows and macOS?

- **A.** It is machine code for both
- **B.** It is bytecode executed by a platform-specific JVM
- **C.** Java converts it manually
- **D.** It cannot

Q4. JIT compilation means...

- **A.** compiling before the program is written
- **B.** compiling frequently used bytecode to machine code at run time
- **C.** compiling only comments
- **D.** running bytecode twice

Interview / exam questions

- Explain the difference between JDK, JRE and JVM.
- Why is Java platform independent?
- Is Java compiled or interpreted?
- What is bytecode?

Watch & read

Video: JVM, JRE, and JDK - Fully Explained in 5 Minutes — Coding with John · 5:28. *How source code becomes bytecode and runs on the JVM; JDK vs JRE vs JVM*

Video: # 03-Difference between JDK JRE JVM | BYTECODE | |JIT COMPILER |JAVA COMPILATION PROCESS | IN HINDI — PortHub Academy · 27:20. *The compile-then-run pipeline step by step (Hindi narration — the idea transfers to Java)*

REVISION CARD (30–60 SECONDS)

- .java → javac → .class (bytecode) → JVM → machine code.
- JDK ⊃ JRE ⊃ JVM.
- Write once, run anywhere: only the JVM differs per OS.
- JIT compiles hot bytecode to native code at run time.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide "Why is JAVA slow?" says Java is slow because linking happens at run time; modern JVMs with JIT compilation are competitive with C++ for long-running programs. Start-up time is where Java is slower.

0.2 The Java Landscape: History, Features, Editions, JVM Memory

Programming & Java Setup · Beginner · Essential Java syntax · about 25 min · complete path

Why Java exists, what it is used for, its editions, and what the JVM does with memory.

YOU WILL BE ABLE TO

- Name Java's key features and what kinds of applications are written in it.
- Distinguish Java SE, EE, ME and JavaFX.
- Describe the JVM's memory areas: heap, stack, method area, and what garbage collection does.
- Explain, with reasons, when "Java is slow" is true and false.

Before this lesson: How Java Works: Source, Bytecode, JVM, JRE, JDK

The idea

Java was created at Sun Microsystems (James Gosling, released 1995) and is now maintained by Oracle and the OpenJDK community. It was designed to be **simple, object-oriented, secure, portable, robust** (strong typing, exceptions, automatic memory management) and **multithreaded**.

Edition	For	Examples
Java SE (Standard)	the core language and libraries — what this course uses	console programs, algorithms
Java EE / Jakarta EE (Enterprise)	large server-side systems	web services, banking back-ends
Java ME (Micro)	small embedded devices	smart cards, older phones
JavaFX	desktop graphical user interfaces	rich client apps

Java applications fall into four families: **standalone** (desktop/console), **web**, **enterprise**, and **mobile** (Android apps are written in Java/Kotlin).

What the JVM does for you

- **Loads** class files and **verifies** that the bytecode is safe.
- **Executes** the bytecode (interpreter + JIT).
- Manages **memory**: you create objects with `new`; the **garbage collector** frees them when nothing refers to them any more. No manual `free()`.



WHY THIS WORKS

Local variables and method calls live on the **stack** (fast, automatic, limited — a runaway recursion overflows it). Objects live on the **heap** (large, flexible, cleaned by the garbage collector). Understanding this split explains StackOverflowError, OutOfMemoryError and pass-by-value of references later in the course.

Syntax

Syntax that appeared after Java 8 (this course targets Java 17+)

feature	since	syntax
var (local type inference)	Java 10	`var n = 5;`
switch expression	Java 14	`int d = switch (x) { case 1 -> 10; default -> 0; };`
text block	Java 15	`String s = """ ...multi-line... """;`
record	Java 16	`record Point(int x, int y) { }`
pattern matching for instanceof	Java 16	`if (o instanceof String s) { ... s.length() ... }`

Long-term-support (LTS) releases are the ones companies stay on: Java 8, 11, 17, 21 and 25. A new feature release appears every six months; `javac --release 17` compiles *as if* for Java 17, even on a newer JDK.

A tiny example

```
int x = 5; // x lives on the stack
int[] numbers = new int[100]; // the array object lives on the heap; 'numbers' (a reference)
    lives on the stack
```

Guided example

RuntimeInfo.java — JVM resources (demonstration)

```
1  public class RuntimeInfo {
2   public static void main(String[] args) {
3   Runtime rt = Runtime.getRuntime();                                          (1)
4   long mb = 1024 * 1024;
5   System.out.println("CPU cores available to the JVM : " + rt.availableProcessors());   (2)
6   System.out.println("Maximum heap the JVM may use : " + rt.maxMemory() / mb + " MB");   (3)
7   System.out.println("Heap currently reserved : " + rt.totalMemory() / mb + " MB");
8   System.out.println("Garbage collector frees objects automatically — you never call
        free().");
9   }
10 }
```

Runtime.getRuntime() gives access to the JVM's view of the machine.

Line-by-line walkthrough

#	Where	What happens
1	line 3	The singleton object that represents the running JVM.
2	line 5	How many CPU cores the JVM may use.
3	line 6	The largest heap it may grow to, converted from bytes to megabytes.

Dry run: watch it execute

Primitives sit in stack slots; arrays and objects live on the heap behind references.

Algorithm: Stack and heap memory. **Input:** x = 7, y = 8.

Pseudocode

```
int n = 5;
int[] a = {x, y};
int[] b = a; // copies the reference
b[0] = 99;
a = new int[]{1};
b = null;
```

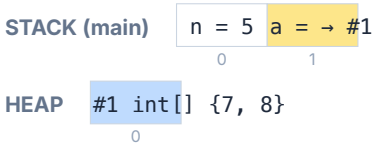
Step 1 of 6. A primitive: the value 5 sits directly in the stack slot n.

stack slots = 1 · heap objects = 0



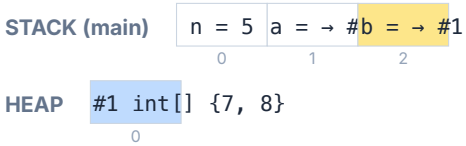
Step 2 of 6. new int[]{7, 8} is built on the HEAP as object #1; the stack variable a only holds a reference to it.

stack slots = 2 · heap objects = 1



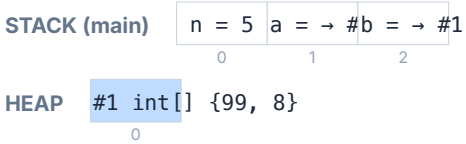
Step 3 of 6. b = a copies the REFERENCE, not the array: a and b now point to the same object #1.

stack slots = 3 · heap objects = 1



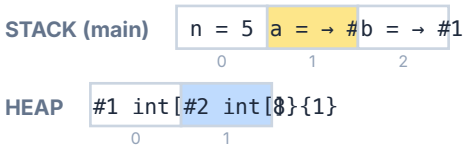
Step 4 of 6. b[0] = 99 changes the shared object, so a[0] is also 99 (a[0] = 99).

stack slots = 3 · heap objects = 1



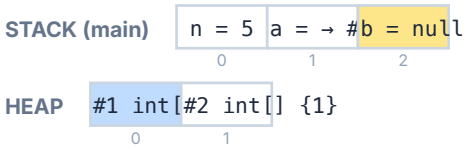
Step 5 of 6. a = new int[]{1} creates a second object and re-points a. b still points to #1.

stack slots = 3 · heap objects = 2



Step 6 of 6. b = null: nothing refers to #1 any more, so the garbage collector may reclaim it (shown empty).

stack slots = 3 · heap objects = 1



> **RESULT**
> a and b shared one array until a was re-pointed; objects without references are garbage-collected

Complexity

Not applicable. (Memory model: stack frames are created and destroyed in LIFO order in O(1); heap allocation is O(1) amortised; garbage collection runs automatically and is not under the programmer's control.)

Common mistakes

Believing Java has no memory leaks

Problem

```
static List<byte[]> cache = new ArrayList<>();
cache.add(new byte[10_000_000]); // never removed
```

Why: The garbage collector frees only unreachable objects. Anything still referenced stays in memory.

Fix: Remove references (set to null / clear the collection) when no longer needed.

Mixing up edition names

Problem

```
// "I need Java EE to write a console algorithm program"
```

Why: Algorithms and this whole course use Java SE.

Fix: Install a standard JDK.

Assuming Java is always slow

Problem

```
// "the slides say dynamic linking makes Java slow"
```

Why: With the JIT, steady-state performance is close to native for most algorithmic code. Short programs pay a start-up cost.

Fix: Measure; do not assume.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Powers of two in memory units — *Beginner*

Read a number of megabytes M and print how many bytes that is (1 MB = 1024 × 1024 bytes). Use long.

- 0 ≤ M ≤ 8 000 000 000

Example input	Expected output
256	268435456

Hint 1: long bytes = (long) M * 1024 * 1024;

Quick check

Answers are in the answer key at the end of this part.

Q1. Where are Java objects stored?

- **A.** stack
- **B.** heap
- **C.** registers
- **D.** method area

Q2. What does the garbage collector do?

- **A.** compiles code
- **B.** frees objects that are no longer reachable
- **C.** sorts memory
- **D.** closes files

Q3. Which edition is used for standard Java programs and algorithms?

- **A.** Java ME
- **B.** Java EE
- **C.** Java SE
- **D.** JavaFX only

Q4. A recursion that never stops usually ends with...

- **A.** OutOfMemoryError: heap
- **B.** StackOverflowError
- **C.** a compile error
- **D.** NullPointerException

Interview / exam questions

- What are the main features of Java?
- Stack vs heap?
- What is garbage collection?
- Is Java pass-by-value or pass-by-reference? (preview)

Watch & read

Video: #26 Stack And Heap in Java — Telusko · 12:37. *Why Java is used, its features, and how the JVM organises memory*

Video: Stack vs Heap Memory - Simple Explanation — Alex Hyett · 5:28. *Stack vs heap memory in the JVM*

REVISION CARD (30–60 SECONDS)

- Java SE = core; EE = enterprise servers; ME = embedded; JavaFX = GUIs.
- Stack: frames/locals. Heap: objects. GC frees unreachable objects.
- Features: OO, portable, robust, secure, multithreaded.

0.3 Installing Java on macOS and Choosing an Editor

Programming & Java Setup · Beginner · Essential Java syntax · about 30 min · ★ on the minimum-viable path

Install a JDK, verify it from the terminal, and run a program from VS Code or IntelliJ IDEA.

YOU WILL BE ABLE TO

- Install a JDK 17+ on macOS and verify `java -version` and `javac -version`.
- Compile and run a program from the Terminal.
- Run the same program from VS Code or IntelliJ IDEA.
- Fix the two most common set-up problems (command not found, wrong version).

Before this lesson: How Java Works: Source, Bytecode, JVM, JRE, JDK

The idea

You need three things: a **JDK**, a **terminal** (built into macOS) and an **editor or IDE**. This course targets **Java 17 (LTS)**: every program compiles with `javac --release 17`, so any newer JDK works too.

1. Install the JDK

1. Install Homebrew-free: download a JDK 17+ installer (**Temurin** from adoptium.net, or Oracle OpenJDK) and run it; **or** with Homebrew: `brew install --cask temurin@17`.
2. Open a new Terminal window.
3. Verify (see below).

2. Verify it

```
java -version # shows e.g. openjdk version "17.0.x"
javac -version # shows e.g. javac 17.0.x
```

3. Compile and run in the terminal

```
cd ~/Desktop
nano Hello.java # type the program, save with Ctrl-O, quit with Ctrl-X
javac Hello.java # creates Hello.class
java Hello # runs it
```

4. Pick an editor

Tool	Good for	Notes
VS Code + "Extension Pack for Java"	lightweight, fast start	install the extension pack, open a folder, click Run above main
IntelliJ IDEA Community	full-featured IDE	File ▸ New ▸ Project, choose your JDK; Run with the green arrow
Terminal + any text editor	learning exactly what happens	javac / java as above

WHY THIS WORKS

An IDE runs exactly the same two commands (`javac`, then `java`) for you when you press Run. Knowing the terminal commands means you are never stuck when the IDE misbehaves and you can use this site's playground, which compiles with the same tools.

Syntax

macOS terminal

```
brew install --cask temurin@17 # a free OpenJDK build (Homebrew); 21 or 25 also work
/usr/libexec/java_home -V # list every JDK installed on this Mac
export JAVA_HOME=$(/usr/libexec/java_home -v 17) # choose one for this terminal session
echo $JAVA_HOME
javac -version && java -version # both must print the same major version
```

> **NOTE**
> Homebrew is optional: you can also download a `.pkg` installer from adoptium.net. The terminal commands above are the same either way.

A tiny example

```
public class VerifyInstall {
 public static void main(String[] args) {
 System.out.println("java.version = " + System.getProperty("java.version"));
 }
}
```

Guided example

VerifyInstall.java — Verify the installation (demonstration)

```
1  public class VerifyInstall {                                          (1)
2   public static void main(String[] args) {
3   System.out.println("If you can read this, your JDK compiler (javac) and runtime (java)
        both work.");
4   System.out.println("java.version = " + System.getProperty("java.version"));
5   System.out.println("java.home = " + System.getProperty("java.home"));   (2)
6   }
7  }
```

If this compiles and prints, your JDK is correctly installed.

Line-by-line walkthrough

#	Where	What happens
1	line 1	The file must be named VerifyInstall.java — a public class must match its file name.
2	line 5	The folder of the JDK that is actually running — useful when several are installed.

> **TROUBLESHOOTING ON MACOS**
> If `javac` is not found but `java` is, you installed only a runtime. Install the JDK. If you have several JDKs, `/usr/libexec/java_home -V` lists them and `export JAVA_HOME=$(/usr/libexec/java_home -v 17)` selects one for the current terminal.

Dry run: watch it execute

The five commands that prove your JDK works.

Algorithm: Checking your Java installation. **Input:** jdk = `21`.

Pseudocode

```
java –version
javac –version
javac Hello.java
ls
java Hello
```

Step 1 of 5. java -version prints the runtime version: 21. This proves a JRE/JDK is on the PATH.

command = java -version · prints = openjdk version "21..."

Step 2 of 5. javac -version prints javac 21...: the COMPILER exists, so this is a JDK and not just a JRE.

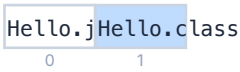
command = javac -version · prints = javac 21... · meets Java 17+ = true

Step 3 of 5. javac Hello.java prints nothing when it succeeds (silence is good news).

command = javac Hello.java · prints = (nothing)

Step 4 of 5. ls now shows a new file Hello.class next to Hello.java: the bytecode.

command = ls

folder 

Step 5 of 5. java Hello (no .class!) runs the program: you see its output.

command = java Hello · prints = Hi

> **RESULT**
> Ready: JDK 21 works for this course

Complexity

Not applicable (set-up lesson).

Common mistakes

File name does not match the class

Problem

```
// saved as hello.java
public class Hello { ... }
```

Why: javac reports "class Hello is public, should be declared in a file named Hello.java". Names are case-sensitive.

Fix: Save as Hello.java.

Running from the wrong folder

Problem

```
$ java Hello
Error: Could not find or load main class Hello
```

Why: The .class file is not in the current directory (or you included the .class extension).

Fix: cd to the folder with Hello.class, run `java Hello`.

Old Java picked up first

Problem

```
$ java -version
java version "1.8.0_…"
```

Why: An older JDK is first on the PATH.

Fix: export JAVA_HOME=$(/usr/libexec/java_home -v 17) (and add it to ~/.zshrc)

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Compile and run it yourself — *Beginner*

In the playground or your terminal run the verification program. Then write a program that prints the two lines `JDK ready` and `Let us begin`.

Example input	Expected output
(no input)	JDK ready ⏎ Let us begin

Hint 1: Two System.out.println statements.

Quick check

Answers are in the answer key at the end of this part.

Q1. Which command compiles Hello.java?

- **A.** java Hello.java
- **B.** javac Hello.java
- **C.** compile Hello
- **D.** jvm Hello

Q2. After compiling you run it with...

- **A.** java Hello
- **B.** java Hello.class
- **C.** javac Hello
- **D.** ./Hello.java

Q3. "javac: command not found" usually means...

- **A.** your code has an error
- **B.** no JDK (compiler) is installed or on the PATH
- **C.** Java is too old
- **D.** the file is empty

Q4. This course compiles programs for which Java language level?

- **A.** Java 8
- **B.** Java 11
- **C.** Java 17
- **D.** Java 21 preview

Interview / exam questions

- What is the PATH and why does javac need to be on it?
- What does an IDE do when you press Run?
- What is JAVA_HOME for?

Watch & read

Video: How to Install Java on Mac | Install Java JDK on macOS — ProgrammingKnowledge · 16:48. *Installing a JDK on macOS and checking it with java -version*

Video: IntelliJ IDEA Setup for Java Beginners | Install IDE + Run Hello World (Step-by-Step) — The Study Room Journal · 5:45. *Setting up an editor/IDE and running a first project*

REVISION CARD (30–60 SECONDS)

- Install JDK 17+; verify java -version and javac -version.
- javac X.java → X.class; java X.
- File name = public class name.
- IDEs just call javac and java for you.

0.4 Your First Program: class, main, statements, comments

Programming & Java Setup · Beginner · Essential Java syntax · about 35 min · ★ on the minimum-viable path

Every part of Hello World explained: the class, the main method, statements, braces, semicolons, comments, files and packages.

YOU WILL BE ABLE TO

- Write, compile and run a program from scratch and explain every keyword in it.
- Use statements, blocks `{}`, semicolons and comments correctly.
- Explain the relationship between file name, class name, package and folder.
- Read a line of input with Scanner for the first time.

Before this lesson: Installing Java on macOS and Choosing an Editor

The idea

```
public class HelloWorld { ← a class: every Java program lives in one
 public static void main(String[] args) { ← main: the JVM starts running here
 System.out.println("Hello, Java!"); ← a statement (ends with ;)
 } ← } closes the method
} ← } closes the class
```

piece	meaning
public	access modifier: visible to everyone (the JVM must be able to call main)
class HelloWorld	declares a class named HelloWorld; the file must be HelloWorld.java
static	belongs to the class itself, so the JVM can call it without creating an object
void	the method returns no value
main	the special name where execution begins
String[] args	command-line arguments, an array of strings (unused for now)
System.out.println(...)	print a line: System is a class, out its output stream, println a method

WHY THIS WORKS

Java is strict: *everything* is inside a class, and execution starts at `public static void main(String[] args)`. Memorise this line as a unit for now; each word is explained fully in the OOP module. A **statement** is one instruction and ends with `;`. Braces `{ }` group statements into a **block**. Indentation does not matter to the compiler but matters hugely to readers.

Files, folders, packages

- One public class per file, and the file is named exactly like the class (`HelloWorld.java`).
- A **package** groups related classes: `package com.example.demo;` as the first line means the file lives in a folder `com/example/demo/`. Small programs use the default (no) package.
- Class names use `UpperCamelCase`; methods and variables `lowerCamelCase`; constants `UPPER_SNAKE_CASE`.

Comments

```
// single-line comment
/* multi-line
 comment */
/** Javadoc: documents the next class or method */
```

Syntax

> **SHORTCUT**
> You can save typing with an IDE template: in IntelliJ type `psvm` + Tab, in VS Code type `main` + Tab.

A tiny example

```
public class HelloWorld {
 public static void main(String[] args) {
 System.out.println("Hello, Java!");
 }
}
```

Guided example

HelloWorld.java — Hello, Java! (tiny example)

```
1  public class HelloWorld {                          (1)
2   public static void main(String[] args) {          (2)
3   System.out.println("Hello, Java!");               (3)
4   }
5  }
```

The smallest complete Java program.

Output when you run it

```
Hello, Java!
```

Line-by-line walkthrough

#	Where	What happens
1	line 1	Declares the class. The name must match the file name HelloWorld.java.
2	line 2	The entry point the JVM looks for.
3	line 3	Prints the text and moves to a new line. Text in double quotes is a String literal.

AnatomyOfAProgram.java — Anatomy: variables and expressions (demonstration)

```
1  // A comment: ignored by the compiler. Use comments to explain WHY, not WHAT.
2  /* A block comment
3   can span several lines. */
4  public class AnatomyOfAProgram { // 1. a class: the container; its name matches the file
       name
5   public static void main(String[] args) { // 2. main: where execution starts
6   int apples = 3; // 3. a statement ends with a semicolon                              (1)
7   int oranges = 4;
8   int fruit = apples + oranges; // 4. an expression computed from variables            (2)
9   System.out.println("Fruit: " + fruit);// 5. output                                  (3)
10  } // braces { } group statements into a block
11 }
```

Shows comments, two int variables, an expression, and string concatenation.

Output when you run it

```
Fruit: 7
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	A variable declaration with a value: type `int`, name `apples`.
2	line 8	An expression is evaluated and its result stored.
3	line 9	`+` joins a String with a number: the number is converted to text.

Greeting.java — Reading a name (demonstration)

```
1  import java.util.Scanner;                                                           (1)
2
3  public class Greeting {
4   public static void main(String[] args) {
5   Scanner in = new Scanner(System.in);                                               (2)
6   System.out.print("What is your name? ");
7   String name = in.nextLine();                                                       (3)
8   System.out.println("Nice to meet you, " + name + "!");
9   }
10 }
```

Scanner reads from the keyboard (System.in).

Sample input

```
Asha
```

Output when you run it

```
What is your name? Nice to meet you, Asha!
```

Line-by-line walkthrough

#	Where	What happens
1	line 1	Brings the Scanner class into scope so we can use its short name.
2	line 5	Create a scanner attached to standard input.
3	line 7	Read a whole line of text the user typed.

Dry run: watch it execute

print keeps the cursor on the line; println ends it.

Algorithm: print vs println. **Input:** words = `A,B,C,D`.

Pseudocode

```
System.out.print(w1);
System.out.print(w2);
System.out.println(w3);
System.out.println(w4);
```

Step 1 of 4. print("A") writes A and leaves the cursor right after it, on the same line.

call = print("A") · cursor line = 1

console (one cell per line)



Step 2 of 4. print("B") writes B and leaves the cursor right after it, on the same line.

call = print("B") · cursor line = 1

console (one cell per line)



Step 3 of 4. println("C") writes C and then moves the cursor to a NEW line.

call = println("C") · cursor line = 2

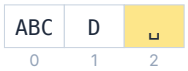
console (one cell per line)



Step 4 of 4. println("D") writes D and then moves the cursor to a NEW line.

call = println("D") · cursor line = 3

console (one cell per line)



> **RESULT**
> ABC ⏎ D

Complexity

Not applicable. (Each statement here takes constant time.)

Common mistakes

Missing semicolon

Problem

```
System.out.println("Hi")
```

Why: Compile error: ";" expected. Every statement ends with a semicolon.

Fix: System.out.println("Hi");

Wrong capitalisation

Problem

```
system.out.println("Hi");
```

Why: Java is case-sensitive: `System` is the class; `system` does not exist (error: package system does not exist).

Fix: System.out.println("Hi");

Mismatched braces

Problem

```
public class A {
 public static void main(String[] a) {
 System.out.println("x");
}
```

Why: The class brace is never closed: "reached end of file while parsing".

Fix: Add the missing } and let the editor auto-indent to see structure.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Print a box — *Beginner*

Print exactly these three lines: `` +--+ |Hi| +--+ ``

Example input	Expected output
(no input)	+--+ ⏎ |Hi| ⏎ +--+

Hint 1: Three println statements.

Exercise 2 · Echo a name — *Beginner*

Read a name (one line) and print `Hello, <name>!`

Example input	Expected output
Asha	Hello, Asha!

Hint 1: Scanner.nextLine() and string concatenation.

Exercise 3 · Sum two numbers — *Intermediate*

Read two integers on one line and print their sum.

Example input	Expected output
7 5	12

Hint 1: sc.nextInt() twice.

Quick check

Answers are in the answer key at the end of this part.

Q1. Where does a Java program start executing?

- **A.** the first line of the file
- **B.** the main method
- **C.** the class name
- **D.** the package statement

Q2. Which line has the compile error?

```
1  public class T {
2   public static void main(String[] args) {
3   System.out.println("Hi")
4   }
5  }
```

Which line contains the bug? _____

Q3. Complete the program.

```
public ______ Hello {
 public static ______ main(String[] args) {
 System.out.println("Hi");
 }
}
```

Q4. What is printed?

```
System.out.print("A");
System.out.print("B");
System.out.println("C");
System.out.println("D");
```

Your answer: _____

Q5. The file for `public class Counter` must be named...

- **A.** counter.java
- **B.** Counter.java
- **C.** Main.java
- **D.** any name

Interview / exam questions

- Why is main static?
- What happens if the file name differs from the public class name?
- What is a package?
- What does String[] args contain?

Watch & read

Video: Java Hello World Program | Meaning of public static void main (String args)| System.out.println() — TechnonTechTV · 8:22. *Anatomy of a first Java program*

Video: Java - Basic Syntax — TutorialsPoint · 8:50. *Comments, statements and basic program structure*

REVISION CARD (30–60 SECONDS)

- public class X {...} in X.java; main = public static void main(String[] args).
- Statements end with ; Blocks use { }.
- Comments: // line, /* block /, /* javadoc */.
- Case-sensitive; classes UpperCamelCase.

0.5 Compile-time, Runtime and Logic Errors

Programming & Java Setup · Beginner · Essential Java syntax · about 30 min · ★ on the minimum-viable path

The three families of bugs, how each one shows up, and how to read an error message.

YOU WILL BE ABLE TO

- Recognise a compile-time error message and locate the line it points to.
- Recognise a runtime exception and read its stack trace.
- Detect a logic error by comparing the output with what you expected.
- Apply a simple debugging routine: read, reproduce, isolate, fix, re-test.

Before this lesson: Your First Program: class, main, statements, comments

The idea

kind	when	what you see	example
Compile-time (syntax / type)	before the program runs	javac refuses; no .class file	missing ; int x = "hi";
Runtime	while it runs	program stops with an exception and a stack trace	10 / 0, array index 5 of 3, null.length()
Logic	always compiles & runs	wrong answer, no message	average = a + b / 2

The compiler is your first teacher: it reports the **file, line number and message**. Read the *first* error first — one mistake often causes a cascade of later complaints.

A typical compile-time error: file:line, the message, the source line with a caret.

```
Hello.java:3: error: ';' expected
 System.out.println("Hi")
 ^
1 error
```

A runtime error: exception type, message, and the stack trace (where it happened).

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
 at Calc.main(Calc.java:6)
```

WHY THIS WORKS

Compile-time errors are the cheapest to fix because the compiler finds them for you. Runtime errors need a test that reaches the faulty line. Logic errors are the most expensive: nothing complains, and the program is simply wrong. That is why you must **check outputs against hand-calculated examples** — the habit this course trains with dry runs and tests.

Syntax

How to read the two kinds of message

```
Hello.java:5: error: ';' expected <- compile-time: file : line : message
 System.out.println("Hi")
 ^ <- the caret points at the problem

Exception in thread "main" java.lang.ArithmeticException: / by zero
 at Hello.main(Hello.java:7) <- run-time: exception type, message, then the call stack
```

message	usually means
';' expected	a missing semicolon on that line or the line above
cannot find symbol	misspelt name, missing import, or variable out of scope
incompatible types	a value of the wrong type (e.g. a double assigned to an int)
missing return statement	some path through the method never returns
ArrayIndexOutOfBoundsException	index < 0 or >= length
NullPointerException	a reference that holds null was dereferenced
NumberFormatException	Integer.parseInt on text that is not a number

A tiny example

```java
int total = 10;
int count = 4;
double average = total / count; // logic error: integer division → 2.0, not 2.5
```

Guided example

ThreeKindsOfErrors.java — Three kinds of errors (demonstration)

```
1  public class ThreeKindsOfErrors {
2   // 1. COMPILE-TIME errors never get this far: the compiler rejects the file (see the
        lesson snippets).
3
4   // 2. RUNTIME error: the program starts, then fails on a bad operation. We catch it to
        show the message.
5   static void runtimeError() {
6   try {
7   int a = 10;
8   int b = 0;
9   System.out.println(a / b);
10  } catch (ArithmeticException e) {                                                    1
11  System.out.println("runtime error caught: " + e.getMessage());                     2
12  }
13  try {
14  int[] arr = new int[3];
15  arr[3] = 1;                                                                          3
16  } catch (ArrayIndexOutOfBoundsException e) {
17  System.out.println("runtime error caught: " + e.getMessage());
18  }
19  }
20
21  // 3. LOGIC error: compiles, runs, prints an answer — the WRONG answer.
22  static double wrongAverage(int a, int b) {
23  return a + b / 2; // precedence: b/2 happens first!                                 4
24  }
25
26  static double rightAverage(int a, int b) {
27  return (a + b) / 2.0;
28  }
29
30  public static void main(String[] args) {
31  runtimeError();
32  System.out.println("average of 10 and 20, buggy : " + wrongAverage(10, 20));
33  System.out.println("average of 10 and 20, correct: " + rightAverage(10, 20));
34  }
35 }
```

try/catch lets the program keep running after a runtime error so we can print the message.

Output when you run it

```
runtime error caught: / by zero
runtime error caught: Index 3 out of bounds for length 3
average of 10 and 20, buggy : 20.0
average of 10 and 20, correct: 15.0
```

Line-by-line walkthrough

#	Where	What happens
1	line 10	Handle the exception named ArithmeticException: dividing an int by zero throws it.
2	line 11	The human-readable detail: "/ by zero".
3	line 15	An array of length 3 has indices 0, 1, 2 — index 3 throws ArrayIndexOutOfBoundsException.
4	line 23	A LOGIC error: `/` binds tighter than `+`, so this is a + (b / 2) = 20, not the average 15.

Dry run: watch it execute

A compile-time error stops everything before the program runs. Try scenarios 2 and 3 as well.

Algorithm: Compile-time, runtime and logic errors. **Input:** kind = `1`.

Pseudocode

```
COMPILE javac checks syntax and types
RUN the JVM executes the bytecode
RESULT is the output what you wanted?
```

Step 1 of 3. The program contains int x = "five"; — a String where an int is required.
stage = source · statement = int x = "five";

Step 2 of 3. javac reports: incompatible types: String cannot be converted to int. No .class file is produced.
stage = COMPILE · outcome = error

Step 3 of 3. The program never runs: a compile-time error is found BEFORE execution, with a line number.
stage = RUN · outcome = not reached

> **RESULT**
> compile-time error (fix the code, then compile again)

Complexity

Not applicable.

Common mistakes

Fixing the last error first

Problem

```
// 12 errors reported; start from the bottom
```

Why: Early errors (a missing brace) cause later ones. Fix the first, recompile.
Fix: Start from the top; recompile after each fix.

Ignoring the stack trace line number

Problem

```
// "NullPointerException" → randomly editing code
```

Why: The trace names the exact line where the exception was thrown.
Fix: Open that line and ask which value is null / out of range.

Trusting output without checking

Problem

```
double avg = (a + b) / 2; // a, b are ints
```

Why: Integer division silently truncates. No error — only a wrong answer.

Fix: double avg = (a + b) / 2.0;

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Correct average — *Beginner*

Read two integers a and b and print their average as a decimal with exactly one digit after the point (e.g. 2.5).

Example input	Expected output
2 3	2.5

Hint 1: Divide by 2.0, not 2. **Hint 2:** System.out.printf("%.1f%n", value);

Exercise 2 · Safe division — *Intermediate*

Read two integers a and b. Print a / b as integer division, or the text `cannot divide by zero` if b is 0 (without crashing).

Example input	Expected output
7 2	3
5 0	cannot divide by zero

Hint 1: Check b first, or catch ArithmeticException.

Quick check

Answers are in the answer key at the end of this part.

Q1. `int x = "hello";` is...

- **A.** a runtime error
- **B.** a compile-time error
- **C.** a logic error
- **D.** fine

Q2. The program prints 20 where 15 was expected, with no error message. This is a...

- **A.** compile-time error
- **B.** runtime error
- **C.** logic error
- **D.** warning

Q3. Which of these is a runtime error?

- **A.** missing semicolon
- **B.** dividing an int by zero
- **C.** misspelled keyword
- **D.** unmatched brace

Q4. What does this print?

```
int a = 7, b = 2;
System.out.println(a / b);
```

Your answer: _____

Q5. In "Hello.java:3: error: ';' expected" what does 3 mean?

- **A.** 3 errors
- **B.** line number
- **C.** column
- **D.** error code

Interview / exam questions

- Difference between compile-time and runtime errors?
- What is a stack trace?
- How do you debug a logic error?
- What is an exception?

Watch & read

Video: Syntax Errors, Runtime Errors & Logical Errors in Java (Demo) — CodeWithHarry · 17:51. *The three kinds of errors and how to recognise them*

Video: Java Tutorial - Debug syntax errors vs runtime exceptions — LinkedIn Learning · 5:34. *Reading compiler messages and fixing common syntax errors*

REVISION CARD (30–60 SECONDS)

• Compile-time: compiler refuses (syntax/type). Runtime: exception while running. Logic: wrong answer, no message.

• Read the first compiler error; use the line number.

• Stack trace: exception type, message, where.

• Check outputs against hand examples.

0.6 What Is an Algorithm?

Programming & Java Setup · Beginner · Core programming & data structures · about 30 min · ★ on the minimum-viable path

An algorithm is a precise, finite list of steps. Learn to describe one, check it for correctness and think about efficiency.

YOU WILL BE ABLE TO

- Define an algorithm and list its essential properties.
- Express an algorithm in plain English, pseudocode and Java.
- Explain correctness and efficiency as the two quality measures.
- Recognise the main families of algorithms you will meet in this course.

Before this lesson: Your First Program: class, main, statements, comments

The idea

An **algorithm** is a set of well-defined instructions — a sequence of steps — that solves a specific problem or performs a task. A recipe is an algorithm for cooking; a route description is an algorithm for travel. To count as an algorithm the steps must be:

- **Unambiguous** — each step has exactly one meaning.
- **Finite** — it stops after a bounded number of steps.
- **Effective** — each step is simple enough to be carried out.
- **Input and output** — it takes zero or more inputs and produces a result.

You can express the same algorithm in several ways: natural language, **pseudocode**, a flowchart, or a programming language. Example — "find the largest number in a list":

```
natural language : Remember the first number. Look at each other number; if it is
 bigger than the remembered one, remember it instead. The
 remembered number at the end is the largest.

pseudocode : largest = list[0]
 for each x in list[1..]
 if x > largest then largest = x
 return largest
```

WHY THIS WORKS

Two questions decide whether an algorithm is good: **Is it correct?** (does it give the right answer for *every* valid input, including edge cases?) and **Is it efficient?** (how much time and memory does it use as the input grows?). A fast wrong algorithm is useless; a correct but hopelessly slow one is also unusable. The rest of this course keeps returning to these two questions — with correctness checked by dry runs and tests, and efficiency expressed with Big-O.

family	idea	examples in this course
searching / sorting	find or order data	linear/binary search, quick sort
number theory	properties of integers	GCD, sieve, φ, CRT
divide and conquer	split, solve, combine	Karatsuba, quick sort

family	idea	examples in this course
greedy	best local choice	weighted string
backtracking	try, undo, try again	N-Queens, maze
dynamic programming	remember sub-answers	grid paths

Syntax

From pseudocode to Java

idea	pseudocode	Java
assignment	`x ← 5`	`x = 5;`
decision	`if a > b then … else …`	`if (a > b) { … } else { … }`
counting loop	`for i from 1 to n`	`for (int i = 1; i <= n; i++)`
conditional loop	`while cond do …`	`while (cond) { … }`
result	`return v`	`return v;`
input / output	`read n` / `print x`	`n = sc.nextInt();` / `System.out.println(x);`

Flowchart symbols: **oval** = start/end, **rectangle** = process step, **diamond** = decision (yes/no), **parallelogram** = input/output, **arrows** = order of execution.

A tiny example

```
int largest = list[0];
for (int i = 1; i < list.length; i++)
 if (list[i] > largest) largest = list[i];
```

Guided example

FindLargest.java — Find the largest number (tiny example)

```
1   import java.util.Scanner;
2
3   public class FindLargest {
4    // An ALGORITHM: a finite list of unambiguous steps that solves a problem.
5    // 1. Take the first number as "largest so far".
6    // 2. Look at each remaining number; if it is bigger, it becomes the largest so far.
7    // 3. When no numbers remain, "largest so far" is the answer.
8    public static void main(String[] args) {
9    Scanner sc = new Scanner(System.in);
10   int n = sc.nextInt();
11   int largest = sc.nextInt();                                    (1)
12   for (int i = 1; i < n; i++) {                                  (2)
13   int x = sc.nextInt();
14   if (x > largest) {                                             (3)
15   largest = x;
16   }
17   }
18   System.out.println("largest = " + largest);                   (4)
19   }
20  }
```

Reads n and then n integers; prints the largest.

Sample input

```
5
3 9 2 14 6
```

Output when you run it

```
largest = 14
```

Line-by-line walkthrough

#	Where	What happens
1	line 11	Step 1: the first number is the largest so far.
2	line 12	Step 2: examine the remaining n − 1 numbers.
3	line 14	A bigger number replaces the remembered one.
4	line 18	Step 3: report the answer.

Dry run: watch it execute

An algorithm in action: a single left-to-right scan that finds the largest value.

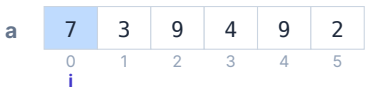
Algorithm: Find the maximum (one scan). **Input:** arr = `7,3,9,4,9,2`.

Pseudocode

```
max = a[0]; maxIndex = 0
for i = 1 .. n−1:
 if a[i] > max:
 max = a[i]; maxIndex = i
print max, maxIndex
```

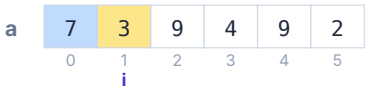
Step 1 of 6. Start by assuming the first element 7 is the maximum.

i = 0 · max = 7 · maxIndex = 0



Step 2 of 6. a[1] = 3 is not larger than 7 (ties do not replace it).

i = 1 · max = 7 · maxIndex = 0



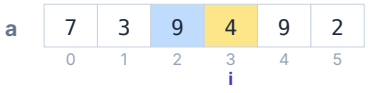
Step 3 of 6. a[2] = 9 is larger than the previous maximum → max = 9.

i = 2 · max = 9 · maxIndex = 2



Step 4 of 6. a[3] = 4 is not larger than 9 (ties do not replace it).

i = 3 · max = 9 · maxIndex = 2



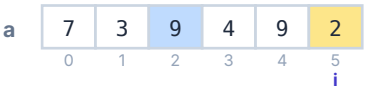
Step 5 of 6. a[4] = 9 is not larger than 9 (ties do not replace it).

i = 4 · max = 9 · maxIndex = 2



Step 6 of 6. a[5] = 2 is not larger than 9 (ties do not replace it).

i = 5 · max = 9 · maxIndex = 2



> **RESULT**
> max = 9 at index 2

Complexity

The algorithm looks at each number exactly once: **n – 1 comparisons → O(n) time, O(1) space.** Doubling the list doubles the work — that "grows in proportion to n" statement is exactly what Big-O captures (see the complexity lessons).

Common mistakes

Starting with 0 as the largest

Problem

```
int largest = 0;
```

Why: If all numbers are negative, 0 is returned although it is not in the list.

Fix: Start with the first element (or Integer.MIN_VALUE).

Forgetting the empty-input case

Problem

```
int largest = sc.nextInt(); // n = 0
```

Why: An algorithm must say what happens for every valid input, including "no numbers".

Fix: Check n ≥ 1 or define the result for an empty list.

An ambiguous step

Problem

```
"Find the biggest element and swap it with a big one."
```

Why: Which big one? Algorithms must be unambiguous.

Fix: Specify exactly which element and how it is chosen.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Find the smallest — *Beginner*

Read n and n integers. Print the smallest.

Example input	Expected output
4 ⏎ 5 2 8 1	1

Hint 1: Mirror the "largest" algorithm with < instead of >.

Exercise 2 · Sum of the numbers — *Beginner*

Read n and n integers. Print their sum.

Example input	Expected output
4 ⏎ 5 2 8 1	16

Hint 1: An accumulator variable that starts at 0.

Exercise 3 · Is it sorted? — *Intermediate*

Read n and n integers. Print YES if they are in non-decreasing order, otherwise NO.

Example input	Expected output
4 ⏎ 1 2 2 3	YES

Hint 1: Compare each number with the previous one.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is an algorithm?

- **A.** A computer program
- **B.** A set of instructions to solve a problem
- **C.** A programming language
- **D.** A hardware component

Q2. Which property is NOT required of an algorithm?

- **A.** finite
- **B.** unambiguous
- **C.** written in Java
- **D.** produces output

Q3. A correct algorithm that takes 10 years on real inputs is...

- **A.** good
- **B.** impractical: efficiency matters too
- **C.** incorrect
- **D.** a heuristic

Q4. To find the largest of n numbers you need at least...

- **A.** 1 comparison
- **B.** n − 1 comparisons
- **C.** n^2 comparisons
- **D.** log n comparisons

Interview / exam questions

- Define an algorithm and its properties.
- Difference between an algorithm and a program?
- How do you prove an algorithm correct?
- Why does efficiency matter?

Watch & read

Video: Flow of Program - Flowcharts & Pseudocode — Kunal Kushwaha · 27:02. *Algorithms, pseudocode and flowcharts*

Video: Data Structures & Algorithms (Episode 1) – Introduction & Why They Matter — Smart Tutorials · 4:49. *Why algorithm choice matters*

REVISION CARD (30–60 SECONDS)

- Algorithm = finite, unambiguous, effective steps from input to output.
- Write as English / pseudocode / code.
- Judge by correctness and efficiency.
- Families: search/sort, number theory, divide & conquer, greedy, backtracking, DP.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The Introduction deck is a single definition and one MCQ ("What is an algorithm?" answer: b). The properties listed here are the standard textbook definition.

Part 0 — Solutions to the exercises

Try the exercises before reading these. Each solution is the exact file that was compiled and tested.

0.1 How Java Works: Source, Bytecode, JVM, JRE, JDK

Exercise 1 · Print your own message

Ex1.java

```java
public class Ex1 {
 public static void main(String[] args) {
 System.out.println("Write once, run anywhere");
 }
}
```

Why it works: A class with a main method that prints the text.

Verified against 1 test case: `(no input)` → `Write once, run anywhere`

0.2 The Java Landscape: History, Features, Editions, JVM Memory

Exercise 1 · Powers of two in memory units

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long megabytes = sc.nextLong();
 long bytes = megabytes * 1024 * 1024; // megabytes is already a long, so no int overflow
 System.out.println(bytes);
 }
}
```

Why it works: Cast to long before multiplying to avoid int overflow.

Verified against 3 test cases: `256⏎` → `268435456`; `1⏎` → `1048576`; `4096⏎` → `4294967296`

0.3 Installing Java on macOS and Choosing an Editor

Exercise 1 · Compile and run it yourself

Ex1.java

```java
public class Ex1 {
 public static void main(String[] args) {
 System.out.println("JDK ready");
 System.out.println("Let us begin");
 }
}
```

Why it works: Two print statements, each followed by a newline.

Verified against 1 test case: `(no input)` → `JDK ready⏎Let us begin`

0.4 Your First Program: class, main, statements, comments

Exercise 1 · Print a box

Ex1.java

```
public class Ex1 {
 public static void main(String[] args) {
 System.out.println("+--+");
 System.out.println("|Hi|");
 System.out.println("+--+");
 }
}
```

Why it works: Each println prints one line.

Verified against 1 test case: `(nc input)` → `+--+⏎|Hi|⏎+--+`

Exercise 2 · Echo a name

Ex2.java

```
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String name = sc.nextLine();
 System.out.println("Hello, " + name + "!");
 }
}
```

Why it works: Read one line, then concatenate.

Verified against 2 test cases: `Asha⏎` → `Hello, Asha!`; `Java Learner⏎` → `Hello, Java Learner!`

Exercise 3 · Sum two numbers

Ex3.java

```
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int b = sc.nextInt();
 System.out.println(a + b);
 }
}
```

Why it works: Scanner.nextInt() reads whitespace-separated integers.

Verified against 3 test cases: `7 5⏎` → `12`; `-3 10⏎` → `7`; `0 0⏎` → `0`

0.5 Compile-time, Runtime and Logic Errors

Exercise 1 · Correct average

Ex1.java

```
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int b = sc.nextInt();
 double average = (a + b) / 2.0; // 2.0 forces floating-point division
 System.out.printf("%.1f%n", average);
 }
}
```

Why it works: Dividing by the double 2.0 forces floating-point division; printf formats one decimal.

Verified against 3 test cases: `2 3⏎` → `2.5`; `10 20⏎` → `15.0`; `-1 0⏎` → `-0.5`

Exercise 2 · Safe division

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int b = sc.nextInt();
 if (b == 0) {
 System.out.println("cannot divide by zero");
 } else {
 System.out.println(a / b);
 }
 }
}
```

Why it works: A guard condition prevents the runtime error.

Verified against 3 test cases: `7 2⏎` → `3`; `5 0⏎` → `cannot divide by zero`; `-7 2⏎` → `-3`

0.6 What Is an Algorithm?

Exercise 1 · Find the smallest

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int smallest = sc.nextInt();
 for (int i = 1; i < n; i++) {
 int x = sc.nextInt();
 if (x < smallest) smallest = x;
 }
 System.out.println(smallest);
 }
}
```

Why it works: Same scan, opposite comparison.

Verified against 3 test cases: `4⏎5 2 8 1⏎` → `1`; `1⏎7⏎` → `7`; `3⏎-5 -9 -1⏎` → `-9`

Exercise 2 · Sum of the numbers

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int sum = 0;
 for (int i = 0; i < n; i++) sum += sc.nextInt();
 System.out.println(sum);
 }
}
```

Why it works: Add each number to a running total.

Verified against 3 test cases: `4⏎5 2 8 1⏎` → `16`; `2⏎-3 3⏎` → `0`; `1⏎100⏎` → `100`

Exercise 3 · Is it sorted?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Is the sequence sorted in non-decreasing order?
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int prev = sc.nextInt();
 boolean sorted = true;
 for (int i = 1; i < n; i++) {
 int x = sc.nextInt();
 if (x < prev) sorted = false;
 prev = x;
 }
 System.out.println(sorted ? "YES" : "NO");
 }
}
```

Why it works: One pass remembering the previous element; any decrease means NO.

Verified against 3 test cases: `4⏎1 2 2 3⏎` → `YES`; `3⏎3 2 1⏎` → `NO`; `1⏎5⏎` → `YES`

Part 0 — Quiz answer key

0.1 How Java Works: Source, Bytecode, JVM, JRE, JDK

	Answer	Why
Q1	B. bytecode (.class files)	javac compiles .java source into .class bytecode.
Q2	C. JDK	The JDK contains the compiler and tools.
Q3	B. It is bytecode executed by a platform-specific JVM	The JVM hides the platform differences.
Q4	B. compiling frequently used bytecode to machine code at run time	Hot code is compiled to native code just in time to speed it up.

0.2 The Java Landscape: History, Features, Editions, JVM Memory

	Answer	Why
Q1	B. heap	Objects live on the heap; references to them may live on the stack.
Q2	B. frees objects that are no longer reachable	Automatic memory management.
Q3	C. Java SE	Java SE (Standard Edition).
Q4	B. StackOverflowError	The call stack is exhausted.

0.3 Installing Java on macOS and Choosing an Editor

	Answer	Why
Q1	B. javac Hello.java	javac is the compiler.
Q2	A. java Hello	Use the class name, without extension.
Q3	B. no JDK (compiler) is installed or on the PATH	The compiler is missing from the PATH.
Q4	C. Java 17	Java 17 (LTS); newer JDKs compile with --release 17.

0.4 Your First Program: class, main, statements, comments

	Answer	Why
Q1	B. the main method	The JVM calls public static void main(String[] args).
Q2	line 3	A semicolon is missing at the end of the statement.
Q3	class | void	A class declaration and a void return type.
Q4	ABC D	print does not add a newline; println does.
Q5	B. Counter.java	A public class must be in a file with exactly its name.

0.5 Compile-time, Runtime and Logic Errors

	Answer	Why
Q1	B. a compile-time error	A type mismatch is found by the compiler.
Q2	C. logic error	It runs fine but is wrong.
Q3	B. dividing an int by zero	ArithmeticException happens while running.
Q4	3	Integer division truncates the .5.

	Answer	Why
Q5	B. line number	Line 3 of Hello.java.

0.6 What Is an Algorithm?

	Answer	Why
Q1	B. A set of instructions to solve a problem	The slide's own MCQ: a set of well-defined instructions that solves a problem.
Q2	C. written in Java	It can be expressed in any notation.
Q3	B. impractical: efficiency matters too	Correctness and efficiency are both needed.
Q4	B. n − 1 comparisons	Each of the other n − 1 numbers must be compared at least once.

Part 1 · Java Syntax & Fundamentals

Everything you need to read and write simple Java programs: data types, casting and overflow, every operator family, Scanner input, if/switch, loops, methods, and how to trace and debug.

	Lesson	Level	Min	Core
1.1	Variables, Constants and Primitive Types	beginner	35	★
1.2	Type Conversion, Casting and Overflow	beginner	35	★
1.3	Arithmetic, Assignment and Increment Operators	beginner	30	★
1.4	Relational, Logical and Ternary Operators; Precedence	beginner	30	★
1.5	Bitwise Operators and Shifts	intermediate	40	★
1.6	Output: print, println, printf and String Concatenation	beginner	30	★
1.7	Input: Scanner, BufferedReader and Command-Line Arguments	beginner	35	★
1.8	Decision Making: if, else, switch	beginner	35	★
1.9	Loops: for, while, do-while, break and continue	beginner	40	★
1.10	Methods: Parameters, Return Values, Scope and Overloading	beginner	40	★
1.11	Tracing and Debugging Programs	beginner	30	★

1.1 Variables, Constants and Primitive Types

Java Syntax & Fundamentals · Beginner · Essential Java syntax · about 35 min · ★ on the minimum-viable path

Named boxes that hold one value of one type: the eight primitives, String, `final`, and naming rules.

YOU WILL BE ABLE TO

- Declare and initialise variables of every primitive type and of type String.
- Choose the right type for a quantity and know the size/range of int and long.
- Make a constant with `final` and follow Java naming conventions.
- Explain the difference between a primitive and a reference type.

Before this lesson: Your First Program: class, main, statements, comments

The idea

A **variable** is a named place in memory that holds a value. Java is **statically typed**: you declare the type once (`int count`), and the compiler guarantees that only values of that type are ever stored there. This catches whole families of mistakes before the program runs.

type	size	holds	example
byte	8 bits	tiny whole numbers −128…127	byte b = 100;
short	16 bits	−32 768…32 767	short s = 30000;
int	32 bits	whole numbers about ±2.1 billion (the default)	int n = 42;
long	64 bits	big whole numbers	long big = 9_000_000_000L;
float	32 bits	decimals (≈7 digits)	float f = 3.14f;
double	64 bits	decimals (≈15 digits), the default	double d = 2.718;
char	16 bits	one character	char c = 'J';
boolean	1 bit*	true / false	boolean ok = true;

String is not a primitive: it is a *reference type* (a class) holding a sequence of characters. Text is written in double quotes `"Java"`; a single character in single quotes `'J'`.

WHY THIS WORKS

Why so many integer types? Memory and range. `int` is almost always right. Use `long` when a value can exceed about 2 billion (factorials, big sums, timestamps), and `double` for measurements. Use the smaller types only for arrays of millions of items, or when the problem says so.

Naming rules and conventions

- Letters, digits, `_` and `$`; cannot start with a digit; cannot be a keyword (`class`, `int`, ...).
- `lowerCamelCase` for variables and methods: `totalMarks`.
- `UPPER_SNAKE_CASE` for constants: `final int MAX_SIZE = 100;`.
- Pick names that say what the value MEANS: `daysLeft`, not `x1`.

Syntax

```
int age; // declaration: reserves a box named age
age = 20; // assignment: puts a value in it
int year = 2026; // declaration + initialisation
int a = 1, b = 2; // several of the same type
final double TAX = 0.18; // constant: assigning again is a compile error
var list = new java.util.ArrayList<String>(); // 'var' lets the compiler infer the type (Java 10+
    )
```

WATCH OUT

A local variable must be assigned **before it is read**: `int x; System.out.println(x);` is a compile error ("variable x might not have been initialized").

A tiny example

```
int apples = 3;
apples = apples + 2; // now 5: the right side is computed first
```

Guided example

VariablesAndTypes.java — A tour of the types (demonstration)

```
1   public class VariablesAndTypes {
2    public static void main(String[] args) {
3    // A variable = a named box that holds ONE value of ONE declared type.
4    byte tiny = 100; // 8-bit -128 .. 127
5    short small = 30_000; // 16-bit
6    int count = 2_000_000_000; // 32-bit about +-2.1 billion (the default for whole numbers)
7    long big = 9_000_000_000L; // 64-bit needs the L suffix                                    (1)
8    float f = 3.14f; // 32-bit decimal, needs the f suffix                                    (2)
9    double d = 2.718281828; // 64-bit decimal (the default for decimals)
10   char letter = 'J'; // one UTF-16 character, single quotes                                 (3)
11   boolean ready = true; // true or false
12   String text = "Java"; // a reference type: a sequence of characters
13
14   final double PI = 3.14159; // final = constant: cannot be reassigned; UPPER_SNAKE_CASE   (4)
15
16   System.out.println(tiny + " " + small + " " + count + " " + big);
17   System.out.println(f + " " + d + " " + letter + " " + ready + " " + text);
18   System.out.println("PI = " + PI);
19   System.out.println("int range: " + Integer.MIN_VALUE + " .. " + Integer.MAX_VALUE);      (5)
20   System.out.println("long max : " + Long.MAX_VALUE);
21   System.out.println("char 'J' has code " + (int) letter);                                 (6)
22   }
23  }
```

Declares one variable of every kind and prints the limits.

Line-by-line walkthrough

#	Where	What happens
1	line 7	Underscores in numbers are only for readability. The `L` suffix is required: without it 9 000 000 000 is read as an int and does not fit.
2	line 8	Decimal literals are double by default; `f` makes it float.
3	line 10	A char is written with single quotes and is stored as a number (its Unicode code).
4	line 14	`final` makes it a constant.
5	line 19	The largest int, 2 147 483 647. Wrapper classes expose the limits of each type.
6	line 21	A cast shows the character's numeric code: 'J' is 74.

Dry run: watch it execute

Swapping needs a temporary variable because assignment overwrites.

Algorithm: Swapping two variables. **Input:** a = `3`, b = `8`.

Pseudocode

```
int temp = a; // keep a safe
a = b;
b = temp;
```

Step 1 of 5. Start: a = 3, b = 8.

a = 3 · b = 8 · temp = —

Step 2 of 5. temp = a stores 3 so it is not lost.

a = 3 · b = 8 · temp = 3

Step 3 of 5. a = b overwrites a with 8. Without temp the old a would be gone.

a = 8 · b = 8 · temp = 3

Step 4 of 5. b = temp gives b the saved 3. Done.

a = 8 · b = 3 · temp = 3

Step 5 of 5. Mistake to avoid: a = b; b = a; would leave BOTH variables equal to 8 because the first line destroyed the original a.

a = 8 · b = 3

> **RESULT**
> a = 8, b = 3

Complexity

Reading or assigning a variable is O(1). A primitive variable takes a fixed number of bytes (int = 4 bytes, long = 8, double = 8).

Common mistakes

Forgetting the L suffix

Problem

```
long big = 9000000000;
```

Why: The literal 9000000000 is an int literal that is too large: "integer number too large".

Fix: long big = 9000000000L;

Using single quotes for a String

Problem

```
String s = 'Java';
```

Why: Single quotes are for ONE char. Java is a String literal and needs double quotes.

Fix: String s = "Java";

Using a variable before it is initialised

Problem

```
int total;
total += 5;
```

Why: total has no value yet; += reads it first.

Fix: int total = 0; total += 5;

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Swap two numbers — *Beginner*

Read two integers a and b and print them swapped: `b a`.

Example input	Expected output
3 8	8 3

Hint 1: You need a third variable to hold one value temporarily.

Exercise 2 · Rectangle — *Beginner*

Read the width and height (decimals). Print `area=…` and `perimeter=…` on two lines using default double printing.

Example input	Expected output
2.5 4	area=10.0 ⏎ perimeter=13.0

Hint 1: double area = w * h;

Exercise 3 · Seconds to h:m:s — *Intermediate*

Read a number of seconds and print it as `hours:minutes:seconds` (no leading zeros).

Example input	Expected output
3725	1:2:5

Hint 1: hours = total / 3600; minutes = (total % 3600) / 60; seconds = total % 60.

Quick check

Answers are in the answer key at the end of this part.

Q1. Which type should hold 5 000 000 000?

- **A.** int
- **B.** short
- **C.** long
- **D.** char

Q2. Which line does not compile?

```
1  int a = 5;
2  double d = 2.5;
3  int c = d;
4  System.out.println(a + c);
```

Which line contains the bug? _____

Q3. Which is a valid variable name?

- **A.** 2ndPlace
- **B.** total-marks
- **C.** totalMarks
- **D.** class

Q4. What is printed?

```
final int LIMIT = 10;
int x = LIMIT * 2;
System.out.println(x);
```

Your answer: _____

Q5. Which of these is NOT a primitive type?

- **A.** int
- **B.** boolean
- **C.** String
- **D.** char

Interview / exam questions

- What are the primitive types in Java and their sizes?
- Difference between float and double?
- What does final mean on a variable?
- Is String a primitive?

Watch & read

Video: Java Tutorial: Variables and Data Types in Java Programming — CodeWithHarry · 24:48. *Declaring variables, primitive types and constants*

Video: Data Types in Java (float,double,char, boolean) | part 2 | JAVA Placement Course — Jenny's Lectures CS IT · 1:05:58. *Primitive types, ranges and final constants*

REVISION CARD (30–60 SECONDS)

- byte/short/int/long; float/double; char; boolean; String is a reference type.
- int default for whole numbers, double for decimals; L and f suffixes.
- final = constant; UPPER_SNAKE_CASE; variables lowerCamelCase.
- Initialise before use.

1.2 Type Conversion, Casting and Overflow

Java Syntax & Fundamentals · Beginner · Essential Java syntax · about 35 min · ★ on the minimum-viable path

When Java converts types for you, when you must cast, and why int arithmetic can silently wrap around.

YOU WILL BE ABLE TO

- Predict the type and value of mixed expressions such as `7 / 2`, `7 / 2.0` and `'A' + 1`.
- Use explicit casts for narrowing conversions and understand what is lost.
- Recognise integer overflow and avoid it by widening before the operation.
- Use `Math.addExact` / `Math.multiplyExact` to detect overflow.

Before this lesson: Variables, Constants and Primitive Types

The idea

Widening (small → big: `int` → `long` → `float` → `double`) is automatic because nothing is lost. **Narrowing** (big → small) can lose information, so Java makes you say so with a **cast**: `(int) 3.99` is `3` — the fraction is **truncated**, not rounded.

```
byte → short → int → long → float → double (automatic →)
 ↑
 char (narrowing needs a cast ←)
```

In an expression the operands are promoted to the **biggest type present** (and at least `int`): `3 + 4.5` is a `double`; `byte + byte` is an `int`.

WHY THIS WORKS

Integers have fixed width. An `int` has 32 bits, so after 2 147 483 647 the next value wraps to −2 147 483 648. No error, no warning: the bits simply roll over, like a car odometer. The cure is to **widen before you compute**: `(long) a * b`, not `(long) (a * b)` — in the second form the multiplication has already overflowed.

Syntax

expression	type	value
7 / 2	int	3
7 / 2.0	double	3.5
(double) 7 / 2	double	3.5 (cast applies to 7 first)
'A' + 1	int	66
(char) ('A' + 1)	char	'B'
(int) 3.99	int	3
(byte) 200	byte	−56 (wraps)
Integer.MAX_VALUE + 1	int	−2147483648

A tiny example

```
int a = 100_000, b = 100_000;
long wrong = a * b; // overflows in int arithmetic first: 1410065408
long right = (long) a * b; // 10000000000
```

Guided example

CastingAndOverflow.java — Casting and overflow tour (demonstration)

```
public class CastingAndOverflow {
 public static void main(String[] args) {
 // WIDENING (automatic, always safe): small type -> bigger type
 int i = 42;
 long l = i;                                                    (1)
 double d = i;
 System.out.println("widening: " + l + " " + d);

 // NARROWING (manual cast, may lose information)
 double pi = 3.99;
 int truncated = (int) pi; // drops the fraction, does NOT round  (2)
 System.out.println("(int) 3.99 = " + truncated);
 System.out.println("rounded = " + Math.round(pi));

 // OVERFLOW wraps around silently
 int max = Integer.MAX_VALUE;
 System.out.println("MAX + 1 = " + (max + 1)); // -2147483648     (3)
 long safe = (long) max + 1; // widen BEFORE adding               (4)
 System.out.println("(long)MAX + 1 = " + safe);

 // Integer division vs floating-point division
 System.out.println("7 / 2 = " + 7 / 2);
 System.out.println("7 / 2.0 = " + 7 / 2.0);
 System.out.println("(double) 7 / 2 = " + (double) 7 / 2);

 // char is a small integer
 char c = 'A';
 c = (char) (c + 2);                                            (5)
 System.out.println("'A' + 2 = " + c);
 System.out.println("'a' - 'A' = " + ('a' - 'A'));

 // the classic trap: a byte cast wraps
 System.out.println("(byte) 200 = " + (byte) 200);

 // detect overflow instead of ignoring it
 try {
 Math.addExact(Integer.MAX_VALUE, 1);                           (6)
 } catch (ArithmeticException e) {
 System.out.println("addExact: " + e.getMessage());
 }
 }
}
```

Every conversion rule in one runnable program.

Line-by-line walkthrough

#	Where	What happens
1	line 5	Widening: no cast needed.
2	line 11	Narrowing: the fraction .99 is cut off. Use Math.round for rounding.
3	line 17	int overflow: wraps to the most negative int.
4	line 18	Cast the operand BEFORE the addition so the arithmetic is done in long.
5	line 28	'A' + 2 is an int (67); casting back gives 'C'.
6	line 37	addExact throws ArithmeticException instead of wrapping silently.

Dry run: watch it execute

A byte silently wraps from 127 to −128.

Algorithm: byte overflow wrap-around. **Input:** start = `120`, add = `5`.

Pseudocode

```
byte b = start
repeat 8 times:
 b = (byte)(b + add) // wraps modulo 256
```

Step 1 of 9. b starts at 120 (binary 01111000).

b = 120 · bits = 01111000

Step 2 of 9. 120 + 5 = 125: fits in a byte.

step = 1 · b + add = 125 · b = 125 · bits = 01111101

Step 3 of 9. 125 + 5 = 130 does not fit; (byte) keeps only the low 8 bits → -126. No error or warning is produced.

step = 2 · b + add = 130 · b = -126 · bits = 10000010

Step 4 of 9. -126 + 5 = -121: fits in a byte.

step = 3 · b + add = -121 · b = -121 · bits = 10000111

Step 5 of 9. -121 + 5 = -116: fits in a byte.

step = 4 · b + add = -116 · b = -116 · bits = 10001100

Step 6 of 9. -116 + 5 = -111: fits in a byte.

step = 5 · b + add = -111 · b = -111 · bits = 10010001

Step 7 of 9. -111 + 5 = -106: fits in a byte.

step = 6 · b + add = -106 · b = -106 · bits = 10010110

Step 8 of 9. -106 + 5 = -101: fits in a byte.

step = 7 · b + add = -101 · b = -101 · bits = 10011011

Step 9 of 9. -101 + 5 = -96: fits in a byte.

step = 8 · b + add = -96 · b = -96 · bits = 10100000

> **RESULT**
> b = -96

Complexity

Casts and arithmetic on primitives are O(1).

Common mistakes

Casting the result instead of the operands

Problem

```
long product = (long) (a * b);
```

Why: a * b is evaluated in int first and may already have overflowed; the cast then widens the wrong value.
Fix: long product = (long) a * b;

Expecting integer division to round

Problem

```
double avg = (a + b) / 2; // a, b ints
```

Why: The division is done on ints and truncated; the result is then widened to double.
Fix: double avg = (a + b) / 2.0;

Assuming char + int is a char

Problem

```
char next = 'a' + 1; // fine only for constants!
char c = 'a';
char d = c + 1; // error
```

Why: char + int is an int. A constant expression that fits may be assigned directly, a variable expression may not.
Fix: char d = (char) (c + 1);

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Big product — *Beginner*

Read two ints a, b (up to 10^9 each) and print a × b exactly.

Example input	Expected output
100000 100000	10000000000

Hint 1: Cast one operand to long before multiplying.

Exercise 2 · Shift a letter — *Intermediate*

Read an uppercase letter and k (0 ≤ k ≤ 1000). Print the letter k places further in the alphabet, wrapping Z to A.

Example input	Expected output
Y 3	B

Hint 1: ('A' + (c - 'A' + k) % 26) then cast to char.

Exercise 3 · Will it overflow? — *Intermediate*

Read two long values a and b. Print OK if a + b fits in an int, otherwise OVERFLOW.

Example input	Expected output
2000000000 2000000000	OVERFLOW

Hint 1: Compute the sum as long and compare with Integer.MIN_VALUE / MAX_VALUE.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
System.out.println(7 / 2 + 7 / 2.0);
```

Your answer: _____

Q2. What is printed?

```
int x = Integer.MAX_VALUE;
x++;
System.out.println(x < 0);
```

Your answer: _____

Q3. Which statement computes 100000 × 100000 correctly?

- **A.** long p = 100000 * 100000;
- **B.** long p = (long) (100000 * 100000);
- **C.** long p = 100000L * 100000;
- **D.** int p = 100000 * 100000;

Q4. What is printed?

```
char c = 'A';
c += 2;
System.out.println(c);
```

Your answer: _____

Q5. (int) 9.99 equals…

- **A.** 9
- **B.** 10
- **C.** 9.99
- **D.** 0

Interview / exam questions

- What is integer overflow and how do you detect it?
- Difference between implicit and explicit conversion?
- Why does (long)(a*b) not fix overflow?
- What does (int) -3.7 give?

Watch & read

Video: Java Type Casting Explained | Widening & Narrowing Casting with Examples | KTU B.Tech, BCA, MCA — TECH MARIA · 14:13. *Implicit widening and explicit narrowing casts*

Video: The Integer Overflow Error — Uma Abu · 4:47. *What happens when an int overflows*

REVISION CARD (30–60 SECONDS)

- Widening automatic; narrowing needs a cast and may lose data.
- int/int is integer division; use 2.0 or a cast.
- Overflow wraps silently; widen BEFORE the operation; Math.xxxExact detects.
- char is a number: char + int → int.

1.3 Arithmetic, Assignment and Increment Operators

Java Syntax & Fundamentals · Beginner · Essential Java syntax · about 30 min · ★ on the minimum-viable path

`+ - * / %`, compound assignment, `++` and `--` — and the two jobs of `%` and `/` that every algorithm uses.

YOU WILL BE ABLE TO

- Evaluate expressions with `+ - * / %`, including negative operands.
- Use `+=`, `-=`, `*=`, `/=`, `%=` and the prefix/postfix forms of `++` and `--`.
- Extract and remove the last digit of a number using `%` and `/`.
- Use `Math` methods for powers, roots and rounding.

Before this lesson: Type Conversion, Casting and Overflow

The idea

operator	meaning	example	result
+	add (also joins Strings)	17 + 5	22
-	subtract	17 - 5	12
*	multiply	17 * 5	85
/	divide (integer division for ints)	17 / 5	3
%	remainder (modulo)	17 % 5	2

WHY THIS WORKS

`/` and `%` are a pair: `a == (a / b) * b + a % b`. On the clock, `% 12` wraps around; on the decimal digits of a number, `% 10` is the **last digit** and `/ 10` removes it. Loops built from these two operations drive digit sums, reversal, base conversion, GCD and half of the algorithms in this course.

Increment/decrement: `x++` means "use the value, then add 1"; `++x` means "add 1, then use the value". As a statement on its own they behave the same.

```
int x = 5;
int y = x++; // y = 5, x = 6 (post-increment: old value)
int z = ++x; // x = 7, z = 7 (pre-increment : new value)
```

Compound assignment `a += b` is shorthand for `a = (type) (a + b)` — it even includes an implicit cast, so `byte b = 10; b += 5;` compiles while `b = b + 5;` does not.

Syntax

Math method	meaning	example
Math.abs(x)	absolute value	Math.abs(-7) = 7
Math.pow(a, b)	a to the power b (double)	Math.pow(2, 10) = 1024.0
Math.sqrt(x)	square root	Math.sqrt(81) = 9.0
Math.max(a, b) / min	larger / smaller	Math.max(3, 9) = 9
Math.floor / ceil / round	round down / up / nearest	Math.round(2.5) = 3

Math method	meaning	example
Math.floorMod(a, b)	modulo that is never negative	Math.floorMod(-7, 3) = 2

A tiny example

```
int n = 4821;
int last = n % 10; // 1
n = n / 10; // 482
```

Guided example

ArithmeticOperators.java — Arithmetic operators in action (demonstration)

```
1  public class ArithmeticOperators {
2   public static void main(String[] args) {
3   int a = 17, b = 5;
4   System.out.println("a + b = " + (a + b));
5   System.out.println("a - b = " + (a - b));
6   System.out.println("a * b = " + (a * b));
7   System.out.println("a / b = " + (a / b) + " (integer division)");
8   System.out.println("a % b = " + (a % b) + " (remainder)");                    (1)
9   System.out.println("-17 % 5 = " + (-17 % 5) + " (sign follows the left operand)");  (2)
10
11  // increment / decrement
12  int x = 5;
13  int y = x++; // y gets the OLD value (5), then x becomes 6                    (3)
14  int z = ++x; // x becomes 7 first, then z gets 7                             (4)
15  System.out.println("x=" + x + " y=" + y + " z=" + z);
16
17  // compound assignment
18  int n = 10;
19  n += 5; // n = n + 5
20  n *= 2; // n = n * 2
21  n %= 7; // n = n % 7                                                         (5)
22  System.out.println("n = " + n);
23
24  // two classic uses of / and %
25  int number = 4821;
26  System.out.println("last digit : " + number % 10);                          (6)
27  System.out.println("without it : " + number / 10);
28   }
29  }
```

Run it and match every printed line to a rule.

Line-by-line walkthrough

#	Where	What happens
1	line 8	Remainder after division: 17 = 3·5 + 2.
2	line 9	The sign of the result follows the LEFT operand: −2, not 3. Use Math.floorMod when you want a non-negative result.
3	line 13	Post-increment: y receives the old value of x.
4	line 14	Pre-increment: x is incremented first.
5	line 21	n = n % 7: 30 % 7 = 2.

#	Where	What happens
6	line 26	The last digit.

Dry run: watch it execute

Precedence and integer division, one reduction at a time.

Algorithm: Evaluating an int expression. **Input:** expr = `7 + 3 * 2 - 8 / 3 % 2`.

Pseudocode

```
1. innermost ( ) first
2. * / % left to right
3. + - left to right
int division truncates; % keeps the dividend's sign
```

Step 1 of 6. Start with the expression as written.
expression = 7 + 3 * 2 - 8 / 3 % 2

Step 2 of 6. Apply 3 * 2 = 6.
operator = * · result = 6 · expression = 7 + 6 - 8 / 3 % 2

Step 3 of 6. Apply 8 / 3 = 2 (integer division drops the fraction: 2.6666666666666665 → 2).
operator = / · result = 2 · expression = 7 + 6 - 2 % 2

Step 4 of 6. Apply 2 % 2 = 0 (the remainder has the sign of the left operand).
operator = % · result = 0 · expression = 7 + 6 - 0

Step 5 of 6. Apply 7 + 6 = 13.
operator = + · result = 13 · expression = 13 - 0

Step 6 of 6. Apply 13 - 0 = 13.
operator = - · result = 13 · expression = 13

> **RESULT**
> 13

Complexity

Each arithmetic operation is O(1). Digit-peeling loops run once per digit: $O(\log_{10} n)$.

Common mistakes

Using ^ for powers

Problem

```
int sq = x ^ 2;
```

Why: `^` is bitwise XOR, not exponent: 5 ^ 2 = 7.
Fix: int sq = x * x; // or (int) Math.pow(x, 2)

Negative remainders

Problem

```
int idx = (i - 1) % n; // i = 0
```

Why: −1 % n is −1: an invalid array index.
Fix: int idx = ((i - 1) % n + n) % n; // or Math.floorMod(i - 1, n)

Using ++ twice in one expression

Problem

```
int r = i++ + i++;
```

Why: Hard to read and easy to get wrong; the value depends on evaluation order.

Fix: Use separate statements.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Digit sum — *Beginner*

Read a positive integer and print the sum of its digits.

Example input	Expected output
4821	15

Hint 1: while (n > 0) { sum += n % 10; n /= 10; }

Exercise 2 · Reverse the digits — *Beginner*

Read a positive integer and print it reversed (trailing zeros of the input vanish: 1200 → 21).

Example input	Expected output
1200	21

Hint 1: reversed = reversed * 10 + n % 10

Exercise 3 · Days to weeks — *Beginner*

Read a number of days and print `W weeks D days`.

Example input	Expected output
30	4 weeks 2 days

Hint 1: days / 7 and days % 7

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
int x = 5;
int y = x++ + ++x;
System.out.println(y + " " + x);
```

Your answer: _____

Q2. What is -17 % 5?

```
System.out.println(-17 % 5);
```

Your answer: _____

Q3. Which expression gives the last digit of n?

- **A.** n / 10
- **B.** n % 10
- **C.** n * 10

- **D.** n - 10

Q4. What is printed?

```
int n = 10;
n += 5;
n *= 2;
n %= 7;
System.out.println(n);
```

Your answer: _____

Q5. What does 5 ^ 2 compute in Java?

- **A.** 25
- **B.** 7
- **C.** 10
- **D.** 3

Interview / exam questions

- Difference between ++i and i++?
- What does % do for negative numbers?
- How do you compute x^2 without Math.pow?
- How to get digits of a number?

Watch & read

Video: Increment and Decrement Operators in Java — Neso Academy · 5:14. *Arithmetic, compound assignment and ++/-- operators*

Video: The Division Operator in Java — Neso Academy · 5:21. *Integer division, modulus and operator precedence*

REVISION CARD (30–60 SECONDS)

- / on ints truncates; % is the remainder (sign of the left operand).
- x++ yields old value; ++x yields new.
- n % 10 = last digit; n / 10 = drop it.
- ^ is XOR, not power.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The "Basic I/O & Operators" deck introduces printing, input and arithmetic together; operators are covered here and I/O in the next two lessons.

1.4 Relational, Logical and Ternary Operators; Precedence

Java Syntax & Fundamentals · Beginner · Essential Java syntax · about 30 min · ★ on the minimum-viable path

Comparing values, combining conditions with `&&` `||` `!`, short-circuiting, `?:`, and the order Java evaluates things.

YOU WILL BE ABLE TO

- Write boolean expressions with `< <= > >= == !=`.
- Combine conditions with `&&`, `||` and `!` and explain short-circuit evaluation.
- Use the ternary operator `cond ? a : b` where it improves clarity.
- Read an expression using precedence and associativity rules (and add parentheses when in doubt).

Before this lesson: Arithmetic, Assignment and Increment Operators

The idea

Relational operators compare two values and produce a `boolean`: `<`, `<=`, `>`, `>=`, `==` (equal), `!=` (not equal). **Logical operators** combine booleans: `&&` (AND), `||` (OR), `!` (NOT).

a	b	a && b	a || b	!a
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

WHY THIS WORKS

`&&` and `||` are **short-circuit**: in `a && b`, if `a` is false the whole result is false, so `b` is never evaluated; in `a || b`, if `a` is true, `b` is skipped. This is how `if (i < n && a[i] > 0)` is safe: when `i == n` the array access never happens.

The **ternary operator** is an if-else that produces a value: `String kind = (n % 2 == 0) ? "even" : "odd";`.

Precedence (highest → lowest) and associativity

level	operators	associativity
1	() [] .	left → right
2	x++ x-- ++x --x +x -x ! ~ (cast)	right → left
3	* / %	left → right
4	+ -	left → right
5	<< >> >>>	left → right
6	< <= > >= instanceof	left → right
7	== !=	left → right
8	& then ^ then |	left → right
9	&& then ||	left → right

level	operators	associativity
10	?:	right → left
11	= += -= *= ...	right → left

Syntax

group	operators	notes
relational	`== != < > <= >=`	produce a boolean; compare numbers/chars (use `.equals` for Strings)
logical	`&& || !`	`&&` and `||` short-circuit (the right side may not run)
non-short-circuit	`& | ^`	on booleans they always evaluate both sides
ternary	`cond ? a : b`	an expression: it has a value (a or b)

Forms

```
int max = (a > b) ? a : b; // ternary
boolean ok = x >= 0 && x <= 100; // range check
boolean either = a == 0 || b == 0;
// precedence, high to low: ! * / % + - < <= > >= == != && || ?: =
```

A tiny example

```
boolean valid = age >= 18 && age <= 65;
int bigger = (a > b) ? a : b;
```

Guided example

LogicalOperators.java — Logic in action (demonstration)

```
1  public class LogicalOperators {
2   static boolean noisy(String name, boolean value) {
3   System.out.println(" evaluating " + name);
4   return value;
5   }
6
7   public static void main(String[] args) {
8   int age = 20;
9   boolean hasTicket = true;
10
11  // relational operators produce a boolean
12  System.out.println(age >= 18); // true
13  System.out.println(age == 20); // true
14  System.out.println(age != 20); // false
15
16  // logical operators combine booleans
17  System.out.println(age >= 18 && hasTicket); // AND
18  System.out.println(age < 13 || age > 65); // OR
19  System.out.println(!hasTicket); // NOT
20
21  // short-circuit: the right side is skipped when the answer is already known
22  System.out.println("false && ...:");
23  boolean r1 = noisy("left", false) && noisy("right", true);                    (1)
24  System.out.println("true || ...:");
25  boolean r2 = noisy("left", true) || noisy("right", false);
26
27  // ternary operator: condition ? valueIfTrue : valueIfFalse
28  int a = 7, b = 12;
29  int larger = (a > b) ? a : b;                                                 (2)
30  String kind = (a % 2 == 0) ? "even" : "odd";
31  System.out.println("larger = " + larger + ", a is " + kind);
32
33  // precedence: && binds tighter than ||
34  System.out.println(true || false && false); // true (&& first)               (3)
35  System.out.println((true || false) && false); // false                       (4)
36  }
37 }
```

Helper method `noisy` prints when it is evaluated, which proves short-circuiting.

Line-by-line walkthrough

#	Where	What happens
1	line 23	Only "left" is printed: the right operand is never evaluated.
2	line 29	Ternary: choose a value by a condition.
3	line 34	&& binds tighter than ||, so this is true || (false && false) = true.
4	line 35	Parentheses override precedence: false.

Dry run: watch it execute

Short-circuit evaluation avoids a division by zero.

Algorithm: Short-circuit evaluation. **Input:** x = `0`, y = `10`, z = `5`.

Pseudocode

```
boolean r = x != 0 && y / x > 2 || z == 5;
String s = r ? "yes" : "no";
```

Step 1 of 5. Precedence: && binds tighter than ||, so the expression is (x != 0 && y / x > 2) || (z == 5).

x = 0 · y = 10 · z = 5

Step 2 of 5. x != 0 → 0 != 0 is false.

x != 0 = false

Step 3 of 5. The left operand of && is false, so the whole && is false. y / x is NOT evaluated — there is no division by zero.

x != 0 && ... = false · y / x evaluated = false

Step 4 of 5. The left operand of || is false, so z == 5 decides: 5 == 5 is true.

z == 5 = true

Step 5 of 5. Ternary: r ? "yes" : "no" picks "yes".

r = true · s = yes

> **RESULT**
> r = true, s = yes

Complexity

All O(1). Short-circuiting can also save time: place the cheaper / more likely-to-fail test first.

Common mistakes

= instead of ==

Problem

```
if (x = 5) { ... }
```

Why: Assignment, not comparison. For ints this is a compile error; for booleans it silently assigns.

Fix: if (x == 5)

Chained comparison

Problem

```
if (0 <= x <= 10)
```

Why: Java evaluates (0 <= x) → boolean, then boolean <= 10: compile error.

Fix: if (0 <= x && x <= 10)

Comparing Strings with ==

Problem

```
if (name == "Asha")
```

Why: == compares references, not content; it may work by accident and fail later.

Fix: if (name.equals("Asha"))

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Leap year — *Beginner*

Read a year. Print `leap` if it is a leap year (divisible by 4, except centuries unless divisible by 400), otherwise `common`.

Example input	Expected output
2000	leap

Hint 1: (y % 4 == 0 && y % 100 != 0) || y % 400 == 0

Exercise 2 · Maximum of three — *Beginner*

Read three integers and print the largest using only ternary operators.

Example input	Expected output
4 9 7	9

Hint 1: Nest a ternary inside a ternary.

Exercise 3 · In range? — *Intermediate*

Read x, lo and hi. Print IN if lo ≤ x ≤ hi, else OUT.

Example input	Expected output
5 1 10	IN

Hint 1: Two comparisons joined by &&.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
int a = 5, b = 0;
System.out.println(b != 0 && a / b > 1);
```

Your answer: _____

Q2. What is printed?

```
System.out.println(true || false && false);
```

Your answer: _____

Q3. Which is the correct way to test 1 ≤ x ≤ 10?

- **A.** 1 <= x <= 10
- **B.** 1 <= x && x <= 10
- **C.** 1 <= x & <= 10
- **D.** x in 1..10

Q4. What does `5 > 3 ? "yes" : "no"` evaluate to?

- **A.** true
- **B.** "yes"
- **C.** "no"
- **D.** a compile error

Q5. Which line is a classic bug?

```
1  int score = 90;
2  if (score = 100) {
3   System.out.println("perfect");
4  }
```

Which line contains the bug? _____

Interview / exam questions

- What is short-circuit evaluation?
- Difference between & and &&?
- Operator precedence of && vs ||?
- How do you compare Strings?

Watch & read

Video: Java Tutorial: Relational and Logical Operators in Java — CodeWithHarry · 12:25. *Comparison, &&, ||, ! and the ternary operator*

Video: Logical Operators in Java — Neso Academy · 6:19. *Short-circuit evaluation and operator precedence*

REVISION CARD (30–60 SECONDS)

- < <= > >= == != give boolean; && || ! combine.
- && and || short-circuit: right side may be skipped.
- cond ? a : b is an expression.
- Precedence: arithmetic > comparison > == > && > || > ?: > =; use parentheses.

1.5 Bitwise Operators and Shifts

Java Syntax & Fundamentals · Intermediate · Essential Java syntax · about 40 min · ★ on the minimum-viable path

`& | ^ ~ << >> >>>` — *operating on the individual bits of an integer, with the everyday bit tricks.*

YOU WILL BE ABLE TO

- Apply AND, OR, XOR, NOT and the three shifts to small numbers by hand, in binary.
- Read, set, clear and toggle a given bit using masks.
- Use `x & (x - 1)` and `x & -x` for power-of-two tests and bit counting.
- Tell `>>` (arithmetic) from `>>>` (logical).

Before this lesson: Relational, Logical and Ternary Operators; Precedence

The idea

Every integer is stored as a row of bits. Bitwise operators work on **each bit position independently**:

```
 a = 1100 (12) b = 1010 (10)
a & b = 1000 (8) both bits 1 AND
a | b = 1110 (14) at least one bit 1 OR
a ^ b = 0110 (6) bits differ XOR
~a = ...0011 flip every bit NOT (= -a - 1)
a << 2 = 110000 (48) shift left, fill 0 × 2²
a >> 2 = 0011 (3) shift right, keep sign ÷ 2²
```

WHY THIS WORKS

A **mask** is a number whose 1-bits select the positions you care about. `x & mask` keeps only those bits; `x | mask` forces them to 1; `x ^ mask` flips them; `x & ~mask` clears them. With `1 << i` you can build a mask for bit i, which gives the four universal bit operations below.

goal	expression
read bit i	(x >> i) & 1
set bit i	x | (1 << i)
clear bit i	x & ~(1 << i)
toggle bit i	x ^ (1 << i)
is x a power of two? (x > 0)	(x & (x - 1)) == 0
clear the lowest set bit	x & (x - 1)
isolate the lowest set bit	x & -x
is x odd?	(x & 1) == 1

Shifts: `x << k` multiplies by 2^k (until overflow). `x >> k` divides by 2^k rounding toward −∞ and **copies the sign bit** (arithmetic shift), so −8 >> 1 is −4. `x >>> k` shifts in zeros (logical shift): −8 >>> 28 is 15 for an int.

Syntax

LITERALS AND A TRAP

Binary literals use `0b`: `0b1100` is 12; hexadecimal uses `0x`: `0xFF` is 255. In Java the shift distance is taken mod 32 for int (mod 64 for long): `1 << 32` is 1, not 0.

A tiny example

```
int x = 0b0101_0000;
boolean bit4 = ((x >> 4) & 1) == 1; // true
x = x | (1 << 0); // 0b0101_0001
```

Guided example

BitwiseOperators.java — Bitwise operators in binary (demonstration)

```
1  public class BitwiseOperators {
2   static String bits(int v) {                                                    (1)
3   String s = Integer.toBinaryString(v & 0xFF); // show the low 8 bits
4   return "0".repeat(8 - s.length()) + s;
5   }
6
7   public static void main(String[] args) {
8   int a = 0b1100; // 12
9   int b = 0b1010; // 10
10  System.out.println("a = " + bits(a) + " (" + a + ")");
11  System.out.println("b = " + bits(b) + " (" + b + ")");
12  System.out.println("a & b = " + bits(a & b) + " AND: 1 only where both are 1");  (2)
13  System.out.println("a | b = " + bits(a | b) + " OR : 1 where either is 1");
14  System.out.println("a ^ b = " + bits(a ^ b) + " XOR: 1 where they differ");
15  System.out.println("~a = " + bits(~a) + " NOT: flip every bit (= -a - 1 = " + (~a) + ")  (3)
        ");
16  System.out.println("a << 2 = " + bits(a << 2) + " multiply by 4 (" + (a << 2) + ")");
17  System.out.println("a >> 2 = " + bits(a >> 2) + " divide by 4 (" + (a >> 2) + ")");
18  System.out.println("-8 >> 1 = " + (-8 >> 1) + " (arithmetic: sign is kept)");  (4)
19  System.out.println("-8 >>> 28 = " + (-8 >>> 28) + " (logical: zeros come in from the  (5)
        left)");
20
21  // everyday bit tricks
22  int x = 0b0101_0000;
23  int bit = 4;
24  System.out.println("bit 4 of x is " + ((x >> bit) & 1));                        (6)
25  System.out.println("set bit 0 : " + bits(x | (1 << 0)));
26  System.out.println("clear bit 4 : " + bits(x & ~(1 << 4)));
27  System.out.println("toggle bit 6 : " + bits(x ^ (1 << 6)));
28  System.out.println("is 64 a power of two? " + (64 > 0 && (64 & (64 - 1)) == 0));
29  }
30 }
```

A helper prints 8-bit patterns so you can verify each operator by eye.

Line-by-line walkthrough

#	Where	What happens
1	line 2	Masking with 0xFF shows only the low 8 bits; padded with zeros for alignment.
2	line 12	Only positions that are 1 in both survive.
3	line 15	Flips every bit; for two's complement ~a == −a − 1.
4	line 18	Arithmetic shift keeps the sign: −4.
5	line 19	Logical shift brings in zeros at the top: 15.
6	line 24	Read one bit: shift it to position 0, then mask with 1.

Dry run: watch it execute

Every bitwise operator on two bytes, in binary.

Algorithm: Bitwise operators in binary. **Input:** a = `12`, b = `10`, k = `2`.

Pseudocode

```
a & b // 1 only where both are 1
a | b // 1 where either is 1
a ^ b // 1 where they differ
~a // flip every bit
a << k // multiply by 2^k
a >> k // divide by 2^k
```

Step 1 of 7. a = 12 is 00001100 and b = 10 is 00001010 in 8 bits.

a = 12 · b = 10

a = 12	0	0	0	0	1	1	0	0
b = 10	0	0	0	0	1	0	1	0

Step 2 of 7. a & b = 8. AND: 1 only where both bits are 1.

a & b = 8 · low 8 bits = 00001000

a = 12	0	0	0	0	1	1	0	0
b = 10	0	0	0	0	1	0	1	0
a & b	0	0	0	0	1	0	0	0

Step 3 of 7. a | b = 14. OR: 1 where at least one bit is 1.

a | b = 14 · low 8 bits = 00001110

a = 12	0	0	0	0	1	1	0	0
b = 10	0	0	0	0	1	0	1	0
a & b	0	0	0	0	1	0	0	0
a | b	0	0	0	0	1	1	1	0

Step 4 of 7. a ^ b = 6. XOR: 1 where the bits differ.

a ^ b = 6 · low 8 bits = 00000110

a = 12	0	0	0	0	1	1	0	0
b = 10	0	0	0	0	1	0	1	0
a & b	0	0	0	0	1	0	0	0
a | b	0	0	0	0	1	1	1	0
a ^ b	0	0	0	0	0	1	1	0

Step 5 of 7. ~a = -13. NOT: every bit flips (a Java int has 32 bits, so the value is negative; the low 8 bits are shown).

~a = -13 · low 8 bits = 11110011

a = 12	0	0	0	0	1	1	0	0
b = 10	0	0	0	0	1	0	1	0
a & b	0	0	0	0	1	0	0	0
a | b	0	0	0	0	1	1	1	0
a ^ b	0	0	0	0	0	1	1	0
~a = −	1	1	1	1	0	0	1	1

Step 6 of 7. a << 2 = 48. shift left 2: the same as a * 4; bits move toward the high end.

a << 2 = 48 · low 8 bits = 00110000

a = 12	0	0	0	0	1	1	0	0
b = 10	0	0	0	0	1	0	1	0
a & b	0	0	0	0	1	0	0	0
a | b	0	0	0	0	1	1	1	0
a ^ b	0	0	0	0	0	1	1	0
~a = −	1	1	1	1	0	0	1	1
a << 2	0	0	1	1	0	0	0	0

Step 7 of 7. a >> 2 = 3. shift right 2: the same as a / 4 (integer division).

a >> 2 = 3 · low 8 bits = 00000011

a = 12	0	0	0	0	1	1	0	0
b = 10	0	0	0	0	1	0	1	0
a & b	0	0	0	0	1	0	0	0
a | b	0	0	0	0	1	1	1	0
a ^ b	0	0	0	0	0	1	1	0
~a = −	1	1	1	1	0	0	1	1
a << 2	0	0	1	1	0	0	0	0
a >> 2	0	0	0	0	0	0	1	1

> **RESULT**
> a & b = 8, a | b = 14, a ^ b = 6, ~a = -13, a << 2 = 48, a >> 2 = 3

Complexity

Every bitwise operation on a machine word is O(1) — a single CPU instruction. Algorithms using them to process one bit at a time on a b-bit number cost O(b) = O(log value).

Common mistakes

Mixing & with &&

Problem

```
if (x > 0 & y / x > 1)
```

Why: & does not short-circuit: y / x is evaluated even if x == 0 → ArithmeticException.

Fix: if (x > 0 && y / x > 1)

Forgetting precedence

Problem

```
if (x & 1 == 1)
```

Why: == binds tighter than &: it parses as x & (1 == 1), a type error.

Fix: if ((x & 1) == 1)

Shifting by negative or too large amounts

Problem

```
int y = 1 << 32; // 1
```

Why: The shift distance is reduced mod 32: you get 1, not 0.

Fix: Use long (1L << 32) or check the distance.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Power of two? — *Beginner*

Read a positive integer (up to 10^{18}). Print YES if it is a power of two, otherwise NO.

Example input	Expected output
64	YES

Hint 1: n & (n − 1) clears the lowest set bit; a power of two has only one.

Exercise 2 · Count the 1 bits — *Intermediate*

Read a non-negative int and print how many 1 bits its binary form has (without Integer.bitCount).

Example input	Expected output
255	8

Hint 1: n &= n − 1 removes one set bit each time.

Exercise 3 · Set, clear or toggle — *Intermediate*

Read n, a bit index i (0-based) and an operation word `set`, `clear` or `toggle`. Print the resulting integer.

Example input	Expected output
5 1 set	7

Hint 1: mask = 1 << i; then |, & ~mask, or ^.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is 12 & 10?

```
System.out.println(12 & 10);
```

Your answer: _____

Q2. What is 5 ^ 3?

```
System.out.println(5 ^ 3);
```

Your answer: _____

Q3. Which expression sets bit 3 of x?

- **A.** x & (1 << 3)
- **B.** x | (1 << 3)
- **C.** x ^ 3
- **D.** x << 3

Q4. What is -16 >> 2?

```
System.out.println(-16 >> 2);
```

Your answer: _____

Q5. x & (x - 1) does what?

- **A.** clears the lowest set bit
- **B.** sets the lowest bit
- **C.** negates x
- **D.** doubles x

Interview / exam questions

- How do you check if a number is a power of two?
- Difference between >> and >>>?
- How do you count set bits?
- Swap two variables without a temp using XOR.

Watch & read

Video: Bitwise operators in Java | and, or, not, xor, left shift, right shift & unsigned right shift — Engineering Digest · 17:40. *AND, OR, XOR, NOT and shifts in Java*

Video: How do I Set, Clear, and Toggle a Single Bit? — Jacob Sorber · 9:26. *Common bit tricks: set, clear, toggle and test a bit*

REVISION CARD (30–60 SECONDS)

- & AND, | OR, ^ XOR, ~ NOT, << >> >>>.
- mask = 1 << i; read (x>>i)&1; set |; clear & ~; toggle ^.
- x & (x−1) clears lowest set bit → power-of-two test, popcount.
- << k = ×2^k; >> keeps sign; >>> fills zeros.

1.6 Output: print, println, printf and String Concatenation

Java Syntax & Fundamentals · Beginner · Essential Java syntax · about 30 min · ★ on the minimum-viable path

Every way to show results, including the famous `+` puzzle and number formatting.

YOU WILL BE ABLE TO

- Choose between `print`, `println` and `printf`.
- Predict the output of mixed `+` expressions of numbers and Strings.
- Format numbers with width, precision, zero padding and thousands separators.
- Know the three standard streams: `System.in`, `System.out`, `System.err`.

Before this lesson: Arithmetic, Assignment and Increment Operators

The idea

Java has three **standard streams**: `System.in` (input), `System.out` (normal output) and `System.err` (error messages). The three output methods:

method	what it does
print(x)	prints x, stays on the same line
println(x)	prints x, then moves to the next line
printf(format, args...)	prints formatted text; add %n for a newline

WHY THIS WORKS

`+` has two jobs. Between two numbers it **adds**; as soon as one operand is a String it **joins** (concatenates) — and Java works **left to right**. That is why `3 + 4 + "x"` is `"7x"` but `"x" + 3 + 4` is `"x34"`. Parentheses fix it: `"x" + (3 + 4)` is `"x7"`.

```
System.out.println(3 + 4); // 7
System.out.println("3" + "4"); // 34
System.out.println("" + 3 + 4); // 34
System.out.println(3 + 4 + 3 + " " + 4 + 3); // 10 43 ← the lecture puzzle
System.out.println("Result: " + 3 + 4); // Result: 34
System.out.println("Result: " + (3 + 4)); // Result: 7
```

format	meaning	example → output
%d	integer	printf("%d", 42) → 42
%5d / %-5d / %05d	width / left-aligned / zero padded	[42] [42] [00042]
%f / %.2f	decimal / 2 decimals	3.141593 / 3.14
%,d	thousands separator	1,234,567
%s	String (any object)	Asha
%c %b %x	char / boolean / hex	z true ff
%n	platform newline	

A `char` prints as a character (`char c = 65;` prints `A`); cast it to see the number: `(int) c`. `char + int` is an int: `y + 1` prints 66 for `y = 'A'`.

Syntax

specifier	meaning	example → output
%d	int / long	`printf("%d", 42)` → `42`
%5d %-5d %05d	width 5, left-justified, zero-padded	`[ 42] [42 ] [00042]`
%,d	thousands separator	`printf("%,d", 1234567)` → `1,234,567`
%f %.2f %8.3f	double (6 decimals default), 2 decimals, width 8 / 3 decimals	`printf("%.2f", 3.14159)` → `3.14`
%s %-10s	String (any object), left-justified in 10	`printf("%s", "hi")` → `hi`
%c %b	char, boolean	`printf("%c", 65)` → `A`
%x %o %e	hex, octal, scientific	`printf("%x", 255)` → `ff`
%n %%	line break, literal percent	

```
System.out.print("no newline");
System.out.println("with newline");
System.out.printf("%-8s|%6.2f%n", "tea", 1.5); // formatted, you add %n yourself
String line = String.format("%03d", 7); // same specifiers, returns a String
```

A tiny example

```
System.out.printf("%s scored %d (%.1f%%)%n", "Asha", 45, 90.0);
// Asha scored 45 (90.0%)
```

Guided example

PrintingDemo.java — Printing in every style (demonstration)

```
1  public class PrintingDemo {
2   public static void main(String[] args) {
3   // print vs println vs printf
4   System.out.print("1. print ");
5   System.out.print("2. print");
6   System.out.println(); // just a newline                                    (1)
7   System.out.println("3. println");
8   System.out.printf("4. printf: %d items cost %.2f each, name=%s%n", 3, 4.5, "pen");
9
10  // the + operator: numbers ADD until a String appears, then it JOINS
11  int a = 3, b = 4;
12  System.out.println(a + b); // 7
13  System.out.println("3" + "4"); // 34
14  System.out.println("" + a + b); // 34
15  System.out.println(3 + 4 + a + " " + b + a);// 10 43                      (2)
16  System.out.println("Result: " + a + b); // Result: 34
17  System.out.println("Result: " + (a + b)); // Result: 7
18
19  // char prints as a character, not its number
20  char x = 65;                                                              (3)
21  char y = 'A';
22  System.out.println(x); // A
23  System.out.println((int) x); // 65
24  System.out.println(y + 1); // 66 (char + int is an int)                    (4)
25  System.out.println((char) (y + 1)); // B
26
27  // formatting
28  System.out.printf("[%5d] [%-5d] [%05d]%n", 42, 42, 42);                   (5)
29  System.out.printf("[%8.3f] [%.0f] [%,d]%n", Math.PI, 2.5, 1234567);
30  System.out.printf("%s is %d years old%n", "Asha", 20);
31  System.out.println(String.format("%x %o %c %b", 255, 8, 'z', true));
32   }
33  }
```

The lecture's "guess the output" examples plus formatting.

Line-by-line walkthrough

#	Where	What happens
1	line 6	println with no argument prints just a newline.
2	line 15	Left to right: 3 + 4 + 3 = 10 (numbers), then " " turns it into a String, then b and a are JOINED: "10 43".
3	line 20	A char initialised with a number prints as its character.
4	line 24	'A' + 1 is an int (66), not a char.
5	line 28	Width 5, left-justified, zero padded.

Dry run: watch it execute

How each printf specifier pads its argument.

Algorithm: How printf formats one line. **Input:** name = `Pen`, price = `2.5`, qty = `7`.

Pseudocode

```
System.out.printf("%-8s|%8.2f|%04d%n", name, price, qty);
%-8s left-justify in 8 columns
%8.2f 2 decimals, right-justify in 8
%04d integer, zero-padded to 4
```

Step 1 of 5. %-8s takes the first argument (a String) and left-justifies it in a field of width 8; "-" means pad on the right. Longer text is never cut.

argument = "Pen" · width = 8 · output = "Pen "

field (· = space)

0	1	2	3	4	5	6	7
P	e	n	·	·	·	·	·

Step 2 of 5. The literal | is copied as it is.

output = Pen |

so far

0	1	2	3	4	5	6	7	8
P	e	n	·	·	·	·	·	|

Step 3 of 5. %8.2f takes the double 2.5: rounds to 2 decimals (HALF_UP) giving 2.50, then right-justifies in width 8.

argument = 2.5 · output = " 2.50"

field (· = space)

0	1	2	3	4	5	6	7
·	·	·	·	2	.	5	0

Step 4 of 5. After another |, %04d takes the int 7: the 0 flag pads with zeros to width 4 → 0007.

argument = 7 · output = "0007"

field

0	1	2	3
0	0	0	7

Step 5 of 5. %n is a platform-independent line break.

line = Pen | 2.50|0007

final line (· = space)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
P	e	n	·	·	·	·	·	|	·	·	·	·	2	.	5	0	|	0	0

20	21
0	7

> **RESULT**
> Pen·····|····2.50|0007 (· = space)

Complexity

Printing is O(length of output). In loops, build a StringBuilder and print once: thousands of tiny println calls are far slower than one large print.

Common mistakes

Forgetting %n or \n in printf

Problem

```
System.out.printf("%d", 5);
System.out.printf("%d", 6); // 56 on one line
```

Why: printf does not add a newline.

Fix: System.out.printf("%d%n", 5);

Wrong format letter

Problem

```
System.out.printf("%d", 3.5);
```

Why: IllegalFormatConversionException: d != java.lang.Double.

Fix: System.out.printf("%.1f", 3.5);

Concatenating before adding

Problem

```
System.out.println("Sum = " + a + b);
```

Why: Prints the digits side by side: "Sum = 34".

Fix: System.out.println("Sum = " + (a + b));

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Times table — *Beginner*

Read n and print `n x i = n·i` for i = 1..5, one line each.

Example input	Expected output
7	7 x 1 = 7 ⏎ 7 x 2 = 14 ⏎ 7 x 3 = 21 ⏎ 7 x 4 = 28 ⏎ 7 x 5 = 35

Hint 1: printf("%d x %d = %d%n", ...)

Exercise 2 · Invoice total — *Intermediate*

Read a unit price (decimal) and a quantity. Print `Total: X` with thousands separators and exactly two decimals.

Example input	Expected output
1999.5 3	Total: 5,998.50

Hint 1: %,.2f

Exercise 3 · Predict, then run — *Beginner*

The program prints the value of `a + b + "=" + a + b` for a = 2, b = 3. Run it and write what it prints (this exercise checks that you reproduce the output exactly).

Example input	Expected output
(no input)	5=23

Hint 1: Left to right.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

Your answer: _____

Q2. What is printed?

```
char c = 'A';
System.out.println(c + 1);
```

Your answer: _____

Q3. Which prints 3.14?

- **A.** printf("%d", 3.14159)
- **B.** printf("%.2f", 3.14159)
- **C.** printf("%s", 3.14159, 2)
- **D.** println(%.2f)

Q4. Which method does NOT end the line?

- **A.** println
- **B.** printf("%n")
- **C.** print
- **D.** println()

Q5. System.err is used for...

- **A.** normal results
- **B.** error messages
- **C.** keyboard input
- **D.** files

Interview / exam questions

- Difference between print and println?
- What does "Hello" + 1 + 2 print?
- How do you format a double to two decimals?
- Why is System.out.println slow in loops?

Watch & read

Video: Java System.out,System.err,append,println,print,printf,format and static import — Brainy Ghosts · 23:10. *print, println and printf formatting*

Video: JAVA || How to format output in java using System.out.printf(...) & String x = String.format(...); — Computer Engineering life · 11:06. *Formatting numbers and strings in output*

REVISION CARD (30–60 SECONDS)

- print (same line), println (newline), printf (format, add %n).
- + adds numbers, joins once a String is involved; left to right.
- %d %f %.2f %s %c %b %x %n; %5d width, %-5d left, %05d zeros, %,d groups.
- char prints as a char; char+int is int.

1.7 Input: Scanner, BufferedReader and Command-Line Arguments

Java Syntax & Fundamentals · Beginner · Essential Java syntax · about 35 min · ★ on the minimum-viable path

Read numbers, words and lines from the keyboard or a file, process input until it ends, and use `args`.

YOU WILL BE ABLE TO

- Read ints, doubles, words and lines with `Scanner`.
- Avoid the `nextInt()` / `nextLine()` newline trap.
- Read until end of input with `hasNext…()`.
- Use `BufferedReader` for fast input and `String[] args` for command-line arguments.

Before this lesson: Output: print, println, printf and String Concatenation

The idea

Input arrives as a stream of characters. **Scanner** splits it into **tokens** (separated by whitespace) and converts them: `nextInt()`, `nextLong()`, `nextDouble()`, `next()` (a word) and `nextLine()` (the rest of the current line).

way	when to use	notes
Scanner(System.in)	most programs	convenient, a bit slow for huge input
BufferedReader + InputStreamReader	large inputs, contests	readLine() returns a String; parse with Integer.parseInt
Console	passwords in a terminal	does not work inside an IDE
String[] args	values given on the command line	always Strings: parse them

WHY THIS WORKS

`nextInt()` reads the digits but leaves the **line break** in the stream. A following `nextLine()` then returns the empty rest of that line instead of the next line. After a numeric read, call `nextLine()` once to consume the newline before reading a full line of text.

```
int age = sc.nextInt();
sc.nextLine(); // eat the newline left behind
String name = sc.nextLine(); // now reads the real next line
```

To process input of unknown length use the **has** methods: `while (sc.hasNextInt()) { … }` ends at end-of-input (Ctrl-D in a macOS terminal). Command-line arguments are passed after the class name: `java Average 67 98 91` → `args = {"67","98","91"}`.

Syntax

```
import java.util.Scanner;
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
double x = sc.nextDouble();
String w = sc.next();
String line = sc.nextLine();
sc.close(); // optional for System.in
```

A tiny example

```
Scanner sc = new Scanner(System.in);
int a = sc.nextInt();
int b = sc.nextInt();
System.out.println(a + b);
```

Guided example

ScannerDemo.java — Scanner in detail (demonstration)

```
1  import java.util.Scanner;
2
3  public class ScannerDemo {
4   public static void main(String[] args) {
5   Scanner sc = new Scanner(System.in);
6   int count = sc.nextInt(); // reads one token and converts it                    (1)
7   double price = sc.nextDouble();
8   String word = sc.next(); // one word (stops at whitespace)
9   sc.nextLine(); // consume the rest of the current line (the newline!)          (2)
10  String sentence = sc.nextLine(); // the whole next line
11  System.out.println("count=" + count + " price=" + price + " word=" + word);
12  System.out.println("sentence=[" + sentence + "]");
13
14  // read until the input ends
15  int total = 0;
16  while (sc.hasNextInt()) {                                                      (3)
17  total += sc.nextInt();
18  }
19  System.out.println("sum of the rest = " + total);
20  }
21 }
```

Sample input: `3 4.5 hello` *then a sentence, then numbers.*

Sample input

```
3 4.5 hello
the quick brown fox
10 20 30
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	Reads one whitespace-delimited token and converts it to int.
2	line 9	Consumes the newline that follows "hello" so the next nextLine() returns the sentence.
3	line 16	True while another int is available — ends cleanly at end-of-input.

FastReader.java — BufferedReader (optimised solution)

```
1  import java.io.BufferedReader;
2  import java.io.IOException;
3  import java.io.InputStreamReader;
4  import java.util.StringTokenizer;
5
6  public class FastReader {
7   // BufferedReader reads big inputs much faster than Scanner; useful in contests.
8   public static void main(String[] args) throws IOException {
9   BufferedReader br = new BufferedReader(new InputStreamReader(System.in));  (1)
10  int n = Integer.parseInt(br.readLine().trim());  (2)
11  StringTokenizer st = new StringTokenizer(br.readLine());  (3)
12  long sum = 0;
13  for (int i = 0; i < n; i++) {
14  sum += Long.parseLong(st.nextToken());
15  }
16  System.out.println("sum = " + sum);
17  }
18 }
```

Much faster for 10^5+ numbers.

Sample input

```
5
1 2 3 4 5
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	Wrap the byte stream System.in in a character reader, then a buffer. Hard to remember, hence a template.
2	line 10	readLine() returns the whole line as a String; convert it yourself.
3	line 11	Splits the line into whitespace-separated tokens.

CommandLineArgs.java — Command-line arguments (demonstration)

```
1  public class CommandLineArgs {
2   // run: java CommandLineArgs 67 98 91
3   public static void main(String[] args) {
4   if (args.length == 0) {                                                    (1)
5   System.out.println("no arguments given; try: java CommandLineArgs 67 98 91");
6   return;
7   }
8   double sum = 0;
9   for (String s : args) {
10  sum += Double.parseDouble(s); // arguments are ALWAYS Strings              (2)
11  }
12  System.out.println("average = " + sum / args.length);
13  if (args.length >= 2) {
14  System.out.println("args[0] + args[1] as text : " + (args[0] + args[1])); // joining   (3)
        strings, NOT adding
15  System.out.println("as numbers they add up to : " + (Integer.parseInt(args[0]) +
        Integer.parseInt(args[1])));
16  }
17  }
18 }
```

Arguments arrive as Strings; "1" + "8" joins to "18"; parseInt converts.

Line-by-line walkthrough

#	Where	What happens
1	line 4	Always check how many arguments you received.
2	line 10	Converts the text "67" to the number 67.0.
3	line 14	The lecture's trap: two Strings are JOINED, not added.

Dry run: watch it execute

nextInt() leaves the line break behind, so nextLine() returns an empty string.

Algorithm: What Scanner really consumes. **Input:** text = `12 7|Alice Smith|`, calls = `nextInt,nextInt,nextLine,nextLine`.

Pseudocode

```
nextInt()/next()/nextDouble(): skip whitespace, then read one token
nextLine(): read up to the end of the current line and consume the line break
```

Step 1 of 5. The characters of the input; the cursor starts before the first one.

cursor = 0

input stream	1	2	␣	7	↵	A	l	i	c	e	␣	S	m	i	t	h	↵
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	▮																

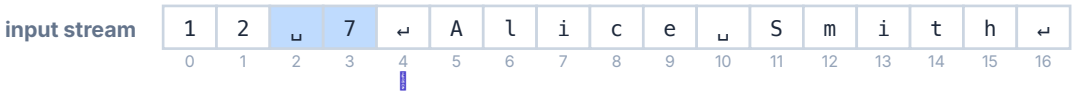
Step 2 of 5. nextInt() skips whitespace and reads the token "12"; the cursor stops right after it, BEFORE the line break.

call = nextInt() · returned = "12" · cursor = 2

input stream	1	2	␣	7	↵	A	l	i	c	e	␣	S	m	i	t	h	↵
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
			▮														

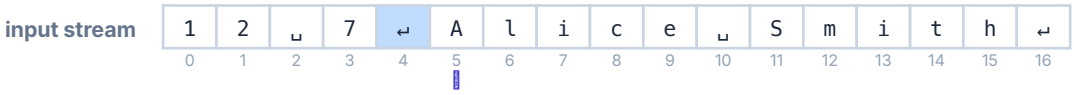
Step 3 of 5. nextInt() skips whitespace and reads the token "7"; the cursor stops right after it, BEFORE the line break.

call = nextInt() · returned = "7" · cursor = 4



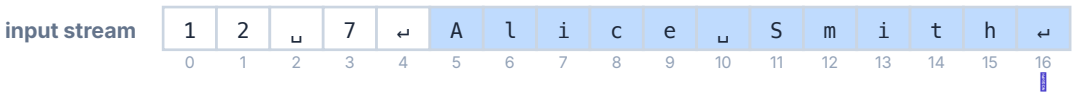
Step 4 of 5. nextLine() returns "" (empty: it only finished the previous line) and moves past the line break.

call = nextLine() · returned = "" · cursor = 5



Step 5 of 5. nextLine() returns "Alice Smith" and moves past the line break.

call = nextLine() · returned = "Alice Smith" · cursor = 17



> **RESULT**
> nextInt() → "12" nextInt() → "7" nextLine() → "" nextLine() → "Alice Smith"

Complexity

Scanner parses roughly 10^6 tokens per second; BufferedReader+StringTokenizer is about 5–10× faster. For n up to 10^5 either is fine; for 10^6+ numbers prefer the buffered reader.

Common mistakes

nextInt() then nextLine()

Problem

```
int age = sc.nextInt();
String name = sc.nextLine(); // returns ""
```

Why: The newline after the number is still in the buffer.

Fix: Call sc.nextLine() once between them.

Adding the Strings from args

Problem

```
System.out.println(args[0] + args[1]); // 1 8 → 18
```

Why: Both are Strings; + concatenates.

Fix: Integer.parseInt(args[0]) + Integer.parseInt(args[1])

Reading more tokens than exist

Problem

```
int x = sc.nextInt(); // input exhausted
```

Why: NoSuchElementException.

Fix: Check hasNextInt() or the stated count.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Average — *Beginner*

Read n and then n decimals. Print their average with two decimals.

Example input	Expected output
3 ⏎ 1.5 2.5 3.5	2.50

Hint 1: printf("%.2f%n", sum / n)

Exercise 2 · Sum until the end — *Intermediate*

The input has any number of integers over several lines. Print their sum.

Example input	Expected output
4 5 6 ⏎ 7	22

Hint 1: while (sc.hasNextLong())

Exercise 3 · Age then name — *Intermediate*

The first line has an age, the second a full name (with spaces). Print `<name> is <age>`.

Example input	Expected output
20 ⏎ Asha Rao	Asha Rao is 20

Hint 1: Consume the newline after nextInt().

Quick check

Answers are in the answer key at the end of this part.

Q1. After `sc.nextInt()` the next `sc.nextLine()` returns...

- **A.** the next full line
- **B.** an empty string (the rest of the number's line)
- **C.** an exception
- **D.** null

Q2. java A 1 8 with System.out.println(args[0] + args[1]) prints...

```
public static void main(String[] args) {
 System.out.println(args[0] + args[1]);
}
```

Your answer: _____

Q3. Which class is best for reading 10^6 numbers fast?

- **A.** Console
- **B.** Scanner
- **C.** BufferedReader
- **D.** System.err

Q4. Complete the end-of-input loop.

```
while (sc._____()) {
 sum += sc.nextInt();
}
```

Interview / exam questions

- Scanner vs BufferedReader?
- Why does nextLine() sometimes return an empty string?
- What type are command-line arguments?
- How do you read input until EOF?

Watch & read

Video: #83 User Input using BufferedReader and Scanner in Java — Telusko · 11:49. *Reading input with Scanner*

Video: Fast Input/Output template for Competitive Programming in JAVA (Reduce TLE) — Tech Tips · 5:15. *BufferedReader for fast input*

REVISION CARD (30–60 SECONDS)

- Scanner: nextInt/nextDouble/next/nextLine; hasNextXxx for EOF.
- After nextInt call nextLine once before reading a line.
- BufferedReader.readLine + parseInt for speed.
- args are Strings; check args.length.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide example `avg = (args[0]+args[1]+…)/5` does not compile (String / int); the slide then fixes it with Float.valueOf — as the lesson shows, convert BEFORE adding. • On the Console slide the code `System.console().readLine()` returns null in IDEs and when input is piped; use Scanner for programs that must work everywhere.

1.8 Decision Making: if, else, switch

Java Syntax & Fundamentals · Beginner · Essential Java syntax · about 35 min · ★ on the minimum-viable path

Choose between actions: if / else if / else, nested conditions, switch with fall-through, and the arrow form.

YOU WILL BE ABLE TO

- Write if, if-else, else-if ladders and nested conditions.
- Use `switch` with `break`, `default` and fall-through, and the arrow form (Java 14+).
- Predict compile errors from the lecture quiz (int condition, missing braces, `break` outside a loop).
- Choose between if-else and switch.

Before this lesson: Relational, Logical and Ternary Operators; Precedence · Input: Scanner, BufferedReader and Command-Line Arguments

The idea

A decision statement runs one block or another depending on a **boolean condition**. In Java the condition must really be a `boolean` — unlike C, an `int` is not accepted.

form	use
if (c) { ... }	run the block only when c is true
if (c) { A } else { B }	two-way choice
if (c1) {...} else if (c2) {...} else {...}	multi-way: the first true condition wins
switch (v) { case x: ... break; default: ... }	multi-way by exact value (int, char, String, enum)

WHY THIS WORKS

An else-if ladder is evaluated **top to bottom and stops at the first true condition**, so the order matters: test the most specific/highest range first (`marks >= 90` before `marks >= 75`). A `switch` jumps straight to the matching `case` — and keeps running into the following cases until it meets `break` (**fall-through**), which is occasionally useful (`case 6: case 7:`) and a frequent bug.

Braces are optional for a single statement, but always use them: the lecture's "Predict the output" quiz shows how a second statement without braces breaks an `else` (`error: 'else' without 'if'`).

Syntax

```
if (marks >= 90) {
 grade = 'A';
} else if (marks >= 75) {
 grade = 'B';
} else {
 grade = 'C';
}

switch (day) {
 case 6:
 case 7: System.out.println("weekend"); break;
 default: System.out.println("weekday");
}

String name = switch (level) { case 1 -> "low"; case 2 -> "medium"; default -> "high"; }; //
    Java 14+
```

A tiny example

```
if (n % 2 == 0) System.out.println("even");
else System.out.println("odd");
```

Guided example

DecisionDemo.java — Decisions (demonstration)

```
import java.util.Scanner;

public class DecisionDemo {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int marks = sc.nextInt();

 // if / else if / else: the FIRST true condition wins, the rest are skipped
 char grade;
 if (marks >= 90) {                                                    (1)
 grade = 'A';
 } else if (marks >= 75) {
 grade = 'B';
 } else if (marks >= 50) {
 grade = 'C';
 } else {
 grade = 'F';
 }
 System.out.println("grade " + grade);

 // nested if
 if (marks >= 50) {
 if (marks % 2 == 0) {                                                 (2)
 System.out.println("pass, even marks");
 } else {
 System.out.println("pass, odd marks");
 }
 }

 // switch: choose by exact value; break stops the fall-through
 int day = sc.nextInt();
 switch (day) {
 case 6:                                                               (3)
 case 7:
 System.out.println("weekend");
 break;
 case 1:
 System.out.println("Monday");
 break;
 default:                                                              (4)
 System.out.println("another weekday");
 }
 }
}
```

Input: marks, then a day number.

Sample input

```
82
7
```

Line-by-line walkthrough

#	Where	What happens
1	line 10	Highest range first — the ladder stops at the first true condition.
2	line 23	Nested if: only reached when marks ≥ 50.
3	line 33	Two case labels with no code between them: 6 and 7 share one block (intentional fall-through).
4	line 40	Runs when no case matches. It may appear anywhere but is conventionally last.

SwitchFallThrough.java — Fall-through and arrow switch (demonstration)

```
1  public class SwitchFallThrough {
2   public static void main(String[] args) {
3   int level = 2;
4   System.out.println("with break missing:");
5   switch (level) {
6   case 1: System.out.println(" case 1");
7   case 2: System.out.println(" case 2"); // matches here...                1
8   case 3: System.out.println(" case 3"); // ...and FALLS THROUGH
9   default: System.out.println(" default");
10  }
11  System.out.println("modern arrow switch (Java 14+), no fall-through:");
12  String name = switch (level) {                                           2
13  case 1 -> "low";
14  case 2 -> "medium";
15  default -> "high";
16  };
17  System.out.println(" " + name);
18  }
19 }
```

Run it: with level = 2 three cases print because there is no break.

Line-by-line walkthrough

#	Where	What happens
1	line 7	The match — but execution continues into case 3 and default.
2	line 12	Arrow-form switch expression: each case yields a value, no fall-through, no break.

NOTE

The lecture quiz answers: `if (x)` with an int → compile error (incompatible types); two statements under `if` without braces followed by `else` → compile error; `if (true) ;` → valid (empty statement), prints nothing; `switch (x + 1 + 1)` with `case 10 + 1 + 1:` → case labels must be constants, and 10+1+1 is a constant expression → prints BYE; `break` inside a plain `if` → error: break outside switch or loop.

Dry run: watch it execute

Conditions are tested top to bottom; the first true one wins.

Algorithm: if – else-if ladder. **Input:** score = `72`.

Pseudocode

```
if score >= 90: grade = "A"
else if score >= 80: grade = "B"
else if score >= 70: grade = "C"
else if score >= 60: grade = "D"
else: grade = "F"
```

Step 1 of 3. Test score >= 90: 72 >= 90 is false. Move to the next condition.

score = 72 · score >= 90 = false

Step 2 of 3. Test score >= 80: 72 >= 80 is false. Move to the next condition.

score = 72 · score >= 80 = false

Step 3 of 3. Test score >= 70: 72 >= 70 is true. Run this branch: grade = "C". All remaining branches are skipped.

score = 72 · score >= 70 = true

> **RESULT**
> grade = C

Complexity

A decision is O(1) per evaluation. An else-if ladder with k branches costs up to k comparisons; a switch on dense ints compiles to a jump table (O(1)).

Common mistakes

Dangling else without braces

Problem

```
if (x > 0)
 System.out.println("pos");
 System.out.println("done");
else
 System.out.println("neg");
```

Why: Only the first println belongs to the if; "else without if" is a compile error. Indentation is ignored by the compiler.

Fix: Always use braces.

Forgetting break

Problem

```
case 1: doA();
case 2: doB();
```

Why: Selecting case 1 runs doA() AND doB().

Fix: case 1: doA(); break;

Semicolon after the condition

Problem

```
if (x > 5);
{
 System.out.println("big");
}
```

Why: The `;` is an empty statement: the block always runs.

Fix: if (x > 5) { ... }

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Grade — *Beginner*

Read marks and print A (≥ 90), B (≥ 75), C (≥ 50) or F.

Example input	Expected output
82	B

Hint 1: Highest range first.

Exercise 2 · Calculator — *Intermediate*

Read `a op b` with op one of + - * /. Print the result of integer arithmetic; for / by zero print `error`; for any other operator print `unknown operator`.

Example input	Expected output
12 / 4	3

Hint 1: switch on the char.

Exercise 3 · Triangle type — *Intermediate*

Read three side lengths. Print `not a triangle`, `equilateral`, `isosceles` or `scalene`.

Example input	Expected output
5 5 8	isosceles

Hint 1: The triangle inequality: every side must be shorter than the sum of the other two.

Quick check

Answers are in the answer key at the end of this part.

Q1. What happens with `int x = 10; if (x) { … }`?

- **A.** runs the block
- **B.** compile-time error
- **C.** runtime error
- **D.** skips the block

Q2. What is printed?

```java
int i = 9;
switch (i) {
 case 0: System.out.println("zero"); break;
 case 1: System.out.println("one"); break;
 default: System.out.println("many");
}
```

Your answer: _____

Q3. What is printed?

```java
int x = 2;
switch (x) {
 case 1: System.out.print("a");
 case 2: System.out.print("b");
 case 3: System.out.print("c");
 default: System.out.print("d");
}
```

Your answer: _____

Q4. What does `if (true) ;` do?

- **A.** compile error
- **B.** nothing — ; is an empty statement
- **C.** infinite loop
- **D.** prints true

Q5. Which line prevents this from compiling?

```java
1  public class A {
2   public static void main(String[] args) {
3   if (true)
4   break;
5   }
6  }
```

Which line contains the bug? _____

Interview / exam questions

- What is fall-through in a switch?
- Which types can a switch use?
- if-else vs switch — when?
- What is the dangling-else problem?

Watch & read

Video: Conditional Statements | If-else, Switch Break | Complete Java Placement Course | Lecture 3 — Apna College · 25:08. *if/else chains and switch statements*

Video: Java Switch Statement Explained | break, default, fall-through | Interview Essential! — ConstantPath · 16:08. *switch, break and fall-through*

REVISION CARD (30–60 SECONDS)

- Condition must be boolean; always use braces.
- else-if ladder: first true wins — order matters.
- switch: break stops fall-through; default for the rest; types int/char/String/enum.
- Arrow switch (Java 14+) has no fall-through.

1.9 Loops: for, while, do-while, break and continue

Java Syntax & Fundamentals · Beginner · Essential Java syntax · about 40 min · ★ on the minimum-viable path

Repeat work: counting loops, condition loops, nested loops, leaving early, skipping, and for-each.

YOU WILL BE ABLE TO

- Choose between `for`, `while` and `do-while` and write each correctly.
- Control a loop with `break` and `continue`; understand nested loops.
- Trace a loop with a table of variable values.
- Count how many times a loop body runs (a first step to Big-O).

Before this lesson: Decision Making: if, else, switch

The idea

loop	form	checks the condition	best when
for	for (init; cond; update) {...}	before each pass	you know the count
while	while (cond) {...}	before each pass	repeat until something changes
do-while	do {...} while (cond);	after each pass	body must run at least once
for-each	for (T x : collection) {...}	—	visit every element

```
for (int i = 1; i <= 3; i++) { body }
 1. init: i = 1
 2. test: i <= 3 ? yes → run body
 3. update: i++ → back to step 2
 ...stops when i becomes 4
```

WHY THIS WORKS

Every loop needs three things: a **starting state**, a **condition** that eventually becomes false, and an **update** that moves the state toward that. If the update never makes the condition false you have an infinite loop. A reliable way to check a loop is a **trace table** — write the variables down after every iteration.

`break` leaves the innermost loop immediately; **`continue`** skips the rest of this pass and goes to the next iteration. In nested loops the inner loop runs completely for every pass of the outer loop, so two loops of n passes run n × n times.

Syntax

The loop statements

```
for (int i = 0; i < n; i++) { /* body */ } // counting: init; condition; update
while (condition) { /* body */ } // test BEFORE each pass (may run 0 times)
do { /* body */ } while (condition); // test AFTER each pass (runs at least once)
for (int x : array) { /* body */ } // enhanced for: every element, no index
break; // leave the innermost loop
continue; // skip to the next iteration
outer: // a label lets break/continue target an outer loop
for (int i = 0; i < n; i++) { for (int j = 0; j < n; j++) { if (found) break outer; } }
```

you know...	use
how many times	for
only a stopping condition	while
must run once before checking	do-while
just want every element	enhanced for

A tiny example

```
int sum = 0;
for (int i = 1; i <= 100; i++) {
 sum += i;
}
// sum = 5050
```

Guided example

LoopsDemo.java — Loops in one program (demonstration)

```
public class LoopsDemo {
 public static void main(String[] args) {
 // for: when you know how many times
 int sum = 0;
 for (int i = 1; i <= 5; i++) {                                    (1)
 sum += i;
 }
 System.out.println("1+2+3+4+5 = " + sum);

 // while: repeat while a condition holds (condition checked BEFORE each pass)
 int n = 1000;
 int digits = 0;
 while (n > 0) {                                                   (2)
 n /= 10;
 digits++;
 }
 System.out.println("1000 has " + digits + " digits");

 // do-while: body runs at least once (condition checked AFTER)
 int k = 10;
 do {
 System.out.println("do-while ran once with k = " + k);
 } while (k < 5);                                                  (3)

 // nested loops: the inner loop restarts for every outer iteration
 for (int row = 1; row <= 3; row++) {
 StringBuilder line = new StringBuilder();
 for (int col = 1; col <= row; col++) {                            (4)
 line.append('*');
 }
 System.out.println(line);
 }

 // break leaves the loop, continue skips to the next iteration
 for (int i = 1; i <= 10; i++) {
 if (i % 2 == 0) continue; // skip evens                           (5)
 if (i > 7) break; // stop completely                              (6)
 System.out.print(i + " ");
 }
 System.out.println();

 // for-each over an array
 int[] marks = {70, 85, 90};
 int total = 0;
 for (int m : marks) {                                             (7)
 total += m;
 }
 System.out.println("total = " + total);
 }
}
```

Trace each section by hand before running it.

Line-by-line walkthrough

#	Where	What happens
1	line 5	Counting loop: i takes 1, 2, 3, 4, 5.
2	line 13	Digit count: dividing by 10 until nothing is left (4 passes for 1000).
3	line 23	The test is at the bottom, so the body runs once even though k < 5 is false.
4	line 28	The inner loop's limit depends on the outer variable → a triangle.
5	line 36	Skips even numbers.
6	line 37	Stops everything when i reaches 8.
7	line 45	for-each: m takes each element in turn.

Dry run: watch it execute

Follow i and sum through every iteration, including the skipped even numbers.

Algorithm: Loop with continue. **Input:** n = `8`.

Pseudocode

```
sum = 0
for i = 1 .. n:
 if i % 2 == 0: continue
 sum = sum + i
print sum
```

The full trace has 18 steps; 14 evenly spaced ones are shown. Run the rest in the website's Dry-run lab.

Step 1 of 18. sum starts at 0.

n = 8 · sum = 0

Step 2 of 18. i = 1: loop condition i <= 8 is true.

i = 1 · sum = 0

Step 4 of 18. i = 2: loop condition i <= 8 is true.

i = 2 · sum = 1

Step 5 of 18. 2 % 2 == 0 → continue: skip the addition and jump to the update i++.

i = 2 · sum = 1 · i % 2 = 0

Step 6 of 18. i = 3: loop condition i <= 8 is true.

i = 3 · sum = 1

Step 8 of 18. i = 4: loop condition i <= 8 is true.

i = 4 · sum = 4

Step 9 of 18. 4 % 2 == 0 → continue: skip the addition and jump to the update i++.

i = 4 · sum = 4 · i % 2 = 0

Step 10 of 18. i = 5: loop condition i <= 8 is true.

i = 5 · sum = 4

Step 11 of 18. 5 is odd → sum = sum + 5 = 9.

i = 5 · sum = 9 · i % 2 = 1

Step 13 of 18. 6 % 2 == 0 → continue: skip the addition and jump to the update i++.

i = 6 · sum = 9 · i % 2 = 0

Step 14 of 18. i = 7: loop condition i <= 8 is true.

i = 7 · sum = 9

Step 15 of 18. 7 is odd → sum = sum + 7 = 16.

i = 7 · sum = 16 · i % 2 = 1

Step 17 of 18. 8 % 2 == 0 → continue: skip the addition and jump to the update i++.

i = 8 · sum = 16 · i % 2 = 0

Step 18 of 18. i = 9: the condition i <= 8 is false, the loop ends.

i = 9 · sum = 16

> **RESULT**
> sum = 16

Complexity

A loop that runs n times is O(n); two nested loops of n are O(n²); a loop that doubles its counter (`i *= 2`) runs $\log_2 n$ times. Counting passes this way is exactly how time complexity is analysed — see the complexity lessons and the lab.

Common mistakes

Off-by-one

Problem

```
for (int i = 0; i <= arr.length; i++) { arr[i] ... }
```

Why: The last index is length − 1; i == length throws ArrayIndexOutOfBoundsException.

Fix: i < arr.length

Infinite loop

Problem

```
int i = 0;
while (i < 10) {
 System.out.println(i);
}
```

Why: i never changes.

Fix: Add i++ inside the body.

Semicolon after for

Problem

```
for (int i = 0; i < 3; i++);
{
 System.out.println("hi");
}
```

Why: The loop body is the empty statement; the block runs once afterwards.

Fix: Remove the ;

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Factorial — *Beginner*

Read n (0 ≤ n ≤ 20) and print n! (use long).

Example input	Expected output
10	3628800

Hint 1: Start with 1 and multiply by 2, 3, ..., n.

Exercise 2 · Pyramid — *Intermediate*

Read n and print a pyramid of n rows of `*`, row i has 2i−1 stars, centred with leading spaces.

Example input	Expected output
3	∗ ⏎ ∗ ⏎ ∗∗∗

Hint 1: Row r: n − r spaces then 2r − 1 stars.

Exercise 3 · Collatz steps — *Intermediate*

Start from n. If n is even divide it by 2, otherwise compute 3n + 1. Count the steps until n becomes 1.

Example input	Expected output
6	8

Hint 1: Use long; a while loop with a counter.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
int s = 0;
for (int i = 1; i <= 4; i++) {
 if (i == 3) continue;
 s += i;
}
System.out.println(s);
```

Your answer: _____

Q2. How many times does the body run?

```
int count = 0;
for (int i = 0; i < 3; i++)
 for (int j = 0; j < 4; j++)
 count++;
System.out.println(count);
```

Your answer: _____

Q3. A loop whose body must run at least once is best written as...

- **A.** for
- **B.** while
- **C.** do-while
- **D.** for-each

Q4. What is printed?

```
int i = 1;
while (i < 100) {
 i *= 3;
}
System.out.println(i);
```

Your answer: _____

Q5. Which line makes this loop infinite?

```
1  int n = 10;
2  while (n > 0) {
3   System.out.println(n);
4   n++;
5  }
```

Which line contains the bug? _____

Interview / exam questions

- for vs while vs do-while?
- What does continue do?
- How many iterations does `for (i = 1; i < n; i *= 2)` run?
- How do you exit nested loops?

Watch & read

Video: Java For Beginners: While, Do While & For Loops (7/10) — Codecourse · 10:59. *for, while and do-while loops*

Video: 15. Break, Continue and Nested Loops in Java — Abhimanyu Kumar Vatsa · 6:59. *break, continue and nested loops*

REVISION CARD (30–60 SECONDS)

- for = known count; while = until condition; do-while = at least once.
- break exits, continue skips to next pass.
- Nested loops multiply: n × n.
- Check start, condition, update; trace with a table.

1.10 Methods: Parameters, Return Values, Scope and Overloading

Java Syntax & Fundamentals · Beginner · Essential Java syntax · about 40 min · ★ on the minimum-viable path

Package code into named, reusable pieces; pass values in, get a value out, and keep variables local.

YOU WILL BE ABLE TO

- Write methods with parameters and return types, and call them.
- Explain pass-by-value for primitive arguments.
- Describe the scope of local variables and parameters.
- Overload a method and decide when a helper method improves a program.

Before this lesson: Loops: for, while, do-while, break and continue

The idea

A **method** is a named block of code that can be called from elsewhere. It has a **signature** (name and parameter types), a **return type** (or `void`), and a **body**.

```
static int square(int x) { // returns an int; takes one int parameter
 return x * x;
}
int y = square(7); // call: 7 is the ARGUMENT, x is the PARAMETER
```

WHY THIS WORKS

Methods let you write a piece of logic once, give it a name that says *what* it does, test it in isolation and reuse it. Algorithms in this course are almost always written as methods (`gcd(a, b)`, `isPrime(n)`, `josephus(n, k)`) with a small `main` that reads input and prints the result.

- **Pass by value:** the method receives a **copy** of each primitive argument. Changing the parameter inside the method does not change the caller's variable.
- **Scope:** a variable exists only inside the block `{ }` in which it is declared. Parameters and locals vanish when the method returns.
- **Overloading:** several methods may share a name if their parameter lists differ (types or count). The compiler picks the best match from the argument types.
- **static:** for now, write helper methods as `static` so `main` (which is static) can call them directly.

Syntax

part	example
modifiers	public static
return type	int / void / boolean / String
name	isPrime (lowerCamelCase, a verb)
parameters	(int n, int k)
body	{ ... return value; }

A tiny example

```
static boolean isEven(int n) {
 return n % 2 == 0;
}
```

Guided example

MethodsDemo.java — Methods demo (demonstration)

```
1  public class MethodsDemo {
2   // A method: name, parameters (inputs), return type (output), body.
3   static int square(int x) {                                          (1)
4   return x * x;
5   }
6
7   // Method OVERLOADING: same name, different parameter lists.
8   static int max(int a, int b) {
9   return a > b ? a : b;
10  }
11
12  static double max(double a, double b) {                             (2)
13  return a > b ? a : b;
14  }
15
16  static int max(int a, int b, int c) {                               (3)
17  return max(max(a, b), c);
18  }
19
20  // A void method returns nothing; it performs an action.
21  static void greet(String name) {
22  System.out.println("Hello, " + name);
23  }
24
25  // Java passes arguments BY VALUE: the method gets a copy of the int.
26  static void tryToChange(int n) {                                    (4)
27  n = 999;
28  }
29
30  // Scope: a variable exists only inside the block where it is declared.
31  static int sumTo(int n) {
32  int total = 0; // local to this method                              (5)
33  for (int i = 1; i <= n; i++) { // i exists only inside the loop
34  total += i;
35  }
36  return total;
37  }
38
39  public static void main(String[] args) {
40  System.out.println(square(7));
41  System.out.println(max(3, 9));
42  System.out.println(max(2.5, 1.5));
43  System.out.println(max(4, 8, 6));
44  greet("Asha");
45  int value = 5;
46  tryToChange(value);
47  System.out.println("value is still " + value);
48  System.out.println(sumTo(100));
49  }
50 }
```

Each helper method illustrates one rule.

Line-by-line walkthrough

#	Where	What happens
1	line 3	A pure function: result depends only on the parameter.
2	line 12	Overload: same name, different parameter types — the call max(2.5, 1.5) picks this one.
3	line 16	Overload with a different number of parameters, reusing the two-argument version.
4	line 26	n is a COPY. Assigning to it does not affect the caller (pass-by-value).
5	line 32	Local variable: exists only during this call; every call starts fresh.

Dry run: watch it execute

Arguments are copied into parameters; each return value flows to the caller.

Algorithm: Method calls and parameters. **Input:** a = 4, b = 9, c = 6.

Pseudocode

```
static int max(int x, int y) { return x > y ? x : y; }
static int max3(int a, int b, int c) { return max(max(a, b), c); }
int m = max3(a, b, c);
```

Step 1 of 6. main calls max3(4, 9, 6): the argument VALUES are copied into the parameters a, b, c.

a = 4 · b = 9 · c = 6

call stack (bottom → top) `max3(a=4, b=9, c=6)`

0

Step 2 of 6. max3 first evaluates the inner call max(a, b) = max(4, 9).

x = 4 · y = 9

call stack (bottom → top) `max3(a=max(x=4,y=9), c=6)`

0 1

Step 3 of 6. x > y is false, so max returns 9. The frame disappears.

x = 4 · y = 9 · return = 9

call stack (bottom → top) `max3(a=max(x=4,y=9), c=6)`

0 1

Step 4 of 6. Now the outer call max(9, 6).

x = 9 · y = 6

call stack (bottom → top) `max3(a=max(x=9,y=6), c=6)`

0 1

Step 5 of 6. x > y is true, so max returns 9.

x = 9 · y = 6 · return = 9

call stack (bottom → top) `max3(a=max(x=9,y=6), c=6)`

0 1

Step 6 of 6. max3 returns 9 to main.

a = 4 · b = 9 · c = 6 · return = 9

call stack (bottom → top) `max3(a=4, b=9, c=6)`

0

> **RESULT**
> max3(4, 9, 6) = 9

Complexity

A method call costs O(1) plus the work done inside it. Each active call occupies a stack frame — relevant for recursion (next module).

Common mistakes

Forgetting to return

Problem

```
static int square(int x) {
 int r = x * x;
}
```

Why: Compile error: missing return statement.

Fix: return r;

Ignoring the return value

Problem

```
square(5); // result discarded
System.out.println(x);
```

Why: The method does not change anything outside; its value must be used.

Fix: int s = square(5);

Expecting a method to modify a primitive argument

Problem

```
static void inc(int n) { n++; }
int a = 1; inc(a); // a is still 1
```

Why: Java passes a copy.

Fix: Return the new value: a = inc(a);

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · isPrime method — *Beginner*

Write `static boolean isPrime(int n)` and use it to print `prime` or `not prime` for the input.

Example input	Expected output
97	prime

Hint 1: Test divisors from 2 while d·d ≤ n.

Exercise 2 · Overloaded power — *Intermediate*

Write two methods named `power`: one for (long base, int exp) and one for (double base, int exp). Read base and exp and print power(base, exp) as long, then print power(2.5, 2).

Example input	Expected output
3 4	81 ⏎ 6.25

Hint 1: Multiply in a loop.

Exercise 3 · Digits helpers — *Intermediate*

Using two helper methods print `<number of digits> <sum of digits>` for a non-negative integer (0 has 1 digit).

Example input	Expected output
90210	5 12

Hint 1: A do-while loop counts digits correctly even for 0.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
static void change(int n) { n = 99; }
public static void main(String[] a) {
 int v = 5;
 change(v);
 System.out.println(v);
}
```

Your answer: _____

Q2. Which pair is a valid overload?

- **A.** int f(int a) and int f(int b)
- **B.** int f(int a) and double f(int a)
- **C.** int f(int a) and int f(double a)
- **D.** void f() and void f()

Q3. Which line is the compile error?

```
1  static int twice(int x) {
2   if (x > 0) {
3   return x * 2;
4   }
5  }
```

Which line contains the bug? _____

Q4. A variable declared inside a for loop body is visible...

- **A.** in the whole method
- **B.** only inside that loop body
- **C.** in the whole class
- **D.** in other methods

Interview / exam questions

- What is method overloading? Overriding (preview)?
- Is Java pass-by-value or pass-by-reference?
- What is a stack frame?
- Why break a program into methods?

Watch & read

Video: Java Methods for Beginners: Arguments, Parameters, Returning Values, — Professor Hank Stalica · 19:57. *Declaring and calling methods*

Video: #25 Method Overloading in Java — Telusko · 6:03. *Overloading and variable scope*

REVISION CARD (30–60 SECONDS)

- returnType name(params) { ... return value; }
- Arguments are copied (pass by value).
- Scope = enclosing braces.
- Overload = same name, different parameters.

1.11 Tracing and Debugging Programs

Java Syntax & Fundamentals · Beginner · Essential Java syntax · about 30 min · ★ on the minimum-viable path

Execute code by hand with a trace table, use print debugging, and learn the IDE debugger workflow.

YOU WILL BE ABLE TO

- Build a trace table for loops and method calls.
- Use temporary print statements to inspect state.
- Use breakpoints, step over, step into and the variables pane of an IDE.
- Apply a systematic debugging routine.

Before this lesson: Methods: Parameters, Return Values, Scope and Overloading · Compile-time, Runtime and Logic Errors

The idea

Tracing means executing the program by hand, writing down the value of every variable after each step. It is the single most useful skill for understanding algorithms — every "dry run" in this course is a trace table.

*Trace of: int x = 1; for (int i = 0; i < 4; i++) x = x * 2 + i; → 27*

i	x before	x after (x = x·2 + i)
0	1	1·2 + 0 = 2
1	2	2·2 + 1 = 5
2	5	5·2 + 2 = 12
3	12	12·2 + 3 = 27

WHY THIS WORKS

When a program misbehaves, guessing wastes time. Debugging is the scientific method: **observe** the wrong output, **hypothesise** which line produces it, **test** the hypothesis by inspecting values, **fix**, and **re-run** the failing case. Printing the variables inside the loop (or stepping through with a debugger) replaces speculation with facts.

IDE debugger quick guide

- Click in the margin next to a line to set a **breakpoint**; run in Debug mode.
- **Step over** runs the current line; **step into** enters a method call; **step out** finishes the method.
- The **variables** pane shows current values; add **watch** expressions for what you care about.
- In VS Code: Run ▸ Start Debugging (F5); IntelliJ: the bug icon (Shift-F9).

Syntax

Two quick tools

```
System.out.println("i=" + i + " sum=" + sum); // print-debugging: label every value
assert count >= 0 : "count went negative"; // checked only when run with: java -ea Main
```

debugger command	what it does
breakpoint	stop before a chosen line executes
step over	run the current line, do not enter called methods
step into	enter the method called on the current line
step out	finish the current method and stop in its caller
resume	run until the next breakpoint
watch / variables pane	show current values of variables and expressions

A **trace table** has one column per variable and one row per step; fill it in by hand while reading the code, then compare with what you expected.

A tiny example

```
for (int i = 1; i <= 3; i++) {
 sum += i * i;
 System.out.println("i=" + i + " sum=" + sum); // temporary debugging print
}
```

Guided example

TraceTable.java — Printing a trace table (demonstration)

```
1  public class TraceTable {
2   // A trace table records the variables after every pass of a loop. Printing it is the
3   // simplest debugger there is.
4   public static void main(String[] args) {
5   int n = 5;
6   int sum = 0;
7   System.out.println("i sum");                                    (1)
8   for (int i = 1; i <= n; i++) {
9   sum += i * i;                                                  (2)
10  System.out.println(i + " " + sum);                             (3)
11  }
12  System.out.println("sum of squares = " + sum);
13  }
14 }
```

Sum of squares with one line printed per iteration.

Line-by-line walkthrough

#	Where	What happens
1	line 7	The header of the trace table.
2	line 9	The state change we want to observe.
3	line 10	One row per iteration — the same table you would write by hand.

FindTheBug.java — Find the bug (demonstration)

```
1  public class FindTheBug {
2   // BUG: the loop is meant to count the even numbers in the array, but it prints 2
        instead of 3.
3   static int countEvensBuggy(int[] a) {
4   int count = 0;
5   for (int i = 1; i < a.length; i++) { // starts at 1: skips a[0]                    1
6   if (a[i] % 2 == 0) {
7   count++;
8   }
9   }
10  return count;
11  }
12
13  static int countEvensFixed(int[] a) {                                             2
14  int count = 0;
15  for (int i = 0; i < a.length; i++) {
16  if (a[i] % 2 == 0) {
17  count++;
18  }
19  }
20  return count;
21  }
22
23  public static void main(String[] args) {
24  int[] a = {2, 7, 4, 9, 6};
25  System.out.println("buggy: " + countEvensBuggy(a));
26  System.out.println("fixed: " + countEvensFixed(a));
27  }
28 }
```

countEvensBuggy starts its loop at index 1 and so ignores the first element.

Line-by-line walkthrough

#	Where	What happens
1	line 5	The bug: the loop skips a[0]. A trace table on {2,7,4,9,6} immediately shows the count missing the first 2.
2	line 13	The fix is a single character: start at 0.

Dry run: watch it execute

A trace table exposes the loop that starts at 1 instead of 0.

Algorithm: A trace table finds the bug. **Input:** arr = `2,7,4,9,6`.

Pseudocode

```
// BUGGY: should count the even numbers
int count = 0;
for (int i = 1; i < a.length; i++) // starts at 1!
 if (a[i] % 2 == 0) count++;
return count;
```

Step 1 of 6. Before the loop count = 0. The loop will start at i = 1, not 0.

count = 0 · expected count = 0

a	2	7	4	9	6
	0	1	2	3	4

Step 2 of 6. i = 1: a[1] = 7 is odd → unchanged.

i = 1 · a[i] = 7 · even = false · count = 0 · a[0] ignored = true

a	2	7	4	9	6
	0	1	2	3	4

Step 3 of 6. i = 2: a[2] = 4 is even → count++.

i = 2 · a[i] = 4 · even = true · count = 1 · a[0] ignored = true

a	2	7	4	9	6
	0	1	2	3	4

Step 4 of 6. i = 3: a[3] = 9 is odd → unchanged.

i = 3 · a[i] = 9 · even = false · count = 1 · a[0] ignored = true

a	2	7	4	9	6
	0	1	2	3	4

Step 5 of 6. i = 4: a[4] = 6 is even → count++.

i = 4 · a[i] = 6 · even = true · count = 2 · a[0] ignored = true

a	2	7	4	9	6
	0	1	2	3	4

Step 6 of 6. The loop never looked at a[0] = 2, which is even. Trace table says count = 2 but the correct answer is 3: the bug is the loop start.

count = 2 · expected = 3

a	2	7	4	9	6
	0	1	2	3	4

> **RESULT**
> buggy count = 2, correct = 3 → off-by-one at the loop start

Complexity

Not applicable. (Debug prints inside a hot loop can dominate running time — remove them afterwards.)

Common mistakes

Debugging by random edits

Problem

```
// change a + to a -, re-run, hope
```

Why: Without a hypothesis you may fix the symptom and add a new bug.

Fix: Trace a small failing input; find the first line where reality differs from your expectation.

Leaving debug prints in submitted code

Problem

```
System.out.println("here 1");
```

Why: Extra output makes tests fail.

Fix: Remove or guard them.

Testing only the happy path

Problem

```
// tested with 5 normal numbers
```

Why: Bugs hide at the edges: empty input, one element, negatives, duplicates, maximum size.

Fix: Write down edge cases first and trace each.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Trace by hand — *Beginner*

What does the program print? (Trace x = 1; for i = 0..3: x = x·2 + i.) Run it to check.

Example input	Expected output
(no input)	27

Hint 1: Write the trace table.

Exercise 2 · Print the trace — *Intermediate*

Read a start value and a number of steps. Repeat value = value·2 + 1 for that many steps and print `step value` after each.

Example input	Expected output
1 3	1 3 ⏎ 2 7 ⏎ 3 15

Hint 1: A loop with one println inside.

Exercise 3 · Fix the maximum — *Intermediate*

A buggy program started `max = 0` and so failed on all-negative inputs. Write the correct version: read n and n integers; print the largest.

Example input	Expected output
3 ⏎ −4 −2 −9	−2

Hint 1: Start from Integer.MIN_VALUE or the first element.

Quick check

Answers are in the answer key at the end of this part.

Q1. Trace this program.

```
int n = 10, count = 0;
while (n > 1) {
 n = n / 2;
 count++;
}
```

- n after pass 1: _____
- n after pass 2: _____
- final count: _____

Q2. In a debugger, "step over" does what?

- **A.** runs the current line without entering called methods
- **B.** enters the called method
- **C.** stops the program
- **D.** restarts

Q3. The best FIRST step when a program gives a wrong answer is...

- **A.** rewrite it
- **B.** trace a small failing input
- **C.** add random prints everywhere
- **D.** change the data type

Q4. What does FindTheBug.countEvensBuggy return for {2, 7, 4, 9, 6}?

```
// loop starts at i = 1
```

Your answer: _____

Interview / exam questions

- How do you debug a program that gives wrong output?
- What is a breakpoint?
- What edge cases do you test?
- What is a trace table?

Watch & read

Video: Debugging Java code in IntelliJ IDEA — IntelliJ IDEA, a JetBrains IDE · 7:26. *Using a debugger: breakpoints and stepping*

Video: how to run the Trace Table | Step-by-Step Dry Run, Variable Tracking, Common Errors & Programming — Computing Knowledge · 23:31. *Hand-tracing a program with a trace table*

REVISION CARD (30–60 SECONDS)

- Trace table: variables after every iteration.
- Debug loop: observe → hypothesise → inspect → fix → retest.
- Breakpoint, step over/into/out, variables pane.
- Test edges: empty, one, negatives, duplicates, max.

Part 1 — Solutions to the exercises

Try the exercises before reading these. Each solution is the exact file that was compiled and tested.

1.1 Variables, Constants and Primitive Types

Exercise 1 · Swap two numbers

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int b = sc.nextInt();
 int temp = a; // temp remembers a before it is overwritten
 a = b;
 b = temp;
 System.out.println(a + " " + b);
 }
}
```

Why it works: temp keeps the first value so it is not lost when a is overwritten.

Verified against 3 test cases: `3 8⏎` → `8 3`; `-1 0⏎` → `0 -1`; `5 5⏎` → `5 5`

Exercise 2 · Rectangle

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 double w = sc.nextDouble();
 double h = sc.nextDouble();
 System.out.println("area=" + w * h);
 System.out.println("perimeter=" + 2 * (w + h));
 }
}
```

Why it works: Plain arithmetic on doubles; string concatenation prints the value.

Verified against 2 test cases: `2.5 4⏎` → `area=10.0⏎perimeter=13.0`; `1 1⏎` → `area=1.0⏎perimeter=4.0`

Exercise 3 · Seconds to h:m:s

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int total = sc.nextInt();
 int hours = total / 3600;
 int minutes = (total % 3600) / 60;
 int seconds = total % 60;
 System.out.println(hours + ":" + minutes + ":" + seconds);
 }
}
```

Why it works: Integer division and remainder peel off the units.

Verified against 3 test cases: `3725⏎` → `1:2:5`; `59⏎` → `0:0:59`; `86399⏎` → `23:59:59`

1.2 Type Conversion, Casting and Overflow

Exercise 1 · Big product

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int b = sc.nextInt();
 long product = (long) a * b; // widen first, then multiply
 System.out.println(product);
 }
}
```

Why it works: (long) a promotes the multiplication to long arithmetic.

Verified against 3 test cases: `100000 100000⏎` → `10000000000`; `2147483647 2147483647⏎` → `4611686014132420609`; `-3 4⏎` → `-12`

Exercise 2 · Shift a letter

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Shift an uppercase letter forward by k positions, wrapping Z -> A.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 char c = sc.next().charAt(0);
 int k = sc.nextInt();
 char shifted = (char) ('A' + (c - 'A' + k) % 26);
 System.out.println(shifted);
 }
}
```

Why it works: Convert to 0..25, add k, wrap with %, convert back.

Verified against 3 test cases: `Y 3⏎` → `B`; `A 0⏎` → `A`; `Z 26⏎` → `Z`

Exercise 3 · Will it overflow?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Print OK if a + b fits in an int, otherwise OVERFLOW.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long a = sc.nextLong();
 long b = sc.nextLong();
 long sum = a + b;
 boolean fits = sum >= Integer.MIN_VALUE && sum <= Integer.MAX_VALUE;
 System.out.println(fits ? "OK" : "OVERFLOW");
 }
}
```

Why it works: A long sum cannot overflow for int-sized inputs; compare it with the int bounds.

Verified against 3 test cases: `2000000000 2000000000⏎` → `OVERFLOW`; `1 2⏎` → `OK`; `-2147483648 -1⏎` → `OVERFLOW`

1.3 Arithmetic, Assignment and Increment Operators

Exercise 1 · Digit sum

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int sum = 0;
 while (n > 0) {
 sum += n % 10;
 n /= 10;
 }
 System.out.println(sum);
 }
}
```

Why it works: Repeatedly take the last digit and drop it.

Verified against 3 test cases: `4821⏎` → `15`; `7⏎` → `7`; `1000⏎` → `1`

Exercise 2 · Reverse the digits

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int reversed = 0;
 while (n > 0) {
 reversed = reversed * 10 + n % 10;
 n /= 10;
 }
 System.out.println(reversed);
 }
}
```

Why it works: Push each last digit onto the new number.

Verified against 3 test cases: `1200⏎` → `21`; `12345⏎` → `54321`; `5⏎` → `5`

Exercise 3 · Days to weeks

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Convert a number of days into "W weeks D days".
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int days = sc.nextInt();
 System.out.println(days / 7 + " weeks " + days % 7 + " days");
 }
}
```

Why it works: Quotient and remainder of division by 7.

Verified against 3 test cases: `30⏎` → `4 weeks 2 days`; `7⏎` → `1 weeks 0 days`; `0⏎` → `0 weeks 0 days`

1.4 Relational, Logical and Ternary Operators; Precedence

Exercise 1 · Leap year

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int y = sc.nextInt();
 boolean leap = (y % 4 == 0 && y % 100 != 0) || y % 400 == 0;
 System.out.println(leap ? "leap" : "common");
 }
}
```

Why it works: Combine the three divisibility rules with && and ||.

Verified against 4 test cases: `2000⏎` → `leap`; `1900⏎` → `common`; `2024⏎` → `leap`

Exercise 2 · Maximum of three

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int b = sc.nextInt();
 int c = sc.nextInt();
 int max = a > b ? (a > c ? a : c) : (b > c ? b : c);
 System.out.println(max);
 }
}
```

Why it works: Compare a with b, then the winner with c.

Verified against 3 test cases: `4 9 7⏎` → `9`; `5 5 5⏎` → `5`; `-1 -2 -3⏎` → `-1`

Exercise 3 · In range?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Print IN if lo <= x <= hi, otherwise OUT.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int lo = sc.nextInt();
 int hi = sc.nextInt();
 System.out.println(x >= lo && x <= hi ? "IN" : "OUT");
 }
}
```

Why it works: x >= lo && x <= hi.

Verified against 3 test cases: `5 1 10⏎` → `IN`; `11 1 10⏎` → `OUT`; `1 1 10⏎` → `IN`

1.5 Bitwise Operators and Shifts

Exercise 1 · Power of two?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 System.out.println(n > 0 && (n & (n - 1)) == 0 ? "YES" : "NO");
 }
}
```

Why it works: A power of two has exactly one 1 bit, so clearing it leaves 0.

Verified against 4 test cases: `64⏎` → `YES`; `12⏎` → `NO`; `1⏎` → `YES`

Exercise 2 · Count the 1 bits

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Count the 1 bits (population count) with the "clear lowest set bit" trick.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int count = 0;
 while (n != 0) {
 n &= n - 1;
 count++;
 }
 System.out.println(count);
 }
}
```

Why it works: Brian Kernighan's method: the loop runs once per set bit.

Verified against 4 test cases: `255⏎` → `8`; `0⏎` → `0`; `1024⏎` → `1`

Exercise 3 · Set, clear or toggle

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Read n, i and an operation (set / clear / toggle). Print the result.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int i = sc.nextInt();
 String op = sc.next();
 int mask = 1 << i;
 int result;
 if (op.equals("set")) result = n | mask;
 else if (op.equals("clear")) result = n & ~mask;
 else result = n ^ mask;
 System.out.println(result);
 }
}
```

Why it works: Build the one-bit mask and apply the matching operator.

Verified against 3 test cases: `5 1 set⏎` → `7`; `5 0 clear⏎` → `4`; `5 2 toggle⏎` → `1`

1.6 Output: print, println, printf and String Concatenation

Exercise 1 · Times table

Ex1.java

```
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 for (int i = 1; i <= 5; i++) {
 System.out.printf("%d x %d = %d%n", n, i, n * i);
 }
 }
}
```

Why it works: A loop with printf.

Verified against 2 test cases: `7⏎` → `7 x 1 = 7⏎7 x 2 = 14⏎7 x 3 = 21⏎7 x 4 = 28⏎7 x 5 = 35`; `1⏎` → `1 x 1 = 1⏎1 x 2 = 2⏎1 x 3 = 3⏎1 x 4 = 4⏎1 x 5 = 5`

Exercise 2 · Invoice total

Ex2.java

```
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 double price = sc.nextDouble();
 int qty = sc.nextInt();
 System.out.printf("Total: %,.2f%n", price * qty);
 }
}
```

Why it works: %,.2f formats with grouping and two decimals.

Verified against 3 test cases: `1999.5 3⏎` → `Total: 5,998.50`; `0.1 3⏎` → `Total: 0.30`; `1000000 1⏎` → `Total: 1,000,000.00`

Exercise 3 · Predict, then run

Ex3.java

```
public class Ex3 {
 // Predict, then verify: what does this print? (the answer is in the quiz)
 public static void main(String[] args) {
 int a = 2, b = 3;
 System.out.println(a + b + "=" + a + b);
 }
}
```

Why it works: 2 + 3 = 5 first; then "=" is joined, then 2 and 3 are joined as text.

Verified against 1 test case: `(no input)` → `5=23`

1.7 Input: Scanner, BufferedReader and Command-Line Arguments

Exercise 1 · Average

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 double sum = 0;
 for (int i = 0; i < n; i++) sum += sc.nextDouble();
 System.out.printf("%.2f%n", sum / n);
 }
}
```

Why it works: Accumulate the sum as a double.

Verified against 3 test cases: `3⏎1.5 2.5 3.5⏎` → `2.50`; `1⏎7⏎` → `7.00`; `2⏎1 2⏎` → `1.50`

Exercise 2 · Sum until the end

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Sum all integers until the end of input (the count is not given).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long sum = 0;
 while (sc.hasNextLong()) sum += sc.nextLong();
 System.out.println(sum);
 }
}
```

Why it works: hasNextLong() ends the loop at end of input.

Verified against 3 test cases: `4 5 6⏎7⏎` → `22`; `10⏎` → `10`; `-5 5⏎` → `0`

Exercise 3 · Age then name

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // A name on the first line, an age on the second: the classic nextInt / nextLine trap.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int age = sc.nextInt();
 sc.nextLine(); // throw away the end of the age line
 String name = sc.nextLine();
 System.out.println(name + " is " + age);
 }
}
```

Why it works: Call nextLine() once to discard the rest of the age line.

Verified against 2 test cases: `20⏎Asha Rao⏎` → `Asha Rao is 20`; `7⏎Bo⏎` → `Bo is 7`

1.8 Decision Making: if, else, switch

Exercise 1 · Grade

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int m = sc.nextInt();
 if (m >= 90) System.out.println("A");
 else if (m >= 75) System.out.println("B");
 else if (m >= 50) System.out.println("C");
 else System.out.println("F");
 }
}
```

Why it works: An else-if ladder ordered from the highest threshold down.

Verified against 4 test cases: `82⏎` → `B`; `90⏎` → `A`; `49⏎` → `F`

Exercise 2 · Calculator

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // A small calculator: "<a> <op> <b>" with op one of + - * /
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 char op = sc.next().charAt(0);
 int b = sc.nextInt();
 switch (op) {
 case '+': System.out.println(a + b); break;
 case '-': System.out.println(a - b); break;
 case '*': System.out.println(a * b); break;
 case '/':
 if (b == 0) System.out.println("error");
 else System.out.println(a / b);
 break;
 default: System.out.println("unknown operator");
 }
 }
}
```

Why it works: A switch on the operator character with a guard for division.

Verified against 4 test cases: `12 / 4⏎` → `3`; `5 / 0⏎` → `error`; `6 * 7⏎` → `42`

Exercise 3 · Triangle type

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int b = sc.nextInt();
 int c = sc.nextInt();
 if (a + b <= c || a + c <= b || b + c <= a) {
 System.out.println("not a triangle");
 } else if (a == b && b == c) {
 System.out.println("equilateral");
 } else if (a == b || b == c || a == c) {
 System.out.println("isosceles");
 } else {
 System.out.println("scalene");
 }
 }
}
```

Why it works: Check validity first, then count equal sides.

Verified against 4 test cases: `5 5 8⏎` → `isosceles`; `3 3 3⏎` → `equilateral`; `3 4 5⏎` → `scalene`

1.9 Loops: for, while, do-while, break and continue

Exercise 1 · Factorial

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long f = 1;
 for (int i = 2; i <= n; i++) f *= i;
 System.out.println(f);
 }
}
```

Why it works: An accumulator multiplied in a for loop.

Verified against 3 test cases: `10⏎` → `3628800`; `0⏎` → `1`; `20⏎` → `2432902008176640000`

Exercise 2 · Pyramid

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 for (int row = 1; row <= n; row++) {
 StringBuilder sb = new StringBuilder();
 for (int c = 1; c <= n - row; c++) sb.append(' ');
 for (int c = 1; c <= 2 * row - 1; c++) sb.append('*');
 System.out.println(sb);
 }
 }
}
```

Why it works: Two inner loops per row: spaces, then stars.

Verified against 3 test cases: `3⏎` → `*⏎  ***⏎*****`; `1⏎` → `*`; `2⏎` → `*⏎***`

Exercise 3 · Collatz steps

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Collatz: n -> n/2 if even, 3n+1 if odd. Count the steps until n becomes 1.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 int steps = 0;
 while (n != 1) {
 n = (n % 2 == 0) ? n / 2 : 3 * n + 1;
 steps++;
 }
 System.out.println(steps);
 }
}
```

Why it works: The loop condition is "n != 1"; the update depends on parity.

Verified against 3 test cases: `6⏎` → `8`; `1⏎` → `0`; `27⏎` → `111`

1.10 Methods: Parameters, Return Values, Scope and Overloading

Exercise 1 · isPrime method

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static boolean isPrime(int n) {
 if (n < 2) return false;
 for (int d = 2; (long) d * d <= n; d++) {
 if (n % d == 0) return false;
 }
 return true;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 System.out.println(isPrime(n) ? "prime" : "not prime");
 }
}
```

Why it works: Trial division up to √n returns false at the first divisor.

Verified against 4 test cases: `97⏎` → `prime`; `1⏎` → `not prime`; `91⏎` → `not prime`

Exercise 2 · Overloaded power

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static long power(long base, int exp) {
 long result = 1;
 for (int i = 0; i < exp; i++) result *= base;
 return result;
 }

 static double power(double base, int exp) { // overloaded version for decimals
 double result = 1;
 for (int i = 0; i < exp; i++) result *= base;
 return result;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long b = sc.nextLong();
 int e = sc.nextInt();
 System.out.println(power(b, e));
 System.out.println(power(2.5, 2));
 }
}
```

Why it works: Overloading chooses the version from the argument types.

Verified against 2 test cases: `3 4⏎` → `81⏎6.25`; `2 10⏎` → `1024⏎6.25`

Exercise 3 · Digits helpers

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Return the number of digits and the digit sum through two separate methods.
 static int digitCount(long n) {
 int c = 0;
 do {
 c++;
 n /= 10;
 } while (n > 0);
 return c;
 }

 static int digitSum(long n) {
 int s = 0;
 while (n > 0) {
 s += n % 10;
 n /= 10;
 }
 return s;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 System.out.println(digitCount(n) + " " + digitSum(n));
 }
}
```

Why it works: Two small methods are easier to test than one big main.

Verified against 3 test cases: `90210⏎` → `5 12`; `0⏎` → `1 0`; `7⏎` → `1 7`

1.11 Tracing and Debugging Programs

Exercise 1 · Trace by hand

Ex1.java

```
public class Ex1 {
 // Trace by hand first. What does this print? (the answer is part of the lesson quiz)
 public static void main(String[] args) {
 int x = 1;
 for (int i = 0; i < 4; i++) {
 x = x * 2 + i;
 }
 System.out.println(x);
 }
}
```

Why it works: See the table in the lesson: 2, 5, 12, 27.

Verified against 1 test case: `(nc input)` → `27`

Exercise 2 · Print the trace

Ex2.java

```
import java.util.Scanner;

public class Ex2 {
 // Print the trace table "i value" for the loop value = value * 2 + 1 starting from the input.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int value = sc.nextInt();
 int steps = sc.nextInt();
 for (int i = 1; i <= steps; i++) {
 value = value * 2 + 1;
 System.out.println(i + " " + value);
 }
 }
}
```

Why it works: The printed lines are the rows of the trace table.

Verified against 2 test cases: `1 3⏎` → `1 3⏎2 7⏎3 15`; `0 2⏎` → `1 1⏎2 3`

Exercise 3 · Fix the maximum

Ex3.java

```
import java.util.Scanner;

public class Ex3 {
 // Fixed version of a buggy "largest of n numbers" that started with max = 0.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int max = Integer.MIN_VALUE;
 for (int i = 0; i < n; i++) {
 int x = sc.nextInt();
 if (x > max) max = x;
 }
 System.out.println(max);
 }
}
```

Why it works: The initial value must be smaller than every possible input.

Verified against 3 test cases: `3⏎-4 -2 -9⏎` → `-2`; `1⏎0⏎` → `0`; `2⏎5 3⏎` → `5`

Part 1 — Quiz answer key

1.1 Variables, Constants and Primitive Types

	Answer	Why
Q1	C. long	It exceeds the int limit of 2.1 billion.
Q2	line 3	A double cannot be assigned to an int without a cast (lossy conversion).
Q3	C. totalMarks	It cannot start with a digit, contain "-", or be a keyword.
Q4	20	LIMIT * 2 = 20.
Q5	C. String	String is a class (reference type).

1.2 Type Conversion, Casting and Overflow

	Answer	Why
Q1	6.5	3 + 3.5 = 6.5.
Q2	true	Overflow wraps to −2147483648.
Q3	C. long p = 100000L * 100000;	One operand is long, so the multiplication happens in long.
Q4	C	Compound assignment includes an implicit cast.
Q5	A. 9	Casting truncates toward zero.

1.3 Arithmetic, Assignment and Increment Operators

	Answer	Why
Q1	12 7	x++ yields 5 (x→6); ++x makes x 7 and yields 7. y = 12, x = 7.
Q2	-2	The remainder has the sign of the dividend.
Q3	B. n % 10	n % 10.
Q4	2	15 → 30 → 2.
Q5	B. 7	^ is XOR: 101 ^ 010 = 111 = 7.

1.4 Relational, Logical and Ternary Operators; Precedence

	Answer	Why
Q1	false	b != 0 is false, so a / b is never evaluated (no exception).
Q2	true	&& is evaluated before ||.
Q3	B. 1 <= x && x <= 10	Two comparisons joined by &&.
Q4	B. "yes"	The condition is true, so the first value.
Q5	line 2	`=` assigns; use `==` to compare.

1.5 Bitwise Operators and Shifts

	Answer	Why
Q1	8	1100 & 1010 = 1000.
Q2	6	101 ^ 011 = 110.
Q3	B. x | (1 << 3)	OR with the mask forces the bit to 1.

	Answer	Why
Q4	-4	Arithmetic shift divides by 4 keeping the sign.
Q5	A. clears the lowest set bit	Subtracting 1 flips the lowest set bit and everything below it.

1.6 Output: print, println, printf and String Concatenation

	Answer	Why
Q1	3345	1 + 2 = 3, then "3" joins → "33", then "4", "5" → 3345.
Q2	66	char + int is an int.
Q3	B. printf("%.2f", 3.14159)	%.2f formats a decimal with two digits.
Q4	C. print	print leaves the cursor on the same line.
Q5	B. error messages	The standard error stream.

1.7 Input: Scanner, BufferedReader and Command-Line Arguments

	Answer	Why
Q1	B. an empty string (the rest of the number's line)	The newline is still waiting.
Q2	18	String concatenation.
Q3	C. BufferedReader	Buffered input is the fastest of the three.
Q4	hasNextInt	hasNextInt() is true while another int is available.

1.8 Decision Making: if, else, switch

	Answer	Why
Q1	B. compile-time error	incompatible types: int cannot be converted to boolean.
Q2	many	No case matches 9, so default runs.
Q3	bcd	Fall-through from case 2.
Q4	B. nothing — ; is an empty statement	The empty statement is legal.
Q5	line 4	break is only allowed inside a loop or switch.

1.9 Loops: for, while, do-while, break and continue

	Answer	Why
Q1	7	1 + 2 + 4 = 7 (3 is skipped).
Q2	12	3 × 4.
Q3	C. do-while	do-while tests after the body.
Q4	243	1, 3, 9, 27, 81, 243 — stops when i ≥ 100.
Q5	line 4	n++ moves n away from 0; it should be n--.

1.10 Methods: Parameters, Return Values, Scope and Overloading

	Answer	Why
Q1	5	The method changed its own copy.
Q2	C. int f(int a) and int f(double a)	Different parameter types; return type alone does not distinguish.

	Answer	Why
Q3	line 5	Not all paths return a value: missing return statement.
Q4	B. only inside that loop body	Scope is the enclosing block.

1.11 Tracing and Debugging Programs

	Answer	Why
Q1	n after pass 1 = 5; n after pass 2 = 2; final count = 3	10 → 5 → 2 → 1: three passes.
Q2	A. runs the current line without entering called methods	Step into enters methods; step over treats them as one step.
Q3	B. trace a small failing input	Reproduce and observe.
Q4	2	It skips a[0] = 2 and counts only 4 and 6.

Part 2 · Core Programming Skills

The building blocks of every algorithm: one- and two-dimensional arrays, strings and StringBuilder, number systems, GCD/primes/modular arithmetic, recursion, references, and writing robust code.

	Lesson	Level	Min	Core
2.1	One-Dimensional Arrays	beginner	40	★
2.2	Two-Dimensional Arrays (Matrices)	beginner	40	★
2.3	Strings, StringBuilder and Characters	beginner	45	★
2.4	Number Systems, Binary and Two's Complement	beginner	40	★
2.5	Divisibility, GCD/LCM, Primes and Modular Arithmetic	beginner	45	★
2.6	Recursion: Base Case, Recursive Case and the Call Stack	intermediate	50	★
2.7	References, Pass-by-Value and Memory	intermediate	35	★
2.8	Boundary Conditions, Validation and Common Mistakes	beginner	30	★

2.1 One-Dimensional Arrays

Core Programming Skills · Beginner · Core programming & data structures · about 40 min · ★ on the minimum-viable path

A fixed-size, indexed list of values of one type: create, traverse, search, reverse, copy — and the aliasing trap.

YOU WILL BE ABLE TO

- Declare, create and initialise arrays; read and write elements by index.
- Traverse with indexed loops and for-each; know the valid index range 0 … length − 1.
- Write the standard array algorithms: sum, maximum, reverse, frequency count.
- Distinguish copying an array from aliasing it, and use `java.util.Arrays`.

Before this lesson: Loops: for, while, do-while, break and continue · Methods: Parameters, Return Values, Scope and Overloading

The idea

An **array** is a numbered row of boxes holding values of the same type, created with a fixed **length**. Elements are accessed by **index**, starting at 0.

```
int[] a = {10, 29, 35, 77, 88};
index : 0 1 2 3 4 a.length == 5
value : 10 29 35 77 88 last index = a.length − 1
```

WHY THIS WORKS

Elements live next to each other in memory, so `a[i]` is found with one multiplication and one addition — **O(1) time** whatever the array size. The price: the length is fixed when the array is created, and inserting in the middle means shifting everything after it (O(n)). Because arrays are **objects**, an array variable holds a *reference* — a copy of the variable shares the same elements.

task	code
create 5 zeros	int[] a = new int[5];
literal	int[] a = {2, 3, 5, 7};
length	a.length (no parentheses!)
last element	a[a.length - 1]
print	Arrays.toString(a)
sort / copy / fill	Arrays.sort(a); Arrays.copyOf(a, n); Arrays.fill(a, v);
compare contents	Arrays.equals(a, b) (a == b compares references)

Syntax

Declare, create, index

```
int[] a = new int[5]; // five zeros; length is fixed
int[] b = {3, 1, 4}; // initialiser list
int[] c = new int[]{1, 2, 3};
a[0] = 10; int x = a[2]; // index 0 .. a.length - 1
int n = a.length; // a field, not a method (no parentheses)
for (int v : b) System.out.println(v);
```

java.util.Arrays	does
Arrays.toString(a)	text like `[3, 1, 4]`
Arrays.sort(a)	ascending, in place
Arrays.fill(a, 7)	set every cell
Arrays.copyOf(a, n)	new array of length n (pads with zeros / truncates)
Arrays.equals(a, b)	same length and same elements
a.clone()	a shallow copy

A tiny example

```
int[] a = {4, 9, 2};
int sum = 0;
for (int x : a) sum += x; // 15
```

Guided example

ArraysDemo.java — Arrays in action (demonstration)

```
import java.util.Arrays;

public class ArraysDemo {
 public static void main(String[] args) {
 // creating arrays
 int[] marks = new int[5]; // five zeros                                    (1)
 int[] primes = {2, 3, 5, 7, 11}; // initialiser list
 String[] names = new String[2]; // two nulls
 System.out.println("length = " + primes.length + " first = " + primes[0] + " last = " +   (2)
     primes[primes.length - 1]);
 System.out.println(Arrays.toString(marks) + " " + Arrays.toString(names));

 // traversing: indexed loop and for-each
 int sum = 0;
 for (int i = 0; i < primes.length; i++) {
 sum += primes[i];
 }
 System.out.println("sum = " + sum);
 for (int p : primes) {
 System.out.print(p * p + " ");
 }
 System.out.println();

 // typical algorithms: max, reverse in place, copy
 int[] a = {4, 9, 2, 7};
 int max = a[0];
 for (int i = 1; i < a.length; i++) {
 if (a[i] > max) max = a[i];
 }
 for (int i = 0, j = a.length - 1; i < j; i++, j--) {                     (3)
 int t = a[i];
 a[i] = a[j];
 a[j] = t;
 }
 int[] copy = Arrays.copyOf(a, a.length); // a real copy                  (4)
 int[] alias = a; // the SAME array                                       (5)
 copy[0] = 100;
 alias[1] = -5;
 System.out.println("max = " + max + ", reversed a = " + Arrays.toString(a) + ", copy =
     " + Arrays.toString(copy));

 // utilities
 int[] b = {5, 1, 4};
 Arrays.sort(b);
 System.out.println(Arrays.toString(b) + " index of 4 = " + Arrays.binarySearch(b, 4));
 Arrays.fill(marks, 7);
 System.out.println(Arrays.toString(marks) + " equals? " + Arrays.equals(b, new int[]{1,
     4, 5}));
 }
}
```

Each block demonstrates one array skill.

Line-by-line walkthrough

#	Where	What happens
1	line 6	Numeric arrays start as all zeros; object arrays start as all null.
2	line 9	The last valid index is length − 1.
3	line 29	Two indices walking towards each other reverse the array in place.
4	line 34	A real, independent copy.
5	line 35	NOT a copy: both variables name the same array, so alias[1] = −5 changes a as well.

Dry run: watch it execute

One pass over an array keeping the best value so far.

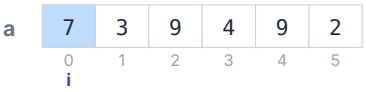
Algorithm: Find the maximum (one scan). **Input:** arr = `7,3,9,4,9,2`.

Pseudocode

```
max = a[0]; maxIndex = 0
for i = 1 .. n−1:
 if a[i] > max:
 max = a[i]; maxIndex = i
print max, maxIndex
```

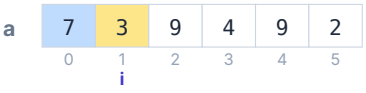
Step 1 of 6. Start by assuming the first element 7 is the maximum.

i = 0 · max = 7 · maxIndex = 0



Step 2 of 6. a[1] = 3 is not larger than 7 (ties do not replace it).

i = 1 · max = 7 · maxIndex = 0



Step 3 of 6. a[2] = 9 is larger than the previous maximum → max = 9.

i = 2 · max = 9 · maxIndex = 2



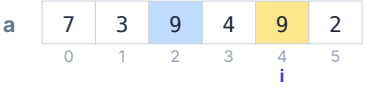
Step 4 of 6. a[3] = 4 is not larger than 9 (ties do not replace it).

i = 3 · max = 9 · maxIndex = 2



Step 5 of 6. a[4] = 9 is not larger than 9 (ties do not replace it).

i = 4 · max = 9 · maxIndex = 2



Step 6 of 6. a[5] = 2 is not larger than 9 (ties do not replace it).

i = 5 · max = 9 · maxIndex = 2



> **RESULT**
> max = 9 at index 2

Complexity

operation	time
read / write a[i]	O(1)
find an element (unsorted)	O(n)
find (sorted, binary search)	O(log n)
insert / delete in the middle	O(n)
copy	O(n)

An array of n ints uses 4n bytes plus a small header (O(n) space).

Common mistakes

ArrayIndexOutOfBoundsException

Problem

```
int[] a = new int[5];
a[5] = 1;
```

Why: Valid indices are 0..4.

Fix: a[4] = 1; // or loop with i < a.length

length vs length()

Problem

```
for (int i = 0; i < a.length(); i++)
```

Why: Arrays have a field `length`; Strings have a method `length()`.

Fix: a.length

Copying with =

Problem

```
int[] b = a;
b[0] = 99; // a[0] changes too
```

Why: Both variables refer to the same array.

Fix: int[] b = a.clone(); // or Arrays.copyOf(a, a.length)

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Print reversed — *Beginner*

Read n and an array; print its elements in reverse order separated by spaces.

Example input	Expected output
5 ⏎ 1 2 3 4 5	5 4 3 2 1

Hint 1: Loop from n − 1 down to 0.

Exercise 2 · Digit frequencies — *Intermediate*

Read n digits (0–9). For every digit that occurs print `digit: count` in increasing digit order.

Example input	Expected output
6 ⏎ 3 1 3 0 9 1	0: 1 ⏎ 1: 2 ⏎ 3: 2 ⏎ 9: 1

Hint 1: A counting array of size 10: freq[d]++.

Exercise 3 · Second largest distinct — *Challenge*

Print the second largest DISTINCT value, or -1 if there is none. One pass, O(1) extra space.

Example input	Expected output
5 ⏎ 4 9 9 2 7	7

Hint 1: Track first and second; update second only when x is strictly between them.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
int[] a = {3, 1, 4, 1, 5};
System.out.println(a.length + " " + a[a.length - 2]);
```

Your answer: _____

Q2. What is printed?

```
int[] a = {1, 2, 3};
int[] b = a;
b[0] = 9;
System.out.println(a[0]);
```

Your answer: _____

Q3. What does `new int[3]` contain?

- **A.** three nulls
- **B.** three zeros
- **C.** garbage
- **D.** nothing

Q4. Which is the valid index range of an array of length n?

- **A.** 1 … n
- **B.** 0 … n
- **C.** 0 … n−1
- **D.** 1 … n−1

Interview / exam questions

- Array vs ArrayList?
- What is the time to access, search, insert?
- How do you copy an array?
- What is ArrayIndexOutOfBoundsException?

Watch & read

Video: Single-Dimensional Arrays in Java (Part 1) — Neso Academy · 9:15. *Declaring, indexing and looping over arrays*

Video: Operations on Arrays in Data Structures: Traversal, Insertion, Deletion and Searching — CodeWithHarry · 28:45. *Typical array operations: search, reverse, max*

REVISION CARD (30–60 SECONDS)

- Indices 0 … length−1; a.length is a field.
- O(1) access; O(n) search/insert.
- = copies the reference; clone / Arrays.copyOf copies the data.
- Arrays.toString / sort / fill / equals.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The Block Swap slide introduces arrays using C-like syntax (`Int Arr[5]= {…}`, `Char Arr[7]={…}` with only 6 initialisers). Java declares `int[] arr = {…};` and the size is taken from the initialiser.

2.2 Two-Dimensional Arrays (Matrices)

Core Programming Skills · Beginner · Core programming & data structures · about 40 min · ★ on the minimum-viable path

Arrays of arrays: rows and columns, nested loops, transpose, diagonals and jagged arrays.

YOU WILL BE ABLE TO

- Create and traverse a rows × columns matrix with nested loops.
- Use `m.length` (rows) and `m[0].length` (columns) correctly.
- Compute row sums, column sums, a diagonal and a transpose.
- Explain row-major order and jagged arrays.

Before this lesson: One-Dimensional Arrays

The idea

A **2-D array** is an array whose elements are themselves arrays — each a **row**. `int[][] m = new int[3][4]` is 3 rows of 4 columns; `m[i][j]` is row i, column j.

```
int[][] m = { {1, 2, 3},
 {4, 5, 6},
 {7, 8, 9} };
 m[1][2] = 6 m.length = 3 rows m[0].length = 3 columns
 diagonal: m[i][i] anti-diagonal: m[i][n-1-i]
```

WHY THIS WORKS

Two nested loops visit every cell: the outer loop picks the row, the inner loop sweeps the columns — **row-major order**, which is also the fastest order in memory. Problems on grids (hourglass, maze, N-Queens, grid paths) all start with exactly this traversal. Many grid problems also use *neighbour offsets* (`dr`, `dc` arrays) to step up/down/left/right without copying code.

Rows need not have the same length (a **jagged** array): `int[][] tri = new int[4][];` then `tri[i] = new int[i + 1];` builds a triangle such as Pascal's.

Syntax

```java
int[][] m = new int[3][4]; // 3 rows, 4 columns, all zeros
int[][] t = {{1, 2, 3}, {4, 5, 6}}; // 2 x 3
int rows = t.length; // number of rows
int cols = t[0].length; // length of row 0
t[1][2] = 60; // row 1, column 2
int[][] jagged = new int[3][]; // rows created separately
for (int i = 0; i < 3; i++) jagged[i] = new int[i + 1];
System.out.println(Arrays.deepToString(t));
```

WATCH OUT

`m[i][j]` is "row i, column j". Mixing the two indices is the most common 2-D bug; name them `r` and `c` while learning.

A tiny example

```
int total = 0;
for (int i = 0; i < m.length; i++)
 for (int j = 0; j < m[i].length; j++)
 total += m[i][j];
```

Guided example

MatrixDemo.java — Matrix operations (demonstration)

```
1  import java.util.Arrays;
2
3  public class MatrixDemo {
4   public static void main(String[] args) {
5   // a 2-D array is an array of rows
6   int[][] m = {
7   {1, 2, 3},
8   {4, 5, 6},
9   {7, 8, 9}
10  };
11  int rows = m.length;
12  int cols = m[0].length;                                                    (1)
13  System.out.println(rows + " x " + cols + ", m[1][2] = " + m[1][2]);
14
15  // row-major traversal, row sums, column sums, diagonal
16  int[] rowSum = new int[rows];
17  int[] colSum = new int[cols];
18  int diagonal = 0;
19  for (int i = 0; i < rows; i++) {
20  for (int j = 0; j < cols; j++) {
21  rowSum[i] += m[i][j];                                                      (2)
22  colSum[j] += m[i][j];
23  if (i == j) diagonal += m[i][j];                                           (3)
24  }
25  }
26  System.out.println("row sums " + Arrays.toString(rowSum) + " column sums " +
        Arrays.toString(colSum) + " diagonal " + diagonal);
27
28  // transpose into a new matrix
29  int[][] t = new int[cols][rows];
30  for (int i = 0; i < rows; i++)
31  for (int j = 0; j < cols; j++)
32  t[j][i] = m[i][j];                                                         (4)
33  System.out.println("transpose: " + Arrays.deepToString(t));
34
35  // jagged array: rows of different length
36  int[][] tri = new int[4][];                                                (5)
37  for (int i = 0; i < 4; i++) {
38  tri[i] = new int[i + 1];
39  tri[i][0] = tri[i][i] = 1;
40  for (int j = 1; j < i; j++) tri[i][j] = tri[i - 1][j - 1] + tri[i - 1][j];
41  }
42  System.out.println("Pascal: " + Arrays.deepToString(tri));
43  }
44 }
```

A 3×3 matrix processed with nested loops.

Line-by-line walkthrough

#	Where	What happens
1	line 12	The number of columns is the length of any row.
2	line 21	Row i accumulates across the inner loop; colSum[j] accumulates down the rows.
3	line 23	Main diagonal: row index equals column index.
4	line 32	Transpose swaps the roles of row and column.
5	line 36	Only the number of rows is given: each row array is created separately and may differ in length.

Dry run: watch it execute

Nested loops: row by row, column by column.

Algorithm: Row sums and the main diagonal. **Input:** n = `4`.

Pseudocode

```
for r = 0 .. n-1:
 rowSum = 0
 for c = 0 .. n-1:
 rowSum += m[r][c]
 if r == c: diag += m[r][c]
```

The full trace has 16 steps; 14 evenly spaced ones are shown. Run the rest in the website's Dry-run lab.

Step 1 of 16. m[0][0] = 1 → rowSum = 1; it is on the diagonal, so diag = 1.

r = 0 · c = 0 · rowSum = 1 · diag = 1

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Step 2 of 16. m[0][1] = 2 → rowSum = 3.

r = 0 · c = 1 · rowSum = 3 · diag = 1

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Step 3 of 16. m[0][2] = 3 → rowSum = 6.

r = 0 · c = 2 · rowSum = 6 · diag = 1

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Step 4 of 16. m[0][3] = 4 → rowSum = 10.

r = 0 · c = 3 · rowSum = 10 · diag = 1

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Step 6 of 16. m[1][1] = 6 → rowSum = 11; it is on the diagonal, so diag = 7.

r = 1 · c = 1 · rowSum = 11 · diag = 7

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Step 7 of 16. m[1][2] = 7 → rowSum = 18.

r = 1 · c = 2 · rowSum = 18 · diag = 7

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Step 8 of 16. m[1][3] = 8 → rowSum = 26.

r = 1 · c = 3 · rowSum = 26 · diag = 7

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Step 9 of 16. m[2][0] = 9 → rowSum = 9.

r = 2 · c = 0 · rowSum = 9 · diag = 7

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Step 10 of 16. m[2][1] = 10 → rowSum = 19.

r = 2 · c = 1 · rowSum = 19 · diag = 7

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Step 11 of 16. m[2][2] = 11 → rowSum = 30; it is on the diagonal, so diag = 18.

r = 2 · c = 2 · rowSum = 30 · diag = 18

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Step 13 of 16. m[3][0] = 13 → rowSum = 13.

r = 3 · c = 0 · rowSum = 13 · diag = 18

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Step 14 of 16. m[3][1] = 14 → rowSum = 27.

r = 3 · c = 1 · rowSum = 27 · diag = 18

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Step 15 of 16. m[3][2] = 15 → rowSum = 42.

r = 3 · c = 2 · rowSum = 42 · diag = 18

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Step 16 of 16. m[3][3] = 16 → rowSum = 58; it is on the diagonal, so diag = 34.

r = 3 · c = 3 · rowSum = 58 · diag = 34

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

> **RESULT**
> diagonal sum = 34

Complexity

Visiting every cell of an R × C matrix is **O(R·C)**; a transpose needs O(R·C) extra space; reading one cell is O(1). Square matrices (n × n) make n^2 cells — so a nested double loop over the matrix is $O(n^2)$ in n but linear in the number of cells.

Common mistakes

Swapping rows and columns in the loop bounds

Problem

```
for (int i = 0; i < m[0].length; i++)
 for (int j = 0; j < m.length; j++)
 use(m[i][j]);
```

Why: For a non-square matrix this runs off the end: i indexes rows (bounded by m.length).

Fix: i < m.length for rows; j < m[i].length for columns

Using the same variable in both loops

Problem

```
for (int i = 0; i < n; i++)
 for (int i = 0; i < n; i++)
```

Why: The inner i is already declared: compile error.

Fix: Use i and j.

Copying a matrix with clone()

Problem

```
int[][] b = a.clone();
```

Why: clone is shallow: b shares the row arrays with a, so writing b[0][0] also changes a[0][0].

Fix: Copy each row: for (int i...) b[i] = a[i].clone();

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Sum of a matrix — *Beginner*

Read R, C and the matrix. Print the sum of all elements.

Example input	Expected output
2 3 ⏎ 1 2 3 ⏎ 4 5 6	21

Hint 1: Two nested loops.

Exercise 2 · Transpose — *Intermediate*

Read R, C and the matrix. Print the C × R transpose, one row per line.

Example input	Expected output
2 3 ⏎ 1 2 3 ⏎ 4 5 6	1 4 ⏎ 2 5 ⏎ 3 6

Hint 1: Swap the loop order when printing.

Exercise 3 · Spiral order — *Challenge*

Read n and an n × n matrix. Print its elements in clockwise spiral order starting at the top-left, on one line.

Example input	Expected output
3 ⏎ 1 2 3 ⏎ 4 5 6 ⏎ 7 8 9	1 2 3 6 9 8 7 4 5

Hint 1: Keep four boundaries: top, bottom, left, right and shrink them.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
int[][] m = {{1, 2}, {3, 4}, {5, 6}};
System.out.println(m.length + " " + m[0].length + " " + m[2][1]);
```

Your answer: _____

Q2. The main diagonal of a square matrix is where...

- **A.** i + j = n
- **B.** i == j
- **C.** i > j
- **D.** j == 0

Q3. The time to read all cells of an R × C matrix is...

- **A.** O(R + C)
- **B.** O(R·C)
- **C.** O(log R)
- **D.** O(1)

Q4. Which statement creates a 3-row jagged array?

- **A.** new int[3][3]
- **B.** new int[3][]
- **C.** new int[][3]
- **D.** new int[3]

Interview / exam questions

- What is row-major order?
- How do you rotate a matrix 90° in place?
- How are 2-D arrays stored in Java?
- Why does clone() on a 2-D array not give an independent copy?

Watch & read

Video: Learn Java 2D arrays in 9 minutes! — Bro Code · 9:38. *Two-dimensional arrays and nested loops*

Video: 2D Arrays | Java Complete Placement Course | Lecture 11 — Apna College · 19:53. *Traversing a matrix by rows, columns and diagonals*

REVISION CARD (30–60 SECONDS)

- m.length = rows; m[i].length = columns.
- Nested loops: outer row, inner column.
- Diagonal i==j; anti-diagonal i+j==n−1; transpose t[j][i]=m[i][j].
- clone() is shallow for 2-D arrays.

2.3 Strings, StringBuilder and Characters

Core Programming Skills · Beginner · Core programming & data structures · about 45 min · ★ on the minimum-viable path

Immutable text, the methods you will use daily, comparing correctly, and StringBuilder for efficient edits.

YOU WILL BE ABLE TO

- Use `length`, `charAt`, `substring`, `indexOf`, `equals`, `compareTo`, `split`, `join`, `trim`, `replace`.
- Explain immutability and why `==` is wrong for comparing content.
- Build and modify text with `StringBuilder`.
- Process a string character by character with `char` arithmetic (`c - 'a'`).

Before this lesson: Methods: Parameters, Return Values, Scope and Overloading · One-Dimensional Arrays

The idea

A **String** is a sequence of characters. It is an **immutable** object: methods such as `toUpperCase()` never change the original; they return a new String. Indexing is zero-based: `"Java".charAt(0)` is `'J'`.

method	returns	example on s = "Java Programming"
length()	number of chars	16
charAt(i)	char at i	s.charAt(5) = 'P'
substring(a, b)	chars a … b−1	s.substring(0, 4) = "Java"
indexOf(x)	first position or −1	s.indexOf("gram") = 8
equals(t) / equalsIgnoreCase(t)	content equality	true / false
compareTo(t)	< 0, 0, > 0 (dictionary order)	"apple".compareTo("banana") < 0
contains / startsWith / endsWith	boolean	s.contains("Pro") = true
trim() toUpperCase() replace(a, b)	new String	s.replace('a', '@')
split(regex) String.join(sep, …)	String[] / String	"a,b".split(",") → {"a","b"}

WHY THIS WORKS

Because Strings are immutable, `s = s + c` inside a loop copies the whole string each time — O(n²) for n appends. **StringBuilder** keeps a growable char buffer: `append`, `insert`, `reverse`, `setCharAt`, `deleteCharAt` modify in place, and `toString()` produces the final String. Use it whenever you build text in a loop.

A **char** is a small number, so letter arithmetic is natural: `c - 'a'` is 0 for `'a'`…25 for `'z'` (an index for a 26-element counter array), and `(char) ('a' + 3)` is `'d'`. `Character.isDigit / isLetter / isUpperCase / toUpperCase` classify and convert.

WATCH OUT

`"hello" == new String("hello")` is **false**: `==` compares object identity. Always use `.equals()` for text.

Syntax

String method	result
s.length()	number of characters
s.charAt(i)	char at index i
s.substring(a, b)	characters a … b−1
s.indexOf("x") / lastIndexOf	first / last position or −1
s.equals(t) / equalsIgnoreCase	content comparison (never `==`)
s.compareTo(t)	negative, 0 or positive (alphabetical)
s.toUpperCase() / toLowerCase()	new String
s.trim() / strip()	without leading/trailing spaces
s.split(" ")	String[] of pieces
s.replace("a", "b")	new String
s.contains("x") / startsWith / endsWith	boolean
s.toCharArray()	char[] copy
String.valueOf(x) / String.join(",", list)	conversion / joining

```
StringBuilder sb = new StringBuilder();
sb.append("ab").append('c').append(42); // "abc42" (mutable, no copying)
sb.insert(0, "X"); sb.reverse(); sb.setCharAt(0, 'Z'); sb.deleteCharAt(1);
String result = sb.toString();
// char helpers
boolean d = Character.isDigit(c), l = Character.isLetter(c);
char up = Character.toUpperCase(c);
int digit = c - '0'; // '7' - '0' == 7
```

A tiny example

```
String s = "level";
boolean pal = new StringBuilder(s).reverse().toString().equals(s); // true
```

Guided example

StringsDemo.java — String toolbox (demonstration)

```
public class StringsDemo {
 public static void main(String[] args) {
  String s = "Java Programming";
  System.out.println(s.length()); // 16
  System.out.println(s.charAt(0) + " " + s.charAt(s.length() - 1));
  System.out.println(s.substring(5)); // Programming
  System.out.println(s.substring(0, 4)); // Java
  System.out.println(s.indexOf("gram") + " " + s.indexOf('z')); // 8 and -1                1
  System.out.println(s.toUpperCase() + " | " + s.replace('a', '@'));
  System.out.println(s.contains("Pro") + " " + s.startsWith("Ja") + " " +
      s.endsWith("ing"));

  // comparing: equals vs ==
  String a = "hello";
  String b = new String("hello");
  System.out.println((a == b) + " " + a.equals(b) + " " + a.equalsIgnoreCase("HELLO"));   2
  System.out.println("apple".compareTo("banana") + " " + "b".compareTo("a"));

  // split / join / trim
  String csv = " red, green ,blue ";
  for (String part : csv.trim().split("\\s*,\\s*")) {                                    3
  System.out.print("[" + part + "]");
  }
  System.out.println();
  System.out.println(String.join("-", "2026", "10", "10"));

  // strings are IMMUTABLE: methods return new strings
  String t = "abc";
  t.toUpperCase(); // result thrown away                                                 4
  System.out.println(t + " " + t.toUpperCase());

  // StringBuilder: mutable, efficient for many edits
  StringBuilder sb = new StringBuilder();
  for (int i = 0; i < 5; i++) {
  sb.append(i).append(',');
  }
  sb.setLength(sb.length() - 1); // drop the trailing comma                             5
  sb.insert(0, '[').append(']');
  System.out.println(sb + " reversed: " + new StringBuilder("stressed").reverse());

  // char operations
  char c = 'q';
  System.out.println(Character.isLetter(c) + " " + Character.isDigit('7') + " " +
      Character.toUpperCase(c) + " " + (char) (c + 1) + " " + (c - 'a'));
  char[] letters = "hello".toCharArray();
  java.util.Arrays.sort(letters);                                                        6
  System.out.println(new String(letters));
  }
 }
```

A tour of everything above.

Line-by-line walkthrough

#	Where	What happens
1	line 8	8 (found) and −1 (not found).
2	line 15	false / true: identity vs content. The reason `==` is a bug for Strings.
3	line 20	split takes a regular expression: here "optional spaces, a comma, optional spaces".
4	line 28	The returned String is discarded, so t is unchanged — immutability.
5	line 36	StringBuilder can be shortened in place to remove a trailing separator.
6	line 44	toCharArray + sort + new String gives the characters in order — a standard trick for anagrams.

Dry run: watch it execute

Two pointers swapping characters until they meet.

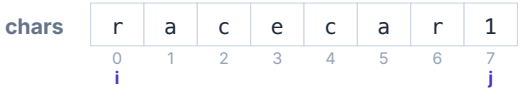
Algorithm: Reverse a string with two pointers. **Input:** s = `racecar1`.

Pseudocode

```
i = 0; j = n − 1
while i < j:
 swap(c[i], c[j])
 i++; j−−
```

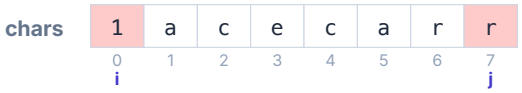
Step 1 of 6. Pointers start at both ends.

i = 0 · j = 7



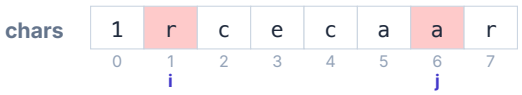
Step 2 of 6. Swap c[0] and c[7].

i = 0 · j = 7



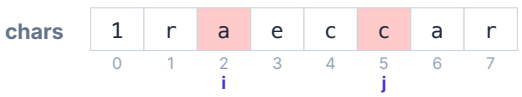
Step 3 of 6. Swap c[1] and c[6].

i = 1 · j = 6



Step 4 of 6. Swap c[2] and c[5].

i = 2 · j = 5



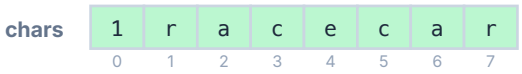
Step 5 of 6. Swap c[3] and c[4].

i = 3 · j = 4



Step 6 of 6. The pointers met or crossed: finished.

i = 4 · j = 3



> **RESULT**
> 1racecar

Complexity

operation	time
length(), charAt(i)	O(1)
substring, concatenation, equals, indexOf	O(n)
s += c repeated n times	O(n²)
StringBuilder.append repeated n times	O(n) amortised
sorting the characters	O(n log n)

Common mistakes

Comparing with ==

Problem

```
if (answer == "yes")
```

Why: Compares references; may be true by accident for literals and false for input read from the keyboard.

Fix: if (answer.equals("yes")) // or "yes".equals(answer) to be null-safe

Ignoring the return value

Problem

```
s.trim();
System.out.println(s);
```

Why: trim returns a new String.

Fix: s = s.trim();

substring end index

Problem

```
s.substring(2, 5) // expecting 5 characters
```

Why: The end index is exclusive: this has 3 characters.

Fix: s.substring(2, 7) // 5 characters

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Count vowels — *Beginner*

Read a line and print how many vowels (a, e, i, o, u, case-insensitive) it contains.

Example input	Expected output
Programming Language	7

Hint 1: "aeiou".indexOf(c) >= 0

Exercise 2 · Anagrams — *Intermediate*

Read two words. Print YES if they are anagrams (same letters, same counts, ignoring case), else NO.

Example input	Expected output
Listen Silent	YES

Hint 1: A 26-element count array: ++ for the first word, −− for the second.

Exercise 3 · Run-length encoding — *Intermediate*

Read a word and print its run-length encoding: each maximal block of equal characters becomes the character followed by the block length.

Example input	Expected output
aaabccdddd	a3b1c2d4

Hint 1: A while loop with an inner scan for the end of the run.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
String s = "hello";
System.out.println(s.substring(1, 3) + s.indexOf('l'));
```

Your answer: _____

Q2. What is printed?

```
String a = "hi";
String b = new String("hi");
System.out.println(a == b);
```

Your answer: _____

Q3. Which class should you use to build a long string in a loop?

- **A.** String
- **B.** StringBuilder
- **C.** char
- **D.** Scanner

Q4. What is printed?

```
char c = 'd';
System.out.println(c - 'a');
```

Your answer: _____

Q5. Strings are...

- **A.** mutable
- **B.** immutable
- **C.** primitive
- **D.** arrays of int

Interview / exam questions

- Why is String immutable?
- String vs StringBuilder vs StringBuffer?
- == vs equals for Strings?
- What is the String pool?

Watch & read

Video: Useful string methods in Java! — Bro Code · 8:36. *String methods, immutability and StringBuilder*

Video: #35 Mutable vs Immutable String in Java — Telusko · 6:23. *String immutability and equals vs ==*

REVISION CARD (30–60 SECONDS)

- Immutable; methods return new Strings.
- Compare with equals, not ==. compareTo for order.
- substring(a, b) excludes b; indexOf → −1 if absent.
- StringBuilder for loops; c − 'a' maps letters to 0..25.

2.4 Number Systems, Binary and Two's Complement

Core Programming Skills · Beginner · Core programming & data structures · about 40 min · ★ on the minimum-viable path

Positional notation in base 2, 8, 10 and 16; converting by repeated division; how negative numbers are stored.

YOU WILL BE ABLE TO

- Read a number written in any base as the sum of digit × base^p^o^s^i^t^i^oⁿ.
- Convert decimal ↔ binary ↔ hexadecimal by hand and in Java.
- Add binary numbers with carries.
- Explain two's complement and compute the n-bit pattern of a negative number.

Before this lesson: Arithmetic, Assignment and Increment Operators · Loops: for, while, do-while, break and continue

The idea

In everyday (**decimal**) notation each position is worth ten times the one to its right: 156 = 1·10² + 5·10¹ + 6·10⁰. **Binary** works the same way with base 2; each digit (a **bit**) is 0 or 1:

```
10011100₂ = 1·128 + 0·64 + 0·32 + 1·16 + 1·8 + 1·4 + 0·2 + 0·1 = 156
weights : 128 64 32 16 8 4 2 1
```

base	name	digits	used for
2	binary	0 1	how computers store everything
8	octal	0–7	file permissions
10	decimal	0–9	people
16	hexadecimal	0–9 A–F	compact binary: 1 hex digit = 4 bits (a nibble)

Decimal → base b: repeated division

```
156 ÷ 2 = 78 r 0
 78 ÷ 2 = 39 r 0
 39 ÷ 2 = 19 r 1
 19 ÷ 2 = 9 r 1
 9 ÷ 2 = 4 r 1
 4 ÷ 2 = 2 r 0
 2 ÷ 2 = 1 r 0
 1 ÷ 2 = 0 r 1 read the remainders from the bottom up → 10011100
```

WHY THIS WORKS

The remainder of n ÷ 2 is the **lowest bit**; dividing by 2 drops that bit. So repeating "take n % 2, then n /= 2" produces the bits from least to most significant — hence they are reversed at the end. The reverse direction (**Horner's rule**: value = value × base + digit) rebuilds the number in one pass.

Two's complement (negative numbers)

Computers store a **signed** n-bit integer so that addition works the same for positive and negative numbers. The top bit has weight -2^{n-1}. To get the pattern of −x: **flip all bits of x, then add 1** (equivalently $2^n - x$).

```
 5 = 0000 0101
~5 = 1111 1010
+1 = 1111 1011 = −5 (8 bits: range −128 … 127)
```

Syntax

Java	does
Integer.toBinaryString(n)	binary text of n (negative → 32-bit pattern)
Integer.toHexString(n)	hex text
Integer.parseInt("9C", 16)	parse text in a base
Integer.toString(n, 7)	any base 2…36
0b1010 0x1F 017	binary, hex, octal literals
Integer.bitCount(n)	number of 1 bits

A tiny example

```java
String bits = Integer.toBinaryString(156); // "10011100"
int back = Integer.parseInt(bits, 2); // 156
```

Guided example

NumberSystems.java — Base conversion by hand (demonstration)

```
1  public class NumberSystems {
2   // Convert a non-negative int to a string in the given base (2..16) by repeated
        division.
3   static String toBase(int n, int base) {
4   if (n == 0) return "0";
5   String digits = "0123456789ABCDEF";
6   StringBuilder sb = new StringBuilder();
7   while (n > 0) {
8   sb.append(digits.charAt(n % base)); // the remainder is the next digit (least
        significant first)                                                          (1)
9   n /= base;                                                                      (2)
10  }
11  return sb.reverse().toString();                                                 (3)
12  }
13
14  // Convert back: Horner's rule.
15  static int fromBase(String s, int base) {
16  int value = 0;
17  for (char c : s.toCharArray()) {
18  value = value * base + Character.digit(c, base);                                (4)
19  }
20  return value;
21  }
22
23  // n-bit two's complement pattern of a possibly negative number
24  static String twosComplement(int x, int bits) {
25  StringBuilder sb = new StringBuilder();
26  for (int i = bits - 1; i >= 0; i--) {
27  sb.append((x >> i) & 1);                                                        (5)
28  }
29  return sb.toString();
30  }
31
32  public static void main(String[] args) {
33  System.out.println("156 in binary : " + toBase(156, 2));
34  System.out.println("156 in octal : " + toBase(156, 8));
35  System.out.println("156 in hex : " + toBase(156, 16));
36  System.out.println("10011100 (2) : " + fromBase("10011100", 2));
37  System.out.println("built-in : " + Integer.toBinaryString(156) + " " +
        Integer.toHexString(156) + " " + Integer.parseInt("9C", 16));
38  System.out.println("-5 in 8 bits : " + twosComplement(-5, 8) + " (flip 00000101 ->
        11111010, add 1)");
39  System.out.println(" 5 in 8 bits : " + twosComplement(5, 8));
40  System.out.println("-1 in 8 bits : " + twosComplement(-1, 8));
41  System.out.println("range of 8-bit two's complement: -128 .. 127");
42  }
43 }
```

Converting without library calls so that you see the arithmetic.

Line-by-line walkthrough

#	Where	What happens
1	line 8	The remainder is the least significant digit; look it up in "0123456789ABCDEF".
2	line 9	Remove that digit.
3	line 11	Digits were produced backwards.
4	line 18	Horner's rule: shift everything left one position and add the new digit.
5	line 27	Reading bit i from the top: Java ints are already in two's complement, so this prints the pattern for negatives too.

Dry run: watch it execute

Repeated division by 2; try −45 for two's complement.

Algorithm: Decimal to binary (and two's complement). **Input:** n = `45`.

Pseudocode

```
while n > 0:
 remainder = n % 2
 n = n / 2
read the remainders bottom-up
negative: invert all 8 bits, add 1
```

Step 1 of 7. 45 ÷ 2 = 22 remainder 1.

n = 45 · quotient = 22 · remainder = 1

remainders so far (first ▯ last)

1
0

Step 2 of 7. 22 ÷ 2 = 11 remainder 0.

n = 22 · quotient = 11 · remainder = 0

remainders so far (first ▯ last)

1	0
0	1

Step 3 of 7. 11 ÷ 2 = 5 remainder 1.

n = 11 · quotient = 5 · remainder = 1

remainders so far (first ▯ last)

1	0	1
0	1	2

Step 4 of 7. 5 ÷ 2 = 2 remainder 1.

n = 5 · quotient = 2 · remainder = 1

remainders so far (first ▯ last)

1	0	1	1
0	1	2	3

Step 5 of 7. 2 ÷ 2 = 1 remainder 0.

n = 2 · quotient = 1 · remainder = 0

remainders so far (first ▯ last)

1	0	1	1	0
0	1	2	3	4

Step 6 of 7. 1 ÷ 2 = 0 remainder 1.

n = 1 · quotient = 0 · remainder = 1

remainders so far (first ▯ last)

1	0	1	1	0	1
0	1	2	3	4	5

Step 7 of 7. Read the remainders from the LAST to the FIRST: 101101 (45 in binary).

binary = 101101 · 8-bit = 00101101

bits (high ▒ low)	0	0	1	0	1	1	0	1
	0	1	2	3	4	5	6	7

> **RESULT**
> 45 = 00101101 (8 bits)

Complexity

Conversion takes one step per digit: log_b(n) steps for a number n in base b — O(log n).

Common mistakes

Reading remainders top-down

Problem

```
// 156 → "00111001"
```

Why: The first remainder is the lowest bit; the result must be reversed.

Fix: Read the remainders bottom-up (or reverse the StringBuilder).

Forgetting leading zeros when a fixed width is needed

Problem

```
Integer.toBinaryString(5) // "101" not "00000101"
```

Why: The method drops leading zeros.

Fix: "0".repeat(8 - s.length()) + s

Confusing the stored bits with the printed text

Problem

```
int x = 0b1010; // "x is binary"
```

Why: x is just a number; binary, decimal and hex are different WRITINGS of the same value.

Fix: Format only when printing.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · To binary — *Beginner*

Read a non-negative integer and print its binary representation (without Integer.toBinaryString).

Example input	Expected output
156	10011100

Hint 1: Repeated division by 2; reverse the remainders; handle 0.

Exercise 2 · From any base — *Intermediate*

Read a number written in base b (2 ≤ b ≤ 16) and the base. Print its decimal value.

Example input	Expected output
9C 16	156

Hint 1: value = value * base + digit

Exercise 3 · Add two binary strings — *Challenge*

Read two binary numbers as strings (up to 1000 digits) and print their sum in binary, without converting to int.

Example input	Expected output
1010 1110	11000

Hint 1: Add digit by digit from the right with a carry.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is 1011_2 in decimal?

```
// 8 + 0 + 2 + 1
```

Your answer: _____

Q2. What is the 8-bit two's complement pattern of −3?

```
// 3 = 00000011
```

Your answer: _____

Q3. One hexadecimal digit corresponds to how many bits?

- **A.** 2
- **B.** 3
- **C.** 4
- **D.** 8

Q4. The range of an 8-bit two's-complement integer is...

- **A.** 0...255
- **B.** −127..127
- **C.** −128...127
- **D.** −256...255

Q5. In repeated division by 2, the first remainder is...

- **A.** the most significant bit
- **B.** the least significant bit
- **C.** always 0
- **D.** the sign

Interview / exam questions

- How do computers represent negative integers?
- Convert 255 to binary and hex.
- Why hexadecimal?
- What is overflow in two's complement?

Watch & read

Video: Number Systems Introduction - Decimal, Binary, Octal & Hexadecimal — The Organic Chemistry Tutor · 10:57. *Converting between binary, decimal and hexadecimal*

Video: Twos complement: Negative numbers in binary — Ben Eater · 13:49. *How negative integers are stored*

REVISION CARD (30–60 SECONDS)

- Position weights: base^k. Binary 128 64 32 16 8 4 2 1.
- Decimal→base: repeated ÷, read remainders upwards. Base→decimal: Horner.
- 1 hex digit = 4 bits.
- Two's complement: $-x = \sim x + 1$; n-bit range $-2^{n-1} \ldots 2^{n-1}-1$.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The Booth deck revises binary addition and two's complement only as screenshots (21 → 10101 by repeated division); this lesson makes those prerequisites explicit.

2.5 Divisibility, GCD/LCM, Primes and Modular Arithmetic

Core Programming Skills · Beginner · Core programming & data structures · about 45 min · ★ on the minimum-viable path

The number-theory vocabulary used by the archive: divisors, gcd, lcm, primes, congruence and fast exponentiation.

YOU WILL BE ABLE TO

- Test divisibility and list the divisors of n in O(√n).
- Define gcd, lcm and coprime; relate lcm to gcd.
- Test primality by trial division up to √n.
- Use modular arithmetic rules and compute a^b mod m by repeated squaring.

Before this lesson: Methods: Parameters, Return Values, Scope and Overloading · Loops: for, while, do-while, break and continue · Arithmetic, Assignment and Increment Operators

The idea

a divides b (written a | b) when b % a == 0. The **divisors** of 36 are 1, 2, 3, 4, 6, 9, 12, 18, 36. A **prime** has exactly two divisors (1 and itself); a **composite** has more. Two numbers are **coprime** when their gcd is 1.

concept	meaning	example
gcd(a, b)	largest number dividing both	gcd(84, 36) = 12
lcm(a, b)	smallest positive number divisible by both	lcm(84, 36) = 252
relation	gcd(a, b) × lcm(a, b) = a × b	12 × 252 = 84 × 36
a mod m	remainder after division by m (0 … m−1)	−7 mod 5 = 3

WHY THIS WORKS

Divisors come in pairs (d, n/d); one of the two is ≤ √n. So to list divisors or test for a divisor you only have to look at d ≤ √n — O(√n) instead of O(n). This single fact is behind trial division, the sieve, Euler's φ and many other algorithms in this course.

Modular arithmetic

Working "mod m" means keeping only remainders: numbers wrap around like a clock of m hours. The key rules (for **+, −, ×**, not division):

```
(a + b) mod m = ((a mod m) + (b mod m)) mod m (a · b) mod m = ((a mod m)
                                              · (b mod m)) mod m
```

That lets you keep numbers small even when the true result is astronomically large — e.g. a^b mod m by **repeated squaring**: square the base each time and multiply it into the result only when the current bit of the exponent is 1. It needs O(log b) multiplications.

Syntax

NEGATIVE REMAINDERS

Java's `%` can return negative numbers. For a result in 0…m−1 use `Math.floorMod(a, m)` or `((a % m) + m) % m`.

A tiny example

```
static boolean isPrime(long n) {
 if (n < 2) return false;
 for (long d = 2; d * d <= n; d++)
 if (n % d == 0) return false;
 return true;
}
```

Guided example

NumberTheoryBasics.java — Number theory toolkit (demonstration)

```
1   import java.util.Scanner;
2
3   public class NumberTheoryBasics {
4    static boolean isPrime(long n) {                                    (1)
5    if (n < 2) return false;
6    if (n % 2 == 0) return n == 2;
7    for (long d = 3; d * d <= n; d += 2) {
8    if (n % d == 0) return false;
9    }
10   return true;
11   }
12
13   static long gcd(long a, long b) {
14   while (b != 0) {
15   long r = a % b;                                                     (2)
16   a = b;
17   b = r;
18   }
19   return a;
20   }
21
22   // a^b mod m by repeated squaring: O(log b) multiplications
23   static long powMod(long a, long b, long m) {
24   long result = 1;
25   a %= m;
26   while (b > 0) {
27   if ((b & 1) == 1) result = result * a % m;                          (3)
28   a = a * a % m;                                                      (4)
29   b >>= 1;
30   }
31   return result;
32   }
33
34   public static void main(String[] args) {
35   Scanner sc = new Scanner(System.in);
36   long a = sc.nextLong();
37   long b = sc.nextLong();
```

(code continued)

```
38    long g = gcd(a, b);
39    System.out.println("gcd = " + g);
40    System.out.println("lcm = " + (a / g * b));
41    System.out.println(a + " prime? " + isPrime(a) + ", " + b + " prime? " + isPrime(b));
42    System.out.println("a divisible by 3 (digit sum rule)? " + (digitSum(a) % 3 == 0));
43    System.out.println("2^100 mod 1000000007 = " + powMod(2, 100, 1_000_000_007L));
44    System.out.println("(-7) mod 5 = " + Math.floorMod(-7, 5) + " (Java's % gives " + (-7 %
          5) + ")");
45    System.out.println("(a * b) mod 13 = " + (a % 13) * (b % 13) % 13);
46    }
47
48    static long digitSum(long n) {
49    long s = 0;
50    while (n > 0) {
51    s += n % 10;
52    n /= 10;
53    }
54    return s;
55    }
56   }
```

5

Sample input: `84 36`.

Sample input

```
84 36
```

Line-by-line walkthrough

#	Where	What happens
1	line 4	Handles 2 separately, then tests odd divisors up to √n.
2	line 15	Euclid's algorithm (full lesson later in the archive module).
3	line 27	Repeated squaring: when the current bit of the exponent is 1, multiply the base into the result.
4	line 28	Square the base and reduce — values stay below m².
5	line 44	3, the mathematically correct residue; −7 % 5 is −2.

Dry run: watch it execute

91 = 7 × 13: found after six trial divisions.

Algorithm: Trial division up to √n. **Input:** n = `91`.

Pseudocode

```
for d = 2; d * d <= n; d++:
 if n % d == 0: composite (d divides n)
otherwise n is prime
```

Step 1 of 7. d = 2: 2² = 4 ≤ 91, so test it: 91 % 2 = 1.

d = 2 · d * d = 4 · n % d = 1

Step 2 of 7. d = 3: 3² = 9 ≤ 91, so test it: 91 % 3 = 1.

d = 3 · d * d = 9 · n % d = 1

Step 3 of 7. d = 4: $4^2 = 16 \le 91$, so test it: 91 % 4 = 3.
d = 4 · d * d = 16 · n % d = 3

Step 4 of 7. d = 5: $5^2 = 25 \le 91$, so test it: 91 % 5 = 1.
d = 5 · d * d = 25 · n % d = 1

Step 5 of 7. d = 6: $6^2 = 36 \le 91$, so test it: 91 % 6 = 1.
d = 6 · d * d = 36 · n % d = 1

Step 6 of 7. d = 7: $7^2 = 49 \le 91$, so test it: 91 % 7 = 0 → 7 divides 91.
d = 7 · d * d = 49 · n % d = 0

Step 7 of 7. 91 = 7 × 13.
smallest divisor = 7 · factorisation = 7 × 13

> **RESULT**
> 91 is composite: 7 × 13 (found after 6 tests)

Complexity

Trial division: O(√n). gcd: O(log min(a, b)). powMod: O(log b). Listing all divisors by the √n trick: O(√n).

Common mistakes

Looping to n instead of √n

Problem

```
for (int d = 2; d < n; d++)
```

Why: For n = 10^9 that is a billion iterations; √n is 31 623.
Fix: for (long d = 2; d * d <= n; d++)

Overflow in the modular product

Problem

```
int r = a * b % m; // a, b up to 10^9
```

Why: a * b overflows int before % is applied.
Fix: long r = (long) a * b % m;

Dividing under a modulus

Problem

```
(a / b) % m
```

Why: Modular division is not ordinary division; you need the modular inverse.
Fix: a * inverse(b) % m (see the Euclid and CRT lessons)

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · gcd and lcm — *Beginner*

Read a and b (up to 10^9). Print gcd and lcm separated by a space.

Example input	Expected output
12 18	6 36

Hint 1: lcm = a / gcd * b with long.

Exercise 2 · All divisors — *Intermediate*

Read n (up to 10^{12}) and print all divisors in increasing order.

Example input	Expected output
36	1 2 3 4 6 9 12 18 36

Hint 1: For each d ≤ √n add both d and n/d to a sorted set.

Exercise 3 · Modular power — *Challenge*

Read a, b, m (a, m ≤ 10^9, b ≤ 10^{18}). Print a^b mod m.

Example input	Expected output
3 200 13	9

Hint 1: Repeated squaring; use long and reduce after every multiplication.

Quick check

Answers are in the answer key at the end of this part.

Q1. gcd(18, 24) × lcm(18, 24) = ?

- **A.** 42
- **B.** 432
- **C.** 6
- **D.** 72

Q2. What does Math.floorMod(-7, 3) return?

```
System.out.println(Math.floorMod(-7, 3));
```

Your answer: _____

Q3. To test whether n is prime by trial division you need to test divisors up to...

- **A.** n
- **B.** n/2
- **C.** √n
- **D.** log n

Q4. What is (17 * 23) % 5?

```
System.out.println((17 % 5) * (23 % 5) % 5);
```

Your answer: _____

Interview / exam questions

- How do you test primality in O(√n)?
- Relation between gcd and lcm?
- Why can you reduce mod m at every step?
- How does fast exponentiation work?

Watch & read

Video: Prime Factorization to find GCD and LCM of two numbers! (So easy!) — Quoc Dat Phung · 5:21. *Divisibility, GCD/LCM, primes and modulus*

Video: L3. Check if a Number if Prime or not | Maths Playlist — take U forward · 7:10. *Checking primes in O(√n)*

REVISION CARD (30–60 SECONDS)

- Divisors pair (d, n/d): only look up to √n.
- gcd·lcm = a·b; coprime ⇔ gcd = 1.
- Modular + − × are safe to reduce at every step; use floorMod.
- powMod: square, multiply when exponent bit is 1.

2.6 Recursion: Base Case, Recursive Case and the Call Stack

Core Programming Skills · Intermediate · Core programming & data structures · about 50 min · ★ on the minimum-viable path

Solve a problem by solving a smaller copy of it — and see exactly what the call stack does.

YOU WILL BE ABLE TO

- Write a recursive method with a base case and a recursive case that moves toward it.
- Trace the call stack and the return values of a recursive call.
- Draw a recursion tree and count calls; compare linear and branching recursion (factorial vs Fibonacci).
- Recognise when recursion is a good fit and when it risks StackOverflowError.

Before this lesson: Methods: Parameters, Return Values, Scope and Overloading · Tracing and Debugging Programs

The idea

A method is **recursive** if it calls itself. Every correct recursive method has two parts:

1. **Base case** — a tiny input whose answer is immediate (so the recursion stops).
2. **Recursive case** — reduce the problem to a **smaller** instance of itself and combine the result.

```
factorial(4)
 = 4 * factorial(3)
 = 3 * factorial(2)
 = 2 * factorial(1)
 = 1 ← base case
 ↑ the answers come back up: 2, 6, 24
```

WHY THIS WORKS

Each call gets its own **stack frame** holding its parameters and local variables. Calls pile up on the **call stack** until a base case returns; then the frames unwind one by one, each combining the value that came back. This is why recursion depth costs memory: factorial(n) needs n frames (O(n) space), and a missing base case ends in `StackOverflowError`.

The shape of the **recursion tree** tells you the cost. `factorial` makes one call per level (a chain, n calls). `fib(n) = fib(n−1) + fib(n−2)` makes two calls per level: the tree doubles, so the number of calls grows like 2^n — fib(20) takes 21 891 calls — although the stack depth is only n. Branching recursion with repeated sub-problems is the cue for **memoisation / dynamic programming**.

pattern	example	calls	depth
linear	factorial(n), sumDigits(n)	n	n
halving	power(b, n/2), binary search	log n	log n
two branches, overlapping	fib(n), grid paths	~2^n	n
divide & conquer	merge sort, quick sort	n	log n

Syntax

The template

```
returnType name(parameters) {
 if (/* smallest case */) {
 return /* answer without recursing */; // 1. BASE CASE: stops the recursion
 }
 // 2. RECURSIVE CASE: call yourself on a SMALLER input
 return /* combine */ name(/* smaller */);
}
```

shape	example	calls
linear	`f(n) = n * f(n - 1)`	1 per level
halving	`f(n) = f(n / 2) + 1`	1 per level, log n levels
two-way	`f(n) = f(n - 1) + f(n - 2)`	2 per level (exponential)
accumulating	`sum(a, i) = a[i] + sum(a, i + 1)`	extra parameter i tracks progress

A tiny example

```
static long factorial(int n) {
 if (n <= 1) return 1; // base case
 return n * factorial(n - 1); // recursive case
}
```

Guided example

RecursionDemo.java — Recursion toolbox (demonstration)

```
1   public class RecursionDemo {
2    static int depth = 0;
3
4    // factorial: n! = n * (n-1)! with 0! = 1 (base case)
5    static long factorial(int n) {
6    if (n <= 1) {                                                    (1)
7    return 1; // BASE CASE: stops the recursion
8    }
9    return n * factorial(n - 1); // RECURSIVE CASE: smaller problem  (2)
10   }
11
12   // see the calls and returns
13   static int sumTo(int n) {
14   String indent = " ".repeat(depth);                               (3)
15   System.out.println(indent + "sumTo(" + n + ")");
16   if (n == 0) {
17   System.out.println(indent + "-> 0");
18   return 0;
19   }
20   depth++;
21   int inner = sumTo(n - 1);
22   depth--;
23   System.out.println(indent + "-> " + (n + inner));
24   return n + inner;
25   }
26
```

(code continued)

```
27    // two recursive calls: the tree of calls grows exponentially
28    static int fib(int n) {
29    if (n <= 1) return n;
30    return fib(n - 1) + fib(n - 2);                                    4
31    }
32
33    static long fibCalls = 0;
34    static int fibCounted(int n) {
35    fibCalls++;
36    if (n <= 1) return n;
37    return fibCounted(n - 1) + fibCounted(n - 2);
38    }
39
40    // fast power: halve the exponent each time
41    static long power(long base, int exp) {
42    if (exp == 0) return 1;
43    long half = power(base, exp / 2);                                  5
44    return exp % 2 == 0 ? half * half : half * half * base;
45    }
46
47    // recursion on a String
48    static boolean isPalindrome(String s) {
49    if (s.length() <= 1) return true;
50    if (s.charAt(0) != s.charAt(s.length() - 1)) return false;
51    return isPalindrome(s.substring(1, s.length() - 1));               6
52    }
53
54    public static void main(String[] args) {
55    System.out.println("5! = " + factorial(5));
56    System.out.println("sumTo(3) trace:");
57    sumTo(3);
58    System.out.println("fib(10) = " + fib(10));
59    fibCounted(20);
60    System.out.println("fib(20) needed " + fibCalls + " calls");
61    System.out.println("2^30 = " + power(2, 30));
62    System.out.println("racecar palindrome? " + isPalindrome("racecar") + ", level? " +
          isPalindrome("leaves"));
63    }
64  }
```

Run it to see the call tree printed with indentation.

Line-by-line walkthrough

#	Where	What happens
1	line 6	Base case: no further call. Without it: infinite recursion → StackOverflowError.
2	line 9	The argument shrinks every call, so the base case is eventually reached.
3	line 14	depth mirrors the stack depth so the trace prints as a staircase.
4	line 30	Two calls: the tree of calls doubles.
5	line 43	One call on half the exponent: depth $\log_2$ exp. Square it (and multiply by base if exp is odd).
6	line 51	Strip both ends and ask the same question of the middle.

Dry run: watch it execute

The call stack grows to the base case and then unwinds.

Algorithm: Recursion call stack: factorial(n). **Input:** n = `4`.

Pseudocode

```
fact(n):
 if n <= 1: return 1 // base case
 return n * fact(n - 1) // recursive case
```

Step 1 of 8. Call fact(4); stack depth is 1.

n = 4 · depth = 1



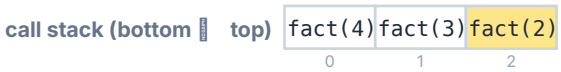
Step 2 of 8. Call fact(3); stack depth is 2.

n = 3 · depth = 2



Step 3 of 8. Call fact(2); stack depth is 3.

n = 2 · depth = 3



Step 4 of 8. Call fact(1); stack depth is 4.

n = 1 · depth = 4



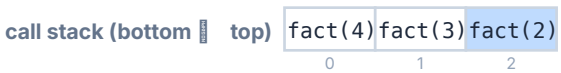
Step 5 of 8. Base case reached: return 1 without recursing.

n = 1 · return = 1



Step 6 of 8. fact(1) returned 1, so fact(2) returns 2 × 1 = 2.

n = 2 · return = 2



Step 7 of 8. fact(2) returned 2, so fact(3) returns 3 × 2 = 6.

n = 3 · return = 6



Step 8 of 8. fact(3) returned 6, so fact(4) returns 4 × 6 = 24.

n = 4 · return = 24



> **RESULT**
> 4! = 24

Complexity

function	time	stack space
factorial(n)	O(n)	O(n)
fib(n) naive	$O(2^n)$ ($\approx 1.6^n$)	O(n)
power(b, n) by halving	O(log n)	O(log n)
isPalindrome(s)	$O(n^2)$ with substring copies	O(n)

Space for recursion = **maximum depth of the call stack** × size of a frame — not the total number of calls. That is the slide's own answer: `int add(int n)` recursing n times uses O(n) space even though it is O(n) time too.

Common mistakes

No base case / not moving toward it

Problem

```
static int f(int n) { return n * f(n - 1); } // never stops
```

Why: StackOverflowError: the stack fills with frames.

Fix: if (n <= 1) return 1;

Recomputing the same sub-problems

Problem

```
fib(50) with the plain recursive version
```

Why: Billions of calls: it recomputes fib(48), fib(47)... repeatedly.

Fix: Memoise in an array, or use a loop.

Forgetting to use the returned value

Problem

```
factorial(n - 1);
return n;
```

Why: The recursive result is thrown away.

Fix: return n * factorial(n - 1);

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Recursive digit sum — *Beginner*

Read n ≥ 0 and print the sum of its digits using recursion (no loops).

Example input	Expected output
4821	15

Hint 1: sumDigits(n) = n % 10 + sumDigits(n / 10); base case n < 10.

Exercise 2 · Towers of Hanoi — *Intermediate*

Move n disks from rod A to rod C using B. Print each move as `Move disk k from X to Y`, then `total moves: M`.

- 1 ≤ n ≤ 10

Example input	Expected output
2	`Move disk 1 from A to B ⏎ Move disk 2 from A to C ⏎ Move disk 1 from B to C ⏎ total moves: 3`

Hint 1: Move n−1 disks to the spare rod, move the biggest, move n−1 on top of it.

Exercise 3 · All subsets — *Challenge*

Read n (1 ≤ n ≤ 10). Print all subsets of {1..n} in the order produced by "skip i" before "take i" (see example).

Example input	Expected output
2	`[] ⏎ [2] ⏎ [1] ⏎ [1, 2]`

Hint 1: subsets(i): skip i then take i, undo after taking.

Quick check

Answers are in the answer key at the end of this part.

Q1. What does f(4) return?

```
static int f(int n) {
 if (n == 0) return 0;
 return n + f(n - 1);
}
```

Your answer: _____

Q2. What happens if a recursive method has no reachable base case?

- **A.** returns 0
- **B.** StackOverflowError
- **C.** compile error
- **D.** nothing

Q3. How many calls does naive fib(5) make (counting fib(5) itself)?

- **A.** 5
- **B.** 9
- **C.** 15
- **D.** 25

Q4. The SPACE used by add(n) = n + add(n−1) is...

- **A.** O(1)
- **B.** O(n)
- **C.** O(n^2)
- **D.** O(log n)

Q5. Recursion tree with overlapping sub-problems suggests...

- **A.** sorting
- **B.** memoisation / dynamic programming
- **C.** binary search
- **D.** a bigger stack

Interview / exam questions

- What is the call stack?
- Recursion vs iteration — when each?
- How do you convert recursion to a loop?
- Why is naive Fibonacci exponential?

Watch & read

Video: Recursion in Java Full Tutorial - How to Create Recursive Methods — Coding with John · 11:11. *Base case, recursive case and the call stack*

Video: Fibonacci Sequence - Anatomy of recursion and space complexity analysis — mycodeschool · 9:39. *Watching the call stack during recursion*

REVISION CARD (30–60 SECONDS)

- Base case + smaller recursive case.
- Stack frame per call; space = depth.
- Chain: n calls. Two branches: ~2^n. Halving: log n.
- Overlap → memoise.

2.7 References, Pass-by-Value and Memory

Core Programming Skills · Intermediate · Core programming & data structures · about 35 min · ★ on the minimum-viable path

What a variable really holds, why arrays and objects can be changed by a method, and what null means.

YOU WILL BE ABLE TO

- Distinguish primitive variables (hold values) from reference variables (hold addresses).
- Explain "Java is always pass-by-value" including for objects.
- Predict aliasing effects and make shallow copies and independent copies.
- Understand `null`, `NullPointerException` and the stack/heap split.

Before this lesson: One-Dimensional Arrays · Methods: Parameters, Return Values, Scope and Overloading

The idea

A **primitive** variable stores the value itself. A **reference** variable (arrays, Strings, any object) stores the **address** of an object that lives on the heap. Assigning a reference copies the address — not the object.

```
int[] a = {1, 2, 3}; stack heap
int[] b = a; a ──────────┐
 b ──────────┼──► [1, 2, 3]
b[0] = 9; → a[0] is 9 too: one array, two names
```

WHY THIS WORKS

Java is strictly pass-by-value. When you call `f(x)`, the *value of x* is copied into the parameter. For a primitive that is the number; for an object reference it is the address. The method can therefore **follow the copied address and change the object** (`arr[0] = 100` is visible to the caller), but it cannot make the caller's variable point somewhere else (`arr = new int[3]` only changes the local copy). Strings are immutable, so there is nothing to change at all.

- `null` means "this reference points to no object". Calling a method on it throws **NullPointerException**.
- `==` on references asks "same object?"; `.equals()` asks "same content?" (for classes that define it).
- Copying: `int[] c = a.clone();` or `Arrays.copyOf(a, a.length)`; for lists `new ArrayList<>(list)`. Nested objects need a **deep copy**.

Syntax

```
int[] a = {1, 2, 3};
int[] b = a; // ALIAS: same array object
int[] c = a.clone(); // independent copy (shallow)
int[] d = Arrays.copyOf(a, a.length);
String s = null; // a reference that points to nothing
if (s != null && s.length() > 0) { /* safe to use s */ } // guard before dereferencing
boolean same = (a == b); // true: the same object
boolean eq1 = a.equals(c); // false: arrays compare by identity
boolean eq2 = Arrays.equals(a, c); // true: same contents
```

kind	variable holds	passed to a method as
int, double, char, boolean ...	the value itself	a copy of the value
arrays, objects, Strings	a reference (address)	a copy of the reference — same object

A tiny example

```
Box b1 = new Box(1);
Box b2 = b1; // alias
b2.value = 42; // b1.value is 42 as well
```

Guided example

ReferencesDemo.java — References and parameters (demonstration)

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class ReferencesDemo {
 static void changePrimitive(int x) {
 x = 100; // changes only the local copy
 }

 static void changeArrayContent(int[] arr) {
 arr[0] = 100; // follows the reference: modifies the SHARED array
 }

 static void reassignReference(int[] arr) {
 arr = new int[] {7, 7, 7}; // only the local reference now points elsewhere
 }

 static void changeString(String s) {
 s = s + "!"; // Strings are immutable: a NEW String, caller unaffected
 }

 static class Box {
 int value;
 Box(int value) { this.value = value; }
 }

 public static void main(String[] args) {
 int n = 5;
 changePrimitive(n);
 System.out.println("primitive: " + n); // 5

 int[] a = {1, 2, 3};
 changeArrayContent(a);
 System.out.println("array content changed: " + Arrays.toString(a)); // [100, 2, 3]
 reassignReference(a);
 System.out.println("after reassignment : " + Arrays.toString(a)); // still [100, 2, 3]

 String s = "hi";
 changeString(s);
 System.out.println("string: " + s); // hi

 // aliases: two variables, one object
```

(code continued)

```
43    Box b1 = new Box(1);
44    Box b2 = b1; // copies the REFERENCE, not the object                    3
45    b2.value = 42;
46    System.out.println("b1.value = " + b1.value + " same object? " + (b1 == b2));
47
48    // copy vs alias for lists
49    List<Integer> original = new ArrayList<>(List.of(1, 2, 3));
50    List<Integer> alias = original;
51    List<Integer> copy = new ArrayList<>(original);                          4
52    original.add(4);
53    System.out.println("alias " + alias + " copy " + copy);
54
55    // null: a reference that points to nothing
56    String missing = null;
57    System.out.println("null check: " + (missing == null));
58    try {
59    System.out.println(missing.length());                                    5
60    } catch (NullPointerException e) {
61    System.out.println("NullPointerException caught");
62    }
63
64    // arrays of objects hold references
65    Box[] boxes = new Box[2];
66    System.out.println("before filling: " + boxes[0]);
67    boxes[0] = new Box(9);
68    System.out.println("after filling : " + boxes[0].value);
69    }
70   }
```

Each static helper isolates one rule.

Line-by-line walkthrough

#	Where	What happens
1	line 10	arr is a COPY of the reference, but it points at the caller's array: changing elements is visible.
2	line 14	Re-pointing the local copy does not affect the caller: still [100, 2, 3].
3	line 44	Copying the reference, not the object: both names refer to one Box.
4	line 51	A genuinely independent list (a shallow copy of the elements).
5	line 59	Dereferencing null throws NullPointerException; the try/catch shows the exception type.

Dry run: watch it execute

The int is copied; the array reference is copied, so the object is shared.

Algorithm: Pass-by-value with an int and an array. **Input:** n = `5`, first = `1`.

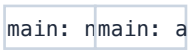
Pseudocode

```
static void f(int x, int[] p) {
 x = 99; // changes the copy
 p[0] = 99; // changes the shared object
 p = new int[]{-1}; // re-points the local copy only
}
f(n, a);
```

Step 1 of 6. main has n = 5 and a → #1 {1, 2}.

n = 5 · a = → #1

variables



Step 2 of 6. Call f(n, a): x gets a COPY of 5; p gets a COPY of the reference, so p also points to #1.

x = 5 · p = → #1

variables



Step 3 of 6. x = 99 changes only the copy inside f; main's n is untouched.

x = 99 · main n = 5

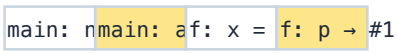
variables



Step 4 of 6. p[0] = 99 follows the reference to the one shared object #1, so main's a now reads {99, 2}.

object #1 = {99, 2}

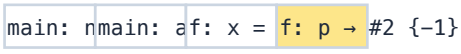
variables



Step 5 of 6. p = new int[]{-1} makes the LOCAL reference point to a new object #2. main's a is not affected.

p = → #2 · main a = → #1

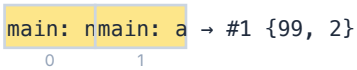
variables



Step 6 of 6. f returns and its variables vanish. main still has n = 5 and a = {99, 2}.

n = 5 · a = {99, 2}

variables



> **RESULT**
> n stays 5; a[0] became 99 (the object was shared)

Complexity

Assigning a reference is O(1); copying an array or list of n items is O(n). Memory: each object has a small header plus its fields; a reference itself is 4–8 bytes.

Common mistakes

Expecting a method to swap two int variables

Problem

```
static void swap(int a, int b) { int t = a; a = b; b = t; }
```

Why: The method swaps its own copies.

Fix: Swap elements inside an array, or return the new values.

Forgetting that arrays default to null/0

Problem

```
String[] names = new String[3];
names[0].length();
```

Why: The array holds three nulls: NullPointerException.

Fix: Assign each element before using it.

Shallow copy surprises

Problem

```
int[][] b = a.clone();
```

Why: The rows are shared.

Fix: Clone every row.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Swap inside an array — *Beginner*

Read n, the array and two indices i and j. Print the array after swapping a[i] and a[j] using a method `swap(int[] a, int i, int j)`.

Example input	Expected output
4 ⏎ 1 2 3 4 ⏎ 0 3	[4, 2, 3, 1]

Hint 1: The method changes the shared array, so it works.

Exercise 2 · Return a new array — *Intermediate*

Read n and an array. Print the original and a NEW array with every element doubled, as `[orig] [doubled]`.

Example input	Expected output
3 ⏎ 1 2 3	[1, 2, 3] [2, 4, 6]

Hint 1: Allocate a second array; do not modify the first.

Exercise 3 · Alias or copy? — *Beginner*

Read mode 1 or 2. Mode 1: b = a; mode 2: b = a.clone(). Then set b[0] = 99 and print `same` if a[0] also became 99, else `different`.

Example input	Expected output
1	same

Hint 1: clone() creates a new array.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
static void f(int[] a) { a[0] = 7; a = new int[]{9}; }
public static void main(String[] x) {
 int[] v = {1, 2};
 f(v);
 System.out.println(v[0]);
}
```

Your answer: _____

Q2. Java passes arguments...

- **A.** by reference
- **B.** by value (references are copied)
- **C.** by name

- **D.** by pointer arithmetic

Q3. Calling a method on a null reference throws...

- **A.** IllegalArgumentException
- **B.** NullPointerException
- **C.** ArithmeticException
- **D.** ClassCastException

Q4. Where do objects live?

- **A.** stack
- **B.** heap
- **C.** registers
- **D.** constant pool only

Interview / exam questions

- Is Java pass-by-value or pass-by-reference?
- What is aliasing?
- Shallow vs deep copy?
- What is a NullPointerException and how do you avoid it?

Watch & read

Video: Java is ALWAYS Pass By Value. Here's Why — Coding with John · 5:22. *Pass-by-value, references and objects*

Video: Null Pointer Exceptions In Java - What EXACTLY They Are and How to Fix Them — Coding with John · 14:20. *References, null and NullPointerException*

REVISION CARD (30–60 SECONDS)

• Primitive variable = value; reference variable = address of a heap object.

• Pass-by-value: the reference is copied; the object can be modified, the caller's variable cannot be re-pointed.

• b = a aliases; clone/copyOf copies; deep copy for nested objects.

• null → NPE.

2.8 Boundary Conditions, Validation and Common Mistakes

Core Programming Skills · Beginner · Core programming & data structures · about 30 min · ★ on the minimum-viable path

The edge cases that break otherwise-correct programs, and the habit of validating input and testing the extremes.

YOU WILL BE ABLE TO

- List the standard boundary conditions for numbers, arrays and strings.
- Validate input with `try/catch` and guard conditions.
- Spot the classic mistakes: off-by-one, overflow, empty input, `Math.abs(MIN_VALUE)`.
- Design a small test list before writing code.

Before this lesson: Tracing and Debugging Programs · One-Dimensional Arrays

The idea

A program is correct only if it is correct **at the edges**. Most real bugs live there. Before coding, write the **boundary cases** next to the problem statement:

input kind	check these
an array / string	empty, one element, all equal, sorted, reverse sorted, maximum length
a number	0, 1, −1, negative, the largest allowed, a prime, a power of two
an index	first, last, one past the end, negative
a loop	zero iterations, one iteration, the final iteration
input from a user	not a number, empty line, too large, extra spaces

WHY THIS WORKS

Validation turns "crashes with a stack trace" into "reports a clear problem": check preconditions (`if (n < 0) …`), convert risky input with `try { Integer.parseInt(s) } catch (NumberFormatException e) { … }`, and never index before checking the bounds. For competition-style problems the input is guaranteed valid — but you must still handle the *valid* extremes (n = 1, n = 10^5, values 10^9).

The usual suspects

- **Off-by-one:** `<` vs `<=`, `length` vs `length - 1`.
- **Overflow:** sums and products of ints; use `long`.
- **Integer division:** `a / b` truncates; `(a + b) / 2` can overflow — use `a + (b - a) / 2`.
- **`Math.abs(Integer.MIN_VALUE)`** is still negative (there is no $+2^{31}$ in an int).
- **Initialising a maximum with 0** when all values may be negative.
- **Using `==` on Strings / boxed Integers.**

Syntax

Defensive patterns

```
if (n < 0) throw new IllegalArgumentException("n must be >= 0"); // reject bad input early
Objects.requireNonNull(name, "name"); // fail fast on null
int total = Math.addExact(a, b); // throws ArithmeticException on overflow
try {
 int v = Integer.parseInt(text); // may throw NumberFormatException
} catch (NumberFormatException e) {
 System.out.println("not a number");
}
for (int i = 0; i < a.length; i++) { /* body */ } // "<" with length; "<=" would read one too far
```

test these inputs	why
empty / length 1	loops that assume a first and last element
0, 1, −1	divisions, signs, base cases
Integer.MAX_VALUE	overflow on +, *
all equal / already sorted / reversed	algorithms with hidden assumptions

A tiny example

```
try {
 int n = Integer.parseInt(token);
} catch (NumberFormatException e) {
 System.out.println("not a number: " + token);
}
```

Guided example

ValidationDemo.java — Defensive code (demonstration)

```
1  import java.util.Scanner;
2
3  public class ValidationDemo {
4   // Parse a token into an int; return empty on any bad input instead of crashing.
5   static java.util.OptionalInt parse(String s) {
6   try {
7   return java.util.OptionalInt.of(Integer.parseInt(s.trim()));                    (1)
8   } catch (NumberFormatException e) {
9   return java.util.OptionalInt.empty();
10  }
11  }
12
13  static String average(int[] a) {
14  if (a == null || a.length == 0) {                                               (2)
15  return "no data"; // an empty array would divide by zero
16  }
17  long sum = 0; // long: the sum of many ints can overflow int                    (3)
18  for (int x : a) sum += x;
19  return String.valueOf((double) sum / a.length);
20  }
21
22  public static void main(String[] args) {
23  Scanner sc = new Scanner(System.in);
24  while (sc.hasNextLine()) {
25  String line = sc.nextLine();
26  System.out.println("\"" + line + "\" -> " + (parse(line).isPresent() ? "number " +
        parse(line).getAsInt() : "invalid"));
27  }
28  System.out.println(average(new int[] {}));
29  System.out.println(average(new int[] {Integer.MAX_VALUE, Integer.MAX_VALUE}));
30
31  // boundary values worth testing: empty, one element, min, max, negative, zero
32  int[] a = {5};
33  System.out.println("last index of a one-element array: " + (a.length - 1));
34  System.out.println("Math.abs(Integer.MIN_VALUE) = " + Math.abs(Integer.MIN_VALUE) + "   (4)
        (still negative!)");
35  }
36 }
```

Feed it lines such as `42`, `abc`, `2147483648`.

Sample input

```
42
 abc
 17
2147483648
-8
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	Throws NumberFormatException for "abc" and for 2147483648 (too large for int).
2	line 14	Guard BEFORE dividing: an empty array would make sum / 0.
3	line 17	The sum of two Integer.MAX_VALUE values only fits in a long.
4	line 34	There is no +2147483648 in an int; the result stays negative.

Dry run: watch it execute

i <= length reads one cell too far.

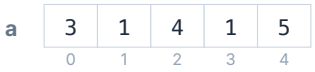
Algorithm: Off-by-one at the array boundary. **Input:** arr = `3,1,4,1,5`, mode = `2`.

Pseudocode

```
int sum = 0;
for (int i = 0; i < a.length; i++) // correct
for (int i = 0; i <= a.length; i++) // BUG
 sum += a[i];
```

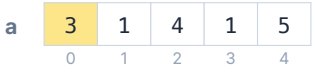
Step 1 of 7. The array has length 5: valid indices are 0 ... 4.

length = 5 · sum = 0



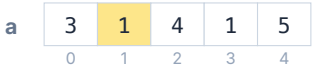
Step 2 of 7. i = 0: sum += a[0] (3) → 3.

i = 0 · a[i] = 3 · sum = 3



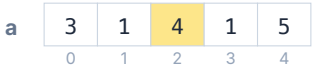
Step 3 of 7. i = 1: sum += a[1] (1) → 4.

i = 1 · a[i] = 1 · sum = 4



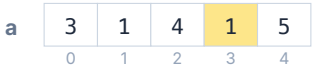
Step 4 of 7. i = 2: sum += a[2] (4) → 8.

i = 2 · a[i] = 4 · sum = 8



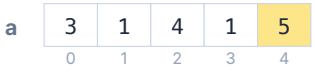
Step 5 of 7. i = 3: sum += a[3] (1) → 9.

i = 3 · a[i] = 1 · sum = 9



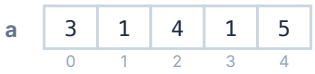
Step 6 of 7. i = 4: sum += a[4] (5) → 14.

i = 4 · a[i] = 5 · sum = 14



Step 7 of 7. i = 5: the condition i <= 5 is still true, so a[5] is evaluated — but there is no such cell. ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5.

i = 5 · length = 5 · sum = 14



> **RESULT**
> crashed with ArrayIndexOutOfBoundsException (index 5, length 5) after summing 14

Complexity

Validation checks are O(1) (or O(length) for scanning a string). They almost never dominate the running time.

Common mistakes

Midpoint overflow

Problem

```
int mid = (lo + hi) / 2; // lo, hi ~ 10^9
```

Why: lo + hi exceeds $2^{31} - 1$ and becomes negative.

Fix: int mid = lo + (hi - lo) / 2;

Assuming at least one element

Problem

```
int first = a[0]; // a may be empty
```

Why: ArrayIndexOutOfBoundsException for n = 0.

Fix: if (a.length == 0) return ...;

Catching everything silently

Problem

```
catch (Exception e) { }
```

Why: Hides real bugs.

Fix: Catch the specific exception and report or handle it.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Sum the valid numbers — *Beginner*

Read whitespace-separated tokens. Print `sum bad` where sum is the total of the tokens that are valid ints and bad is how many are not.

Example input	Expected output
5 x 7 3.5 −2	10 2

Hint 1: try { Integer.parseInt } catch (NumberFormatException e)

Exercise 2 · Maximum or empty — *Beginner*

Read n, then n integers. Print the maximum, or `empty` when n = 0.

Example input	Expected output
`0`	`empty`

Hint 1: Handle n = 0 first.

Exercise 3 · Safe element access — *Intermediate*

Read n, an array and an index i. Print a[i] if 0 ≤ i < n, else `out of range`.

Example input	Expected output
`3 ⏎ 10 20 30 ⏎ 3`	`out of range`

Hint 1: Check both ends of the range.

Quick check

Answers are in the answer key at the end of this part.

Q1. What does this print?

```
System.out.println(Math.abs(Integer.MIN_VALUE) < 0);
```

Your answer: _____

Q2. The safe way to compute the midpoint of lo and hi is...

- **A.** (lo + hi) / 2
- **B.** lo + (hi - lo) / 2
- **C.** hi / 2 + lo
- **D.** lo * hi / 2

Q3. Which input is a boundary case for "find the maximum of an array"?

- **A.** an array of 5 random numbers
- **B.** an empty array
- **C.** an array of 100 numbers
- **D.** an array sorted ascending

Q4. Integer.parseInt("12.5") throws...

- **A.** ArithmeticException
- **B.** NumberFormatException
- **C.** NullPointerException
- **D.** nothing

Interview / exam questions

- What edge cases would you test for a function that reverses a string?
- How do you avoid overflow when averaging?
- When should you validate input?

Watch & read

Video: Error in Arrays:Array index out of bounds exception in java with example — Bhargava Teja Maddula · 2:57. *Off-by-one and index bounds errors*

Video: Boundary Value Testing | Black Testing | Software Engineering with Real Life examples — Gate Smashers · 6:13. *Thinking about edge cases and input validation*

REVISION CARD (30–60 SECONDS)

- Test: empty, one, max, negative, equal, sorted.
- Validate before use; try/catch for parsing.
- mid = lo + (hi − lo) / 2; use long for sums.
- Math.abs(MIN_VALUE) < 0.

Part 2 — Solutions to the exercises

Try the exercises before reading these. Each solution is the exact file that was compiled and tested.

2.1 One-Dimensional Arrays

Exercise 1 · Print reversed

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int i = n - 1; i >= 0; i--) sb.append(i == n - 1 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

Why it works: A downward loop; no need to change the array.

Verified against 3 test cases: `5⏎1 2 3 4 5⏎` → `5 4 3 2 1`; `1⏎7⏎` → `7`; `3⏎-1 0 1⏎` → `1 0 -1`

Exercise 2 · Digit frequencies

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Frequency of each value 0..9 in the array.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] freq = new int[10];
 for (int i = 0; i < n; i++) freq[sc.nextInt()]++;
 for (int d = 0; d < 10; d++) {
 if (freq[d] > 0) System.out.println(d + ": " + freq[d]);
 }
 }
}
```

Why it works: The value itself is the index of its counter.

Verified against 2 test cases: `6⏎3 1 3 0 9 1⏎` → `0: 1⏎1: 2⏎3: 2⏎9: 1`; `1⏎5⏎` → `5: 1`

Exercise 3 · Second largest distinct

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Second largest DISTINCT value, or -1 if there is none. One pass, O(1) space.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long first = Long.MIN_VALUE, second = Long.MIN_VALUE;
 for (int i = 0; i < n; i++) {
 long x = sc.nextLong();
 if (x > first) {
 second = first;
 first = x;
 } else if (x > second && x < first) {
 second = x;
 }
 }
 System.out.println(second == Long.MIN_VALUE ? "-1" : String.valueOf(second));
 }
}
```

Why it works: Two variables updated in one pass; duplicates of the maximum must not become the second.

Verified against 3 test cases: `5⏎4 9 9 2 7⏎` → `7`; `3⏎5 5 5⏎` → `-1`; `2⏎-3 -1⏎` → `-3`

2.2 Two-Dimensional Arrays (Matrices)

Exercise 1 · Sum of a matrix

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 long sum = 0;
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) sum += sc.nextInt();
 System.out.println(sum);
 }
}
```

Why it works: Accumulate while reading.

Verified against 2 test cases: `2 3⏎1 2 3⏎4 5 6⏎` → `21`; `1 1⏎-4⏎` → `-4`

Exercise 2 · Transpose

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Print the transpose of an r x c matrix.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 int[][] m = new int[r][c];
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) m[i][j] = sc.nextInt();
 for (int j = 0; j < c; j++) {
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < r; i++) sb.append(i == 0 ? "" : " ").append(m[i][j]);
 System.out.println(sb);
 }
 }
}
```

Why it works: Print column j of the original as row j.

Verified against 2 test cases: `2 3⏎1 2 3⏎4 5 6⏎` → `1 4⏎2 5⏎3 6`; `1 2⏎7 8⏎` → `7⏎8`

Exercise 3 · Spiral order

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Print an n x n matrix in clockwise spiral order, space separated.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[][] m = new int[n][n];
 for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) m[i][j] = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 int top = 0, bottom = n - 1, left = 0, right = n - 1;
 while (top <= bottom && left <= right) {
 for (int j = left; j <= right; j++) sb.append(m[top][j]).append(' ');
 top++;
 for (int i = top; i <= bottom; i++) sb.append(m[i][right]).append(' ');
 right--;
 if (top <= bottom) {
 for (int j = right; j >= left; j--) sb.append(m[bottom][j]).append(' ');
 bottom--;
 }
 if (left <= right) {
 for (int i = bottom; i >= top; i--) sb.append(m[i][left]).append(' ');
 left++;
 }
 }
 System.out.println(sb.toString().trim());
 }
}
```

Why it works: Peel the outer ring, then recurse inwards by moving the boundaries.

Verified against 3 test cases: `3⏎1 2 3⏎4 5 6⏎7 8 9⏎` → `1 2 3 6 9 8 7 4 5`; `1⏎5⏎` → `5`; `2⏎1 2⏎3 4⏎` → `1 2 4 3`

2.3 Strings, StringBuilder and Characters

Exercise 1 · Count vowels

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.nextLine().toLowerCase();
 int v = 0;
 for (char c : s.toCharArray()) {
 if ("aeiou".indexOf(c) >= 0) v++;
 }
 System.out.println(v);
 }
}
```

Why it works: Lower-case the text and test each char for membership.

Verified against 3 test cases: `Programming Language⏎` → `7`; `rhythm⏎` → `0`; `AEIOU aeiou⏎` → `10`

Exercise 2 · Anagrams

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Are the two words anagrams (same letters, same counts, ignoring case)?
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String a = sc.next().toLowerCase();
 String b = sc.next().toLowerCase();
 int[] count = new int[26];
 for (char c : a.toCharArray()) count[c - 'a']++;
 for (char c : b.toCharArray()) count[c - 'a']--;
 boolean ok = a.length() == b.length();
 for (int x : count) if (x != 0) ok = false;
 System.out.println(ok ? "YES" : "NO");
 }
}
```

Why it works: Equal multisets of letters give all-zero counters.

Verified against 3 test cases: `Listen Silent⏎` → `YES`; `abc abd⏎` → `NO`; `aa a⏎` → `NO`

Exercise 3 · Run-length encoding

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Run-length encode: aaabccdddd -> a3b1c2d4
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 StringBuilder out = new StringBuilder();
 int i = 0;
 while (i < s.length()) {
 int j = i;
 while (j < s.length() && s.charAt(j) == s.charAt(i)) j++;
 out.append(s.charAt(i)).append(j - i);
 i = j;
 }
 System.out.println(out);
 }
}
```

Why it works: Find the end j of the run starting at i, append char and count, continue at j.

Verified against 3 test cases: `aaabccdddd⏎` → `a3b1c2d4`; `a⏎` → `a1`; `abab⏎` → `a1b1a1b1`

2.4 Number Systems, Binary and Two's Complement

Exercise 1 · To binary

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 if (n == 0) {
 System.out.println("0");
 return;
 }
 StringBuilder sb = new StringBuilder();
 while (n > 0) {
 sb.append(n % 2);
 n /= 2;
 }
 System.out.println(sb.reverse());
 }
}
```

Why it works: Remainders are the bits from low to high.

Verified against 4 test cases: `156⏎` → `10011100`; `0⏎` → `0`; `1⏎` → `1`

Exercise 2 · From any base

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 int base = sc.nextInt();
 long value = 0;
 for (char c : s.toCharArray()) value = value * base + Character.digit(c, base);
 System.out.println(value);
 }
}
```

Why it works: Horner's rule over the digits.

Verified against 3 test cases: `9C 16⏎` → `156`; `1010 2⏎` → `10`; `777 8⏎` → `511`

Exercise 3 · Add two binary strings

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Add two binary numbers given as strings (no conversion to int).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String a = sc.next();
 String b = sc.next();
 StringBuilder sb = new StringBuilder();
 int i = a.length() - 1, j = b.length() - 1, carry = 0;
 while (i >= 0 || j >= 0 || carry > 0) {
 int sum = carry;
 if (i >= 0) sum += a.charAt(i--) - '0';
 if (j >= 0) sum += b.charAt(j--) - '0';
 sb.append(sum % 2);
 carry = sum / 2;
 }
 System.out.println(sb.reverse());
 }
}
```

Why it works: Column addition: sum = a + b + carry; digit = sum % 2; carry = sum / 2.

Verified against 4 test cases: `1010 1110⏎` → `11000`; `1 1⏎` → `10`; `0 0⏎` → `0`

2.5 Divisibility, GCD/LCM, Primes and Modular Arithmetic

Exercise 1 · gcd and lcm

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long a = sc.nextLong(), b = sc.nextLong();
 long x = a, y = b;
 while (y != 0) {
 long r = x % y;
 x = y;
 y = r;
 }
 System.out.println(x + " " + (a / x * b));
 }
}
```

Why it works: Euclid for the gcd; divide first to avoid overflow in the lcm.

Verified against 3 test cases: `12 18⏎` → `6 36`; `7 13⏎` → `1 91`; `1000000000 1000000000⏎` → `1000000000 1000000000`

Exercise 2 · All divisors

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Print all divisors of n in increasing order using the sqrt trick (pairs d, n/d).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 java.util.TreeSet<Long> divisors = new java.util.TreeSet<>();
 for (long d = 1; d * d <= n; d++) {
 if (n % d == 0) {
 divisors.add(d);
 divisors.add(n / d);
 }
 }
 StringBuilder sb = new StringBuilder();
 for (long d : divisors) sb.append(sb.length() == 0 ? "" : " ").append(d);
 System.out.println(sb);
 }
}
```

Why it works: Divisors pair up; a TreeSet keeps them sorted and removes the duplicate when d = n/d.

Verified against 3 test cases: `36⏎` → `1 2 3 4 6 9 12 18 36`; `1⏎` → `1`; `97⏎` → `1 97`

Exercise 3 · Modular power

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // a^b mod m with fast exponentiation
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long a = sc.nextLong(), b = sc.nextLong(), m = sc.nextLong();
 long result = 1 % m;
 a %= m;
 while (b > 0) {
 if ((b & 1) == 1) result = result * a % m;
 a = a * a % m;
 b >>= 1;
 }
 System.out.println(result);
 }
}
```

Why it works: O(log b) squarings; each product is below $m^2 \le 10^{18}$ so long suffices.

Verified against 3 test cases: `3 200 13⏎` → `9`; `2 10 1000⏎` → `24`; `5 0 7⏎` → `1`

2.6 Recursion: Base Case, Recursive Case and the Call Stack

Exercise 1 · Recursive digit sum

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static int sumDigits(int n) {
 if (n < 10) return n;
 return n % 10 + sumDigits(n / 10);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 System.out.println(sumDigits(sc.nextInt()));
 }
}
```

Why it works: The last digit plus the digit sum of the rest.

Verified against 3 test cases: `4821⏎` → `15`; `0⏎` → `0`; `999⏎` → `27`

Exercise 2 · Towers of Hanoi

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Towers of Hanoi: print the moves for n disks from rod A to rod C using B.
 static int moves = 0;
 static StringBuilder out = new StringBuilder();

 static void hanoi(int n, char from, char to, char via) {
 if (n == 0) return;
 hanoi(n - 1, from, via, to);
 out.append("Move disk ").append(n).append(" from ").append(from).append(" to ").append(to)
     .append('\n');
 moves++;
 hanoi(n - 1, via, to, from);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 hanoi(n, 'A', 'C', 'B');
 System.out.print(out);
 System.out.println("total moves: " + moves);
 }
}
```

Why it works: T(n) = 2T(n−1) + 1 = 2^n − 1 moves.

Verified against 2 test cases: `2⏎` → `Move disk 1 from A to B⏎Move disk 2 from A to C⏎Move disk 1 from B to C⏎total moves: 3`; `1⏎` → `Move disk 1 from A to C⏎total moves: 1`

Exercise 3 · All subsets

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Print every subset of {1..n} (in the order: without the element first, then with it).
 static StringBuilder out = new StringBuilder();

 static void subsets(int i, int n, java.util.ArrayList<Integer> cur) {
 if (i > n) {
 out.append(cur.toString()).append('\n');
 return;
 }
 subsets(i + 1, n, cur); // skip i
 cur.add(i);
 subsets(i + 1, n, cur); // take i
 cur.remove(cur.size() - 1); // undo
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 subsets(1, sc.nextInt(), new java.util.ArrayList<>());
 System.out.print(out);
 }
}
```

Why it works: Each element is either excluded or included: 2^n leaves.

Verified against 2 test cases: `2⏎` → `[]⏎[2]⏎[1]⏎[1, 2]`; `1⏎` → `[]⏎[1]`

2.7 References, Pass-by-Value and Memory

Exercise 1 · Swap inside an array

Ex1.java

```java
import java.util.Arrays;
import java.util.Scanner;

public class Ex1 {
 // Write a method that swaps a[i] and a[j] IN the array (this works because arrays are objects).
 static void swap(int[] a, int i, int j) {
 int t = a[i];
 a[i] = a[j];
 a[j] = t;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int k = 0; k < n; k++) a[k] = sc.nextInt();
 int i = sc.nextInt(), j = sc.nextInt();
 swap(a, i, j);
 System.out.println(Arrays.toString(a));
 }
}
```

Why it works: The parameter references the caller's array.

Verified against 2 test cases: `4⏎1 2 3 4⏎0 3⏎` → `[4, 2, 3, 1]`; `2⏎5 6⏎0 0⏎` → `[5, 6]`

Exercise 2 · Return a new array

Ex2.java

```java
import java.util.Arrays;
import java.util.Scanner;

public class Ex2 {
 // Return a NEW array with every element doubled; the input array must stay unchanged.
 static int[] doubled(int[] a) {
 int[] r = new int[a.length];
 for (int i = 0; i < a.length; i++) r[i] = a[i] * 2;
 return r;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int k = 0; k < n; k++) a[k] = sc.nextInt();
 int[] d = doubled(a);
 System.out.println(Arrays.toString(a) + " " + Arrays.toString(d));
 }
}
```

Why it works: Creating a new array keeps the caller's data intact.

Verified against 2 test cases: `3⏎1 2 3⏎` → `[1, 2, 3] [2, 4, 6]`; `1⏎0⏎` → `[0] [0]`

Exercise 3 · Alias or copy?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Prints "same" or "different" for two array variables that refer to the same / different
     arrays.
 // Mode 1: b = a (alias). Mode 2: b = a.clone() (copy).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int mode = sc.nextInt();
 int[] a = {1, 2, 3};
 int[] b = mode == 1 ? a : a.clone();
 b[0] = 99;
 System.out.println(a[0] == 99 ? "same" : "different");
 }
}
```

Why it works: An alias shares the object; a clone does not.

Verified against 2 test cases: `1⏎` → `same`; `2⏎` → `different`

2.8 Boundary Conditions, Validation and Common Mistakes

Exercise 1 · Sum the valid numbers

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Read tokens; print the sum of the ones that are valid integers and the count of invalid
     tokens.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long sum = 0;
 int bad = 0;
 while (sc.hasNext()) {
 String t = sc.next();
 try {
 sum += Integer.parseInt(t);
 } catch (NumberFormatException e) {
 bad++;
 }
 }
 System.out.println(sum + " " + bad);
 }
}
```

Why it works: Each token is validated by attempting the conversion.

Verified against 3 test cases: `5 x 7 3.5 -2⏎` → `10 2`; `a b⏎` → `0 2`; `10 20⏎` → `30 0`

Exercise 2 · Maximum or empty

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Largest element, or `empty` when n = 0.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 if (n == 0) {
 System.out.println("empty");
 return;
 }
 int max = Integer.MIN_VALUE;
 for (int i = 0; i < n; i++) max = Math.max(max, sc.nextInt());
 System.out.println(max);
 }
}
```

Why it works: A guard for the empty case; MIN_VALUE as the initial maximum.

Verified against 3 test cases: `0⏎` → `empty`; `3⏎-5 -2 -9⏎` → `-2`; `1⏎7⏎` → `7`

Exercise 3 · Safe element access

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Safe element access: print a[i] if 0 <= i < n else `out of range`.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int k = 0; k < n; k++) a[k] = sc.nextInt();
 int i = sc.nextInt();
 System.out.println(i >= 0 && i < n ? String.valueOf(a[i]) : "out of range");
 }
}
```

Why it works: The bounds check comes before the access.

Verified against 3 test cases: `3⏎10 20 30⏎3⏎` → `out of range`; `3⏎10 20 30⏎0⏎` → `10`; `3⏎10 20 30⏎-1⏎` → `out of range`

Part 2 — Quiz answer key

2.1 One-Dimensional Arrays

	Answer	Why
Q1	5 1	length is 5; index 3 holds 1.
Q2	9	b is an alias of a.
Q3	B. three zeros	Numeric arrays are zero-initialised.
Q4	C. 0 … n−1	Zero-based indexing.

2.2 Two-Dimensional Arrays (Matrices)

	Answer	Why
Q1	3 2 6	3 rows, 2 columns, element in row 2 col 1.
Q2	B. i == j	Row index equals column index.
Q3	B. O(R·C)	Every cell once.
Q4	B. new int[3][]	Only the first dimension is required.

2.3 Strings, StringBuilder and Characters

	Answer	Why
Q1	el2	substring(1,3) = "el"; first l is at index 2.
Q2	false	Different objects.
Q3	B. StringBuilder	It avoids $O(n^2)$ copying.
Q4	3	char arithmetic: 100 − 97.
Q5	B. immutable	Their contents never change after creation.

2.4 Number Systems, Binary and Two's Complement

	Answer	Why
Q1	11	1·8 + 0·4 + 1·2 + 1·1.
Q2	11111101	Flip → 11111100, add 1 → 11111101.
Q3	C. 4	$16 = 2^4$.
Q4	C. −128..127	$-2^7 \ldots 2^7 - 1$.
Q5	B. the least significant bit	It is n mod 2.

2.5 Divisibility, GCD/LCM, Primes and Modular Arithmetic

	Answer	Why
Q1	B. 432	It equals 18 × 24 = 432.
Q2	2	−7 = −3·3 + 2.
Q3	C. √n	A composite has a divisor ≤ √n.
Q4	1	2 · 3 = 6, 6 mod 5 = 1.

2.6 Recursion: Base Case, Recursive Case and the Call Stack

	Answer	Why
Q1	10	4 + 3 + 2 + 1 + 0.
Q2	B. StackOverflowError	Frames pile up until the stack is exhausted.
Q3	C. 15	calls(n) = calls(n−1) + calls(n−2) + 1: 1,1,3,5,9,15.
Q4	B. O(n)	n stack frames at the deepest point.
Q5	B. memoisation / dynamic programming	Cache each sub-answer once.

2.7 References, Pass-by-Value and Memory

	Answer	Why
Q1	7	The element change is seen; the reassignment is not.
Q2	B. by value (references are copied)	Always by value; for objects the value is the reference.
Q3	B. NullPointerException	NullPointerException.
Q4	B. heap	The heap; references to them may be on the stack.

2.8 Boundary Conditions, Validation and Common Mistakes

	Answer	Why
Q1	true	The absolute value overflows back to MIN_VALUE.
Q2	B. lo + (hi - lo) / 2	It cannot overflow when lo ≤ hi.
Q3	B. an empty array	There is no maximum of nothing.
Q4	B. NumberFormatException	It is not a valid int.

Part 3 · Object-Oriented Programming

Classes and objects, encapsulation, inheritance, polymorphism, abstraction, interfaces, static/final, packages, exceptions, generics, the Java Collections Framework and sorting with comparators.

	Lesson	Level	Min	Core
3.1	Classes, Objects, Fields, Constructors and this	beginner	40	
3.2	Encapsulation and Access Modifiers	beginner	30	
3.3	Inheritance, Overriding and Polymorphism	intermediate	45	
3.4	Abstract Classes and Interfaces	intermediate	40	
3.5	static, final and Initialisation Order	intermediate	30	
3.6	Packages and Imports	beginner	20	
3.7	Exceptions: try, catch, finally, throw and throws	intermediate	35	
3.8	Generics and Lists: ArrayList and LinkedList	beginner	40	★
3.9	Sets and Maps: HashSet, TreeSet, HashMap, TreeMap	beginner	40	★
3.10	Queues, Deques, Stacks and PriorityQueue	beginner	35	★
3.11	Sorting with Comparable and Comparator	intermediate	35	★
3.12	equals(), hashCode() and Immutability	intermediate	35	

3.1 Classes, Objects, Fields, Constructors and this

Object-Oriented Programming · Beginner · Essential Java syntax · about 40 min · complete path

Define your own types: a blueprint (class) with data (fields) and behaviour (methods), and objects built from it.

YOU WILL BE ABLE TO

- Declare a class with fields, constructors and methods, and create objects with `new`.
- Explain what `this` refers to and use constructor overloading and `this(...)`.
- Override `toString()` to print an object readably.
- Describe how object variables hold references (aliasing).

Before this lesson: Methods: Parameters, Return Values, Scope and Overloading · References, Pass-by-Value and Memory

The idea

So far all data has been numbers, Strings and arrays. A **class** lets you invent a new kind of thing: it bundles **fields** (what an object *has*) with **methods** (what it can *do*). An **object** is one concrete instance built from the class with `new`.

```
class Student { ← blueprint
 String name; ← fields (state)
 int marks;
 Student(String name, int marks) { this.name = name; this.marks = marks; } ← constructor
 boolean passed() { return marks >= 40; } ← method
}
Student asha = new Student("Asha", 82); ← an object (instance)
```

WHY THIS WORKS

Grouping data and the operations on it removes whole classes of mistakes: a Student always has both a name and marks, and the only code that interprets marks lives next to the data. In algorithm code you will mostly use small classes as **records** (`Point`, `Edge`, `Interval`) and let library classes (`ArrayList`, `HashMap`) do the heavy lifting.

- **Constructor:** same name as the class, no return type; runs on `new` and sets the fields up. If you write none, Java supplies an empty default constructor.
- **`this`:** the object the method/constructor is working on. It disambiguates `this.name = name;` and lets one constructor call another with `this(...)`.
- **Object variables hold references:** `Student b = a;` copies the reference, so both names reach one object (see the references lesson).
- **`toString()`:** called automatically when an object is printed or concatenated to a String — override it to avoid output such as `Student@1b6d3586`.

Syntax

```
public class Student {
 private String name; // fields (the object's state)
 private int age;

 public Student(String name, int age) { // constructor: same name as the class, no return type
 this.name = name; // this.field = parameter
 this.age = age;
 }
 public Student(String name) { this(name, 18); } // constructor chaining

 public String getName() { return name; } // methods (behaviour)
 public void birthday() { age++; }
 @Override public String toString() { return name + " (" + age + ")"; }
}
Student s = new Student("Ana", 20); // create an object
s.birthday(); // call a method
```

A tiny example

```
class Point {
 int x, y;
 Point(int x, int y) { this.x = x; this.y = y; }
}
Point p = new Point(3, 4);
System.out.println(p.x + p.y); // 7
```

Guided example

ClassesDemo.java — A Student class (demonstration)

```
public class ClassesDemo {
 // A CLASS is a blueprint; an OBJECT is one thing built from it.
 static class Student {
 // fields: the data each object carries
 String name;                                                                    (1)
 int marks;

 // constructor: runs when you write `new Student(...)`; same name as the class, no return type
 Student(String name, int marks) {
 this.name = name; // `this` is the object being built; it separates the field from the parameter   (2)
 this.marks = marks;
 }

 // second constructor (overloading), delegating to the first with this(...)
 Student(String name) {
 this(name, 0);                                                                  (3)
 }

 // a method: behaviour that works on the object's own fields
 boolean passed() {                                                              (4)
 return marks >= 40;
 }

 @Override
 public String toString() {                                                      (5)
 return name + " (" + marks + ")";
 }
 }

 public static void main(String[] args) {
 Student asha = new Student("Asha", 82); // an object on the heap
 Student ravi = new Student("Ravi");
 ravi.marks = 35;
 System.out.println(asha + " passed? " + asha.passed());
 System.out.println(ravi + " passed? " + ravi.passed());

 Student alias = asha; // another name for the SAME object                       (6)
 alias.marks = 90;
 System.out.println(asha);

 Student[] roll = {asha, ravi, new Student("Mei", 71)};
 int total = 0;
 for (Student s : roll) {
 total += s.marks;
 }
 System.out.println("average = " + total / roll.length);
 }
}
```

The class is nested as `static class Student` only to keep everything in one file.

Line-by-line walkthrough

#	Where	What happens
1	line 5	Fields: each Student object has its own copy.
2	line 10	`this.name` is the field; plain `name` is the parameter that shadows it.
3	line 16	Constructor chaining: reuse the two-argument constructor.
4	line 20	A method can use the fields of its own object directly.
5	line 25	Overrides Object.toString so printing shows "Asha (82)".
6	line 37	No new object: alias and asha refer to the same Student.

Dry run: watch it execute

new allocates, the constructor initialises, references alias.

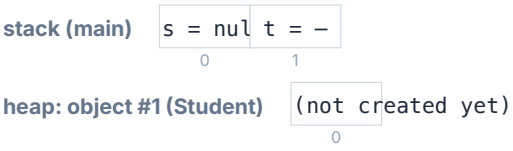
Algorithm: What new Student(...) does. **Input:** name = `Ana`, age = `20`.

Pseudocode

```
Student s;
s = new Student(name, age);
Student t = s;
t.age++;
```

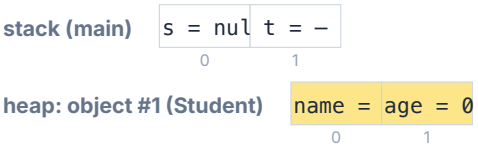
Step 1 of 6. Student s; declares a reference variable. No object exists yet: s is null (it holds no address).

s = null



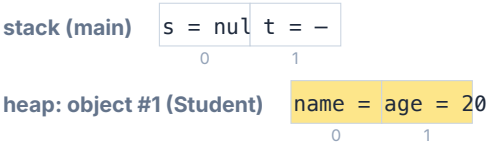
Step 2 of 6. new Student(...) first allocates memory for one Student object on the heap. Every field gets its default value: null for references, 0 for numbers.

name field = null · age field = 0



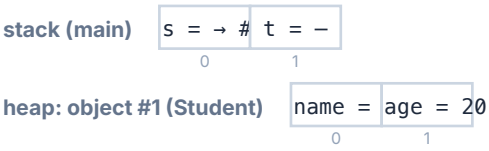
Step 3 of 6. Now the constructor runs. this.name = name and this.age = age copy the arguments into the new object's fields.

this.name = "Ana" · this.age = 20



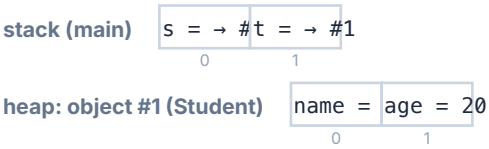
Step 4 of 6. The assignment s = ... stores the ADDRESS of the new object in s.

s = → #1



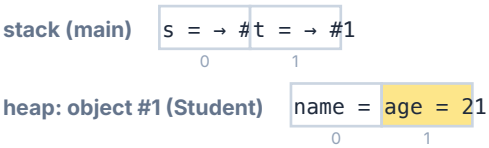
Step 5 of 6. Student t = s copies the reference: t and s are two names for the SAME object. No second Student is created.

s = → #1 · t = → #1



Step 6 of 6. t.age++ follows the reference and changes the one object. Reading s.age now gives 21.

s.age = 21 · t.age = 21



> **RESULT**
> one object, two references; s.age == t.age == 21

Complexity

Creating an object is O(1) plus its constructor work; accessing a field or calling a method is O(1) plus the method's own cost. Memory: the sum of the field sizes plus a small object header (about 12–16 bytes).

Common mistakes

Assigning the parameter to itself

Problem

```
Student(String name, int marks) {
 name = name; // does nothing useful
}
```

Why: Both sides are the parameter; the field stays null.

Fix: this.name = name;

Calling an instance method without an object

Problem

```
public static void main(...) { passed(); }
```

Why: passed() needs an object to work on.

Fix: asha.passed();

Using an object before creating it

Problem

```
Student s;
System.out.println(s.marks);
```

Why: The variable is unassigned / null (NullPointerException).

Fix: Student s = new Student("A", 1);

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Rectangle class — *Beginner*

Create a class Rectangle with width and height, and methods area() and perimeter(). Read width and height (decimals) and print `area perimeter`.

Example input	Expected output
2.5 4	10.0 13.0

Hint 1: A constructor that stores both fields.

Exercise 2 · Counter — *Intermediate*

Create a Counter with increment(), add(k) and get(). Read n operations: `inc` or `add k`. Print the final value.

Example input	Expected output
4 ⏎ inc ⏎ add 5 ⏎ inc ⏎ add 10	17

Hint 1: A private int field.

Exercise 3 · Distance between points — *Intermediate*

Create a Point class with a method distanceTo(Point). Read x1 y1 x2 y2 and print the distance with three decimals.

Example input	Expected output
0 0 3 4	5.000

Hint 1: Math.sqrt(dx*dx + dy*dy)

Quick check

Answers are in the answer key at the end of this part.

Q1. What is the difference between a class and an object?

- **A.** none
- **B.** a class is a blueprint; an object is an instance of it
- **C.** an object is a blueprint
- **D.** a class is created with new

Q2. What is printed?

```
class P { int v; P(int v) { this.v = v; } }
P a = new P(1);
P b = a;
b.v = 5;
System.out.println(a.v);
```

Your answer: _____

Q3. A constructor...

- **A.** has a return type void
- **B.** has the same name as the class and no return type
- **C.** must be static
- **D.** is called manually

Q4. What does `this` refer to inside a method?

- **A.** the class
- **B.** the object the method was called on
- **C.** the parent class
- **D.** the main method

Interview / exam questions

- Class vs object?
- What is a constructor? Can it be overloaded?
- What happens if you do not define a constructor?
- Why override toString()?

Watch & read

Video: This Keyword in Java Full Tutorial - How to Use "this" — Coding with John · 7:39. *Classes, objects, constructors and this*

Video: Java OOPs in One Shot | Object Oriented Programming | Java Language | Placement Course — Apna College · 1:06:27. *Object-oriented basics*

REVISION CARD (30–60 SECONDS)

- class = fields + constructors + methods; object = new Class(...).
- this = current object; this(...) chains constructors.
- Object variables are references; = aliases.
- Override toString for printing.

3.2 Encapsulation and Access Modifiers

Object-Oriented Programming · Beginner · Essential Java syntax · about 30 min · complete path

Hide the data, expose a safe interface: private fields, getters/setters, and the four access levels.

YOU WILL BE ABLE TO

- Make fields private and expose behaviour through public methods.
- Enforce an invariant (a rule that must always hold) inside the class.
- List what `private`, package-private, `protected` and `public` allow.
- Know when a getter/setter pair is appropriate and when it is not.

Before this lesson: Classes, Objects, Fields, Constructors and this

The idea

Encapsulation means keeping an object's data private and letting outsiders touch it only through methods the class controls. The class can then guarantee its **invariants** — for example "a bank balance never goes negative".

modifier	same class	same package	subclass (other package)	everyone
private	✓	✗	✗	✗
(none) package-private	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

WHY THIS WORKS

If `balance` were public, any line of code anywhere could set it to −1000 and you would have to search the whole program to find the culprit. With a private field and a `withdraw` method that refuses overdrafts, **one** method is the only place the rule can be broken — and the only place to fix. Private fields also let you change the internal representation later (cents as `long` instead of dollars as `double`) without touching any caller.

- Start with everything `private`; widen only when needed.
- Provide a getter only if outsiders need to read the value; provide a setter only if arbitrary values are valid (or validate inside it).
- Prefer *meaningful* methods (`deposit`, `withdraw`) to bare setters.
- Mark fields that never change `final`.

Syntax

modifier	same class	same package	subclass (other package)	anywhere
`private`	✓	✗	✗	✗
(none) package-private	✓	✓	✗	✗
`protected`	✓	✓	✓	✗
`public`	✓	✓	✓	✓

Getter / validating setter

```
public class Account {
 private double balance; // hidden state
 public double getBalance() { return balance; } // read access
 public void deposit(double amount) { // controlled change
 if (amount <= 0) throw new IllegalArgumentException("amount must be positive");
 balance += amount;
 }
}
```

A tiny example

```
class Account {
 private long cents;
 void deposit(long c) { if (c > 0) cents += c; }
 long balance() { return cents; }
}
```

Guided example

EncapsulationDemo.java — Protecting an invariant (demonstration)

```
public class EncapsulationDemo {
 // Encapsulation: hide the data (private) and expose a small, safe interface (public
     methods).
 static class BankAccount {
 private final String owner; // never changes                                    (1)
 private long balanceCents; // can only change through deposit/withdraw

 BankAccount(String owner, long openingCents) {
 if (openingCents < 0) {                                                        (2)
 throw new IllegalArgumentException("opening balance cannot be negative");
 }
 this.owner = owner;
 this.balanceCents = openingCents;
 }

 public void deposit(long cents) {
 if (cents <= 0) {
 throw new IllegalArgumentException("deposit must be positive");
 }
 balanceCents += cents;
 }

 public boolean withdraw(long cents) {
 if (cents <= 0 || cents > balanceCents) {                                      (3)
 return false; // refuse instead of breaking the rule "balance >= 0"
 }
 balanceCents -= cents;
 return true;
 }

 public String getOwner() { return owner; }
 public long getBalanceCents() { return balanceCents; }                         (4)
 }

 public static void main(String[] args) {
 BankAccount acct = new BankAccount("Asha", 10_000);
 acct.deposit(2_500);
 System.out.println("withdraw 5000: " + acct.withdraw(5_000));
 System.out.println("withdraw 99999: " + acct.withdraw(99_999));
 System.out.println(acct.getOwner() + " has " + acct.getBalanceCents() / 100.0);
 // acct.balanceCents = -1; // would NOT compile: the field is private
 try {
 acct.deposit(-5);
 } catch (IllegalArgumentException e) {
 System.out.println("rejected: " + e.getMessage());
 }
 }
}
```

All state is private; only deposit/withdraw change it.

Line-by-line walkthrough

#	Where	What happens
1	line 4	private + final: set once in the constructor and never visible for modification.
2	line 8	The constructor validates: an object can never be created in an invalid state.
3	line 23	withdraw refuses rather than break the "balance ≥ 0" rule.
4	line 31	A read-only getter: callers can see the balance but not set it.

Dry run: watch it execute

Change the modifier and the caller to fill the table.

Algorithm: Who may touch a member?. **Input:** modifier = `private`, caller = `same-package`.

Pseudocode

```
class A { <modifier> int x; }
// code in <caller> writes: a.x = 1;
private: only A itself
package: A + classes in A's package
protected: package + subclasses elsewhere
public: everyone
```

Step 1 of 2. The field is declared private. The caller is: same package.

modifier = private · caller = same-package

Step 2 of 2. The table row "private" has a ✗ in column "same-package": the compiler reports an error ("x has private access in A").

allowed = false

	same-c	same-p	subcla	other
privat	✓	✗	✗	✗
packag	✓	✓	✗	✗
protec	✓	✓	✓	✗
public	✓	✓	✓	✓

> **RESULT**
> compile-time error: not accessible

Complexity

Access control is a compile-time check: no run-time cost.

Common mistakes

Public fields

Problem

```
public int balance;
```

Why: Any code can write an invalid value.

Fix: private long balanceCents; with methods

A setter that validates nothing

Problem

```
void setAge(int a) { age = a; }
```

Why: It is just a public field with extra typing.

Fix: if (a < 0) throw new IllegalArgumentException(...)

Returning an internal mutable object

Problem

```
List<String> getItems() { return items; }
```

Why: The caller can modify your list directly.

Fix: return Collections.unmodifiableList(items);

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Validated temperature — *Beginner*

Create a Temperature class whose setCelsius rejects values below −273.15 by throwing IllegalArgumentException. Read a Celsius value; print the Fahrenheit value with one decimal, or `invalid: below absolute zero`.

Example input	Expected output
36.6	97.9

Hint 1: catch the exception in main.

Exercise 2 · Bounded stack — *Intermediate*

Implement an int stack of capacity 3 with push (returns false if full) and pop. Read n commands (`push x` or `pop`); print `ok`/`full` for pushes and the popped value or `empty` for pops.

Example input	Expected output
5 ⏎ push 1 ⏎ push 2 ⏎ pop ⏎ pop ⏎ pop	ok ok 2 1 empty

Hint 1: A private array and a private size.

Exercise 3 · Access levels — *Beginner*

Run the program: it prints the values of four fields declared with private, package-private, protected and public inside a class. (The output `42 7 3 1` shows they are all reachable from inside the same top-level class.)

Example input	Expected output
(no input)	42 7 3 1

Hint 1: Read the lesson table: same-class access is always allowed.

Quick check

Answers are in the answer key at the end of this part.

Q1. Which modifier makes a member visible only inside its own class?

- **A.** public
- **B.** protected
- **C.** private
- **D.** default

Q2. Why make fields private?

- **A.** to make the program faster
- **B.** so the class can enforce its rules and change internals later
- **C.** because Java requires it
- **D.** to save memory

Q3. A member with no modifier is visible...

- **A.** everywhere
- **B.** in the same package
- **C.** only in the class
- **D.** in subclasses only

Q4. Which line breaks encapsulation?

```
1  class Account {
2   public long balance;
3   void deposit(long c) { balance += c; }
4  }
```

Which line contains the bug? _____

Interview / exam questions

- What is encapsulation?
- Difference between protected and default access?
- Why avoid returning internal collections?
- What is an invariant?

Watch & read

Video: Java Tutorial: Access modifiers, getters & setters in Java — CodeWithHarry · 12:56. *Private fields, getters/setters and access modifiers*

Video: Access Specifiers in Java | Public Private Protected Default | Explained with Examples | JAVA Course — Jenny's Lectures CS IT · 38:36. *The four access levels*

REVISION CARD (30–60 SECONDS)

- private fields + public methods.
- private < package < protected < public.
- Validate in constructors/setters.
- Do not leak mutable internals.

3.3 Inheritance, Overriding and Polymorphism

Object-Oriented Programming · Intermediate · Essential Java syntax · about 45 min · complete path

Build new classes from existing ones with `extends`, redefine behaviour with `@Override`, and let one variable type stand for many.

YOU WILL BE ABLE TO

- Write a subclass with `extends`, `super(...)` and `@Override`.
- Explain dynamic dispatch: the method that runs depends on the object's actual type.
- Use a superclass reference to hold subclass objects and `instanceof` to test them.
- Know when composition ("has-a") is better than inheritance ("is-a").

Before this lesson: Encapsulation and Access Modifiers

The idea

Inheritance lets a class (the **subclass**) reuse and extend another (the **superclass**): `class Circle extends Shape`. A Circle *is a* Shape, so it has all of Shape's non-private members and can be used wherever a Shape is expected.

A subclass may **override** a method — provide its own version with the same signature. **Polymorphism** ("many forms"): a variable of the superclass type may refer to any subclass object, and the call `shape.area()` runs the **subclass's** version, chosen at run time.

```
Shape s = new Circle(1); static (declared) type: Shape
s.area(); actual type: Circle → Circle.area() runs

Shape ← Rectangle ← Square a Square is also a Rectangle and a Shape
```

WHY THIS WORKS

Polymorphism lets you write code against the general type (`for (Shape s : shapes) total += s.area();`) and add new shapes later without touching that loop. Constructors run **parent first**: `super(...)` must be the first statement of a subclass constructor (the compiler inserts `super()` if you omit it). `super.method()` calls the parent's version from inside an override.

- Use `@Override` — the compiler then checks that you really override something.
- `final` classes cannot be extended; `final` methods cannot be overridden.
- `instanceof` tests the actual type; casting `(Rectangle) s` is checked at run time (ClassCastException if wrong).
- Prefer **composition** when the relationship is "has-a" (a Car *has* an Engine) or when you only want to reuse code.

Syntax

```
class Animal {
 String name;
 Animal(String name) { this.name = name; }
 void speak() { System.out.println("..."); }
}
class Dog extends Animal { // Dog IS-A Animal
 Dog(String name) { super(name); } // call the parent constructor FIRST
 @Override void speak() { // @Override: compiler checks the signature
 super.speak(); // optionally reuse the parent version
 System.out.println("Woof");
 }
}
Animal a = new Dog("Rex"); // parent-type variable, child object
a.speak(); // Dog's version runs (dynamic dispatch)
if (a instanceof Dog d) { d.name.length(); } // test + cast (Java 16+)
```

- `final class X` cannot be extended; `final void m()` cannot be overridden.
- A class has exactly one superclass (`extends`) but may implement many interfaces.

A tiny example

```
class Animal { String sound() { return "..."; } }
class Dog extends Animal { @Override String sound() { return "Woof"; } }
Animal a = new Dog();
System.out.println(a.sound()); // Woof
```

Guided example

ShapesDemo.java — A shape hierarchy (demonstration)

```
1  import java.util.List;
2
3  public class ShapesDemo {
4   static class Shape {
5   protected final String name;
6
7   Shape(String name) {
8   this.name = name;
9   }
10
11  double area() {
12  return 0;
13  }
14
15  @Override                                                    (2)
16  public String toString() {
17  return name + " area=" + String.format("%.2f", area());
18  }
19  }
20
21  static class Circle extends Shape { // "is-a": a Circle IS a Shape
22  private final double r;
23
24  Circle(double r) {
25  super("Circle"); // call the parent constructor first        (1)
```

(code continued)

```
    this.r = r;
    }

    @Override
    double area() { // OVERRIDING: same signature, new behaviour
    return Math.PI * r * r;
    }
    }

    static class Rectangle extends Shape {
    private final double w, h;

    Rectangle(double w, double h) {
    super("Rectangle");
    this.w = w;
    this.h = h;
    }

    @Override
    double area() {
    return w * h;
    }
    }

    static class Square extends Rectangle {
    Square(double side) {
    super(side, side);
    }

    @Override
    public String toString() {
    return "Square! " + super.toString(); // reuse the parent's version                3
    }
    }

    public static void main(String[] args) {
    // POLYMORPHISM: one variable type (Shape), many actual types; the right area() runs at
        run time.
    List<Shape> shapes = List.of(new Circle(1), new Rectangle(2, 3), new Square(4), new      4
        Shape("Blob"));
    double total = 0;
    for (Shape s : shapes) {
    System.out.println(s);
    total += s.area();
    }
    System.out.printf("total area = %.2f%n", total);

    Shape s = new Square(2);
    System.out.println(s instanceof Rectangle); // true
    System.out.println(s instanceof Circle); // false
    if (s instanceof Rectangle rect) { // pattern matching (Java 16+)                   5
    System.out.println("it is a rectangle with area " + rect.area());
    }
    }
  }
```

The list holds different kinds of shapes; area() is chosen per object.

Line-by-line walkthrough

#	Where	What happens
1	line 25	Call the parent constructor first.
2	line 15	Same signature as Shape.area(); the compiler verifies it.
3	line 57	super.toString() reuses Shape's version.
4	line 63	One list type, four different object types.
5	line 74	Pattern matching: test and cast in one step (Java 16+).

Dry run: watch it execute

The JVM starts at the real class and walks up until it finds speak().

Algorithm: Which speak() runs?. **Input:** cls = `Puppy`.

Pseudocode

```
Animal a = new <Class>();
a.speak();
Animal.speak() → "..."
Dog.speak() → "Woof"
Cat.speak() → "Meow"
Puppy extends Dog (no speak)
```

Step 1 of 5. Compile time: the variable a has the DECLARED type Animal. The compiler only checks that Animal has a method speak(): it does, so the call compiles.

declared type = Animal

Step 2 of 5. Run time: the object created by new Puppy() has the REAL class Puppy.

real class = Puppy

class chain (real class first)

Puppy	Dog	Animal
0	1	2

Step 3 of 5. Puppy does not declare speak(): look in its superclass Dog.

looking in = Puppy · has speak() = false

class chain (real class first)

Puppy	Dog	Animal
0	1	2

Step 4 of 5. Dog declares speak(): this is the version that runs.

looking in = Dog · has speak() = true

class chain (real class first)

Puppy	Dog	Animal
0	1	2

Step 5 of 5. Dynamic dispatch picks Dog.speak() and prints "Woof", although the variable's type is Animal.

output = Woof

> **RESULT**
> prints Woof

Complexity

A virtual method call costs a table lookup — O(1) and nearly free in practice (the JIT often inlines it).

Common mistakes

Overloading instead of overriding

Problem

```
class Dog extends Animal {
 String sound(int times) { ... } // different parameters!
}
```

Why: This is a new method, not an override; `animal.sound()` still returns "...". @Override would have flagged it.

Fix: @Override String sound() { ... }

Forgetting super(...) when the parent has no default constructor

Problem

```
class Child extends Parent {
 Child() { } // Parent(int) only
}
```

Why: The compiler cannot insert super() because Parent has no no-argument constructor.

Fix: Child() { super(5); }

Calling a subclass-only method through the parent type

Problem

```
Shape s = new Circle(1);
s.radius();
```

Why: The static type Shape has no radius().

Fix: ((Circle) s).radius(); // after an instanceof test

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Animal sounds — *Beginner*

Create Animal (sound "..."), Dog ("Woof") and Cat ("Meow"). Read n kinds (`dog`, `cat` or anything else = plain animal) and print each sound separated by spaces.

Example input	Expected output
3 ⏎ dog ⏎ cat ⏎ fish	Woof Meow ...

Hint 1: Store each as an Animal reference and call sound().

Exercise 2 · Payroll — *Intermediate*

Employee pays base; Manager pays 1.5 × base. Read n lines `role name base` (role manager or staff) and print the total pay with two decimals.

Example input	Expected output
2 ⏎ manager Ann 1000 ⏎ staff Bob 800	2300.00

Hint 1: Override pay() in Manager.

Exercise 3 · Predict the order — *Beginner*

Run the program and write down the output: the constructors of a parent and child class, and a call through a parent-typed variable.

Example input	Expected output
(no input)	A() ⏎ B() ⏎ B

Hint 1: Parent constructor first; the object is a B.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
class A { String f() { return "A"; } }
class B extends A { String f() { return "B"; } }
A x = new B();
System.out.println(x.f());
```

Your answer: _____

Q2. Which keyword calls the parent constructor?

- **A.** this
- **B.** super
- **C.** parent
- **D.** base

Q3. Which relationship suggests composition rather than inheritance?

- **A.** Dog is an Animal
- **B.** Car has an Engine
- **C.** Circle is a Shape
- **D.** Manager is an Employee

Q4. In the parent-child constructor chain, which runs first?

- **A.** child body
- **B.** parent constructor
- **C.** static main
- **D.** neither

Interview / exam questions

- Overloading vs overriding?
- What is polymorphism?
- Can you override a static method? a private method?
- Why prefer composition?

Watch & read

Video: #6.4 Java Tutorial | Method Overriding | Super Keyword — Telusko · 4:59. *Inheritance, overriding and polymorphism*

Video: #56 Dynamic Method Dispatch in Java — Telusko · 8:04. *Dynamic dispatch and upcasting*

REVISION CARD (30–60 SECONDS)

- extends, super(...), @Override.
- Dynamic dispatch: actual type decides.
- instanceof + cast.
- is-a → inheritance; has-a → composition.

3.4 Abstract Classes and Interfaces

Object-Oriented Programming · Intermediate · Essential Java syntax · about 40 min · complete path

Define what something must do without saying how: abstract methods, interfaces, default methods and Comparable.

YOU WILL BE ABLE TO

- Declare an abstract class with abstract methods and extend it.
- Declare and implement an interface, including default methods.
- Choose between an abstract class and an interface.
- Implement `Comparable<T>` to give your class a natural order.

Before this lesson: Inheritance, Overriding and Polymorphism

The idea

An **abstract class** is a partly finished class: it can hold fields and real methods, but also **abstract methods** (a signature with no body) that every concrete subclass must supply. You cannot create an object of an abstract class.

An **interface** is a pure contract: a list of methods a type promises to have. A class can **implement many interfaces** but extend only one class. Interface methods are `public abstract` by default; since Java 8 they may also have **default** (with a body) and **static** methods.

	abstract class	interface
state (fields)	yes	only constants
constructors	yes	no
multiple per class	no (one superclass)	yes
use when	subclasses share code/state	unrelated classes share a capability

WHY THIS WORKS

Programming against an interface decouples **what** from **how**: a method taking `List<Integer>` works with an `ArrayList` or a `LinkedList`; a sort that needs `Comparable` works for any class that implements it. This is the single most important design idea for reusable code, and it is how the Java Collections Framework is organised.

Syntax

```
interface Payable { double amountDue(); default String describe() { return "pay " + amountDue();
    } }
abstract class Employee implements Payable { abstract double monthlyPay(); public double
    amountDue() { return monthlyPay(); } }
class Hourly extends Employee { double monthlyPay() { return 25 * 80; } }
```

A tiny example

```
interface Shape { double area(); }
class Square implements Shape {
 double s;
 public double area() { return s * s; } // interface methods are public
}
```

Guided example

AbstractionDemo.java — Payable and Employee (demonstration)

```
1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class AbstractionDemo {
5   // An INTERFACE is a contract: what a type can do, with no state.
6   interface Payable {                                                    (1)
7   double amountDue();
8
9   default String describe() { // default method (Java 8+)                (2)
10  return "pay " + String.format("%.2f", amountDue());
11  }
12  }
13
14  // An ABSTRACT CLASS is a partly finished class: shared state and code, plus abstract
        holes to fill in.
15  static abstract class Employee implements Payable {                    (3)
16  protected final String name;
17
18  Employee(String name) {
19  this.name = name;
20  }
21
22  abstract double monthlyPay(); // must be implemented by subclasses      (4)
23
24  @Override
25  public double amountDue() {
26  return monthlyPay();
27  }
28  }
29
30  static class Salaried extends Employee {
31  private final double yearly;
32
33  Salaried(String name, double yearly) {
34  super(name);
35  this.yearly = yearly;
36  }
37
38  @Override
39  double monthlyPay() {
40  return yearly / 12;
41  }
42  }
43
44  static class Hourly extends Employee {
45  private final double rate;
```

(code continued)

```
    private final int hours;

    Hourly(String name, double rate, int hours) {
    super(name);
    this.rate = rate;
    this.hours = hours;
    }

    @Override
    double monthlyPay() {
    return rate * hours;
    }
    }

    static class Invoice implements Payable { // unrelated to Employee, still Payable
    private final double total;

    Invoice(double total) {
    this.total = total;
    }

    @Override
    public double amountDue() {
    return total;
    }
    }

    public static void main(String[] args) {
    List<Payable> bills = new ArrayList<>();
    bills.add(new Salaried("Asha", 120_000));
    bills.add(new Hourly("Ravi", 25.0, 80));
    bills.add(new Invoice(499.99));
    double sum = 0;
    for (Payable p : bills) {
    System.out.println(p.describe());
    sum += p.amountDue();
    }
    System.out.printf("total due: %.2f%n", sum);
    // new Employee("x"); // would NOT compile: abstract classes cannot be instantiated
    }
   }
```

5

One list holds employees and an invoice because they share the Payable contract.

Line-by-line walkthrough

#	Where	What happens
1	line 6	The contract: anything Payable can report an amount due.
2	line 9	A default method gives implementers free behaviour that they may override.
3	line 15	An abstract class may implement an interface and leave some methods for subclasses.
4	line 22	No body: each concrete subclass MUST implement it.
5	line 60	Unrelated to Employee yet usable wherever Payable is expected.

Dry run: watch it execute

A concrete class that forgets one method does not compile.

Algorithm: Does the class satisfy the interface?. **Input:** impl = `area,perimeter`, abs = `no`.

Pseudocode

```
interface Shape { double area(); double perimeter(); String name(); }
class Circle implements Shape { …only the methods listed… }
concrete class → must implement all three
```

Step 1 of 5. Shape requires three abstract methods.



Step 2 of 5. area() is implemented by Circle.

method = area() · implemented = true



Step 3 of 5. perimeter() is implemented by Circle.

method = perimeter() · implemented = true



Step 4 of 5. name() is NOT implemented by Circle.

method = name() · implemented = false



Step 5 of 5. The compiler reports: Circle is not abstract and does not override abstract method name() in Shape.

compiles = false

> **RESULT**
> compile-time error: name() missing

Complexity

Interface dispatch is O(1). The design choice affects maintainability, not asymptotic cost.

Common mistakes

Instantiating an abstract class

Problem

```
Employee e = new Employee("x");
```

Why: Compile error: Employee is abstract; cannot be instantiated.

Fix: new Salaried("x", 1000)

Weaker access in the implementation

Problem

```
class Square implements Shape {
 double area() { ... } // package-private
```

Why: Interface methods are public; an implementation cannot reduce visibility.

Fix: public double area()

Forgetting to implement all abstract methods

Problem

```
class Hourly extends Employee { }
```

Why: Hourly is not abstract, so it must implement monthlyPay().

Fix: Implement it, or declare Hourly abstract.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Total area — *Beginner*

Define an interface Shape with area(); implement Circle and Square. Read n lines `circle r` or `square s` and print the total area with two decimals.

Example input	Expected output
3 ⏎ circle 1 ⏎ square 2 ⏎ circle 2	19.71

Hint 1: A list/array of Shape.

Exercise 2 · Abstract Vehicle — *Intermediate*

Create an abstract class Vehicle with an abstract wheels() and a concrete describe() that prints `<ClassName> has <n> wheels`. Read `bike` or `car` and print the description.

Example input	Expected output
bike	Bike has 2 wheels

Hint 1: getClass().getSimpleName()

Exercise 3 · Comparable versions — *Intermediate*

Create class Version implementing Comparable<Version> for strings like `1.10`. Read n versions and print them sorted by major then minor numerically.

Example input	Expected output
4 ⏎ 1.10 ⏎ 1.2 ⏎ 2.0 ⏎ 1.9	1.2 1.9 1.10 2.0

Hint 1: Integer.compare on major, then minor.

Quick check

Answers are in the answer key at the end of this part.

Q1. Which can a class do?

- **A.** extend two classes
- **B.** implement several interfaces
- **C.** extend an interface
- **D.** instantiate an interface

Q2. An abstract method...

- **A.** has a body
- **B.** has no body and must be implemented by a concrete subclass
- **C.** is private
- **D.** cannot be in a class

Q3. To make `Collections.sort(list)` work for your class, implement...

- **A.** Runnable
- **B.** Comparable<T>
- **C.** Serializable
- **D.** Cloneable

Q4. Unrelated classes that share a capability are best modelled with...

- **A.** an abstract class
- **B.** an interface
- **C.** a static method
- **D.** a constructor

Interview / exam questions

- Abstract class vs interface?
- What is a default method?
- Why program to an interface?
- What is Comparable vs Comparator?

Watch & read

Video: Java Tutorial: Abstract Classes Vs Interfaces — CodeWithHarry · 14:50. *Abstract classes and interfaces compared*

Video: 8.22 Interface in Java 8 Default , Static Methods | New features — Telusko · 18:20. *Interfaces in modern Java*

REVISION CARD (30–60 SECONDS)

- abstract class: state + partial code; cannot be instantiated.
- interface: contract; many per class; default methods.
- Program to the interface.
- Comparable<T>.compareTo → natural ordering.

3.5 static, final and Initialisation Order

Object-Oriented Programming · Intermediate · Essential Java syntax · about 30 min · complete path

Members that belong to the class rather than to objects, constants, and what runs when.

YOU WILL BE ABLE TO

- Distinguish static (class-level) and instance (object-level) members.
- Use `final` for variables, methods and classes, and `static final` for constants.
- Predict the order: static blocks → instance initialisers → constructor.
- Explain why `main` is static and why a static method cannot use `this`.

Before this lesson: Classes, Objects, Fields, Constructors and this

The idea

Instance members belong to each object (every Student has its own `marks`). **`static`** members belong to the **class** itself: one shared copy, accessible as `ClassName.member` without any object. `Math.sqrt(x)` and `Integer.MAX_VALUE` are static.

keyword	on a variable	on a method	on a class
static	one shared copy	called on the class; no `this`	nested class independent of an outer object
final	assigned once (a constant)	cannot be overridden	cannot be extended (String is final)

WHY THIS WORKS

`main` is `static` because the JVM must be able to start the program **before any object exists**. For the same reason a static method cannot touch instance fields — there is no `this`. `final` protects the *variable*, not the object it refers to: a `final int[]` can still have its elements changed.

Order of initialisation

1. Class loading: **static** fields/blocks run once, top to bottom.
2. Each `new`: instance field initialisers and **instance blocks** run, in source order...
3. ...then the **constructor** body.

Syntax

```
class Counter {
 static int created = 0; // one copy shared by all objects
 final int id; // per object; assigned exactly once
 static final double TAX = 0.18; // a constant: static + final, NAMES_IN_CAPS

 static { System.out.println("class loaded"); } // static block: once, at class loading
 { created++; } // instance block: before every constructor body
 Counter() { id = created; }
 static int total() { return created; } // static method: call as Counter.total()
}
```

	static member	instance member
belongs to	the class	one object
accessed with	`ClassName.member`	`object.member`
can use `this`?	no	yes

A tiny example

```
class Counter {
 static int created = 0;
 static final int MAX = 100;
 Counter() { created++; }
}
```

Guided example

StaticFinalDemo.java — Static, final and init order (demonstration)

```
1  public class StaticFinalDemo {
2   static class Ticket {
3   static int issued = 0; // ONE copy shared by all tickets                    (1)
4   static final double TAX_RATE = 0.18; // constant: static + final           (2)
5   private final int number; // one per object, set exactly once
6
7   static { // static block: runs once, when the class is loaded               (3)
8   System.out.println("[Ticket class loaded]");
9   }
10
11  { // instance initialiser: runs before every constructor body               (4)
12  issued++;
13  }
14
15  Ticket() {
16  number = issued;
17  System.out.println("[Ticket #" + number + " created]");
18  }
19
20  static double withTax(double price) { // static method: no `this`, called on the class  (5)
21  return price * (1 + TAX_RATE);
22  }
23
24  int getNumber() {
25  return number;
26  }
27  }
28
29  public static void main(String[] args) {
30  System.out.println("start");
31  Ticket a = new Ticket();
32  Ticket b = new Ticket();
33  System.out.println("issued so far: " + Ticket.issued);
34  System.out.printf("100 with tax: %.2f%n", Ticket.withTax(100));
35  System.out.println("numbers: " + a.getNumber() + ", " + b.getNumber());
36  // Ticket.TAX_RATE = 0.2; // would NOT compile: final
37  final int[] arr = {1, 2, 3};                                               (6)
38  arr[0] = 99; // allowed: final protects the reference, not the contents
39  System.out.println("final array first element: " + arr[0]);
40  }
41 }
```

Watch the order of the bracketed messages.

Line-by-line walkthrough

#	Where	What happens
1	line 3	Shared by all tickets: a counter of how many exist.
2	line 4	A true constant: one copy, never reassigned, named in UPPER_SNAKE_CASE.
3	line 7	Static block: runs once, the first time the class is used.
4	line 11	Instance block: runs for EVERY new object before the constructor body.
5	line 20	A static method: called as Ticket.withTax(100) with no object.

#	Where	What happens
6	line 37	final = the variable cannot be re-pointed; the array elements can still change.

Dry run: watch it execute

static block once, then instance block + constructor per object.

Algorithm: Initialisation order. **Input:** n = 2.

Pseudocode

```
class Demo {
 static int count;
 static { print("static block"); }
 { print("instance block"); count++; }
 Demo() { print("constructor, count=" + count); }
 main: print("main starts"); new Demo() × n
}
```

Step 1 of 6. The JVM loads and initialises class Demo BEFORE calling main: static initialisers run, exactly once.

count = 0

console output `static block`

Step 2 of 6. main begins.

count = 0

console output `static main starts`

Step 3 of 6. new Demo() #1: the instance block runs first (count becomes 1).

count = 1

console output `static main stinstance block`

Step 4 of 6. Then the constructor body runs for object #1.

count = 1

console output `static main stinstanconstructor, count=1`

Step 5 of 6. new Demo() #2: the instance block runs first (count becomes 2).

count = 2

console output `static main stinstancconstruinstance block`

Step 6 of 6. Then the constructor body runs for object #2.

count = 2

console output `static main stinstancconstruinstanconstructor, count=2`

> **RESULT**
> static block → main starts → instance block → constructor, count=1 → instance block → constructor, count=2

Complexity

No run-time cost. A static field uses one memory location for all objects.

Common mistakes

Using an instance member from static context

Problem

```
static void f() { System.out.println(count); } // count is an instance field
```

Why: No object, so no count: "non-static variable cannot be referenced from a static context".

Fix: Make it static or call through an object.

Mutable static state

Problem

```
static List<Integer> cache = new ArrayList<>(); // shared by everything
```

Why: Hidden coupling: one test or caller affects another.

Fix: Prefer instance state; use static only for constants and pure helpers.

Assuming final means immutable object

Problem

```
final List<String> names = new ArrayList<>();
names.add("x"); // allowed!
```

Why: Only the reference is final.

Fix: Use List.of(...) or Collections.unmodifiableList for read-only data.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · ID generator — *Beginner*

Create a class with a static counter and a static method nextId(). Read n and print n generated IDs starting at 1, space separated.

Example input	Expected output
4	1 2 3 4

Hint 1: private static int next = 1; return next++;

Exercise 2 · A constant — *Beginner*

Define `static final double PI_APPROX = 3.14159` and a method circleArea(r). Read r and print the area with three decimals.

Example input	Expected output
2	12.566

Hint 1: printf("%.3f%n", ...)

Exercise 3 · Initialisation order — *Intermediate*

Run the program that creates two objects of a class with a static block, an instance block and a constructor; write down the order of the five lines.

Example input	Expected output
(no input)	static block ⏎ instance block ⏎ constructor ⏎ instance block ⏎ constructor

Hint 1: Static block once; instance block + constructor for every object.

Quick check

Answers are in the answer key at the end of this part.

Q1. Why is main declared static?

- **A.** for speed
- **B.** the JVM calls it before any object exists
- **C.** to hide it
- **D.** because of the String[] parameter

Q2. How many copies of a static field exist?

- **A.** one per object
- **B.** one per class
- **C.** one per method
- **D.** zero

Q3. What is printed?

```
class C { static int n; C() { n++; } }
new C(); new C(); new C();
System.out.println(C.n);
```

Your answer: _____

Q4. A `final` variable of type int[]...

- **A.** cannot be re-assigned but elements can change
- **B.** is completely immutable
- **C.** cannot be read
- **D.** must be static

Interview / exam questions

- static vs instance?
- What does final do on a class, method, variable?
- Order of static and instance initialisation?
- Can a static method be overridden?

Watch & read

Video: #57 Final keyword in java — Telusko · 6:15. *static members and final*

Video: #39 Static Block in java — Telusko · 7:18. *Initialisation order*

REVISION CARD (30–60 SECONDS)

- static = per class; instance = per object.
- final = assign once / no override / no extend.
- Order: static blocks → instance blocks → constructor.
- static methods have no this.

3.6 Packages and Imports

Object-Oriented Programming · Beginner · Essential Java syntax · about 20 min · complete path

How Java organises classes into namespaces, how `import` works, and the standard packages you will use.

YOU WILL BE ABLE TO

- Write a package declaration and map it to a folder structure.
- Use single-class, wildcard and static imports.
- Resolve name clashes with fully qualified names.
- Name the packages you will use most.

Before this lesson: Your First Program: class, main, statements, comments

The idea

A **package** is a named group of related classes — a namespace that avoids name clashes and organises code. The declaration `package com.shop.billing;` must be the first statement; the file then lives in the folder `com/shop/billing/`.

you write	meaning
import java.util.List;	use `List` without the full name
import java.util.*;	import every class of the package (not sub-packages)
import static java.lang.Math.max;	use `max(a, b)` directly
java.util.Date d;	fully qualified name: no import needed

WHY THIS WORKS

`java.lang` (String, Math, System, Integer, Exception...) is imported automatically. Imports are only a **compile-time convenience** — they do not load anything; they just let you write short names. When two imported packages both contain a class called `Date`, use the fully qualified name for at least one of them.

package	contents you will meet
java.lang	String, Math, Integer, Object, Thread, exceptions
java.util	Scanner, ArrayList, HashMap, Arrays, Collections, Random, Optional
java.io	BufferedReader, IOException, File
java.math	BigInteger, BigDecimal
java.util.function / java.util.stream	lambdas and streams

Syntax

```
package com.school.util; // first statement: the package of this file

import java.util.List; // one class
import java.util.*; // every class of the package
import static java.lang.Math.sqrt; // a static member without the class prefix
```

Compile and run from the project root

```
src/com/school/util/Tools.java // directory = package name
javac -d out src/com/school/util/Tools.java src/com/school/Main.java
java -cp out com.school.Main // run with the FULL class name
```

A tiny example

```
import java.util.ArrayList;
import java.util.List;
List<String> names = new ArrayList<>();
```

Guided example

ImportsDemo.java — Imports (demonstration)

```
1  // A package statement (if any) must be the FIRST line: package com.example.demo;
2  // This file is in the default package, so there is none.
3
4  import java.util.ArrayList; // import one class
5  import java.util.List;
6  import java.util.Map;
7  import java.util.HashMap;
8  import static java.lang.Math.max; // static import: use max(...) without "Math."          (1)
9
10 public class ImportsDemo {
11  public static void main(String[] args) {
12  // java.lang is imported automatically: String, Math, System, Integer ...
13  List<String> names = new ArrayList<>();
14  names.add("Asha");
15  Map<String, Integer> ages = new HashMap<>();
16  ages.put("Asha", 20);
17
18  // a fully qualified name needs no import (useful when two classes share a simple name)
19  java.util.Date now = null;                                                              (2)
20  java.sql.Date sqlDate = null;
21
22  System.out.println(names + " " + ages + " max=" + max(3, 9));
23  System.out.println(now == null && sqlDate == null);
24  System.out.println(ImportsDemo.class.getName() + " in package '" +                      (3)
        ImportsDemo.class.getPackageName() + "'");
25  }
26 }
```

The default package is used here, so no package statement appears.

Line-by-line walkthrough

#	Where	What happens
1	line 8	Static import: call max(...) without the class name.
2	line 19	A fully qualified name works without any import.
3	line 24	Prints the package of a class — empty here (default package).

> **COMMAND LINE**
> Compiling a program that is inside a package: from the project root run `javac -d out com/shop/billing/Invoice.java` and run `java -cp out com.shop.billing.Invoice`. IDEs do this for you.

Dry run: watch it execute

The order in which the compiler resolves a simple class name.

Algorithm: How the compiler finds a class name. **Input:** imports = `java.util.*`, name = `List`.

Pseudocode

```
1. a class in this file / this package
2. a single-type import import java.util.List;
3. java.lang (implicit)
4. an on-demand import import java.util.*;
otherwise: cannot find symbol
```

Step 1 of 5. The code uses the simple name List. The compiler must decide which class that is.
name = List

Step 2 of 5. Step 1: is there a class named List in this file or this package? No.
found = false

Step 3 of 5. Step 2: no single-type import ends in .List.
found = false

Step 4 of 5. Step 3: not in java.lang.
found = false

Step 5 of 5. Step 4: the on-demand import java.util.* covers every public class of java.util, including List.
found = true · via = java.util.*

> **RESULT**
> resolved: java.util.List

Complexity

Not applicable (compile-time feature).

Common mistakes

Forgetting an import

Problem

```
List<String> a = new ArrayList<>(); // no import
```

Why: cannot find symbol: class List.

Fix: import java.util.List; import java.util.ArrayList;

Folder does not match package

Problem

```
// package a.b; but the file is in the folder src/
```

Why: The compiler/IDE cannot locate the class.

Fix: Put the file in a/b/.

Importing a clashing wildcard

Problem

```
import java.util.*; import java.sql.*;
Date d;
```

Why: Reference to Date is ambiguous.

Fix: java.util.Date d;

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Static import — *Beginner*

Use a static import of Math.sqrt and print the square root of the input number.

Example input	Expected output
16	4.0

Hint 1: import static java.lang.Math.sqrt;

Exercise 2 · Sort with Arrays — *Beginner*

Read n and an array. Import java.util.Arrays and print the sorted array as `[a, b, c]`.

Example input	Expected output
4 ⏎ 9 3 7 1	[1, 3, 7, 9]

Hint 1: Arrays.sort then Arrays.toString

Exercise 3 · Same name, two packages — *Intermediate*

Create a java.util.Date and a java.sql.Date using fully qualified names and print their class names (one per line).

Example input	Expected output
(no input)	java.util.Date ⏎ java.sql.Date

Hint 1: No imports needed.

Quick check

Answers are in the answer key at the end of this part.

Q1. Which package is imported automatically?

- **A.** java.util
- **B.** java.io
- **C.** java.lang
- **D.** java.math

Q2. Where must the package statement appear?

- **A.** anywhere
- **B.** first in the file
- **C.** after imports
- **D.** inside the class

Q3. Does `import java.util.*;` import java.util.function.Function?

- **A.** yes
- **B.** no — sub-packages are separate

Q4. The folder for `package com.shop.billing;` is...

- **A.** com.shop.billing/
- **B.** com/shop/billing/
- **C.** billing/
- **D.** shop/

Interview / exam questions

- What is a package? Why use them?
- Import vs fully qualified name?
- What is the default package?
- What does import static do?

Watch & read

Video: Java Packages and Import Statements - Java programming — Will Tollefson · 10:43. *Packages and imports*

Video: #53 Packages in Java — Telusko · 12:20. *Compiling and running code in packages*

REVISION CARD (30–60 SECONDS)

- package a.b.c; = folder a/b/c.
- import X; import pkg.*; import static ...;
- java.lang is automatic.
- Clash → fully qualified name.

3.7 Exceptions: try, catch, finally, throw and throws

Object-Oriented Programming · Intermediate · Essential Java syntax · about 35 min · complete path

Handle run-time errors on purpose: checked vs unchecked, catching, cleaning up, and defining your own.

YOU WILL BE ABLE TO

- Use try/catch/finally and multi-catch correctly.
- Distinguish checked and unchecked exceptions and apply `throws`.
- Throw built-in and custom exceptions with a good message.
- Use try-with-resources to close resources automatically.

Before this lesson: Compile-time, Runtime and Logic Errors · Inheritance, Overriding and Polymorphism

The idea

An **exception** is an object that represents a problem. When code `throw`s one, normal execution stops and Java looks **up the call stack** for a matching `catch`. If none exists the program ends with a stack trace.

```
Throwable
 ├─ Error (JVM problems: OutOfMemoryError — don't catch)
 └─ Exception
 ├─ IOException ... CHECKED: compiler forces you to handle or declare
 └─ RuntimeException UNCHECKED: programming mistakes
 ├─ ArithmeticException, NullPointerException
 ├─ ArrayIndexOutOfBoundsException, ClassCastException
 └─ IllegalArgumentException, NumberFormatException
```

WHY THIS WORKS

Checked exceptions model problems a correct program can still meet (a missing file); the compiler makes you decide. **Unchecked** ones signal bugs (a null, a bad index); fix the code instead of catching. `finally` runs **whatever happens** (even after a `return`), so it is the place for cleanup; try-with-resources does that for anything implementing `AutoCloseable`.

- Catch the **most specific** exception first; order matters.
- Never write an empty catch block; at least log or rethrow.
- Throw early (validate arguments) and catch late (where you can do something about it).
- Custom exception: `class MyException extends Exception` (checked) or `extends RuntimeException` (unchecked).

Syntax

```
try {
 risky();
} catch (NumberFormatException | ArithmeticException e) {
 System.out.println(e.getMessage());
} finally {
 cleanup();
}

void read() throws IOException { /* may throw */ } // declare
throw new IllegalArgumentException("n < 0"); // raise
try (BufferedReader r = new BufferedReader(new FileReader("in.txt"))) { /* use r */ } //
    auto-close
```

A tiny example

```
try {
 int x = Integer.parseInt("12x");
} catch (NumberFormatException e) {
 System.out.println("not a number");
}
```

Guided example

ExceptionsDemo.java — Exceptions tour (demonstration)

```
1  import java.io.BufferedReader;
2  import java.io.IOException;
3  import java.io.StringReader;
4
5  public class ExceptionsDemo {
6   // A CUSTOM checked exception
7   static class InsufficientFundsException extends Exception {                    (1)
8   InsufficientFundsException(String message) {
9   super(message);
10  }
11  }
12
13  static void withdraw(int balance, int amount) throws InsufficientFundsException {   (2)
14  if (amount > balance) {
15  throw new InsufficientFundsException("need " + (amount - balance) + " more");
16  }
17  System.out.println("withdrew " + amount);
18  }
19
20  static int parseOrDefault(String s, int fallback) {
21  try {
22  return Integer.parseInt(s);
23  } catch (NumberFormatException e) {
24  return fallback;
25  } finally {                                                                    (3)
26  System.out.println(" (finally always runs for \"" + s + "\")");
27  }
28  }
29
30  public static void main(String[] args) {
```

(code continued)

```
31    // try / catch / finally
32    System.out.println(parseOrDefault("123", -1));
33    System.out.println(parseOrDefault("12x", -1));
34
35    // catching several exception types
36    int[] data = {10, 0};
37    try {
38    System.out.println(data[0] / data[1]);
39    } catch (ArithmeticException | ArrayIndexOutOfBoundsException e) {                    4
40    System.out.println("caught " + e.getClass().getSimpleName() + ": " + e.getMessage());
41    }
42
43    // checked exceptions must be handled or declared
44    try {
45    withdraw(100, 30);
46    withdraw(100, 250);
47    } catch (InsufficientFundsException e) {
48    System.out.println("refused: " + e.getMessage());
49    }
50
51    // try-with-resources closes the resource automatically
52    try (BufferedReader r = new BufferedReader(new StringReader("line one\nline two"))) {    5
53    System.out.println(r.readLine());
54    } catch (IOException e) {
55    System.out.println("io problem");
56    }
57
58    // unchecked: NullPointerException, IllegalArgumentException ... need no declaration
59    try {
60    Object o = "text";
61    Integer i = (Integer) o;
62    } catch (ClassCastException e) {
63    System.out.println("ClassCastException caught");
64    }
65    }
66   }
```

Each section triggers one kind of exception.

Line-by-line walkthrough

#	Where	What happens
1	line 7	A custom CHECKED exception because it extends Exception (not RuntimeException).
2	line 13	Declares to callers that this method can throw it.
3	line 25	Runs after the try (and catch), including when the try returned early.
4	line 39	Multi-catch: one handler for two unrelated types.
5	line 52	The reader is closed automatically at the end of the block.

Dry run: watch it execute

A thrown exception skips the rest of try; finally always runs.

Algorithm: try / catch / finally. **Input:** d = `0`, idx = `1`.

Pseudocode

```
try {
 int r = 100 / d;
 int[] a = new int[3]; a[idx] = r;
 print("stored");
} catch (ArithmeticException e) { print("math"); }
catch (ArrayIndexOutOfBoundsException e) { print("index"); }
finally { print("cleanup"); }
print("after");
```

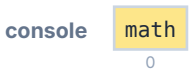
Step 1 of 5. Execution enters the try block.

d = 0 · idx = 1

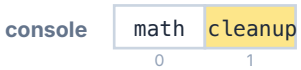
Step 2 of 5. 100 / d with d = 0 throws ArithmeticException. The remaining lines of the try block are SKIPPED.

thrown = ArithmeticException

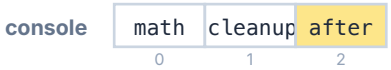
Step 3 of 5. The thrown ArithmeticException matches the first catch, which prints "math".



Step 4 of 5. finally ALWAYS runs: after a normal finish, after a caught exception, even after return.



Step 5 of 5. Because the exception was handled, execution continues after the whole try statement.



> **RESULT**
> math → cleanup → after

Complexity

Throwing and catching an exception is relatively expensive (building the stack trace). Do not use exceptions for ordinary control flow in a hot loop.

Common mistakes

Catching Exception everywhere

Problem

```
catch (Exception e) { }
```

Why: Swallows real bugs and hides the cause.

Fix: Catch the specific type and handle or rethrow.

Wrong catch order

Problem

```
catch (Exception e) {...}
catch (IOException e) {...}
```

Why: Compile error: the second catch is unreachable.

Fix: Put the subclass first.

Using return in finally

Problem

```
finally { return 0; }
```

Why: It overrides any exception or earlier return.

Fix: Use finally only for cleanup.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Safe division — *Beginner*

Read two tokens. Print a / b as integers, `bad number` if either is not an integer, or `division by zero`.

Example input	Expected output
12 4	3

Hint 1: Two catch clauses.

Exercise 2 · Custom exception — *Intermediate*

Define NegativeAgeException (unchecked). Read n ages; print `ok <age>` for valid ones, or the exception message `age cannot be negative: <age>`.

Example input	Expected output
3 ⏎ 20 ⏎ −5 ⏎ 0	ok 20 ⏎ age cannot be negative: −5 ⏎ ok 0

Hint 1: extends RuntimeException; call super(message).

Exercise 3 · try / finally order — *Intermediate*

Run the program: a method prints in try, returns 1, and prints in finally. Write the output order.

Example input	Expected output
(no input)	try ⏎ finally ⏎ 1

Hint 1: finally runs before the value is delivered to the caller.

Quick check

Answers are in the answer key at the end of this part.

Q1. Which is a checked exception?

- **A.** ArithmeticException
- **B.** NullPointerException
- **C.** IOException
- **D.** IllegalArgumentException

Q2. What is printed?

```
try {
 System.out.print("A");
 int x = 1 / 0;
 System.out.print("B");
} catch (ArithmeticException e) {
 System.out.print("C");
} finally {
 System.out.print("D");
}
```

Your answer: _____

Q3. What does `throws` do in a method signature?

- **A.** throws an exception
- **B.** declares that the method may throw a checked exception
- **C.** catches an exception
- **D.** ignores errors

Q4. try-with-resources requires the resource to implement...

- **A.** Serializable
- **B.** AutoCloseable
- **C.** Runnable
- **D.** Comparable

Interview / exam questions

- Checked vs unchecked?
- Does finally always run?
- How do you create a custom exception?
- What is try-with-resources?

Watch & read

Video: Java Exception Handling in One Shot | try-catch, finally & throw vs throws — The Curious Coder · 35:54.
try/catch/finally, throw and throws

Video: Checked vs. Unchecked Exceptions in Java Tutorial - What's The Difference? — Coding with John · 10:14.
Checked and unchecked exceptions; custom exceptions

REVISION CARD (30–60 SECONDS)

- try / catch(specific first) / finally.
- Checked: handle or declare; unchecked: fix the bug.
- throw new X("msg"); throws X.
- try-with-resources closes automatically.

3.8 Generics and Lists: ArrayList and LinkedList

Object-Oriented Programming · Beginner · Core programming & data structures · about 40 min · ★ on the minimum-viable path

Type-safe containers with `<T>`: the resizable array, the linked list, autoboxing and the remove() trap.

YOU WILL BE ABLE TO

- Create and use `List<T>` with `ArrayList` and `LinkedList`.
- Explain generics (`<T>`) and why primitives need wrapper types.
- Avoid the `remove(int)` vs `remove(Object)` trap.
- Choose a list type from the costs of its operations.

Before this lesson: One-Dimensional Arrays · Classes, Objects, Fields, Constructors and this

The idea

Arrays have a fixed size. A **`List<E>`** is a resizable sequence from `java.util`. The type in angle brackets is a **generic type argument**: `List<Integer>` can only hold Integers, and the compiler checks it (no casts needed).

	ArrayList	LinkedList
inside	a resizable array	a doubly-linked chain of nodes
get(i) / set(i)	O(1)	O(n)
add at end	O(1) amortised	O(1)
insert / remove in the middle	O(n) (shifts)	O(1) once positioned, O(n) to find
add/remove at the front	O(n)	O(1)
memory	compact	node object per element
default choice	yes	rarely — use ArrayDeque for queues

WHY THIS WORKS

Generics were added so containers can be type-safe without code duplication: one `ArrayList` class works for `String`, `Integer`, or your own `Student`. They only work with **reference types**, so `List<int>` is illegal; Java automatically **boxes** an `int` into an `Integer` and unboxes it back (autoboxing).

TWO CLASSIC TRAPS

`List<Integer> a; a.remove(1)` removes the element at **index 1**. To remove the **value** 1 write `a.remove(Integer.valueOf(1))`. Likewise `Integer p = 1000, q = 1000; p == q` is **false** — compare boxed values with `.equals()`.

Syntax

```
List<Integer> a = new ArrayList<>(); // diamond: type inferred
a.add(5); a.add(0, 9); // append / insert at index
a.get(0); a.set(0, 1); a.size(); a.isEmpty();
a.contains(5); a.indexOf(5);
a.remove(0); a.remove(Integer.valueOf(5)); // by index / by value
Collections.sort(a); Collections.reverse(a); Collections.max(a);
List<Integer> fixed = List.of(1, 2, 3); // immutable
int[] arr = a.stream().mapToInt(Integer::intValue).toArray();
```

A tiny example

```
List<String> names = new ArrayList<>();
names.add("Asha");
for (String n : names) System.out.println(n);
```

Guided example

ListsDemo.java — Lists and generics (demonstration)

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedList;
import java.util.List;

public class ListsDemo {
 // A GENERIC method: works for a list of any element type T.
 static <T> void swapFirstAndLast(List<T> list) {                    (1)
 if (list.size() < 2) return;
 T first = list.get(0);
 list.set(0, list.get(list.size() - 1));
 list.set(list.size() - 1, first);
 }

 public static void main(String[] args) {
 // ArrayList<E>: a resizable array. The type argument says what it holds.
 List<Integer> nums = new ArrayList<>();
 nums.add(30);
 nums.add(10);
 nums.add(20);
 nums.add(1, 99); // insert at index 1: shifts the rest right      (2)
 System.out.println(nums + " size=" + nums.size() + " get(2)=" + nums.get(2));

 nums.set(0, 31);
 nums.remove(Integer.valueOf(99)); // remove by VALUE               (3)
 nums.remove(0); // remove by INDEX
 System.out.println(nums + " contains 20? " + nums.contains(20) + " indexOf(10)=" +
     nums.indexOf(10));

 // the classic trap: a List<Integer> has two remove methods
 List<Integer> a = new ArrayList<>(List.of(5, 6, 7, 1));
 a.remove(1); // index 1 -> removes the 6                           (4)
 a.remove(Integer.valueOf(1)); // the VALUE 1
 System.out.println(a);

 // loops
```

(code continued)

```
    int sum = 0;
    for (int x : a) sum += x;
    System.out.println("sum " + sum);

    // sorting and utilities
    List<String> words = new ArrayList<>(List.of("pear", "apple", "fig"));
    Collections.sort(words);
    System.out.println(words + " max=" + Collections.max(words) + " reversed=" +
        reversed(words));
    swapFirstAndLast(words);
    System.out.println(words);

    // LinkedList: fast insert/remove at both ends, slow get(i)
    LinkedList<String> ll = new LinkedList<>(words);                    (5)
    ll.addFirst("kiwi");
    ll.removeLast();
    System.out.println(ll);

    // autoboxing: int <-> Integer is automatic, but == on Integer objects is dangerous
    Integer p = 127, q = 127, r = 1000, s = 1000;                       (6)
    System.out.println((p == q) + " " + (r == s) + " " + r.equals(s));

    // a list cannot hold primitives: List<int> is illegal, use the wrapper type Integer
    System.out.println(List.of(1, 2, 3).getClass().getSimpleName().startsWith("List"));
    }

    static <T> List<T> reversed(List<T> in) {
    List<T> out = new ArrayList<>(in);
    Collections.reverse(out);
    return out;
    }
}
```

Compare each printed line with the code that produced it.

Line-by-line walkthrough

#	Where	What happens
1	line 8	A generic method: `<T>` declares a type parameter used in the signature.
2	line 21	Insert at index 1; elements to the right shift.
3	line 25	Remove by VALUE: passing an Integer object selects remove(Object).
4	line 31	Remove by INDEX: an int literal selects remove(int).
5	line 48	Same List interface, different internals: addFirst/removeLast are O(1).
6	line 54	Integers −128..127 are cached so == happens to work; 1000 is not cached so r == s is false.

Dry run: watch it execute

Growth by 1.5× and the cost of shifting.

Algorithm: ArrayList growth and shifting. **Input:** cap = `4`, ops = `add 5,add 7,add 9,add 2,add 8,add@0 1,remove@2`.

Pseudocode

```
new ArrayList<>(cap)
add(v) // append; grow by 1.5x when full
add(i, v) // shift [i..] right
remove(i) // shift (i..] left
```

Step 1 of 9. new ArrayList<>(4): an array of 4 cells, size 0.

size = 0 · capacity = 4

backing array (capacity 4, size 0)



Step 2 of 9. add(5) writes into the next free cell (index 0). O(1).

size = 1 · capacity = 4

backing array (capacity 4, size 1)



Step 3 of 9. add(7) writes into the next free cell (index 1). O(1).

size = 2 · capacity = 4

backing array (capacity 4, size 2)



Step 4 of 9. add(9) writes into the next free cell (index 2). O(1).

size = 3 · capacity = 4

backing array (capacity 4, size 3)



Step 5 of 9. add(2) writes into the next free cell (index 3). O(1).

size = 4 · capacity = 4

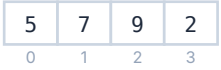
backing array (capacity 4, size 4)



Step 6 of 9. The array is full (4/4): a new array of 6 cells is allocated and every element is copied. This is the expensive step.

old capacity = 4 · new capacity = 6

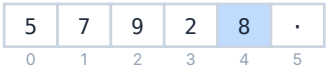
backing array (capacity 4, size 4)



Step 7 of 9. add(8) writes into the next free cell (index 4). O(1).

size = 5 · capacity = 6

backing array (capacity 6, size 5)



Step 8 of 9. add(0, 1) first shifts 5 elements one cell to the right, then writes the value. O(n).

size = 6 · shifts = 5

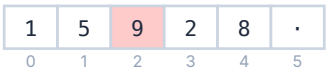
backing array (capacity 6, size 6)



Step 9 of 9. remove(2) takes out 7 and shifts 3 elements one cell to the left. O(n).

size = 5 · shifts = 3

backing array (capacity 6, size 5)



> **RESULT**
> [1, 5, 9, 2, 8] — size 5, capacity 6

Complexity

ArrayList doubles its internal array when full, so `add` is **amortised O(1)**. `contains`/`indexOf` are O(n) on any list. Sorting is O(n log n). Prefer `ArrayList` unless you do many insertions at the front.

Common mistakes

ConcurrentModificationException

Problem

```
for (Integer x : list) if (x == 3) list.remove(x);
```

Why: Modifying a list while iterating over it with for-each.

Fix: list.removeIf(x → x == 3); or use an Iterator

remove(int) vs remove(Object)

Problem

```
list.remove(5); // meant the value 5
```

Why: Removes the element at index 5 (or throws IndexOutOfBounds).

Fix: list.remove(Integer.valueOf(5));

Using == on boxed numbers

Problem

```
if (list.get(0) == list.get(1))
```

Why: Compares references: true only for the small cached values.

Fix: list.get(0).equals(list.get(1))

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Remove duplicates — *Beginner*

Read n integers and print them with duplicates removed, keeping the first occurrence of each.

Example input	Expected output
6 ⏎ 3 1 3 2 1 4	3 1 2 4

Hint 1: Add to a result list only if !contains.

Exercise 2 · Remove a value — *Intermediate*

Read n integers and a target. Print the list (as `[a, b]`) with every occurrence of the target removed — without ConcurrentModificationException.

Example input	Expected output
6 ⏎ 3 1 3 2 1 4 ⏎ 3	[1, 2, 1, 4]

Hint 1: list.removeIf

Exercise 3 · Generic maximum — *Challenge*

Write `static <T extends Comparable<T>> T largest(List<T> list)` and use it on n words; print the largest alphabetically.

Example input	Expected output
4 ⏎ pear apple zebra fig	zebra

Hint 1: `T extends Comparable<T>` lets you call compareTo.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
List<Integer> a = new ArrayList<>(List.of(5, 6, 7));
a.remove(1);
System.out.println(a);
```

Your answer: _____

Q2. Which list is best for random access get(i)?

- **A.** LinkedList
- **B.** ArrayList
- **C.** neither
- **D.** both equal

Q3. Why is List<int> illegal?

- **A.** ints are too big
- **B.** generics need reference types; use Integer
- **C.** lists are only for strings
- **D.** it is legal

Q4. What is printed?

```
Integer a = 1000, b = 1000;
System.out.println(a == b);
```

Your answer: _____

Interview / exam questions

- ArrayList vs LinkedList?
- What are generics and type erasure?
- What is autoboxing?
- Why does for-each + remove throw?

Watch & read

Video: LinkedList vs ArrayList in Java Tutorial - Which Should You Use? — Coding with John · 11:43. *Generics, ArrayList and LinkedList*

Video: 14.5 LinkedList vs ArrayList in Java — Telusko · 9:16. *When to use ArrayList vs LinkedList*

REVISION CARD (30–60 SECONDS)

- List<E>: ArrayList (array) vs LinkedList (chain).
- Wrapper types for primitives; remove(int index) vs remove(Object).
- Modify during for-each → use removeIf / Iterator.
- ArrayList add amortised O(1); contains O(n).

3.9 Sets and Maps: HashSet, TreeSet, HashMap, TreeMap

Object-Oriented Programming · Beginner · Core programming & data structures · about 40 min · ★ on the minimum-viable path

Fast membership and lookup: sets remove duplicates, maps connect keys to values, and counting with `merge`.

YOU WILL BE ABLE TO

- Use `HashSet` / `TreeSet` for uniqueness and membership.
- Use `HashMap` / `TreeMap` for counting and lookup, iterating with `entrySet()`.
- Apply `getOrDefault`, `merge` and `computeIfAbsent`.
- Pick hash vs tree vs linked variants by ordering needs and cost.

Before this lesson: Generics and Lists: ArrayList and LinkedList

The idea

A **Set** stores each value at most once and answers "is x in here?" quickly. A **Map** stores **key → value** pairs with unique keys and answers "what is the value for this key?".

	Hash...	Tree...	LinkedHash...
order	none	sorted by key	insertion order
add / contains / get	O(1) average	O(log n)	O(1) average
needs	equals + hashCode	Comparable / Comparator	equals + hashCode
use when	default	sorted output, floor/ceiling	predictable iteration

WHY THIS WORKS

A **hash table** turns a key into an array index with `hashCode()`, so lookup does not depend on how many elements there are. That is why counting words in a `HashMap` is O(n) for n words while counting with a list (`contains` + search) is O(n²). The price: no ordering, and keys must have consistent `equals`/`hashCode` (see the last lesson of this module).

Idiomatic counting: `freq.merge(key, 1, Integer::sum)` or `freq.put(key, freq.getOrDefault(key, 0) + 1)`. Grouping: `map.computeIfAbsent(k, x -> new ArrayList<>()).add(v)`.

Syntax

```
Set<Integer> s = new HashSet<>();
s.add(3); s.contains(3); s.remove(3); s.size();
Map<String, Integer> m = new HashMap<>();
m.put("a", 1); m.get("a"); m.getOrDefault("z", 0);
m.containsKey("a"); m.remove("a");
for (Map.Entry<String, Integer> e : m.entrySet()) e.getKey(); e.getValue();
m.keySet(); m.values();
```

A tiny example

```
Map<Character, Integer> count = new HashMap<>();
for (char c : "banana".toCharArray()) count.merge(c, 1, Integer::sum);
// {a=3, b=1, n=2}
```

Guided example

SetsMapsDemo.java — Sets and maps (demonstration)

```
1  import java.util.HashMap;
2  import java.util.HashSet;
3  import java.util.LinkedHashMap;
4  import java.util.Map;
5  import java.util.Set;
6  import java.util.TreeMap;
7  import java.util.TreeSet;
8
9  public class SetsMapsDemo {
10  public static void main(String[] args) {
11  // SET: no duplicates. HashSet = fast, unordered; TreeSet = sorted; LinkedHashSet =
        insertion order.
12  Set<Integer> seen = new HashSet<>();
13  for (int x : new int[] {3, 1, 3, 2, 1}) {
14  if (!seen.add(x)) { // add returns false when the element was already present                1
15  System.out.println("duplicate: " + x);
16  }
17  }
18  System.out.println("size " + seen.size() + " contains 2? " + seen.contains(2));
19  Set<String> sorted = new TreeSet<>(java.util.List.of("pear", "apple", "fig"));
20  System.out.println(sorted);
21
22  // MAP: key -> value. HashMap O(1) average, TreeMap sorted by key, LinkedHashMap
        insertion order.
23  String text = "to be or not to be";
24  Map<String, Integer> freq = new HashMap<>();
25  for (String w : text.split(" ")) {
26  freq.put(w, freq.getOrDefault(w, 0) + 1);                                                   2
27  }
28  System.out.println(new TreeMap<>(freq)); // print in key order                              4
29
30  Map<Character, Integer> counts = new LinkedHashMap<>();
31  for (char c : "banana".toCharArray()) {
32  counts.merge(c, 1, Integer::sum); // add 1, or insert 1                                     3
33  }
34  System.out.println(counts);
35
36  for (Map.Entry<Character, Integer> e : counts.entrySet()) {
37  System.out.print(e.getKey() + "=" + e.getValue() + ";");
38  }
39  System.out.println();
40  System.out.println("has 'n'? " + counts.containsKey('n') + ", value of 'z' = " +
        counts.get('z'));
41
42  // TWO-SUM with a map: remember what we have seen and where
43  int[] a = {2, 7, 11, 15};
44  int target = 18;
45  Map<Integer, Integer> index = new HashMap<>();
```

(code continued)

```
46    for (int i = 0; i < a.length; i++) {
47    Integer j = index.get(target - a[i]);                    5
48    if (j != null) {
49    System.out.println("two-sum: indices " + j + " and " + i);
50    }
51    index.put(a[i], i);
52    }
53    }
54   }
```

Four idioms you will reuse in many algorithm problems.

Line-by-line walkthrough

#	Where	What happens
1	line 14	add returns false when the value was already there — a free "have I seen this?" test.
2	line 26	Increment a counter, treating a missing key as 0.
3	line 32	The same in one call: insert 1, or combine the old value with 1 using sum.
4	line 28	Copy into a TreeMap to print in sorted key order.
5	line 47	Two-sum: remember where each value was seen; look up the complement in O(1).

Dry run: watch it execute

Real String.hashCode values decide the bucket.

Algorithm: Where HashMap / HashSet put a key. **Input:** keys = `cat,dog,bird,emu,ant,cow,dog`.

Pseudocode

```
h = key.hashCode()
h = h ^ (h >>> 16) // spread the high bits
index = h & (capacity - 1) // capacity = 16
bucket empty → store; same key → replace/ignore; else chain
```

Step 1 of 7. "cat": hashCode = 98262, spread = 98263, index = 98263 & 15 = 7. The bucket is empty: store the key there.

key = cat · hashCode = 98262 · index = 7 · size = 1

buckets 0–15

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
·	·	·	·	·	·	·	cat	·	·	·	·	·	·	·	·

Step 2 of 7. "dog": hashCode = 99644, spread = 99645, index = 99645 & 15 = 13. The bucket is empty: store the key there.

key = dog · hashCode = 99644 · index = 13 · size = 2

buckets 0–15

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
·	·	·	·	·	·	·	cat	·	·	·	·	·	dog	·	·

Step 3 of 7. "bird": hashCode = 3024057 → bucket 7, which already holds cat: a COLLISION, so the key is chained after it.

key = bird · hashCode = 3024057 · index = 7 · size = 3

buckets 0–15

0	1	2	3	4	5	6	7	8	9	10	11
·	·	·	·	·	·	·	cat → b	·	·	·	·

12	13	14	15
·	dog	·	·

Step 4 of 7. "emu": hashCode = 100557, spread = 100556, index = 100556 & 15 = 12. The bucket is empty: store the key there.

key = emu · hashCode = 100557 · index = 12 · size = 4



Step 5 of 7. "ant": hashCode = 96743, spread = 96742, index = 96742 & 15 = 6. The bucket is empty: store the key there.

key = ant · hashCode = 96743 · index = 6 · size = 5



Step 6 of 7. "cow": hashCode = 98699, spread = 98698, index = 98698 & 15 = 10. The bucket is empty: store the key there.

key = cow · hashCode = 98699 · index = 10 · size = 6



Step 7 of 7. "dog": hashCode = 99644, bucket 13. The key is ALREADY in that bucket (equals matches): a map would replace the value, a set ignores it.

key = dog · hashCode = 99644 · index = 13 · size = 6



> **RESULT**
> size 6; 1 collision; 1 duplicate ignored

Complexity

HashSet/HashMap: add, remove, contains/get are **O(1) average** (O(n) worst case with terrible hashing). TreeSet/TreeMap: **O(log n)**. Iterating a HashMap is O(capacity + size). Memory: O(n) with a noticeable constant per entry.

Common mistakes

Mutating a key after inserting it

Problem

```
Point p = new Point(1, 2);
set.add(p);
p.x = 9; // lookup now fails
```

Why: The hash code changed, so the entry sits in the wrong bucket.

Fix: Use immutable keys.

Expecting HashMap order

Problem

```
for (String k : map.keySet()) // assumed insertion order
```

Why: HashMap order is arbitrary and may change.

Fix: LinkedHashMap (insertion) or TreeMap (sorted).

NullPointerException on unboxing

Problem

```
int v = map.get("missing");
```

Why: get returns null; unboxing null fails.

Fix: int v = map.getOrDefault("missing", 0);

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Distinct count — *Beginner*

Read n integers and print how many distinct values there are.

Example input	Expected output
6 ⏎ 3 1 3 2 1 4	4

Hint 1: A HashSet and its size().

Exercise 2 · Word frequency — *Intermediate*

Read words until the end of input (case-insensitive). Print each distinct word and its count, one `word count` per line, alphabetically.

Example input	Expected output
the cat and The hat	and 1 ⏎ cat 1 ⏎ hat 1 ⏎ the 2

Hint 1: TreeMap<String,Integer> + merge.

Exercise 3 · Two-sum — *Challenge*

Read n, target and an array. Print the indices `i j` (i < j) of the pair with the smallest j such that a[i] + a[j] = target, or `none`. Use O(n).

Example input	Expected output
4 18 ⏎ 2 7 11 15	1 2

Hint 1: For each j look up target − a[j] among the earlier elements.

Quick check

Answers are in the answer key at the end of this part.

Q1. What does Set.add return when the element is already present?

- **A.** true
- **B.** false
- **C.** null
- **D.** throws

Q2. Which keeps keys sorted?

- **A.** HashMap
- **B.** TreeMap
- **C.** LinkedHashMap
- **D.** Hashtable

Q3. Average cost of HashMap.get?

- **A.** O(1)
- **B.** O(log n)
- **C.** O(n)
- **D.** O(n log n)

Q4. What is printed?

```
Map<String,Integer> m = new HashMap<>();
m.merge("a", 1, Integer::sum);
m.merge("a", 1, Integer::sum);
System.out.println(m.get("a"));
```

Your answer: _____

Interview / exam questions

- How does a HashMap work internally?
- HashMap vs TreeMap?
- Why must keys be immutable?
- How do you iterate a map?

Watch & read

Video: Map and HashMap in Java - Full Tutorial — Coding with John · 10:10. *HashSet, HashMap and sorted variants*

Video: Hashmap in Java | Internal Working of Hashmap in Java | Hashmap Implementation | DSA-One Course #30 — Anuj Kumar Sharma · 16:39. *How HashMap works internally*

REVISION CARD (30–60 SECONDS)

- Set = unique; Map = key → value.
- Hash O(1) avg unordered; Tree O(log n) sorted; LinkedHash insertion.
- Count: merge(k, 1, Integer::sum) / getOrDefault.
- Keys need equals + hashCode and should not change.

3.10 Queues, Deques, Stacks and PriorityQueue

Object-Oriented Programming · Beginner · Core programming & data structures · about 35 min · ★ on the minimum-viable path

FIFO and LIFO structures in Java: `Queue`, `ArrayDeque` as a stack, and the min-heap `PriorityQueue`.

YOU WILL BE ABLE TO

- Use `Queue` (FIFO) with `offer`, `poll`, `peek` and `Deque`/`ArrayDeque` as a stack (LIFO).
- Explain why `java.util.Stack` should be avoided.
- Use `PriorityQueue` as a min-heap and max-heap and as "k largest" tool.
- Solve bracket matching with a stack and the Josephus order with a queue.

Before this lesson: Generics and Lists: ArrayList and LinkedList

The idea

structure	rule	real life	Java
Queue	first in, first out	a line at a counter	Queue<E> q = new ArrayDeque<>(); offer / poll / peek
Stack	last in, first out	a pile of plates; undo	Deque<E> s = new ArrayDeque<>(); push / pop / peek
Deque	both ends	a queue you may also join at the front	addFirst / addLast / pollFirst / pollLast
PriorityQueue	smallest (or best) first	emergency room	new PriorityQueue<>(); poll() = minimum

WHY THIS WORKS

A stack naturally matches **nesting** (brackets, recursion, undo): the most recently opened item must be closed first. A queue matches **fairness and level-by-level exploration** (BFS in the maze lesson: the cells found first are expanded first). A priority queue is a **binary heap**: insert and remove-min are O(log n), peek is O(1) — the basis of Dijkstra's algorithm and of "keep the k best".

WHICH CLASS?

Use `ArrayDeque` for stacks and queues. The old `java.util.Stack` class is synchronised, inherits from Vector and is slower; the `Queue` methods that **return a special value** (`offer`, `poll`, `peek`) are safer than the ones that throw (`add`, `remove`, `element`).

Syntax

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); stack.push(2); stack.peek(); stack.pop(); // 2, then 2 removed
Queue<Integer> queue = new ArrayDeque<>();
queue.offer(1); queue.offer(2); queue.peek(); queue.poll(); // 1
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
```

A tiny example

```
Deque<Character> st = new ArrayDeque<>();
st.push('(');
char top = st.pop();
```

Guided example

QueueDequeDemo.java — Queue, stack, deque, heap (demonstration)

```
1  import java.util.ArrayDeque;
2  import java.util.Collections;
3  import java.util.Deque;
4  import java.util.LinkedList;
5  import java.util.PriorityQueue;
6  import java.util.Queue;
7
8  public class QueueDequeDemo {
9   // balanced brackets with a stack
10  static boolean balanced(String s) {
11  Deque<Character> stack = new ArrayDeque<>();
12  for (char c : s.toCharArray()) {
13  if (c == '(' || c == '[' || c == '{') {
14  stack.push(c);
15  } else if (c == ')' || c == ']' || c == '}') {
16  if (stack.isEmpty()) return false;                                    5
17  char open = stack.pop();
18  if ((c == ')' && open != '(') || (c == ']' && open != '[') || (c == '}' && open != '{'))
        return false;
19  }
20  }
21  return stack.isEmpty();                                               6
22  }
23
24  public static void main(String[] args) {
25  // QUEUE: first in, first out. offer = add at the back, poll = remove from the front.
26  Queue<String> line = new LinkedList<>();
27  line.offer("Asha");                                                   1
28  line.offer("Ravi");
29  line.offer("Mei");
30  System.out.println(line.peek() + " is first; served: " + line.poll() + ", then " +   2
        line.poll());
31
32  // STACK: last in, first out. Use ArrayDeque (java.util.Stack is legacy).
33  Deque<Integer> stack = new ArrayDeque<>();
34  stack.push(1);                                                        3
35  stack.push(2);
36  stack.push(3);
37  System.out.println("top " + stack.peek() + ", pop " + stack.pop() + ", pop " +
        stack.pop());
38
39  // DEQUE: both ends
40  Deque<Integer> dq = new ArrayDeque<>();
41  dq.addFirst(2);
42  dq.addLast(3);
43  dq.addFirst(1);
44  System.out.println(dq + " first=" + dq.peekFirst() + " last=" + dq.peekLast());
45
```

(code continued)

```
46    // PRIORITY QUEUE: poll() returns the SMALLEST (min-heap) unless a comparator says
          otherwise
47    PriorityQueue<Integer> pq = new PriorityQueue<>();
48    for (int x : new int[] {5, 1, 4, 2}) pq.offer(x);
49    StringBuilder sb = new StringBuilder();
50    while (!pq.isEmpty()) sb.append(pq.poll()).append(' ');
51    System.out.println("min-heap order: " + sb.toString().trim());
52    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());          4
53    maxHeap.addAll(java.util.List.of(5, 1, 4, 2));
54    System.out.println("max-heap top: " + maxHeap.peek());
55
56    System.out.println(balanced("{[()]}") + " " + balanced("([)]") + " " + balanced("(("));
57    }
58  }
```

Each structure used once, then a bracket-matching function.

Line-by-line walkthrough

#	Where	What happens
1	line 27	Add at the back of the queue.
2	line 30	Remove and return the front (null if empty).
3	line 34	Push adds at the FRONT of the deque; pop removes from the front — LIFO.
4	line 52	A max-heap: the comparator reverses the order.
5	line 16	A closing bracket with nothing open is unbalanced.
6	line 21	Anything left open at the end is unbalanced too.

Dry run: watch it execute

The same adds and removes applied to a stack and to a queue.

Algorithm: Stack (LIFO) vs queue (FIFO). **Input:** ops = `a b c - - d -`.

Pseudocode

```
for each token:
 letter: stack.push(x); queue.offer(x)
 "-": stack.pop(); queue.poll()
```

Step 1 of 7. Add a to both.

token = a

stack (top = last)

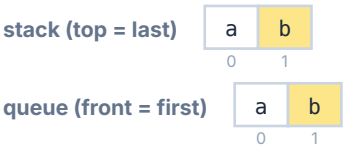


queue (front = first)



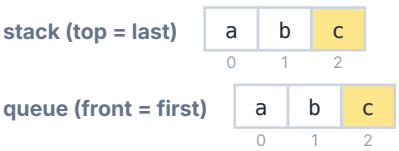
Step 2 of 7. Add b to both.

token = b



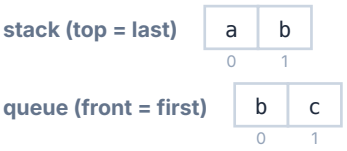
Step 3 of 7. Add c to both.

token = c



Step 4 of 7. pop() removes the NEWEST item c; poll() removes the OLDEST item a.

token = - · stack.pop = c · queue.poll = a



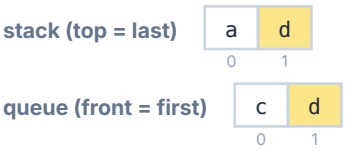
Step 5 of 7. pop() removes the NEWEST item b; poll() removes the OLDEST item b.

token = - · stack.pop = b · queue.poll = b



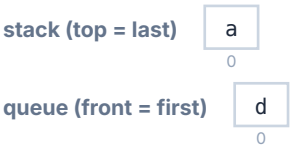
Step 6 of 7. Add d to both.

token = d



Step 7 of 7. pop() removes the NEWEST item d; poll() removes the OLDEST item c.

token = - · stack.pop = d · queue.poll = c



> **RESULT**
> removed (stack/queue): c/a, b/b, d/c

Complexity

operation	ArrayDeque	PriorityQueue
add / push / offer	O(1) amortised	O(log n)
poll / pop	O(1)	O(log n)
peek	O(1)	O(1)
contains	O(n)	O(n)

Common mistakes

Iterating a PriorityQueue and expecting sorted order

Problem

```
for (int x : pq) System.out.println(x);
```

Why: The iterator walks the heap array, not sorted order.

Fix: while (!pq.isEmpty()) pq.poll();

Using Stack/Vector

Problem

```
Stack<Integer> s = new Stack<>();
```

Why: Legacy and slower.

Fix: Deque<Integer> s = new ArrayDeque<>();

null elements

Problem

```
ArrayDeque<String> d = ...; d.add(null);
```

Why: ArrayDeque forbids null (poll() uses null to mean empty).

Fix: Use a sentinel value or Optional.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Balanced brackets — *Beginner*

Read a string of brackets ()[]{}. Print YES if balanced, else NO.

Example input	Expected output
{[()]}	YES

Hint 1: Push openers; on a closer pop and compare.

Exercise 2 · Josephus with a queue — *Intermediate*

Read n and k. People 1..n stand in a queue; repeatedly move k−1 people to the back and remove the next. Print the removal order.

Example input	Expected output
5 2	2 4 1 5 3

Hint 1: q.add(q.poll()) k−1 times, then poll.

Exercise 3 · k-th largest — *Challenge*

Read n, k and an array. Print the k-th largest element using a min-heap of size k (O(n log k)).

Example input	Expected output
6 2 ⏎ 3 2 1 5 6 4	5

Hint 1: Add each element; if size > k, poll the minimum.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
Deque<Integer> s = new ArrayDeque<>();
s.push(1); s.push(2); s.push(3);
System.out.println(s.pop() + " " + s.peek());
```

Your answer: _____

Q2. Which data structure is best for BFS?

- **A.** stack
- **B.** queue
- **C.** priority queue
- **D.** set

Q3. PriorityQueue<Integer>.poll() returns...

- **A.** the first inserted
- **B.** the largest
- **C.** the smallest
- **D.** a random one

Q4. Insert into a PriorityQueue costs...

- **A.** O(1)
- **B.** O(log n)
- **C.** O(n)
- **D.** O(n log n)

Interview / exam questions

- Stack vs queue?
- Implement a queue with two stacks.
- How does a PriorityQueue work?
- Why not java.util.Stack?

Watch & read

Video: #13 Queue Implementation using Java Part 1 | EnQueue — Telusko · 9:38. *Queue, Deque, Stack and PriorityQueue*

Video: Learn Priority Queue data structures in 5 minutes — Bro Code · 5:01. *PriorityQueue as a min-heap with a comparator*

REVISION CARD (30–60 SECONDS)

- Queue FIFO offer/poll/peek; Stack LIFO push/pop/peek on ArrayDeque.
- PriorityQueue = min-heap, O(log n) add/poll; reverseOrder for max.
- Brackets → stack; BFS → queue; top-k → heap.
- Avoid java.util.Stack.

3.11 Sorting with Comparable and Comparator

Object-Oriented Programming · Intermediate · Core programming & data structures · about 35 min · ★ on the minimum-viable path

Order objects any way you like: natural order, comparator lambdas, multi-key sorts and stability.

YOU WILL BE ABLE TO

- Sort lists and arrays with `Collections.sort`, `List.sort` and `Arrays.sort`.
- Implement `Comparable<T>` and write a `Comparator<T>` as a lambda.
- Chain keys with `thenComparing` and reverse with `reversed()`.
- Explain stability and why a comparator must be consistent.

Before this lesson: Generics and Lists: ArrayList and LinkedList · Abstract Classes and Interfaces

The idea

Sorting needs a rule for "which of two items comes first?". Java expresses it as a number: **negative** = first goes first, **zero** = tie, **positive** = second goes first.

tool	where defined	use for
Comparable<T>.compareTo	inside the class	the single natural order (String, Integer, Version)
Comparator<T>.compare	outside the class	any other order; many per class

WHY THIS WORKS

Because a Comparator is a small function, you can build orders by composition: `Comparator.comparingInt((Student s) -> s.marks).reversed().thenComparing(s -> s.age)` reads as "highest marks first, ties by younger age". The library sorts (TimSort for objects, dual-pivot quicksort for primitives) in **O(n log n)**; object sorts are **stable** — items that compare equal keep their original relative order — so sorting by one key and then another works as expected.

A comparator must be a consistent total order: `compare(a, b) = −compare(b, a)` and transitivity. Do **not** compute it as `a - b` for ints: the subtraction can overflow. Use `Integer.compare(a, b)`.

Syntax

```
Collections.sort(list); // natural order
list.sort(Comparator.comparing(Student::getName));
list.sort((a, b) -> Integer.compare(b.marks, a.marks)); // marks descending
Arrays.sort(arr, Collections.reverseOrder()); // Integer[] only
Arrays.sort(pairs, (x, y) -> Integer.compare(x[0], y[0]));
```

A tiny example

```
List<String> w = new ArrayList<>(List.of("pear", "fig", "apple"));
w.sort(Comparator.comparingInt(String::length));
// [fig, pear, apple]
```

Guided example

SortingDemo.java — Comparators in practice (demonstration)

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

public class SortingDemo {
 static class Student implements Comparable<Student> {
 final String name;
 final int marks;
 final int age;

 Student(String name, int marks, int age) {
 this.name = name;
 this.marks = marks;
 this.age = age;
 }

 // NATURAL ORDER: by name. A negative result means "this comes first".
 @Override
 public int compareTo(Student o) {                                    (1)
 return name.compareTo(o.name);
 }

 @Override
 public String toString() {
 return name + ":" + marks + ":" + age;
 }
 }

 public static void main(String[] args) {
 List<Student> list = new ArrayList<>(List.of(
 new Student("Mei", 71, 20), new Student("Asha", 90, 19),
 new Student("Ravi", 71, 18), new Student("Zoe", 55, 21)));

 Collections.sort(list); // uses compareTo
 System.out.println("by name : " + list);

 list.sort(Comparator.comparingInt((Student s) -> s.marks).reversed()); // marks   (2)
     descending
 System.out.println("by marks desc: " + list); // stable: Mei before Ravi (both 71)

 list.sort(Comparator.comparingInt((Student s) -> s.marks).reversed().thenComparing(s ->   (3)
     s.age));
 System.out.println("marks desc, then age: " + list);

 // arrays
 Integer[] boxed = {5, 2, 9, 1};
 Arrays.sort(boxed, Collections.reverseOrder());                      (4)
 System.out.println(Arrays.toString(boxed));
 String[] words = {"banana", "fig", "apple", "kiwi"};
 Arrays.sort(words, Comparator.comparing(String::length)
     .thenComparing(Comparator.naturalOrder()));
 System.out.println(Arrays.toString(words));

```

(code continued)

```
53    // a comparator written the long way: negative, zero, positive
54    Comparator<int[]> byFirstThenSecond = (x, y) -> x[0] != y[0] ? Integer.compare(x[0],  5
          y[0]) : Integer.compare(x[1], y[1]);
55    int[][] pairs = {{2, 5}, {1, 9}, {2, 1}};
56    Arrays.sort(pairs, byFirstThenSecond);
57    System.out.println(Arrays.deepToString(pairs));
58    }
59  }
```

Students sorted by name, by marks (stable), then by marks and age.

Line-by-line walkthrough

#	Where	What happens
1	line 21	The natural order: by name. String.compareTo already returns negative/zero/positive.
2	line 39	Order by marks, then flip it to descending.
3	line 42	Tie-breaker: when marks are equal, younger first.
4	line 47	Arrays.sort with a comparator needs object elements (Integer[], not int[]).
5	line 54	A hand-written comparator for int[] pairs; never use subtraction.

Dry run: watch it execute

The sort just keeps asking compare(a, b).

Algorithm: Sorting with a comparator. **Input:** words = `pear,fig,apple,kiwi,plum`, mode = `2`.

Pseudocode

```
compare(a, b):
 1: a.compareTo(b)
 2: Integer.compare(a.length(), b.length()), then a.compareTo(b)
 3: b.compareTo(a)
insertion sort using compare
```

Step 1 of 9. The list before sorting.

comparator = length, then natural

	0	1	2	3	4
words	pear	fig	apple	kiwi	plum

Step 2 of 9. compare("pear", "fig") = positive → "pear" must come after "fig": shift it right.

compare result = 1 · key = fig · j = 0

	0	1	2	3	4
words	pear	fig	apple	kiwi	plum

Step 3 of 9. compare("pear", "apple") = negative → "pear" stays before "apple".

compare result = -1 · key = apple · j = 1

	0	1	2	3	4
words	fig	pear	apple	kiwi	plum

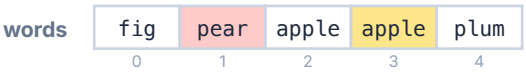
Step 4 of 9. compare("apple", "kiwi") = positive → "apple" must come after "kiwi": shift it right.

compare result = 1 · key = kiwi · j = 2

	0	1	2	3	4
words	fig	pear	apple	kiwi	plum

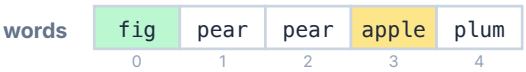
Step 5 of 9. compare("pear", "kiwi") = positive → "pear" must come after "kiwi": shift it right.

compare result = 1 · key = kiwi · j = 1



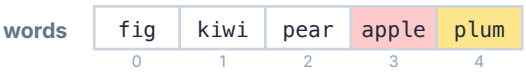
Step 6 of 9. compare("fig", "kiwi") = negative → "fig" stays before "kiwi".

compare result = -1 · key = kiwi · j = 0



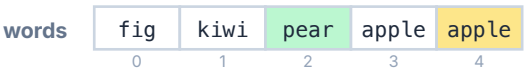
Step 7 of 9. compare("apple", "plum") = positive → "apple" must come after "plum": shift it right.

compare result = 1 · key = plum · j = 3

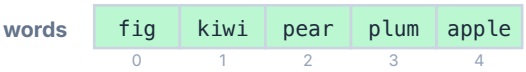


Step 8 of 9. compare("pear", "plum") = negative → "pear" stays before "plum".

compare result = -1 · key = plum · j = 2



Step 9 of 9. Sorted.



> **RESULT**
> fig, kiwi, pear, plum, apple

Complexity

Sorting n items takes O(n log n) comparisons; each comparison costs the comparator's time (O(1) for primitive keys, O(L) for strings of length L). TimSort uses O(n) extra space and is O(n) on already-sorted input.

Common mistakes

Comparator by subtraction

Problem

```
(a, b) -> a - b
```

Why: Overflows for large/negative values: e.g. Integer.MIN_VALUE − 1.

Fix: Integer.compare(a, b)

Sorting a primitive array with a comparator

Problem

```
Arrays.sort(intArray, Collections.reverseOrder());
```

Why: Compile error: comparators need object arrays.

Fix: Use Integer[] or sort ascending and reverse.

Inconsistent comparator

Problem

```
(a, b) -> a.x < b.x ? -1 : 1 // never 0
```

Why: Violates the contract (equal items compare as "greater" both ways): "Comparison method violates its general contract!"

Fix: Return 0 for ties: use Integer.compare.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Descending — *Beginner*

Read n integers and print them in descending order.

Example input	Expected output
`5 ⏎ 3 9 1 7 5`	`9 7 5 3 1`

Hint 1: Collections.reverseOrder()

Exercise 2 · By length then alphabet — *Intermediate*

Read n words and print them sorted by length, ties alphabetically.

Example input	Expected output
`5 ⏎ pear fig apple kiwi plum`	`fig kiwi pear plum apple`

Hint 1: comparingInt(String::length).thenComparing(...)

Exercise 3 · Merge intervals — *Challenge*

Read n intervals `start end`. Merge overlapping ones (touching counts) and print them as `[s,e]` separated by spaces in increasing order.

Example input	Expected output
`4 ⏎ 1 3 ⏎ 8 10 ⏎ 2 6 ⏎ 15 18`	`[1,6] [8,10] [15,18]`

Hint 1: Sort by start, then sweep keeping the current merged interval.

Quick check

Answers are in the answer key at the end of this part.

Q1. A comparator returning a negative number means...

- **A.** a goes before b
- **B.** b goes before a
- **C.** they are equal
- **D.** an error

Q2. Why is `(a, b) -> a - b` dangerous?

- **A.** it is slower
- **B.** integer overflow
- **C.** it sorts descending
- **D.** it does not compile

Q3. A stable sort...

- **A.** never crashes
- **B.** keeps equal elements in their original order
- **C.** is always O(n)
- **D.** uses no memory

Q4. Time complexity of Collections.sort on n items?

- **A.** O(n)
- **B.** O(n log n)
- **C.** O(n^2)
- **D.** O(log n)

Interview / exam questions

- Comparable vs Comparator?
- What is a stable sort and why does it matter?
- How does Java sort primitives vs objects?
- How do you sort by two keys?

Watch & read

Video: #95 Comparator vs Comparable in Java — Telusko · 15:43. *Sorting objects with Comparable and Comparator*

Video: Java Custom Lambda Expression In Arrays.sort and Collections.sort? — Pepcoding · 34:46. *Sorting with lambdas and comparators*

REVISION CARD (30–60 SECONDS)

- compare → negative / 0 / positive.
- Comparable: natural; Comparator: any order; thenComparing, reversed.
- Use Integer.compare, never a − b.
- Object sorts are stable, O(n log n).

3.12 equals(), hashCode() and Immutability

Object-Oriented Programming · Intermediate · Core programming & data structures · about 35 min · complete path

When two objects count as "the same", why hash-based collections need both methods, and how to build immutable classes.

YOU WILL BE ABLE TO

- Override `equals` and `hashCode` consistently and explain the contract.
- Explain what goes wrong in a `HashSet` if you override only one.
- Build an immutable class (final fields, no setters, defensive copies).
- Spot common object-related errors: `==` on objects, mutable keys, leaked references.

Before this lesson: Sets and Maps: HashSet, TreeSet, HashMap, TreeMap · Encapsulation and Access Modifiers

The idea

By default `a.equals(b)` is the same as `a == b`: **identity** (same object). For value-like classes (a `Point`, a `Fraction`) you want **equality of content**, so you override `equals`.

```
The contract:
 • a.equals(a) is true; symmetric; transitive; consistent
 • equal objects MUST have equal hashCode() ← the one people forget
 • unequal objects MAY share a hash code (a collision)
```

WHY THIS WORKS

A HashSet first computes `hashCode()` to choose a **bucket**, then uses `equals()` to compare inside that bucket. If you override `equals` but not `hashCode`, two equal points land in different buckets and the set treats them as different — the exact bug the demo shows (`raw set size 2`). Using `Objects.hash(x, y)` and `Objects.equals` makes both methods short and correct.

Immutability

- Declare the class `final` (or all constructors private).
- All fields `private final`; no setters.
- Copy mutable inputs in the constructor and mutable outputs in getters (defensive copies), or return unmodifiable views.
- Methods return **new** objects (`String.concat`, `BigInteger.add`).

Immutable objects are safe to share, safe as map keys, and trivially thread-safe — that is why `String`, `Integer` and `BigInteger` are immutable.

Syntax

```java
@Override public boolean equals(Object o) {
 if (this == o) return true;
 if (!(o instanceof Point)) return false;
 Point p = (Point) o;
 return x == p.x && y == p.y;
}
@Override public int hashCode() { return Objects.hash(x, y); }
```

A tiny example

```
new Point(1, 2).equals(new Point(1, 2)); // false without an override, true with it
```

Guided example

EqualsHashCodeDemo.java — Value equality (demonstration)

```
import java.util.HashSet;
import java.util.Objects;
import java.util.Set;

public class EqualsHashCodeDemo {
 // WITHOUT equals/hashCode: two logically equal points are different objects.
 static class RawPoint {                                                    (1)
 final int x, y;
 RawPoint(int x, int y) { this.x = x; this.y = y; }
 }

 // WITH equals + hashCode: value-based identity. The contract: equal objects MUST have
     equal hash codes.
 static final class Point {                                                 (2)
 private final int x, y; // final + no setters = immutable

 Point(int x, int y) {
 this.x = x;
 this.y = y;
 }

 @Override
 public boolean equals(Object o) {
 if (this == o) return true;
 if (!(o instanceof Point)) return false;                                   (3)
 Point p = (Point) o;
 return x == p.x && y == p.y;
 }

 @Override
 public int hashCode() {
 return Objects.hash(x, y);                                                 (4)
 }

 @Override
 public String toString() {
 return "(" + x + "," + y + ")";
 }
 }

 public static void main(String[] args) {
 RawPoint r1 = new RawPoint(1, 2);
 RawPoint r2 = new RawPoint(1, 2);
 System.out.println("raw equals? " + r1.equals(r2)); // false: compares references

 Point p1 = new Point(1, 2);
 Point p2 = new Point(1, 2);
 System.out.println("point equals? " + p1.equals(p2) + ", same hash? " + (p1.hashCode()
     == p2.hashCode()));

```

(code continued)

```
49    Set<RawPoint> rawSet = new HashSet<>();
50    rawSet.add(r1);
51    rawSet.add(r2);
52    Set<Point> set = new HashSet<>();
53    set.add(p1);
54    set.add(p2);
55    System.out.println("raw set size " + rawSet.size() + ", point set size " + set.size());  5
56    System.out.println("contains (1,2)? " + set.contains(new Point(1, 2)));
57
58    // String is immutable too: methods return new objects
59    String s = "abc";
60    String t = s.concat("d");
61    System.out.println(s + " " + t);
62    }
63   }
```

RawPoint vs Point stored in HashSets.

Line-by-line walkthrough

#	Where	What happens
1	line 7	No overrides: equality is identity, so two (1,2) points are different.
2	line 13	final class with private final fields: immutable.
3	line 24	Guards against null and unrelated types in one test.
4	line 31	Same fields as equals → equal objects get equal hash codes.
5	line 55	raw: 2 entries; value-equal Points collapse to 1.

Dry run: watch it execute

Overriding equals without hashCode breaks HashSet.

Algorithm: HashSet<Point> and the equals/hashCode contract. **Input:** mode = `2`, pts = `1:2,1:2,3:4`.

Pseudocode

```
set.add(p):
 h = p.hashCode() → bucket
 only entries with the SAME hash are compared with equals()
 equal → already present, not added
```

Step 1 of 4. Point overrides equals but NOT hashCode (the contract is broken).

equals = true · hashCode = false

Step 2 of 4. add(Point(1,2)): hash 100000 → bucket 1. No entry with this hash exists, so equals() is never even called: added.

hash = 100000 · bucket = 1 · size = 1

set contents `(1,2) h=100000`
0

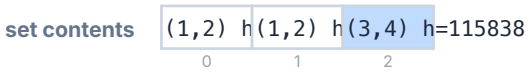
Step 3 of 4. add(Point(1,2)): hash 107919 → bucket 14. No entry with this hash exists, so equals() is never even called: added. (An equal point is already stored in another bucket, but nobody looked there!)

hash = 107919 · bucket = 14 · size = 2

set contents `(1,2) h` `(1,2) h=107919`
0 1

Step 4 of 4. add(Point(3,4)): hash 115838 → bucket 15. No entry with this hash exists, so equals() is never even called: added.

hash = 115838 · bucket = 15 · size = 3



> **RESULT**
> size = 3 (logically equal points are counted twice)

Complexity

A good hashCode makes HashSet/HashMap operations O(1) average; a constant hashCode degrades them to O(n). equals/hashCode themselves are O(number of fields).

Common mistakes

Overriding equals without hashCode

Problem

```
@Override public boolean equals(Object o) { ... }
```

Why: HashSet/HashMap look in the wrong bucket; contains() returns false for equal objects.

Fix: Always override both, using the same fields.

Wrong signature

Problem

```
public boolean equals(Point p) { ... }
```

Why: That OVERLOADS equals(Object); collections still call the Object version. @Override would have caught it.

Fix: public boolean equals(Object o)

Leaking a mutable field

Problem

```
public List<String> getMembers() { return members; }
```

Why: The "immutable" class can be changed from outside.

Fix: return Collections.unmodifiableList(members);

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Distinct people — *Intermediate*

Create a Person(name, age) class with equals/hashCode. Read n lines `name age` and print the number of distinct persons.

Example input	Expected output
4 ⏎ Asha 20 ⏎ Ravi 21 ⏎ Asha 20 ⏎ Asha 22	3

Hint 1: Put Persons in a HashSet.

Exercise 2 · Immutable money — *Intermediate*

Run the Money class whose plus() returns a new object; print the amounts of a and b = a.plus(175¢) for a = 250¢.

Example input	Expected output
(no input)	250 425

Hint 1: The original object is unchanged.

Exercise 3 · Defensive copying — *Challenge*

A Team stores a copy of the member list given to its constructor and returns an unmodifiable view. Run it: the source list is changed afterwards (the team must not change) and a caller tries to add to the returned list.

Example input	Expected output
(no input)	[Asha, Ravi] ⏎ read-only

Hint 1: new ArrayList<>(members) in; Collections.unmodifiableList out.

Quick check

Answers are in the answer key at the end of this part.

Q1. If a.equals(b) is true then...

- **A.** a.hashCode() == b.hashCode() must hold
- **B.** hash codes are unrelated
- **C.** a == b must be true
- **D.** they must be the same class only

Q2. Which makes a class immutable?

- **A.** public fields
- **B.** private final fields and no setters
- **C.** static methods
- **D.** a toString method

Q3. What does a HashSet<Point> print for size if equals is overridden but hashCode is not?

```
set.add(new Point(1, 2));
set.add(new Point(1, 2));
System.out.println(set.size());
```

Your answer: _____

Q4. Which of these classes is immutable?

- **A.** StringBuilder
- **B.** ArrayList
- **C.** String
- **D.** int[]

Interview / exam questions

- Why override hashCode with equals?
- What is immutability and its benefits?
- == vs equals?
- How does HashMap find an entry?

Watch & read

Video: 02. Equals and HashCode Contract & Different Variations — Ankit Wasankar · 11:19. *The equals/hashCode contract*

Video: Immutable Classes and Objects in Java — Neso Academy · 5:06. *Designing immutable classes*

REVISION CARD (30–60 SECONDS)

- Override equals AND hashCode with the same fields.
- Hash → bucket; equals → match inside bucket.
- Immutable: final class, private final fields, no setters, defensive copies.
- Use Objects.hash / Objects.equals.

Part 3 — Solutions to the exercises

Try the exercises before reading these. Each solution is the exact file that was compiled and tested.

3.1 Classes, Objects, Fields, Constructors and this

Exercise 1 · Rectangle class

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static class Rectangle {
 double width, height;

 Rectangle(double width, double height) {
 this.width = width;
 this.height = height;
 }

 double area() { return width * height; }
 double perimeter() { return 2 * (width + height); }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 Rectangle r = new Rectangle(sc.nextDouble(), sc.nextDouble());
 System.out.println(r.area() + " " + r.perimeter());
 }
}
```

Why it works: Fields + constructor + two methods that use the fields.

Verified against 2 test cases: `2.5 4⏎` → `10.0 13.0`; `1 1⏎` → `1.0 4.0`

Exercise 2 · Counter

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static class Counter {
 private int count;

 void increment() { count++; }
 void add(int k) { count += k; }
 int get() { return count; }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 Counter c = new Counter();
 for (int i = 0; i < n; i++) {
 String op = sc.next();
 if (op.equals("inc")) c.increment();
 else c.add(sc.nextInt());
 }
 System.out.println(c.get());
 }
}
```

Why it works: State in a field changed only through methods.

Verified against 2 test cases: `4⏎inc⏎add 5⏎inc⏎add 10⏎` → `17`; `1⏎inc⏎` → `1`

Exercise 3 · Distance between points

Ex3.java

```
import java.util.Scanner;

public class Ex3 {
 static class Point {
 final int x, y;

 Point(int x, int y) {
 this.x = x;
 this.y = y;
 }

 double distanceTo(Point other) {
 int dx = x - other.x;
 int dy = y - other.y;
 return Math.sqrt(dx * dx + dy * dy);
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 Point a = new Point(sc.nextInt(), sc.nextInt());
 Point b = new Point(sc.nextInt(), sc.nextInt());
 System.out.printf("%.3f%n", a.distanceTo(b));
 }
}
```

Why it works: A method that takes another object of the same class.

Verified against 3 test cases: `0 0 3 4⏎` → `5.000`; `1 1 1 1⏎` → `0.000`; `-1 -1 2 3⏎` → `5.000`

3.2 Encapsulation and Access Modifiers

Exercise 1 · Validated temperature

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static class Temperature {
 private double celsius;

 void setCelsius(double c) {
 if (c < -273.15) throw new IllegalArgumentException("below absolute zero");
 celsius = c;
 }

 double getCelsius() { return celsius; }
 double getFahrenheit() { return celsius * 9 / 5 + 32; }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 Temperature t = new Temperature();
 try {
 t.setCelsius(sc.nextDouble());
 System.out.printf("%.1f%n", t.getFahrenheit());
 } catch (IllegalArgumentException e) {
 System.out.println("invalid: " + e.getMessage());
 }
 }
}
```

Why it works: The setter protects the invariant.

Verified against 3 test cases: `36.6⏎` → `97.9`; `-300⏎` → `invalid: below absolute zero`; `100⏎` → `212.0`

Exercise 2 · Bounded stack

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static class Stack {
 private final int[] data;
 private int size;

 Stack(int capacity) { data = new int[capacity]; }

 boolean push(int x) {
 if (size == data.length) return false;
 data[size++] = x;
 return true;
 }

 int pop() {
 if (size == 0) throw new IllegalStateException("empty");
 return data[--size];
 }

 boolean isEmpty() { return size == 0; }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 Stack s = new Stack(3);
 int n = sc.nextInt();
 StringBuilder out = new StringBuilder();
 for (int i = 0; i < n; i++) {
 String op = sc.next();
 if (op.equals("push")) {
 out.append(s.push(sc.nextInt()) ? "ok " : "full ");
 } else {
 out.append(s.isEmpty() ? "empty " : s.pop() + " ");
 }
 }
 System.out.println(out.toString().trim());
 }
}
```

Why it works: The array is hidden: callers cannot corrupt size.

Verified against 2 test cases: `5⏎push 1⏎push 2⏎pop⏎pop⏎pop⏎` → `ok ok 2 1 empty`; `5⏎push 1⏎push 2⏎push 3⏎push 4⏎pop⏎` → `ok ok ok full 3`

Exercise 3 · Access levels

Ex3.java

```java
public class Ex3 {
 // Which members are accessible? A tiny demonstration with output you can predict.
 static class Vault {
 private int secret = 42;
 int packageLevel = 7;
 protected int family = 3;
 public int open = 1;

 int reveal() { return secret; } // inside the class: private is accessible
 }

 public static void main(String[] args) {
 Vault v = new Vault();
 // v.secret would be fine here only because Vault is a nested class of Ex3; the lesson explains the rule.
 System.out.println(v.reveal() + " " + v.packageLevel + " " + v.family + " " + v.open);
 }
}
```

Why it works: Nested classes share access with their enclosing class; across top-level classes in different packages only public (and protected for subclasses) would be visible.

Verified against 1 test case: `(nc input)` → `42 7 3 1`

3.3 Inheritance, Overriding and Polymorphism

Exercise 1 · Animal sounds

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static class Animal {
 String sound() { return "..."; }
 }

 static class Dog extends Animal {
 @Override String sound() { return "Woof"; }
 }

 static class Cat extends Animal {
 @Override String sound() { return "Meow"; }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) {
 String kind = sc.next();
 Animal a = kind.equals("dog") ? new Dog() : kind.equals("cat") ? new Cat() : new Animal();
 sb.append(sb.length() == 0 ? "" : " ").append(a.sound());
 }
 System.out.println(sb);
 }
}
```

Why it works: Polymorphic call: the object's class decides the output.

Verified against 2 test cases: `3⏎dog⏎cat⏎fish⏎` → `Woof Meow ...`; `1⏎cat⏎` → `Meow`

Exercise 2 · Payroll

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static class Employee {
 final String name;
 final double base;

 Employee(String name, double base) {
 this.name = name;
 this.base = base;
 }

 double pay() { return base; }
 }

 static class Manager extends Employee {
 Manager(String name, double base) { super(name, base); }

 @Override double pay() { return base * 1.5; } // 50% bonus
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 double total = 0;
 for (int i = 0; i < n; i++) {
 String role = sc.next();
 String name = sc.next();
 double base = sc.nextDouble();
 Employee e = role.equals("manager") ? new Manager(name, base) : new Employee(name, base);
 total += e.pay();
 }
 System.out.printf("%.2f%n", total);
 }
}
```

Why it works: The same loop works for every employee type.

Verified against 2 test cases: `2⏎manager Ann 1000⏎staff Bob 800⏎` → `2300.00`; `1⏎staff Z 10⏎` → `10.00`

Exercise 3 · Predict the order

Ex3.java

```java
public class Ex3 {
 // Predict the output: constructors run parent-first; the overridden method is chosen by the
     OBJECT's type.
 static class A {
 A() { System.out.println("A()"); }
 String who() { return "A"; }
 }

 static class B extends A {
 B() { System.out.println("B()"); }
 @Override String who() { return "B"; }
 }

 public static void main(String[] args) {
 A x = new B();
 System.out.println(x.who());
 }
}
```

Why it works: new B() runs A() then B(); who() is dispatched on the actual object (B).

Verified against 1 test case: `(nc input)` → `A()⏎B()⏎B`

3.4 Abstract Classes and Interfaces

Exercise 1 · Total area

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 interface Shape {
 double area();
 }

 static class Circle implements Shape {
 final double r;
 Circle(double r) { this.r = r; }
 public double area() { return Math.PI * r * r; }
 }

 static class Square implements Shape {
 final double s;
 Square(double s) { this.s = s; }
 public double area() { return s * s; }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 double total = 0;
 for (int i = 0; i < n; i++) {
 String kind = sc.next();
 double v = sc.nextDouble();
 Shape s = kind.equals("circle") ? new Circle(v) : new Square(v);
 total += s.area();
 }
 System.out.printf("%.2f%n", total);
 }
}
```

Why it works: The loop sees only Shape; each object supplies its own area.

Verified against 2 test cases: `3⏎circle 1⏎square 2⏎circle 2⏎` → `19.71`; `1⏎square 3⏎` → `9.00`

Exercise 2 · Abstract Vehicle

Ex2.java

```
import java.util.Scanner;

public class Ex2 {
 static abstract class Vehicle {
 abstract int wheels();

 String describe() {
 return getClass().getSimpleName() + " has " + wheels() + " wheels";
 }
 }

 static class Bike extends Vehicle { int wheels() { return 2; } }
 static class Car extends Vehicle { int wheels() { return 4; } }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 Vehicle v = sc.next().equals("bike") ? new Bike() : new Car();
 System.out.println(v.describe());
 }
}
```

Why it works: Shared code in the abstract class, variable part in the subclass.

Verified against 2 test cases: `bike⏎` → `Bike has 2 wheels`; `car⏎` → `Car has 4 wheels`

Exercise 3 · Comparable versions

Ex3.java

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Scanner;

public class Ex3 {
 // Implementing Comparable<T> gives a class a natural ordering usable by sort().
 static class Version implements Comparable<Version> {
 final int major, minor;

 Version(String s) {
 String[] p = s.split("\\.");
 major = Integer.parseInt(p[0]);
 minor = Integer.parseInt(p[1]);
 }

 @Override
 public int compareTo(Version o) {
 if (major != o.major) return Integer.compare(major, o.major);
 return Integer.compare(minor, o.minor);
 }

 @Override
 public String toString() {
 return major + "." + minor;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<Version> list = new ArrayList<>();
 for (int i = 0; i < n; i++) list.add(new Version(sc.next()));
 Collections.sort(list);
 StringBuilder sb = new StringBuilder();
 for (Version v : list) sb.append(sb.length() == 0 ? "" : " ").append(v);
 System.out.println(sb);
 }
}
```

Why it works: compareTo defines the natural order used by Collections.sort.

Verified against 2 test cases: `4⏎1.10⏎1.2⏎2.0⏎1.9⏎` → `1.2 1.9 1.10 2.0`; `2⏎3.1⏎3.0⏎` → `3.0 3.1`

3.5 static, final and Initialisation Order

Exercise 1 · ID generator

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static class IdGenerator {
 private static int next = 1;

 static int nextId() {
 return next++;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(sb.length() == 0 ? "" : " ").append(IdGenerator.nextId());
 System.out.println(sb);
 }
}
```

Why it works: One shared counter increments on every call.

Verified against 2 test cases: `4⏎` → `1 2 3 4`; `1⏎` → `1`

Exercise 2 · A constant

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static final double PI_APPROX = 3.14159;

 static double circleArea(double r) {
 return PI_APPROX * r * r;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 System.out.printf("%.3f%n", circleArea(sc.nextDouble()));
 }
}
```

Why it works: A named constant avoids magic numbers.

Verified against 2 test cases: `2⏎` → `12.566`; `1⏎` → `3.142`

Exercise 3 · Initialisation order

Ex3.java

```java
public class Ex3 {
 // Predict the output of the initialisation order.
 static class Init {
 static { System.out.println("static block"); }
 { System.out.println("instance block"); }
 Init() { System.out.println("constructor"); }
 }

 public static void main(String[] args) {
 new Init();
 new Init();
 }
}
```

Why it works: The class is loaded once; each object repeats instance init + constructor.

Verified against 1 test case: `(nc input)` → `static block⏎instance block⏎constructor⏎instance block⏎constructor`

3.6 Packages and Imports

Exercise 1 · Static import

Ex1.java

```java
import java.util.Scanner;
import static java.lang.Math.sqrt;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 double a = sc.nextDouble();
 System.out.println(sqrt(a));
 }
}
```

Why it works: The static import lets you write sqrt(x).

Verified against 2 test cases: `16⏎` → `4.0`; `2⏎` → `1.4142135623730951`

Exercise 2 · Sort with Arrays

Ex2.java

```java
import java.util.Arrays;
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 Arrays.sort(a);
 System.out.println(Arrays.toString(a));
 }
}
```

Why it works: Two utility methods from one imported class.

Verified against 2 test cases: `4⏎9 3 7 1⏎` → `[1, 3, 7, 9]`; `1⏎5⏎` → `[5]`

Exercise 3 · Same name, two packages

Ex3.java

```java
public class Ex3 {
 // Two classes named Date live in different packages; use fully qualified names to tell them
    apart.
 public static void main(String[] args) {
 java.util.Date util = new java.util.Date(0);
 java.sql.Date sql = new java.sql.Date(0);
 System.out.println(util.getClass().getName());
 System.out.println(sql.getClass().getName());
 }
}
```

Why it works: Fully qualified names disambiguate clashes.

Verified against 1 test case: `(nc input)` → `java.util.Date⏎java.sql.Date`

3.7 Exceptions: try, catch, finally, throw and throws

Exercise 1 · Safe division

Ex1.java

```
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String a = sc.next();
 String b = sc.next();
 try {
 int x = Integer.parseInt(a);
 int y = Integer.parseInt(b);
 System.out.println(x / y);
 } catch (NumberFormatException e) {
 System.out.println("bad number");
 } catch (ArithmeticException e) {
 System.out.println("division by zero");
 }
 }
}
```

Why it works: parseInt throws NumberFormatException; the division throws ArithmeticException.

Verified against 3 test cases: `12 4⏎` → `3`; `5 0⏎` → `division by zero`; `x 2⏎` → `bad number`

Exercise 2 · Custom exception

Ex2.java

```
import java.util.Scanner;

public class Ex2 {
 static class NegativeAgeException extends RuntimeException {
 NegativeAgeException(int age) {
 super("age cannot be negative: " + age);
 }
 }

 static void check(int age) {
 if (age < 0) throw new NegativeAgeException(age);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 for (int i = 0; i < n; i++) {
 int age = sc.nextInt();
 try {
 check(age);
 System.out.println("ok " + age);
 } catch (NegativeAgeException e) {
 System.out.println(e.getMessage());
 }
 }
 }
}
```

Why it works: throw inside the check method; catch in the loop.

Verified against 2 test cases: `3⏎20⏎-5⏎0⏎` → `ok 20⏎age cannot be negative: -5⏎ok 0`; `1⏎-1⏎` → `age cannot be negative: -1`

Exercise 3 · try / finally order

Ex3.java

```
public class Ex3 {
 // Predict: which lines print, and in what order?
 static int f() {
 try {
 System.out.println("try");
 return 1;
 } finally {
 System.out.println("finally");
 }
 }

 public static void main(String[] args) {
 System.out.println(f());
 }
}
```

Why it works: The return value is computed, finally runs, then the value returns.

Verified against 1 test case: `(nc input)` → `try⏎finally⏎1`

3.8 Generics and Lists: ArrayList and LinkedList

Exercise 1 · Remove duplicates

Ex1.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex1 {
 // Remove all duplicates keeping the first occurrence (order preserved), using a List only.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<Integer> out = new ArrayList<>();
 for (int i = 0; i < n; i++) {
 int x = sc.nextInt();
 if (!out.contains(x)) out.add(x);
 }
 StringBuilder sb = new StringBuilder();
 for (int x : out) sb.append(sb.length() == 0 ? "" : " ").append(x);
 System.out.println(sb);
 }
}
```

Why it works: contains is O(n), so this is O(n²) — a HashSet makes it O(n) (next lesson).

Verified against 2 test cases: `6⏎3 1 3 2 1 4⏎` → `3 1 2 4`; `3⏎7 7 7⏎` → `7`

Exercise 2 · Remove a value

Ex2.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex2 {
 // Delete every element equal to the given value from a list, safely (no
     ConcurrentModificationException).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<Integer> list = new ArrayList<>();
 for (int i = 0; i < n; i++) list.add(sc.nextInt());
 int target = sc.nextInt();
 list.removeIf(x -> x == target);
 System.out.println(list);
 }
}
```

Why it works: removeIf iterates safely and in one pass.

Verified against 2 test cases: `6⏎3 1 3 2 1 4⏎3⏎` → `[1, 2, 1, 4]`; `2⏎5 5⏎5⏎` → `[]`

Exercise 3 · Generic maximum

Ex3.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex3 {
 // Generic method: return the largest element of any list of Comparable items.
 static <T extends Comparable<T>> T largest(List<T> list) {
 T best = list.get(0);
 for (T x : list) if (x.compareTo(best) > 0) best = x;
 return best;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<String> words = new ArrayList<>();
 for (int i = 0; i < n; i++) words.add(sc.next());
 System.out.println(largest(words));
 }
}
```

Why it works: A bounded generic method works for any comparable type.

Verified against 2 test cases: `4⏎pear apple zebra fig⏎` → `zebra`; `1⏎x⏎` → `x`

3.9 Sets and Maps: HashSet, TreeSet, HashMap, TreeMap

Exercise 1 · Distinct count

Ex1.java

```java
import java.util.HashSet;
import java.util.Scanner;
import java.util.Set;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 Set<Integer> set = new HashSet<>();
 for (int i = 0; i < n; i++) set.add(sc.nextInt());
 System.out.println(set.size());
 }
}
```

Why it works: A set keeps each value once.

Verified against 2 test cases: `6⏎3 1 3 2 1 4⏎` → `4`; `1⏎9⏎` → `1`

Exercise 2 · Word frequency

Ex2.java

```java
import java.util.Scanner;
import java.util.TreeMap;

public class Ex2 {
 // Word frequencies, words printed in alphabetical order.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 TreeMap<String, Integer> freq = new TreeMap<>();
 while (sc.hasNext()) {
 freq.merge(sc.next().toLowerCase(), 1, Integer::sum);
 }
 for (var e : freq.entrySet()) {
 System.out.println(e.getKey() + " " + e.getValue());
 }
 }
}
```

Why it works: A TreeMap gives sorted iteration for free.

Verified against 2 test cases: `the cat and The hat and the bat⏎` → `and 2⏎bat 1⏎cat 1⏎hat 1⏎the 3`; `a⏎` → `a 1`

Exercise 3 · Two-sum

Ex3.java

```java
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class Ex3 {
 // Two-sum: print the indices (smaller first) of the first pair that adds up to target, or
    `none`.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int target = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 Map<Integer, Integer> seen = new HashMap<>();
 for (int i = 0; i < n; i++) {
 Integer j = seen.get(target - a[i]);
 if (j != null) {
 System.out.println(j + " " + i);
 return;
 }
 seen.put(a[i], i);
 }
 System.out.println("none");
 }
}
```

Why it works: A map from value to index finds the complement in O(1).

Verified against 3 test cases: `4 18⏎2 7 11 15⏎` → `1 2`; `3 100⏎1 2 3⏎` → `none`; `4 6⏎3 2 4 3⏎` → `1 2`

3.10 Queues, Deques, Stacks and PriorityQueue

Exercise 1 · Balanced brackets

Ex1.java

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 Deque<Character> st = new ArrayDeque<>();
 boolean ok = true;
 for (char c : s.toCharArray()) {
 if ("([{".indexOf(c) >= 0) st.push(c);
 else {
 if (st.isEmpty()) { ok = false; break; }
 char o = st.pop();
 if ("([{".indexOf(o) != ")]}".indexOf(c)) { ok = false; break; }
 }
 }
 System.out.println(ok && st.isEmpty() ? "YES" : "NO");
 }
}
```

Why it works: A stack holds the currently open brackets in order.

Verified against 4 test cases: `{[()]}⏎` → `YES`; `([)]⏎` → `NO`; `((⏎` → `NO`

Exercise 2 · Josephus with a queue

Ex2.java

```java
import java.util.ArrayDeque;
import java.util.Queue;
import java.util.Scanner;

public class Ex2 {
 // Josephus with a queue: n people, every k-th leaves; print the order of leaving.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 Queue<Integer> q = new ArrayDeque<>();
 for (int i = 1; i <= n; i++) q.add(i);
 StringBuilder sb = new StringBuilder();
 while (!q.isEmpty()) {
 for (int i = 1; i < k; i++) q.add(q.poll()); // rotate k-1 people to the back
 sb.append(sb.length() == 0 ? "" : " ").append(q.poll());
 }
 System.out.println(sb);
 }
}
```

Why it works: A circle is a queue whose front keeps rotating to the back.

Verified against 2 test cases: `5 2⏎` → `2 4 1 5 3`; `7 3⏎` → `3 6 2 7 5 1 4`

Exercise 3 · k-th largest

Ex3.java

```java
import java.util.PriorityQueue;
import java.util.Scanner;

public class Ex3 {
 // k-th largest element using a min-heap of size k.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 PriorityQueue<Integer> heap = new PriorityQueue<>();
 for (int i = 0; i < n; i++) {
 heap.add(sc.nextInt());
 if (heap.size() > k) heap.poll(); // drop the smallest: the heap keeps the k largest
 }
 System.out.println(heap.peek());
 }
}
```

Why it works: The heap always contains the k largest seen so far; its top is the k-th largest.

Verified against 3 test cases: `6 2⏎3 2 1 5 6 4⏎` → `5`; `3 3⏎9 8 7⏎` → `7`; `1 1⏎4⏎` → `4`

3.11 Sorting with Comparable and Comparator

Exercise 1 · Descending

Ex1.java

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<Integer> list = new ArrayList<>();
 for (int i = 0; i < n; i++) list.add(sc.nextInt());
 Collections.sort(list, Collections.reverseOrder());
 StringBuilder sb = new StringBuilder();
 for (int x : list) sb.append(sb.length() == 0 ? "" : " ").append(x);
 System.out.println(sb);
 }
}
```

Why it works: A reversed natural order.

Verified against 2 test cases: `5⏎3 9 1 7 5⏎` → `9 7 5 3 1`; `1⏎2⏎` → `2`

Exercise 2 · By length then alphabet

Ex2.java

```java
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Scanner;

public class Ex2 {
 // Sort words by length, then alphabetically.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<String> words = new ArrayList<>();
 for (int i = 0; i < n; i++) words.add(sc.next());
 words.sort(Comparator.comparingInt(String::length).thenComparing(Comparator.naturalOrder()));
 System.out.println(String.join(" ", words));
 }
}
```

Why it works: A two-key comparator.

Verified against 2 test cases: `5⏎pear fig apple kiwi plum⏎` → `fig kiwi pear plum apple`; `2⏎b a⏎` → `a b`

Exercise 3 · Merge intervals

Ex3.java

```java
import java.util.Arrays;
import java.util.Scanner;

public class Ex3 {
 // Intervals: sort by start, then merge overlapping ones.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[][] iv = new int[n][2];
 for (int i = 0; i < n; i++) {
 iv[i][0] = sc.nextInt();
 iv[i][1] = sc.nextInt();
 }
 Arrays.sort(iv, (a, b) -> Integer.compare(a[0], b[0]));
 StringBuilder sb = new StringBuilder();
 int start = iv[0][0], end = iv[0][1];
 for (int i = 1; i < n; i++) {
 if (iv[i][0] <= end) {
 end = Math.max(end, iv[i][1]);
 } else {
 sb.append('[').append(start).append(',').append(end).append("] ");
 start = iv[i][0];
 end = iv[i][1];
 }
 }
 sb.append('[').append(start).append(',').append(end).append(']');
 System.out.println(sb);
 }
}
```

Why it works: After sorting by start, overlapping intervals are adjacent: extend the end while the next start ≤ end.

Verified against 3 test cases: `4⏎1 3⏎8 10⏎2 6⏎15 18⏎` → `[1,6] [8,10] [15,18]`; `2⏎1 4⏎4 5⏎` → `[1,5]`; `1⏎3 7⏎` → `[3,7]`

3.12 equals(), hashCode() and Immutability

Exercise 1 · Distinct people

Ex1.java

```java
import java.util.HashSet;
import java.util.Objects;
import java.util.Scanner;
import java.util.Set;

public class Ex1 {
 static final class Person {
 final String name;
 final int age;

 Person(String name, int age) {
 this.name = name;
 this.age = age;
 }

 @Override
 public boolean equals(Object o) {
 if (this == o) return true;
 if (!(o instanceof Person)) return false;
 Person p = (Person) o;
 return age == p.age && name.equals(p.name);
 }

 @Override
 public int hashCode() {
 return Objects.hash(name, age);
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 Set<Person> set = new HashSet<>();
 for (int i = 0; i < n; i++) set.add(new Person(sc.next(), sc.nextInt()));
 System.out.println(set.size());
 }
}
```

Why it works: With equals + hashCode, the HashSet de-duplicates by content.

Verified against 2 test cases: `4⏎Asha 20⏎Ravi 21⏎Asha 20⏎Asha 22⏎` → `3`; `2⏎A 1⏎A 1⏎` → `1`

Exercise 2 · Immutable money

Ex2.java

```
public class Ex2 {
 // An immutable "with" method: returns a NEW object instead of changing this one.
 static final class Money {
 private final long cents;

 Money(long cents) { this.cents = cents; }

 Money plus(Money other) { return new Money(cents + other.cents); }

 long cents() { return cents; }
 }

 public static void main(String[] args) {
 Money a = new Money(250);
 Money b = a.plus(new Money(175));
 System.out.println(a.cents() + " " + b.cents());
 }
}
```

Why it works: An immutable class returns new instances.

Verified against 1 test case: `(no input)` → `250 425`

Exercise 3 · Defensive copying

Ex3.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Ex3 {
 // An "immutable" class that leaks its list: fix by copying defensively.
 static final class Team {
 private final List<String> members;

 Team(List<String> members) {
 this.members = new ArrayList<>(members); // defensive copy in
 }

 List<String> getMembers() {
 return Collections.unmodifiableList(members); // read-only view out
 }
 }

 public static void main(String[] args) {
 List<String> source = new ArrayList<>(List.of("Asha", "Ravi"));
 Team t = new Team(source);
 source.add("Intruder"); // does not affect the Team
 System.out.println(t.getMembers());
 try {
 t.getMembers().add("Hacker");
 } catch (UnsupportedOperationException e) {
 System.out.println("read-only");
 }
 }
}
```

Why it works: Copy in, read-only view out.

Verified against 1 test case: `(no input)` → `[Asha, Ravi]⏎read-only`

Part 3 — Quiz answer key

3.1 Classes, Objects, Fields, Constructors and this

	Answer	Why
Q1	B. a class is a blueprint; an object is an instance of it	new Student(...) builds an object from the Student class.
Q2	5	a and b are the same object.
Q3	B. has the same name as the class and no return type	Written like a method without a return type.
Q4	B. the object the method was called on	The current object.

3.2 Encapsulation and Access Modifiers

	Answer	Why
Q1	C. private	private.
Q2	B. so the class can enforce its rules and change internals later	Encapsulation protects invariants.
Q3	B. in the same package	Package-private.
Q4	line 2	A public mutable field can be set to anything.

3.3 Inheritance, Overriding and Polymorphism

	Answer	Why
Q1	B	Dynamic dispatch uses the actual type.
Q2	B. super	super(...).
Q3	B. Car has an Engine	"has-a".
Q4	B. parent constructor	Parent first.

3.4 Abstract Classes and Interfaces

	Answer	Why
Q1	B. implement several interfaces	One superclass, many interfaces.
Q2	B. has no body and must be implemented by a concrete subclass	It declares a requirement.
Q3	B. Comparable<T>	compareTo defines the order.
Q4	B. an interface	An interface.

3.5 static, final and Initialisation Order

	Answer	Why
Q1	B. the JVM calls it before any object exists	No object is needed to start the program.
Q2	B. one per class	A single shared copy.
Q3	3	The static counter is shared.
Q4	A. cannot be re-assigned but elements can change	final protects the reference.

3.6 Packages and Imports

	Answer	Why
Q1	C. java.lang	java.lang.
Q2	B. first in the file	It is the first statement (comments aside).
Q3	B. no — sub-packages are separate	Wildcards cover one package only.
Q4	B. com/shop/billing/	Dots become folder separators.

3.7 Exceptions: try, catch, finally, throw and throws

	Answer	Why
Q1	C. IOException	IOException must be handled or declared.
Q2	ACD	B is skipped; the catch and finally run.
Q3	B. declares that the method may throw a checked exception	It warns callers.
Q4	B. AutoCloseable	close() is called automatically.

3.8 Generics and Lists: ArrayList and LinkedList

	Answer	Why
Q1	[5, 7]	remove(int) removes index 1 (the 6).
Q2	B. ArrayList	ArrayList is O(1).
Q3	B. generics need reference types; use Integer	Type arguments must be objects.
Q4	false	Outside the −128..127 cache, boxed values are different objects.

3.9 Sets and Maps: HashSet, TreeSet, HashMap, TreeMap

	Answer	Why
Q1	B. false	false — the set is unchanged.
Q2	B. TreeMap	TreeMap, in O(log n).
Q3	A. O(1)	Hashing jumps to the bucket.
Q4	2	merge sums existing and new values.

3.10 Queues, Deques, Stacks and PriorityQueue

	Answer	Why
Q1	3 2	LIFO: 3 comes out first; 2 is now on top.
Q2	B. queue	FIFO explores level by level.
Q3	C. the smallest	Natural ordering is a min-heap.
Q4	B. O(log n)	Sift up through the heap.

3.11 Sorting with Comparable and Comparator

	Answer	Why
Q1	A. a goes before b	Negative: first argument first.

	Answer	Why
Q2	B. integer overflow	The subtraction can overflow.
Q3	B. keeps equal elements in their original order	List.sort and Collections.sort are stable.
Q4	B. O(n log n)	Merge-sort-like TimSort.

3.12 equals(), hashCode() and Immutability

	Answer	Why
Q1	A. a.hashCode() == b.hashCode() must hold	That is the contract.
Q2	B. private final fields and no setters	State cannot change after construction.
Q3	2	Almost certainly: different hash codes put them in different buckets, so equals is never consulted.
Q4	C. String	String never changes.

Part 4 · Data Structures & Algorithm Foundations

The data structures behind the problems, linear and binary search, elementary sorting, iteration vs recursion, algorithm paradigms, and time/space complexity analysis.

	Lesson	Level	Min	Core
4.1	Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables	beginner	45	★
4.2	Trees, Graphs and Heaps	intermediate	50	
4.3	Searching: Linear and Binary Search	beginner	40	★
4.4	Elementary Sorting: Bubble, Insertion, Selection; Stability	beginner	45	★
4.5	Merge Sort and Divide and Conquer	intermediate	45	★
4.6	Iteration versus Recursion	intermediate	35	
4.7	Time Complexity and Big-O Notation	beginner	45	★
4.8	Algorithm Design Paradigms	intermediate	50	★
4.9	Analysing Loops: Counting Iterations	intermediate	50	★
4.10	Analysing Recursion: Recurrence Relations	intermediate	40	
4.11	Space Complexity and the Call Stack	beginner	30	★
4.12	Best, Average and Worst Case	intermediate	30	

4.1 Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables

Data Structures & Algorithm Foundations · Beginner · Core programming & data structures · about 45 min · ★ on the minimum-viable path

The main ways to organise data in memory, what each makes fast or slow, and a hand-built linked list, stack and queue.

YOU WILL BE ABLE TO

- Compare arrays, linked lists, stacks, queues and hash tables by the cost of their operations.
- Build a singly linked list with a `Node` class and reverse it.
- Implement a stack on a list and a queue on a ring buffer.
- Choose a data structure from the operations a problem needs.

Before this lesson: Generics and Lists: ArrayList and LinkedList · References, Pass-by-Value and Memory

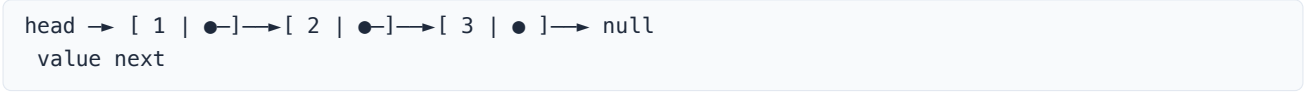
The idea

A **data structure** is a way of arranging data so that the operations you need are cheap. Every structure trades something: fast access for slow insertion, speed for memory, order for convenience.

structure	access by index	search	insert / delete	good for
array	O(1)	O(n) / O(log n) sorted	O(n) middle, O(1) end	fixed-size sequences, matrices
linked list	O(n)	O(n)	O(1) at a known node / head	frequent insertion at the front, building queues
stack (LIFO)	—	—	push/pop O(1)	undo, brackets, recursion, DFS
queue (FIFO)	—	—	enqueue/dequeue O(1)	scheduling, BFS
hash table	—	O(1) average	O(1) average	lookup, counting, de-duplication
balanced tree / TreeMap	—	O(log n)	O(log n)	sorted data, range queries
heap / priority queue	—	min only O(1)	O(log n)	repeatedly take the best item

WHY THIS WORKS

A linked list stores each value in a separate **node** that holds a reference to the next node. There is no way to jump to element i without walking i links (O(n)), but inserting at the head — or after a node you already hold — only re-wires two references (O(1)), with no shifting. An array is the opposite. Stacks and queues are not new storage; they are **disciplines**: restrict *where* you may add and remove, and every operation becomes O(1).



Syntax

structure	Java type	typical calls
array	`int[] a`	`a[i]`, `a.length`
dynamic array	`ArrayList<Integer>`	`add(x)`, `get(i)`, `set(i, x)`, `remove(i)`, `size()`
linked list	`LinkedList<Integer>`	`addFirst`, `removeFirst`, `getFirst`
stack	`ArrayDeque<Integer>`	`push(x)`, `pop()`, `peek()`
queue	`ArrayDeque<Integer>`	`offer(x)`, `poll()`, `peek()`
hash table	`HashMap<K,V>` / `HashSet<E>`	`put`, `get`, `containsKey`, `add`, `contains`

```
class Node { // hand-made linked list node
 int value;
 Node next;
 Node(int value) { this.value = value; }
}
Node head = new Node(1);
head.next = new Node(2); // head -> 1 -> 2 -> null
```

A tiny example

```
class Node {
 int value;
 Node next;
 Node(int value) { this.value = value; }
}
Node head = new Node(1);
head.next = new Node(2);
```

Guided example

LinkedStructures.java — Linked list, stack and queue from scratch (demonstration)

```
public class LinkedStructures {
 // A NODE: one value plus a reference to the next node.
 static class Node {
 int value;
 Node next;

 Node(int value) {
 this.value = value;
 }
 }

 // A singly linked list with head insertion/removal in O(1) and tail append in O(n).
 static class LinkedList {
 Node head;
 int size;

 void addFirst(int v) {
 Node n = new Node(v);
 n.next = head;
 head = n;
```

(code continued)

```
    size++;
    }

    void addLast(int v) {
    Node n = new Node(v);
    if (head == null) {
    head = n;
    } else {
    Node cur = head;
    while (cur.next != null) cur = cur.next; // walk to the end: O(n)
    cur.next = n;
    }
    size++;
    }

    int removeFirst() {
    int v = head.value;
    head = head.next;
    size--;
    return v;
    }

    void reverse() {
    Node prev = null;
    Node cur = head;
    while (cur != null) {
    Node next = cur.next; // remember the rest
    cur.next = prev; // flip the link
    prev = cur;
    cur = next;
    }
    head = prev;
    }

    @Override
    public String toString() {
    StringBuilder sb = new StringBuilder("[");
    for (Node c = head; c != null; c = c.next) {
    sb.append(c.value).append(c.next != null ? " -> " : "");
    }
    return sb.append("]").toString();
    }
    }

    // A STACK built on the list: push/pop at the head, both O(1).
    static class Stack {
    private final LinkedList list = new LinkedList();
    void push(int v) { list.addFirst(v); }
    int pop() { return list.removeFirst(); }
    boolean isEmpty() { return list.size == 0; }
    }

    // A QUEUE built on an array ring buffer: enqueue/dequeue both O(1).
    static class Queue {
    private final int[] data;
    private int head, size;

```

(code continued)

```
78    Queue(int capacity) { data = new int[capacity]; }
79
80    boolean enqueue(int v) {
81    if (size == data.length) return false;
82    data[(head + size) % data.length] = v;                    5
83    size++;
84    return true;
85    }
86
87    int dequeue() {
88    int v = data[head];
89    head = (head + 1) % data.length;
90    size--;
91    return v;
92    }
93
94    boolean isEmpty() { return size == 0; }
95    }
96
97    public static void main(String[] args) {
98    LinkedList list = new LinkedList();
99    list.addFirst(2);
100   list.addFirst(1);
101   list.addLast(3);
102   list.addLast(4);
103   System.out.println(list + " size " + list.size);
104   list.reverse();
105   System.out.println("reversed " + list);
106
107   Stack st = new Stack();
108   for (int i = 1; i <= 3; i++) st.push(i);
109   StringBuilder sb = new StringBuilder("stack pops:");
110   while (!st.isEmpty()) sb.append(' ').append(st.pop());
111   System.out.println(sb);
112
113   Queue q = new Queue(3);
114   q.enqueue(10);
115   q.enqueue(20);
116   q.enqueue(30);
117   System.out.println("fourth enqueue accepted? " + q.enqueue(40));
118   System.out.println("dequeued " + q.dequeue() + ", then enqueue 40 -> " + q.enqueue(40));
119   sb = new StringBuilder("queue order:");
120   while (!q.isEmpty()) sb.append(' ').append(q.dequeue());
121   System.out.println(sb);
122   }
123  }
```

No library collections: only Node and arrays.

Line-by-line walkthrough

#	Where	What happens
1	line 30	Reaching the tail requires walking the whole list: addLast is O(n) unless you also keep a tail reference.
2	line 47	Reversal: save the next node BEFORE overwriting the link, otherwise the rest of the list is lost.
3	line 48	Each node now points backwards.
4	line 68	A stack is the head operations of the list: both O(1).
5	line 82	A circular buffer: the modulo wraps indices around, so neither end ever needs shifting.

Dry run: watch it execute

prev / cur / next flipping one link at a time.

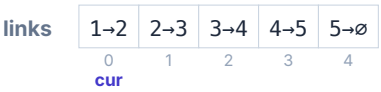
Algorithm: Reverse a singly linked list. **Input:** arr = `1,2,3,4,5`.

Pseudocode

```
prev = null; cur = head
while cur != null:
 next = cur.next // remember the rest
 cur.next = prev // flip the link
 prev = cur; cur = next
head = prev
```

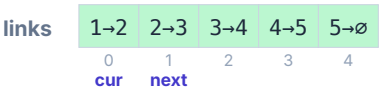
Step 1 of 12. prev is null, cur is the head.

prev = null · cur = 1



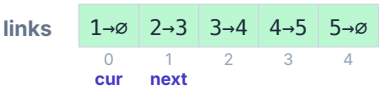
Step 2 of 12. Save next = 2 before changing cur.next.

prev = null · cur = 1 · next = 2



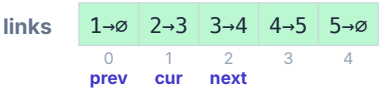
Step 3 of 12. Flip: 1 now points to null.

prev = null · cur = 1



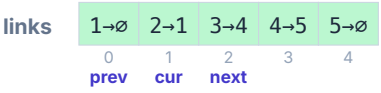
Step 4 of 12. Save next = 3 before changing cur.next.

prev = 1 · cur = 2 · next = 3



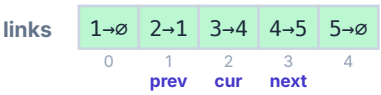
Step 5 of 12. Flip: 2 now points to 1.

prev = 1 · cur = 2



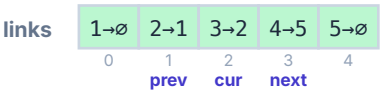
Step 6 of 12. Save next = 4 before changing cur.next.

prev = 2 · cur = 3 · next = 4



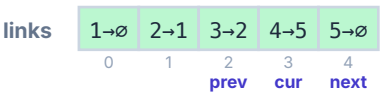
Step 7 of 12. Flip: 3 now points to 2.

prev = 2 · cur = 3



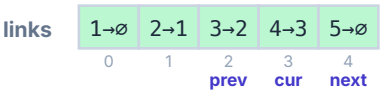
Step 8 of 12. Save next = 5 before changing cur.next.

prev = 3 · cur = 4 · next = 5



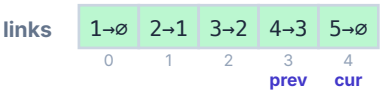
Step 9 of 12. Flip: 4 now points to 3.

prev = 3 · cur = 4



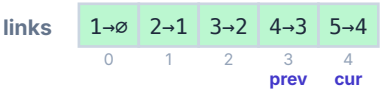
Step 10 of 12. Save next = null before changing cur.next.

prev = 4 · cur = 5 · next = null



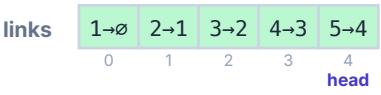
Step 11 of 12. Flip: 5 now points to 4.

prev = 4 · cur = 5



Step 12 of 12. cur is null: the new head is 5.

head = 5



> **RESULT**
> 5 → 4 → 3 → 2 → 1

Complexity

See the table above. Space: an array of n ints uses 4n bytes; a linked list uses about 16–24 bytes per element (value + reference + object header) — noticeably more.

Common mistakes

Losing the rest of the list

Problem

```
cur.next = prev; // before saving cur.next
```

Why: The only reference to the remaining nodes is overwritten.

Fix: Node next = cur.next; cur.next = prev; ...

Dereferencing null

Problem

```
head.next.value // empty list or last node
```

Why: NullPointerException.

Fix: Check head != null / cur.next != null.

Off-by-one in a ring buffer

Problem

```
data[head + size] // missing % capacity
```

Why: Index runs off the end of the array.

Fix: data[(head + size) % capacity]

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Reverse a linked list — *Beginner*

Read n and n numbers into a singly linked list (your own Node class); reverse it and print it.

Example input	Expected output
5 ⏎ 1 2 3 4 5	5 4 3 2 1

Hint 1: Use three references: prev, cur, next.

Exercise 2 · Palindromic list — *Beginner*

Read n numbers; print YES if the sequence reads the same forwards and backwards.

Example input	Expected output
5 ⏎ 1 2 3 2 1	YES

Hint 1: Two indices moving inwards.

Exercise 3 · Circular queue — *Intermediate*

Implement a queue of capacity c on an array. Read c and n commands `enq x` / `deq`; print `ok`/`full` for enq and the removed value or `empty` for deq.

Example input	Expected output
2 6 ⏎ enq 1 ⏎ enq 2 ⏎ enq 3 ⏎ deq ⏎ enq 3 ⏎ deq	ok ok full 1 ok 2

Hint 1: head and size; index (head + size) % c.

Quick check

Answers are in the answer key at the end of this part.

Q1. Match the structure to its best-known operation cost.

Item	Your match
array index lookup	_____
linked list lookup by position	_____
hash table lookup (average)	_____
sorted tree lookup	_____

Choose from: `O(1)`, `O(log n)`, `O(n)`

Q2. Which structure matches "undo"?

- **A.** queue
- **B.** stack
- **C.** set
- **D.** heap

Q3. Insert at the head of a singly linked list costs...

- **A.** O(1)
- **B.** O(log n)
- **C.** O(n)
- **D.** $O(n^2)$

Q4. Which is best for BFS?

- **A.** stack
- **B.** queue
- **C.** array
- **D.** tree

Interview / exam questions

- Array vs linked list?
- How do you reverse a linked list?
- Implement a queue with two stacks.
- Detect a cycle in a linked list (Floyd).

Watch & read

Video: Data Structures Made Easy:Animated Explanations of Arrays,Linked Lists,Stacks,Queues,Heaps, HashMaps — Coding Society · 4:35. *Overview of the fundamental data structures*

Video: #5 Linked List Implementation in Java Part 1 | Data Structures — Telusko · 23:05. *Building a linked list*

REVISION CARD (30–60 SECONDS)

- Array: O(1) index, O(n) insert. Linked list: O(1) head insert, O(n) index.
- Stack LIFO, queue FIFO, hash table O(1) average lookup.
- Reverse: save next, flip link, advance.
- Ring buffer: (head + size) % capacity.

4.2 Trees, Graphs and Heaps

Data Structures & Algorithm Foundations · Intermediate · Core programming & data structures · about 50 min · complete path

Hierarchical and networked data: binary search trees, graph representations with BFS/DFS, and the array-based min-heap.

YOU WILL BE ABLE TO

- Describe trees, binary search trees, and traversals (inorder, preorder, postorder).
- Represent a graph as an adjacency matrix and an adjacency list; know when each is better.
- Write BFS (queue) and DFS (recursion/stack) and state their cost.
- Explain a binary heap stored in an array and its sift-up / sift-down operations.

Before this lesson: Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables · Recursion: Base Case, Recursive Case and the Call Stack · Queues, Deques, Stacks and PriorityQueue

The idea

A **tree** is a connected structure of nodes without cycles: one **root**, and every node has **children**. In a **binary search tree (BST)** each node's key is larger than everything in its left subtree and smaller than everything in its right subtree, so **inorder traversal prints keys in sorted order** and searching discards half the tree at each step (O(height), which is O(log n) when balanced).

A **graph** is a set of **vertices** joined by **edges** (directed or undirected). The Hamiltonian-cycle lesson stored one as an **adjacency matrix** (`graph[i][j] == 1`: V^2 memory, O(1) edge test). An **adjacency list** (`List<List<Integer>>`: each vertex lists its neighbours) uses O(V + E) memory and is best for sparse graphs.

```
BST insert 50, 30, 70, 20, 40, 60, 80 graph: 0–1 0–2 1–3 2–3 3–4
 50 BFS from 0: 0 1 2 3 4 (by distance)
 / \ DFS from 0: 0 1 3 2 4 (as deep as possible first)
 30 70
 / \ / \
 20 40 60 80 inorder: 20 30 40 50 60 70 80
```

WHY THIS WORKS

BFS uses a queue, so vertices are processed in order of distance from the start: the first time it reaches a vertex is along a shortest path (fewest edges). **DFS** goes as deep as possible then backtracks — exactly the structure of the N-Queens, maze and Hamiltonian-cycle searches. Both visit every vertex and edge once: O(V + E) with an adjacency list.

Heap

A **binary min-heap** is a complete binary tree stored in an array where every parent ≤ its children. For index i: parent = (i−1)/2, children = 2i+1 and 2i+2. **add** puts the item at the end and **sifts up**; **removeMin** moves the last item to the root and **sifts down**. Both are O(log n); reading the minimum is O(1). This is what `PriorityQueue` does.

Syntax

```
class TreeNode { int key; TreeNode left, right; TreeNode(int k) { key = k; } }

List<List<Integer>> adj = new ArrayList<>(); // adjacency list for V vertices
for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
adj.get(u).add(v); adj.get(v).add(u); // undirected edge

PriorityQueue<Integer> minHeap = new PriorityQueue<>(); // smallest first
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

Deque<Integer> queue = new ArrayDeque<>(); // BFS
queue.offer(start); seen[start] = true;
while (!queue.isEmpty()) {
 int v = queue.poll();
 for (int w : adj.get(v)) if (!seen[w]) { seen[w] = true; queue.offer(w); }
}
```

A tiny example

```
static boolean contains(TreeNode t, int key) {
 while (t != null) {
 if (key == t.key) return true;
 t = key < t.key ? t.left : t.right;
 }
 return false;
}
```

Guided example

TreeGraphDemo.java — Trees, graphs and heaps (demonstration)

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Queue;

public class TreeGraphDemo {
 // ---------- binary search tree ----------
 static class TreeNode {
 int key;
 TreeNode left, right;

 TreeNode(int key) { this.key = key; }
 }

 static TreeNode insert(TreeNode root, int key) {                    1
 if (root == null) return new TreeNode(key);
 if (key < root.key) root.left = insert(root.left, key);
 else if (key > root.key) root.right = insert(root.right, key);
 return root;
 }

 static boolean contains(TreeNode root, int key) {
 while (root != null) {
 if (key == root.key) return true;
```

(code continued)

```
    root = key < root.key ? root.left : root.right; // discard one half each step
    }
    return false;
    }

    static void inorder(TreeNode t, List<Integer> out) { // left, node, right: SORTED order  (2)
        for a BST
    if (t == null) return;
    inorder(t.left, out);
    out.add(t.key);
    inorder(t.right, out);
    }

    static int height(TreeNode t) {
    return t == null ? 0 : 1 + Math.max(height(t.left), height(t.right));
    }

    // ---------- graph as adjacency list ----------
    static List<List<Integer>> graph(int n, int[][] edges) {
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    for (int[] e : edges) {
    adj.get(e[0]).add(e[1]);
    adj.get(e[1]).add(e[0]); // undirected: add both directions  (3)
    }
    return adj;
    }

    static List<Integer> bfs(List<List<Integer>> adj, int start) {
    boolean[] seen = new boolean[adj.size()];  (4)
    List<Integer> order = new ArrayList<>();
    Queue<Integer> q = new ArrayDeque<>();
    q.add(start);
    seen[start] = true;
    while (!q.isEmpty()) {
    int u = q.poll();
    order.add(u);
    for (int v : adj.get(u)) {
    if (!seen[v]) {
    seen[v] = true;
    q.add(v);
    }
    }
    }
    return order;
    }

    static void dfs(List<List<Integer>> adj, int u, boolean[] seen, List<Integer> order) {
    seen[u] = true;
    order.add(u);
    for (int v : adj.get(u)) {
    if (!seen[v]) dfs(adj, v, seen, order);
    }
    }

    // ---------- binary min-heap in an array ----------
    static class MinHeap {
```

(code continued)

```
    int[] a = new int[16];
    int size;

    void add(int x) {
    a[size] = x;
    int i = size++;
    while (i > 0 && a[(i - 1) / 2] > a[i]) { // sift up: parent of i is (i-1)/2
    int p = (i - 1) / 2;
    int t = a[p]; a[p] = a[i]; a[i] = t;
    i = p;
    }
    }

    int removeMin() {
    int min = a[0];
    a[0] = a[--size];
    int i = 0;
    while (true) { // sift down: children 2i+1, 2i+2
    int l = 2 * i + 1, r = l + 1, s = i;
    if (l < size && a[l] < a[s]) s = l;
    if (r < size && a[r] < a[s]) s = r;
    if (s == i) break;
    int t = a[s]; a[s] = a[i]; a[i] = t;
    i = s;
    }
    return min;
    }
    }

    public static void main(String[] args) {
    TreeNode root = null;
    for (int k : new int[] {50, 30, 70, 20, 40, 60, 80, 30}) root = insert(root, k); // 30
        twice: ignored
    List<Integer> sorted = new ArrayList<>();
    inorder(root, sorted);
    System.out.println("inorder : " + sorted);
    System.out.println("height : " + height(root) + ", contains 60? " + contains(root, 60) +
        ", 65? " + contains(root, 65));

    int[][] edges = {{0, 1}, {0, 2}, {1, 3}, {2, 3}, {3, 4}};
    List<List<Integer>> adj = graph(5, edges);
    System.out.println("adjacency: " + adj);
    System.out.println("BFS from 0: " + bfs(adj, 0));
    List<Integer> d = new ArrayList<>();
    dfs(adj, 0, new boolean[5], d);
    System.out.println("DFS from 0: " + d);

    MinHeap h = new MinHeap();
    for (int x : new int[] {9, 4, 7, 1, 8, 2}) h.add(x);
    int[] out = new int[6];
    for (int i = 0; i < 6; i++) out[i] = h.removeMin();
    System.out.println("heap order: " + Arrays.toString(out));
    }
   }
```

Three structures in one file.

Line-by-line walkthrough

#	Where	What happens
1	line 16	Recursive BST insertion: go left or right until an empty spot; duplicates are ignored.
2	line 31	Left subtree, then the node, then the right subtree: sorted output.
3	line 48	Undirected graph: store the edge in both directions.
4	line 54	Mark vertices when they are ENQUEUED so each is queued once.
5	line 88	Sift up: swap with the parent while smaller than it.
6	line 100	Sift down: pick the smaller child and swap if it is smaller than a[i].

Dry run: watch it execute

BFS discovers vertices level by level.

Algorithm: Breadth-first search. **Input:** edges = `0-1,0-2,1-3,2-3,3-4`, start = `0`.

Pseudocode

```
queue = [start]; dist[start] = 0
while queue not empty:
 v = queue.poll()
 for each neighbour w of v:
 if w not seen: dist[w] = dist[v] + 1; queue.add(w)
```

Step 1 of 6. Enqueue the start vertex 0 with distance 0.

queue = [0]

dist by vertex	0	1	2	3	4
	0	·	·	·	·

Step 2 of 6. Poll 0. Newly discovered: 1, 2 at distance 1.

v = 0 · queue = [1, 2] · visited = 0

dist by vertex	0	1	2	3	4
	0	1	1	·	·

Step 3 of 6. Poll 1. Newly discovered: 3 at distance 2.

v = 1 · queue = [2, 3] · visited = 0 1

dist by vertex	0	1	2	3	4
	0	1	1	2	·

Step 4 of 6. Poll 2. No new neighbours.

v = 2 · queue = [3] · visited = 0 1 2

dist by vertex	0	1	2	3	4
	0	1	1	2	·

Step 5 of 6. Poll 3. Newly discovered: 4 at distance 3.

v = 3 · queue = [4] · visited = 0 1 2 3

dist by vertex	0	1	2	3	4
	0	1	1	2	3

Step 6 of 6. Poll 4. No new neighbours.

v = 4 · queue = [] · visited = 0 1 2 3 4

dist by vertex	0	1	2	3	4
	0	1	1	2	3

> **RESULT**
> visit order: 0 1 2 3 4

Complexity

operation	time
BST search/insert (balanced)	O(log n)
BST search/insert (degenerate: sorted input)	O(n)
BFS / DFS (adjacency list)	O(V + E)
BFS / DFS (adjacency matrix)	$O(V^2)$
heap add / removeMin	O(log n)
heap peek	O(1)
build a heap from n items	O(n)

Common mistakes

Marking visited too late in BFS

Problem

```
seen[v] = true; // only when polled
```

Why: A vertex may be added to the queue many times.

Fix: Mark when enqueuing.

Assuming a BST is balanced

Problem

```
inserting 1, 2, 3, 4, 5 in order
```

Why: The tree degenerates into a linked list: O(n) operations.

Fix: Use TreeMap (red-black tree) or shuffle/balance.

Treating the heap array as sorted

Problem

```
printing the array of a heap
```

Why: Only the parent ≤ child rule holds, not full sortedness.

Fix: Repeatedly removeMin to obtain sorted order.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Height of a BST — *Beginner*

Read n distinct keys, insert them into a BST in the given order, and print its height (number of nodes on the longest root-to-leaf path).

Example input	Expected output
`7 ⏎ 50 30 70 20 40 60 80`	3

Hint 1: height(t) = 1 + max(height(left), height(right)); null has height 0.

Exercise 2 · Connected components — *Intermediate*

Read n, m and m undirected edges. Print the number of connected components.

Example input	Expected output
`6 3 ⏎ 0 1 ⏎ 1 2 ⏎ 3 4`	`3`

Hint 1: Start a BFS from every unvisited vertex and count the starts.

Exercise 3 · Shortest path length — *Challenge*

Read n, m and the edges of an undirected graph. Print the minimum number of edges from vertex 0 to vertex n−1, or -1.

Example input	Expected output
`5 5 ⏎ 0 1 ⏎ 0 2 ⏎ 1 3 ⏎ 2 3 ⏎ 3 4`	`3`

Hint 1: BFS with a distance array.

Quick check

Answers are in the answer key at the end of this part.

Q1. Inorder traversal of a BST yields...

- **A.** sorted keys
- **B.** level order
- **C.** reverse order
- **D.** random order

Q2. Which data structure drives BFS?

- **A.** stack
- **B.** queue
- **C.** heap
- **D.** hash set

Q3. The children of heap index 3 (0-based) are at...

- **A.** 6 and 7
- **B.** 7 and 8
- **C.** 4 and 5
- **D.** 1 and 2

Q4. Memory for an adjacency matrix of V vertices is...

- **A.** O(V)
- **B.** O(V + E)
- **C.** $O(V^2)$
- **D.** $O(E^2)$

Interview / exam questions

- BFS vs DFS?
- Why is a BST search O(log n) only if balanced?
- Adjacency list vs matrix?
- How does a priority queue work?

Watch & read

Video: Lec-53: Binary Search Tree in Data Structure | Insertion and Traversal in BST — Gate Smashers · 11:02. *Trees, BSTs and traversals*

Video: Graphs: Edge List, Adjacency Matrix, Adjacency List, DFS, BFS - DSA Course in Python Lecture 11 — Greg Hogg · 32:11. *Graphs, BFS and DFS (Python — the idea transfers to Java)*

REVISION CARD (30–60 SECONDS)

- BST: left < node < right; inorder sorted; O(height).
- Graph: matrix $O(V^2)$ / list O(V+E). BFS queue → shortest edges; DFS recursion → depth.
- Heap: parent (i−1)/2, children 2i+1, 2i+2; sift up/down O(log n).
- Mark visited when enqueuing.

4.3 Searching: Linear and Binary Search

Data Structures & Algorithm Foundations · Beginner · Core programming & data structures · about 40 min · ★ on the minimum-viable path

Find an element by scanning, or by halving a sorted range — and the lower-bound variant for duplicates.

YOU WILL BE ABLE TO

- Write linear search and binary search (iterative and recursive).
- Explain why binary search needs sorted data and runs in O(log n).
- Avoid the `(lo + hi) / 2` overflow and the common off-by-one mistakes.
- Use lower bound to count occurrences and "binary search on the answer".

Before this lesson: One-Dimensional Arrays · Loops: for, while, do-while, break and continue · Boundary Conditions, Validation and Common Mistakes

The idea

Linear search inspects elements one by one until it finds the target: works on anything, O(n). **Binary search** needs a **sorted** array: compare the target with the middle element; if the target is smaller it must be in the left half, if larger in the right half — so **half of the remaining candidates vanish every step**.

```
find 9 in [1 3 5 7 9 11 13]
 lo=0 hi=6 mid=3 a[3]=7 <9 → lo=4
 lo=4 hi=6 mid=5 a[5]=11 >9 → hi=4
 lo=4 hi=4 mid=4 a[4]=9 == → found at index 4 (3 steps for 7 elements)
```

WHY THIS WORKS

After k steps at most n / 2^k elements remain; the search ends when that reaches 1: k = $\log_2 n$. A billion sorted elements need only about 30 comparisons. The loop invariant is "if the target is in the array, it is in a[lo..hi]"; each branch preserves it by discarding a half that cannot contain the target.

Beyond "is it there?"

- **Lower bound:** first index with `a[i] >= target`. Then `count = lowerBound(t + 1) - lowerBound(t)` gives the number of copies in O(log n).
- **Binary search on the answer:** when a yes/no test is monotone (false... false true... true) you can binary search the *value*. The Alice's-apple-tree lesson searches the smallest r with apples(r) ≥ needed.

Syntax

The two templates

```
// closed range [lo, hi] — find an exact match
int lo = 0, hi = a.length - 1;
while (lo <= hi) {
 int mid = lo + (hi - lo) / 2;
 if (a[mid] == target) return mid;
 if (a[mid] < target) lo = mid + 1; else hi = mid - 1;
}
return -1;

// half-open [lo, hi) — first index with a[i] >= target (lower bound)
int lo = 0, hi = a.length;
while (lo < hi) {
 int mid = lo + (hi - lo) / 2;
 if (a[mid] < target) lo = mid + 1; else hi = mid;
}
return lo;
```

The library does it for you on **sorted** data: `Arrays.binarySearch(a, key)` returns the index, or `-(insertionPoint) - 1` when the key is absent.

A tiny example

```
int lo = 0, hi = a.length - 1;
while (lo <= hi) {
 int mid = lo + (hi - lo) / 2;
 if (a[mid] == target) return mid;
 if (a[mid] < target) lo = mid + 1; else hi = mid - 1;
}
return -1;
```

Guided example

SearchDemo.java — Search methods (demonstration)

```
1  import java.util.Arrays;
2
3  public class SearchDemo {
4   static int comparisons;
5
6   // LINEAR SEARCH: works on any array; O(n)
7   static int linear(int[] a, int target) {
8   for (int i = 0; i < a.length; i++) {
9   comparisons++;
10  if (a[i] == target) return i;
11  }
12  return -1;
13  }
14
15  // BINARY SEARCH: needs a SORTED array; halves the range each step; O(log n)
16  static int binary(int[] a, int target) {
17  int lo = 0, hi = a.length - 1;
18  while (lo <= hi) {
19  int mid = lo + (hi - lo) / 2; // never (lo + hi) / 2: that can overflow
20  comparisons++;
```

1

(code continued)

```
21    if (a[mid] == target) return mid;
22    if (a[mid] < target) lo = mid + 1; // target can only be on the right                2
23    else hi = mid - 1; // or on the left                                                 3
24    }
25    return -1;
26    }
27
28    // first index whose value is >= target (the "lower bound"): useful when duplicates
          exist
29    static int lowerBound(int[] a, int target) {
30    int lo = 0, hi = a.length; // hi is exclusive                                        4
31    while (lo < hi) {
32    int mid = lo + (hi - lo) / 2;
33    if (a[mid] < target) lo = mid + 1;
34    else hi = mid;                                                                       5
35    }
36    return lo;
37    }
38
39    static int binaryRecursive(int[] a, int target, int lo, int hi) {
40    if (lo > hi) return -1;
41    int mid = lo + (hi - lo) / 2;
42    if (a[mid] == target) return mid;
43    return a[mid] < target ? binaryRecursive(a, target, mid + 1, hi) : binaryRecursive(a,
          target, lo, mid - 1);
44    }
45
46    public static void main(String[] args) {
47    int[] a = new int[1000];
48    for (int i = 0; i < a.length; i++) a[i] = 2 * i + 1; // 1, 3, 5, ... 1999 (sorted,
          distinct)
49    comparisons = 0;
50    System.out.println("linear 1999 -> index " + linear(a, 1999) + " after " + comparisons +
          " comparisons");
51    comparisons = 0;
52    System.out.println("binary 1999 -> index " + binary(a, 1999) + " after " + comparisons +
          " comparisons");
53    System.out.println("binary 1000 (absent) -> " + binary(a, 1000));
54    System.out.println("recursive 501 -> " + binaryRecursive(a, 501, 0, a.length - 1));
55    int[] dup = {1, 2, 2, 2, 5, 9};
56    System.out.println("lowerBound(2) = " + lowerBound(dup, 2) + ", lowerBound(3) = " +
          lowerBound(dup, 3));
57    System.out.println("library: " + Arrays.binarySearch(a, 501));
58    }
59  }
```

Comparison counters show 1000 vs 10 steps.

Line-by-line walkthrough

#	Where	What happens
1	line 19	Equivalent to (lo + hi) / 2 but cannot overflow for large indices.
2	line 22	Target is to the right of mid: mid itself is excluded (+1).
3	line 23	Target is to the left: exclude mid (−1). Forgetting ±1 causes infinite loops.
4	line 30	Half-open range [lo, hi): the loop ends with lo == hi at the answer.

#	Where	What happens
5	line 34	mid might be the answer, so keep it.

Dry run: watch it execute

The search window halves on every probe.

Algorithm: Binary search. **Input:** arr = `2,5,8,12,16,23,38,56,72,91`, target = `23`.

Pseudocode

```
lo = 0; hi = n - 1
while lo <= hi:
 mid = lo + (hi - lo) / 2
 if a[mid] == target: return mid
 if a[mid] < target: lo = mid + 1
 else: hi = mid - 1
return -1
```

Step 1 of 3. mid = 4, a[4] = 16 is less than 23: discard the left half.

lo = 0 · hi = 9 · mid = 4 · a[mid] = 16



Step 2 of 3. mid = 7, a[7] = 56 is greater than 23: discard the right half.

lo = 5 · hi = 9 · mid = 7 · a[mid] = 56



Step 3 of 3. mid = 5, a[5] = 23 equals the target 23: found.

lo = 5 · hi = 6 · mid = 5 · a[mid] = 23



> **RESULT**
> found at index 5 after 3 probe(s)

Complexity

search	time	needs
linear	O(n)	nothing
binary	O(log n)	sorted data, random access
hash lookup	O(1) average	a hash structure

Binary search is O(1) space iteratively and O(log n) stack space recursively.

Common mistakes

Binary search on unsorted data

Problem

```
Arrays.binarySearch(unsorted, x)
```

Why: The result is undefined (wrong index or negative).

Fix: Sort first (O(n log n)) or use linear search.

lo = mid instead of mid + 1

Problem

```
if (a[mid] < target) lo = mid;
```

Why: With lo == mid the range never shrinks → infinite loop.

Fix: lo = mid + 1;

Overflowing midpoint

Problem

```
int mid = (lo + hi) / 2;
```

Why: lo + hi can exceed $2^{31} - 1$.

Fix: lo + (hi - lo) / 2

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · First occurrence — *Beginner*

Read n, target and n numbers (unsorted). Print the index of the first occurrence of the target, or -1.

Example input	Expected output
5 7 ⏎ 4 9 7 1 7	2

Hint 1: A for loop with an early exit.

Exercise 2 · Binary search — *Intermediate*

Read n, target and a SORTED array of distinct numbers. Print the index of the target or -1 using binary search.

Example input	Expected output
6 9 ⏎ 1 3 5 7 9 11	4

Hint 1: lo, hi, mid; discard half each time.

Exercise 3 · Count occurrences — *Challenge*

In a sorted array (duplicates allowed) print how many times the target occurs, in O(log n).

Example input	Expected output
8 2 ⏎ 1 2 2 2 2 3 5 9	4

Hint 1: lowerBound(target + 1) − lowerBound(target).

Quick check

Answers are in the answer key at the end of this part.

Q1. How many comparisons does binary search need (at most) for 1 000 000 sorted elements?

- **A.** 10
- **B.** 20
- **C.** 1000
- **D.** 500000

Q2. Binary search requires the data to be...

- **A.** unique
- **B.** sorted
- **C.** positive
- **D.** in a list

Q3. Binary search for 7 in [1, 3, 5, 7, 9, 11, 13].

```
int lo = 0, hi = 6;
while (lo <= hi) {
 int mid = lo + (hi - lo) / 2;
 if (a[mid] == 7) break;
 if (a[mid] < 7) lo = mid + 1; else hi = mid - 1;
}
```

- first mid: _____
- a[first mid]: _____

Q4. The time complexity of binary search is...

- **A.** O(1)
- **B.** O(log n)
- **C.** O(n)
- **D.** O(n log n)

Interview / exam questions

- Why is binary search O(log n)?
- What is the invariant?
- How do you search a rotated sorted array?
- What is "binary search on the answer"?

Watch & read

Video: Binary Search in Java - Full Simple Coding Tutorial — Coding with John · 17:48. *Binary search step by step*

Video: Time Complexity of Linear Search and Binary Search || Lesson 33 || Algorithms || Learning Monkey || — Wisdomers - Computer Science and Engineering · 7:53. *Linear vs binary search*

REVISION CARD (30–60 SECONDS)

- Linear O(n) any data; binary O(log n) sorted data.
- mid = lo + (hi − lo)/2; lo = mid+1 / hi = mid−1.
- lowerBound for duplicates and counting.
- Monotone test → binary search on the answer.

4.4 Elementary Sorting: Bubble, Insertion, Selection; Stability

Data Structures & Algorithm Foundations · Beginner · Core programming & data structures · about 45 min · ★ on the minimum-viable path

The three O(n²) sorts side by side, with comparison and swap counts, and what "stable" and "in-place" mean.

YOU WILL BE ABLE TO

- Trace bubble, insertion and selection sort by hand.
- State best, average and worst-case time of each and their swap/shift behaviour.
- Define stable and in-place and classify each algorithm.
- Explain why insertion sort is the best of the three on nearly sorted data.

Before this lesson: One-Dimensional Arrays · Loops: for, while, do-while, break and continue

The idea

algorithm	idea	best	worst	swaps / shifts	stable	in place
bubble	swap adjacent out-of-order pairs; largest sinks each pass	O(n) with early exit	O(n²)	up to n²/2 swaps	yes	yes
insertion	insert the next item into the sorted prefix	O(n)	O(n²)	one shift per inversion	yes	yes
selection	pick the minimum, swap to the front	O(n²)	O(n²)	≤ n − 1 swaps	no	yes

WHY THIS WORKS

Every adjacent-swap sort must fix each **inversion** (a pair out of order) one at a time, and a random array has about n²/4 inversions — hence Θ(n²). Insertion sort does work proportional to n + inversions, so it is **O(n) on nearly sorted input** and is used inside real library sorts for small ranges. **Stable** means equal keys keep their input order (essential when sorting by one field after another); **in place** means O(1) extra memory.

Rule of thumb: for n ≤ ~50, insertion sort is perfectly fine; beyond that use O(n log n) algorithms (merge sort, quick sort, `Arrays.sort`).

Syntax

```
// the swap idiom (needs a temporary)
int t = a[i]; a[i] = a[j]; a[j] = t;

// library sorts — in practice use these
Arrays.sort(a); // int[] ascending (dual-pivot quicksort)
Arrays.sort(a, 2, 7); // only indices 2 … 6
Integer[] boxed = {5, 2, 9};
Arrays.sort(boxed, Collections.reverseOrder()); // objects need Integer[], not int[]
Collections.sort(list); // List<T> ascending (stable merge sort / TimSort)
list.sort((x, y) -> y - x); // with a comparator
```

A tiny example

```
for (int i = 1; i < a.length; i++) {
 int key = a[i], j = i - 1;
 while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; j--; }
 a[j + 1] = key;
}
```

Guided example

ElementarySorts.java — Three sorts, one test harness (demonstration)

```
1   import java.util.Arrays;
2
3   public class ElementarySorts {
4    static long comparisons, swaps;
5
6    static void swap(int[] a, int i, int j) {
7    int t = a[i];
8    a[i] = a[j];
9    a[j] = t;
10   swaps++;
11   }
12
13   // BUBBLE SORT: repeatedly swap adjacent out-of-order pairs; the largest "bubbles" to
         the end each pass.
14   static void bubble(int[] a) {
15   for (int pass = 0; pass < a.length - 1; pass++) {
16   boolean swapped = false;
17   for (int i = 0; i < a.length - 1 - pass; i++) {
18   comparisons++;
19   if (a[i] > a[i + 1]) {
20   swap(a, i, i + 1);
21   swapped = true;
22   }
23   }
24   if (!swapped) break; // already sorted: O(n) best case                    (1)
25   }
26   }
27
28   // INSERTION SORT: grow a sorted prefix; insert the next element by shifting larger
         ones right.
29   static void insertion(int[] a) {
30   for (int i = 1; i < a.length; i++) {
31   int key = a[i];
32   int j = i - 1;
33   while (j >= 0) {
34   comparisons++;
35   if (a[j] <= key) break; // <= keeps equal elements in order: STABLE       (2)
36   a[j + 1] = a[j];                                                         (3)
37   j--;
38   }
39   a[j + 1] = key;
40   }
41   }
42
43   static void selection(int[] a) {
44   for (int i = 0; i < a.length - 1; i++) {
```

(code continued)

```
    int m = i;
    for (int j = i + 1; j < a.length; j++) {
    comparisons++;
    if (a[j] < a[m]) m = j;
    }
    if (m != i) swap(a, i, m);
    }
    }

    static void run(String name, java.util.function.Consumer<int[]> sorter, int[] input) {
    int[] a = input.clone();
    comparisons = 0;
    swaps = 0;
    sorter.accept(a);
    System.out.printf("%-10s %s comparisons=%d swaps=%d%n", name, Arrays.toString(a),
        comparisons, swaps);
    }

    public static void main(String[] args) {
    int[] data = {5, 2, 9, 1, 5, 6};
    System.out.println("input " + Arrays.toString(data));
    run("bubble", ElementarySorts::bubble, data);
    run("insertion", ElementarySorts::insertion, data);
    run("selection", ElementarySorts::selection, data);

    int[] sorted = {1, 2, 3, 4, 5, 6};
    System.out.println("already sorted input:");
    run("bubble", ElementarySorts::bubble, sorted);
    run("insertion", ElementarySorts::insertion, sorted);
    run("selection", ElementarySorts::selection, sorted);

    // stability: records with equal keys. Insertion sort keeps "b1" before "b2".
    String[] items = {"b1", "a1", "b2", "a2"};
    String[] stable = items.clone();
    for (int i = 1; i < stable.length; i++) {
    String key = stable[i];
    int j = i - 1;
    while (j >= 0 && stable[j].charAt(0) > key.charAt(0)) { // compare letters only
    stable[j + 1] = stable[j];
    j--;
    }
    stable[j + 1] = key;
    }
    System.out.println("stable sort by letter: " + Arrays.toString(stable));
    }
   }
```

The harness prints comparisons and swaps for random and already sorted input.

Line-by-line walkthrough

#	Where	What happens
1	line 24	Early exit: a pass without swaps proves the array is sorted — best case O(n).
2	line 35	<= stops at equal elements, so they keep their order (stable).
3	line 36	Shifting right instead of swapping: one write per step.
4	line 67	Method references pass each algorithm to the same measuring code.

Dry run: watch it execute

A sorted prefix grows by one element per step.

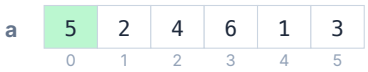
Algorithm: Insertion sort. **Input:** arr = `5,2,4,6,1,3`.

Pseudocode

```
for i = 1 .. n-1:
 key = a[i]; j = i - 1
 while j >= 0 and a[j] > key:
 a[j+1] = a[j]; j--
 a[j+1] = key
```

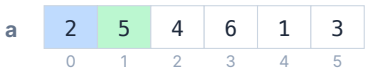
Step 1 of 6. The first element is a sorted prefix of length 1.

i = 0



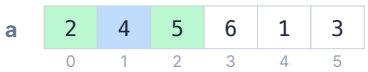
Step 2 of 6. Insert key 2 at index 0; the prefix a[0..1] is sorted.

i = 1 · key = 2 · comparisons so far = 1 · shifts so far = 1



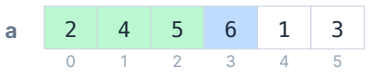
Step 3 of 6. Insert key 4 at index 1; the prefix a[0..2] is sorted.

i = 2 · key = 4 · comparisons so far = 3 · shifts so far = 2



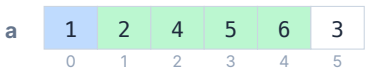
Step 4 of 6. Insert key 6 at index 3; the prefix a[0..3] is sorted.

i = 3 · key = 6 · comparisons so far = 4 · shifts so far = 2



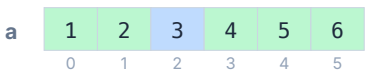
Step 5 of 6. Insert key 1 at index 0; the prefix a[0..4] is sorted.

i = 4 · key = 1 · comparisons so far = 8 · shifts so far = 6



Step 6 of 6. Insert key 3 at index 2; the prefix a[0..5] is sorted.

i = 5 · key = 3 · comparisons so far = 12 · shifts so far = 9



> **RESULT**
> [1, 2, 3, 4, 5, 6] — 12 comparisons, 9 shifts

Complexity

All three are **O(n²)** in the worst and average case; space O(1). The counters on a sorted input show bubble (with early exit) and insertion doing only n−1 comparisons while selection still does n(n−1)/2.

Common mistakes

Bubble sort without the shrinking bound

Problem

```
for (int i = 0; i < n - 1; i++) // every pass
```

Why: Correct but re-examines the already-placed tail; wastes ~half the work.

Fix: i < n - 1 - pass

Insertion sort overwriting the key

Problem

```
a[j + 1] = a[j]; // key not saved
```

Why: The value being inserted is lost.

Fix: int key = a[i]; … a[j + 1] = key;

Using > instead of >= for stability

Problem

```
while (j >= 0 && a[j] >= key)
```

Why: Moves equal elements past each other: not stable.

Fix: a[j] > key

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Bubble sort with swap count — *Beginner*

Read n numbers. Bubble-sort them; print the sorted array and, on the next line, the number of swaps performed.

Example input	Expected output
5 ⏎ 4 3 2 1 5	1 2 3 4 5 ⏎ 6

Hint 1: The swap count equals the number of inversions.

Exercise 2 · Insertion steps — *Intermediate*

Read n numbers. After each insertion step i = 1..n−1 print the whole array.

Example input	Expected output
4 ⏎ 5 2 4 1	2 5 4 1 ⏎ 2 4 5 1 ⏎ 1 2 4 5

Hint 1: Print after placing the key.

Exercise 3 · Stable by key — *Challenge*

Read n pairs `key label`. Sort them by key with a stable algorithm and print the labels in order, separated by spaces.

Example input	Expected output
5 ⏎ 2 a ⏎ 1 b ⏎ 2 c ⏎ 1 d ⏎ 2 e	b d a c e

Hint 1: Insertion sort with a strict > comparison on the keys.

Quick check

Answers are in the answer key at the end of this part.

Q1. Which of these sorts is NOT stable?

- **A.** bubble
- **B.** insertion
- **C.** selection
- **D.** merge

Q2. Insertion sort on an already sorted array takes...

- **A.** O(n)
- **B.** O(n log n)
- **C.** O(n²)
- **D.** O(1)

Q3. How many swaps does bubble sort make on {3, 2, 1}?

```
// adjacent swaps only
```

Your answer: _____

Q4. In-place means...

- **A.** the array is already sorted
- **B.** O(1) extra memory
- **C.** no comparisons
- **D.** stable

Interview / exam questions

- Which elementary sort do you prefer and why?
- What is an inversion?
- Stable vs unstable?
- Why does insertion sort suit small arrays?

Watch & read

Video: Sorting Algorithms | Bubble Sort, Selection Sort & Insertion Sort | DSA Series by Shradha Ma'am — Apna College · 34:33. *Comparing the three elementary sorts*

Video: What are Adaptive/ In-Place/ Stable Algorithms? | All Sorting algo Explained | Algorithm | Gate CSE — GATE for Geeks · 8:10. *Stability and in-place sorting*

REVISION CARD (30–60 SECONDS)

• Bubble: adjacent swaps, early exit O(n) best. Insertion: shift into sorted prefix, adaptive. Selection: ≤ n−1 swaps, always n².

• All O(n²), O(1) space; bubble & insertion stable, selection not.

• Swaps = inversions.

• n small → insertion; else O(n log n).

4.5 Merge Sort and Divide and Conquer

Data Structures & Algorithm Foundations · Intermediate · Core programming & data structures · about 45 min · ★ on the minimum-viable path

Split, sort each half, merge: a guaranteed O(n log n), stable sort — and counting inversions as a bonus.

YOU WILL BE ABLE TO

- Describe divide and conquer: divide, conquer, combine.
- Implement the merge of two sorted ranges and the recursive merge sort.
- Derive O(n log n) from the recursion tree and state its O(n) extra space.
- Use the merge step to count inversions.

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Elementary Sorting: Bubble, Insertion, Selection; Stability

The idea

Divide and conquer solves a problem by (1) **dividing** it into smaller instances of the same problem, (2) **conquering** them recursively, (3) **combining** the answers. Merge sort: split the array in half, sort each half, then **merge** the two sorted halves.

```
[38 27 43 3 9 82 10]
 ├─ [38 27 43 3] ├─ [9 82 10]
 │ ├─ [38 27] → [27 38] │ ├─ [9] [82 10] → [10 82]
 │ └─ [43 3] → [3 43] │ └─ merge → [9 10 82]
 └─ merge → [3 27 38 43]
merge → [3 9 10 27 38 43 82]
```

WHY THIS WORKS

Merging two sorted lists is easy and linear: keep a pointer at the front of each and repeatedly take the smaller front element. The recursion has $\log_2 n$ levels (halving), and every level touches each element once while merging → $n \cdot \log_2 n$. Unlike quick sort the split is always exactly in the middle, so the worst case is the same as the average. Taking the LEFT element on ties makes the sort **stable**.

The same merge can **count inversions**: whenever an element of the right half is taken before the remaining left elements, it jumps over all of them — add `mid - i` to the counter. That counts every out-of-order pair in O(n log n).

Syntax

Skeleton (half-open ranges)

```
static void mergeSort(int[] a, int[] tmp, int lo, int hi) { // sorts a[lo .. hi)
 if (hi - lo < 2) return; // base case
 int mid = (lo + hi) >>> 1;
 mergeSort(a, tmp, lo, mid);
 mergeSort(a, tmp, mid, hi);
 int i = lo, j = mid, k = lo;
 while (i < mid && j < hi) tmp[k++] = a[i] <= a[j] ? a[i++] : a[j++];
 while (i < mid) tmp[k++] = a[i++];
 while (j < hi) tmp[k++] = a[j++];
 System.arraycopy(tmp, lo, a, lo, hi - lo); // copy the merged run back
}
mergeSort(a, new int[a.length], 0, a.length);
```

> **TIP**
> `System.arraycopy(src, srcPos, dest, destPos, length)` and `Arrays.copyOfRange(a, from, to)` are the fast ways to move blocks of an array.

A tiny example

```
// merge a[lo..mid) and a[mid..hi) using tmp
while (i < mid && j < hi) tmp[k++] = a[i] <= a[j] ? a[i++] : a[j++];
while (i < mid) tmp[k++] = a[i++];
while (j < hi) tmp[k++] = a[j++];
```

Guided example

MergeSortDemo.java — Merge sort (optimised solution)

```
1  import java.util.Arrays;
2
3  public class MergeSortDemo {
4   static long comparisons;
5
6   // DIVIDE AND CONQUER: split in half, sort each half recursively, MERGE the two sorted
        halves.
7   static void mergeSort(int[] a, int lo, int hi, int[] tmp) {
8   if (hi - lo < 2) return; // 0 or 1 element: already sorted                    (1)
9   int mid = (lo + hi) >>> 1;                                                   (2)
10  mergeSort(a, lo, mid, tmp);
11  mergeSort(a, mid, hi, tmp);
12  merge(a, lo, mid, hi, tmp);
13  }
14
15  // merge a[lo..mid) and a[mid..hi), both sorted, into sorted order
16  static void merge(int[] a, int lo, int mid, int hi, int[] tmp) {
17  int i = lo, j = mid, k = lo;
18  while (i < mid && j < hi) {
19  comparisons++;
20  if (a[i] <= a[j]) tmp[k++] = a[i++]; // <= takes the left one on ties: STABLE  (3)
21  else tmp[k++] = a[j++];
22  }
23  while (i < mid) tmp[k++] = a[i++];
24  while (j < hi) tmp[k++] = a[j++];
```

(code continued)

```
25    System.arraycopy(tmp, lo, a, lo, hi - lo);                                    4
26    }
27
28    // The same merge also counts INVERSIONS (pairs i<j with a[i] > a[j]) in O(n log n).
29    static long countInversions(int[] a, int lo, int hi, int[] tmp) {
30    if (hi - lo < 2) return 0;
31    int mid = (lo + hi) >>> 1;
32    long inv = countInversions(a, lo, mid, tmp) + countInversions(a, mid, hi, tmp);
33    int i = lo, j = mid, k = lo;
34    while (i < mid && j < hi) {
35    if (a[i] <= a[j]) tmp[k++] = a[i++];
36    else {
37    inv += mid - i; // a[j] jumps over every remaining left element              5
38    tmp[k++] = a[j++];
39    }
40    }
41    while (i < mid) tmp[k++] = a[i++];
42    while (j < hi) tmp[k++] = a[j++];
43    System.arraycopy(tmp, lo, a, lo, hi - lo);
44    return inv;
45    }
46
47    public static void main(String[] args) {
48    int[] a = {38, 27, 43, 3, 9, 82, 10};
49    int[] copy = a.clone();
50    mergeSort(a, 0, a.length, new int[a.length]);
51    System.out.println(Arrays.toString(a) + " comparisons=" + comparisons);
52    System.out.println("inversions in " + Arrays.toString(copy) + " = " +
          countInversions(copy, 0, copy.length, new int[copy.length]));
53
54    int n = 1 << 16; // 65 536 elements, reverse sorted
55    int[] big = new int[n];
56    for (int i = 0; i < n; i++) big[i] = n - i;
57    comparisons = 0;
58    mergeSort(big, 0, n, new int[n]);
59    System.out.println("n = " + n + ": " + comparisons + " comparisons (n log2 n = " + n *
          16L + ")");
60    }
61   }
```

Half-open ranges [lo, hi) and one reusable temporary array.

Line-by-line walkthrough

#	Where	What happens
1	line 8	Base case: 0 or 1 elements are sorted.
2	line 9	Unsigned shift: overflow-safe midpoint.
3	line 20	<= takes from the left on ties: stability.
4	line 25	Copy the merged range back into the original array.
5	line 37	a[j] is smaller than all remaining left elements: that is (mid − i) inversions at once.

Dry run: watch it execute

Small sorted runs merge into bigger ones.

Algorithm: Merge sort. **Input:** arr = `38,27,43,3,9,82,10`.

Pseudocode

```
mergeSort(lo, hi):
 if hi − lo < 2: return
 mid = (lo + hi) / 2
 mergeSort(lo, mid); mergeSort(mid, hi)
 merge a[lo..mid) and a[mid..hi)
```

Step 1 of 7. Unsorted array.

comparisons = 0

a

0	1	2	3	4	5	6
38	27	43	3	9	82	10

Step 2 of 7. Merge [27] and [43] → [27, 43] (positions 1–2).

lo = 1 · mid = 2 · hi = 3 · depth = 2 · comparisons = 1

a

0	1	2	3	4	5	6
38	27	43	3	9	82	10

Step 3 of 7. Merge [38] and [27, 43] → [27, 38, 43] (positions 0–2).

lo = 0 · mid = 1 · hi = 3 · depth = 1 · comparisons = 3

a

0	1	2	3	4	5	6
27	38	43	3	9	82	10

Step 4 of 7. Merge [3] and [9] → [3, 9] (positions 3–4).

lo = 3 · mid = 4 · hi = 5 · depth = 2 · comparisons = 4

a

0	1	2	3	4	5	6
27	38	43	3	9	82	10

Step 5 of 7. Merge [82] and [10] → [10, 82] (positions 5–6).

lo = 5 · mid = 6 · hi = 7 · depth = 2 · comparisons = 5

a

0	1	2	3	4	5	6
27	38	43	3	9	10	82

Step 6 of 7. Merge [3, 9] and [10, 82] → [3, 9, 10, 82] (positions 3–6).

lo = 3 · mid = 5 · hi = 7 · depth = 1 · comparisons = 7

a

0	1	2	3	4	5	6
27	38	43	3	9	10	82

Step 7 of 7. Merge [27, 38, 43] and [3, 9, 10, 82] → [3, 9, 10, 27, 38, 43, 82] (positions 0–6).

lo = 0 · mid = 3 · hi = 7 · depth = 0 · comparisons = 13

a

0	1	2	3	4	5	6
3	9	10	27	38	43	82

> **RESULT**
> [3, 9, 10, 27, 38, 43, 82] — 13 comparisons

Complexity

$$T(n) = 2 \cdot T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$$

Every level of the recursion tree does about n work and there are $\log_2 n$ levels. For n = 65 536 the demo needs 524 288 comparisons (about $n \cdot \log_2 n/2$, since merging stops when one side empties). Extra space **O(n)** for the temporary array; recursion depth O(log n). Merge sort is the algorithm behind `List.sort` for objects (as TimSort, a merge-sort hybrid).

Common mistakes

Allocating a new array in every merge

Problem

```
int[] tmp = new int[hi - lo]; // inside merge
```

Why: Many small allocations slow it down (still correct).

Fix: Allocate one tmp array of size n once.

Forgetting the leftovers

Problem

```
// after the main while loop, nothing else
```

Why: When one side runs out the other side's remaining elements are never copied.

Fix: while (i < mid) ...; while (j < hi) ...;

Wrong midpoint for half-open ranges

Problem

```
mergeSort(a, lo, mid - 1); mergeSort(a, mid, hi);
```

Why: Mixing inclusive and exclusive bounds drops an element.

Fix: Use [lo, mid) and [mid, hi) consistently.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Merge sort — *Beginner*

Read n numbers and print them sorted using your own merge sort.

Example input	Expected output
7 ⏎ 38 27 43 3 9 82 10	3 9 10 27 38 43 82

Hint 1: Recurse on halves, then merge with a temp array.

Exercise 2 · Merge two sorted arrays — *Intermediate*

Read n, m and two sorted arrays. Print their merge as one sorted array.

Example input	Expected output
3 4 ⏎ 1 4 9 ⏎ 2 3 10 11	1 2 3 4 9 10 11

Hint 1: Two pointers; take the smaller front value.

Exercise 3 · Count inversions — *Challenge*

Read n (≤ 10^5) numbers and print the number of pairs i < j with a[i] > a[j]. Use O(n log n).

Example input	Expected output
5 ⏎ 2 4 1 3 5	3

Hint 1: During the merge, when you take from the right half add mid − i.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is the time complexity of merge sort in the worst case?

- **A.** O(n)
- **B.** O(n log n)
- **C.** O(n^2)
- **D.** O(log n)

Q2. Merge sort's extra space is...

- **A.** O(1)
- **B.** O(log n)
- **C.** O(n)
- **D.** O(n^2)

Q3. The three steps of divide and conquer are...

- **A.** sort, search, merge
- **B.** divide, conquer, combine
- **C.** split, copy, delete
- **D.** read, compute, print

Q4. How many inversions does {3, 1, 2} have?

```
// pairs (3,1), (3,2)
```

Your answer: _____

Interview / exam questions

- Why is merge sort O(n log n)?
- Merge sort vs quick sort?
- Why is merge sort good for linked lists?
- How do you count inversions?

Watch & read

Video: 2.7.2. Merge Sort Algorithm — Abdul Bari · 20:23. *How merge sort splits and merges*

Video: Learn Merge Sort in 13 minutes — Bro Code · 13:45. *Merge sort implementation and O(n log n)*

REVISION CARD (30–60 SECONDS)

- Divide → recurse → merge; base case size ≤ 1.
- Merge two sorted ranges in O(n); take left on ties for stability.
- T(n) = 2T(n/2) + n → O(n log n); O(n) space.
- Inversions: add mid − i when taking from the right.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- Merge sort is not itself a slide topic; it is added because divide and conquer (Karatsuba, quick sort) needs a clean worked example and "merge sort" appears in the lecture's Big-O cheat sheet as the O(n log n) example.

4.6 Iteration versus Recursion

Data Structures & Algorithm Foundations · Intermediate · Core programming & data structures · about 35 min · complete path

The same problem solved with loops and with recursion: cost, memoisation, stack depth, and converting between them.

YOU WILL BE ABLE TO

- Convert simple recursions into loops and vice versa.
- Compare time and stack costs of the two styles (factorial, Fibonacci, Josephus).
- Apply memoisation to a branching recursion.
- Decide which style is clearer and safer for a given problem.

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Loops: for, while, do-while, break and continue

The idea

Anything a recursion can compute a loop can compute (with an explicit stack if needed), and vice versa. The choice is about **clarity** and **resource use**.

	iteration	recursion
memory	O(1) loop variables	one stack frame per active call (O(depth))
speed	no call overhead	small overhead per call
clarity	good for linear scans	excellent for trees, backtracking, divide and conquer
risk	off-by-one in bounds	StackOverflowError for very deep chains
when	simple counting / accumulating	the problem is defined in terms of smaller copies of itself

WHY THIS WORKS

Josephus, grid paths and Fibonacci show both faces. The Josephus recurrence `J(n) = (J(n−1) + k) mod n` is naturally recursive, but because each call depends only on the previous one it becomes a 3-line loop that runs bottom-up with O(1) memory. Fibonacci's two overlapping calls explode exponentially; **memoisation** (cache each answer in a map) cuts 2.7 million calls for fib(30) to 59, and the bottom-up loop needs only two variables.

- **Tail-position recursion** can always be turned into a loop mechanically; Java does not do this optimisation for you.
- Deep recursion (10^5 levels or more) risks `StackOverflowError`; iterate or use an explicit `Deque` as a stack.
- Exploring all choices (subsets, permutations, N-Queens) is much shorter recursively; converting it to loops needs an explicit stack.

Syntax

```
// recursion
long f(int n) { return n <= 1 ? 1 : n * f(n - 1); }
// the same computation as a loop
long r = 1;
for (int i = 2; i <= n; i++) r *= i;

// memoisation: a cache turns exponential recursion into linear
static long[] memo = new long[91];
static long fib(int n) {
 if (n <= 1) return n;
 if (memo[n] != 0) return memo[n];
 return memo[n] = fib(n - 1) + fib(n - 2);
}

// explicit stack replaces the call stack
Deque<Integer> stack = new ArrayDeque<>();
stack.push(start);
while (!stack.isEmpty()) { int x = stack.pop(); /* process x, push children */ }
```

A tiny example

```
// recursion
long f(int n) { return n <= 1 ? 1 : n * f(n - 1); }
// iteration
long r = 1;
for (int i = 2; i <= n; i++) r *= i;
```

Guided example

IterationVsRecursion.java — Same answers, different costs (demonstration)

```
1   import java.util.HashMap;
2   import java.util.Map;
3
4   public class IterationVsRecursion {
5    static long calls;
6
7    // factorial: loop and recursion do the same job; the loop needs O(1) stack, the
         recursion O(n)
8    static long factIterative(int n) {
9    long r = 1;
10   for (int i = 2; i <= n; i++) r *= i;
11   return r;
12   }
13
14   static long factRecursive(int n) {
15   return n <= 1 ? 1 : n * factRecursive(n - 1);
16   }
17
18   // Fibonacci three ways
19   static long fibRecursive(int n) {
20   calls++;
21   return n <= 1 ? n : fibRecursive(n - 1) + fibRecursive(n - 2);
22   }
23
```

(code continued)

```
    static Map<Integer, Long> memo = new HashMap<>();

    static long fibMemo(int n) {                                                    (1)
    calls++;
    if (n <= 1) return n;
    Long cached = memo.get(n);
    if (cached != null) return cached;
    long v = fibMemo(n - 1) + fibMemo(n - 2);
    memo.put(n, v);
    return v;
    }

    static long fibIterative(int n) {                                               (2)
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) {
    long next = a + b;
    a = b;
    b = next;
    }
    return a;
    }

    // a recursion that is NOT easy to turn into a loop: it explores a tree (subset sum)
    static boolean subsetSum(int[] a, int i, int target) {                          (3)
    if (target == 0) return true;
    if (i == a.length || target < 0) return false;
    return subsetSum(a, i + 1, target - a[i]) || subsetSum(a, i + 1, target);
    }

    // recursion converted to an explicit stack to avoid StackOverflowError on a long chain
    static long sumRecursive(int n) {
    return n == 0 ? 0 : n + sumRecursive(n - 1);
    }

    static long sumWithExplicitStack(int n) {
    java.util.ArrayDeque<Integer> stack = new java.util.ArrayDeque<>();
    for (int i = n; i > 0; i--) stack.push(i);
    long total = 0;
    while (!stack.isEmpty()) total += stack.pop();
    return total;
    }

    public static void main(String[] args) {
    System.out.println(factIterative(10) + " " + factRecursive(10));
    calls = 0;
    System.out.println("fibRecursive(30) = " + fibRecursive(30) + " in " + calls + " calls")
        ;
    calls = 0;
    System.out.println("fibMemo(30) = " + fibMemo(30) + " in " + calls + " calls");
    System.out.println("fibIterative(30) = " + fibIterative(30) + " in 30 loop steps");
    System.out.println("subsetSum {3,34,4,12,5,2}, 9 -> " + subsetSum(new int[] {3, 34, 4,
        12, 5, 2}, 0, 9));
    try {
    System.out.println(sumRecursive(10_000_000));
    } catch (StackOverflowError e) {                                                (4)
    System.out.println("StackOverflowError for depth 10 000 000");
    }
```

(code continued)

```
79      System.out.println("explicit stack: " + sumWithExplicitStack(1_000_000));
80      }
81    }
```

fib(30) takes 2 692 537 calls recursively, 59 with memoisation and 30 loop steps iteratively.

Line-by-line walkthrough

#	Where	What happens
1	line 26	Check the cache first; compute and store only when missing.
2	line 36	Bottom-up: keep only the last two values.
3	line 47	Two choices per element: take it or leave it — a natural recursion.
4	line 76	Depth 10 000 000 overflows the default stack; the explicit-stack loop below handles it.

Dry run: watch it execute

Cache hits replace the exponential repeated work.

Algorithm: Memoised Fibonacci. **Input:** n = `7`.

Pseudocode

```
fib(n):
 if n <= 1: return n
 if memo has n: return memo[n] // cache hit
 memo[n] = fib(n-1) + fib(n-2)
 return memo[n]
```

Step 1 of 13. fib(1) is a base case → 1.

call = 7 · k = 1 · cache hits = 0

memo[0..n]

0	1	2	3	4	5	6	7
·	·	·	·	·	·	·	·

Step 2 of 13. fib(0) is a base case → 0.

call = 8 · k = 0 · cache hits = 0

memo[0..n]

0	1	2	3	4	5	6	7
·	·	·	·	·	·	·	·

Step 3 of 13. fib(2) = fib(1) + fib(0) = 1; store it.

call = 8 · k = 2 · cache hits = 0

memo[0..n]

0	1	2	3	4	5	6	7
·	·	1	·	·	·	·	·

Step 4 of 13. fib(1) is a base case → 1.

call = 9 · k = 1 · cache hits = 0

memo[0..n]

0	1	2	3	4	5	6	7
·	·	1	·	·	·	·	·

Step 5 of 13. fib(3) = fib(2) + fib(1) = 2; store it.

call = 9 · k = 3 · cache hits = 0

memo[0..n]

0	1	2	3	4	5	6	7
·	·	1	2	·	·	·	·

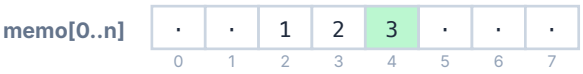
Step 6 of 13. fib(2) is already in the cache: reuse 1 (no recursion).

call = 10 · k = 2 · cache hits = 1



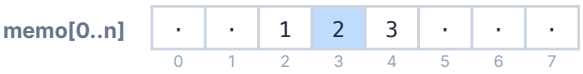
Step 7 of 13. fib(4) = fib(3) + fib(2) = 3; store it.

call = 10 · k = 4 · cache hits = 1



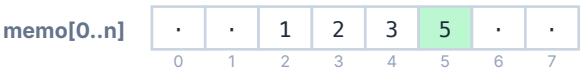
Step 8 of 13. fib(3) is already in the cache: reuse 2 (no recursion).

call = 11 · k = 3 · cache hits = 2



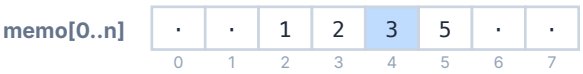
Step 9 of 13. fib(5) = fib(4) + fib(3) = 5; store it.

call = 11 · k = 5 · cache hits = 2



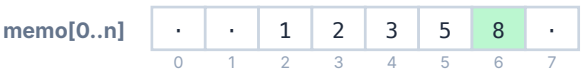
Step 10 of 13. fib(4) is already in the cache: reuse 3 (no recursion).

call = 12 · k = 4 · cache hits = 3



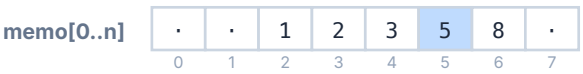
Step 11 of 13. fib(6) = fib(5) + fib(4) = 8; store it.

call = 12 · k = 6 · cache hits = 3



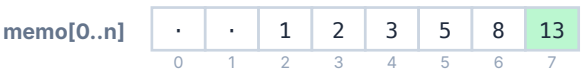
Step 12 of 13. fib(5) is already in the cache: reuse 5 (no recursion).

call = 13 · k = 5 · cache hits = 4



Step 13 of 13. fib(7) = fib(6) + fib(5) = 13; store it.

call = 13 · k = 7 · cache hits = 4



> **RESULT**
> fib(7) = 13 using 13 calls (naive recursion needs far more)

Complexity

problem	recursive time	iterative time	extra space
factorial	O(n)	O(n)	O(n) stack vs O(1)
fib naive	$O(\varphi^n)$	O(n)	O(n) stack vs O(1)
fib memoised	O(n)	—	O(n)
Josephus	O(n)	O(n)	O(n) stack vs O(1)

Common mistakes

Using naive recursion for Fibonacci/DP

Problem

```
fib(50) // ~2·10^10 calls
```

Why: Exponential time.

Fix: Memoise or iterate.

Recursing 10^6 deep

Problem

```
sumRecursive(1_000_000)
```

Why: StackOverflowError on default settings.

Fix: Loop (or raise -Xss only as a last resort).

Forgetting the cache check

Problem

```
long v = fib(n-1) + fib(n-2); memo.put(n, v); // never reads memo
```

Why: No benefit: still exponential.

Fix: if (memo.containsKey(n)) return memo.get(n);

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · gcd both ways — *Beginner*

Read a and b. Print the gcd computed iteratively and recursively, separated by a space.

Example input	Expected output
84 36	12 12

Hint 1: The loop and the recursion use the same rule.

Exercise 2 · Climbing stairs — *Intermediate*

You may climb 1 or 2 steps at a time. Read n and print the number of distinct ways to climb n steps, using a loop.

Example input	Expected output
10	89

Hint 1: ways(n) = ways(n−1) + ways(n−2): Fibonacci.

Exercise 3 · Reverse a string — *Beginner*

Read a word. Print its reverse computed recursively, a space, then computed with a loop.

Example input	Expected output
stressed	desserts desserts

Hint 1: reverse(s) = reverse(s.substring(1)) + s.charAt(0).

Quick check

Answers are in the answer key at the end of this part.

Q1. Which uses O(1) extra space for factorial?

- **A.** recursion
- **B.** loop
- **C.** both
- **D.** neither

Q2. Memoisation turns naive Fibonacci from...

- **A.** O(n) to O(1)
- **B.** $O(2^n)$ to O(n)
- **C.** $O(n^2)$ to O(n)
- **D.** O(n) to O(log n)

Q3. Which is best solved by recursion?

- **A.** sum of an array
- **B.** all subsets of a set
- **C.** maximum of an array
- **D.** counting digits

Q4. What error may deep recursion cause?

- **A.** OutOfMemoryError: heap
- **B.** StackOverflowError
- **C.** ArithmeticException
- **D.** ClassCastException

Interview / exam questions

- How would you convert recursion to iteration?
- What is memoisation?
- Tail recursion?
- When is recursion clearer?

Watch & read

Video: Recursion vs Iteration : Understand with Printing Fibonacci Series Example — PrepBytes · 12:52. *When to use recursion vs loops*

Video: Fibonacci Sequence - Recursion with memoization — mycodeschool · 9:12. *Memoising a recursion*

REVISION CARD (30–60 SECONDS)

- Loop O(1) space; recursion O(depth) stack.
- Branching recursion → memoise / bottom-up.
- Deep chains → loop or explicit stack.
- Trees, backtracking, D&C → recursion reads best.

4.7 Time Complexity and Big-O Notation

Data Structures & Algorithm Foundations · Beginner · Core programming & data structures · about 45 min · ★ on the minimum-viable path

Measure an algorithm by how its work grows with the input: Big-O, Big-Ω, Big-Θ, the common classes and the cheat sheet.

YOU WILL BE ABLE TO

- Define time complexity as a count of basic operations as a function of input size n.
- State and use Big-O (upper bound), Big-Ω (lower bound) and Big-Θ (tight bound).
- Order the common classes from O(1) to O(n!) and give an example of each.
- Simplify an operation count by dropping constants and lower-order terms.

Before this lesson: Loops: for, while, do-while, break and continue · What Is an Algorithm?

The idea

We cannot compare algorithms by stopwatch: the answer depends on the computer, the language and the data. Instead we **count basic operations** as a function of the input size **n** and look at how that function **grows**. Constant factors and small terms stop mattering when n is large.

The Big-O cheat sheet from the lecture, with sizes.

class	name	example	n = 10	n = 1000	n = 10^6
O(1)	constant	odd or even; array access	1	1	1
O(log n)	logarithmic	binary search	3	10	20
O(√n)	square root	trial division	3	32	1000
O(n)	linear	find the maximum	10	10^3	10^6
O(n log n)	linearithmic	merge sort	33	10^4	$2 \cdot 10^7$
O(n^2)	quadratic	bubble sort	100	10^6	10^{12}
O(n^3)	cubic	naive matrix multiply	10^3	10^9	10^{18}
O(2^n)	exponential	all subsets	1024	10^{301}	—
O(n!)	factorial	all permutations	$3.6 \cdot 10^6$	—	—

WHY THIS WORKS

Big-O says "f(n) is at most proportional to g(n) for large n": f(n) = O(g(n)) means there are constants c and n_0 with $f(n) \le c \cdot g(n)$ for all $n \ge n_0$. **Big-**Ω is the matching lower bound and **Big-**Θ means both: the growth is *exactly* of that order. When people say "the time complexity is O(n log n)" they usually mean the tight bound for the *worst case*. Rules: drop constants ($3n \to n$), keep the dominant term ($n^2 + n \to n^2$), and add sequential blocks / multiply nested ones.

```
f(n) = 3n² + 10n + 7 → O(n²) (n² dominates)
for … n { … } then for … n { … } → O(n + n) = O(n)
for … n { for … n { … } } → O(n · n) = O(n²)
```

A SLIDE CORRECTION

The slide that says "quadratic time complexity means the running time increases **exponentially** with the input size" is wrong: quadratic growth is **polynomial** (n^2); exponential means 2^n.

Syntax

notation	read as	meaning
f(n) = O(g(n))	big-O	f(n) ≤ c·g(n) for all large n — an upper bound
f(n) = Ω(g(n))	big-omega	f(n) ≥ c·g(n) for all large n — a lower bound
f(n) = Θ(g(n))	big-theta	both: the growth is exactly of that order
log n	log base 2 here	times a number must be doubled to reach n

useful sum	value
1 + 2 + … + n	n(n+1)/2
1 + 2 + 4 + … + 2^k	2^k⁺¹ − 1
1 + 1/2 + 1/3 + … + 1/n	≈ ln n
n·(n−1)/2 pairs	number of pairs i < j

A tiny example

```
for (int i = 0; i < n; i++) sum += a[i]; // n additions → O(n)
```

Guided example

GrowthTable.java — Counting steps (demonstration)

```
1  public class GrowthTable {
2   static long steps;
3
4   static void constant(int n) { steps++; } // O(1)
5   static void logarithmic(int n) { for (int i = n; i > 1; i /= 2) steps++; } // O(log n)   1
6   static void linear(int n) { for (int i = 0; i < n; i++) steps++; } // O(n)
7   static void linearithmic(int n) { // O(n log n)                                          2
8   for (int i = 0; i < n; i++)
9   for (int j = 1; j < n; j *= 2) steps++;
10  }
11  static void quadratic(int n) { // O(n^2)
12  for (int i = 0; i < n; i++)
13  for (int j = 0; j < n; j++) steps++;
14  }
15  static void cubic(int n) { // O(n^3)
16  for (int i = 0; i < n; i++)
17  for (int j = 0; j < n; j++)
18  for (int k = 0; k < n; k++) steps++;
19  }
20  static void exponential(int n) { // O(2^n): all subsets                                  3
21  if (n == 0) { steps++; return; }
22  exponential(n - 1);
23  exponential(n - 1);
24  }
```

(code continued)

```
25
26    interface Algo { void run(int n); }
27
28    static long measure(Algo a, int n) {
29    steps = 0;
30    a.run(n);
31    return steps;
32    }
33
34    public static void main(String[] args) {
35    int[] sizes = {8, 16, 32, 64};
36    System.out.printf("%-12s", "n");
37    for (int n : sizes) System.out.printf("%12d", n);
38    System.out.println();
39    Object[][] rows = {
40    {"O(1)", (Algo) GrowthTable::constant},
41    {"O(log n)", (Algo) GrowthTable::logarithmic},
42    {"O(n)", (Algo) GrowthTable::linear},
43    {"O(n log n)", (Algo) GrowthTable::linearithmic},
44    {"O(n^2)", (Algo) GrowthTable::quadratic},
45    {"O(n^3)", (Algo) GrowthTable::cubic},
46    };
47    for (Object[] r : rows) {
48    System.out.printf("%-12s", r[0]);
49    for (int n : sizes) System.out.printf("%12d", measure((Algo) r[1], n));
50    System.out.println();
51    }
52    System.out.printf("%-12s", "O(2^n)");
53    for (int n : new int[] {8, 12, 16, 20}) System.out.printf("%12d",
          measure(GrowthTable::exponential, n));
54    System.out.println(" (n = 8, 12, 16, 20)");
55    }
56   }
```

4

Each method increments a static counter once per basic step.

Line-by-line walkthrough

#	Where	What happens
1	line 5	Halving: $\log_2 n$ steps (8 → 3, 64 → 6).
2	line 7	n outer × log n inner.
3	line 20	Two recursive calls of size n−1: 2^n leaves — at n = 20 already a million.
4	line 53	Smaller n for the exponential row: otherwise the program would never finish.

Dry run: watch it execute

Double n and see which columns explode.

Algorithm: How fast do the classes grow?. **Input:** max = `16`.

Pseudocode

```
for n = 1, 2, 4, 8, … :
 log2 n n n·log2 n n² 2ⁿ
```

Step 1 of 5. n = 1: everything is tiny.

n = 1

n	$\log_2 n$	$n \log_2$	n^2	2^n
1	0	0	1	2

Step 2 of 5. n = 2: doubling n added 1 to $\log_2 n$, doubled n, roughly quadrupled n^2, and SQUARED 2^n.

n = 2

n	$\log_2 n$	$n \log_2$	n^2	2^n
1	0	0	1	2
2	1	2	4	4

Step 3 of 5. n = 4: doubling n added 1 to $\log_2 n$, doubled n, roughly quadrupled n^2, and SQUARED 2^n.

n = 4

n	$\log_2 n$	$n \log_2$	n^2	2^n
1	0	0	1	2
2	1	2	4	4
4	2	8	16	16

Step 4 of 5. n = 8: doubling n added 1 to $\log_2 n$, doubled n, roughly quadrupled n^2, and SQUARED 2^n.

n = 8

n	$\log_2 n$	$n \log_2$	n^2	2^n
1	0	0	1	2
2	1	2	4	4
4	2	8	16	16
8	3	24	64	256

Step 5 of 5. n = 16: doubling n added 1 to $\log_2 n$, doubled n, roughly quadrupled n^2, and SQUARED 2^n.

n = 16

n	$\log_2 n$	$n \log_2$	n^2	2^n
1	0	0	1	2
2	1	2	4	4
4	2	8	16	16
8	3	24	64	256
16	4	64	256	65536

> **RESULT**
> at n = 16: $n^2 = 256$ but $2^n = 65536$

Complexity

This lesson *is* about complexity. Remember the hierarchy: **$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$**. For ~$10^8$ simple operations per second, n up to about 10^8 is feasible for O(n), 10^4 for $O(n^2)$, 500 for $O(n^3)$, 25 for $O(2^n)$.

Common mistakes

Keeping constants

Problem

```
"This is O(3n + 5)"
```

Why: Big-O hides constants and lower-order terms: O(n).

Fix: O(n)

Confusing O with "always"

Problem

```
"binary search is O(n²)" (technically true but useless)
```

Why: Big-O is an upper bound, so loose bounds are true but not informative.

Fix: State the tightest bound: Θ(log n).

Counting the wrong n

Problem

```
matrix n × n scanned "in O(n)"
```

Why: It has n^2 cells; scanning them is $O(n^2)$ in terms of the side length.

Fix: Say $O(R \cdot C)$ or $O(n^2)$ with n the side.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Count the triangle — *Beginner*

Read n and print how many times the innermost statement runs in: for i = 1..n, for j = 1..i, count++.

Example input	Expected output
5	15

Hint 1: 1 + 2 + … + n = n(n+1)/2.

Exercise 2 · Passes of a halving loop — *Beginner*

Read n and print how many times `n /= 2` runs in `while (n > 1)`.

Example input	Expected output
1000	9

Hint 1: $\lfloor \log_2 n \rfloor$.

Exercise 3 · The crossover point — *Challenge*

An $O(n^2)$ algorithm does n^2 steps; an $O(n \log_2 n)$ one does `factor · n · log₂ n` steps. Read factor and print the smallest n ≥ 2 for which the quadratic algorithm does MORE steps.

Example input	Expected output
10	59

Hint 1: Search upwards from n = 2.

Quick check

Answers are in the answer key at the end of this part.

Q1. Match each algorithm to its time complexity.

Item	Your match
binary search	_____
find max in unsorted array	_____
merge sort	_____
bubble sort	_____
all subsets	_____
all permutations	_____

Choose from: `O(2ⁿ)`, `O(log n)`, `O(n log n)`, `O(n!)`, `O(n)`, `O(n²)`

Q2. f(n) = $4n^2 + 100n + 7$ is...

- **A.** O(n)
- **B.** $O(n^2)$
- **C.** $O(n^3)$ only
- **D.** O(1)

Q3. Which means "tight bound"?

- **A.** Big-O
- **B.** Big-Ω
- **C.** Big-Θ
- **D.** little-o

Q4. Which is the fastest-growing?

- **A.** n^3
- **B.** 2^n
- **C.** n log n
- **D.** n^2

Q5. The time complexity of accessing an element of an array by index is...

- **A.** O(1)
- **B.** O(n)
- **C.** O(log n)
- **D.** $O(n^2)$

Interview / exam questions

- Explain Big-O, Big-Θ and Big-Ω.
- Why do we ignore constants?
- Rank 2^n, n!, n^2, n log n, log n.
- What is the complexity of nested loops with i and j ≤ i?

Watch & read

Video: Big O Notation, Time Complexity | DSA — Telusko · 21:17. *Big-O, Big-Ω and Big-Θ*

Video: Iterative Time Complexity Analysis | Examples | O(1) - O(n) - O(logn) - O(√n) - O(m+n) - O(n^2) — Badhon Nandi · 13:13. *Common complexity classes*

REVISION CARD (30–60 SECONDS)

- Count basic operations vs n; drop constants & low-order terms.
- O upper, Ω lower, Θ tight.
- $1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$
- Sequential: add; nested: multiply.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide on the time-complexity chart says quadratic time complexity "increases exponentially with the input size"; quadratic is polynomial. • Slide 10 attributes $O(n^2 \cdot 2^n)$ to brute-force travelling salesman and slide 11 $O(n!)$; both describe brute-force enumeration — the Held–Karp DP achieves $O(n^2 \cdot 2^n)$.

4.8 Algorithm Design Paradigms

Data Structures & Algorithm Foundations · Intermediate · Core programming & data structures · about 50 min · ★ on the minimum-viable path

Brute force, divide and conquer, greedy, backtracking and dynamic programming — recognise the pattern, pick the tool.

YOU WILL BE ABLE TO

- Describe each paradigm in one sentence and give an archive example.
- Decide when greedy is safe (and show a counterexample when it is not).
- Use dynamic programming to repair a failing greedy solution (coin change).
- Recognise the signals: sorted data, choices, overlapping sub-problems, constraints.

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Time Complexity and Big-O Notation

The idea

paradigm	idea	signal	archive examples
brute force	try every candidate	tiny input; need a baseline	trial division, $O(n^2)$ leaders, all-subsets sum
divide & conquer	split, solve, combine	problem splits into independent halves	merge sort, quick sort, Karatsuba, block swap
greedy	take the best local choice	a "best next step" provably extends to the optimum	weighted string, activity selection, Euclid
backtracking	build step by step, undo	constraints prune a search tree	N-Queens, maze, Hamiltonian cycle, permutations, knight's tour
dynamic programming	store answers to overlapping sub-problems	same sub-problem appears again	grid paths, Fibonacci, coin change

WHY THIS WORKS

Greedy is fast but only valid when a local best choice never ruins the global optimum (the weights 1, 3, 12, 60 each divide the next, so taking the biggest letter first is optimal). With coins {4, 3, 1} and amount 6 greedy takes 4+1+1 (3 coins) but 3+3 uses 2: greedy fails, and **dynamic programming** — which tries every coin for every smaller amount and remembers the best — is correct. **Backtracking** is brute force with early rejection: abandon a partial solution the moment it violates a constraint.

How to choose

1. Can you solve it by brute force? How big is the input? (If $n \le 20$ maybe 2^n is fine.)
2. Is there a sorted order, a monotone property or a "best choice"? Try greedy — then try to break it.
3. Do sub-problems repeat? Define the DP state and recurrence.
4. Are there constraints that cut a search tree? Backtrack.
5. Does the problem split into independent halves? Divide and conquer.

Syntax

Skeletons (pseudo-Java)

```
// GREEDY: sort, then take the best remaining choice
sort(items by key); for (item : items) if (fits(item)) take(item);

// BACKTRACKING: choose - explore - un-choose
void solve(int step) {
 if (complete(step)) { record(); return; }
 for (option : options(step)) {
 if (!valid(option)) continue; // prune
 apply(option); solve(step + 1); undo(option);
 }
}

// DYNAMIC PROGRAMMING: state, recurrence, base
long[] dp = new long[n + 1];
dp[0] = base;
for (int i = 1; i <= n; i++) dp[i] = combine(dp[i - 1], dp[i - 2] /* ... */);

// DIVIDE AND CONQUER: split, solve, combine
R solve(P p) { if (small(p)) return direct(p); return combine(solve(left(p)), solve(right(p))); }
```

A tiny example

```
// greedy fails: coins {4, 3, 1}, amount 6
// greedy: 4 + 1 + 1 = 3 coins
// optimal (DP): 3 + 3 = 2 coins
```

Guided example

Paradigms.java — Five paradigms (demonstration)

```
1  import java.util.Arrays;
2
3  public class Paradigms {
4   // 1. BRUTE FORCE: try everything. Subset sum over all 2^n subsets using bit masks.
5   static boolean subsetSumBrute(int[] a, int target) {
6   int n = a.length;
7   for (int mask = 0; mask < (1 << n); mask++) {                    (1)
8   int sum = 0;
9   for (int i = 0; i < n; i++) {
10  if ((mask >> i & 1) == 1) sum += a[i];
11  }
12  if (sum == target) return true;
13  }
14  return false;
15  }
16
17  // 2. DIVIDE AND CONQUER: maximum of an array by splitting it in halves.
18  static int maxDC(int[] a, int lo, int hi) {
19  if (lo == hi) return a[lo];
20  int mid = (lo + hi) >>> 1;
21  return Math.max(maxDC(a, lo, mid), maxDC(a, mid + 1, hi));       (2)
22  }
23
24  // 3. GREEDY: take the largest coin that fits. Optimal for canonical systems only.
```

(code continued)

```
25    static int coinsGreedy(int[] coinsDescending, int amount) {
26    int count = 0;
27    for (int c : coinsDescending) {
28    count += amount / c;                                                    3
29    amount %= c;
30    }
31    return amount == 0 ? count : -1;
32    }
33
34    // 4. DYNAMIC PROGRAMMING: fewest coins for ANY coin system; reuse the answers for
          smaller amounts.
35    static int coinsDP(int[] coins, int amount) {
36    int[] best = new int[amount + 1];
37    Arrays.fill(best, Integer.MAX_VALUE);
38    best[0] = 0;
39    for (int x = 1; x <= amount; x++) {
40    for (int c : coins) {
41    if (c <= x && best[x - c] != Integer.MAX_VALUE) {
42    best[x] = Math.min(best[x], best[x - c] + 1);                           4
43    }
44    }
45    }
46    return best[amount] == Integer.MAX_VALUE ? -1 : best[amount];
47    }
48
49    // 5. BACKTRACKING: all ways to write n as an ordered sum of 1s and 2s (print them).
50    static void ways(int remaining, String soFar, java.util.List<String> out) {
51    if (remaining == 0) {
52    out.add(soFar.trim());
53    return;
54    }
55    if (remaining >= 1) ways(remaining - 1, soFar + " 1", out); // choose 1, explore,   5
          (un-choose is implicit)
56    if (remaining >= 2) ways(remaining - 2, soFar + " 2", out);
57    }
58
59    public static void main(String[] args) {
60    System.out.println("brute force: subset of {3, 34, 4, 12, 5, 2} summing to 9? " +
          subsetSumBrute(new int[] {3, 34, 4, 12, 5, 2}, 9));
61    System.out.println("divide & conquer max: " + maxDC(new int[] {4, 17, 3, 9, 25, 1}, 0,
          5));
62    int[] canonical = {25, 10, 5, 1};
63    int[] tricky = {4, 3, 1};
64    System.out.println("greedy 63 with {25,10,5,1}: " + coinsGreedy(canonical, 63) + "
          coins");
65    System.out.println("greedy 6 with {4,3,1} : " + coinsGreedy(tricky, 6) + " coins (4+1+1)
          - NOT optimal");
66    System.out.println("DP 6 with {4,3,1} : " + coinsDP(tricky, 6) + " coins (3+3)");
67    java.util.List<String> out = new java.util.ArrayList<>();
68    ways(4, "", out);
69    System.out.println("ways to sum 4 with 1s and 2s: " + out);
70    }
71   }
```

Each static method is a minimal example of its paradigm.

Line-by-line walkthrough

#	Where	What happens
1	line 7	Brute force: every subset is one n-bit mask: 2^n candidates.
2	line 21	Divide and conquer: combine the answers of two halves.
3	line 28	Greedy: take as many of the biggest coin as possible, then move on.
4	line 42	DP: the best way for amount x extends the best way for x − c by one coin.
5	line 55	Backtracking: try a 1, then a 2, by recursion — the "undo" is implicit because the string is immutable.

Dry run: watch it execute

DP finds 3 + 3 where greedy settles for 4 + 1 + 1.

Algorithm: Fewest coins (dynamic programming). **Input:** coins = `4,3,1`, amount = `6`.

Pseudocode

```
best[0] = 0
for x = 1 .. amount:
 best[x] = INF
 for each coin c <= x:
 best[x] = min(best[x], best[x−c] + 1)
```

Step 1 of 7. best[0] = 0: no coins are needed for amount 0.

x = 0

best[x]

0
0

Step 2 of 7. Amount 1: use coin 1 on top of best[0] = 0 → 1 coin(s).

x = 1 · best[x] = 1

best[x]

0	1
0	1

Step 3 of 7. Amount 2: use coin 1 on top of best[1] = 1 → 2 coin(s).

x = 2 · best[x] = 2

best[x]

0	1	2
0	1	2

Step 4 of 7. Amount 3: use coin 3 on top of best[0] = 0 → 1 coin(s).

x = 3 · best[x] = 1

best[x]

0	1	2	1
0	1	2	3

Step 5 of 7. Amount 4: use coin 4 on top of best[0] = 0 → 1 coin(s).

x = 4 · best[x] = 1

best[x]

0	1	2	1	1
0	1	2	3	4

Step 6 of 7. Amount 5: use coin 4 on top of best[1] = 1 → 2 coin(s).

x = 5 · best[x] = 2

best[x]

0	1	2	1	1	2
0	1	2	3	4	5

Step 7 of 7. Amount 6: use coin 3 on top of best[3] = 1 → 2 coin(s).

x = 6 · best[x] = 2

best[x]	0	1	2	3	4	5	6
	0	1	2	1	1	2	2

> **RESULT**
> 2 coin(s) (greedy would use 3)

Complexity

paradigm	typical cost
brute force subsets	O(2ⁿ · n)
divide & conquer (merge sort)	O(n log n)
greedy (sort + scan)	O(n log n)
backtracking	exponential worst case, much less with pruning
DP coin change	O(amount × coins)

Common mistakes

Assuming greedy is always right

Problem

```
coinsGreedy({4, 3, 1}, 6)
```

Why: Counterexample above.

Fix: Prove it, or test against brute force/DP on small inputs.

Using DP without identifying the state

Problem

```
// "memoise everything"
```

Why: If every sub-problem is distinct there is nothing to reuse.

Fix: Write the recurrence and check that sub-problems overlap.

Backtracking without pruning

Problem

```
generate all N^N boards then test
```

Why: Hopelessly slow.

Fix: Test constraints at each step (isSafe).

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Activity selection (greedy) — *Intermediate*

Read n intervals `start end`. Print the maximum number of mutually non-overlapping intervals (touching endpoints are allowed).

Example input	Expected output
5 ⏎ 1 3 ⏎ 2 4 ⏎ 3 5 ⏎ 0 6 ⏎ 5 7	3

Hint 1: Sort by end time; always take the earliest-finishing compatible interval.

Exercise 2 · Fewest coins (DP) — *Intermediate*

Read k, the k coin values and an amount. Print the fewest coins needed, or -1.

Example input	Expected output
3 ⏎ 4 3 1 ⏎ 6	2

Hint 1: best[x] = 1 + min over coins of best[x − c].

Exercise 3 · No two adjacent ones (backtracking) — *Challenge*

Read n (≤ 10). Print all binary strings of length n with no two consecutive 1s, in increasing order.

Example input	Expected output
3	000 ⏎ 001 ⏎ 010 ⏎ 100 ⏎ 101

Hint 1: Recurse by length; only append 1 if the last char is not 1.

Quick check

Answers are in the answer key at the end of this part.

Q1. Match each problem to its paradigm.

Item	Your match
N-Queens	_____
merge sort	_____
shortest string of given weight	_____
grid paths count	_____

Choose from: `backtracking`, `divide and conquer`, `dynamic programming`, `greedy`

Q2. How many coins does the greedy method use for 30 with {25, 10, 5, 1}?

```
// greedy takes 25, then 5
```

Your answer: _____

Q3. A greedy algorithm is correct when...

- **A.** it is fast
- **B.** the best local choice leads to a global optimum
- **C.** the array is sorted
- **D.** recursion is used

Q4. Overlapping sub-problems suggest...

- **A.** greedy
- **B.** dynamic programming
- **C.** binary search
- **D.** hashing

Interview / exam questions

- Greedy vs DP?
- Give a case where greedy fails.
- Backtracking vs brute force?
- How do you recognise a DP problem?

Watch & read

Video: L-5.1: Introduction to Dynamic Programming | Greedy Vs Dynamic Programming | Algorithm(DAA) — Gate Smashers · 9:08. *Overview of algorithm design paradigms*

Video: DP#2: Coin Change Problem Number of ways to get total | Dynamic Programming | Algorithms — Jenny's Lectures CS IT · 22:32. *Greedy vs dynamic programming*

REVISION CARD (30–60 SECONDS)

• Brute force (try all), D&C (split-solve-combine), greedy (local best), backtracking (choose-explore-undo), DP (store sub-answers).

• Greedy needs proof — coins {4,3,1} break it.

• DP: state + recurrence + base.

• Prune early in backtracking.

4.9 Analysing Loops: Counting Iterations

Data Structures & Algorithm Foundations · Intermediate · Core programming & data structures · about 50 min · ★ on the minimum-viable path

A step-by-step method for the 20 classic "predict the complexity" snippets: linear, nested, doubling, harmonic, sums.

YOU WILL BE ABLE TO

- Count the iterations of a single loop from its start, condition and update.
- Handle nested loops, dependent bounds (j ≤ i), step sizes (j += i) and `break`.
- Recognise log n (doubling/halving), √n (i·i ≤ n) and harmonic n log n patterns.
- Correct the mistakes in the lecture solutions for the n⁴ and n log n problems.

Before this lesson: Time Complexity and Big-O Notation · Loops: for, while, do-while, break and continue

The idea

Method: (1) find how many times the **innermost statement** runs; (2) express it as a sum over the loop variables; (3) keep the dominant term.

loop	iterations	class
for (i = 1; i < n; i++)	n − 1	O(n)
for (i = 1; i <= √n; i++) / i·i ≤ n	√n	O(√n)
for (i = 1; i <= n; i *= 2)	$\lfloor \log_2 n \rfloor + 1$	O(log n)
while (n > 1) n /= 2	$\lfloor \log_2 n \rfloor$	O(log n)
for i<n { for j<n }	n^2	O(n^2)
for i<n { for j<n { break; } }	n	O(n)
for i<n { for (j = 1; j < n; j += i) }	n(1 + 1/2 + … + 1/n) ≈ n ln n	O(n log n)
for i<n { for (j = 1; j < n; j *= 2) }	n · $\log_2 n$	O(n log n)
for i=1..n { for j=1..i { 20× } }	20·n(n+1)/2	O(n^2)
for i { for j ≤ i^2 { for k ≤ n/2 } }	Σ i^2 · n/2 ≈ n^4/6	O(n^4)
while (s <= n) { i++; s += i; }	i(i+1)/2 > n → i ≈ √(2n)	O(√n)
while (n > 0) { n /= 10; }	number of digits	O(log n)

WHY THIS WORKS

A loop with a **multiplicative** update (`i *= 2`, `n /= 2`) shrinks/grows geometrically, so it runs a logarithmic number of times. A **dependent** inner bound (`j <= i`) turns a rectangle into a triangle: the sum 1 + 2 + … + n is n(n+1)/2, still O(n^2). The **harmonic** pattern (`j += i`) gives n/i inner iterations for each i, and 1 + 1/2 + 1/3 + … + 1/n ≈ ln n, so the total is n ln n.

Where the lecture answers need correcting

- `for i<n { for j=1; j<n; j*=2 }` is **O(n log n)** (n × log n). The lecture's derivation of "log² n" multiplies the wrong things: the outer loop is linear, not logarithmic.
- `for i<=n { for j<=i*i { for k<=n/2 } }` is **O(n⁴)** (the lecture's choice D is right, but the written explanation arrives at n³).
- `for i = n/2..n { for j<=n/2 { for k *= 2 } }` is O(n² log n): (n/2)·(n/2)·log n.

Syntax

loop header	iterations
`for (i = a; i < b; i++)`	b − a
`for (i = a; i < b; i += s)`	⌈(b − a) / s⌉
`for (i = 1; i < n; i *= c)`	⌈log_c n⌉
`for (i = n; i > 1; i /= c)`	⌊log_c n⌋
`for (i = 1; i * i <= n; i++)`	⌊√n⌋
`for (j = 0; j < i; j++)` inside `for i = 0..n−1`	0 + 1 + … + (n−1) = n(n−1)/2
`for (j = i; j <= n; j += i)` inside `for i = 1..n`	n/1 + n/2 + … + n/n ≈ n ln n

Rule: total = Σ over the outer variable of the inner count; then keep the dominant term.

A tiny example

```
for (int i = 1; i < n; i = i * 2) { // i = 1, 2, 4, 8 ... → log2(n) iterations
 work();
}
```

Guided example

LoopCounts.java — Measured iteration counts (demonstration)

```
1  import java.util.function.IntToLongFunction;
2
3  public class LoopCounts {
4   // The lecture's "predict the time complexity" snippets, executed with counters.
5   static long linear(int n) { long c = 0; for (int i = 1; i < n; i++) c++; return c; } //
        O(n)
6   static long sqrtBound(int n) { long c = 0; for (long i = 1; i * i <= n; i++) c++;
        return c; } // O(sqrt n)                                                          (1)
7   static long doubling(int n) { long c = 0; for (long i = 1; i <= n; i *= 2) c++; return
        c; } // O(log n)
8   static long halving(int n) { long c = 0; while (n > 1) { n /= 2; c++; } return c; } //
        O(log n)
9   static long nested(int n) { long c = 0; for (int i = 1; i < n; i++) for (int j = 1; j <
        n; j++) c++; return c; } // O(n^2)
10  static long withBreak(int n) { long c = 0; for (int i = 1; i < n; i++) for (int j = 1;
        j < n; j++) { c++; break; } return c; } // O(n)
11  static long harmonic(int n) { long c = 0; for (int i = 1; i < n; i++) for (int j = 1; j
        < n; j += i) c++; return c; } // O(n log n)                                      (2)
12  static long nLogN(int n) { long c = 0; for (int i = 1; i < n; i++) for (long j = 1; j <
        n; j *= 2) c++; return c; } // O(n log n)
13  static long triangle(int n) { long c = 0; for (int i = 1; i <= n; i++) for (int j = 1;
        j <= i; j++) for (int k = 1; k <= 20; k++) c++; return c; } // O(n^2)
14  static long quartic(int n) { long c = 0; for (long i = 1; i <= n; i++) for (long j = 1;
        j <= i * i; j++) c += n / 2; return c; } // O(n^4)                               (3)
15  static long sumLimit(int n) { long c = 0, s = 1, i = 1; while (s <= n) { i++; s += i; c+
        +; } return c; } // O(sqrt n)
16  static long digits(int n) { long c = 0; while (n > 0) { n /= 10; c++; } return c; } //
        O(log n)
17
18  static void row(String name, String bigO, IntToLongFunction f, int[] sizes) {       (4)
19  StringBuilder sb = new StringBuilder(String.format("%-22s %-11s", name, bigO));
20  for (int n : sizes) sb.append(String.format(" n=%-6d -> %-12d", n, f.applyAsLong(n)));
21  System.out.println(sb);
22  }
23
24  public static void main(String[] args) {
25  int[] big = {100, 1_000, 10_000};
26  int[] mid = {100, 200, 400}; // quadratic-ish: keep n small
27  int[] small = {10, 20, 40}; // quartic
28  row("for (i<n)", "O(n)", LoopCounts::linear, big);
29  row("i*i <= n", "O(sqrt n)", LoopCounts::sqrtBound, big);
30  row("i *= 2", "O(log n)", LoopCounts::doubling, big);
31  row("n /= 2", "O(log n)", LoopCounts::halving, big);
32  row("n x n", "O(n^2)", LoopCounts::nested, mid);
33  row("inner break", "O(n)", LoopCounts::withBreak, big);
34  row("j += i", "O(n log n)", LoopCounts::harmonic, big);
35  row("i x (j *= 2)", "O(n log n)", LoopCounts::nLogN, big);
36  row("triangle x 20", "O(n^2)", LoopCounts::triangle, mid);
37  row("j <= i*i, k <= n/2", "O(n^4)", LoopCounts::quartic, small);
38  row("s += i until s > n", "O(sqrt n)", LoopCounts::sumLimit, big);
39  row("digits of n", "O(log n)", LoopCounts::digits, big);
40  }
41 }
```

Verify every claim in the table above by running it.

Line-by-line walkthrough

#	Where	What happens
1	line 6	`i * i <= n` is written with long so i·i cannot overflow.
2	line 11	The inner step grows with i: total ≈ n ln n (566 for n = 100 versus n^2 = 10 000).
3	line 14	Adds n/2 inside the double loop: Σ i^2 · n/2 — the n^4 result; kept to n ≤ 40 so it finishes instantly.
4	line 18	Prints count for three n values: if the count multiplies by ~10 when n does, it is linear; by ~100, quadratic; by ~1 + small, logarithmic.

Open **Dry-run & complexity lab** → **Complexity lab** to drag n and watch the iteration counts of the same snippets update live.

Dry run: watch it execute

Every shaded cell is one execution of the inner statement.

Algorithm: Counting nested-loop iterations. **Input:** n = `6`, mode = `1`.

Pseudocode

```
count = 0
for i = 1 .. n:
 for j = 1 .. i: // mode 1 (triangle)
 count++
print count
```

Step 1 of 6. Row i = 1: the inner loop ran 1 time(s). Total so far: 1.

i = 1 · inner runs = 1 · count = 1

1	·	·	·	·	·
·	·	·	·	·	·
·	·	·	·	·	·
·	·	·	·	·	·
·	·	·	·	·	·
·	·	·	·	·	·

Step 2 of 6. Row i = 2: the inner loop ran 2 time(s). Total so far: 3.

i = 2 · inner runs = 2 · count = 3

1	·	·	·	·	·
2	3	·	·	·	·
·	·	·	·	·	·
·	·	·	·	·	·
·	·	·	·	·	·
·	·	·	·	·	·

Step 3 of 6. Row i = 3: the inner loop ran 3 time(s). Total so far: 6.

i = 3 · inner runs = 3 · count = 6

1	·	·	·	·	·
2	3	·	·	·	·
4	5	6	·	·	·
·	·	·	·	·	·
·	·	·	·	·	·
·	·	·	·	·	·

Step 4 of 6. Row i = 4: the inner loop ran 4 time(s). Total so far: 10.

i = 4 · inner runs = 4 · count = 10

1	·	·	·	·	·
2	3	·	·	·	·
4	5	6	·	·	·
7	8	9	10	·	·
·	·	·	·	·	·
·	·	·	·	·	·

Step 5 of 6. Row i = 5: the inner loop ran 5 time(s). Total so far: 15.

i = 5 · inner runs = 5 · count = 15

1	·	·	·	·	·
2	3	·	·	·	·
4	5	6	·	·	·
7	8	9	10	·	·
11	12	13	14	15	·
·	·	·	·	·	·

Step 6 of 6. Row i = 6: the inner loop ran 6 time(s). Total so far: 21.

i = 6 · inner runs = 6 · count = 21

1	·	·	·	·	·
2	3	·	·	·	·
4	5	6	·	·	·
7	8	9	10	·	·
11	12	13	14	15	·
16	17	18	19	20	21

> **RESULT**
> count = 21 (n(n+1)/2 → O(n²))

Complexity

The program itself is O(sum of the snippet costs) — dominated by the n x n rows. The point is the growth *ratios*: when n grows 10× the counts should grow ≈10× (O(n)), ≈100× (O(n²)), ≈1.3× (O(log n)), ≈3.2× (O(√n)).

Common mistakes

Treating a break as free

Problem

```
for i<n { for j<n { work(); break; } } → "O(n²)"
```

Why: The inner loop runs once per outer pass: O(n).

Fix: Count executed iterations, not the written bound.

log base confusion

Problem

```
"O(log₂ n) ≠ O(log₁₀ n)"
```

Why: They differ by a constant factor ($\log_2 n = 3.32 \log_{10} n$), which Big-O ignores. The lecture's digit-sum loop is $O(\log_{10} n) = O(\log n)$.

Fix: Write O(log n).

Adding where you should multiply

Problem

```
two nested loops "O(n + n)"
```

Why: Nested loops multiply: n × n.

Fix: Only sequential blocks add.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Tripling outer loop — *Intermediate*

Read n. Count how many times `count++` runs in: for (i = 1; i <= n; i *= 3) for (j = 0; j < n; j++) count++.

Example input	Expected output
30	120

Hint 1: The outer loop runs $\lfloor \log_3 n \rfloor + 1$ times.

Exercise 2 · Half-linear times log — *Intermediate*

Read n. Count: for (i = n/2; i >= 1; i--) for (j = 1; j <= n; j *= 2) count++.

Example input	Expected output
16	40

Hint 1: (n/2) outer × ($\lfloor \log_2 n \rfloor + 1$) inner.

Exercise 3 · Measure the growth ratio — *Challenge*

For the triangular loop (j ≤ i) let count(n) be the iteration count. Read n and print count(2n) / count(n) with one decimal.

Example input	Expected output
500	4.0

Hint 1: A ratio near 4 means quadratic growth.

Quick check

Answers are in the answer key at the end of this part.

Q1. for (i = 1; i < n; i = i * 2) runs...

- **A.** n times
- **B.** $\log_2 n$ times
- **C.** $\sqrt{n}$ times
- **D.** n/2 times

Q2. for (i=1;i<=n;i++) for (j=1;j<=i;j++) is...

- **A.** O(n)
- **B.** O(n log n)
- **C.** $O(n^2)$
- **D.** $O(2^n)$

Q3. for (i=1;i<n;i++) for (j=1;j<n;j=j+i) is...

- **A.** O(n)
- **B.** O(n log n)
- **C.** $O(n^2)$
- **D.** O(log n)

Q4. int fun(){ for(i=1;i<n;i++) for(j=1;j<sqrt(n);j++) ... } is...

- **A.** O(n)
- **B.** $O(n\sqrt{n})$
- **C.** $O(n^2)$
- **D.** O(n log n)

Q5. The loop `while(n > 0) { ans += n % 10; n /= 10; }` runs...

- **A.** n times
- **B.** $\log_{10} n$ times (the number of digits)
- **C.** 10 times
- **D.** n/10 times

Interview / exam questions

- What is the complexity of `for (i=1;i<=n;i*=2)`?
- Nested loops where the inner bound is i — complexity?
- How do you count `j += i`?
- Sum of the first n squares is...?

Watch & read

Video: Time Complexity of Nested Loops (Independent Loops) — Neso Academy · 15:44. *Counting loop iterations*

Video: Time Complexity of Nested Loops (Dependent Loops) — Neso Academy · 13:14. *Logarithmic loops and harmonic sums*

REVISION CARD (30–60 SECONDS)

- i *= c or n /= c → log n. i·i ≤ n → √n.
- Nested multiply; dependent j ≤ i → $n^2/2$; j += i → n log n.
- break/early exit changes the count.
- Verify by measuring with a counter.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• Slide 22: the written derivation of $O(\log^2 n)$ is incorrect; the total is $O(n \log n)$ (the multiple-choice option C is the right one). • Slide 26: option D (n^4) is correct but the explanation concludes n^3. • Slide 20 claims that with `break` the loop is O(n) — correct — but its prose says "the inner loop always runs only once", which is also correct; the worry is only the suggested answer list that omits O(n) in that choice set. • Slide 33 correctly derives $O(\sqrt{n})$ for `s = s + i`.

4.10 Analysing Recursion: Recurrence Relations

Data Structures & Algorithm Foundations · Intermediate · Core programming & data structures · about 40 min · complete path

Write T(n) for a recursive algorithm and solve it: linear, halving, branching, divide and conquer, and Karatsuba.

YOU WILL BE ABLE TO

- Turn a recursive method into a recurrence relation T(n).
- Solve the five common shapes by unrolling or a recursion tree.
- State the master-theorem results for T(n) = aT(n/b) + O(n^d).
- Predict the number of calls and the stack depth.

Before this lesson: Analysing Loops: Counting Iterations · Recursion: Base Case, Recursive Case and the Call Stack

The idea

The running time of a recursive algorithm satisfies an equation in terms of itself — a **recurrence**. Solving it (by unrolling or by drawing the recursion tree and adding level by level) gives the closed form.

recurrence	shape	solution	example
T(n) = T(n−1) + O(1)	chain	O(n)	factorial, linear recursion
T(n) = T(n−1) + O(n)	chain with linear work	$O(n^2)$	worst-case quick sort, selection sort
T(n) = T(n/2) + O(1)	halving	O(log n)	binary search, fast power
T(n) = 2T(n/2) + O(n)	balanced split	O(n log n)	merge sort
T(n) = 2T(n−1) + O(1)	binary branching	$O(2^n)$	Towers of Hanoi, subsets
T(n) = T(n−1) + T(n−2) + O(1)	Fibonacci	$O(\varphi^n) \approx 1.618^n$	naive Fibonacci
T(n) = 3T(n/2) + O(n)	three half-size calls	$O(n^{\log_2 3}) \approx$ n^1.585	Karatsuba

WHY THIS WORKS

Master theorem for T(n) = a·T(n/b) + O(n^d): compare the work at the root (n^d) with the work at the leaves (n^(log_b a)). If the leaves dominate (d < log_b a) → O(n^(log_b a)); if equal (d = log_b a) → O(n^d log n); if the root dominates → O(n^d). Karatsuba: a = 3, b = 2, d = 1, $\log_2 3 \approx 1.585 > 1$ → leaves dominate → O(n^1.585). Merge sort: a = 2, b = 2, d = 1 = $\log_2 2$ → O(n log n).

```
merge sort recursion tree level work levels = log2 n
 n n
 n/2 n/2 n total n·log2 n
 n/4 n/4 n/4 n/4 n
```

Syntax

```
T(n) = a · T(n / b) + f(n) a ≥ 1 subproblems of size n/b, plus f(n) work to split and combine
```

Master theorem

compare f(n) with n^(log_b a)	result
f(n) grows slower	T(n) = Θ(n^(log_b a)) (leaves dominate)
same order, f(n) = Θ(n^(log_b a))	T(n) = Θ(n^(log_b a) · log n)
f(n) grows faster (and regularity holds)	T(n) = Θ(f(n)) (root dominates)

```
// from code to recurrence: count the calls and the work outside them
int f(int n) { // T(n) = 2 T(n/2) + n
 if (n <= 1) return 1;
 int s = f(n / 2) + f(n / 2);
 for (int i = 0; i < n; i++) s++; // the + n
 return s;
}
```

A tiny example

```
static int f(int n) { return n <= 1 ? 1 : f(n - 1) + f(n - 1); } // T(n) = 2T(n-1) + 1 → 2^n
```

Guided example

RecurrenceCounts.java — Counting calls (demonstration)

```
1  public class RecurrenceCounts {
2   static long calls;
3
4   // T(n) = T(n-1) + O(1) -> n calls O(n)
5   static long linear(int n) {
6   calls++;
7   return n <= 1 ? 1 : 1 + linear(n - 1);
8   }
9
10  // T(n) = T(n/2) + O(1) -> log2 n calls O(log n)
11  static long halving(int n) {                                    (1)
12  calls++;
13  return n <= 1 ? 1 : 1 + halving(n / 2);
14  }
15
16  // T(n) = 2T(n/2) + O(n) -> n log n work (merge sort shape)
17  static long splitTwoLinear(int n) {
18  calls++;
19  if (n <= 1) return 0;
20  long work = n; // the "merge" step costs n                      (2)
21  return work + splitTwoLinear(n / 2) + splitTwoLinear(n - n / 2);
22  }
23
24  // T(n) = T(n-1) + T(n-2) + O(1) -> ~1.618^n (Fibonacci shape)
25  static long fib(int n) {
26  calls++;
27  return n <= 1 ? n : fib(n - 1) + fib(n - 2);                    (3)
28  }
29
30  // T(n) = 2T(n-1) + O(1) -> 2^n (Towers of Hanoi shape)
31  static long hanoi(int n) {
32  calls++;
33  return n == 0 ? 0 : 1 + 2 * hanoi(n - 1);                       (4)
34  }
35
36  public static void main(String[] args) {
37  System.out.println("n linear halving n*log(n) work fib calls hanoi moves");
38  for (int n : new int[] {8, 16, 24, 32}) {
39  calls = 0; linear(n); long a = calls;
40  calls = 0; halving(n); long b = calls;
41  calls = 0; long w = splitTwoLinear(n);
42  calls = 0; fib(n); long f = calls;
43  calls = 0; long h = hanoi(Math.min(n, 20));
44  System.out.printf("%-6d %-7d %-8d %-15d %-11d %d%n", n, a, b, w, f, h);
45  }
46  }
47 }
```

A static counter is incremented on entry to each recursive function.

Line-by-line walkthrough

#	Where	What happens
1	line 11	One call on n/2: $\log_2 n + 1$ calls in total.
2	line 20	The linear combine cost at each node (like the merge step).
3	line 27	Two calls of nearly equal size → exponential in n (3193 calls for n = 16, 7 million for n = 32).
4	line 33	Moves for Hanoi: $2^n - 1$.

Dry run: watch it execute

Merge sort's recursion tree: n work on each of log n levels.

Algorithm: Recursion tree for T(n) = a·T(n/b) + n. **Input:** a = `2`, b = `2`, n = `16`.

Pseudocode

```
level k: a^k nodes, each of size n / b^k
work at level k = a^k · (n / b^k)
levels continue until size 1
a = b → n per level × log n levels → θ(n log n)
```

Step 1 of 6. Level 0: 1 node of size 16 → 1 × 16 = 16 units of work. Running total 16.

level = 0 · nodes = 1 · size = 16 · work = 16 · total = 16

level	nodes	size e	work
0	1	16	16

Step 2 of 6. Level 0: 2 nodes of size 8 → 2 × 8 = 16 units of work. Running total 32.

level = 0 · nodes = 2 · size = 8 · work = 16 · total = 32

level	nodes	size e	work
0	1	16	16
0	2	8	16

Step 3 of 6. Level 0: 4 nodes of size 4 → 4 × 4 = 16 units of work. Running total 48.

level = 0 · nodes = 4 · size = 4 · work = 16 · total = 48

level	nodes	size e	work
0	1	16	16
0	2	8	16
0	4	4	16

Step 4 of 6. Level 0: 8 nodes of size 2 → 8 × 2 = 16 units of work. Running total 64.

level = 0 · nodes = 8 · size = 2 · work = 16 · total = 64

level	nodes	size e	work
0	1	16	16
0	2	8	16
0	4	4	16
0	8	2	16

Step 5 of 6. Level 0: 16 nodes of size 1 → 16 × 1 = 16 units of work. Running total 80.

level = 0 · nodes = 16 · size = 1 · work = 16 · total = 80

level	nodes	size e	work
0	1	16	16
0	2	8	16
0	4	4	16
0	8	2	16
0	16	1	16

Step 6 of 6. Total work 80. Master theorem: log_2(2) = 1.00 compared with 1 → Θ(n log n): every level costs about n.

total = 80 · log_b a = 1

> **RESULT**
> 80 operations — Θ(n log n): every level costs about n

Complexity

Space is the recursion **depth**: chain O(n), halving O(log n), balanced D&C O(log n) (+ output buffers), Fibonacci O(n). Time is the total of the tree.

Common mistakes

Counting depth instead of calls

Problem

```
"fib(n) is O(n) because it recurses n deep"
```

Why: Depth is n, but the number of calls is exponential.

Fix: Time = number of nodes in the call tree.

Forgetting the combine cost

Problem

```
merge sort "T(n) = 2T(n/2)"
```

Why: Without + O(n) the solution would be O(n), which is impossible for comparison sorting.

Fix: T(n) = 2T(n/2) + O(n)

Using the master theorem when a, b are not constants

Problem

```
T(n) = T(n−1) + O(1)
```

Why: Subtracting one is not dividing by b; solve by unrolling.

Fix: T(n) = O(n)

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Count the calls — *Intermediate*

Read n. Count the calls of f where f(n) = f(n−1) + f(n−1) for n > 1 and returns 1 for n ≤ 1 (count every call including the first).

Example input	Expected output
10	1023

Hint 1: calls(n) = 1 + 2·calls(n−1), calls(1) = 1 → $2^n - 1$.

Exercise 2 · Two half calls — *Intermediate*

g(n) = g(n/2) + g(n/2) for n > 1 (integer division), 0 for n ≤ 1. Count all calls including the first for the input n.

Example input	Expected output
16	31

Hint 1: For n a power of two the tree has 2n − 1 nodes.

Exercise 3 · Karatsuba work — *Challenge*

Define t(n): if n ≤ 1 add 1 to a counter; otherwise add n, then call t(n/2) three times. For n a power of two print the final counter value.

Example input	Expected output
8	65

Hint 1: Level k has 3^k nodes each adding n/2^k; leaves add 1.

Quick check

Answers are in the answer key at the end of this part.

Q1. Match the recurrence with its solution.

Item	Your match
T(n) = T(n−1) + 1	_____
T(n) = T(n/2) + 1	_____
T(n) = 2T(n/2) + n	_____
T(n) = 2T(n−1) + 1	_____

Choose from: `O(2ⁿ)`, `O(log n)`, `O(n log n)`, `O(n)`

Q2. By the master theorem T(n) = 3T(n/2) + n is...

- **A.** O(n)
- **B.** O(n log n)
- **C.** O(n^1.585)
- **D.** O(n²)

Q3. What is the recursion depth of merge sort?

- **A.** n
- **B.** log n
- **C.** n log n
- **D.** 1

Q4. Naive fib(n) makes about...

- **A.** n calls
- **B.** n^2 calls
- **C.** 1.618^n calls
- **D.** log n calls

Interview / exam questions

- Solve T(n) = 2T(n/2) + n.
- State the master theorem.
- Why is naive Fibonacci exponential?
- What is the worst-case recurrence of quick sort?

Watch & read

Video: L-2.9: Recurrence Relation T(n)= 2T(n/2) +cn | Recursive Tree method | Algorithm — Gate Smashers · 7:14. *Solving recurrences*

Video: L-2.6: Recurrence Relation T(n)= 8T(n/2) + n^2 | Master Theorem | Example#1 | Algorithm — Gate Smashers · 6:34. *The master theorem*

REVISION CARD (30–60 SECONDS)

- Write T(n); solve by unrolling or a recursion tree.
- T(n−1)+1 → n; T(n/2)+1 → log n; 2T(n/2)+n → n log n; 2T(n−1)+1 → 2^n.
- Master: compare n^d with n^log_b(a).
- Space = depth.

4.11 Space Complexity and the Call Stack

Data Structures & Algorithm Foundations · Beginner · Core programming & data structures · about 30 min · ★ on the minimum-viable path

How much memory an algorithm needs beyond its input: auxiliary space, data-structure cost and recursion depth.

YOU WILL BE ABLE TO

- Distinguish space complexity from auxiliary space.
- Compute the space of variables, arrays, matrices and recursion.
- Explain why recursion depth, not call count, determines stack space.
- Trade space for time and vice versa.

Before this lesson: Time Complexity and Big-O Notation · Recursion: Base Case, Recursive Case and the Call Stack

The idea

Space complexity is the total memory an algorithm uses as a function of the input size, **including the input**. **Auxiliary space** is only the *extra* memory it allocates. Sorting an array in place has O(1) auxiliary space although its space complexity is O(n) (the array itself).

what you allocate	space
a fixed number of variables	O(1)
an array of n elements	O(n)
an n × n matrix	$O(n^2)$
a recursion of depth d	O(d) stack frames
a HashMap with n entries	O(n)
a StringBuilder of length n	O(n)

WHY THIS WORKS

Each recursive call keeps its parameters and local variables in a **stack frame** until it returns. Only the calls that are active *at the same time* occupy memory, so space = **maximum depth × frame size** — not the total number of calls. `add(n) = n + add(n−1)` is O(n) space; naive `fibonacci(n)` makes exponentially many calls but is only n deep, so it needs **O(n)** stack space (the lecture's answer b); `power(b, e)` halving the exponent needs O(log n).

Time and space trade off: memoising Fibonacci spends O(n) memory to cut time from $O(2^n)$ to O(n); the iterative version needs only two variables. A hash set removes duplicates in O(n) time using O(n) memory, while sorting in place needs O(n log n) time and O(1) extra memory.

Syntax

Rough sizes (they vary by JVM; use them for orders of magnitude)

Java item (64-bit HotSpot, typical)	memory
int / float	4 bytes
long / double	8 bytes
boolean / byte	1 byte (in arrays)
reference	4 bytes (compressed) — 8 otherwise
int[n]	≈ 16 + 4n bytes
Integer object	≈ 16 bytes (+ the reference)
String of n chars	≈ 40 + n bytes

```
int[] a = new int[n]; // O(n) extra space
int[][] grid = new int[n][n]; // O(n^2)
// recursion adds one frame per active call: depth d => O(d) stack space
```

A tiny example

```
int add(int n) { return n <= 0 ? 0 : n + add(n - 1); } // O(n) stack space
```

Guided example

SpaceDemo.java — Measuring space (demonstration)

```
public class SpaceDemo {
 static int maxDepth;
 static int depth;

 // O(1) extra space: a loop with a few variables
 static long sumLoop(int[] a) {
 long s = 0;
 for (int x : a) s += x;
 return s;
 }

 // O(n) auxiliary space: builds a second array of the same size
 static int[] prefixSums(int[] a) {                    3
 int[] p = new int[a.length];
 int run = 0;
 for (int i = 0; i < a.length; i++) {
 run += a[i];
 p[i] = run;
 }
 return p;
 }

 // O(n) STACK space: depth n, even though there is no array
 static long sumRecursive(int n) {
 depth++;                                              1
 maxDepth = Math.max(maxDepth, depth);
 long r = n == 0 ? 0 : n + sumRecursive(n - 1);
```

(code continued)

```
    depth--;                                                                    2
    return r;
    }

    // O(log n) stack space: depth log2 n
    static long power(long b, int e) {
    depth++;
    maxDepth = Math.max(maxDepth, depth);
    long r;
    if (e == 0) r = 1;
    else {
    long h = power(b, e / 2);
    r = e % 2 == 0 ? h * h : h * h * b;
    }
    depth--;
    return r;
    }

    // in place vs extra copy: reversing an array
    static void reverseInPlace(int[] a) { // O(1) auxiliary                     4
    for (int i = 0, j = a.length - 1; i < j; i++, j--) {
    int t = a[i]; a[i] = a[j]; a[j] = t;
    }
    }

    static int[] reverseCopy(int[] a) { // O(n) auxiliary
    int[] r = new int[a.length];
    for (int i = 0; i < a.length; i++) r[a.length - 1 - i] = a[i];
    return r;
    }

    public static void main(String[] args) {
    int[] a = {3, 1, 4, 1, 5};
    System.out.println("sumLoop : " + sumLoop(a) + " extra space O(1)");
    System.out.println("prefixSums : " + java.util.Arrays.toString(prefixSums(a)) + " extra
        space O(n)");
    maxDepth = depth = 0;
    sumRecursive(1000);
    System.out.println("sumRecursive(1000) : max stack depth " + maxDepth + " O(n) stack
        space");
    maxDepth = depth = 0;
    power(2, 1000);
    System.out.println("power(2, 1000) : max stack depth " + maxDepth + " O(log n) stack
        space");
    int[] b = a.clone();
    reverseInPlace(b);
    System.out.println("reverseInPlace : " + java.util.Arrays.toString(b));
    System.out.println("reverseCopy : " + java.util.Arrays.toString(reverseCopy(a)));
    }
   }
```

depth/maxDepth static counters track the live stack depth.

Line-by-line walkthrough

#	Where	What happens
1	line 25	Entering a call: one more frame is live.
2	line 28	Leaving the call: the frame is released.
3	line 13	Allocates a second array of size n: O(n) auxiliary space.
4	line 47	Two indices and one temp: O(1) auxiliary space.

Dry run: watch it execute

One frame per active call: depth n for a chain, log n for halving (mode 2).

Algorithm: Recursion depth = stack space. **Input:** n = `6`, mode = `1`.

Pseudocode

```
sum(n): if n == 0 return 0; return n + sum(n - 1)
f(n): if n <= 1 return 0; return 1 + f(n / 2)
```

Step 1 of 14. Call sum(6): one more frame on the stack (depth 1).

depth = 1 · max depth = 1

call stack (bottom ▒ top) `sum(6)`
0

Step 2 of 14. Call sum(5): one more frame on the stack (depth 2).

depth = 2 · max depth = 2

call stack (bottom ▒ top) `sum(6)` `sum(5)`
0 1

Step 3 of 14. Call sum(4): one more frame on the stack (depth 3).

depth = 3 · max depth = 3

call stack (bottom ▒ top) `sum(6)` `sum(5)` `sum(4)`
0 1 2

Step 4 of 14. Call sum(3): one more frame on the stack (depth 4).

depth = 4 · max depth = 4

call stack (bottom ▒ top) `sum(6)` `sum(5)` `sum(4)` `sum(3)`
0 1 2 3

Step 5 of 14. Call sum(2): one more frame on the stack (depth 5).

depth = 5 · max depth = 5

call stack (bottom ▒ top) `sum(6)` `sum(5)` `sum(4)` `sum(3)` `sum(2)`
0 1 2 3 4

Step 6 of 14. Call sum(1): one more frame on the stack (depth 6).

depth = 6 · max depth = 6

call stack (bottom ▒ top) `sum(6)` `sum(5)` `sum(4)` `sum(3)` `sum(2)` `sum(1)`
0 1 2 3 4 5

Step 7 of 14. Call sum(0): one more frame on the stack (depth 7).

depth = 7 · max depth = 7

call stack (bottom ▒ top) `sum(6)` `sum(5)` `sum(4)` `sum(3)` `sum(2)` `sum(1)` `sum(0)`
0 1 2 3 4 5 6

Step 8 of 14. sum(0) returns 0; its frame is about to be popped.

depth = 7 · max depth = 7

call stack (bottom → top) `sum(6)` `sum(5)` `sum(4)` `sum(3)` `sum(2)` `sum(1)` `sum(0)`

0 1 2 3 4 5 6

Step 9 of 14. sum(1) returns 1; its frame is about to be popped.

depth = 6 · max depth = 7

call stack (bottom → top) `sum(6)` `sum(5)` `sum(4)` `sum(3)` `sum(2)` `sum(1)`

0 1 2 3 4 5

Step 10 of 14. sum(2) returns 3; its frame is about to be popped.

depth = 5 · max depth = 7

call stack (bottom → top) `sum(6)` `sum(5)` `sum(4)` `sum(3)` `sum(2)`

0 1 2 3 4

Step 11 of 14. sum(3) returns 6; its frame is about to be popped.

depth = 4 · max depth = 7

call stack (bottom → top) `sum(6)` `sum(5)` `sum(4)` `sum(3)`

0 1 2 3

Step 12 of 14. sum(4) returns 10; its frame is about to be popped.

depth = 3 · max depth = 7

call stack (bottom → top) `sum(6)` `sum(5)` `sum(4)`

0 1 2

Step 13 of 14. sum(5) returns 15; its frame is about to be popped.

depth = 2 · max depth = 7

call stack (bottom → top) `sum(6)` `sum(5)`

0 1

Step 14 of 14. sum(6) returns 21; its frame is about to be popped.

depth = 1 · max depth = 7

call stack (bottom → top) `sum(6)`

0

> **RESULT**
> result 21; maximum depth 7 → O(n): 7 frames for n = 6

Complexity

Remember the three-part habit when analysing an algorithm: **time**, **auxiliary space**, and — for recursion — **stack depth**.

Common mistakes

Counting the input as extra space

Problem

```
"in-place reverse uses O(n) space"
```

Why: The auxiliary space is O(1); the input is given.

Fix: Say "O(1) auxiliary space".

Ignoring the recursion stack

Problem

```
"recursive sum uses O(1) space — no arrays"
```

Why: n frames are live at the deepest point.

Fix: O(n) stack space.

Confusing calls with depth

Problem

```
"fib(n) uses O(2ⁿ) space"
```

Why: Calls are made sequentially along each branch; only a path of length n is live.

Fix: O(n).

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Maximum recursion depth — *Beginner*

f(n) = f(n−1) + 1 for n > 1, f(1) = 1. Read n and print the maximum number of simultaneously active calls.

Example input	Expected output
50	50

Hint 1: It equals n.

Exercise 2 · Fibonacci in O(1) space — *Intermediate*

Read n (≤ 90) and print the n-th Fibonacci number (F(0) = 0, F(1) = 1) using only two variables.

Example input	Expected output
50	12586269025

Hint 1: Keep the last two values and slide them forward.

Exercise 3 · Duplicates with O(1) extra space — *Challenge*

Read n numbers; print YES if any value appears twice. Do it by sorting the array in place (no HashSet).

Example input	Expected output
5 ⏎ 4 1 7 1 9	YES

Hint 1: After sorting, duplicates are adjacent.

Quick check

Answers are in the answer key at the end of this part.

Q1. Match each lecture snippet with its space complexity.

Item	Your match
int add(int n) recursive sum	_____
loop with sum += pairSum(i, i+1)	_____
new int[n] filled in a loop	_____
int sum; for (i<n) sum += i	_____

Item	Your match
recursive fibonacci(n) (stack)	_____
linked list with n inserts	_____

Choose from: `O(1)`, `O(n)`

Q2. Auxiliary space is...

- **A.** the space of the input
- **B.** the extra space used besides the input
- **C.** the stack only
- **D.** the heap only

Q3. A recursion that halves n each call has stack space...

- **A.** O(1)
- **B.** O(log n)
- **C.** O(n)
- **D.** $O(n^2)$

Q4. A 1000 × 1000 int matrix uses about...

- **A.** 4 KB
- **B.** 4 MB
- **C.** 4 GB
- **D.** 1 MB

Interview / exam questions

- Space complexity vs auxiliary space?
- What is the space complexity of recursive Fibonacci?
- Can you trade time for space? Example.
- Is merge sort in place?

Watch & read

Video: Space Complexity of Recursive Algorithms (The Call Stack) — Neso Academy · 13:55. *Space complexity and the call stack*

Video: Time and Space Complexity - Strivers A2Z DSA Course — take U forward · 35:16. *Time and space together*

REVISION CARD (30–60 SECONDS)

- Space = auxiliary + input; recursion = max depth × frame.
- Variables O(1); array O(n); matrix $O(n^2)$.
- fib naive: O(n) stack; halving recursion O(log n).
- Trade: memo O(n) memory for O(n) time.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- Slide 9 answers O(n) for recursive fibonacci space — correct (depth n), but the printed reason talks about "recursive calls n times"; precisely it is the depth of the call tree, n, not the number of calls.

4.12 Best, Average and Worst Case

Data Structures & Algorithm Foundations · Intermediate · Core programming & data structures · about 30 min · complete path

The same algorithm can behave very differently on different inputs: what each case means and when it matters.

YOU WILL BE ABLE TO

- Define best, average and worst case of an algorithm for inputs of the same size.
- Find the best/worst input for linear search, insertion sort and quick sort.
- Explain why worst-case Big-O is the usual guarantee, and when average case is the better description.
- Distinguish a case from a bound (O/Ω/Θ).

Before this lesson: Time Complexity and Big-O Notation · Searching: Linear and Binary Search

The idea

For a fixed input **size n**, different inputs can cost very different amounts. The **best case** is the cheapest input, the **worst case** the most expensive, and the **average case** the expected cost over some distribution of inputs.

algorithm	best	average	worst	worst-case input
linear search	O(1): target is first	O(n): ≈ n/2	O(n): last / absent	target at the end
binary search	O(1)	O(log n)	O(log n)	target absent
insertion sort	O(n) sorted	$O(n^2)$	$O(n^2)$	reverse sorted
selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	any
quick sort (last pivot)	O(n log n)	O(n log n)	$O(n^2)$	already sorted
merge sort	O(n log n)	O(n log n)	O(n log n)	any
hash table lookup	O(1)	O(1)	O(n) all keys collide	terrible hashCode

WHY THIS WORKS

The **worst case** is a *guarantee*: it holds for every input, so it is what real-time and safety-critical software designs for. The **average case** describes typical behaviour but needs an assumption about the input distribution. Quick sort illustrates both: its worst case is quadratic (and happens with a fixed last-element pivot on sorted data), yet its average is n log n with tiny constants, so it is usually the fastest in practice — randomising the pivot makes the bad input unlikely.

A *case* and a *bound* are different axes: you can give an upper (O), lower (Ω) or tight (Θ) bound for the best case, the average case or the worst case. "Linear search is Θ(1) in the best case and Θ(n) in the worst case."

Syntax

```
T_best(n) = min over inputs of size n T_worst(n) = max over inputs of size n
```

```
T_avg(n) = Σ P(input) · T(input) (for a uniform distribution: the plain mean)
```

symbol	means
O(T_worst)	an upper bound that holds for EVERY input
Ω(T_best)	no input is cheaper than this
Θ(T_avg)	typical behaviour under a stated input distribution

A tiny example

```
// linear search: target is a[0] → 1 comparison; target is last or absent → n comparisons
```

Guided example

CasesDemo.java — Measuring the cases (demonstration)

```
import java.util.Random;

public class CasesDemo {
 static long linearComparisons(int[] a, int target) {
 long c = 0;
 for (int x : a) {
 c++;
 if (x == target) break;
 }
 return c;
 }

 static long insertionComparisons(int[] a) {
 a = a.clone();
 long c = 0;
 for (int i = 1; i < a.length; i++) {
 int key = a[i], j = i - 1;
 while (j >= 0) {
 c++;
 if (a[j] <= key) break;
 a[j + 1] = a[j];
 j--;
 }
 a[j + 1] = key;
 }
 return c;
 }

 static long quickDepthWork;

 // quick sort on the LAST element as pivot: counts comparisons
 static void quick(int[] a, int lo, int hi) {
```

1

(code continued)

```
    if (lo >= hi) return;
    int pivot = a[hi], i = lo - 1;
    for (int j = lo; j < hi; j++) {
    quickDepthWork++;
    if (a[j] <= pivot) {
    i++;
    int t = a[i]; a[i] = a[j]; a[j] = t;
    }
    }
    int t = a[i + 1]; a[i + 1] = a[hi]; a[hi] = t;
    quick(a, lo, i);
    quick(a, i + 2, hi);
    }

    public static void main(String[] args) {
    int n = 1000;
    int[] a = new int[n];
    for (int i = 0; i < n; i++) a[i] = i;
    System.out.println("LINEAR SEARCH in " + n + " elements");
    System.out.println(" best (target is first) : " + linearComparisons(a, 0));
    System.out.println(" worst (target is last) : " + linearComparisons(a, n - 1));
    Random rnd = new Random(1);
    long total = 0;
    for (int t = 0; t < 10_000; t++) total += linearComparisons(a, rnd.nextInt(n));          (2)
    System.out.println(" average (10 000 random) : " + total / 10_000);

    int[] sortedIn = a.clone();
    int[] reversed = new int[n];                                                            (3)
    for (int i = 0; i < n; i++) reversed[i] = n - i;
    int[] random = new int[n];
    for (int i = 0; i < n; i++) random[i] = rnd.nextInt(n);
    System.out.println("INSERTION SORT comparisons for n = " + n);
    System.out.println(" best (sorted) : " + insertionComparisons(sortedIn));
    System.out.println(" worst (reversed) : " + insertionComparisons(reversed));
    System.out.println(" random : " + insertionComparisons(random));

    System.out.println("QUICK SORT (last-element pivot) comparisons for n = " + n);
    quickDepthWork = 0;
    quick(random.clone(), 0, n - 1);
    System.out.println(" random data : " + quickDepthWork);
    quickDepthWork = 0;
    quick(sortedIn.clone(), 0, n - 1);                                                      (4)
    System.out.println(" already sorted : " + quickDepthWork + " (worst case n(n-1)/2 = " +
        n * (n - 1) / 2 + ")");
    }
}
```

The numbers match the table: 1 / ~500 / 1000 for linear search, 999 vs 499 500 for insertion sort, 499 500 for quick sort on sorted input.

Line-by-line walkthrough

#	Where	What happens
1	line 4	Counts comparisons until the target is found.
2	line 56	Average: 10 000 random targets — about n/2 = 500.
3	line 60	The worst input for insertion sort: every element must travel to the front.
4	line 74	A sorted array with the last element as pivot makes every partition maximally unbalanced.

Dry run: watch it execute

The same scan costs 1, (n+1)/2 or n comparisons depending on the target.

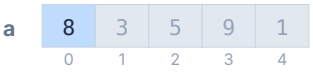
Algorithm: Best, average and worst case of linear search. **Input:** arr = `8,3,5,9,1`.

Pseudocode

```
for i in 0..n-1:
 comparisons++
 if a[i] == target: return i
return -1 // after n comparisons
```

Step 1 of 7. target = 8 (at index 0): the scan makes 1 comparison — the BEST case.

target = 8 · comparisons = 1



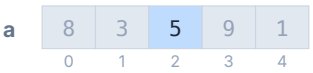
Step 2 of 7. target = 3 (at index 1): the scan makes 2 comparisons.

target = 3 · comparisons = 2



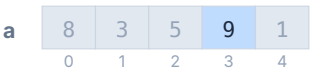
Step 3 of 7. target = 5 (at index 2): the scan makes 3 comparisons.

target = 5 · comparisons = 3



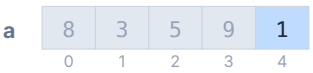
Step 4 of 7. target = 9 (at index 3): the scan makes 4 comparisons.

target = 9 · comparisons = 4



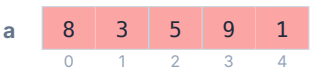
Step 5 of 7. target = 1 (at index 4): the scan makes 5 comparisons — the WORST successful case.

target = 1 · comparisons = 5



Step 6 of 7. A target that is not in the array costs all 5 comparisons — the worst case overall.

comparisons = 5



Step 7 of 7. If every position is equally likely, the average number of comparisons is (1 + 2 + ... + 5) / 5 = 3, i.e. (n + 1) / 2.

best = 1 · average = 3 · worst = 5

> **RESULT**
> best 1, average 3, worst 5 comparisons

Complexity

Always specify the **case** with the bound. If an interviewer says "what is the complexity?" give the worst case, then mention the average if it differs importantly.

Common mistakes

Quoting the best case as the complexity

Problem

```
"insertion sort is O(n)"
```

Why: Only for (nearly) sorted input; the guarantee is $O(n^2)$.

Fix: $O(n^2)$ worst/average, $O(n)$ best.

Assuming "average" means "typical for my data"

Problem

```
"average O(1) hashing" for adversarial keys
```

Why: The average assumes a good hash function and non-adversarial input.

Fix: Know the assumption; use randomised/secure hashing if needed.

Mixing case and bound

Problem

```
"O is the worst case, Ω the best case"
```

Why: O/Ω/Θ are bounds on a function; each case is itself a function that can have all three.

Fix: Say "$\Theta(n^2)$ in the worst case".

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Comparisons in a linear search — *Beginner*

Read n, a target and n numbers. Print how many comparisons a linear search makes (n if the target is absent).

Example input	Expected output
5 7 ⏎ 4 9 7 1 7	3

Hint 1: Stop counting at the first match.

Exercise 2 · Average successful search — *Beginner*

Read n. Print the average number of comparisons of a successful linear search over n elements when every position is equally likely, with one decimal.

Example input	Expected output
9	5.0

Hint 1: (1 + 2 + ... + n) / n = (n + 1) / 2.

Exercise 3 · Insertion sort comparisons — *Intermediate*

Read n numbers and print the number of comparisons insertion sort makes (compare the key with elements to its left until one is ≤ key).

Example input	Expected output
`5 ⏎ 5 4 3 2 1`	`10`

Hint 1: A reversed array of 5 gives 4+3+2+1.

Quick check

Answers are in the answer key at the end of this part.

Q1. Worst-case input for insertion sort?

- **A.** sorted
- **B.** reverse sorted
- **C.** all equal
- **D.** random

Q2. Quick sort with the last element as pivot is $O(n^2)$ on...

- **A.** random data
- **B.** already sorted data
- **C.** all different data
- **D.** never

Q3. Which case is a guarantee for every input?

- **A.** best
- **B.** average
- **C.** worst
- **D.** typical

Q4. Merge sort's best, average and worst case are...

- **A.** O(n), O(n log n), $O(n^2)$
- **B.** all O(n log n)
- **C.** all O(n)
- **D.** O(log n), O(n), $O(n^2)$

Interview / exam questions

- Best vs average vs worst case?
- Why is quick sort used despite $O(n^2)$ worst case?
- How do you avoid the worst case?
- What is amortised analysis?

Watch & read

Video: 1.11 Best Worst and Average Case Analysis — Abdul Bari · 18:56. *Best, average and worst case*

Video: L-1.3: Asymptotic Notations | Big O | Big Omega | Theta Notations | Most Imp Topic Of Algorithm — Gate Smashers · 14:25. *Cases versus asymptotic bounds*

REVISION CARD (30–60 SECONDS)

- Case = which input; bound = O/Ω/Θ of its cost.
- Linear search 1 / n/2 / n; insertion sort n / n^2 / n^2; quick sort n log n / n log n / n^2.
- Worst case = guarantee.
- Randomise the pivot.

Part 4 — Solutions to the exercises

Try the exercises before reading these. Each solution is the exact file that was compiled and tested.

4.1 Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables

Exercise 1 · Reverse a linked list

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static class Node {
 int v;
 Node next;
 Node(int v) { this.v = v; }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 Node head = null;
 Node tail = null;
 for (int i = 0; i < n; i++) {
 Node node = new Node(sc.nextInt());
 if (head == null) head = tail = node;
 else {
 tail.next = node;
 tail = node;
 }
 }
 // reverse
 Node prev = null;
 for (Node c = head; c != null; ) {
 Node nx = c.next;
 c.next = prev;
 prev = c;
 c = nx;
 }
 StringBuilder sb = new StringBuilder();
 for (Node c = prev; c != null; c = c.next) sb.append(sb.length() == 0 ? "" : " ").append(c.v);
 System.out.println(sb);
 }
}
```

Why it works: One pass flipping each link.

Verified against 2 test cases: `5⏎1 2 3 4 5⏎` → `5 4 3 2 1`; `1⏎9⏎` → `9`

Exercise 2 · Palindromic list

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Is a linked list of n numbers a palindrome? Use an array to hold the values (simple version).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 boolean ok = true;
 for (int i = 0, j = n - 1; i < j; i++, j--) if (a[i] != a[j]) ok = false;
 System.out.println(ok ? "YES" : "NO");
 }
}
```

Why it works: Compare the i-th from the front with the i-th from the back.

Verified against 3 test cases: `5⏎1 2 3 2 1⏎` → `YES`; `3⏎1 2 3⏎` → `NO`; `1⏎7⏎` → `YES`

Exercise 3 · Circular queue

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // A circular queue (ring buffer) of capacity c. Commands: enq x -> ok/full ; deq -> value/empty.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int cap = sc.nextInt();
 int n = sc.nextInt();
 int[] data = new int[cap];
 int head = 0, size = 0;
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) {
 String op = sc.next();
 if (op.equals("enq")) {
 int x = sc.nextInt();
 if (size == cap) sb.append("full ");
 else {
 data[(head + size) % cap] = x;
 size++;
 sb.append("ok ");
 }
 } else {
 if (size == 0) sb.append("empty ");
 else {
 sb.append(data[head]).append(' ');
 head = (head + 1) % cap;
 size--;
 }
 }
 }
 System.out.println(sb.toString().trim());
 }
}
```

Why it works: Modulo arithmetic turns the array into a ring.

Verified against 2 test cases: `2 6⏎enq 1⏎enq 2⏎enq 3⏎deq⏎enq 3⏎deq⏎` → `ok ok full 1 ok 2`; `1 2⏎deq⏎enq 5⏎` → `empty ok`

4.2 Trees, Graphs and Heaps

Exercise 1 · Height of a BST

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static class T { int k; T l, r; T(int k) { this.k = k; } }

 static T ins(T t, int k) {
 if (t == null) return new T(k);
 if (k < t.k) t.l = ins(t.l, k);
 else if (k > t.k) t.r = ins(t.r, k);
 return t;
 }

 static int height(T t) { return t == null ? 0 : 1 + Math.max(height(t.l), height(t.r)); }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 T root = null;
 for (int i = 0; i < n; i++) root = ins(root, sc.nextInt());
 System.out.println(height(root));
 }
}
```

Why it works: Recursive definition mirrors the tree.

Verified against 3 test cases: `7⏎50 30 70 20 40 60 80⏎` → `3`; `5⏎1 2 3 4 5⏎` → `5`; `1⏎7⏎` → `1`

Exercise 2 · Connected components

Ex2.java

```java
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.List;
import java.util.Queue;
import java.util.Scanner;

public class Ex2 {
 // Number of connected components of an undirected graph.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int m = sc.nextInt();
 List<List<Integer>> adj = new ArrayList<>();
 for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
 for (int i = 0; i < m; i++) {
 int a = sc.nextInt(), b = sc.nextInt();
 adj.get(a).add(b);
 adj.get(b).add(a);
 }
 boolean[] seen = new boolean[n];
 int components = 0;
 for (int s = 0; s < n; s++) {
 if (seen[s]) continue;
 components++;
 Queue<Integer> q = new ArrayDeque<>();
 q.add(s);
 seen[s] = true;
 while (!q.isEmpty()) {
 int u = q.poll();
 for (int v : adj.get(u)) {
 if (!seen[v]) {
 seen[v] = true;
 q.add(v);
 }
 }
 }
 }
 System.out.println(components);
 }
}
```

Why it works: Each BFS/DFS discovers exactly one component.

Verified against 3 test cases: `6 3⏎0 1⏎1 2⏎3 4⏎` → `3`; `3 0⏎` → `3`; `2 1⏎0 1⏎` → `1`

Exercise 3 · Shortest path length

Ex3.java

```java
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Queue;
import java.util.Scanner;

public class Ex3 {
 // Fewest edges from vertex 0 to vertex n-1 in an undirected graph, or -1.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int m = sc.nextInt();
 List<List<Integer>> adj = new ArrayList<>();
 for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
 for (int i = 0; i < m; i++) {
 int a = sc.nextInt(), b = sc.nextInt();
 adj.get(a).add(b);
 adj.get(b).add(a);
 }
 int[] dist = new int[n];
 Arrays.fill(dist, -1);
 Queue<Integer> q = new ArrayDeque<>();
 q.add(0);
 dist[0] = 0;
 while (!q.isEmpty()) {
 int u = q.poll();
 for (int v : adj.get(u)) {
 if (dist[v] == -1) {
 dist[v] = dist[u] + 1;
 q.add(v);
 }
 }
 }
 System.out.println(dist[n - 1]);
 }
}
```

Why it works: BFS reaches each vertex first by a shortest path.

Verified against 3 test cases: `5 5⏎0 1⏎0 2⏎1 3⏎2 3⏎3 4⏎` → `3`; `3 1⏎0 1⏎` → `-1`; `1 0⏎` → `0`

4.3 Searching: Linear and Binary Search

Exercise 1 · First occurrence

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int target = sc.nextInt();
 int found = -1;
 for (int i = 0; i < n; i++) {
 int x = sc.nextInt();
 if (x == target && found == -1) found = i;
 }
 System.out.println(found);
 }
}
```

Why it works: Linear search remembers the first match.

Verified against 2 test cases: `5 7⏎4 9 7 1 7⏎` → `2`; `3 5⏎1 2 3⏎` → `-1`

Exercise 2 · Binary search

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Binary search in a sorted array; print the index or -1.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int target = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int lo = 0, hi = n - 1, ans = -1;
 while (lo <= hi) {
 int mid = lo + (hi - lo) / 2;
 if (a[mid] == target) { ans = mid; break; }
 if (a[mid] < target) lo = mid + 1;
 else hi = mid - 1;
 }
 System.out.println(ans);
 }
}
```

Why it works: O(log n) with the standard invariant.

Verified against 3 test cases: `6 9⏎1 3 5 7 9 11⏎` → `4`; `3 4⏎1 2 3⏎` → `-1`; `1 5⏎5⏎` → `0`

Exercise 3 · Count occurrences

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Count how many times target occurs in a sorted array in O(log n): upper bound − lower bound.
 static int lower(int[] a, int t) {
 int lo = 0, hi = a.length;
 while (lo < hi) {
 int mid = lo + (hi − lo) / 2;
 if (a[mid] < t) lo = mid + 1;
 else hi = mid;
 }
 return lo;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int target = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 System.out.println(lower(a, target + 1) − lower(a, target));
 }
}
```

Why it works: Two lower-bound searches bracket the run of equal values.

Verified against 3 test cases: `8 2⏎1 2 2 2 2 3 5 9⏎` → `4`; `3 7⏎1 2 3⏎` → `0`; `4 1⏎1 1 1 1⏎` → `4`

4.4 Elementary Sorting: Bubble, Insertion, Selection; Stability

Exercise 1 · Bubble sort with swap count

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Bubble sort; print the sorted array and the number of swaps (= number of inversions).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int swaps = 0;
 for (int pass = 0; pass < n − 1; pass++) {
 boolean any = false;
 for (int i = 0; i < n − 1 − pass; i++) {
 if (a[i] > a[i + 1]) {
 int t = a[i]; a[i] = a[i + 1]; a[i + 1] = t;
 swaps++;
 any = true;
 }
 }
 if (!any) break;
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 System.out.println(swaps);
 }
}
```

Why it works: Each swap removes exactly one inversion.

Verified against 3 test cases: `5⏎4 3 2 1 5⏎` → `1 2 3 4 5⏎`6; `3⏎1 2 3⏎` → `1 2 3⏎`0; `2⏎2 1⏎` → `1 2⏎1`

Exercise 2 · Insertion steps

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Insertion sort: print the array after every insertion step i = 1..n-1.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 for (int i = 1; i < n; i++) {
 int key = a[i];
 int j = i - 1;
 while (j >= 0 && a[j] > key) {
 a[j + 1] = a[j];
 j--;
 }
 a[j + 1] = key;
 StringBuilder sb = new StringBuilder();
 for (int k = 0; k < n; k++) sb.append(k == 0 ? "" : " ").append(a[k]);
 System.out.println(sb);
 }
 }
}
```

Why it works: The sorted prefix grows by one element per step.

Verified against 2 test cases: `4⏎5 2 4 1⏎` → `2 5 4 1⏎`2 4 5 1⏎1 2 4 5; `2⏎1 2⏎` → `1 2`

Exercise 3 · Stable by key

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Is the sort stable? Input: n pairs "key label". Sort by key with insertion sort and print the labels.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] key = new int[n];
 String[] label = new String[n];
 for (int i = 0; i < n; i++) {
 key[i] = sc.nextInt();
 label[i] = sc.next();
 }
 for (int i = 1; i < n; i++) {
 int k = key[i];
 String l = label[i];
 int j = i - 1;
 while (j >= 0 && key[j] > k) {
 key[j + 1] = key[j];
 label[j + 1] = label[j];
 j--;
 }
 key[j + 1] = k;
 label[j + 1] = l;
 }
 System.out.println(String.join(" ", label));
 }
}
```

Why it works: Equal keys never pass each other, so labels with equal keys keep input order.

Verified against 2 test cases: `5⏎2 a⏎1 b⏎2 c⏎1 d⏎2 e⏎` → `b d a c e`; `2⏎1 x⏎1 y⏎` → `x y`

4.5 Merge Sort and Divide and Conquer

Exercise 1 · Merge sort

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static void sort(int[] a, int lo, int hi, int[] t) {
 if (hi - lo < 2) return;
 int mid = (lo + hi) >>> 1;
 sort(a, lo, mid, t);
 sort(a, mid, hi, t);
 int i = lo, j = mid, k = lo;
 while (i < mid && j < hi) t[k++] = a[i] <= a[j] ? a[i++] : a[j++];
 while (i < mid) t[k++] = a[i++];
 while (j < hi) t[k++] = a[j++];
 System.arraycopy(t, lo, a, lo, hi - lo);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 sort(a, 0, n, new int[n]);
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

Why it works: Standard top-down merge sort.

Verified against 3 test cases: `7⏎38 27 43 3 9 82 10⏎` → `3 9 10 27 38 43 82`; `1⏎5⏎` → `5`; `4⏎2 2 1 1⏎` → `1 1 2 2`

Exercise 2 · Merge two sorted arrays

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Merge two sorted arrays into one sorted array.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int m = sc.nextInt();
 int[] a = new int[n], b = new int[m];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 for (int i = 0; i < m; i++) b[i] = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 int i = 0, j = 0;
 while (i < n || j < m) {
 int v = (j >= m || (i < n && a[i] <= b[j])) ? a[i++] : b[j++];
 sb.append(sb.length() == 0 ? "" : " ").append(v);
 }
 System.out.println(sb);
 }
}
```

Why it works: The merge step on its own: O(n + m).

Verified against 2 test cases: `3 4⏎1 4 9⏎2 3 10 11⏎` → `1 2 3 4 9 10 11`; `1 1⏎5⏎5⏎` → `5 5`

Exercise 3 · Count inversions

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static long count(int[] a, int lo, int hi, int[] t) {
 if (hi - lo < 2) return 0;
 int mid = (lo + hi) >>> 1;
 long inv = count(a, lo, mid, t) + count(a, mid, hi, t);
 int i = lo, j = mid, k = lo;
 while (i < mid && j < hi) {
 if (a[i] <= a[j]) t[k++] = a[i++];
 else {
 inv += mid - i;
 t[k++] = a[j++];
 }
 }
 while (i < mid) t[k++] = a[i++];
 while (j < hi) t[k++] = a[j++];
 System.arraycopy(t, lo, a, lo, hi - lo);
 return inv;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 System.out.println(count(a, 0, n, new int[n]));
 }
}
```

Why it works: Each right-half element that jumps ahead passes all remaining left-half elements.

Verified against 3 test cases: `5⏎2 4 1 3 5⏎` → `3`; `3⏎3 2 1⏎` → `3`; `4⏎1 2 3 4⏎` → `0`

4.6 Iteration versus Recursion

Exercise 1 · gcd both ways

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // gcd both ways; print iterative then recursive result
 static int gcdLoop(int a, int b) {
 while (b != 0) {
 int t = a % b;
 a = b;
 b = t;
 }
 return a;
 }

 static int gcdRec(int a, int b) {
 return b == 0 ? a : gcdRec(b, a % b);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt(), b = sc.nextInt();
 System.out.println(gcdLoop(a, b) + " " + gcdRec(a, b));
 }
}
```

Why it works: while (b != 0) vs return gcd(b, a % b).

Verified against 2 test cases: `84 36⏎` → `12 12`; `7 13⏎` → `1 1`

Exercise 2 · Climbing stairs

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Climbing stairs: ways to climb n steps taking 1 or 2 at a time (Fibonacci in disguise),
     iteratively.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long a = 1, b = 1;
 for (int i = 2; i <= n; i++) {
 long next = a + b;
 a = b;
 b = next;
 }
 System.out.println(b);
 }
}
```

Why it works: Bottom-up with two variables.

Verified against 3 test cases: `10⏎` → `89`; `1⏎` → `1`; `2⏎` → `2`

Exercise 3 · Reverse a string

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Binary-search count of steps both ways would be the same; here: convert the recursive
 // "reverse a string" into a loop and print both results.
 static String revRec(String s) {
 return s.isEmpty() ? "" : revRec(s.substring(1)) + s.charAt(0);
 }

 static String revLoop(String s) {
 StringBuilder sb = new StringBuilder();
 for (int i = s.length() - 1; i >= 0; i--) sb.append(s.charAt(i));
 return sb.toString();
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 System.out.println(revRec(s) + " " + revLoop(s));
 }
}
```

Why it works: Same result; the recursion creates a new string at each level.

Verified against 2 test cases: `stressed⏎` → `desserts desserts`; `a⏎` → `a a`

4.7 Time Complexity and Big-O Notation

Exercise 1 · Count the triangle

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Count how many times the inner statement runs for a double loop i in 1..n, j in 1..i.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long count = 0;
 for (int i = 1; i <= n; i++)
 for (int j = 1; j <= i; j++) count++;
 System.out.println(count);
 }
}
```

Why it works: Triangular numbers: $O(n^2)$.

Verified against 3 test cases: `5⏎` → `15`; `1⏎` → `1`; `100⏎` → `5050`

Exercise 2 · Passes of a halving loop

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Count the passes of: while (n > 1) n /= 2; (the floor of log2 n)
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 int passes = 0;
 while (n > 1) {
 n /= 2;
 passes++;
 }
 System.out.println(passes);
 }
}
```

Why it works: Each pass halves n; $\lfloor \log_2 1000 \rfloor = 9$.

Verified against 3 test cases: `1000⏎` → `9`; `1⏎` → `0`; `1024⏎` → `10`

Exercise 3 · The crossover point

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Smallest n for which n^2 exceeds 10 * n * log2(n) ... i.e. when does a quadratic algorithm
    become
 // slower than an n log n one that does 10x more work per step? Print that n (n >= 2).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int factor = sc.nextInt();
 int n = 2;
 while (!((double) n * n > factor * n * (Math.log(n) / Math.log(2)))) n++;
 System.out.println(n);
 }
}
```

Why it works: $n^2 > \text{factor} \cdot n \cdot \log_2 n \Leftrightarrow n > \text{factor} \cdot \log_2 n$; for factor 10 this first holds at n = 59.

Verified against 2 test cases: `10⏎` → `59`; `2⏎` → `5`

4.8 Algorithm Design Paradigms

Exercise 1 · Activity selection (greedy)

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Greedy activity selection: maximum number of non-overlapping intervals (sort by END time).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[][] iv = new int[n][2];
 for (int i = 0; i < n; i++) {
 iv[i][0] = sc.nextInt();
 iv[i][1] = sc.nextInt();
 }
 java.util.Arrays.sort(iv, (a, b) -> Integer.compare(a[1], b[1]));
 int count = 0, lastEnd = Integer.MIN_VALUE;
 for (int[] x : iv) {
 if (x[0] >= lastEnd) {
 count++;
 lastEnd = x[1];
 }
 }
 System.out.println(count);
 }
}
```

Why it works: The earliest finish leaves the most room — a provable greedy choice.

Verified against 3 test cases: `5⏎1 3⏎2 4⏎3 5⏎0 6⏎5 7⏎` → `3`; `1⏎0 1⏎` → `1`; `3⏎1 5⏎2 3⏎3 4⏎` → `2`

Exercise 2 · Fewest coins (DP)

Ex2.java

```java
import java.util.Arrays;
import java.util.Scanner;

public class Ex2 {
 // Fewest coins to make the amount (any coin system), or -1.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int k = sc.nextInt();
 int[] coins = new int[k];
 for (int i = 0; i < k; i++) coins[i] = sc.nextInt();
 int amount = sc.nextInt();
 int INF = Integer.MAX_VALUE;
 int[] best = new int[amount + 1];
 Arrays.fill(best, INF);
 best[0] = 0;
 for (int x = 1; x <= amount; x++)
 for (int c : coins)
 if (c <= x && best[x - c] != INF) best[x] = Math.min(best[x], best[x - c] + 1);
 System.out.println(best[amount] == INF ? -1 : best[amount]);
 }
}
```

Why it works: Bottom-up over amounts 1..amount.

Verified against 3 test cases: `3⏎4 3 1⏎6⏎` → `2`; `1⏎2⏎3⏎` → `-1`; `3⏎25 10 1⏎30⏎` → `3`

Exercise 3 · No two adjacent ones (backtracking)

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Backtracking: print all binary strings of length n with no two consecutive 1s, in order.
 static StringBuilder out = new StringBuilder();

 static void gen(int n, String s, boolean lastOne) {
 if (s.length() == n) {
 out.append(s).append('\n');
 return;
 }
 gen(n, s + "0", false);
 if (!lastOne) gen(n, s + "1", true);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 gen(sc.nextInt(), "", false);
 System.out.print(out);
 }
}
```

Why it works: The constraint is checked as the string grows, pruning invalid branches immediately.

Verified against 3 test cases: `3⏎` → `000⏎001⏎010⏎100⏎101`; `1⏎` → `0⏎1`; `2⏎` → `00⏎01⏎10`

4.9 Analysing Loops: Counting Iterations

Exercise 1 · Tripling outer loop

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // How many times does "count++" execute? for (i = 1; i <= n; i *= 3) for (j = 0; j < n; j++)
     count++;
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long count = 0;
 for (long i = 1; i <= n; i *= 3)
 for (int j = 0; j < n; j++) count++;
 System.out.println(count);
 }
}
```

Why it works: 4 outer iterations (1, 3, 9, 27) × 30 = 120: O(n log n).

Verified against 3 test cases: `30⏎` → `120`; `1⏎` → `1`; `100⏎` → `500`

Exercise 2 · Half-linear times log

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // for (i = n/2; i >= 1; i--) for (j = 1; j <= n; j *= 2) count++;
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long count = 0;
 for (int i = n / 2; i >= 1; i--)
 for (long j = 1; j <= n; j *= 2) count++;
 System.out.println(count);
 }
}
```

Why it works: 8 × 5 = 40: O(n log n).

Verified against 3 test cases: `16⏎` → `40`; `2⏎` → `2`; `100⏎` → `350`

Exercise 3 · Measure the growth ratio

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Measure the growth: print count(2n) / count(n) rounded to 1 decimal for a double loop where j
     runs i times.
 static long count(int n) {
 long c = 0;
 for (int i = 1; i <= n; i++)
 for (int j = 1; j <= i; j++) c++;
 return c;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 System.out.printf("%.1f%n", (double) count(2 * n) / count(n));
 }
}
```

Why it works: Doubling n quadruples n(n+1)/2 (almost exactly), the signature of $O(n^2)$.

Verified against 2 test cases: `500⏎` → `4.0`; `1000⏎` → `4.0`

4.10 Analysing Recursion: Recurrence Relations

Exercise 1 · Count the calls

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static long calls;

 static int f(int n) {
 calls++;
 if (n <= 1) return 1;
 return f(n - 1) + f(n - 1); // two calls, not one!
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 f(sc.nextInt());
 System.out.println(calls);
 }
}
```

Why it works: A complete binary tree of depth n−1.

Verified against 3 test cases: `10⏎` → `1023`; `1⏎` → `1`; `5⏎` → `31`

Exercise 2 · Two half calls

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static long calls;

 static int g(int n) {
 calls++;
 if (n <= 1) return 0;
 return g(n / 2) + g(n / 2); // two calls on HALF the size: T(n) = 2T(n/2) + 1
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 g(sc.nextInt());
 System.out.println(calls);
 }
}
```

Why it works: T(n) = 2T(n/2) + 1 = O(n): a full binary tree with n leaves.

Verified against 3 test cases: `16⏎` → `31`; `1⏎` → `1`; `8⏎` → `15`

Exercise 3 · Karatsuba work

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static long work;

 // T(n) = 3T(n/2) + n (Karatsuba shape): count the "n" work terms for n a power of two.
 static void t(int n) {
 if (n <= 1) {
 work += 1;
 return;
 }
 work += n;
 t(n / 2);
 t(n / 2);
 t(n / 2);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 t(sc.nextInt());
 System.out.println(work);
 }
}
```

Why it works: Σ n(3/2)^k + 3^(log n) = Θ(n^1.585).

Verified against 3 test cases: `8⏎` → `65`; `2⏎` → `5`; `1⏎` → `1`

4.11 Space Complexity and the Call Stack

Exercise 1 · Maximum recursion depth

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static int depth, maxDepth;

 static long f(int n) {
 depth++;
 maxDepth = Math.max(maxDepth, depth);
 long r = n <= 1 ? 1 : f(n - 1) + 1;
 depth--;
 return r;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 f(sc.nextInt());
 System.out.println(maxDepth);
 }
}
```

Why it works: A chain of n calls is all active at the base case.

Verified against 3 test cases: `50⏎` → `50`; `1⏎` → `1`; `200⏎` → `200`

Exercise 2 · Fibonacci in O(1) space

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Space-efficient: nth Fibonacci with O(1) space (two variables) instead of an array of n values.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long a = 0, b = 1;
 for (int i = 0; i < n; i++) {
 long c = a + b;
 a = b;
 b = c;
 }
 System.out.println(a);
 }
}
```

Why it works: The loop only needs F(i−1) and F(i).

Verified against 4 test cases: `50⏎` → `12586269025`; `0⏎` → `0`; `1⏎` → `1`

Exercise 3 · Duplicates with O(1) extra space

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Does the array contain a duplicate? O(n log n) time but only O(1) extra space apart from sorting in place.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 java.util.Arrays.sort(a);
 boolean dup = false;
 for (int i = 1; i < n; i++) if (a[i] == a[i - 1]) dup = true;
 System.out.println(dup ? "YES" : "NO");
 }
}
```

Why it works: Trade time (O(n log n) sort) for space (no extra structure).

Verified against 3 test cases: `5⏎4 1 7 1 9⏎` → `YES`; `3⏎1 2 3⏎` → `NO`; `1⏎5⏎` → `NO`

4.12 Best, Average and Worst Case

Exercise 1 · Comparisons in a linear search

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Comparisons a linear search makes for the target (n if absent).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int target = sc.nextInt();
 int comparisons = 0;
 boolean found = false;
 for (int i = 0; i < n; i++) {
 int x = sc.nextInt();
 if (!found) {
 comparisons++;
 if (x == target) found = true;
 }
 }
 System.out.println(comparisons);
 }
}
```

Why it works: The count equals index + 1 for a hit and n for a miss.

Verified against 3 test cases: `5 7⏎4 9 7 1 7⏎` → `3`; `3 9⏎1 2 3⏎` → `3`; `2 1⏎1 2⏎` → `1`

Exercise 2 · Average successful search

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Average comparisons of a successful linear search over n elements when every position is
     equally likely.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 System.out.printf("%.1f%n", (n + 1) / 2.0);
 }
}
```

Why it works: Average of 1..n.

Verified against 3 test cases: `9⏎` → `5.0`; `1⏎` → `1.0`; `10⏎` → `5.5`

Exercise 3 · Insertion sort comparisons

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Insertion sort comparisons for the given array (the lesson's counting rule).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 long c = 0;
 for (int i = 1; i < n; i++) {
 int key = a[i], j = i - 1;
 while (j >= 0) {
 c++;
 if (a[j] <= key) break;
 a[j + 1] = a[j];
 j--;
 }
 a[j + 1] = key;
 }
 System.out.println(c);
 }
}
```

Why it works: Each element is compared with every larger predecessor plus one stopping comparison (when it exists).

Verified against 3 test cases: `5⏎5 4 3 2 1⏎` → `10`; `5⏎1 2 3 4 5⏎` → `4`; `1⏎1⏎` → `0`

Part 4 — Quiz answer key

4.1 Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables

	Answer	Why
Q1	array index lookup → O(1); linked list lookup by position → O(n); hash table lookup (average) → O(1); sorted tree lookup → O(log n)	Arrays jump by arithmetic; lists walk; hashing jumps by hash; trees halve.
Q2	B. stack	The most recent action is undone first: LIFO.
Q3	A. O(1)	Re-wire two references.
Q4	B. queue	FIFO explores by levels.

4.2 Trees, Graphs and Heaps

	Answer	Why
Q1	A. sorted keys	left, node, right.
Q2	B. queue	FIFO queue.
Q3	B. 7 and 8	$2 \cdot 3+1 = 7$ and $2 \cdot 3+2 = 8$.
Q4	C. $O(V^2)$	One cell per vertex pair.

4.3 Searching: Linear and Binary Search

	Answer	Why
Q1	B. 20	$\lceil \log_2 10^6 \rceil = 20$.
Q2	B. sorted	Sorted order lets us discard a half.
Q3	first mid = 3; a[first mid] = 7	mid = 3 and a[3] = 7: found at the first probe.
Q4	B. O(log n)	The range halves each step.

4.4 Elementary Sorting: Bubble, Insertion, Selection; Stability

	Answer	Why
Q1	C. selection	Selection sort swaps across equal elements.
Q2	A. O(n)	One comparison per element.
Q3	3	3 inversions → 3 swaps.
Q4	B. O(1) extra memory	Rearranges within the input.

4.5 Merge Sort and Divide and Conquer

	Answer	Why
Q1	B. O(n log n)	Always n log n.
Q2	C. O(n)	The temporary array.
Q3	B. divide, conquer, combine	Divide, conquer (recurse), combine.
Q4	2	$3 > 1$ and $3 > 2$; $1 < 2$.

4.6 Iteration versus Recursion

	Answer	Why
Q1	B. loop	Recursion needs a stack frame per level.
Q2	B. $O(2^n)$ to $O(n)$	Each value is computed once.
Q3	B. all subsets of a set	Branching choices form a tree.
Q4	B. StackOverflowError	The call stack is exhausted.

4.7 Time Complexity and Big-O Notation

	Answer	Why
Q1	binary search → $O(\log n)$; find max in unsorted array → $O(n)$; merge sort → $O(n \log n)$; bubble sort → $O(n^2)$; all subsets → $O(2^n)$; all permutations → $O(n!)$	The lecture's Big-O cheat sheet.
Q2	B. $O(n^2)$	The n^2 term dominates (n^3 is true but loose).
Q3	C. Big-Θ	Θ is both an upper and a lower bound.
Q4	B. 2^n	Exponential beats every polynomial.
Q5	A. $O(1)$	Constant: address arithmetic.

4.8 Algorithm Design Paradigms

	Answer	Why
Q1	N-Queens → backtracking; merge sort → divide and conquer; shortest string of given weight → greedy; grid paths count → dynamic programming	See the table in the lesson.
Q2	2	25 + 5.
Q3	B. the best local choice leads to a global optimum	Provability is the key.
Q4	B. dynamic programming	Store and reuse answers.

4.9 Analysing Loops: Counting Iterations

	Answer	Why
Q1	B. $\log_2 n$ times	Geometric growth.
Q2	C. $O(n^2)$	$n(n+1)/2$.
Q3	B. $O(n \log n)$	Harmonic series: $n \cdot H(n)$.
Q4	B. $O(n\sqrt{n})$	n outer × $\sqrt{n}$ inner.
Q5	B. $\log_{10} n$ times (the number of digits)	One iteration per digit.

4.10 Analysing Recursion: Recurrence Relations

	Answer	Why
Q1	$T(n) = T(n-1) + 1 \rightarrow O(n)$; $T(n) = T(n/2) + 1 \rightarrow O(\log n)$; $T(n) = 2T(n/2) + n \rightarrow O(n \log n)$; $T(n) = 2T(n-1) + 1 \rightarrow O(2^n)$	See the table.

	Answer	Why
Q2	C. O(n^1.585)	Leaves dominate: n^log₂3.
Q3	B. log n	Halving.
Q4	C. 1.618ⁿ calls	Exponential in the golden ratio.

4.11 Space Complexity and the Call Stack

	Answer	Why
Q1	int add(int n) recursive sum → O(n); loop with sum += pairSum(i, i+1) → O(1); new int[n] filled in a loop → O(n); int sum; for (i<n) sum += i → O(1); recursive fibonacci(n) (stack) → O(n); linked list with n inserts → O(n)	The six "predict the space complexity" slides.
Q2	B. the extra space used besides the input	Temporary memory.
Q3	B. O(log n)	Depth $\log_2 n$.
Q4	B. 4 MB	$10^6 \times 4$ bytes.

4.12 Best, Average and Worst Case

	Answer	Why
Q1	B. reverse sorted	Every element moves to the front.
Q2	B. already sorted data	Unbalanced partitions.
Q3	C. worst	The worst case bounds all inputs.
Q4	B. all O(n log n)	It always splits evenly.

Part 5 · Archive Algorithm Mastery

One full lesson for each algorithm and programming problem in your course archive — numbers and bits, arrays and matrices, strings, recursion and backtracking, and sorting — with naive and optimized solutions, dry runs, practice and quizzes.

	Lesson	Level	Min	Core
5.1	Euclid's Algorithm (GCD)	beginner	40	★
5.2	Euler's Phi (Totient) Function	intermediate	45	★
5.3	Chinese Remainder Theorem	intermediate	45	
5.4	Combinations (nCr)	beginner	40	
5.5	Karatsuba's Multiplication	advanced	55	
5.6	Booth's Algorithm (signed binary multiplication)	advanced	60	
5.7	Simple Sieve of Eratosthenes	beginner	40	★
5.8	Segmented & Incremental Sieve	advanced	55	
5.9	Binary Palindrome	beginner	35	★
5.10	Swap Two Nibbles in a Byte	beginner	30	
5.11	Longest Sequence of 1s (and after flipping a bit)	intermediate	45	
5.12	Josephus Problem	intermediate	50	★
5.13	Block Swap Algorithm (Array Rotation)	intermediate	50	★
5.14	Leaders in an Array	beginner	30	★
5.15	Majority Element (Boyer–Moore Voting)	intermediate	50	★
5.16	Maximum Product Subarray	intermediate	50	★
5.17	Maximum Equilibrium Sum	beginner	35	★
5.18	Maximum Sum of an Hourglass in a Matrix	beginner	30	★
5.19	Toggle the Switch (100 Doors)	intermediate	40	★
5.20	Alice's Apple Tree (Minimum Garden Perimeter)	advanced	45	
5.21	Weighted Substring (Smallest String of a Given Weight)	advanced	50	
5.22	Move Hyphen to the Beginning	beginner	30	★
5.23	Strobogrammatic Number	intermediate	40	
5.24	Lexicographically First Palindromic Rearrangement	intermediate	40	★
5.25	Manacher's Algorithm (Longest Palindromic Substring)	advanced	65	
5.26	Natural Sort Order	intermediate	40	
5.27	Maneuvering (Counting Grid Paths)	intermediate	45	★
5.28	Sorted Distinct Permutations	intermediate	50	

	Lesson	Level	Min	Core
5.29	N-Queens (Backtracking)	advanced	60	★
5.30	Maze Solving (Rat in a Maze)	advanced	55	★
5.31	Hamiltonian Cycle	advanced	55	
5.32	Knight's Tour & Warnsdorff's Rule	advanced	55	
5.33	Selection Sort	beginner	35	★
5.34	Quick Sort	intermediate	55	★

5.1 Euclid's Algorithm (GCD)

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 40 min · ★ on the minimum-viable path

Find the greatest common divisor of two numbers in a handful of steps by repeatedly taking remainders.

YOU WILL BE ABLE TO

- Explain what a GCD is and compute small ones by listing divisors.
- Explain WHY gcd(a, b) = gcd(b, a mod b) — not just that it works.
- Write Euclid's algorithm in Java both as a loop and recursively, and trace it by hand.
- State its running time, O(log min(a, b)), and why it is so much faster than trying every divisor.

Before this lesson: Loops: for, while, do-while, break and continue · Methods: Parameters, Return Values, Scope and Overloading · Divisibility, GCD/LCM, Primes and Modular Arithmetic · Recursion: Base Case, Recursive Case and the Call Stack

The idea

The **greatest common divisor** (GCD) of two numbers is the largest number that divides both of them with no remainder. gcd(12, 18) = 6 because 6 divides 12 and 18, and nothing bigger does.

Picture a floor that is 50 tiles long and 12 tiles wide. You want the **largest square tile** that covers the whole floor with no cutting. Euclid's trick: cut off as many 12×12 squares as you can. Four fit (4 × 12 = 48), leaving a thin strip 12 long and **2** wide. Any tile that works for the big floor must also work for the leftover strip, so now solve the strip: cut 2×2 squares from 12×2 — six fit and nothing is left over. The tile is 2×2, so gcd(50, 12) = 2.

Each line replaces the pair (a, b) with the smaller pair (b, a mod b).

```
50 = 4 × 12 + 2 → gcd(50, 12) = gcd(12, 2)
12 = 6 × 2 + 0 → gcd(12, 2) = gcd( 2, 0) = 2
```

WHY THIS WORKS

Let d be any number that divides both a and b. Write a = q·b + r. Then r = a − q·b, which is also divisible by d. So d divides b and r too. The same argument works backwards: anything dividing b and r also divides a = q·b + r. Hence **a,b and b,r have exactly the same common divisors**, so they have the same greatest one. That single idea is the entire algorithm; the program is just that idea repeated until the remainder is 0.

Syntax

You need three Java ideas, all from earlier lessons:

- `%` is the remainder operator: `50 % 12` is `2`.
- A `while (b != 0)` loop repeats until the condition becomes false.
- A method can call itself (recursion) as long as it has a **base case** that stops it.

A tiny example

The smallest useful version just does what the pen-and-paper table does:

```
int a = 50, b = 12;
while (b != 0) {
 int r = a % b; // remainder of this step
 a = b; // the old divisor becomes the new dividend
 b = r; // the remainder becomes the new divisor
}
System.out.println(a); // 2
```

Guided example

The lecture slides write the work as a **Q | A | B | R** table (quotient, dividend, divisor, remainder). Each new row takes A from the previous B and B from the previous R. For gcd(33, 12):

The answer is the A on the last row, where B has become 0: gcd(33, 12) = 3.

Q	A	B	R
2	33	12	9
1	12	9	3
3	9	3	0
—	3	0	—

The program `EuclidGcd` prints exactly this table so you can compare it with your hand calculation. `NaiveGcd` shows the slow alternative — try every candidate from `min(a, b)` downwards — so you can feel the difference.

NaiveGcd.java — Naive GCD (try every divisor) (simple solution)

```
1  import java.util.Scanner;
2
3  public class NaiveGcd {
4   // Try every number from the smaller input down to 1.
5   // The first one that divides BOTH numbers is the greatest common divisor.
6   static int gcd(int a, int b) {
7   int smaller = Math.min(a, b);                                          (1)
8   for (int d = smaller; d >= 1; d--) {                                   (2)
9   if (a % d == 0 && b % d == 0) {                                        (3)
10  return d;
11  }
12  }
13  return 1; // only reached if a or b is 0 or negative; see the lesson's edge-case notes   (4)
14  }
15
16  public static void main(String[] args) {
17  Scanner sc = new Scanner(System.in);
18  int a = sc.nextInt();
19  int b = sc.nextInt();
20  System.out.println("gcd(" + a + ", " + b + ") = " + gcd(a, b));
21  }
22 }
```

Counts down from min(a, b) and returns the first number that divides both. Easy to believe, but slow for big inputs.

Sample input

```
50 12
```

Output when you run it

```
gcd(50, 12) = 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	A common divisor can never be larger than the smaller number, so we start there.
2	line 8	Counting DOWN means the first hit is automatically the greatest.
3	line 9	d is a common divisor only if it divides both with remainder 0.
4	line 13	Reached only for inputs the lesson excludes (0 or negatives). Every positive pair shares divisor 1, so the loop always returns earlier.

EuclidGcd.java — Euclid's algorithm (iterative) (optimised solution)

```
1  import java.util.Scanner;
2
3  public class EuclidGcd {
4   // Euclid's algorithm: gcd(a, b) = gcd(b, a mod b), and gcd(a, 0) = a.
5   static long gcd(long a, long b) {
6   a = Math.abs(a);                                                    (1)
7   b = Math.abs(b);
8   while (b != 0) {                                                    (2)
9   long remainder = a % b;                                             (3)
10  a = b;                                                              (4)
11  b = remainder;                                                      (5)
12  }
13  return a;
14  }
15
16  public static void main(String[] args) {
17  Scanner sc = new Scanner(System.in);
18  long a = sc.nextLong();
19  long b = sc.nextLong();
20
21  // Print the same Q | A | B | R table used in the lecture slides.
22  System.out.println(" Q A B R");
23  long x = Math.abs(a);
24  long y = Math.abs(b);
25  while (y != 0) {
26  long q = x / y;                                                     (6)
27  long r = x % y;
28  System.out.printf("%4d %10d %10d %10d%n", q, x, y, r);
29  x = y;
30  y = r;
31  }
32  System.out.printf("%4s %10d %10d %10s%n", "-", x, y, "-");
33  System.out.println("gcd(" + a + ", " + b + ") = " + gcd(a, b));
34  }
35 }
```

Replaces (a, b) by (b, a mod b) until b is 0. Uses long and Math.abs so large and negative inputs behave.

Sample input

```
33 12
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	gcd ignores signs: gcd(-12, 18) = gcd(12, 18) = 6.
2	line 8	Stop when the divisor reaches 0; at that moment a holds the answer.
3	line 9	Compute the remainder BEFORE overwriting a — the order of the next three lines matters.
4	line 10	Slide the window: old divisor becomes the new dividend.
5	line 11	The remainder becomes the new divisor (it is strictly smaller than b, so progress is guaranteed).
6	line 26	The table printing loop repeats the algorithm just to display Q, A, B, R. The real work is the gcd method above.

EuclidRecursive.java — Euclid's algorithm (recursive) (demonstration)

```
1  public class EuclidRecursive {
2   // The recursive form is a direct copy of the rule gcd(a, b) = gcd(b, a mod b).
3   static int gcd(int a, int b) {
4   System.out.println("gcd(" + a + ", " + b + ")");
5   if (b == 0) {                                                    (1)
6   return a; // base case: nothing left to divide by
7   }
8   return gcd(b, a % b); // smaller problem: same answer, smaller numbers   (2)
9   }
10
11  public static void main(String[] args) {
12  System.out.println("Answer: " + gcd(83, 19));
13  }
14 }
```

The same rule written as a self-calling method, with a print so you can watch the call stack unwind.

Line-by-line walkthrough

#	Where	What happens
1	line 5	Base case — without it the method would recurse forever (and then divide by zero).
2	line 8	The recursive step is the rule gcd(a, b) = gcd(b, a mod b), copied literally.

Dry run: watch it execute

Slide example 2: gcd(83, 19) — the numbers shrink until the remainder reaches 0.

Algorithm: Euclid's GCD. **Input:** a = `83`, b = `19`.

Pseudocode

```
while (b != 0) {
 r = a % b;
 a = b;
 b = r;
}
return a;
```

Step 1 of 12. Start with a = 83, b = 19.

a = 83 · b = 19 · r = —

Step 2 of 12. 83 mod 19 = 7 (quotient 4)

a = 83 · b = 19 · r = 7

Step 3 of 12. Slide: a becomes 19, b becomes 7.

a = 19 · b = 7 · r = 7

Step 4 of 12. 19 mod 7 = 5 (quotient 2)

a = 19 · b = 7 · r = 5

Step 5 of 12. Slide: a becomes 7, b becomes 5.

a = 7 · b = 5 · r = 5

Step 6 of 12. 7 mod 5 = 2 (quotient 1)

a = 7 · b = 5 · r = 2

Step 7 of 12. Slide: a becomes 5, b becomes 2.

a = 5 · b = 2 · r = 2

Step 8 of 12. 5 mod 2 = 1 (quotient 2)

a = 5 · b = 2 · r = 1

Step 9 of 12. Slide: a becomes 2, b becomes 1.

a = 2 · b = 1 · r = 1

Step 10 of 12. 2 mod 1 = 0 (quotient 2)

a = 2 · b = 1 · r = 0

Step 11 of 12. Slide: a becomes 1, b becomes 0.

a = 1 · b = 0 · r = 0

Step 12 of 12. b is 0, so the answer is a = 1.

a = 1 · b = 0 · r = 0

> **RESULT**
> gcd = 1

Complexity

Time

Naive: the loop may run `min(a, b)` times → O(min(a, b)). For gcd(1 000 000 007, 1 000 000 000) that is up to a billion iterations.

Euclid: look at two consecutive steps. Claim: a mod b < a/2 always.

- If b ≤ a/2, then a mod b < b ≤ a/2.
- If b > a/2, then a mod b = a − b < a/2.

So every step the value in the "A" position is more than halved after two steps. A number can only be halved about $\log_2(n)$ times before it hits 0, so the loop runs at most about $2 \cdot \log_2(\min(a, b))$ times.

```
time = O(log min(a, b)) space = O(1) iterative, O(log min(a, b))
                        recursive (call stack)
```

The worst case for Euclid is two consecutive Fibonacci numbers (Lamé's theorem), and even that is only logarithmic.

Input	Naive iterations (worst)	Euclid steps
gcd(50, 12)	12	2
gcd(83, 19)	19	5
gcd(1 000 000 007, 1 000 000 000)	≈ 1 000 000 000	≈ 6
gcd(Fibonacci F_{40}, F_{39})	≈ 63 000 000	38 (the worst case)

Naive vs optimized

Both are correct. Euclid wins by a factor of up to **millions** on large inputs because every step throws away a big chunk of the problem instead of one candidate.

Common mistakes

Overwriting a before using it

Problem

```
while (b != 0) {
 a = b;
 b = a % b; // a was already replaced!
}
```

Why: After `a = b`, the expression `a % b` is `b % b` which is always 0, so the loop stops with the wrong answer.

Fix: while (b != 0) { int r = a % b; a = b; b = r; }

Forgetting the base case

Problem

```
static int gcd(int a, int b) {
 return gcd(b, a % b);
}
```

Why: When b becomes 0, `a % 0` throws ArithmeticException ("/ by zero"). Recursion needs a stopping rule.

Fix: static int gcd(int a, int b) { if (b == 0) return a; return gcd(b, a % b); }

LCM computed as a b / gcd*

Problem

```
int lcm = a * b / gcd(a, b); // a * b can overflow int
```

Why: For a = b = 100 000 the product 10 000 000 000 does not fit in an int, so the result is garbage.

Fix: long lcm = (long) a / gcd(a, b) * b; // divide first, then multiply

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Reduce a fraction — *Beginner*

Read a numerator and a denominator (both positive). Print the fraction in lowest terms as `numerator/denominator`.

- 1 ≤ numerator, denominator ≤ 10^9

Example input	Expected output
12 18	2/3

Hint 1: What number can you divide BOTH parts by without leaving a remainder? **Hint 2:** Use Euclid's algorithm for that number, then divide.

Exercise 2 · LCM of a list — *Intermediate*

Read n and then n positive integers. Print their least common multiple. The answer fits in a `long`.

- 1 ≤ n ≤ 20
- 1 ≤ each number ≤ 10^6
- answer ≤ $9 \cdot 10^{18}$

Example input	Expected output
3 ⏎ 4 6 8	24

Hint 1: lcm(a, b) = a / gcd(a, b) * b. **Hint 2:** Fold the list: lcm = lcm(lcm, next). **Hint 3:** Use long and divide before multiplying.

Exercise 3 · Modular inverse — *Challenge*

Given a and m, print the number x in [0, m−1] such that (a · x) mod m = 1, or -1 if no such x exists.

- 1 ≤ a, m ≤ 10^9
- m ≥ 2

Example input	Expected output
3 11	4
6 9	−1

Hint 1: An inverse exists exactly when gcd(a, m) = 1. **Hint 2:** Extended Euclid finds x and y with a·x + m·y = gcd(a, m). Then x is the inverse. **Hint 3:** The x it returns can be negative — shift it into [0, m−1] with ((x % m) + m) % m.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is gcd(48, 18)?

- **A.** 3
- **B.** 6
- **C.** 9
- **D.** 12

Q2. What does this program print?

```
int a = 84, b = 36;
while (b != 0) {
 int t = a % b;
 a = b;
 b = t;
}
System.out.println(a);
```

Your answer: _____

Q3. Which line makes this gcd method return the wrong answer?

```
1  static int gcd(int a, int b) {
2   while (b != 0) {
3   a = b;
4   b = a % b;
5   }
6   return a;
7  }
```

Which line contains the bug? _____

Q4. Complete the recursive base case and recursive call.

```
static int gcd(int a, int b) {
 if (b == ______) return ______;
 return gcd(______, a % b);
}
```

Q5. What is the running time of Euclid's algorithm for gcd(a, b)?

- **A.** O(min(a, b))
- **B.** O(log min(a, b))
- **C.** O(a + b)
- **D.** O(1)

Q6. Trace Euclid's algorithm for (a, b) = (50, 12).

```
1  while (b != 0) {
2   int r = a % b;
3   a = b;
4   b = r;
5  }
```

- a after iteration 1: _____
- b after iteration 1: _____
- b after iteration 2: _____
- final a: _____

Interview / exam questions

- Prove that gcd(a, b) = gcd(b, a mod b).
- What is the worst-case input for Euclid's algorithm and why?
- How do you compute the GCD of an array of n numbers? What is the total cost?
- How is the LCM related to the GCD, and how do you avoid overflow when computing it?
- What does the extended Euclidean algorithm compute and where is it used?

Watch & read

Video: GCD - Euclidean Algorithm (Method 1) — Neso Academy · 14:38. *Why gcd(a, b) = gcd(b, a mod b)*

Video: Euclidean Algorithm (GCD). Java — Aleksandr Azimut · 8:08. *Writing Euclid's algorithm in Java*

REVISION CARD (30–60 SECONDS)

- gcd(a, b) = gcd(b, a mod b); gcd(a, 0) = a.
- Loop: r = a % b; a = b; b = r; stop when b == 0, answer is a.
- Time O(log min(a, b)); naive is O(min(a, b)).
- lcm(a, b) = a / gcd(a, b) * b (divide first to avoid overflow).
- Worst case: consecutive Fibonacci numbers.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide pseudocode states the prerequisite "a > b". It is not needed: if a < b the first step swaps them (gcd(12, 50) → gcd(50, 12)).

5.2 Euler's Phi (Totient) Function

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 45 min · ★ on the minimum-viable path

Count the numbers up to n that share no factor with n — using prime factorisation instead of testing each one.

YOU WILL BE ABLE TO

- Define coprime numbers and Euler's φ(n) and compute φ for small n by hand.
- Derive φ(n) = n · ∏(1 − 1/p) from the idea "remove the multiples of each prime factor".
- Write the O(√n) Java method and explain every line, including the "leftover prime" step.
- Spot and explain the error in the lecture's worked example φ(60).

Before this lesson: Euclid's Algorithm (GCD) · Divisibility, GCD/LCM, Primes and Modular Arithmetic · Loops: for, while, do-while, break and continue

The idea

Two numbers are **coprime** when their only common divisor is 1 (gcd = 1). 8 and 15 are coprime; 8 and 12 are not (both divisible by 4). **Euler's totient** φ(n) counts how many numbers from 1 to n are coprime to n.

A number fails to be coprime to 12 exactly when it shares the factor 2 or the factor 3.

```
n = 12 → 1 2 3 4 5 6 7 8 9 10 11 12
coprime → ✓ · · · ✓ · ✓ · · · ✓ · φ(12) = 4 (1, 5, 7, 11)
```

So φ(n) is "n minus everything that shares a prime factor with n". Take n = 12 = 2² · 3:

- Half of the numbers are multiples of 2 → keep **1/2** of them (× (1 − 1/2)).
- A third are multiples of 3 → keep **2/3** of those (× (1 − 1/3)).

$$\varphi(12) = 12 \times (1 - 1/2) \times (1 - 1/3) = 12 \times 1/2 \times 2/3 = 4$$

WHY THIS WORKS

The formula φ(n) = n · ∏ (1 − 1/p) takes the product over the **distinct** primes p that divide n. Multiples of different primes are independent of each other (that is what the Chinese Remainder Theorem says), so the "survival fractions" multiply. A prime appearing 5 times in n counts only once — its multiples are the same numbers however many times it divides n.

Syntax

Java ideas used: `for` loops with a condition that involves multiplication (`p * p <= n`), the remainder operator `%`, and `n /= p` (shorthand for `n = n / p`). Use `long` so that `p * p` cannot overflow for large n.

A tiny example

The smallest correct program literally counts coprime numbers (using the Euclid lesson):

```
int count = 0;
for (int k = 1; k <= n; k++) {
 if (gcd(k, n) == 1) count++;
}
```

Guided example

The efficient program never tests individual numbers. It **finds the prime factors of n** by trial division and applies "× (1 − 1/p)" once for each. Trace n = 60 = $2^2 \cdot 3 \cdot 5$:

φ(60) = 16.

step	p	n before	p divides n?	n after	result
start	—	60	—	60	60
1	2	60	yes (strip 2,2)	15	60 − 60/2 = 30
2	3	15	yes (strip 3)	5	30 − 30/3 = 20
3	4	5	loop stops: 4·4 > 5	5	20
leftover	—	5 > 1	prime factor 5 remains	5	20 − 20/5 = 16

BruteForcePhi.java — Brute force φ(n) (simple solution)

```
1  import java.util.Scanner;
2
3  public class BruteForcePhi {
4   static int gcd(int a, int b) {                                    (1)
5   while (b != 0) {
6   int r = a % b;
7   a = b;
8   b = r;
9   }
10  return a;
11  }
12
13  // phi(n) = how many numbers k in 1..n have gcd(k, n) = 1 ("coprime to n")
14  static int phi(int n) {
15  int count = 0;
16  for (int k = 1; k <= n; k++) {                                    (2)
17  if (gcd(k, n) == 1) {                                             (3)
18  count++;
19  }
20  }
21  return count;
22  }
23
24  public static void main(String[] args) {
25  Scanner sc = new Scanner(System.in);
26  int n = sc.nextInt();
27  System.out.println("phi(" + n + ") = " + phi(n));
28  }
29 }
```

Counts every k in 1..n with gcd(k, n) = 1. O(n log n).

Sample input

```
24
```

Line-by-line walkthrough

#	Where	What happens
1	line 4	Euclid's algorithm from the previous lesson, reused as a helper.
2	line 16	Test every number from 1 to n.
3	line 17	k is coprime to n exactly when the gcd is 1.

EulerPhi.java — φ(n) from prime factors (optimised solution)

```
1  import java.util.Scanner;
2
3  public class EulerPhi {
4   // phi(n) = n * (1 - 1/p1) * (1 - 1/p2) * ... over the DISTINCT primes p dividing n.
5   // We apply each factor as result -= result / p which stays in whole numbers.
6   static long phi(long n) {
7   long result = n;                                                    (1)
8   for (long p = 2; p * p <= n; p++) {                                 (2)
9   if (n % p == 0) { // p is a prime factor of n                       (3)
10  while (n % p == 0) { // strip every copy of p so we never test it again
11  n /= p;                                                             (4)
12  }
13  result -= result / p; // multiply result by (1 - 1/p)               (5)
14  }
15  }
16  if (n > 1) { // what is left is one prime factor larger than sqrt(original n)  (6)
17  result -= result / n;
18  }
19  return result;
20  }
21
22  public static void main(String[] args) {
23  Scanner sc = new Scanner(System.in);
24  long n = sc.nextLong();
25  System.out.println("phi(" + n + ") = " + phi(n));
26  }
27 }
```

Finds each distinct prime factor p ≤ √n and applies result × (1 − 1/p) in integer form. O(√n).

Sample input

```
60
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	Start with "all n numbers survive" and remove the unwanted ones prime by prime.
2	line 8	A composite number has a prime factor ≤ its square root, so after this loop at most ONE prime factor is left over. Using `long` prevents p*p overflowing.
3	line 9	The first p that divides n is always prime (any smaller prime factor was already removed).
4	line 11	Strip every copy of p so that n shrinks and p is never counted twice.

#	Where	What happens
5	line 13	Multiply result by (1 – 1/p). `result / p` is exact because result is still divisible by p, so there is no rounding.
6	line 16	Whatever is left is one prime factor bigger than √(original n). It needs its own (1 – 1/p) factor.

EulerPhiTable.java — φ for every number up to N (demonstration)

```
1  public class EulerPhiTable {
2   // Computes phi(1..limit) for all numbers at once with a sieve-like pass.
3   static int[] phiUpTo(int limit) {
4   int[] phi = new int[limit + 1];
5   for (int i = 0; i <= limit; i++) {
6   phi[i] = i;                                                    (1)
7   }
8   for (int p = 2; p <= limit; p++) {
9   if (phi[p] == p) { // untouched so far => p is prime           (2)
10  for (int multiple = p; multiple <= limit; multiple += p) {
11  phi[multiple] -= phi[multiple] / p;                            (3)
12  }
13  }
14  }
15  return phi;
16  }
17
18  public static void main(String[] args) {
19  int[] phi = phiUpTo(20);
20  for (int n = 1; n <= 20; n++) {
21  System.out.println("phi(" + n + ") = " + phi[n]);
22  }
23  }
24 }
```

Same formula applied sieve-style: for each prime p, update all its multiples. O(N log log N).

Line-by-line walkthrough

#	Where	What happens
1	line 6	Every φ starts at i itself.
2	line 9	Nobody has reduced phi[p] yet, so p has no smaller prime factor: p is prime.
3	line 11	Apply (1 – 1/p) to every multiple of the prime p.

Dry run: watch it execute

φ(60): strip each prime factor of n and multiply result by (1 – 1/p).

Algorithm: Euler's φ (totient). **Input:** n = `60`.

Pseudocode

```
result = n;
for (p = 2; p * p <= n; p++) {
 if (n % p == 0) {
 while (n % p == 0) n /= p;
 result -= result / p;
 }
}
if (n > 1) result -= result / n;
return result;
```

Step 1 of 6. result starts as n = 60.

p = — · n = 60 · result = 60

Step 2 of 6. 2 divides n: strip every 2 (n becomes 15); result = 60 − 60/2 = 30.

p = 2 · n = 15 · result = 30

Step 3 of 6. 3 divides n: strip every 3 (n becomes 5); result = 30 − 30/3 = 20.

p = 3 · n = 5 · result = 20

Step 4 of 6. Loop ends: p·p = 16 > n = 5.

p = 4 · n = 5 · result = 20

Step 5 of 6. Leftover prime 5: result = 20 − 20/5 = 16.

p = 4 · n = 5 · result = 16

Step 6 of 6. φ(60) = 16.

p = 4 · n = 5 · result = 16

> **RESULT**
> φ(60) = 16

Complexity

Brute force: n gcd calls, each O(log n) → O(n log n).

Factorisation method: the loop variable p goes up to √n (n only gets smaller) and the divisions inside the loop add at most O(log n) in total → **O(√n) time, O(1) space**. For n ≈ 10^{12} that is about 10^6 steps instead of 10^{12}.

Table for 1..N: each prime p touches N/p multiples; the sum of N/p over primes is N · log log N.

n	Brute force steps	Factor method steps
10^6	10^6 gcds	≤ 1 000
10^{12}	10^{12} gcds (impossible)	≤ 10^6

Common mistakes

Using the formula with (p – 1) · p^(k–1) but also dividing again

Problem

```
// φ(60) the "slide" way: (2–1)*2^1 * (2–1)*2^2 * (3–1)*3^0 * (5–1)*5^0 * (5–1)*5^1 = 32
```

Why: The lecture example repeats the factors for 2 and 5. Each distinct prime contributes ONE factor (p − 1)·p^(k−1). The correct product is (2−1)·2^1 × (3−1)·3^0 × (5−1)·5^0 = 2 × 2 × 4 = 16.

Fix: // one factor per distinct prime phi = (2-1)*2* (3-1)*1* (5-1)*1; // = 16

Forgetting the leftover prime

Problem

```
for (int p = 2; p * p <= n; p++) { ... }
return result; // missing: if (n > 1) result -= result / n;
```

Why: For n = 14 the loop strips 2 and leaves n = 7. 7·7 > 7 ends the loop before 7 is handled, so φ(14) would come out as 7 instead of 6.

Fix: if (n > 1) result -= result / n;

*int overflow in p p**

Problem

```
for (int p = 2; p * p <= n; p++) // n up to 2·10^9
```

Why: When p reaches ~46 341, p * p exceeds Integer.MAX_VALUE and becomes negative, so the loop never ends or ends wrongly.

Fix: for (long p = 2; p * p <= n; p++)

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · List the coprime numbers — *Beginner*

Read n. Print, in increasing order and separated by spaces, every k with 1 ≤ k ≤ n and gcd(k, n) = 1.

- 1 ≤ n ≤ 10^5

Example input	Expected output
12	1 5 7 11

Hint 1: You need a gcd helper — reuse Euclid. **Hint 2:** Build the output in a StringBuilder and print once.

Exercise 2 · Sum of totients — *Intermediate*

Read n and print φ(1) + φ(2) + ... + φ(n).

- 1 ≤ n ≤ 10^6

Example input	Expected output
10	32

Hint 1: Computing φ separately for each i with trial division is too slow in the worst case — use the table approach. **Hint 2:** Remember the answer can exceed int range for large n: use long for the sum.

Exercise 3 · Coprime pairs — *Challenge*

Count ordered pairs (a, b) with 1 ≤ a, b ≤ n and gcd(a, b) = 1.

- 1 ≤ n ≤ 10^6

Example input	Expected output
3	7

Hint 1: For a fixed larger value b ≥ 2, how many a < b are coprime to b? **Hint 2:** Each such unordered pair counts twice (a,b) and (b,a). Don't forget (1,1).

Quick check

Answers are in the answer key at the end of this part.

Q1. What is φ(17)?

- **A.** 8
- **B.** 15
- **C.** 16
- **D.** 17

Q2. What is φ(60)?

- **A.** 16
- **B.** 24
- **C.** 32
- **D.** 48

Q3. What does phi(36) return for the optimized method?

```
long result = 36, n = 36;
for (long p = 2; p * p <= n; p++) {
 if (n % p == 0) {
 while (n % p == 0) n /= p;
 result -= result / p;
 }
}
if (n > 1) result -= result / n;
System.out.println(result);
```

Your answer: _____

Q4. Complete the line that applies the factor (1 – 1/p).

```
if (n % p == 0) {
 while (n % p == 0) n /= p;
 result ______ result / p;
}
```

Q5. Why is `if (n > 1)` needed after the loop?

- **A.** To avoid dividing by zero
- **B.** A prime factor larger than √n can remain
- **C.** Because n is always 1 after the loop
- **D.** To handle n = 0

Q6. Match each n with φ(n).

Item	Your match
φ(1)	_____
φ(9)	_____
φ(10)	_____
φ(13)	_____

Choose from: `1`, `12`, `4`, `6`

Interview / exam questions

- What does Euler's totient count? Give φ(p) and φ(p^k).
- Prove or explain why φ(ab) = φ(a)φ(b) when gcd(a, b) = 1.
- How would you compute φ for all numbers up to 10^7?
- Where does φ appear? (Euler's theorem, RSA, modular inverses).

Watch & read

Video: Euler's Totient Function (Phi Function) — Neso Academy · 8:40. *Definition and meaning of φ(n)*

Video: Euler's Totient Function (Solved Examples) — Neso Academy · 12:53. *Computing φ(n) from prime factors*

REVISION CARD (30–60 SECONDS)

- φ(n) = count of k in 1..n with gcd(k, n) = 1.
- φ(n) = n · Π(1 − 1/p) over DISTINCT primes p | n.
- Code: for p ≤ √n strip p, result −= result/p; finally if n > 1: result −= result/n.
- O(√n) time, O(1) space. Prime: φ(p) = p − 1.
- φ(60) = 16 (not 32).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide computes φ(60) = 32. The correct value is 16: 60 × (1/2) × (2/3) × (4/5) = 16. The slide repeated factors for the primes 2 and 5. The Java code on the slide is correct and returns 16. • The slide's product formula φ(n) = $(p_1-1)(p_2-1)\ldots \cdot p_1\text{^}(k_1-1)\ldots$ is right, but its intermediate text for φ(60) lists the factors of 5 twice.

5.3 Chinese Remainder Theorem

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 45 min · complete path

Recover a number from its remainders modulo several pairwise-coprime numbers.

YOU WILL BE ABLE TO

- Read a system of congruences x ≡ r (mod m) and explain what it asks.
- Solve small systems by searching, and larger ones with the CRT formula.
- Compute a modular inverse and use it inside the formula.
- Explain why the answer is unique only modulo M = $m_1 \cdot m_2 \cdot \ldots$

Before this lesson: Euclid's Algorithm (GCD) · Divisibility, GCD/LCM, Primes and Modular Arithmetic

The idea

A general lines up soldiers in rows of 3 and 2 are left over; in rows of 5, 3 are left over; in rows of 7, 2 are left over. How many soldiers are there? The question "find x such that x % 3 = 2, x % 5 = 3, x % 7 = 2" is called a **system of congruences**, and it is written x ≡ 2 (mod 3), x ≡ 3 (mod 5), x ≡ 2 (mod 7).

Think of three clocks with 3, 5 and 7 positions. A counter x makes all three tick. The three clock positions together are different for every x between 0 and 104, and the pattern repeats every 3 × 5 × 7 = 105 steps. So **the triple of remainders identifies x uniquely, up to multiples of 105**. That is the theorem (it needs the moduli to be pairwise coprime — share no factors).

WHY THIS WORKS

To build the answer we make three "switches". For modulus 3 we need a number that is 1 mod 3 but a multiple of 5 and 7, i.e. a multiple of M_1 = 105/3 = 35 that is ≡ 1 (mod 3): 35 · inverse(35 mod 3) = 35 · 2 = 70. Multiply by the wanted remainder 2 and it is "on" for modulus 3 and invisible (≡ 0) for 5 and 7. Add the three switched-on parts and take mod 105.

```
x = 2·(35·2) + 3·(21·1) + 2·(15·1) = 140 + 63 + 30 = 233
233 mod 105 = 23 ← the smallest non-negative solution
check: 23 mod 3 = 2 ✓ 23 mod 5 = 3 ✓ 23 mod 7 = 2 ✓
```

Syntax

Java ideas: arrays for the moduli and remainders, nested loops with `break`, `long` arithmetic for products of moduli, and the remainder operator on possibly large numbers.

A tiny example

Brute force: try x = 0, 1, 2, ... until all three remainders match.

```java
int x = 0;
while (!(x % 3 == 2 && x % 5 == 3 && x % 7 == 2)) {
 x++;
}
System.out.println(x); // 23
```

Guided example

The slide solution is exactly this search generalised to arrays. It is perfectly fine while M is small, but needs up to M iterations. The formula needs only a few multiplications and one modular inverse per modulus.

CrtBruteForce.java — CRT by search (simple solution)

```
import java.util.Scanner;

public class CrtBruteForce {
 // Slide approach: try x = 1, 2, 3, ... until x % m[i] == r[i] for every i.
 static long solve(int[] mods, int[] rems) {
 long limit = 1;
 for (int m : mods) {
 limit *= m; // the answer repeats every M = product of moduli                    (1)
 }
 for (long x = 0; x < limit; x++) {
 boolean ok = true;
 for (int i = 0; i < mods.length; i++) {
 if (x % mods[i] != rems[i]) {                                                    (2)
 ok = false;
 break;                                                                           (3)
 }
 }
 if (ok) {
 return x;
 }
 }
 return -1; // no solution (moduli not pairwise coprime / inconsistent)
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] mods = new int[n];
 int[] rems = new int[n];
 for (int i = 0; i < n; i++) mods[i] = sc.nextInt();
 for (int i = 0; i < n; i++) rems[i] = sc.nextInt();
 System.out.println(solve(mods, rems));
 }
}
```

Tries x = 0 … M−1 until x % m[i] == r[i] for every i. At most M iterations.

Sample input

```
3
3 5 7
2 3 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	The answers repeat every M = $m_1 \cdot m_2 \cdot \ldots$, so searching below M is enough; if nothing is found there is no solution.
2	line 13	One failed congruence rejects x immediately.
3	line 15	Leave the inner loop early — no need to test the remaining moduli.

CrtFormula.java — CRT by formula (optimised solution)

```
1  import java.util.Scanner;
2
3  public class CrtFormula {
4   // modular inverse of a modulo m by trying 1..m-1 (fine for small moduli; see Euclid
        lesson for the fast way)
5   static long inverse(long a, long m) {
6   a %= m;
7   for (long x = 1; x < m; x++) {
8   if (a * x % m == 1) {
9   return x;
10  }
11  }
12  throw new IllegalArgumentException("no inverse: moduli are not coprime");
13  }
14
15  // x = sum( r_i * M_i * inv(M_i mod m_i) ) mod M where M_i = M / m_i
16  static long solve(int[] mods, int[] rems) {
17  long M = 1;
18  for (int m : mods) {
19  M *= m;
20  }
21  long x = 0;
22  for (int i = 0; i < mods.length; i++) {
23  long Mi = M / mods[i];                                                    (1)
24  long b = inverse(Mi, mods[i]);                                           (2)
25  x += rems[i] * Mi % M * b % M;                                           (3)
26  x %= M;
27  }
28  return x;
29  }
30
31  public static void main(String[] args) {
32  Scanner sc = new Scanner(System.in);
33  int n = sc.nextInt();
34  int[] mods = new int[n];
35  int[] rems = new int[n];
36  for (int i = 0; i < n; i++) mods[i] = sc.nextInt();
37  for (int i = 0; i < n; i++) rems[i] = sc.nextInt();
38  System.out.println(solve(mods, rems));
39  }
40 }
```

x = Σ r_i · M_i · inverse(M_i mod m_i), reduced mod M.

Sample input

```
3
3 5 7
2 3 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 23	M_i is the product of all the OTHER moduli. It is ≡ 0 modulo each of them and invertible modulo m_i.
2	line 24	b·M_i ≡ 1 (mod m_i). Here the inverse is found by trying 1..m−1; the Euclid lesson gives a faster way.
3	line 25	Scale the "switch" by the required remainder. `% M` after each multiplication keeps numbers small.

Dry run: watch it execute

Sun Tzu's classic: x ≡ 2 (mod 3), 3 (mod 5), 2 (mod 7) → 23.

Algorithm: Chinese remainder theorem by sieving. **Input:** pairs = `2:3,3:5,2:7`.

Pseudocode

```
x = r1; step = m1
for each further (r, m):
 while x % m != r: x += step
 step = lcm(step, m)
x is the smallest solution
```

Step 1 of 5. x ≡ 2 (mod 3): the candidates are 2, 5, 8, ... (step 3).

x = 2 · step = 3

Step 2 of 5. Next condition x ≡ 3 (mod 5). Candidates 2, 5, 8: 8 % 5 = 3 ✓.

x = 8 · step = 3 · mod = 5

candidates for mod 5

0	1	2
2	5	8

Step 3 of 5. All later solutions differ by the lcm(3, 5) = 15, so the step grows to 15.

x = 8 · step = 15

Step 4 of 5. Next condition x ≡ 2 (mod 7). Candidates 8, 23: 23 % 7 = 2 ✓.

x = 23 · step = 15 · mod = 7

candidates for mod 7

0	1
8	23

Step 5 of 5. All later solutions differ by the lcm(15, 7) = 105, so the step grows to 105.

x = 23 · step = 105

> **RESULT**
> x = 23 (repeats every 105)

Complexity

Search: O(M · k) where M = ∏ m_i and k is the number of congruences — exponential in the number of moduli.

Formula: for each of the k congruences one inverse (O(log m) with extended Euclid, O(m) with the loop used here) → O(k · log M) with Euclid. Space O(k).

Common mistakes

Moduli that are not coprime

Problem

```
// x % 6 == 2 and x % 4 == 1 → no x exists, but the formula silently returns garbage
```

Why: The CRT formula needs gcd(m_i, m_j) = 1. 6 and 4 share the factor 2, so x must be even (from mod 6) and odd (from mod 4) — impossible. Without coprimality the inverse does not exist.

Fix: // check gcd(mi, mj) == 1 first, or use the general merge from the challenge exercise

Reading the answer as THE solution

Problem

```
// "x = 233" for the slide example
```

Why: Any x + 105k is a solution. The conventional answer is the smallest non-negative one: 233 mod 105 = 23.

Fix: x = ((x % M) + M) % M;

Overflow in the products

Problem

```
x += rems[i] * Mi * b; // can exceed long for big moduli
```

Why: Mi alone may be near 10^{12}, and times b and remainder may overflow 64 bits.

Fix: x = (x + rems[i] % M * Mi % M * b) % M;

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Two congruences — *Beginner*

Given coprime moduli a and b and remainders r1, r2 (read in the order `a r1 b r2`), print the smallest non-negative x with x % a = r1 and x % b = r2.

- 2 ≤ a, b ≤ 1000, gcd(a, b) = 1
- 0 ≤ r1 < a, 0 ≤ r2 < b

Example input	Expected output
3 2 5 3	8

Hint 1: The answer is below a·b. **Hint 2:** Loop x from 0 and test both conditions.

Exercise 2 · CRT with the formula — *Intermediate*

Read n, then n lines `m r`. All moduli are pairwise coprime. Print the smallest non-negative x with x ≡ r (mod m) for all lines.

- 1 ≤ n ≤ 5
- 2 ≤ m ≤ 1000
- product of moduli ≤ 10^{12}

Example input	Expected output
3 ⏎ 3 2 ⏎ 5 3 ⏎ 7 2	23

Hint 1: M_i = M / m_i. **Hint 2:** Inverse of M_i modulo m_i: extended Euclid, or try 1..m_i−1. **Hint 3:** Reduce mod M after every multiplication.

Exercise 3 · Non-coprime moduli — *Challenge*

Read `r1 m1 r2 m2` (the moduli may share factors). Print the smallest non-negative x with x ≡ r1 (mod m1) and x ≡ r2 (mod m2), or -1 if none exists.

- 1 ≤ m1, m2 ≤ 10^6
- 0 ≤ r1 < m1, 0 ≤ r2 < m2

Example input	Expected output
2 6 4 10	14
2 6 1 4	−1

Hint 1: Only numbers r1, r1+m1, r1+2·m1, … satisfy the first congruence. **Hint 2:** Their remainders modulo m2 repeat after at most m2 steps — try that many.

Quick check

Answers are in the answer key at the end of this part.

Q1. The smallest non-negative x with x ≡ 2 (mod 3), x ≡ 3 (mod 5), x ≡ 2 (mod 7) is…

- **A.** 23
- **B.** 233
- **C.** 105
- **D.** 35

Q2. Which condition do the moduli need for the CRT formula?

- **A.** All equal
- **B.** All prime
- **C.** Pairwise coprime
- **D.** Sum to 100

Q3. What is printed?

```
int x = 0;
while (!(x % 4 == 1 && x % 5 == 2)) x++;
System.out.println(x);
```

Your answer: _____

Q4. In the formula, what is M_i?

- **A.** m_i squared
- **B.** M / m_i
- **C.** M − m_i
- **D.** r_i · m_i

Q5. The system x ≡ 1 (mod 6), x ≡ 2 (mod 4) has…

- **A.** x = 13
- **B.** x = 5
- **C.** no solution
- **D.** infinitely many but none below 24

Interview / exam questions

- State the CRT. When is the solution unique?
- Compute the modular inverse of 35 modulo 3 by hand.
- How would you solve a system whose moduli are not coprime?
- Where does CRT appear in cryptography (RSA speed-up)?

Watch & read

Video: The Chinese Remainder Theorem (Solved Example 1) — Neso Academy · 14:22. *Solving simultaneous congruences*

Video: The Chinese Remainder Theorem (Solved Example 2) — Neso Academy · 12:06. *A worked CRT example*

REVISION CARD (30–60 SECONDS)

- x ≡ r_i (mod m_i), m_i pairwise coprime → unique x modulo M = Πm_i.
- x = Σ r_i·M_i·inv(M_i mod m_i), M_i = M / m_i, then x mod M.
- Slide example: x = 233 ≡ 23 (mod 105).
- Not coprime? Check consistency (gcd) first.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide gives the answer as "x ≡ 233 (mod 105)". That is a correct congruence but the canonical answer is the reduced value 23. • The slide's Java program has a compile error (`for(int i=0; i<n; i++)` uses n, which is never declared — it should be `size`) and its brute-force loop never terminates when no solution exists. The versions in this lesson fix both.

ASSUMPTIONS MADE

• The slide's program takes the divisors first and then the remainders; the lesson programs use the same input order (`n`, then moduli, then remainders).

5.4 Combinations (nCr)

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 40 min · complete path

Count the ways to choose r items from n, and list them — without overflowing on big factorials.

YOU WILL BE ABLE TO

- Explain the difference between a permutation (order matters) and a combination (order does not).
- Compute nCr with the factorial formula and see why it overflows for n ≥ 13.
- Compute nCr safely with a multiply-then-divide loop and with Pascal's triangle.
- Generate all combinations of an array with recursion and backtracking.

Before this lesson: Loops: for, while, do-while, break and continue · Methods: Parameters, Return Values, Scope and Overloading · Recursion: Base Case, Recursive Case and the Call Stack

The idea

Choosing 2 friends from {Ann, Ben, Cy} for a pair project: {Ann, Ben}, {Ann, Cy}, {Ben, Cy} — **3** ways. {Ann, Ben} and {Ben, Ann} are the same team, so **order does not matter**. That count is written **C(3, 2)** or ${}^{3}C_{2}$ ("3 choose 2").

If order did matter (president and vice-president), there would be 3 × 2 = 6 arrangements (a **permutation**). Every team of 2 was counted 2! = 2 times, so the number of teams is 6 / 2 = 3. In general:

```
C(n, r) = n! / (r! · (n − r)!)  (what the lecture slide uses)
```

WHY THIS WORKS

Another way to see the formula is Pascal's rule: C(n, r) = C(n−1, r−1) + C(n−1, r). Take one special item. Either it is in the team (choose r−1 more from the other n−1) or it is not (choose all r from the other n−1). That recurrence builds Pascal's triangle and never needs a factorial.

Syntax

```
long result = 1;
for (int i = 1; i <= r; i++) result = result * (n - r + i) / i; // multiply THEN divide: stays
    whole

// import java.math.BigInteger; // when long overflows (n > ~60)
BigInteger f = BigInteger.ONE;
f = f.multiply(BigInteger.valueOf(k)).divide(BigInteger.valueOf(i));
long safe = Math.multiplyExact(a, b); // throws ArithmeticException instead of wrapping
```

A tiny example

```
// 5C2 = 5! / (2! * 3!) = 120 / (2 * 6) = 10
```

Guided example

The lecture program puts numbers in an `ArrayList` and uses only its **size** n = 5 — the list contents (9, 12, 19, 61, 19) play no role in counting. `fact` is written with a `while` loop. It prints 10.

> **WATCH OUT**
> `int` can only hold up to 2 147 483 647. 13! = 6 227 020 800 already overflows, so the factorial formula silently gives wrong answers for n ≥ 13 — even when the final answer is small.

CombinationNaive.java — nCr with factorials (simple solution)

```
1  import java.util.Scanner;
2
3  public class CombinationNaive {
4   // The slide's formula nCr = n! / (r! * (n-r)!) -- correct, but n! overflows int for n
        > 12.
5   static int fact(int k) {                                                    (1)
6   int f = 1;
7   for (int j = 2; j <= k; j++) {
8   f *= j;
9   }
10  return f;
11  }
12
13  public static void main(String[] args) {
14  Scanner sc = new Scanner(System.in);
15  int n = sc.nextInt();
16  int r = sc.nextInt();
17  System.out.println(n + "C" + r + " = " + fact(n) / (fact(r) * fact(n - r)));   (2)
18  }
19 }
```

Direct translation of n! / (r! (n−r)!). Correct for n ≤ 12, breaks for larger n because of overflow.

Sample input

```
5 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 5	Multiplies 2 · 3 · … · k. The starting value 1 also handles 0! = 1.
2	line 17	Integer division is exact because the true quotient is a whole number — if nothing overflowed.

Combination.java — nCr multiplicatively (optimised solution)

```
1  import java.util.Scanner;
2
3  public class Combination {
4   // nCr without ever building n!: multiply and divide one step at a time.
5   // After step i the running value is exactly C(n - r + i, i), always a whole number.
6   static long nCr(int n, int r) {
7   if (r < 0 || r > n) {
8   return 0;
9   }
10  r = Math.min(r, n - r); // C(n, r) == C(n, n - r); fewer steps                    (1)
11  long result = 1;
12  for (int i = 1; i <= r; i++) {
13  result = result * (n - r + i) / i;                                              (2)
14  }
15  return result;
16  }
17
18  public static void main(String[] args) {
19  Scanner sc = new Scanner(System.in);
20  int n = sc.nextInt();
21  int r = sc.nextInt();
22  System.out.println(n + "C" + r + " = " + nCr(n, r));
23  }
24 }
```

Builds the answer in r steps: after step i the value is C(n−r+i, i). Uses long and the symmetry C(n, r) = C(n, n−r).

Sample input

```
30 15
```

Line-by-line walkthrough

#	Where	What happens
1	line 10	C(100, 98) = C(100, 2): choose the smaller side so the loop is short.
2	line 13	Multiply FIRST then divide. The product of i consecutive numbers is always divisible by i!, so `/ i` is exact at every step.

PascalTriangle.java — Pascal's triangle (demonstration)

```
1  public class PascalTriangle {
2   // C(n, r) = C(n-1, r-1) + C(n-1, r) -- pick the last item or skip it.
3   public static void main(String[] args) {
4   int rows = 6;
5   long[][] c = new long[rows][rows];
6   for (int n = 0; n < rows; n++) {
7   c[n][0] = 1;                                                    (1)
8   for (int r = 1; r <= n; r++) {
9   c[n][r] = c[n - 1][r - 1] + (r <= n - 1 ? c[n - 1][r] : 0);     (2)
10  }
11  }
12  for (int n = 0; n < rows; n++) {
13  StringBuilder line = new StringBuilder();
14  for (int r = 0; r <= n; r++) {
15  line.append(c[n][r]).append(' ');
16  }
17  System.out.println(line.toString().trim());
18  }
19  }
20 }
```

Each entry is the sum of the two entries above it. Only addition, so no overflow until the numbers themselves are huge.

Line-by-line walkthrough

#	Where	What happens
1	line 7	There is one way to choose nothing.
2	line 9	Pascal's rule: include the last item, or skip it.

CombinationsList.java — List all combinations (backtracking) (demonstration)

```
1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class CombinationsList {
5   // Prints every way to choose r items from the array (the "harder" part mentioned in
        the lecture).
6   static void combos(int[] items, int r, int start, List<Integer> current) {
7   if (current.size() == r) {                                      (1)
8   System.out.println(current);
9   return;
10  }
11  for (int i = start; i < items.length; i++) {
12  current.add(items[i]); // choose items[i]                       (2)
13  combos(items, r, i + 1, current); // go on with the items to its right   (3)
14  current.remove(current.size() - 1); // un-choose (backtrack)    (4)
15  }
16  }
17
18  public static void main(String[] args) {
19  combos(new int[] {9, 12, 19, 61}, 2, 0, new ArrayList<>());
20  }
21 }
```

Choose an item, recurse on the items to its right, then un-choose it and try the next item.

Line-by-line walkthrough

#	Where	What happens
1	line 7	Base case: we have exactly r items — print this combination.
2	line 12	Choose.
3	line 13	Explore: only items to the RIGHT can be added, which prevents duplicates like [12, 9].
4	line 14	Un-choose (backtrack) so the next loop iteration starts from a clean selection.

Dry run: watch it execute

Pascal's triangle and the multiplicative formula agree.

Algorithm: nCr from Pascal's triangle and from the formula. **Input:** n = `6`, r = `2`.

Pseudocode

```
Pascal: C(i, j) = C(i-1, j-1) + C(i-1, j)
multiplicative:
r = min(r, n - r); result = 1
for i = 1..r: result = result * (n - r + i) / i
```

Step 1 of 11. Row 0: the edges are always 1.

row = 0

1						

Step 2 of 11. Row 1: the edges are always 1.

row = 1

1						
1	1					

Step 3 of 11. Row 2: each inner entry is the sum of the two entries above it.

row = 2

1						
1	1					
1	2	1				

Step 4 of 11. Row 3: each inner entry is the sum of the two entries above it.

row = 3

1						
1	1					
1	2	1				
1	3	3	1			

Step 5 of 11. Row 4: each inner entry is the sum of the two entries above it.

row = 4

1						
1	1					
1	2	1				
1	3	3	1			
1	4	6	4	1		

Step 6 of 11. Row 5: each inner entry is the sum of the two entries above it.

row = 5

1						
1	1					
1	2	1				
1	3	3	1			
1	4	6	4	1		
1	5	10	10	5	1	

Step 7 of 11. Row 6: each inner entry is the sum of the two entries above it.

row = 6

1						
1	1					
1	2	1				
1	3	3	1			
1	4	6	4	1		
1	5	10	10	5	1	
1	6	15	20	15	6	1

Step 8 of 11. C(6, 2) is the entry in row 6, column 2 = 15.

C(6,2) = 15

1						
1	1					
1	2	1				
1	3	3	1			
1	4	6	4	1		
1	5	10	10	5	1	
1	6	15	20	15	6	1

Step 9 of 11. Formula version: use symmetry C(6, 2) = C(6, 4) and take r = 2 (fewer steps). result starts at 1.

r = 2 · result = 1

Step 10 of 11. i = 1: result = result × (n − r + i) / i = × 5 / 1 → 5. It is always a whole number, so integer division is exact.

i = 1 · result = 5

Step 11 of 11. i = 2: result = result × (n − r + i) / i = × 6 / 2 → 15. It is always a whole number, so integer division is exact.

i = 2 · result = 15

> **RESULT**
> 6C2 = 15

Complexity

Factorial formula: O(n) multiplications, O(1) space — but incorrect for n ≥ 13 in `int` (n ≥ 21 in `long`).

Multiplicative loop: O(min(r, n−r)) time, O(1) space. Valid whenever the *result* fits in the type.

Pascal: O(n · r) time, O(n · r) space (can be O(r) with a single row).

Listing all combinations: there are C(n, r) of them, each costing O(r) to print → O(r · C(n, r)). The count itself explodes — C(40, 20) ≈ 1.4 × 10^{11}.

Common mistakes

Overflow in the factorial

Problem

```
int c = fact(20) / (fact(2) * fact(18));
```

Why: 20! overflows int (and even long is borderline). The result printed is nonsense, with no error message.

Fix: long result = 1; for (int i = 1; i <= r; i++) result = result * (n - r + i) / i;

Divide before multiply

Problem

```
result = result / i * (n - r + i);
```

Why: Integer division truncates: 4/3 becomes 1 and the answer is wrong.

Fix: result = result * (n - r + i) / i; // multiply first

Treating order as important

Problem

```
// printing [9,12] and [12,9] as two different combinations
```

Why: A combination is a set. Starting the recursive loop at i + 1 (not 0) prevents revisiting earlier items.

Fix: for (int i = start; i < items.length; i++) ...

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · nCr as a number — *Beginner*

Read n and r (0 ≤ r ≤ n ≤ 30). Print C(n, r).

Example input	Expected output
5 2	10

Hint 1: Multiply first, then divide, one step at a time. **Hint 2:** Use long.

Exercise 2 · nCr modulo 1 000 000 007 — *Intermediate*

Read n and r (0 ≤ r ≤ n ≤ 1000). Print C(n, r) mod 1 000 000 007.

Example input	Expected output
10 3	120

Hint 1: The true value is far too large for long — use Pascal's rule and take % MOD after each addition.

Exercise 3 · List all combinations — *Challenge*

Read n and r (1 ≤ r ≤ n ≤ 10). Print every r-element subset of {1..n}, one per line, elements in increasing order, subsets in lexicographic order.

Example input	Expected output
4 2	1 2 ⏎ 1 3 ⏎ 1 4 ⏎ 2 3 ⏎ 2 4 ⏎ 3 4

Hint 1: Recursion: choose(start, count). **Hint 2:** Only pick values larger than the previous one.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is C(6, 2)?

- **A.** 12
- **B.** 15
- **C.** 30
- **D.** 36

Q2. How many ways to arrange (order matters) 2 of 6 items?

- **A.** 12
- **B.** 15
- **C.** 30
- **D.** 36

Q3. What is printed?

```
long result = 1;
int n = 6, r = 2;
for (int i = 1; i <= r; i++)
 result = result * (n - r + i) / i;
System.out.println(result);
```

Your answer: _____

Q4. Why does `fact(15) / (fact(2) * fact(13))` print a wrong answer in int arithmetic?

- **A.** Division is wrong
- **B.** fact(15) overflows int
- **C.** r is too small
- **D.** nCr is undefined for n=15

Q5. C(n, r) equals...

- **A.** C(n, n−r)
- **B.** C(n, r+1)
- **C.** C(n−1, r)
- **D.** n · r

Interview / exam questions

- Difference between nCr and nPr?
- Compute C(1000, 3) without overflow.
- Why is Pascal's rule true?
- How many subsets does an n-element set have? ($2^n = \Sigma\ C(n, r)$)

Watch & read

Video: The Combination Formula and Why it Works | Combinatorics, Combinations and Permutations — Wrath of Math · 8:14. *The nCr formula and its computation*

Video: Pascal Triangle | Finding nCr in minimal time — take U forward · 26:45. *Pascal's triangle and nCr by DP*

REVISION CARD (30–60 SECONDS)

- C(n,r) = n! / (r!(n−r)!)
- C(n,r) = C(n,n−r); C(n,0) = C(n,n) = 1
- Loop: result = result*(n−r+i)/i for i=1..r
- Pascal: C(n,r) = C(n−1,r−1) + C(n−1,r)
- Overflow: 13! > int, 21! > long

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide places an ArrayList of 5 numbers into the program, but only list.size() is ever used, so the "9, 12, 19, 61, 19" values have no effect on the result (10). • The slide's factorial uses int and would overflow for n ≥ 13 — fine for the sample, wrong in general.

5.5 Karatsuba's Multiplication

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 55 min · complete path

Multiply two big numbers with three recursive multiplications instead of four — a first taste of divide and conquer.

YOU WILL BE ABLE TO

- Explain school-book multiplication and its $O(n^2)$ digit multiplications.
- Derive the 3-multiplication trick (a+b)(c+d) − ac − bd = ad + bc.
- Implement Karatsuba for long and for BigInteger and test it against *.
- Solve the recurrence T(n) = 3T(n/2) + O(n) = O(n^1.585).

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Algorithm Design Paradigms · Euclid's Algorithm (GCD)

The idea

Multiply 1234 × 5678 by splitting each number into two halves: 1234 = **12**·100 + **34** and 5678 = **56**·100 + **78**. Call the halves a = 12, b = 34, c = 56, d = 78.

$$(a \cdot 100 + b)(c \cdot 100 + d) = ac \cdot 100^2 + (ad + bc) \cdot 100 + bd$$

That needs **four** smaller multiplications: ac, ad, bc, bd. Karatsuba noticed that the middle term can be obtained from ONE extra multiplication that we already half-have:

$$(a + b)(c + d) = ac + ad + bc + bd \Rightarrow ad + bc = (a+b)(c+d) - ac - bd$$

WHY THIS WORKS

We already compute ac and bd for the outer terms. One more product, (a+b)(c+d), gives the middle term after two subtractions. Subtractions are cheap (linear time), multiplications are expensive, so trading one multiplication for a few additions makes the recursion branch 3 ways instead of 4. Over many levels that is the difference between n^2 and n^1.585.

```
1234 × 5678 a=12 b=34 c=56 d=78
ac = 12 × 56 = 672
bd = 34 × 78 = 2652
(a+b)(c+d)= 46 × 134 = 6164
ad + bc = 6164 − 672 − 2652 = 2840
result = 672·10000 + 2840·100 + 2652 = 7 006 652 ✓
```

Syntax

Java ideas: recursion with a base case, `Long.toString(x).length()` to count digits, integer division `/` and remainder `%` to split a number, and `BigInteger` with `shiftLeft/shiftRight` for the bit-splitting version.

A tiny example

```
// split 1234 into halves with p = 100
long a = 1234 / 100; // 12
long b = 1234 % 100; // 34
```

Guided example

Karatsuba.java — Karatsuba (decimal split) (simple solution)

```
1  import java.util.Scanner;
2
3  public class Karatsuba {
4   static int baseCases = 0; // how many single-digit multiplications we performed
5
6   static long pow10(int k) {
7   long p = 1;
8   for (int i = 0; i < k; i++) {
9   p *= 10;
10  }
11  return p;
12  }
13
14  // Multiplies two non-negative numbers using 3 recursive multiplications instead of 4.
15  static long multiply(long x, long y) {
16  if (x < 10 || y < 10) { // base case: a one-digit factor => plain multiplication          (1)
17  baseCases++;
18  return x * y;
19  }
20  int n = Math.max(Long.toString(x).length(), Long.toString(y).length());
21  int half = n / 2; // number of digits in the LOW part                                    (2)
22  long p = pow10(half);
23
24  long a = x / p, b = x % p; // x = a * 10^half + b                                       (3)
25  long c = y / p, d = y % p; // y = c * 10^half + d
26
27  long ac = multiply(a, c);
28  long bd = multiply(b, d);
29  long abcd = multiply(a + b, c + d); // (a+b)(c+d) = ac + ad + bc + bd                   (4)
30  long adPlusBc = abcd - ac - bd; // the middle term without a 4th multiplication        (5)
31
32  return ac * p * p + adPlusBc * p + bd;                                                 (6)
33  }
34
35  public static void main(String[] args) {
36  Scanner sc = new Scanner(System.in);
37  long x = sc.nextLong();
38  long y = sc.nextLong();
39  long result = multiply(x, y);
40  System.out.println(x + " x " + y + " = " + result);
41  System.out.println("check with *: " + (x * y) + " -> " + (result == x * y ? "match" :
        "MISMATCH"));
42  System.out.println("single-digit multiplications used: " + baseCases);
43  }
44 }
```

Splits both numbers at half their digit count and recurses on three products.

Sample input

```
1234 5678
```

Line-by-line walkthrough

#	Where	What happens
1	line 16	Base case: if either factor has one digit, ordinary multiplication is already cheap.
2	line 21	The LOW part gets `half` digits; the high part gets the rest. This differs from the slide (which used (n+1)/2 with Math.pow) but both are valid.
3	line 24	x = a·10^half + b, so division gives the high part and remainder gives the low part.
4	line 29	The single extra multiplication that replaces both ad and bc.
5	line 30	Remove the terms we already know. What remains is exactly ad + bc.
6	line 32	Shift the three parts into place and add them (shifting = multiplying by powers of 10).

KaratsubaBig.java — Karatsuba (bit split, BigInteger) (optimised solution)

```
1  import java.math.BigInteger;
2  import java.util.Random;
3
4  public class KaratsubaBig {
5   // Karatsuba on BigInteger, splitting by BITS (shifts are cheaper than powers of 10).
6   static BigInteger multiply(BigInteger x, BigInteger y) {
7   int n = Math.max(x.bitLength(), y.bitLength());
8   if (n <= 32) { // small enough: use the machine multiply                    (1)
9   return x.multiply(y);
10  }
11  int half = n / 2;
12  BigInteger a = x.shiftRight(half); // high part                            (2)
13  BigInteger b = x.subtract(a.shiftLeft(half)); // low part
14  BigInteger c = y.shiftRight(half);
15  BigInteger d = y.subtract(c.shiftLeft(half));
16
17  BigInteger ac = multiply(a, c);
18  BigInteger bd = multiply(b, d);
19  BigInteger mid = multiply(a.add(b), c.add(d)).subtract(ac).subtract(bd);   (3)
20
21  return ac.shiftLeft(2 * half).add(mid.shiftLeft(half)).add(bd);           (4)
22  }
23
24  public static void main(String[] args) {
25  Random rnd = new Random(42);
26  int ok = 0;
27  for (int t = 0; t < 200; t++) {
28  BigInteger x = new BigInteger(200 + rnd.nextInt(300), rnd);
29  BigInteger y = new BigInteger(200 + rnd.nextInt(300), rnd);
30  if (multiply(x, y).equals(x.multiply(y))) {
31  ok++;
32  }
33  }
34  System.out.println(ok + " of 200 random 200-500 bit products match BigInteger.multiply")
        ;
35  BigInteger big = new BigInteger("123456789012345678901234567890");
36  System.out.println(multiply(big, big));
37  }
38 }
```

Same idea using shifts on BigInteger; handles numbers of hundreds of digits and is self-checked against BigInteger.multiply.

Line-by-line walkthrough

#	Where	What happens
1	line 8	Small enough for the hardware: use the built-in multiply instead of recursing further.
2	line 12	Shifting right by `half` bits is x / 2^half — the high part.
3	line 19	The same three-product identity, with BigInteger operations.
4	line 21	Multiplying by 2^(2·half) is a left shift — much cheaper than a decimal power.

For 1234 × 5678 the program reports **11** single-digit multiplications. The school method needs 4 × 4 = 16 digit multiplications — and the gap widens fast with the number of digits.

Dry run: watch it execute

Three recursive multiplications per level instead of four.

Algorithm: Karatsuba multiplication. **Input:** x = `1234`, y = `5678`.

Pseudocode

```
multiply(x, y):
 if x < 10 or y < 10: return x * y
 p = 10^(n/2); x = a·p + b; y = c·p + d
 ac = multiply(a, c); bd = multiply(b, d)
 abcd = multiply(a + b, c + d)
 return ac·p² + (abcd − ac − bd)·p + bd
```

The full trace has 16 steps; 14 evenly spaced ones are shown. Run the rest in the website's Dry-run lab.

Step 1 of 16. base case: 1 × 5 = 5 (one factor has a single digit).

depth = 2 · x = 1 · y = 5 · result = 5 · base multiplications = 1

Step 2 of 16. base case: 2 × 6 = 12 (one factor has a single digit).

depth = 2 · x = 2 · y = 6 · result = 12 · base multiplications = 2

Step 3 of 16. base case: 3 × 11 = 33 (one factor has a single digit).

depth = 2 · x = 3 · y = 11 · result = 33 · base multiplications = 3

Step 4 of 16. 12 × 56: split at 10^1 → a=1, b=2, c=5, d=6. ac=5, bd=12, (a+b)(c+d)=33, middle = 33 − 5 − 12 = 16. Result 5·100 + 16·10 + 12 = 672.

depth = 1 · ac = 5 · bd = 12 · a·d + b·c = 16 · result = 672

Step 6 of 16. base case: 4 × 8 = 32 (one factor has a single digit).

depth = 2 · x = 4 · y = 8 · result = 32 · base multiplications = 5

Step 7 of 16. base case: 7 × 15 = 105 (one factor has a single digit).

depth = 2 · x = 7 · y = 15 · result = 105 · base multiplications = 6

Step 8 of 16. 34 × 78: split at 10^1 → a=3, b=4, c=7, d=8. ac=21, bd=32, (a+b)(c+d)=105, middle = 105 − 21 − 32 = 52. Result 21·100 + 52·10 + 32 = 2652.

depth = 1 · ac = 21 · bd = 32 · a·d + b·c = 52 · result = 2652

Step 9 of 16. base case: 4 × 13 = 52 (one factor has a single digit).

depth = 2 · x = 4 · y = 13 · result = 52 · base multiplications = 7

Step 10 of 16. base case: 6 × 4 = 24 (one factor has a single digit).

depth = 2 · x = 6 · y = 4 · result = 24 · base multiplications = 8

Step 11 of 16. base case: 1 × 1 = 1 (one factor has a single digit).

depth = 3 · x = 1 · y = 1 · result = 1 · base multiplications = 9

Step 13 of 16. base case: 1 × 8 = 8 (one factor has a single digit).

depth = 3 · x = 1 · y = 8 · result = 8 · base multiplications = 11

Step 14 of 16. 10 × 17: split at 10^1 → a=1, b=0, c=1, d=7. ac=1, bd=0, (a+b)(c+d)=8, middle = 8 − 1 − 0 = 7. Result 1·100 + 7·10 + 0 = 170.

depth = 2 · ac = 1 · bd = 0 · a·d + b·c = 7 · result = 170

Step 15 of 16. 46 × 134: split at 10^1 → a=4, b=6, c=13, d=4. ac=52, bd=24, (a+b)(c+d)=170, middle = 170 − 52 − 24 = 94. Result 52·100 + 94·10 + 24 = 6164.

depth = 1 · ac = 52 · bd = 24 · a·d + b·c = 94 · result = 6164

Step 16 of 16. 1234 × 5678: split at 10^2 → a=12, b=34, c=56, d=78. ac=672, bd=2652, (a+b)(c+d)=6164, middle = 6164 − 672 − 2652 = 2840. Result 672·10000 + 2840·100 + 2652 = 7006652.

depth = 0 · ac = 672 · bd = 2652 · a·d + b·c = 2840 · result = 7006652

RESULT
1234 × 5678 = 7006652 (matches *), 11 single-digit multiplications

Complexity

Let T(n) be the cost for n-digit numbers. We make **3** recursive calls on numbers about half as long, plus linear work for splitting, adding and subtracting:

$$T(n) = 3 \cdot T(n/2) + O(n) \Rightarrow T(n) = O(n^{(\log_2 3)}) \approx O(n^{1.585})$$

School-book multiplication satisfies $T(n) = 4 \cdot T(n/2) + O(n) = O(n^2)$. For 1000-digit numbers that is about 10^6 vs $1000^{1.585} \approx 5.6 \times 10^4$ digit operations.

Space: the recursion depth is $\log_2 n$, but every level allocates numbers of size O(n), so O(n) extra space overall.

digits n	school n^2	Karatsuba n^1.585	speed-up
10	100	38	≈ 2.6×
100	10 000	1 479	≈ 6.8×
1 000	1 000 000	56 000	≈ 18×
10 000	100 000 000	2 200 000	≈ 45×

NOTE
For small inputs ordinary multiplication is faster because Karatsuba has overhead. Real libraries switch to schoolbook below a threshold (BigInteger uses 80 ints) — exactly what the base case does.

Common mistakes

Using Math.pow for powers of 10

Problem

```
long p = (long) Math.pow(10, half);
```

Why: Math.pow returns a double. Beyond 2^{53} it cannot represent every integer, so results can silently be off by one.

Fix: long p = 1; for (int i = 0; i < half; i++) p *= 10;

Declaring a variable named ad+bc

Problem

```
long ad+bc = multiply(a + b, c + d) − ac − bd; // the slide
```

Why: Identifiers cannot contain "+". This is a compile-time error.

Fix: long adPlusBc = multiply(a + b, c + d) - ac - bd;

Forgetting that a+b may have more digits

Problem

```
// assuming (a+b) has n/2 digits
```

Why: The sum of two half-length numbers can carry into one more digit. The algorithm still works because every recursion strictly decreases the numbers, but the digit count of a+b is not n/2.

Fix: Recompute the length inside every call (the code above does this)

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · School-book multiplication — *Beginner*

Read two non-negative integers as digit strings (up to 100 digits each). Print their product without using BigInteger or `long` arithmetic on the whole numbers.

Example input	Expected output
12345 6789	83810205

Hint 1: Digit i of x times digit j of y contributes to position i + j + 1 of the result. **Hint 2:** Keep a carry as you add.

Exercise 2 · Count the base multiplications — *Intermediate*

Run the lesson's Karatsuba algorithm (split at n/2 low digits; base case when either factor < 10) on x and y (both < 10^9). Print how many times the base case is reached.

Example input	Expected output
1234 5678	11

Hint 1: Add a static counter incremented in the base case. **Hint 2:** Do not change the splitting rule — the answer depends on it.

Exercise 3 · Big Karatsuba — *Challenge*

Read two non-negative integers (up to 1000 digits). Multiply them with your own Karatsuba on BigInteger (do not call BigInteger.multiply except below a 30-bit threshold) and print the product.

Example input	Expected output
123456789 987654321	121932631112635269

Hint 1: Use bitLength() to decide the split point. **Hint 2:** Use shiftLeft/shiftRight and subtract to obtain the low part.

Quick check

Answers are in the answer key at the end of this part.

Q1. How many recursive multiplications does Karatsuba make per level?

- **A.** 2
- **B.** 3
- **C.** 4
- **D.** 5

Q2. Which expression equals ad + bc?

- **A.** (a+b)(c+d) − ac − bd
- **B.** (a−b)(c−d)
- **C.** ac + bd
- **D.** (a+c)(b+d)

Q3. The running time of Karatsuba is...

- **A.** O(n)
- **B.** O(n log n)
- **C.** O(n^1.585)
- **D.** O(n^2)

Q4. With a=12, b=34, c=56, d=78, what is (a+b)(c+d) − ac − bd?

```
// ac = 672, bd = 2652
long abcd = 46 * 134;
System.out.println(abcd - 672 - 2652);
```

Your answer: _____

Q5. Why do libraries use school-book multiplication for small numbers?

- **A.** It is more accurate
- **B.** Karatsuba has overhead that dominates for small n
- **C.** It is required by the JVM
- **D.** Karatsuba cannot handle small numbers

Interview / exam questions

- Derive Karatsuba's 3-multiplication identity.
- Solve T(n) = 3T(n/2) + n with the master theorem.
- Why does BigInteger switch algorithms with size?
- Name an algorithm faster than Karatsuba for huge numbers (Toom–Cook, FFT/Schönhage–Strassen).

Watch & read

Video: 1 3 Karatsuba Multiplication 13 min — Stanford Algorithms · 12:40. *Three multiplications instead of four*

Video: Understanding Karatsuba Algorithm based on Divide and Conquer Implementation — MathPhy · 20:36. *Why Karatsuba is O(n^1.585)*

REVISION CARD (30–60 SECONDS)

- x=a·10^m+b, y=c·10^m+d
- ac, bd, (a+b)(c+d); middle = third − ac − bd
- x·y = ac·10^(2m) + mid·10^m + bd
- T(n) = 3T(n/2) + O(n) → O(n^1.585)
- Base case: small factor → plain multiply

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide code contains `long ad+bc = …` and later uses `adbc`; neither compiles. The slide also uses Math.pow (floating point) to build powers of ten. • The slide's explanation labels z0 = ac in one paragraph and z0 = bd in the next. In this lesson ac is the high product and bd the low product, consistently. • The slide says the three-product count "reduces overall complexity" but never states it: it is O(n^1.585) versus O(n²).

5.6 Booth's Algorithm (signed binary multiplication)

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 60 min · complete path

Multiply two signed two's-complement numbers with additions, subtractions and shifts — the way hardware does.

YOU WILL BE ABLE TO

- Convert numbers to and from n-bit two's complement and negate them.
- Explain an arithmetic right shift and why it preserves the sign.
- Apply the three Booth rules (Q_0Q_{-1} = 00/11, 10, 01) by hand for n = 4.
- Implement the algorithm in Java and verify it against * for every input.

Before this lesson: Number Systems, Binary and Two's Complement · Bitwise Operators and Shifts · Loops: for, while, do-while, break and continue

The idea

Ordinary binary multiplication adds a shifted copy of the multiplicand for every 1 in the multiplier. 0111 (7) has three 1s so it costs three additions. But 7 = 8 − 1: **one shift and one subtraction**. A run of 1s from bit position j to k equals 2^(k+1) − 2^j.

Booth's algorithm scans the multiplier bit pairs from right to left and uses that observation: **subtract** the multiplicand where a run of 1s begins, **add** it where the run ends, and do nothing inside a run or inside a run of 0s.

Booth's conditions — the slide's "CONDITIONS" box.

Q_0	Q_{-1}	meaning	action
0	0	inside a run of 0s	only shift
1	1	inside a run of 1s	only shift
1	0	a run of 1s begins	AC = AC − M, then shift
0	1	a run of 1s ends	AC = AC + M, then shift

WHY THIS WORKS

Treat the multiplier as the sum $\Sigma(q_{i-1} - q_i)\cdot 2^i$. Each pair contributes +1, 0 or −1 times the multiplicand shifted into place. Doing the multiply with signed digits works for NEGATIVE multipliers too, because in two's complement the leading bit has a negative weight and the same pair rule gives it the right sign. The arithmetic shift (copying the sign bit) keeps the partial product's sign.

Syntax

Two's complement represents −x in n bits as (2^n − x): flip every bit and add 1. For n = 4: 7 = 0111; −7 = 1000 + 1 = 1001.

Arithmetic shift right moves every bit one place right and keeps the leftmost (sign) bit: 1100 → 1110. Java's `>>` does this for `int`; `>>>` shifts in zeros instead.

Java bit tools: `x & mask` (keep bits), `x | y`, `x << k`, `x >> k`, `(1 << n) - 1` (n ones).

A tiny example

```
int M = 7; // 0111
int negM = (~M + 1); // flip bits and add 1 → −7
System.out.println(negM); // −7
System.out.println(negM & 0xF); // 9 (= 1001, the 4-bit pattern of −7)
```

Guided example

The slide traces M = 0111 (7) and Q = 0011 (3) with registers **AC** (accumulator), **Q** (multiplier) and Q_{-1}. The table below is produced by the program `BoothMultiply` on the same input:

Result = AC:Q = 0001 0101 = 21 = 7 × 3

cycle	Q_0 Q_{-1}	operation	AC	Q	Q_{-1}
init			0000	0011	0
1	1 0	AC = AC − M, ASR	1100	1001	1
2	1 1	shift only	1110	0100	1
3	0 1	AC = AC + M, ASR	0010	1010	0
4	0 0	shift only	0001	0101	0

BoothMultiply.java — Booth's algorithm with trace (optimised solution)

```
import java.util.Scanner;

public class BoothMultiply {
 static String bits(int value, int width) {
 String s = Integer.toBinaryString(value & ((1 << width) - 1));
 return "0".repeat(width - s.length()) + s;
 }

 // Booth's algorithm for two signed numbers that each fit in `width` bits (two's complement).
 static int multiply(int multiplicand, int multiplier, int width, boolean trace) {
 if (multiplicand == -(1 << (width - 1))) {                                    (1)
 width++; // the most negative number cannot be negated in `width` bits: use one extra bit
 }
 int mask = (1 << width) - 1;
 int M = multiplicand & mask; // multiplicand register
 int negM = (-multiplicand) & mask; // two's complement of M (flip bits, add 1)   (2)
 int AC = 0; // accumulator
 int Q = multiplier & mask; // multiplier register
 int qMinus1 = 0; // the extra bit to the right of Q

 if (trace) {
 System.out.println("M = " + bits(M, width) + " -M = " + bits(negM, width));
 System.out.println("cycle Q0 Q-1 operation AC Q Q-1 (state AFTER the shift)");
 System.out.println("init - " + bits(AC, width) + " " + bits(Q, width) + " " + qMinus1);
 }
 for (int cycle = 1; cycle <= width; cycle++) {
 int q0 = Q & 1;
 int pairQ = qMinus1;
 String op = "shift only";
 if (q0 == 1 && qMinus1 == 0) { // pair "10": a run of 1s begins -> subtract M   (3)
```

(code continued)

```
31    AC = (AC + negM) & mask;
32    op = "AC = AC - M, ASR";
33    } else if (q0 == 0 && qMinus1 == 1) { // pair "01": a run of 1s ends -> add M      4
34    AC = (AC + M) & mask;
35    op = "AC = AC + M, ASR";
36    }
37    // arithmetic shift right of the combined register AC | Q | Q-1
38    qMinus1 = Q & 1;                                                                  5
39    Q = ((Q >> 1) | ((AC & 1) << (width - 1))) & mask;                                6
40    int sign = AC & (1 << (width - 1)); // remember the sign bit...
41    AC = (AC >> 1) | sign; // ...and put it back on the left                          7
42    if (trace) {
43    System.out.println(" " + cycle + " " + q0 + " " + pairQ + " " + String.format("%-15s",
          op) + " " + bits(AC, width) + " " + bits(Q, width) + " " + qMinus1);
44    }
45    }
46    long raw = ((long) AC << width) | Q; // 2*width-bit result AC:Q                  8
47    long signBit = 1L << (2 * width - 1);
48    return (int) ((raw & signBit) != 0 ? raw - (1L << (2 * width)) : raw);           9
49    }
50
51    public static void main(String[] args) {
52    Scanner sc = new Scanner(System.in);
53    int m = sc.nextInt();
54    int q = sc.nextInt();
55    int width = sc.nextInt();
56    int product = multiply(m, q, width, true);
57    System.out.println(m + " x " + q + " = " + product + " (check: " + (m * q) + ")");
58    }
59   }
```

Simulates AC, Q, Q_{-1} registers of `width` bits. Handles negative operands and the most-negative-number edge case.

Sample input

```
7 3 4
```

Line-by-line walkthrough

#	Where	What happens
1	line 11	Edge case: the most negative n-bit number (−8 in 4 bits) has no positive partner in n bits, so −M cannot be formed. One extra bit (sign extension) fixes it.
2	line 16	Two's complement of M as an n-bit pattern, precomputed so the loop can just ADD it to subtract M.
3	line 30	Pair "10": a run of 1s begins → AC = AC − M (adding −M).
4	line 33	Pair "01": a run of 1s ends → AC = AC + M.
5	line 38	Arithmetic shift of the 2n+1-bit chain AC|Q|Q_{-1}: the lowest bit of Q falls into Q_{-1}...
6	line 39	...Q shifts right and receives the lowest bit of AC at its top...
7	line 41	...and AC shifts right while the sign bit is kept: that is the arithmetic shift.
8	line 46	The 2n-bit product is AC concatenated with Q.
9	line 48	If the top bit is set the number is negative: subtract 2^(2n) to read it as signed.

BoothCheck.java — Exhaustive test (demonstration)

```
public class BoothCheck {
 // Compact copy of Booth's algorithm (no printing) so this file is self-contained.
 static int booth(int multiplicand, int multiplier, int width) {
 if (multiplicand == -(1 << (width - 1))) {
 width++;
 }
 int mask = (1 << width) - 1;
 int M = multiplicand & mask;
 int negM = (-multiplicand) & mask;
 int AC = 0;
 int Q = multiplier & mask;
 int qMinus1 = 0;
 for (int cycle = 0; cycle < width; cycle++) {
 int q0 = Q & 1;
 if (q0 == 1 && qMinus1 == 0) {
 AC = (AC + negM) & mask;
 } else if (q0 == 0 && qMinus1 == 1) {
 AC = (AC + M) & mask;
 }
 qMinus1 = Q & 1;
 Q = ((Q >> 1) | ((AC & 1) << (width - 1))) & mask;
 AC = (AC >> 1) | (AC & (1 << (width - 1)));
 }
 long raw = ((long) AC << width) | Q;
 return (int) ((raw & (1L << (2 * width - 1))) != 0 ? raw - (1L << (2 * width)) : raw);
 }

 // Exhaustively compares Booth's algorithm with the * operator for every pair of 6-bit
     signed numbers.
 public static void main(String[] args) {
 int width = 6;
 int lo = -(1 << (width - 1)); // -32
 int hi = (1 << (width - 1)) - 1; // 31
 int checked = 0;
 for (int m = lo; m <= hi; m++) {                                                    1
 for (int q = lo; q <= hi; q++) {
 int got = booth(m, q, width);
 if (got != m * q) {                                                                 2
 System.out.println("MISMATCH " + m + " x " + q + " gave " + got);
 return;
 }
 checked++;
 }
 }
 System.out.println("all " + checked + " products correct (including " + lo + " x " + lo
     + ")");
 }
}
```

*A compact copy of the algorithm compared with the * operator for all 64 × 64 six-bit pairs, including (−32) × (−32).*

Line-by-line walkthrough

#	Where	What happens
1	line 34	Every signed 6-bit multiplicand from −32 to 31.
2	line 37	Any disagreement with Java's own multiplication is reported immediately.

Dry run: watch it execute

The slide example M = 7, Q = 3 in 4 bits.

Algorithm: Booth's multiplication. **Input:** m = `7`, q = `3`, bits = `4`.

Pseudocode

```
AC = 0; Q−1 = 0
repeat n times:
 if (Q0,Q−1) == 10: AC = AC − M
 if (Q0,Q−1) == 01: AC = AC + M
 arithmetic shift right (AC, Q, Q−1)
product = AC:Q
```

Step 1 of 6. M = 0111 (7), −M = 1001, Q = 0011 (3). AC = 0, Q_{-1} = 0.

cycle = 0 · Q0 Q-1 = — — · AC = 0000 · Q = 0011 · Q-1 = 0

Step 2 of 6. Cycle 1: Q_0Q_{-1} = 10 → AC = AC − M (AC becomes 1001), then arithmetic shift right.

cycle = 1 · Q0 Q-1 = 10 · AC = 1100 · Q = 1001 · Q-1 = 1

Step 3 of 6. Cycle 2: Q_0Q_{-1} = 11 → only shift, then arithmetic shift right.

cycle = 2 · Q0 Q-1 = 11 · AC = 1110 · Q = 0100 · Q-1 = 1

Step 4 of 6. Cycle 3: Q_0Q_{-1} = 01 → AC = AC + M (AC becomes 0101), then arithmetic shift right.

cycle = 3 · Q0 Q-1 = 01 · AC = 0010 · Q = 1010 · Q-1 = 0

Step 5 of 6. Cycle 4: Q_0Q_{-1} = 00 → only shift, then arithmetic shift right.

cycle = 4 · Q0 Q-1 = 00 · AC = 0001 · Q = 0101 · Q-1 = 0

Step 6 of 6. Product = AC:Q = 0001 0101 = 21.

cycle = 4 · Q0 Q-1 = · AC = 0001 · Q = 0101 · Q-1 = 0

> **RESULT**
> 7 × 3 = 21

Complexity

The loop runs once per multiplier bit, and each cycle does at most one n-bit addition and one shift → **$O(n^2)$ bit operations** for n-bit operands (n cycles × n-bit add), O(n) cycles. Space O(n).

Why bother? In hardware shifting is nearly free, so the cost is the number of add/subtract operations. Booth turns a run of k ones into 2 operations instead of k. For multipliers with long runs of 1s (like −1 = 1111…1) that is a huge saving; worst case (alternating 0101…) is still n operations.

Common mistakes

Logical shift instead of arithmetic shift

Problem

```
AC = AC >> 1; // on an n-bit pattern held in a larger int
```

Why: The top bit of the n-bit pattern is not the int's sign bit, so >> shifts in 0. Negative partial products become positive.

Fix: AC = (AC >> 1) | (AC & signBit); // re-insert the sign bit

Forgetting to discard the carry

Problem

```
AC = AC + M; // AC is int, bit n silently appears
```

Why: The addition can produce an (n+1)th bit. Hardware drops it; in software you must mask.

Fix: AC = (AC + M) & mask;

Negating the most negative number

Problem

```
booth(-8, 3, 4) // -(-8) is 8, which does not fit in 4-bit two's complement
```

Why: −8 = 1000 and its two's complement is again 1000. The subtraction adds the wrong value.

Fix: Use width + 1 bits when the multiplicand is −2^(n−1) (as BoothMultiply does).

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · n-bit two's complement — *Beginner*

Read an integer x and a width n (1 ≤ n ≤ 16, x fits in n bits). Print the n-bit two's complement bit pattern of x.

Example input	Expected output
−5 8	11111011

Hint 1: Walk from the top bit n−1 down to 0 and print (x >> bit) & 1. **Hint 2:** Java's >> on a negative int already yields the two's complement bits.

Exercise 2 · Booth's operations — *Intermediate*

Read an n-bit multiplier as a bit string. For each cycle (from the rightmost bit to the leftmost), print the operation Booth's algorithm performs: `-M`, `+M` or `0`. Separate with spaces.

Example input	Expected output
0110	0 −M 0 +M

Hint 1: Keep the previous bit (starting at 0). **Hint 2:** (1,0) → −M; (0,1) → +M; otherwise 0.

Exercise 3 · Booth multiplication — *Challenge*

Read two signed integers (each fits in 16-bit two's complement). Print their product, computed by simulating Booth's algorithm on registers (no `*`).

Example input	Expected output
−3 4	−12

Hint 1: Use width = 16 (17 if the multiplicand is −32768). **Hint 2:** Mask AC and M to the register width after every addition. **Hint 3:** Combine AC and Q at the end and sign-extend the 2n-bit value.

Quick check

Answers are in the answer key at the end of this part.

Q1. Booth's algorithm subtracts the multiplicand when (Q_0, Q_{-1}) is...

- **A.** 00
- **B.** 01
- **C.** 10
- **D.** 11

Q2. What is the 4-bit two's complement of 0111 (7)?

```
// flip: 1000, add 1
```

Your answer: _____

Q3. An arithmetic right shift of 1100 (4-bit) gives...

- **A.** 0110
- **B.** 1110
- **C.** 1100
- **D.** 0100

Q4. What is 0111 × 0011 after 4 Booth cycles in AC:Q?

- **A.** 00010101
- **B.** 00011001
- **C.** 00001111
- **D.** 00010011

Q5. Which multiplier makes Booth's algorithm most efficient?

- **A.** 01010101
- **B.** 11111111
- **C.** 10101010
- **D.** 00000001

Interview / exam questions

- Why does Booth's algorithm work for negative multipliers?
- What are the three registers and what does each do?
- Why is an extra bit needed for the most negative multiplicand?
- How does Booth compare to shift-and-add on an input with many consecutive 1s?

Watch & read

Video: Booth's Algorithm for Signed Multiplication — TutorialsPoint · 14:32. *Booth's signed multiplication*

Video: Booth's algorithm - Binary multiplication example | Computer Organization — Scholarly Things · 5:39. *A worked Booth example*

REVISION CARD (30–60 SECONDS)

- Pairs (Q0 Q-1): 10 → AC−M, 01 → AC+M, 00/11 → nothing; then arithmetic shift right.
- n cycles; product = AC:Q (2n bits).
- −M = ~M + 1.
- Run of 1s = 2^(k+1) − 2^j: one subtract, one add.
- Example: 7 × 3 = 0001 0101 = 21.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The Booth deck is entirely screenshots from a video lecture; it has no Java code. The program here is original and verified exhaustively.

ASSUMPTIONS MADE

- The slides use M = 0111, Q = 0011 (n = 4) and a second example with M = 01011, Q = 01110 (n = 5). Both are supported by the program.

5.7 Simple Sieve of Eratosthenes

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 40 min · ★ on the minimum-viable path

Find every prime up to n by crossing out multiples — far faster than testing each number.

YOU WILL BE ABLE TO

- Explain how crossing out multiples finds primes.
- Justify the two optimisations: start at p·p and stop at √n.
- Write the sieve with a boolean array and count / print primes.
- State O(n log log n) time and O(n) space and when a sieve is the right tool.

Before this lesson: One-Dimensional Arrays · Loops: for, while, do-while, break and continue · Divisibility, GCD/LCM, Primes and Modular Arithmetic

The idea

A **prime** is a number greater than 1 whose only divisors are 1 and itself. To list the primes up to 50 you could test each number — or you can do what Eratosthenes did 2 200 years ago: write the numbers down, and repeatedly cross out multiples.

1. Write 2, 3, 4, ..., n. Nothing is crossed out.
2. Take the first number that is not crossed out (2). It is prime. Cross out its multiples: 4, 6, 8, ...
3. Take the next surviving number (3). It is prime. Cross out its multiples 6, 9, 12, ...
4. Repeat. Every surviving number is prime.

```
n = 30 2 3 4 5 6 7 8 9 10 11 12 13 14 15 ...
p = 2 2 3 ✗ 5 ✗ 7 ✗ 9 ✗ 11 ✗ 13 ✗ 15 ...
p = 3 2 3 5 7 ✗ 11 13 ✗ ...
p = 5 2 3 5 7 11 13 ... (25 crossed; 5·5 = 25 ≤ 30)
```

WHY THIS WORKS

Why start at p·p? Any multiple p·k with k < p has a smaller prime factor than p (one of the factors of k), so it was already crossed out earlier. **Why stop at √n?** If a composite number m ≤ n had no prime factor ≤ √n, then m would be at least (√n)·(√n) > n — contradiction. So every composite is crossed out by a prime ≤ √n; primes larger than √n have nothing new to cross out.

Syntax

A `boolean[]` of size n+1 is indexed by the number itself: `isPrime[17]` answers "is 17 prime?". Java initialises boolean arrays to `false`, so the program either sets all to `true` first or flips the meaning to `composite[]`.

A tiny example

```
int n = 20;
boolean[] isPrime = new boolean[n + 1];
for (int i = 2; i <= n; i++) isPrime[i] = true;
for (int p = 2; p * p <= n; p++)
 if (isPrime[p])
 for (int m = p * p; m <= n; m += p)
 isPrime[m] = false;
```

Guided example

Compare with testing each number separately by trial division up to √k:

TrialDivision.java — Trial division (simple solution)

```
1  import java.util.Scanner;
2
3  public class TrialDivision {
4   // Test each number separately: n is prime if no d with 2 <= d <= sqrt(n) divides it.
5   static boolean isPrime(int n) {
6   if (n < 2) {                                                    1
7   return false;                                                   3
8   }
9   for (int d = 2; (long) d * d <= n; d++) {                       2
10  if (n % d == 0) {
11  return false;
12  }
13  }
14  return true;
15  }
16
17  public static void main(String[] args) {
18  Scanner sc = new Scanner(System.in);
19  int limit = sc.nextInt();
20  StringBuilder sb = new StringBuilder();
21  int count = 0;
22  for (int n = 2; n <= limit; n++) {
23  if (isPrime(n)) {
24  sb.append(n).append(' ');
25  count++;
26  }
27  }
28  System.out.println(sb.toString().trim());
29  System.out.println("count = " + count);
30  }
31 }
```

For every number k, test divisors up to √k. Total about n·√n operations.

Sample input

```
50
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	0 and 1 are not prime by definition.
2	line 9	Only divisors up to √n need testing; casting to long avoids overflow.
3	line 7	Found a divisor — n is composite. Return immediately.

SimpleSieve.java — Sieve of Eratosthenes (optimised solution)

```
1  import java.util.Scanner;
2
3  public class SimpleSieve {
4   // Sieve of Eratosthenes: cross out the multiples of every prime; whatever survives is
        prime.
5   static boolean[] sieve(int limit) {
6   boolean[] isPrime = new boolean[limit + 1];
7   for (int i = 2; i <= limit; i++) {
8   isPrime[i] = true; // assume prime until crossed out                                    (1)
9   }
10  for (int p = 2; (long) p * p <= limit; p++) { // primes bigger than sqrt(limit) have    (2)
        nothing left to cross out
11  if (isPrime[p]) {                                                                      (3)
12  for (int m = p * p; m <= limit; m += p) { // smaller multiples were crossed out by     (4)
        smaller primes
13  isPrime[m] = false;
14  }
15  }
16  }
17  return isPrime;
18  }
19
20  public static void main(String[] args) {
21  Scanner sc = new Scanner(System.in);
22  int limit = sc.nextInt();
23  boolean[] isPrime = sieve(limit);
24  StringBuilder sb = new StringBuilder();
25  int count = 0;
26  for (int n = 2; n <= limit; n++) {
27  if (isPrime[n]) {
28  sb.append(n).append(' ');
29  count++;
30  }
31  }
32  System.out.println(sb.toString().trim());
33  System.out.println("count = " + count);
34  }
35 }
```

Crosses out multiples of each surviving prime starting at p·p. O(n log log n).

Sample input

```
50
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	Optimistically mark every number prime; crossing out will correct this.
2	line 10	Only primes up to √limit need to cross out multiples.
3	line 11	p survived all smaller primes so p is prime. Composite p can be skipped because their multiples were already handled.
4	line 12	Smaller multiples were crossed out by smaller primes; stepping by p visits each multiple once.

Both print the primes up to 50: `2 3 5 7 11 13 17 19 23 29 31 37 41 43 47` (15 primes).

Dry run: watch it execute

Primes up to 30: watch each prime cross out its multiples.

Algorithm: Sieve of Eratosthenes. **Input:** n = `30`.

Pseudocode

```
isPrime[2..n] = true
for (p = 2; p * p <= n; p++)
 if (isPrime[p])
 for (m = p * p; m <= n; m += p)
 isPrime[m] = false
primes = all i with isPrime[i]
```

Step 1 of 6. Write down 2..n. Nothing is crossed out yet.

p = —

numbers (= crossed out)

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

20	21	22	23	24	25	26	27	28	29	30
20	21	22	23	24	25	26	27	28	29	30

Step 2 of 6. 2 is prime. Cross out 2·2 = 4, then every 2th number after it.

p = 2 · start p·p = 4

numbers (= crossed out)

2	3	✗	5	✗	7	✗	9	✗	11	✗	13	✗	15	✗	17	✗	19
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

✗	21	✗	23	✗	25	✗	27	✗	29	✗
20	21	22	23	24	25	26	27	28	29	30

Step 3 of 6. 3 is prime. Cross out 3·3 = 9, then every 3th number after it.

p = 3 · start p·p = 9

numbers (= crossed out)

2	3	✗	5	✗	7	✗	✗	✗	11	✗	13	✗	✗	✗	17	✗	19
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

✗	✗	✗	23	✗	25	✗	✗	✗	29	✗
20	21	22	23	24	25	26	27	28	29	30

Step 4 of 6. 4 is already crossed out, so it is not prime — skip it.

p = 4

numbers (= crossed out)

2	3	✗	5	✗	7	✗	✗	✗	11	✗	13	✗	✗	✗	17	✗	19
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

✗	✗	✗	23	✗	25	✗	✗	✗	29	✗
20	21	22	23	24	25	26	27	28	29	30

Step 5 of 6. 5 is prime. Cross out 5·5 = 25, then every 5th number after it.

p = 5 · start p·p = 25

numbers (= crossed out)

2	3	✗	5	✗	7	✗	✗	✗	11	✗	13	✗	✗	✗	17	✗	19
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

✗	✗	✗	23	✗	✗	✗	✗	✗	29	✗
20	21	22	23	24	25	26	27	28	29	30

Step 6 of 6. p·p exceeds 30, so we stop. Survivors are the primes.

count = 10

numbers (= crossed out)

2	3	✗	5	✗	7	✗	✗	✗	11	✗	13	✗	✗	✗	17	✗	19
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

✗	✗	✗	23	✗	✗	✗	✗	✗	29	✗
20	21	22	23	24	25	26	27	28	29	30

RESULT

10 primes: 2 3 5 7 11 13 17 19 23 29

Complexity

Prime p crosses out about n/p numbers. Summed over primes p ≤ √n that is n·(1/2 + 1/3 + 1/5 + ...) = n · log log n (a very slowly growing function: for n = 10^9 it is about 3).

```
time = O(n log log n) space = O(n) (one boolean per number)
```

n	Trial division ≈ n√n	Sieve ≈ n log log n
10^6	10^9	3×10^6
10^7	3×10^{10}	3×10^7

NOTE

Use a sieve when you need ALL primes up to n, or many primality queries. Use trial division (or Miller–Rabin) when you need to test a single large number. When n is too large for memory but the range is narrow, use the segmented sieve (next lesson).

Common mistakes

Index out of bounds

Problem

```
boolean[] isPrime = new boolean[limit];
isPrime[limit] = true;
```

Why: A boolean[limit] has indices 0..limit−1. To use the number `limit` itself the size must be limit + 1.

Fix: new boolean[limit + 1]

Starting at 2·p

Problem

```
for (int m = 2 * p; m <= limit; m += p)
```

Why: Correct, but does redundant work — those multiples were already crossed out. It does NOT break correctness, only speed (a common beginner version).

Fix: for (int m = p * p; ...)

Counting 0 and 1 as prime

Problem

```
for (int i = 0; i <= limit; i++) if (isPrime[i]) count++;
```

Why: With the "all true" initialisation, 0 and 1 stay true unless set to false.

Fix: Start both the initialisation and the counting loops at 2.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Count primes — *Beginner*

Read n and print how many primes are ≤ n.

- $1 \le n \le 10^7$

Example input	Expected output
100	25

Hint 1: Use a boolean array of size n + 1. **Hint 2:** Count while you sieve.

Exercise 2 · The k-th prime — *Intermediate*

Read k (1 ≤ k ≤ 100 000) and print the k-th prime (2 is the 1st).

Example input	Expected output
1000	7919

Hint 1: How large can the 100 000-th prime be? About 1.3 million. **Hint 2:** Sieve up to a fixed safe bound and count.

Exercise 3 · Prime factorisation queries — *Challenge*

Read q, then q numbers x ($2 \le x \le 10^6$). For each, print its prime factorisation as `x = p1^e1 * p2^e2` (exponent omitted when 1).

Example input	Expected output
3 ⏎ 12 ⏎ 97 ⏎ 360	12 = 2^2 * 3 ⏎ 97 = 97 ⏎ 360 = 2^3 * 3^2 * 5

Hint 1: Build an array spf[x] = smallest prime factor with a sieve. **Hint 2:** Repeatedly divide x by spf[x].

Quick check

Answers are in the answer key at the end of this part.

Q1. In the sieve, why can the outer loop stop at p·p ≤ n?

- **A.** Because larger primes are not prime
- **B.** Because every composite ≤ n has a prime factor ≤ √n
- **C.** Because arrays are limited to √n
- **D.** Because p·p is always prime

Q2. Which numbers does p = 3 cross out for n = 30 (starting at p·p)?

```
for (int m = 9; m <= 30; m += 3) System.out.print(m + " ");
```

Your answer: _____

Q3. Time complexity of the sieve of Eratosthenes?

- **A.** O(n)
- **B.** O(n log log n)
- **C.** O(n^2)
- **D.** O(n√n)

Q4. This sieve reports no primes at all. Which line is wrong?

```
1  boolean[] isPrime = new boolean[n + 1];
2  Arrays.fill(isPrime, true);
3  for (int p = 2; p * p <= n; p++)
4   for (int m = p; m <= n; m += p)
5   isPrime[m] = false;
```

Which line contains the bug? _____

Q5. How many primes are there below 50?

- **A.** 13
- **B.** 15
- **C.** 17
- **D.** 19

Interview / exam questions

- Why is the sieve O(n log log n)?
- How would you reduce the memory of a sieve?
- Difference between sieve and trial division — when to use which?
- How do you get the smallest prime factor of every number?

Watch & read

Video: Sieve of Eratosthenes | Journey into cryptography | Computer Science | Khan Academy — Khan Academy Labs · 4:12. *Crossing out multiples to find primes*

Video: L6. Sieve of Eratosthenes | Maths Playlist — take U forward · 18:27. *Sieve implementation in Java*

REVISION CARD (30–60 SECONDS)

- isPrime[0..n]=true (except 0,1)
- for p with p·p ≤ n and isPrime[p]: cross out p·p, p·p+p, ...
- Time O(n log log n), space O(n)
- Use for "all primes ≤ n" and many queries

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide walk-through for n = 20 is correct, but its last numbered step is cut off ("11. The rem..."); the final answer is 2 3 5 7 11 13 17 19.

5.8 Segmented & Incremental Sieve

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 55 min · complete path

Find primes inside a window [L, R] with R up to 10^{12}, or generate primes one at a time with no upper bound.

YOU WILL BE ABLE TO

- Explain why a simple sieve cannot handle R = 10^{12} and how segmenting fixes it.
- Implement the segmented sieve: base primes up to √R, then mark multiples inside [L, R].
- Handle the tricky parts: first multiple ≥ L, never marking p itself, and L < 2.
- Describe an incremental sieve that produces primes one at a time using a map of upcoming composites.

Before this lesson: Simple Sieve of Eratosthenes · Sets and Maps: HashSet, TreeSet, HashMap, TreeMap

The idea

The ordinary sieve needs an array of size n. To find the primes between 10^{12} and $10^{12} + 10^6$ it would need a trillion cells. But the interval itself is only a million numbers wide. The key fact: **a composite number in [L, R] has a prime factor ≤ √R ≤ 10^6**. So we only need the small primes up to √R to cross out everything in the window.

1. Run the simple sieve to get all **base primes** up to √R.
2. Create a small array for the window; cell i stands for the number L + i.
3. For each base prime p find its first multiple inside the window and cross out every p-th cell.
4. The cells never crossed out are the primes in [L, R].

```
window [10, 30], base primes ≤ √30 = 5 → 2, 3, 5

index 0 1 2 3 4 5 ... number = 10 + index
n 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
p=2 ✗ ✗ ✗ ✗ ✗ ✗ ✗ ✗ ✗ ✗
p=3 · ✗ ✗ ✗ ✗ ✗ ✗ ✗
p=5 ✗ ✗ ✗ (10, 15, 20, 25, 30)
primes: 11 13 17 19 23 29
```

WHY THIS WORKS

The first multiple of p that is ≥ L is ⌈L/p⌉·p. But we must also never start below p·p: if L ≤ p ≤ R, then p itself lies in the window and must not be crossed out. Starting at max(p·p, ⌈L/p⌉·p) gives both guarantees.

Incremental sieve

The incremental (a.k.a. unbounded or "rolling") sieve produces **the next prime on demand**, with no limit decided in advance. It keeps a dictionary mapping an upcoming composite number to the list of primes that will cross it out. When the counter reaches a number: if it is in the dictionary it is composite — move each of its primes to the next multiple; if it is not, it is prime and we schedule its square.

Syntax

New Java here: `HashMap<Long, List<Long>>` (a map from a number to a list of numbers), `computeIfAbsent(key, k -> new ArrayList<>())` (get the list, creating it if missing), and `map.remove(key)` which returns the removed value or `null`.

A tiny example

(low + p − 1) / p rounds low/p UP using integer division.

```
long low = 10, p = 3;
long start = Math.max(p * p, (low + p - 1) / p * p); // 12: first multiple of 3 that is >= 10
```

Guided example

SegmentedSieve.java — Segmented sieve (optimised solution)

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class SegmentedSieve {
 // Step 1: ordinary sieve that returns every prime up to `limit`.
 static List<Integer> simpleSieve(int limit) {
 boolean[] composite = new boolean[limit + 1];
 List<Integer> primes = new ArrayList<>();
 for (int i = 2; i <= limit; i++) {
 if (!composite[i]) {
 primes.add(i);
 for (long m = (long) i * i; m <= limit; m += i) {
 composite[(int) m] = true;
 }
 }
 }
 return primes;
 }

 // Step 2: sieve only the window [low, high] using the small primes up to sqrt(high).
 static List<Long> segmentedSieve(long low, long high) {
 int root = (int) Math.sqrt((double) high);                                    (1)
 while ((long) root * root > high) root--; // guard against floating-point rounding
 while ((long) (root + 1) * (root + 1) <= high) root++;
 List<Integer> basePrimes = simpleSieve(root);                                 (2)

 boolean[] composite = new boolean[(int) (high - low + 1)]; // index i <-> number low + i (3)
 for (int p : basePrimes) {
 // first multiple of p that is >= low, but never below p*p (p itself must stay prime)
 long start = Math.max((long) p * p, (low + p - 1) / p * p);                   (4)
 for (long m = start; m <= high; m += p) {
 composite[(int) (m - low)] = true;                                            (5)
 }
 }
 List<Long> primes = new ArrayList<>();
 for (long n = Math.max(low, 2); n <= high; n++) {                             (6)
 if (!composite[(int) (n - low)]) {
 primes.add(n);
 }
 }
 return primes;
```

(code continued)

```
    }

    public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    long low = sc.nextLong();
    long high = sc.nextLong();
    List<Long> primes = segmentedSieve(low, high);
    StringBuilder sb = new StringBuilder();
    for (long p : primes) {
    sb.append(p).append(' ');
    }
    System.out.println(sb.toString().trim());
    System.out.println("count = " + primes.size());
    }
   }
```

Simple sieve up to √high, then marks multiples inside the window. Memory O(√high + (high − low)).

Sample input

```
10 30
```

Line-by-line walkthrough

#	Where	What happens
1	line 23	We need base primes up to √high. The two while loops correct floating-point rounding.
2	line 26	The ordinary sieve from the previous lesson, but returning the primes.
3	line 28	Only the window is stored. Index i represents the number low + i.
4	line 31	First multiple ≥ low, but not below p·p so that p itself (if in the window) survives.
5	line 33	Translate the number m into its array index by subtracting low.
6	line 37	Numbers below 2 are not prime, so a window starting at 0 or 1 must skip them.

IncrementalSieve.java — Incremental sieve (assumed from the slide description) (demonstration)

```java
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class IncrementalSieve {
 // An incremental (unbounded) sieve: no upper limit is needed in advance.
 // `composites` remembers, for each upcoming composite number, which primes will cross it out.
 private final Map<Long, List<Long>> composites = new HashMap<>();
 private long candidate = 1;

 long nextPrime() {
 while (true) {
 candidate++;
 List<Long> crossers = composites.remove(candidate);                    (1)
 if (crossers == null) {
 // nobody has crossed candidate out, so it is prime; schedule its first multiple p*p
 composites.computeIfAbsent(candidate * candidate, k -> new ArrayList<>()).add(candidate);  (2)
 return candidate;
 }
 for (long p : crossers) { // candidate is composite: move each p on to its next multiple
 composites.computeIfAbsent(candidate + p, k -> new ArrayList<>()).add(p);  (3)
 }
 }
 }

 public static void main(String[] args) {
 IncrementalSieve s = new IncrementalSieve();
 StringBuilder first = new StringBuilder();
 for (int i = 0; i < 15; i++) {
 first.append(s.nextPrime()).append(' ');
 }
 System.out.println("first 15 primes: " + first.toString().trim());

 // "start from N": keep generating and skip everything below N
 long N = 100;
 IncrementalSieve t = new IncrementalSieve();
 long p = t.nextPrime();
 while (p < N) {
 p = t.nextPrime();
 }
 StringBuilder from = new StringBuilder();
 for (int i = 0; i < 5; i++) {
 from.append(p).append(' ');
 p = t.nextPrime();
 }
 System.out.println("first 5 primes >= " + N + ": " + from.toString().trim());
 }
}
```

An unbounded generator of primes using a map of upcoming composites. The slide only describes this algorithm in words, so the implementation is inferred.

Line-by-line walkthrough

#	Where	What happens
1	line 15	If someone scheduled this number, it is composite and the list holds its prime factors seen so far.
2	line 18	A new prime p can first cross out p·p (smaller multiples have smaller factors).
3	line 22	Each prime that hit a composite moves on to its next multiple.

Dry run: watch it execute

Window [10, 30]: base primes 2, 3, 5 cross out cells; survivors are prime.

Algorithm: Segmented sieve [L, R]. **Input:** low = `10`, high = `30`.

Pseudocode

```
base = primes up to sqrt(R)
window[i] stands for L + i
for each base prime p:
 start = max(p*p, ceil(L/p)*p)
 cross out start, start+p, ... <= R
survivors (>= 2) are prime
```

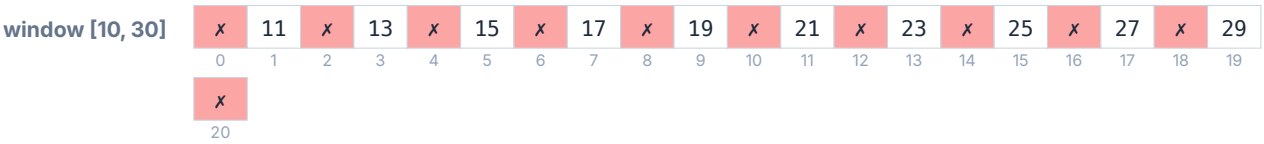
Step 1 of 5. √30 ≈ 5, so the base primes are 2, 3, 5.

base primes = 2 3 5



Step 2 of 5. p = 2: first multiple ≥ 10 is 10; starting at max(4, that) = 10.

p = 2 · start = 10



Step 3 of 5. p = 3: first multiple ≥ 10 is 12; starting at max(9, that) = 12.

p = 3 · start = 12



Step 4 of 5. p = 5: first multiple ≥ 10 is 10; starting at max(25, that) = 25.

p = 5 · start = 25



Step 5 of 5. Cells never crossed out (and ≥ 2) are prime.

count = 6



RESULT
6 primes: 11 13 17 19 23 29

Complexity

Segmented: base primes cost O(√R log log R). Marking the window costs about (R − L) · log log R. So time ≈ O((R − L) log log R + √R) and space O(√R + (R − L)). For R = 10^{12}, R − L = 10^6 this is a few million operations.

Incremental: each new prime p inserts one map entry; each composite c causes one map move per distinct prime factor. Memory is proportional to the number of primes found up to √(current) — the map keeps only "next composite" per prime.

Common mistakes

Marking the prime itself

Problem

```
for (long m = (low / p) * p; m <= high; m += p) composite[...] = true; // slide style
```

Why: If low ≤ p (for example low = 2), this crosses out p itself, so 2 is reported as composite.
Fix: long start = Math.max((long) p * p, (low + p - 1) / p * p);

Using int for the window bounds

Problem

```
int low = 1_000_000_000_000;
```

Why: 10^{12} is far above Integer.MAX_VALUE (2.1·10^9): it does not compile; computations on int bounds overflow silently.
Fix: long low, high; // but keep the ARRAY index (high − low) as an int

Forgetting to initialise (slide)

Problem

```
boolean prime[] = new boolean[h + 1]; // all false, then marks them false again
```

Why: The slide's array starts as all false and the commented-out Arrays.fill is never run, so the printed result is empty. It also writes `False` (capital F), which does not compile.
Fix: Use a composite[] array: false means "still possibly prime", so no fill is needed.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Primes in a small range — *Beginner*

Read L and R (1 ≤ L ≤ R ≤ 10^6). Print how many primes are in [L, R].

Example input	Expected output
10 30	6

Hint 1: A plain sieve up to R is fine here. **Hint 2:** Count only values ≥ max(L, 2).

Exercise 2 · Primes in a huge range — *Intermediate*

Read L and R with R ≤ 10^{12} and R − L ≤ 10^{6}. Print how many primes lie in [L, R].

Example input	Expected output
1000000000000 1000000000100	4

Hint 1: The simple sieve cannot allocate 10^{12} cells. **Hint 2:** Base primes up to 10^{6}, then window marking.

Exercise 3 · Largest prime gap in a window — *Challenge*

Read L and R (R − L ≤ 10^{6}, R ≤ 10^{12}). Print the largest difference between two consecutive primes in [L, R], or -1 if there are fewer than two primes.

Example input	Expected output
1 100	8

Hint 1: Sieve the window, then scan keeping the previous prime.

Quick check

Answers are in the answer key at the end of this part.

Q1. To sieve [L, R] you need base primes up to...

- **A.** R
- **B.** R/2
- **C.** √R
- **D.** L

Q2. What is the first multiple of 7 that is ≥ 100 and ≥ 49?

```
long low = 100, p = 7;
System.out.println(Math.max(p * p, (low + p - 1) / p * p));
```

Your answer: _____

Q3. Why start marking at max(p·p, first multiple ≥ L)?

- **A.** To speed up only
- **B.** To avoid marking p itself and values below p·p
- **C.** Because arrays start at p
- **D.** It makes memory smaller

Q4. Which statement describes an incremental sieve?

- **A.** Needs the limit up front
- **B.** Produces primes one after another, remembering upcoming composites
- **C.** Only works on [L, R]
- **D.** Uses recursion on array halves

Interview / exam questions

- How do you find primes in [10^{12}, 10^{12} + 10^{6}]?
- Why is √R enough for base primes?
- What is the memory of a segmented sieve?
- How does an incremental sieve avoid fixing the upper bound?

Watch & read

Video: Segmented Sieve | Sample Video II for Essential Maths for CP | GeeksforGeeks — GeeksforGeeks · 11:42. *Sieving a range [L, R] with base primes*

Video: 15.1 Sieve of Eratosthenes | Challenge | C++ Placement Course — Apna College · 10:47. *Segmented sieve implementation (C++ — the idea transfers to Java)*

REVISION CARD (30–60 SECONDS)

- Base primes ≤ √R via simple sieve.
- window[i] ↔ L + i
- start = max(p·p, ⌈L/p⌉·p)
- skip numbers < 2
- Incremental: map composite → primes crossing it

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide's SegSieve does not compile (`False`, a commented Arrays.fill) and, if repaired naively, marks small primes in the window as composite and ignores the p·p optimisation. • The slide's array has size h + 1 (memory proportional to R), which defeats the purpose of segmenting.

ASSUMPTIONS MADE

• The slide describes the incremental sieve only in words ("check N + i using primes up to L"). The lesson implements the standard incremental (dictionary-based) sieve and labels it as such.

5.9 Binary Palindrome

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 35 min · ★ on the minimum-viable path

Decide whether the binary form of a number reads the same forwards and backwards, using strings and then pure bit operations.

YOU WILL BE ABLE TO

- Write a number in binary and decide by hand whether it is a palindrome.
- Explain what `x & 1`, `x >> 1`, `r << 1` and `r | bit` do and combine them to reverse bits.
- Implement both a String-based and a bit-based palindrome check and compare them.
- Understand why leading zeros are NOT part of the binary representation.

Before this lesson: Number Systems, Binary and Two's Complement · Bitwise Operators and Shifts · Loops: for, while, do-while, break and continue

The idea

A palindrome reads the same in both directions (`racecar`, `12321`). A **binary palindrome** is a number whose binary digits form a palindrome. 17 is `10001` — reversed it is still `10001`, so it is a binary palindrome. 16 is `10000`; reversed it is `00001`, which is not the same, so 16 is not.

n	binary	reversed	palindrome?
5	101	101	yes
6	110	011 → 11 ≠ 110	no
9	1001	1001	yes
10	1010	0101	no
51	110011	110011	yes

WHY THIS WORKS

Binary has **no leading zeros**: 6 is written `110`, never `0110`. So a number whose binary form ends in zeros can never be a palindrome (its reverse would start with zeros that vanish). This is why the bit-reversal trick below can compare plain integers: if the reversed bits give the same integer, they gave the same bit string.

To reverse the bits of x with arithmetic, peel bits off the right end of x (`x & 1` reads the lowest bit; `x >> 1` removes it) and push them onto the right end of a new number (`r << 1` makes room, `r | bit` adds the bit).

Syntax

expression	meaning	example with x = 13 (1101)
x & 1	lowest bit	1
x >> 1	drop lowest bit (÷ 2)	110 = 6
r << 1	make room (× 2)	1 → 10
r | b	set the lowest bit to b	10 | 1 = 11

A tiny example

```
int x = 5; // 101
System.out.println(Integer.toBinaryString(x)); // "101"
```

Guided example

BinaryPalindromeString.java — Binary palindrome with a String (simple solution)

```
1  import java.util.Scanner;
2
3  public class BinaryPalindromeString {
4   // Convert to a binary String and compare characters from both ends.
5   static boolean isBinaryPalindrome(int x) {
6   String bits = Integer.toBinaryString(x); // no leading zeros, e.g. 17 -> "10001"   (1)
7   int left = 0;                                                                      (2)
8   int right = bits.length() - 1;
9   while (left < right) {
10  if (bits.charAt(left) != bits.charAt(right)) {                                     (3)
11  return false;
12  }
13  left++;                                                                            (4)
14  right--;
15  }
16  return true;
17  }
18
19  public static void main(String[] args) {
20  Scanner sc = new Scanner(System.in);
21  int x = sc.nextInt();
22  System.out.println(x + " (" + Integer.toBinaryString(x) + ") " + (isBinaryPalindrome(x)
        ? "is" : "is not") + " a binary palindrome");
23  }
24 }
```

Uses Integer.toBinaryString and two pointers moving inwards.

Sample input

```
17
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	Gives the bits without leading zeros, e.g. 17 → "10001".
2	line 7	Two pointers start at the two ends of the string.
3	line 10	One mismatch is enough to answer "no".
4	line 13	Move both pointers one step towards the middle.

BinaryPalindromeBits.java — Binary palindrome with bit operations (optimised solution)

```
1  import java.util.Scanner;
2
3  public class BinaryPalindromeBits {
4   // Build the bit-reversal of x with shifts and compare with x. O(log x) time, O(1)
        space, no String.
5   static boolean isBinaryPalindrome(int x) {
6   int reversed = 0;                                                              (1)
7   int rest = x;
8   while (rest > 0) {                                                             (2)
9   reversed <<= 1; // make room on the right                                      (3)
10  reversed |= (rest & 1); // copy the lowest bit of rest into the new lowest bit (4)
11  rest >>= 1; // drop the bit we just used                                       (5)
12  }
13  return reversed == x;                                                          (6)
14  }
15
16  public static void main(String[] args) {
17  Scanner sc = new Scanner(System.in);
18  int x = sc.nextInt();
19  System.out.println(x + (isBinaryPalindrome(x) ? " has a binary palindrome
        representation." : " does not have a binary palindrome representation."));
20  }
21 }
```

Builds the bit-reversed number and compares it with the original. No String, no array.

Sample input

```
51
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	The number being built, starting empty.
2	line 8	Each iteration consumes one bit of the copy `rest`; the loop ends when no bits remain.
3	line 9	Shift left: make room for the next bit on the right.
4	line 10	`rest & 1` is the lowest bit of rest; OR-ing puts it into the freed spot.
5	line 11	Remove the bit we just copied.
6	line 13	Both numbers have the same number of bits, so they are equal exactly when the bit strings are mirror images.

Dry run: watch it execute

Peel bits off x one at a time and push them onto reversed.

Algorithm: Binary palindrome (bit reversal). **Input:** n = `51`.

Pseudocode

```
reversed = 0; rest = x;
while (rest > 0) {
 reversed <<= 1;
 reversed |= (rest & 1);
 rest >>= 1;
}
return reversed == x;
```

Step 1 of 8. x = 51 = 110011 in binary.

x = 51 · rest = 51 · reversed = 0 · rest bin = 110011 · reversed bin = 0

Step 2 of 8. Lowest bit of rest was 1: shift reversed left and OR it in; then shift rest right.

x = 51 · rest = 25 · reversed = 1 · rest bin = 11001 · reversed bin = 1

Step 3 of 8. Lowest bit of rest was 1: shift reversed left and OR it in; then shift rest right.

x = 51 · rest = 12 · reversed = 3 · rest bin = 1100 · reversed bin = 11

Step 4 of 8. Lowest bit of rest was 0: shift reversed left and OR it in; then shift rest right.

x = 51 · rest = 6 · reversed = 6 · rest bin = 110 · reversed bin = 110

Step 5 of 8. Lowest bit of rest was 0: shift reversed left and OR it in; then shift rest right.

x = 51 · rest = 3 · reversed = 12 · rest bin = 11 · reversed bin = 1100

Step 6 of 8. Lowest bit of rest was 1: shift reversed left and OR it in; then shift rest right.

x = 51 · rest = 1 · reversed = 25 · rest bin = 1 · reversed bin = 11001

Step 7 of 8. Lowest bit of rest was 1: shift reversed left and OR it in; then shift rest right.

x = 51 · rest = 0 · reversed = 51 · rest bin = 0 · reversed bin = 110011

Step 8 of 8. reversed (51) equals x (51): a binary palindrome.

x = 51 · reversed = 51 · palindrome = true

> **RESULT**
> 51 (110011) is a binary palindrome

Complexity

A number x has $\lfloor \log_2 x \rfloor + 1$ bits, and each loop iteration removes one: **O(log x) time, O(1) space** for the bit method. The String method is also O(log x) time but allocates a String of that length (O(log x) space).

Why log? Doubling the number adds only one more bit, so 10^9 has just 30 bits — 30 iterations.

Common mistakes

Forgetting that zero bits at the end matter

Problem

```
// "10" is 2 in binary; someone treats 010 as a palindrome
```

Why: Leading zeros are not stored. 2 = `10` which reversed is `01` = 1 ≠ 2, so 2 is not a palindrome.

Fix: Compare the integer reversal with the original (the lesson code) — it handles this automatically.

Reversing the same variable you are reading

Problem

```
while (x > 0) {
 reversed = (reversed << 1) | (x & 1);
 x >>= 1;
}
return reversed == x; // x is 0 now!
```

Why: The loop destroys x. After the loop x == 0, so the comparison is almost always false.

Fix: int rest = x; // work on a copy; compare with the untouched x

Negative numbers

Problem

```
isBinaryPalindrome(-5)
```

Why: Java stores negatives in two's complement (32 bits starting with 1...). The loop `while (x > 0)` never runs and returns `0 == -5` → false, which is a reasonable answer but a different question.

Fix: Decide the problem's rule (usually: n ≥ 0) and document it.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Is it a binary palindrome? — *Beginner*

Read a non-negative integer n. Print YES if its binary representation is a palindrome, otherwise NO.

- $0 \le n \le 2^{31} - 1$

Example input	Expected output
5	YES
10	NO

Hint 1: Compare the number with its bit reversal. **Hint 2:** Remember 0 is written "0" — a palindrome.

Exercise 2 · Count binary palindromes up to n — *Intermediate*

Read n. Print how many integers from 1 to n have a palindromic binary representation.

- $1 \le n \le 10^6$

Example input	Expected output
10	5

Hint 1: Loop and reuse your checker. **Hint 2:** Check your answer by listing them for n = 10.

Exercise 3 · Next binary palindrome — *Challenge*

Read n. Print the smallest binary palindrome strictly greater than n.

- $0 \le n \le 10^9$

Example input	Expected output
7	9

Hint 1: Palindromes are fairly dense: the gap is never larger than about 2^(bits/2). **Hint 2:** Start at n + 1 and keep going.

Quick check

Answers are in the answer key at the end of this part.

Q1. Which of these numbers is a binary palindrome?

- **A.** 6 (110)
- **B.** 9 (1001)
- **C.** 12 (1100)
- **D.** 14 (1110)

Q2. What does this print?

```
int x = 6, r = 0;
while (x > 0) {
 r = (r << 1) | (x & 1);
 x >>= 1;
}
System.out.println(r);
```

Your answer: _____

Q3. What does `r << 1` do to the number r?

- **A.** divides by 2
- **B.** multiplies by 2
- **C.** adds 1
- **D.** flips all bits

Q4. Fill in the bit that is copied each step.

```
reversed = (reversed << 1) | (rest ______ 1);
rest = rest >> 1;
```

Q5. Time complexity of the bit-reversal palindrome test?

- **A.** O(1)
- **B.** O(log n)
- **C.** O(n)
- **D.** O(n log n)

Interview / exam questions

- How would you check a binary palindrome without converting to a String?
- Why can't a number that is a multiple of 2 be a binary palindrome (other than 0)?
- How would you reverse the 32 bits of an int?

Watch & read

Video: C++ Challenge: Check If a Number's Binary Form Is a Palindrome — Atshk.h programing · 6:09. *Testing a binary palindrome with bit operations (C++ — the idea transfers to Java)*

Video: Reverse Bits - Binary - Leetcode 190 - Python — NeetCode · 10:13. *Reversing the bits of an integer (Python — the idea transfers to Java)*

REVISION CARD (30–60 SECONDS)

- x & 1 = lowest bit; x >> 1 drops it; r << 1 | bit pushes it.
- Reverse the bits, compare with the original.
- O(log n) time, O(1) space.
- Even numbers (> 0) are never binary palindromes: they end in 0 but start with 1.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slides list 16 (10000) as a non-palindrome and 17 (10001) as a palindrome — both correct. The explanation text calls the reversed value "00001"; mathematically the reversed integer is 1.

5.10 Swap Two Nibbles in a Byte

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 30 min · complete path

Exchange the high and low 4-bit halves of a byte using masks and shifts.

YOU WILL BE ABLE TO

- Define a nibble and split a byte into its two nibbles with masks 0xF0 and 0x0F.
- Combine shifts and OR to rebuild the byte with nibbles swapped.
- Explain why Java's signed `byte` needs `& 0xFF` before shifting.
- Generalise: swap adjacent bits and reverse all 32 bits.

Before this lesson: Bitwise Operators and Shifts · Number Systems, Binary and Two's Complement · Type Conversion, Casting and Overflow

The idea

A **bit** is 0 or 1. A **nibble** is 4 bits (half a byte; it holds exactly one hexadecimal digit, 0–F). A **byte** is 8 bits = two nibbles. In `1010 1100` (0xAC) the high nibble is `1010` (A) and the low nibble is `1100` (C). Swapping the nibbles gives `1100 1010` (0xCA).

```
1010 1100 0xAC
& 0xF0 → 1010 0000 (keep high nibble) >>> 4 → 0000 1010
& 0x0F → 0000 1100 (keep low nibble) << 4 → 1100 0000
OR → 1100 1010 0xCA
```

WHY THIS WORKS

A mask is a pattern of 1s over the bits you want to keep: `x & mask` zeroes everything else. Shifting by 4 slides a nibble across the byte boundary: multiplying by 16 (`<< 4`) moves the low nibble up, dividing by 16 (`>>> 4`) moves the high nibble down. OR glues the two moved pieces back together because they occupy different bit positions.

Syntax

Hexadecimal literals start with `0x`: `0xF0` = 240 = `1111 0000`, `0x0F` = 15 = `0000 1111`.

BYTE IS SIGNED

Java's `byte` is **signed** (−128..127). When a byte is used in an expression it is converted to `int` with sign extension: `(byte) 0xAB` becomes `0xFFFFFFAB`. Always mask with `& 0xFF` to get the unsigned 0–255 value first.

A tiny example

```java
int b = 0xAC;
int swapped = ((b & 0xF0) >>> 4) | ((b & 0x0F) << 4);
System.out.println(Integer.toHexString(swapped)); // ca
```

Guided example

NibbleSwap.java — Swap nibbles (int based) (optimised solution)

```
1  import java.util.Scanner;
2
3  public class NibbleSwap {
4   // A byte has 8 bits = two 4-bit nibbles. Swap high and low nibble.
5   static int swapNibbles(int value) {
6   int b = value & 0xFF; // work on exactly one byte (0..255)                    (1)
7   int upper = (b & 0xF0) >>> 4; // keep the high 4 bits and move them to the right  (2)
8   int lower = (b & 0x0F) << 4; // keep the low 4 bits and move them to the left    (3)
9   return upper | lower; // glue them back together                              (4)
10  }
11
12  static String bits(int v) {
13  String s = Integer.toBinaryString(v & 0xFF);                                  (5)
14  return "0".repeat(8 - s.length()) + s;
15  }
16
17  public static void main(String[] args) {
18  Scanner sc = new Scanner(System.in);
19  int value = sc.nextInt();
20  int swapped = swapNibbles(value);
21  System.out.println("original: " + value + " = " + bits(value));
22  System.out.println("swapped : " + swapped + " = " + bits(swapped));
23  }
24 }
```

Works on an int holding 0..255, which avoids all the signed-byte surprises.

Sample input

```
172
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	Keep exactly 8 bits so the value is 0–255 regardless of the input's sign.
2	line 7	Isolate the high nibble and slide it to the low position.
3	line 8	Isolate the low nibble and slide it to the high position.
4	line 9	OR combines the two non-overlapping pieces.
5	line 13	Pad with leading zeros so all 8 bits are shown.

NibbleSwapByte.java — Swap nibbles with the byte type (lecture version) (from the slides)

```
1  public class NibbleSwapByte {
2   // The lecture version, using the Java type byte (signed!). Casting back to byte needs
        care.
3   public static byte swapNibbles(byte b) {
4   byte upperNibble = (byte) ((b & 0xF0) >>> 4); // b is promoted to int; & 0xF0 removes
        sign-extension bits                                                              (1)
5   byte lowerNibble = (byte) (b & 0x0F);
6   return (byte) ((lowerNibble << 4) | upperNibble);
7   }
8
9   public static void main(String[] args) {
10  byte byteValue = (byte) 0xAB; // 1010 1011
11  byte swappedByte = swapNibbles(byteValue);
12  System.out.println("Original byte: " + Integer.toBinaryString(byteValue & 0xFF));   (2)
13  System.out.println("Swapped byte: " + Integer.toBinaryString(swappedByte & 0xFF));
14  System.out.println("as hex: " + Integer.toHexString(byteValue & 0xFF) + " -> " +
        Integer.toHexString(swappedByte & 0xFF));
15  }
16 }
```

Uses (byte) casts. Correct, but see the notes about sign extension.

Line-by-line walkthrough

#	Where	What happens
1	line 4	b is promoted to int; the mask clears the sign-extension bits.
2	line 12	Mask when printing, otherwise a negative byte prints as a 32-bit pattern with many leading 1s.

Dry run: watch it execute

172 = 1010 1100 → 1100 1010 = 202.

Algorithm: Swap nibbles. **Input:** n = `172`.

Pseudocode

```
b = value & 0xFF;
upper = (b & 0xF0) >>> 4;
lower = (b & 0x0F) << 4;
return upper | lower;
```

Step 1 of 4. b = 172 = 10101100

b = 172 · bits = 10101100

Step 2 of 4. b & 0xF0 keeps the high nibble 10100000; >>> 4 moves it to 00001010.

b = 172 · upper = 10 · upper bits = 00001010

Step 3 of 4. b & 0x0F keeps the low nibble 00001100; << 4 moves it to 11000000.

upper = 10 · lower = 192 · lower bits = 11000000

Step 4 of 4. upper | lower = 11001010 = 202.

result = 202 · bits = 11001010

> **RESULT**
> 10101100 → 11001010 (172 → 202)

Complexity

A fixed number of operations on a fixed-size value: **O(1) time, O(1) space**. (For an n-bit word the same pattern still takes O(1) word operations.)

Common mistakes

Using >> instead of >>> on a negative byte

Problem

```
byte b = (byte) 0xAB;
int high = (b & 0xF0) >> 4;
```

Why: Here the mask already removes the sign, so it happens to work — but without the mask `b >> 4` copies the sign bit and gives 0xFFFFFFFA. Always mask first.

Fix: int high = (b & 0xF0) >>> 4;

Printing a byte directly

Problem

```
System.out.println(Integer.toBinaryString(b)); // b = (byte) 0xAB
```

Why: b is sign-extended to 32 bits: prints 11111111111111111111111110101011.

Fix: System.out.println(Integer.toBinaryString(b & 0xFF));

Forgetting that << can overflow a byte

Problem

```
byte low = (byte) (b & 0x0F);
byte r = low << 4; // compile error: int to byte
```

Why: Arithmetic on bytes yields int; assigning back needs a cast.

Fix: byte r = (byte) (low << 4);

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Swap the nibbles — *Beginner*

Read an integer 0 ≤ x ≤ 255 and print the value obtained by swapping its two 4-bit halves.

Example input	Expected output
171	186

Hint 1: High nibble: (x & 0xF0) >>> 4 **Hint 2:** Low nibble: (x & 0x0F) << 4

Exercise 2 · Swap adjacent bits — *Intermediate*

Read an unsigned 32-bit integer and swap bit 0 with bit 1, bit 2 with bit 3, and so on. Print the result as an unsigned number.

Example input	Expected output
23	43

Hint 1: Even-position bits: x & 0x55555555; odd-position bits: x & 0xAAAAAAAA. **Hint 2:** Shift evens left by 1 and odds right by 1 (use >>>). **Hint 3:** Print `result & 0xFFFFFFFFL` to see it unsigned.

Exercise 3 · Reverse 32 bits — *Challenge*

Read an unsigned 32-bit integer and print the integer whose binary form is the original's 32-bit pattern reversed.

Example input	Expected output
43261596	964176192

Hint 1: Loop 32 times: shift result left, OR in the lowest bit of x, shift x right with >>>. **Hint 2:** Print as unsigned (long).

Quick check

Answers are in the answer key at the end of this part.

Q1. How many bits are in a nibble?

- **A.** 2
- **B.** 4
- **C.** 8
- **D.** 16

Q2. What does this print (decimal)?

```
int b = 0x3C;
System.out.println(((b & 0xF0) >>> 4) | ((b & 0x0F) << 4));
```

Your answer: _____

Q3. Which mask keeps only the high nibble of a byte?

- **A.** 0x0F
- **B.** 0xF0
- **C.** 0xFF
- **D.** 0x80

Q4. Why write `b & 0xFF` for a Java byte b?

- **A.** To make it faster
- **B.** To remove sign extension and get 0–255
- **C.** To convert to hexadecimal
- **D.** To make it a char

Interview / exam questions

- How would you swap the two halves of a 32-bit int?
- What is the difference between >> and >>>?
- How many hex digits does a byte have?

Watch & read

Video: Swap two nibbles in a byte | GFG POTD | Bit Manipulation | CPP | AlgoBazaar — AlgoBazaar · 9:06. *Swapping the two nibbles of a byte (C++ — the idea transfers to Java)*

Video: Bitwise Operators in C (Part 2) — Neso Academy · 4:59. *Masks and shifts (C — the idea transfers to Java)*

REVISION CARD (30–60 SECONDS)

- byte = 8 bits = 2 nibbles; one nibble = one hex digit.
- swap = ((b & 0xF0) >>> 4) | ((b & 0x0F) << 4)
- Mask with & 0xFF before shifting a Java byte.
- O(1).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide's English steps are written as Python (`swap_nibbles(byte)`, `bin()`) but the program on the next slide is Java — the Java is what this lesson explains. In the example, the slide moves from `1010 1100` (0xAC) to `1100 1010` (0xCA), while its code demo uses 0xAB → 0xBA; both are consistent.

5.11 Longest Sequence of 1s (and after flipping a bit)

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 45 min · complete path

Find the longest run of consecutive 1 bits, then the longest run you can create by flipping a single 0.

YOU WILL BE ABLE TO

- Count the longest run of 1s with a scan using `count` and `maxCount`.
- Handle the "flip one 0" variant by remembering the run before the last 0.
- Do it on 32-bit ints without strings and handle negative numbers and the all-ones case.
- Verify a bit trick against brute force.

Before this lesson: Binary Palindrome · Bitwise Operators and Shifts · Strings, StringBuilder and Characters

The idea

Look at a binary number as a string of 0s and 1s and find the longest block of consecutive 1s. `1101110111111001` has runs 11, 111, 111111, 1 — the longest is **6**.

```
1 1 0 1 1 1 0 1 1 1 1 1 1 0 0 1
└┘ └──┘ └─────────┘ └ 1
 2 3 6
```

Now you may flip exactly one 0 to a 1. Flipping the 0 between `111` and `111111` joins both runs: 3 + 1 + 6 = **10**. In general a flipped 0 can join the run on its left, the run on its right, and itself.

WHY THIS WORKS

Scan the bits once, always remembering two things: the length of the run of 1s you are in now (`current`) and the length of the run just before the last 0 (`previous`). When you stand on a 0, flipping it would give `previous + 1 + current` — but only if that 0 is a single 0 (if two zeros in a row precede, `previous` is not adjacent). So one pass with two counters is enough.

Syntax

call / idiom	result
`Integer.toBinaryString(x)`	the bits without leading zeros
`Integer.bitCount(x)`	number of 1 bits
`Integer.highestOneBit(x)`	only the top set bit
`Integer.numberOfTrailingZeros(x)`	index of the lowest set bit
`x & 1`	the lowest bit
`x >>= 1` / `x >>> 1`	drop the lowest bit (arithmetic / logical)
`x & (x - 1)`	clear the lowest set bit
`x | (x << 1)`	every run of 1s grows by one position

A tiny example

```
int current = 0, best = 0;
for (int i = 0; i < 32; i++) {
 if ((x & 1) == 1) { current++; best = Math.max(best, current); }
 else current = 0;
 x >>>= 1;
}
```

Guided example

LongestOnes.java — Longest run of 1s (simple solution)

```
1  import java.util.Scanner;
2
3  public class LongestOnes {
4   // Longest run of consecutive 1 bits, scanning the 32 bits with shifts (no String
        needed).
5   static int longestRun(int x) {
6   int current = 0;                                                          4
7   int best = 0;
8   for (int i = 0; i < 32; i++) {                                            1
9   if ((x & 1) == 1) {                                                       2
10  current++;
11  best = Math.max(best, current);                                           3
12  } else {
13  current = 0; // a 0 ends the run
14  }
15  x >>>= 1; // unsigned shift so negatives are handled                      5
16  }
17  return best;
18  }
19
20  public static void main(String[] args) {
21  Scanner sc = new Scanner(System.in);
22  int x = sc.nextInt();
23  System.out.println(Integer.toBinaryString(x) + " -> longest run of 1s = " +
        longestRun(x));
24  }
25 }
```

Examines all 32 bits with shifts; no String needed, negative numbers work.

Sample input

```
56825
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	An int has exactly 32 bits, so 32 iterations cover every bit including leading zeros.
2	line 9	The lowest bit is 1: the current run grows.
3	line 11	Keep the best run seen so far.
4	line 6	A 0 ends the run.
5	line 15	Unsigned shift so the sign bit does not repeat for negatives.

FlipOneBit.java — Longest run after flipping one bit (optimised solution)

```
1  import java.util.Scanner;
2
3  public class FlipOneBit {
4   // Longest run of 1s you can get by flipping at most one 0 to 1 (32-bit int; negatives
        handled).
5   // Track the length of the current run and the run before the last 0.
6   static int longestAfterFlip(int x) {
7   if (x == -1) {                                                                    (1)
8   return 32; // already all ones: nothing to flip
9   }
10  int current = 0; // run of 1s ending at this bit
11  int previous = 0; // run of 1s just before the most recent 0
12  int best = 1; // flipping any 0 gives at least 1
13  for (int i = 0; i < 32; i++) {
14  if ((x & 1) == 1) {
15  current++;
16  } else {
17  previous = ((x & 2) == 0) ? 0 : current; // two zeros in a row: the flipped 0 cannot   (2)
        join two runs
18  current = 0;
19  }
20  best = Math.max(best, previous + current + 1);                                    (3)
21  x >>>= 1;
22  }
23  return Math.min(best, 32);                                                        (4)
24  }
25
26  public static void main(String[] args) {
27  Scanner sc = new Scanner(System.in);
28  int x = sc.nextInt();
29  System.out.println(Integer.toBinaryString(x) + " -> longest run after flipping one bit
        = " + longestAfterFlip(x));
30  }
31 }
```

Tracks `current` and `previous`; the candidate at every position is previous + current + 1.

Sample input

```
478
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	All 32 bits are already 1: there is nothing to flip, the answer is 32 (not 33).
2	line 17	On a 0: if the NEXT bit is also 0, flipping this 0 cannot connect to the earlier run (the other 0 is in the way), so previous restarts at 0; otherwise the run we just finished becomes `previous`.
3	line 20	Left run + the flipped bit + right run (the run we are in).
4	line 23	A 32-bit int cannot hold a longer run.

FlipCheck.java — Brute-force verification (demonstration)

```
public class FlipCheck {
 static int fast(int x) {
 if (x == -1) return 32;
 int current = 0, previous = 0, best = 1;
 for (int i = 0; i < 32; i++) {
 if ((x & 1) == 1) {
 current++;
 } else {
 previous = ((x & 2) == 0) ? 0 : current;
 current = 0;
 }
 best = Math.max(best, previous + current + 1);
 x >>>= 1;
 }
 return Math.min(best, 32);
 }

 // Try flipping each of the 32 bits and measure the longest run directly.
 static int brute(int x) {
 int best = 0;
 for (int bit = 0; bit < 32; bit++) {
 int y = x | (1 << bit);
 int run = 0, longest = 0;
 for (int i = 0; i < 32; i++) {
 if (((y >>> i) & 1) == 1) {
 run++;
 longest = Math.max(longest, run);
 } else run = 0;
 }
 best = Math.max(best, longest);
 }
 return best;
 }

 public static void main(String[] args) {
 java.util.Random rnd = new java.util.Random(7);
 int[] special = {0, 1, -1, 15, Integer.MIN_VALUE, Integer.MAX_VALUE, 0b111011110,
     0b1101110111111001};
 for (int x : special) {
 if (fast(x) != brute(x)) { System.out.println("MISMATCH " + x); return; }
 }
 for (int t = 0; t < 200000; t++) {
 int x = rnd.nextInt();
 if (fast(x) != brute(x)) { System.out.println("MISMATCH " + x); return; }
 }
 System.out.println("flip-one-bit matches brute force on 200000 random ints and the
     special cases");
 }
}
```

1 2

Tries flipping every one of the 32 bits and measures the run directly, comparing with the fast version.

Line-by-line walkthrough

#	Where	What happens
1	line 22	Force bit number `bit` to 1 (flipping a 0, or leaving a 1 unchanged).
2	line 42	200 000 random 32-bit integers, including negatives.

Dry run: watch it execute

478 = 111011110: flipping the 0 in the middle gives 3 + 1 + 4 = 8.

Algorithm: Longest run of 1s / flip one bit. **Input:** n = `478`.

Pseudocode

```
for each bit (low to high):
 if bit == 1: current++
 else: previous = current; current = 0
 best = max(best, previous + current + 1)
```

Step 1 of 11. 478 in binary is 111011110. We scan from the lowest bit (index 0) upwards, including one leading 0.

current = 0 · previous = 0 · best = 1

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 2 of 11. bit 0 is 0: previous = 0, current resets to 0.

current = 0 · previous = 0 · best = 1 · longest plain run = 0

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 3 of 11. bit 1 is 1: the current run grows to 1.

current = 1 · previous = 0 · best = 2 · longest plain run = 1

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 4 of 11. bit 2 is 1: the current run grows to 2.

current = 2 · previous = 0 · best = 3 · longest plain run = 2

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 5 of 11. bit 3 is 1: the current run grows to 3.

current = 3 · previous = 0 · best = 4 · longest plain run = 3

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 6 of 11. bit 4 is 1: the current run grows to 4.

current = 4 · previous = 0 · best = 5 · longest plain run = 4

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 7 of 11. bit 5 is 0: previous = 4, current resets to 0.

current = 0 · previous = 4 · best = 5 · longest plain run = 4

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 8 of 11. bit 6 is 1: the current run grows to 1.

current = 1 · previous = 4 · best = 6 · longest plain run = 4

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 9 of 11. bit 7 is 1: the current run grows to 2.

current = 2 · previous = 4 · best = 7 · longest plain run = 4

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 10 of 11. bit 8 is 1: the current run grows to 3.

current = 3 · previous = 4 · best = 8 · longest plain run = 4

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 11 of 11. bit 9 is 0: previous = 0, current resets to 0.

current = 0 · previous = 0 · best = 8 · longest plain run = 4

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

> **RESULT**
> longest run of 1s = 4; after flipping one bit = 8

Complexity

Both programs look at each of the 32 bits once: **O(w)** where w = 32 is the word size (equivalently O(log n) for the number's value). Space O(1).

The String-based version on the slides is also O(log n) time but uses O(log n) memory and breaks for negative numbers, where `Integer.toBinaryString(-1)` has 32 ones and the +1 gives 33.

Common mistakes

Wrong decimal value in the slide example

Problem

```
0b1101110111111001 // "87769" and answer "5"
```

Why: 0b1101110111111001 is 56 825, and its longest run of 1s is 6, not 5.

Fix: Always verify examples by running them (java LongestOnes with 56825 prints 6).

Using toBinaryString on a negative number

Problem

```
Integer.toBinaryString(-1).length() // 32
```

Why: Works but then "flip one bit" returns 33 for -1; the result must be capped at the 32 bits.

Fix: if (x == -1) return 32;

Resetting current without saving previous

Problem

```
else { current = 0; } // flip variant
```

Why: You lose the run to the left of the zero; the join length cannot be computed.

Fix: previous = current; current = 0;

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Longest run of 1s — *Beginner*

Read an int and print the length of the longest block of consecutive 1 bits in its 32-bit two's-complement representation.

Example input	Expected output
56825	6

Hint 1: Scan 32 bits with `x & 1` and `x >>>= 1`.

Exercise 2 · Flip one bit — *Intermediate*

Read an int. You may flip exactly one bit (0 → 1 or leave it). Print the longest run of 1s you can obtain among the 32 bits.

Example input	Expected output
478	8

Hint 1: Brute force is fine: try each of the 32 bits. **Hint 2:** Or track current and previous runs in one pass.

Exercise 3 · Flip up to k zeros — *Challenge*

Read an int x and k. Print the longest run of 1s in the 32-bit representation after flipping at most k zeros to ones.

- 0 ≤ k ≤ 32

Example input	Expected output
478 2	9

Hint 1: Sliding window over the 32 bits: keep the number of zeros inside the window ≤ k. **Hint 2:** Move the left end forward whenever the window has too many zeros.

Quick check

Answers are in the answer key at the end of this part.

Q1. What does this print?

```
int x = 0b0110_1110;
int cur = 0, best = 0;
for (int i = 0; i < 8; i++) {
 if ((x & 1) == 1) best = Math.max(best, ++cur);
 else cur = 0;
 x >>>= 1;
}
System.out.println(best);
```

Your answer: _____

Q2. In the flip variant, which value is the candidate on a 0?

- **A.** current
- **B.** previous
- **C.** previous + current + 1
- **D.** current × 2

Q3. The longest run of 1s after flipping one bit in -1 (all ones) is...

- **A.** 31
- **B.** 32
- **C.** 33
- **D.** 1

Q4. What is the time complexity of the bit scan?

- **A.** O(1)
- **B.** O(log n) (32 steps)
- **C.** O(n)
- **D.** O(n²)

Interview / exam questions

- Flip one bit to get the longest sequence of 1s — outline the O(w) solution.
- Why do you need `previous` in addition to `current`?
- What changes if you may flip k bits?

Watch & read

Video: Maximizing Contiguous Ones: Find the Longest Sequence in a Binary Array with a Single Flip | Medium — Tutorial Horizon · 12:00. *Longest run of 1s after one flip*

Video: L4. Max Consecutive Ones III | 2 Pointers and Sliding Window Playlist — take U forward · 29:58. *The sliding-window view*

REVISION CARD (30–60 SECONDS)

• Scan bits: current++ on 1, reset on 0, best = max.
• Flip variant: candidate = previous + current + 1 at each 0; previous = current (unless next bit is 0 too).
• All ones (−1) → 32.
• O(32) time, O(1) space.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide states that 0b1101110111111001 equals 87769 and that its longest run is 5. It equals 56825 and the longest run is 6. • The slide's Longafterflip code uses Integer.toBinaryString and char loops; its last line `maxLength = max(maxLength, currentLength + previousLength + 1)` yields 33 for -1. This lesson caps the answer at 32.

5.12 Josephus Problem

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 50 min · ★ on the minimum-viable path

People stand in a circle and every k-th one is removed. Who is left? A recursion that becomes a three-line loop.

YOU WILL BE ABLE TO

- Simulate the elimination with a list and print the order of elimination.
- Derive the recurrence J(n) = (J(n−1) + k) mod n and explain each term.
- Implement the recursive and iterative O(n) solutions and convert between 0-based and 1-based positions.
- Use the k = 2 shortcut 2·(n − 2^m) + 1.

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Divisibility, GCD/LCM, Primes and Modular Arithmetic · Generics and Lists: ArrayList and LinkedList

The idea

N people stand in a circle, numbered 1..N. Starting at person 1, count k people; the k-th person leaves the circle. Counting continues from the next person. The last person left is the survivor. For N = 7, k = 3 the order of leaving is 3, 6, 2, 7, 5, 1 and person **4** survives.

```
N = 5, k = 2 1 2 3 4 5 start counting at 1
 kill 2 → 1 _ 3 4 5 next count starts at 3
 kill 4 → 1 _ 3 _ 5 next count starts at 5
 kill 1 → _ _ 3 _ 5 next count starts at 3
 kill 5 → _ _ 3 _ _ survivor: 3
```

WHY THIS WORKS

Suppose you know the answer for a circle of n−1 people, counted from the person right after the first victim. The only difference between that circle and the real circle of n people is that positions are shifted: the first victim stood at position k (0-based k−1), and the new count starts k places further round. Translating a position p of the small circle back to the big circle therefore adds k and wraps around n: **J(n) = (J(n−1) + k) mod n** (0-based). With 1 person the survivor is position 0, which seeds the loop.

Syntax

The `%` remainder operator wraps a position around a circle: `(index + k) % size` always lands inside 0..size−1. Lists (`ArrayList<Integer>`) support `remove(index)`, which shifts the later elements left.

A tiny example

```
int ans = 0; // 1 person: position 0
for (int people = 2; people <= n; people++)
 ans = (ans + k) % people;
return ans + 1; // 1-based
```

Guided example

Check the formula against the slide examples by running the recursive program: (5, 2) → 3, (7, 3) → 4, (6, 2) → 5, (14, 2) → 13.

JosephusSimulation.java — Josephus by simulation (simple solution)

```
1  import java.util.ArrayList;
2  import java.util.List;
3  import java.util.Scanner;
4
5  public class JosephusSimulation {
6   // Literally act out the circle: remove every k-th person until one is left.
7   static int survivor(int n, int k, List<Integer> order) {
8   List<Integer> circle = new ArrayList<>();
9   for (int person = 1; person <= n; person++) {
10  circle.add(person);
11  }
12  int index = 0; // where counting starts
13  while (circle.size() > 1) {
14  index = (index + k - 1) % circle.size(); // skip k-1 people, land on the k-th          (1)
15  order.add(circle.remove(index)); // that person is out; the next count starts at the  (2)
        same index
16  }
17  return circle.get(0);                                                                 (3)
18  }
19
20  public static void main(String[] args) {
21  Scanner sc = new Scanner(System.in);
22  int n = sc.nextInt();
23  int k = sc.nextInt();
24  List<Integer> order = new ArrayList<>();
25  int last = survivor(n, k, order);
26  System.out.println("eliminated in order: " + order);
27  System.out.println("survivor: " + last);
28  }
29 }
```

Keeps the circle as a list and removes the k-th person repeatedly. O(n²) because removal from the middle shifts elements.

Sample input

```
7 3
```

Line-by-line walkthrough

#	Where	What happens
1	line 14	Skip k−1 people (counting the current one as 1) and wrap with %.
2	line 15	Remove the victim. The person who was after them slides into `index`, which is exactly where the next count starts.
3	line 17	One person remains.

JosephusRecursive.java — Josephus recursively (optimised solution)

```
1  public class JosephusRecursive {
2   // survivor's position (1-based) among n people when every k-th is eliminated.
3   // Idea: after the first elimination there are n-1 people and counting restarts right
        after the
4   // eliminated person. Solve that smaller circle, then translate its answer back to the
        n-circle.
5   static int josephus(int n, int k) {
6   if (n == 1) {                                                                    (1)
7   return 1; // one person left: they survive
8   }
9   return (josephus(n - 1, k) + k - 1) % n + 1; // shift by k places and wrap around the  (3)
        circle of n
10  }
11
12  public static void main(String[] args) {
13  int[][] cases = {{5, 2}, {7, 3}, {6, 2}, {14, 2}};
14  for (int[] c : cases) {
15  System.out.println("josephus(" + c[0] + ", " + c[1] + ") = " + josephus(c[0], c[1]));
16  }
17  }
18 }
```

J(1) = 1; J(n) = (J(n−1) + k − 1) mod n + 1 in 1-based form.

Line-by-line walkthrough

#	Where	What happens
1	line 6	Base case: one person left.
2	line 9	Solve the smaller circle first.
3	line 9	Convert to 0-based (−1), add k, wrap in n people, convert back (+1). The slide folds the "−1 … +1" into this expression.

JosephusIterative.java — Josephus iteratively (optimised solution)

```
1  import java.util.Scanner;
2
3  public class JosephusIterative {
4   // The same recurrence without recursion: build the answer for 1, 2, 3, ... n people
        (0-based positions).
5   static int josephus(int n, int k) {
6   int ans = 0; // with 1 person the survivor is at position 0                      (1)
7   for (int people = 2; people <= n; people++) {
8   ans = (ans + k) % people;                                                       (2)
9   }
10  return ans + 1; // convert to 1-based                                           (3)
11  }
12
13  public static void main(String[] args) {
14  Scanner sc = new Scanner(System.in);
15  int n = sc.nextInt();
16  int k = sc.nextInt();
17  System.out.println(josephus(n, k));
18  }
19 }
```

Same recurrence, built bottom-up: O(n) time, O(1) space, no recursion depth problem.

Sample input

```
14 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	With one person the survivor is at (0-based) position 0.
2	line 8	Grow the circle by one person: shift by k and wrap.
3	line 10	Convert to 1-based numbering.

JosephusPowerOfTwo.java — Josephus for k = 2 (optimised solution)

```
1  import java.util.Scanner;
2
3  public class JosephusPowerOfTwo {
4   // Special case k = 2: write n = 2^m + r (largest power of two not above n). Answer =
        2r + 1.
5   static long josephusK2(long n) {
6   long highestPowerOfTwo = Long.highestOneBit(n);                    (1)
7   return 2 * (n - highestPowerOfTwo) + 1;                            (2)
8   }
9
10  public static void main(String[] args) {
11  Scanner sc = new Scanner(System.in);
12  long n = sc.nextLong();
13  System.out.println(josephusK2(n));
14  }
15 }
```

Write n = 2^m + r; the survivor is 2r + 1. O(1).

Sample input

```
14
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	The largest power of two not above n (e.g. 14 → 8).
2	line 7	After r = n − 2^m eliminations the circle has exactly 2^m people and the next person to count is 2r + 1 — and with k = 2 and a power of two people, the first person always survives.

Dry run: watch it execute

Watch the circle shrink as every 3rd person leaves.

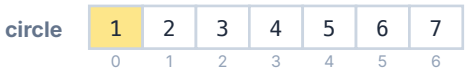
Algorithm: Josephus problem. **Input:** n = `7`, k = `3`.

Pseudocode

```
circle = [1..n]; index = 0
while (circle.size() > 1) {
 index = (index + k - 1) % size
 remove circle[index]
}
survivor = circle[0]
```

Step 1 of 8. 7 people stand in a circle; counting starts at person 1.

index = 0 · size = 7



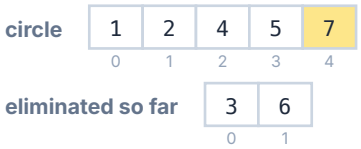
Step 2 of 8. Count 3: person 3 leaves. Counting restarts at 4.

eliminated = 3 · index = 2 · size = 6



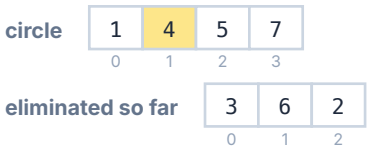
Step 3 of 8. Count 3: person 6 leaves. Counting restarts at 7.

eliminated = 6 · index = 4 · size = 5



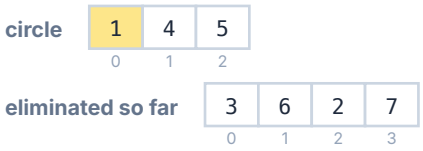
Step 4 of 8. Count 3: person 2 leaves. Counting restarts at 4.

eliminated = 2 · index = 1 · size = 4



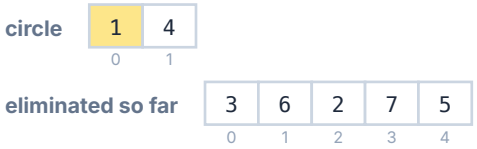
Step 5 of 8. Count 3: person 7 leaves. Counting restarts at 1.

eliminated = 7 · index = 0 · size = 3



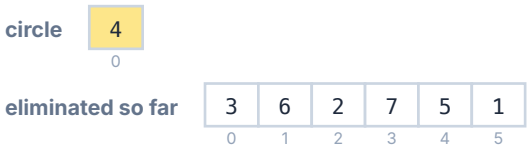
Step 6 of 8. Count 3: person 5 leaves. Counting restarts at 1.

eliminated = 5 · index = 0 · size = 2



Step 7 of 8. Count 3: person 1 leaves. Counting restarts at 4.

eliminated = 1 · index = 0 · size = 1



Step 8 of 8. Only 4 is left: the survivor.

survivor = 4



> **RESULT**
> survivor = 4 (order out: 3 6 2 7 5 1)

Complexity

Simulation: each of the n−1 removals costs O(n) (list shift) → $O(n^2)$. With a linked list or a Fenwick tree this improves but is still not simple.

Recurrence: n steps of O(1) → **O(n) time**. Recursive form uses O(n) stack space; the loop uses O(1).

k = 2: O(1) with a bit operation (O(log n) without highestOneBit).

> **NOTE**
> For huge n and small k there is an O(k log n) algorithm; for k = 2 it collapses to the bit trick above.

Common mistakes

Mixing 0-based and 1-based positions

Problem

```
return (josephus(n - 1, k) + k) % n; // returns 0..n-1 but the problem is 1-based
```

Why: The answer is off by one. Decide which indexing you use and convert once, at the end (or use the slide's +k−1 ... +1 form).

Fix: return (josephus(n - 1, k) + k - 1) % n + 1;

Skipping k instead of k – 1

Problem

```
index = (index + k) % circle.size(); // simulation
```

Why: The current person counts as the first; the k-th person is k−1 steps further. Using k kills the wrong person.

Fix: index = (index + k - 1) % circle.size();

Stack overflow on large n

Problem

```
josephus(1_000_000, 2) // recursive
```

Why: One stack frame per person: about 10^5–10^6 deep recursion throws StackOverflowError.

Fix: Use the iterative loop (same recurrence).

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Elimination order — *Beginner*

Read N and k. Print the numbers in the order in which people are removed; the survivor is printed last.

Example input	Expected output
5 2	2 4 1 5 3

Hint 1: Keep an ArrayList 1..N and an index. **Hint 2:** index = (index + k − 1) % size, then remove(index).

Exercise 2 · Survivor for large N — *Intermediate*

Read N and k and print the survivor's position (1-based).

- $1 \le N \le 10^7$
- $1 \le k \le 10^9$

Example input	Expected output
10 3	4

Hint 1: Simulation is too slow for N = 10^7 — use the recurrence. **Hint 2:** Use long for (ans + k).

Exercise 3 · Josephus with k = 2 and N up to 10^{18} — *Challenge*

Every second person is removed (k = 2). Read N and print the survivor.

- $1 \le N \le 10^{18}$

Example input	Expected output
100	73

Hint 1: Write N = 2^m + r. What happens after r eliminations? **Hint 2:** Long.highestOneBit gives 2^m.

Quick check

Answers are in the answer key at the end of this part.

Q1. For N = 6 and k = 2, who survives?

- **A.** 1
- **B.** 3
- **C.** 5
- **D.** 6

Q2. What does this print?

```
int ans = 0;
for (int people = 2; people <= 5; people++)
 ans = (ans + 2) % people;
System.out.println(ans + 1);
```

Your answer: _____

Q3. What is the time complexity of the iterative Josephus recurrence?

- **A.** O(1)
- **B.** O(log n)
- **C.** O(n)
- **D.** O(n^2)

Q4. Complete the recurrence for 0-based positions.

```
ans = (ans ______ k) % people;
```

Q5. For k = 2 and N = 20 (= 16 + 4) the survivor is...

- **A.** 9
- **B.** 17
- **C.** 8
- **D.** 4

Interview / exam questions

- Derive J(n) = (J(n−1) + k) mod n.
- How would you print the elimination order efficiently for N = 10^5?
- Is there a closed form for k = 2?
- What data structure speeds up the simulation?

Watch & read

Video: The Josephus Problem - Numberphile — Numberphile · 13:58. *The Josephus problem*

Video: Josephus Problem | GeeksforGeeks — GeeksforGeeks · 12:13. *The Josephus recurrence*

REVISION CARD (30–60 SECONDS)

- J(1) = 0; J(n) = (J(n−1) + k) mod n (0-based). Add 1 for 1-based.
- Simulation: index = (index + k − 1) % size; remove.
- k = 2: n = 2^m + r → 2r + 1.
- Iterative O(n) time, O(1) space; recursion depth n.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide's explanatory text says "recursively delete the Kth position until only one person is left", but its code does not delete anything: it uses the arithmetic recurrence. The slide diagram (steps 01–06) uses start = (start + k) % size with k = 1 in 0-based form, which is the same idea as the recurrence run forwards.

5.13 Block Swap Algorithm (Array Rotation)

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 50 min · ★ on the minimum-viable path

Rotate an array left by d positions in O(n) time and O(1) extra space by swapping blocks.

YOU WILL BE ABLE TO

- Define left rotation by d and compute it by hand.
- Implement the O(n·d) one-by-one rotation and the O(n) temp-array version.
- Explain and trace the block swap algorithm that works in place.
- Compare it with the three-reversal trick and verify all variants by testing.

Before this lesson: One-Dimensional Arrays · Loops: for, while, do-while, break and continue · Methods: Parameters, Return Values, Scope and Overloading

The idea

Rotating `{1,2,3,4,5,6,7}` left by 2 moves the first two elements to the end: `{3,4,5,6,7,1,2}`. Split the array into **A** = the first d elements and **B** = the rest. Rotation turns **AB** into **BA**.

```
d = 2: [ 1 2 | 3 4 5 6 7 ] A = [1 2] B = [3 4 5 6 7]
 A shorter than B → swap A with the LAST |A| elements of B:
 [ 6 7 | 3 4 5 | 1 2 ] → A is now in its final place at the end
 remaining: [6 7 | 3 4 5] , swap again:
 [ 3 4 | 6 7 | 5 ] ...
```

WHY THIS WORKS

Block swap is "Euclid's algorithm on blocks". If A is shorter than B, swapping A with the last |A| elements of B puts A in its final position (at the end) and leaves a smaller problem: rotate the front part `A_new B_front`. If A is longer, swapping B with the first |B| elements of A puts B at the front, and the remaining part `A_rest …` is a smaller rotation. Each step fixes some elements permanently and shrinks the problem like the subtraction form of Euclid's GCD, until both blocks are equal and one swap finishes.

Alternative: **reversal**. Reverse A, reverse B, then reverse everything: (A^r B^r)^r = B A. Three passes, 2 loops of swaps, trivial to code.

Syntax

```java
int t = a[i]; a[i] = a[j]; a[j] = t; // swap two cells
System.arraycopy(a, from, b, to, len); // block copy (handles overlap)
int[] part = Arrays.copyOfRange(a, from, toExclusive); // new array
Collections.rotate(list, -d); // library rotation of a List by -d (left d)
static void reverse(int[] a, int lo, int hi) { // reverse a[lo..hi] in place
 while (lo < hi) { int t = a[lo]; a[lo++] = a[hi]; a[hi--] = t; }
}
```

A tiny example

```
int first = a[0];
for (int i = 0; i < n - 1; i++) a[i] = a[i + 1];
a[n - 1] = first; // rotate left by 1: O(n)
```

Guided example

Rotating by d with the tiny loop d times costs d·n moves. The block swap moves each element at most once or twice.

RotateOneByOne.java — Rotate one position at a time (simple solution)

```
1  import java.util.Arrays;
2  import java.util.Scanner;
3
4  public class RotateOneByOne {
5   // Rotate left by one position, d times. Easy to understand, but d * n element moves.
6   static void leftRotate(int[] a, int d) {
7   int n = a.length;
8   d %= n;                                                    (1)
9   for (int step = 0; step < d; step++) {
10  int first = a[0];                                          (2)
11  for (int i = 0; i < n - 1; i++) {
12  a[i] = a[i + 1];                                           (3)
13  }
14  a[n - 1] = first;                                          (4)
15  }
16  }
17
18  public static void main(String[] args) {
19  Scanner sc = new Scanner(System.in);
20  int n = sc.nextInt();
21  int d = sc.nextInt();
22  int[] a = new int[n];
23  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
24  leftRotate(a, d);
25  System.out.println(Arrays.toString(a));
26  }
27 }
```

Left-rotates by 1, d times. O(n·d) time, O(1) space.

Sample input

```
8 2
7 9 8 0 5 1 6 4
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	Rotating by n returns the array to the start, so only d mod n matters.
2	line 10	The element that falls off the front must be saved.
3	line 12	Everyone else moves one step left.
4	line 14	The saved element lands at the end.

BlockSwapRotate.java — Block swap rotation (optimised solution)

```java
import java.util.Arrays;
import java.util.Scanner;

public class BlockSwapRotate {
 // swap d elements starting at index fi with d elements starting at index si
 static void swap(int[] a, int fi, int si, int d) {
 for (int i = 0; i < d; i++) {
 int temp = a[fi + i];
 a[fi + i] = a[si + i];
 a[si + i] = temp;
 }
 }

 // Left-rotate a[0..n-1] by d using block swaps. A = a[0..d-1], B = a[d..n-1].
 static void leftRotate(int[] a, int d) {
 int n = a.length;
 if (n == 0) return;
 d %= n;                                                          (1)
 if (d == 0) return;
 int i = d; // size of block A (still to be placed)               (2)
 int j = n - d; // size of block B
 while (i != j) {
 if (i < j) { // A is shorter: swap A with the LAST i elements of B  (3)
 swap(a, d - i, d + j - i, i);                                    (4)
 j -= i; // those i elements of B are now final                   (5)
 } else { // A is longer: swap the LAST j elements of A with B
 swap(a, d - i, d, j);                                            (6)
 i -= j;
 }
 }
 swap(a, d - i, d, i); // equal sizes: one final block swap        (7)
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int d = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 leftRotate(a, d);
 System.out.println(Arrays.toString(a));
 }
}
```

A = a[0..d−1], B = a[d..n−1]. Repeatedly swap the shorter block with part of the longer one.

Sample input

```
10 5
6 8 7 9 0 5 1 3 2 4
```

Line-by-line walkthrough

#	Where	What happens
1	line 18	Normalise d (the slide handles d > n the same way).
2	line 20	i = size of A still to be placed; j = size of B. They shrink like the two numbers in Euclid's algorithm.
3	line 23	A is shorter: swap it with the LAST i elements of B. Those elements of B now hold A's old values and are in their final position.
4	line 24	Swap i elements starting at d−i (the end of A) with i elements starting at d+j−i (the tail of B).
5	line 25	The i swapped elements are final, so B effectively gets shorter.
6	line 27	A is longer: swap the last j elements of A with all of B; B is now in its final front position.
7	line 31	When i == j the two blocks are the same size and one swap finishes the job.

RotateReversal.java — Rotation by three reversals (optimised solution)

```
1  import java.util.Arrays;
2  import java.util.Scanner;
3
4  public class RotateReversal {
5   static void reverse(int[] a, int lo, int hi) {
6   while (lo < hi) {
7   int t = a[lo];
8   a[lo++] = a[hi];
9   a[hi--] = t;
10  }
11  }
12
13  // Reverse A, reverse B, then reverse the whole array: (A^r B^r)^r = B A
14  static void leftRotate(int[] a, int d) {
15  int n = a.length;
16  d %= n;
17  reverse(a, 0, d - 1);                                    (1)
18  reverse(a, d, n - 1);                                    (2)
19  reverse(a, 0, n - 1);                                    (3)
20  }
21
22  public static void main(String[] args) {
23  Scanner sc = new Scanner(System.in);
24  int n = sc.nextInt();
25  int d = sc.nextInt();
26  int[] a = new int[n];
27  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
28  leftRotate(a, d);
29  System.out.println(Arrays.toString(a));
30  }
31 }
```

reverse(A), reverse(B), reverse(whole). Easy to remember, O(n) time, O(1) space.

Sample input

```
7 2
1 2 3 4 5 6 7
```

Line-by-line walkthrough

#	Where	What happens
1	line 17	Reverse the first block A in place.
2	line 18	Reverse the second block B in place.
3	line 19	Reversing everything turns (A^r B^r) into B A.

BlockSwapCheck.java — Exhaustive check (demonstration)

```
import java.util.Arrays;

public class BlockSwapCheck {
 static void swap(int[] a, int fi, int si, int d) {
 for (int i = 0; i < d; i++) {
 int t = a[fi + i];
 a[fi + i] = a[si + i];
 a[si + i] = t;
 }
 }

 static void blockSwap(int[] a, int d) {
 int n = a.length;
 d %= n;
 if (d == 0) return;
 int i = d, j = n - d;
 while (i != j) {
 if (i < j) {
 swap(a, d - i, d + j - i, i);
 j -= i;
 } else {
 swap(a, d - i, d, j);
 i -= j;
 }
 }
 swap(a, d - i, d, i);
 }

 // Compares block swap with the obvious "rotate through a temp array" for every n <= 40 and every d.
 public static void main(String[] args) {
 int checked = 0;
 for (int n = 1; n <= 40; n++) {
 for (int d = 0; d <= 2 * n; d++) {
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = i + 1;
 int[] expected = new int[n];
 for (int i = 0; i < n; i++) expected[i] = a[(i + d) % n];
 blockSwap(a, d);
 if (!Arrays.equals(a, expected)) {
 System.out.println("MISMATCH n=" + n + " d=" + d + " got " + Arrays.toString(a));
 return;
 }
 checked++;
 }
 }
 System.out.println("block swap matches the reference for all " + checked + " (n, d) pairs");
 }
}
```

Compares the block swap output with `a[(i + d) % n]` for 1 680 combinations of n and d (including d > n and d = 0).

Line-by-line walkthrough

#	Where	What happens
1	line 37	The definition of a left rotation by d: element i of the result is element (i + d) mod n of the input.

Dry run: watch it execute

The two shrinking blocks i and j are swapped until they are equal.

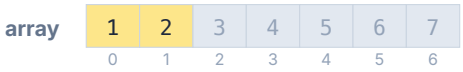
Algorithm: Block swap rotation. **Input:** arr = `1,2,3,4,5,6,7`, d = `2`.

Pseudocode

```
i = d; j = n − d
while (i != j) {
 if (i < j) { swap(d−i, d+j−i, i); j −= i; }
 else { swap(d−i, d, j); i −= j; }
}
swap(d − i, d, i)
```

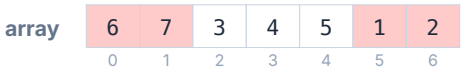
Step 1 of 5. Rotate left by 2. Block A = first 2 elements, block B = the other 5.

i = 2 · j = 5 · d = 2



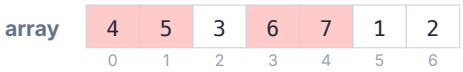
Step 2 of 5. A (size 2) is shorter: swap it with the last 2 elements of B; B shrinks to 3.

i = 2 · j = 3



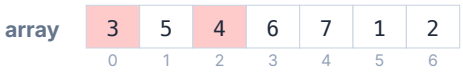
Step 3 of 5. A (size 2) is shorter: swap it with the last 2 elements of B; B shrinks to 1.

i = 2 · j = 1



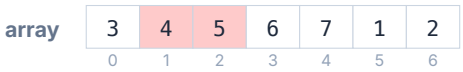
Step 4 of 5. A is longer: swap the last 1 elements of A with B; A shrinks to 1.

i = 1 · j = 1



Step 5 of 5. Blocks are equal (size 1): one final swap.

i = 1 · j = 1



> **RESULT**
> [3, 4, 5, 6, 7, 1, 2]

Complexity

One-by-one: d passes × n moves → O(n·d) time, O(1) space.

Temp array: copy the first d elements aside, shift the rest, copy back → O(n) time, O(d) space.

Block swap / reversal: every element is swapped O(1) times → **O(n) time, O(1) auxiliary space** (iterative).

> **CHECK THE CLAIM**
> The slide says the recursive version uses O(log n) space. Recursion depth of the subtraction-style block swap can reach about n (try d = 1), so O(log n) is not guaranteed; the iterative loop above avoids the issue entirely.

method	time	extra space
rotate by 1, d times	O(n·d)	O(1)
temp array	O(n)	O(d)
block swap (iterative)	O(n)	O(1)
three reversals	O(n)	O(1)

Common mistakes

Using d without taking d mod n

Problem

```
for (int i = 0; i < n; i++) result[i] = a[i + d];
```

Why: Throws ArrayIndexOutOfBoundsException for i + d ≥ n, and never wraps around.

Fix: result[i] = a[(i + d) % n];

Mixing up left and right rotation

Problem

```
// "rotate by 2" for {1,2,3,4,5} printed as {4,5,1,2,3}
```

Why: That is a RIGHT rotation. Left rotation by 2 gives {3,4,5,1,2}. Always state the direction.

Fix: Right rotation by d == left rotation by n − d.

Slide example 2 typo

Problem

```
// {6,8,7,9,0,5,1,3,2,4}, d = 5 → "{5,1,2,3,4,6,8,7,9,0}"
```

Why: The elements 3 and 2 are in the wrong order in the slide. The correct left rotation is {5,1,3,2,4,6,8,7,9,0} (verified by running the program).

Fix: Trace by hand: take indices 5..9 then 0..4.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Left rotation — *Beginner*

Read n, d and an array of n integers. Print the array after rotating it left by d positions.

- $1 \le n \le 10^5$
- $0 \le d \le 10^9$

Example input	Expected output
5 2 ⏎ 1 2 3 4 5	3 4 5 1 2

Hint 1: Only d mod n matters. **Hint 2:** Output element i is input element (i + d) mod n.

Exercise 2 · Right rotation in place — *Intermediate*

Read n, d and the array. Print the array rotated RIGHT by d, using O(1) extra memory (reversals).

Example input	Expected output
5 2 ⏎ 1 2 3 4 5	4 5 1 2 3

Hint 1: A right rotation by d is a left rotation by n − d. **Hint 2:** Reverse three times.

Exercise 3 · How many rotations? — *Challenge*

A sorted array of distinct integers was rotated left an unknown number of times. Read n and the array and print how many left rotations produced it (the index of the minimum). Use O(log n) time.

Example input	Expected output
6 ⏎ 15 18 2 3 6 12	2

Hint 1: The minimum is the only element smaller than its predecessor. **Hint 2:** Binary search: compare a[mid] with a[hi].

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
int[] a = {1, 2, 3, 4, 5};
int d = 2, n = a.length;
for (int i = 0; i < n; i++)
 System.out.print(a[(i + d) % n] + " ");
```

Your answer: _____

Q2. Which method rotates in O(n) time and O(1) extra space?

- **A.** Rotate by one, d times
- **B.** Copy to a temp array of size n
- **C.** Three reversals
- **D.** Sorting

Q3. Right rotation by 2 of a 7-element array equals left rotation by...

- **A.** 2
- **B.** 5
- **C.** 7
- **D.** 9

Q4. The time complexity of the one-by-one rotation by d is...

- **A.** O(n)
- **B.** O(n + d)
- **C.** O(n·d)
- **D.** O(log n)

Q5. After reversal rotation of {1,2,3,4,5,6} left by 2, what is the 1st element?

```
// reverse(0,1) -> 2 1 3 4 5 6
// reverse(2,5) -> 2 1 6 5 4 3
// reverse(0,5) -> ?
```

Your answer: _____

Interview / exam questions

- Rotate an array in place without extra space — name two ways.
- Why does the reversal method work? (A^r B^r)^r = ?
- What is the juggling algorithm and how does it use gcd(n, d)?
- How would you search a rotated sorted array?

Watch & read

Video: Block swap algorithm for Array rotation || easy explanation — Code with Med · 13:42. *Rotating an array by block swap*

Video: Rotate Array - Leetcode 189 - Python — NeetCode · 8:59. *Reversal algorithm for rotation (Python — the idea transfers to Java)*

REVISION CARD (30–60 SECONDS)

- Left rotate by d: result[i] = a[(i + d) % n]; use d mod n.
- AB → BA. Block swap shrinks like Euclid; three reversals is simplest.
- O(n) time, O(1) space.
- Right by d = left by n − d.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• Example 2 on the slide prints {5,1,2,3,4,6,8,7,9,0}; the correct result is {5,1,3,2,4,6,8,7,9,0}. • The slide says "O(log n)" auxiliary space for the recursive version. The depth can be O(n); the iterative version is O(1). • The slide's `leftRotate` and the helper `swap` appear in a confusing order across two slides; they belong to the same class.

5.14 Leaders in an Array

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 30 min · ★ on the minimum-viable path

An element is a leader if it is greater than everything to its right. Brute force vs a single right-to-left scan.

YOU WILL BE ABLE TO

- State the definition of a leader and find all leaders in a small array by hand.
- Write the O(n²) brute force with a `break`.
- Write the O(n) right-to-left scan and explain why it works.
- Adapt it to non-strict leaders and next-greater-element.

Before this lesson: One-Dimensional Arrays · Loops: for, while, do-while, break and continue

The idea

Imagine people standing in a line, all facing right. Someone is a **leader** if they are **taller than everybody standing to their right** — nobody ahead can look over them. In `{14, 20, 3, 6, 5}` the leaders are 20, 6 and 5 (the last person has nobody to their right, so is always a leader).

```
index : 0 1 2 3 4
value : 14 20 3 6 5
right max:20 6 6 5 –
leader? ✗ ✓ ✗ ✓ ✓ → 20, 6, 5
```

WHY THIS WORKS

Looking from the right, the only thing you need to know about "everything to my right" is its **maximum**. Walk from the last element to the first, keeping `maxFromRight`. If the current element is greater than that maximum it is a leader (and becomes the new maximum). One pass, one variable — the O(n²) comparisons collapse because the maximum summarises the whole suffix.

Syntax

```
int max = Integer.MIN_VALUE; // largest value seen so far from the right
List<Integer> leaders = new ArrayList<>();
for (int i = a.length - 1; i >= 0; i--) { // right-to-left loop
 if (a[i] > max) { leaders.add(a[i]); max = a[i]; }
}
Collections.reverse(leaders); // restore left-to-right order
```

A tiny example

```
int max = a[n - 1]; // the last element is always a leader
for (int i = n - 2; i >= 0; i--)
 if (a[i] > max) max = a[i]; // a[i] is a leader
```

Guided example

The lecture slides show both approaches and print the leaders of `{16, 17, 4, 3, 5, 2}`: **17, 5, 2** (left to right). The right-to-left scan naturally finds them in the order 2, 5, 17, so the lesson program stores them and prints in the original order.

LeadersBruteForce.java — Brute force (simple solution)

```
1  import java.util.Scanner;
2
3  public class LeadersBruteForce {
4   // An element is a leader if it is strictly greater than EVERY element to its right.
5   public static void main(String[] args) {
6   Scanner sc = new Scanner(System.in);
7   int n = sc.nextInt();
8   int[] a = new int[n];
9   for (int i = 0; i < n; i++) a[i] = sc.nextInt();
10
11  StringBuilder sb = new StringBuilder();
12  for (int i = 0; i < n; i++) {
13  boolean isLeader = true;
14  for (int j = i + 1; j < n; j++) { // compare with everything on the right   1
15  if (a[i] <= a[j]) {                                                        2
16  isLeader = false;
17  break; // one bigger-or-equal element is enough                           3
18  }
19  }
20  if (isLeader) {
21  sb.append(sb.length() == 0 ? "" : " ").append(a[i]);
22  }
23  }
24  System.out.println(sb);
25  }
26 }
```

For every element, compare it with every element on its right.

Sample input

```
6
16 17 4 3 5 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 14	Only elements to the RIGHT of i matter.
2	line 15	A single element that is greater than or equal disqualifies a[i] ("greater than all" means strictly greater).
3	line 17	No need to look further once a[i] has been disqualified.

LeadersRightScan.java — Right-to-left scan (optimised solution)

```
1  import java.util.Scanner;
2
3  public class LeadersRightScan {
4   // Scan from the right keeping the largest value seen so far. One pass, O(1) extra
        space for the scan.
5   public static void main(String[] args) {
6   Scanner sc = new Scanner(System.in);
7   int n = sc.nextInt();
8   int[] a = new int[n];
9   for (int i = 0; i < n; i++) a[i] = sc.nextInt();
10
11  int[] leaders = new int[n];
12  int count = 0;
13  int maxFromRight = Integer.MIN_VALUE;                                    (1)
14  for (int i = n - 1; i >= 0; i--) {
15  if (a[i] > maxFromRight) { // bigger than everything to its right        (2)
16  leaders[count++] = a[i];
17  maxFromRight = a[i];                                                     (3)
18  }
19  }
20  // leaders were found right-to-left; print them in the original left-to-right order
21  StringBuilder sb = new StringBuilder();
22  for (int i = count - 1; i >= 0; i--) {                                   (4)
23  sb.append(sb.length() == 0 ? "" : " ").append(leaders[i]);
24  }
25  System.out.println(sb);
26  }
27 }
```

One pass from the right with a running maximum. O(n) time.

Sample input

```
6
16 17 4 3 5 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 13	Nothing to the right of the last element, so any value beats this.
2	line 15	Strictly greater than every element to its right.
3	line 17	The new leader is also the new maximum of the suffix.
4	line 22	Reverse the stored leaders to print them in left-to-right order.

Dry run: watch it execute

Scan right to left; each new maximum is a leader.

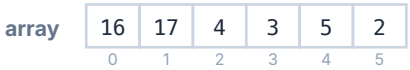
Algorithm: Leaders in an array. **Input:** arr = `16,17,4,3,5,2`.

Pseudocode

```
max = -infinity
for (i = n-1; i >= 0; i--)
 if (a[i] > max) {
 leader: a[i]
 max = a[i]
 }
```

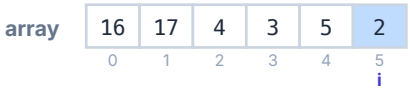
Step 1 of 7. Scan from the right. Nothing is to the right of the last element, so it is always a leader.

max = −∞ · leaders =



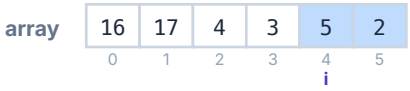
Step 2 of 7. 2 > max: leader! max becomes 2.

i = 5 · max = 2 · leaders = 2



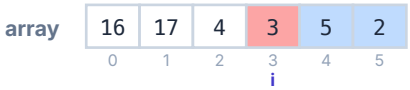
Step 3 of 7. 5 > max: leader! max becomes 5.

i = 4 · max = 5 · leaders = 5 2



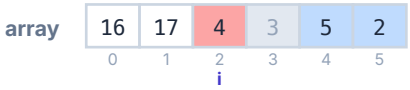
Step 4 of 7. 3 ≤ max (5): not a leader.

i = 3 · max = 5 · leaders = 5 2



Step 5 of 7. 4 ≤ max (5): not a leader.

i = 2 · max = 5 · leaders = 5 2



Step 6 of 7. 17 > max: leader! max becomes 17.

i = 1 · max = 17 · leaders = 17 5 2



Step 7 of 7. 16 ≤ max (17): not a leader.

i = 0 · max = 17 · leaders = 17 5 2



> **RESULT**
> leaders (left→right): 17 5 2

Complexity

Brute force: element i is compared with up to n−i−1 later elements. When every element is a leader (a strictly decreasing array) the inner loop never breaks, so the total is n(n−1)/2 comparisons → **O(n²)** worst case.

Right scan: one comparison per element → **O(n) time**, O(1) extra space besides the output.

n	brute force comparisons (worst)	right scan
1 000	≈ 500 000	1 000
100 000	≈ 5 × 10⁹	100 000

Common mistakes

Using >= and printing equal elements

Problem

```
if (a[i] >= maxFromRight) // for the standard (strict) definition
```

Why: With {7, 7} the first 7 is NOT greater than the 7 to its right. The strict definition excludes it.

Fix: if (a[i] > maxFromRight)

Forgetting the last element

Problem

```
for (int i = n - 2; i >= 0; i--) { ... } // never prints a[n-1]
```

Why: The last element has nothing to its right so it is always a leader and must be included.

Fix: start the scan at i = n − 1 with maxFromRight = MIN_VALUE

Initialising max to 0

Problem

```
int max = 0;
```

Why: For an all-negative array like {-5, -7} nothing is greater than 0, so no leader is found.

Fix: int max = Integer.MIN_VALUE;

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Print the leaders — *Beginner*

Read n and an array. Print all leaders (strictly greater than every element to their right) from left to right.

Example input	Expected output
6 ⏎ 16 17 4 3 5 2	17 5 2

Hint 1: Scan from the right with a running maximum. **Hint 2:** Store, then print in reverse.

Exercise 2 · Non-strict leaders — *Intermediate*

Same as before, but an element counts as a leader when it is greater than OR EQUAL to every element on its right.

Example input	Expected output
5 ⏎ 7 10 4 10 6	10 10 6

Hint 1: Only the comparison changes.

Exercise 3 · Next greater element — *Challenge*

For each element print the first element to its right that is strictly greater, or -1 if there is none.

Example input	Expected output
4 ⏎ 4 5 2 25	5 25 25 −1

Hint 1: Scan from the right with a stack of candidates. **Hint 2:** Pop candidates that are not greater than the current element.

Quick check

Answers are in the answer key at the end of this part.

Q1. The leaders of {16, 17, 4, 3, 5, 2} are...

- **A.** 17 5 2
- **B.** 16 17 5 2
- **C.** 2 5 17
- **D.** 17 4 3 5 2

Q2. This leader finder gives wrong results for {-3, -5}. Which line is the bug?

```
1  int max = 0;
2  for (int i = n - 1; i >= 0; i--) {
3   if (a[i] > max) {
4   max = a[i];
5   print(a[i]);
6   }
7  }
```

Which line contains the bug? _____

Q3. The right-to-left scan runs in...

- **A.** O(1)
- **B.** O(n)
- **C.** O(n log n)
- **D.** O(n²)

Q4. Is the last element of an array always a leader?

- **A.** Yes
- **B.** No
- **C.** Only if positive
- **D.** Only if largest

Interview / exam questions

- Why is the last element always a leader?
- Can you find leaders in O(n) using O(1) extra space?
- How is the leader problem related to the "next greater element" problem?

Watch & read

Video: Leaders in an Array | Brute - Optimal | Strivers A2Z DSA Course — take U forward · 11:53. *Finding leaders by scanning from the right*

Video: Leaders in an Array | Right-to-Left Scan — Beyond the Function · 4:47. *The right-to-left scan*

REVISION CARD (30–60 SECONDS)

- Leader = strictly greater than all elements to its right; last element is always one.
- Scan from the right, keep max, a[i] > max → leader.
- O(n^2) brute force vs O(n) scan.
- Print in original order by reversing the stored list.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The Leaders deck is a screen-recorded video presentation (slides are images). Its right-to-left code prints leaders in reverse order (2 5 17) — the lesson prints them left-to-right to match the definition.

5.15 Majority Element (Boyer–Moore Voting)

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 50 min · ★ on the minimum-viable path

Find the element that appears more than n/2 times: counting, sorting, and the clever O(1)-space voting algorithm.

YOU WILL BE ABLE TO

- State the majority-element definition (more than n/2 occurrences) precisely.
- Implement counting (two loops), sorting, and hash-map solutions and compare their costs.
- Explain Boyer–Moore voting: why cancelling different pairs leaves the majority, and why a verification pass is required.
- Extend the idea to elements appearing more than n/3 times.

Before this lesson: One-Dimensional Arrays · Sets and Maps: HashSet, TreeSet, HashMap, TreeMap

The idea

An element is the **majority element** of an array of length n if it occurs **more than n/2 times**. In `{4,7,4,4,7,4,4,9,4,3}` (n = 10) the value 4 occurs 6 times > 5, so 4 is the majority. In `{1,7,8,2,6,8,1,3,2,8}` the best is 8 with 3 occurrences — there is **no** majority element.

Voting analogy: every element is a voter for itself. Pair off two voters who disagree and send both home: the winner (who has more than half the votes) can never be completely eliminated, because every pair removed contains at most one of its voters. Whoever is left must be the only possible winner — but if there was no real winner, a loser is left too, so we have to **count** to confirm.

```
4 7 4 4 7 4 4 9 4 3 candidate count
4 4 1
 7 (differs → cancel) 4 0
 4 (count==0 → new) 4 1
 4 4 2
 7 (cancel) 4 1
 4 4 2
 4 4 3
 9 (cancel) 4 2
 4 4 3
 3 (cancel) 4 2 → candidate 4, then verify: 4 appears 6 > 5 ✓
```

WHY THIS WORKS

Each time the counter decrements, one occurrence of the candidate is cancelled against one occurrence of a *different* value. If x appears more than n/2 times it has more occurrences than all other values combined, so after cancelling pairs of unequal elements at least one x remains. The candidate at the end is therefore x. If no value is a majority the algorithm still returns *some* value, so verify with one more pass.

Syntax

```
int candidate = 0, count = 0; // Boyer–Moore vote
for (int x : a) {
 if (count == 0) candidate = x;
 count += (x == candidate) ? 1 : -1;
}

Map<Integer, Integer> freq = new HashMap<>(); // counting with a map
for (int x : a) freq.merge(x, 1, Integer::sum); // or: freq.put(x, freq.getOrDefault(x, 0) + 1)
```

A tiny example

```
int candidate = 0, count = 0;
for (int x : a) {
 if (count == 0) { candidate = x; count = 1; }
 else if (x == candidate) count++;
 else count--;
}
```

Guided example

MajorityBruteForce.java — Two nested loops (simple solution)

```
1  import java.util.Scanner;
2
3  public class MajorityBruteForce {
4   // Count how many times each element occurs (two loops). O(n^2).
5   static Integer majority(int[] a) {
6   int n = a.length;
7   for (int i = 0; i < n; i++) {
8   int count = 0;
9   for (int j = 0; j < n; j++) {                                    (1)
10  if (a[j] == a[i]) {
11  count++;
12  }
13  }
14  if (count > n / 2) { // strictly more than half                  (2)
15  return a[i];
16  }
17  }
18  return null;
19  }
20
21  public static void main(String[] args) {
22  Scanner sc = new Scanner(System.in);
23  int n = sc.nextInt();
24  int[] a = new int[n];
25  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
26  Integer m = majority(a);
27  System.out.println(m == null ? "No majority element" : "Majority element: " + m);
28  }
29 }
```

Counts how often each element occurs. Simple, but O(n²).

Sample input

```
10
4 7 4 4 7 4 4 9 4 3
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	For each candidate a[i], scan the whole array and count equal elements.
2	line 14	"More than half": strictly greater than n/2 (integer division works: for n = 5, n/2 = 2, need ≥ 3).

MajoritySorting.java — Sort, then count runs (simple solution)

```
1  import java.util.Arrays;
2  import java.util.Scanner;
3
4  public class MajoritySorting {
5   // After sorting, equal values sit together, so one pass counts every run. O(n log n).
6   static Integer majority(int[] original) {
7   int[] a = original.clone();                                    (1)
8   Arrays.sort(a);                                                (2)
9   int n = a.length;
10  int run = 1;
11  for (int i = 1; i <= n; i++) {
12  if (i < n && a[i] == a[i - 1]) {
13  run++;
14  } else {
15  if (run > n / 2) {                                             (3)
16  return a[i - 1];
17  }
18  run = 1;
19  }
20  }
21  return null;
22  }
23
24  public static void main(String[] args) {
25  Scanner sc = new Scanner(System.in);
26  int n = sc.nextInt();
27  int[] a = new int[n];
28  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
29  Integer m = majority(a);
30  System.out.println(m == null ? "No majority element" : "Majority element: " + m);
31  }
32 }
```

After sorting, equal values are adjacent, so one pass counts every run. O(n log n).

Sample input

```
10
1 7 8 2 6 8 1 3 2 8
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	Sort a copy so the caller's array is not changed.
2	line 8	O(n log n) dominates the running time.
3	line 15	Check at the end of every run of equal values.

BoyerMoore.java — Boyer–Moore majority vote (optimised solution)

```
1  import java.util.Scanner;
2
3  public class BoyerMoore {
4   // Boyer–Moore voting: O(n) time, O(1) space.
5   static Integer majority(int[] a) {
6   // Step 1: find a CANDIDATE. Each different pair of elements "cancels out"; a true
        majority survives.
7   Integer candidate = null;
8   int count = 0;
9   for (int x : a) {
10  if (count == 0) {                                                    1
11  candidate = x;
12  count = 1;
13  } else if (x == candidate) {                                         2
14  count++;
15  } else {
16  count--;                                                             3
17  }
18  }
19  if (candidate == null) {
20  return null;
21  }
22  // Step 2: VERIFY. The survivor is only a candidate: it need not be a majority.   4
23  int occurrences = 0;
24  for (int x : a) {
25  if (x == candidate) {
26  occurrences++;
27  }
28  }
29  return occurrences > a.length / 2 ? candidate : null;                5
30  }
31
32  public static void main(String[] args) {
33  Scanner sc = new Scanner(System.in);
34  int n = sc.nextInt();
35  int[] a = new int[n];
36  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
37  Integer m = majority(a);
38  System.out.println(m == null ? "No majority element" : "Majority element: " + m);
39  }
40 }
```

Phase 1 finds the only possible candidate; phase 2 confirms it.

Sample input

```
5
2 1 2 4 7
```

Line-by-line walkthrough

#	Where	What happens
1	line 10	No surviving voter: adopt the current element as the new candidate.
2	line 13	Same value: one more vote for the candidate.
3	line 16	Different value: the two cancel each other out.
4	line 22	The candidate is only a *possible* majority. Count its real occurrences.
5	line 29	Strict majority test; otherwise report no majority.

> **NOTE**
> The slide uses `{2,1,2,4,7}` to show why step 2 is needed: after step 1 the candidate is 7 with count 1, yet 7 appears only once in 5 elements. Try it: `BoyerMoore.in` contains exactly that array and prints "No majority element".

Dry run: watch it execute

Boyer–Moore: candidate and count after each element; the final pass verifies it.

Algorithm: Boyer–Moore majority vote. **Input:** arr = `4,7,4,4,7,4,4,9,4,3`.

Pseudocode

```
candidate = none; count = 0
for x in a:
 if count == 0: candidate = x; count = 1
 else if x == candidate: count++
 else count--
verify: occurrences(candidate) > n/2 ?
```

Step 1 of 12. Start with no candidate.

candidate = — · count = 0

array	4	7	4	4	7	4	4	9	4	3
	0	1	2	3	4	5	6	7	8	9

Step 2 of 12. count was 0: 4 becomes the candidate.

candidate = 4 · count = 1

array	4	7	4	4	7	4	4	9	4	3
	0	1	2	3	4	5	6	7	8	9

Step 3 of 12. 7 differs from 4: they cancel, count--.

candidate = 4 · count = 0

array	4	7	4	4	7	4	4	9	4	3
	0	1	2	3	4	5	6	7	8	9

Step 4 of 12. count was 0: 4 becomes the candidate.

candidate = 4 · count = 1

array	4	7	4	4	7	4	4	9	4	3
	0	1	2	3	4	5	6	7	8	9

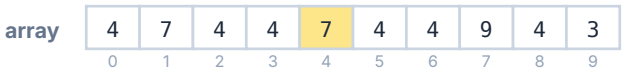
Step 5 of 12. 4 equals the candidate: count++.

candidate = 4 · count = 2



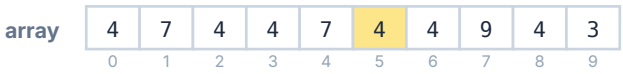
Step 6 of 12. 7 differs from 4: they cancel, count--.

candidate = 4 · count = 1



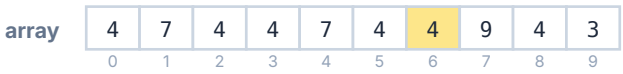
Step 7 of 12. 4 equals the candidate: count++.

candidate = 4 · count = 2



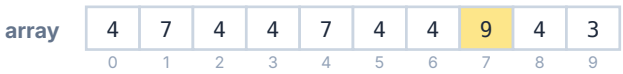
Step 8 of 12. 4 equals the candidate: count++.

candidate = 4 · count = 3



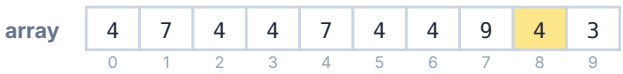
Step 9 of 12. 9 differs from 4: they cancel, count--.

candidate = 4 · count = 2



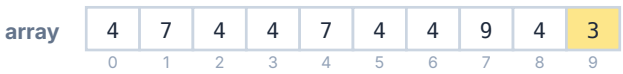
Step 10 of 12. 4 equals the candidate: count++.

candidate = 4 · count = 3



Step 11 of 12. 3 differs from 4: they cancel, count--.

candidate = 4 · count = 2



Step 12 of 12. Verify: 4 occurs 6 times; n/2 = 5. Majority confirmed.

candidate = 4 · occurrences = 6 · majority = true



> **RESULT**
> majority element = 4

Complexity

approach	time	space
two loops	O(n²)	O(1)
sorting	O(n log n)	O(1)–O(n)
HashMap of counts	O(n)	O(n)
Boyer–Moore	O(n)	O(1)

Boyer–Moore does exactly two passes over the array (2n comparisons) and uses two variables. This matches the slide's final statement: **Time O(n), Space O(1)**.

Common mistakes

Skipping the verification pass

Problem

```
return candidate; // after phase 1
```

Why: For {2, 1, 2, 4, 7} phase 1 ends with candidate 7, which is not a majority. Without phase 2 you report a wrong answer.

Fix: Count occurrences of the candidate and compare with n/2.

Using ≥ n/2 instead of > n/2

Problem

```
if (count >= n / 2)
```

Why: For {1, 2} the value 1 has exactly n/2 occurrences — not a majority.

Fix: if (count > n / 2)

Comparing Integer objects with ==

Problem

```
Integer candidate = ...;
if (x == candidate) ...
```

Why: When one side is an Integer object and the other an int, Java unboxes — fine; but comparing two Integer objects with == compares references (works only for −128..127).

Fix: Use int primitives, or candidate.equals(x).

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Majority with a HashMap — *Beginner*

Read n and an array. Print the majority element or the text `No majority element`.

Example input	Expected output
5 ⏎ 2 2 1 1 1	1

Hint 1: Count frequencies with a HashMap<Integer,Integer>. **Hint 2:** Majority means more than n/2.

Exercise 2 · Boyer–Moore — *Intermediate*

Same task, but use O(1) extra memory: no sorting, no hash map.

- $1 \le n \le 10^6$

Example input	Expected output
7 ⏎ 3 3 4 2 4 4 2	No majority element

Hint 1: Two variables: candidate and count. **Hint 2:** Verify with a second pass.

Exercise 3 · Elements appearing more than n/3 times — *Challenge*

Print all values that occur more than $\lfloor n/3 \rfloor$ times, in increasing order, or `None`.

Example input	Expected output
8 ⏎ 1 1 1 3 3 2 2 2	1 2

Hint 1: At most two values can exceed n/3. **Hint 2:** Keep TWO candidates with two counters; decrement both when a third value arrives. **Hint 3:** Verify both candidates in a second pass.

Quick check

Answers are in the answer key at the end of this part.

Q1. In an array of 9 elements, how many occurrences make a value the majority?

- **A.** at least 4
- **B.** at least 5
- **C.** exactly 4
- **D.** at least 9

Q2. Run phase 1 of Boyer–Moore on {2, 1, 2, 4, 7}.

```
for (int x : a) {
 if (count == 0) { candidate = x; count = 1; }
 else if (x == candidate) count++;
 else count--;
}
```

- count after 2nd element (1): _____
- candidate after 3rd element (2): _____
- count after 4th element (4): _____
- final candidate: _____

Q3. Why is the second pass of Boyer–Moore needed?

- **A.** To sort the array
- **B.** The candidate may not be a majority
- **C.** To reset the counter
- **D.** To avoid overflow

Q4. Which approach uses O(1) extra space and O(n) time?

- **A.** HashMap counting
- **B.** Sorting
- **C.** Boyer–Moore
- **D.** Two loops

Q5. Sorting-based majority detection costs...

- **A.** O(n)
- **B.** O(n log n)
- **C.** $O(n^2)$
- **D.** O(log n)

Interview / exam questions

- Prove that Boyer–Moore returns the majority element if one exists.
- How would you find elements appearing more than n/k times?
- What if the array is too large to fit in memory?
- Why does the final verification matter in an interview answer?

Watch & read

Video: Majority Element I | Brute-Better-Optimal | Moore's Voting Algorithm | Intuition |Brute to Optimal — take U forward · 18:13. *Boyer–Moore majority vote*

Video: Majority Element - Leetcode 169 - Python — NeetCode · 14:39. *Majority element solutions (Python — the idea transfers to Java)*

REVISION CARD (30–60 SECONDS)

- Majority: count > n/2.
- Boyer–Moore: candidate/count; count == 0 → adopt; same → ++; different → −−.
- ALWAYS verify the candidate with a second pass.
- O(n) time, O(1) space; > n/3 needs two candidates.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide cites https://www.youtube.com/watch?v=fCGLjIsEsNA as the video for this topic. YouTube's own metadata shows that video is titled "Maximum average subarray of size k" (IDeserve) — it is not about majority elements. It is NOT used in this library.

ASSUMPTIONS MADE

- The slide deck is a set of images from an IDeserve lecture; the Java fragments in the screenshots are reconstructed into the complete program `BoyerMoore.java`.

5.16 Maximum Product Subarray

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 50 min · ★ on the minimum-viable path

Find the contiguous subarray with the largest product — negatives and zeros make this trickier than the sum version.

YOU WILL BE ABLE TO

- Define subarray and count them: n(n+1)/2.
- Solve with brute force in O(n²) and understand the cost.
- Explain why you must track BOTH the maximum and minimum product ending at each position.
- Handle zeros, all-negative arrays, and overflow, and verify against brute force.

Before this lesson: One-Dimensional Arrays · Loops: for, while, do-while, break and continue · Type Conversion, Casting and Overflow

The idea

A **subarray** is a block of consecutive elements. `{4, 0, −2, 6}` has 4·5/2 = 10 subarrays (1 + 2 + 3 + 4). Their products are 4, 0, 0, 0, 0, 0, −2, −12, 0 and 6 — so the maximum is **6** (the slide lists them all).

For sums, "best subarray ending here" needs only one number. For products it needs two, because multiplying by a **negative** number flips big positive into big negative and the other way round:

```
a = { −5, −4, −1, −2 }
 max ending here min ending here
 −5 −5 −5
 −4 20 (−5·−4) −4
 −1 4 (−4·−1) −20 (20·−1)
 −2 40 (−20·−2) −8 (4·−2) answer = 40
```

WHY THIS WORKS

A subarray ending at i is either just a[i] or a[i] times a subarray ending at i−1. If a[i] > 0 we want the biggest product ending at i−1 to make the biggest result, and the smallest to make the smallest result. If a[i] < 0 the roles are swapped: the **smallest** (most negative) previous product becomes the biggest. A zero resets both. That is why `max` and `min` are updated together and swapped when a[i] is negative.

Syntax

```
long best = a[0], curMax = a[0], curMin = a[0]; // products can overflow int: use long
for (int i = 1; i < a.length; i++) {
 long x = a[i];
 long hi = curMax * x, lo = curMin * x; // a negative x swaps the roles of max and min
 curMax = Math.max(x, Math.max(hi, lo));
 curMin = Math.min(x, Math.min(hi, lo));
 best = Math.max(best, curMax);
}
```

A tiny example

```
long mx = a[0], mn = a[0], best = a[0];
for (int i = 1; i < n; i++) {
 long x = a[i];
 if (x < 0) { long t = mx; mx = mn; mn = t; }
 mx = Math.max(x, mx * x);
 mn = Math.min(x, mn * x);
 best = Math.max(best, mx);
}
```

Guided example

MaxProductBrute.java — Brute force over all subarrays (simple solution)

```
1  import java.util.Scanner;
2
3  public class MaxProductBrute {
4   // Try every subarray (start i, end j): there are n(n+1)/2 of them. O(n^2).
5   static long maxProduct(int[] a) {
6   long best = a[0];
7   for (int i = 0; i < a.length; i++) {
8   long product = 1;                                                    (1)
9   for (int j = i; j < a.length; j++) {
10  product *= a[j]; // product of a[i..j], extended one element at a time  (2)
11  best = Math.max(best, product);                                      (3)
12  }
13  }
14  return best;
15  }
16
17  public static void main(String[] args) {
18  Scanner sc = new Scanner(System.in);
19  int n = sc.nextInt();
20  int[] a = new int[n];
21  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
22  System.out.println(maxProduct(a));
23  }
24 }
```

For each start i, extend the end j and keep a running product.

Sample input

```
4
4 0 -2 6
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	Product of the empty prefix; reset for every new start index.
2	line 10	Extending the subarray by one element needs only one multiplication, not a recomputation.
3	line 11	Keep the largest product seen over all (i, j).

MaxProduct.java — Max/min product ending at each index (optimised solution)

```
1  import java.util.Scanner;
2
3  public class MaxProduct {
4   // Keep the best AND the worst product of a subarray ending at i. A negative number
        turns the
5   // worst (most negative) product into the best one, so both are needed. O(n) time, O(1)
        space.
6   static long maxProduct(int[] a) {
7   long maxEnding = a[0];                                                    (1)
8   long minEnding = a[0];                                                    (2)
9   long best = a[0];
10  for (int i = 1; i < a.length; i++) {
11  long x = a[i];
12  if (x < 0) { // multiplying by a negative swaps the roles                 (3)
13  long t = maxEnding;
14  maxEnding = minEnding;
15  minEnding = t;
16  }
17  maxEnding = Math.max(x, maxEnding * x); // extend the previous run, or start fresh at x  (4)
18  minEnding = Math.min(x, minEnding * x);
19  best = Math.max(best, maxEnding);                                         (5)
20  }
21  return best;
22  }
23
24  public static void main(String[] args) {
25  Scanner sc = new Scanner(System.in);
26  int n = sc.nextInt();
27  int[] a = new int[n];
28  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
29  System.out.println(maxProduct(a));
30  }
31 }
```

One pass, O(1) space. Swapping on negatives replaces the slide's three-way if/else.

Sample input

```
4
-5 -4 -1 -2
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	Best product of a subarray ending at the current index.
2	line 8	Worst (most negative) product ending here — it may become the best after a negative factor.
3	line 12	A negative factor reverses the order of products: swap the roles.
4	line 17	Either start fresh at x (the earlier product hurts) or extend.
5	line 19	Track the overall best.

MaxProductCheck.java — Randomised check (demonstration)

```
import java.util.Random;

public class MaxProductCheck {
 static long brute(int[] a) {
 long best = Long.MIN_VALUE;
 for (int i = 0; i < a.length; i++) {
 long p = 1;
 for (int j = i; j < a.length; j++) {
 p *= a[j];
 best = Math.max(best, p);
 }
 }
 return best;
 }

 static long fast(int[] a) {
 long mx = a[0], mn = a[0], best = a[0];
 for (int i = 1; i < a.length; i++) {
 long x = a[i];
 if (x < 0) {
 long t = mx;
 mx = mn;
 mn = t;
 }
 mx = Math.max(x, mx * x);
 mn = Math.min(x, mn * x);
 best = Math.max(best, mx);
 }
 return best;
 }

 // 20 000 random arrays with values in [-4, 4] (zeros and negatives included) against
     brute force.
 public static void main(String[] args) {
 Random rnd = new Random(2024);
 for (int t = 0; t < 20000; t++) {
 int n = 1 + rnd.nextInt(9);
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = rnd.nextInt(9) - 4;
 if (brute(a) != fast(a)) {
 System.out.println("MISMATCH on " + java.util.Arrays.toString(a));
 return;
 }
 }
 System.out.println("fast version matches brute force on 20000 random arrays");
 }
}
```

20 000 random arrays over {−4..4} including zeros; the fast and brute versions must agree.

Line-by-line walkthrough

#	Where	What happens
1	line 38	Values −4..4: lots of zeros and sign changes, which is where bugs hide.

Dry run: watch it execute

The slide's execution trace: max and min after each element.

Algorithm: Maximum product subarray. **Input:** arr = `-5,-4,-1,-2`.

Pseudocode

```
mx = mn = best = a[0]
for i = 1..n-1:
 if a[i] < 0: swap(mx, mn)
 mx = max(a[i], mx * a[i])
 mn = min(a[i], mn * a[i])
 best = max(best, mx)
```

Step 1 of 4. Start: max = min = best = a[0] = -5.

max = -5 · min = -5 · best = -5

array	−5	−4	−1	−2
	0	1	2	3

Step 2 of 4. a[1] = -4. Negative: swap max and min. max = 20, min = -4, best = 20.

max = 20 · min = -4 · best = 20

array	−5	−4	−1	−2
	0	1	2	3

Step 3 of 4. a[2] = -1. Negative: swap max and min. max = 4, min = -20, best = 20.

max = 4 · min = -20 · best = 20

array	−5	−4	−1	−2
	0	1	2	3

Step 4 of 4. a[3] = -2. Negative: swap max and min. max = 40, min = -8, best = 40.

max = 40 · min = -8 · best = 40

array	−5	−4	−1	−2
	0	1	2	3

RESULT

maximum product = 40

Complexity

Brute force: n(n+1)/2 subarrays, O(1) each with the running product → **$O(n^2)$ time**, O(1) space.

Max/min: a single pass → **O(n) time, O(1) space**.

WATCH OUT

Products grow fast: 2^{63} is reached after only 63 elements equal to 2. Use `long` (or BigInteger) and state the constraints; the answer is guaranteed to fit in the problem statement.

Common mistakes

Tracking only the maximum

Problem

```
maxEnding = Math.max(a[i], maxEnding * a[i]);
```

Why: For {−2, 3, −4} the answer is 24, but a max-only version stores 3 then 3·(−4) = −12 → 3·... it misses (−2·3·−4). The minimum −6 is needed.

Fix: Keep maxEnding AND minEnding; swap when a[i] < 0.

Hard-coding the length (slide)

Problem

```
for (int i = 1; i < 4; i++) // array happens to have 4 elements
```

Why: Any other array length silently skips elements or throws.

Fix: for (int i = 1; i < a.length; i++)

Initialising best with 0

Problem

```
long best = 0;
```

Why: For {−3} the answer is −3 (a subarray must contain at least one element) but best stays 0.

Fix: long best = a[0];

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Count subarrays with positive product — *Beginner*

Read n and an array. Print the number of subarrays whose product is strictly positive.

- 1 ≤ n ≤ 2000

Example input	Expected output
5 ⏎ 2 3 −2 4 0	4

Hint 1: A subarray is positive when it has no zero and an even number of negatives. **Hint 2:** For each start extend until a zero appears.

Exercise 2 · Maximum product subarray — *Intermediate*

Read n and an array of integers. Print the largest product of a contiguous non-empty subarray.

- 1 ≤ n ≤ 10^5
- −10 ≤ a[i] ≤ 10
- the answer fits in a 64-bit signed integer

Example input	Expected output
4 ⏎ 2 3 −2 4	6

Hint 1: Keep both max and min ending at i. **Hint 2:** Swap them when a[i] is negative.

Exercise 3 · Which subarray? — *Challenge*

Print three numbers: the maximum product, then the start and end index (0-based) of the first subarray that achieves it (earliest start, then earliest end).

- 1 ≤ n ≤ 1000

Example input	Expected output
4 ⏎ 2 3 −2 4	6 0 1

Hint 1: The brute force O(n²) is fine for n ≤ 1000. **Hint 2:** Only update on a STRICTLY larger product to keep the first one.

Quick check

Answers are in the answer key at the end of this part.

Q1. How many subarrays does an array of 5 elements have?

- **A.** 10
- **B.** 15
- **C.** 20
- **D.** 25

Q2. What does the lesson's algorithm return for {−2, 3, −4}?

```
// max/min product ending at each index
```

Your answer: _____

Q3. Why do we keep the MINIMUM product ending at i?

- **A.** To find the minimum answer
- **B.** A negative number can turn it into the maximum
- **C.** To detect zeros
- **D.** For overflow protection

Q4. Time complexity of the max/min algorithm?

- **A.** O(n)
- **B.** O(n log n)
- **C.** O(n²)
- **D.** O(2ⁿ)

Q5. Max product of {0, 2}?

```
// a zero resets the running products
```

Your answer: _____

Interview / exam questions

- Why does the maximum-sum subarray (Kadane) not directly work for products?
- How do zeros and negatives affect the algorithm?
- What if the numbers are huge — how do you avoid overflow?
- Solve it by splitting at zeros and counting negatives.

Watch & read

Video: Maximum Product Subarray - Best Intuitive Approach Discussed — take U forward · 20:27. *Tracking max and min products*

Video: Maximum Product Subarray - Dynamic Programming - Leetcode 152 — NeetCode · 15:31. *Kadane-style solution*

REVISION CARD (30–60 SECONDS)

- Keep max & min product ending here; swap if a[i] < 0.
- max = max(a[i], max·a[i]); min = min(a[i], min·a[i]); best = max(best, max).
- Zero resets; initialise from a[0], not 0.
- O(n) time, O(1) space; use long.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide prints `{4, 0, −2, 6}` products with a stray "x1 = …" notation, but its conclusion (6) is correct. • The slide code loops `i < 4` regardless of array length and uses a three-way branch with a temp variable; the swap form in this lesson is equivalent and shorter.

5.17 Maximum Equilibrium Sum

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 35 min · ★ on the minimum-viable path

Find indices where the left sum equals the right sum, and report the largest such sum — in one pass with a running total.

YOU WILL BE ABLE TO

- Define an equilibrium index and find one by hand.
- Solve with brute force, then with a prefix/running sum in O(n).
- Compute the maximum equilibrium sum and distinguish the two common definitions.
- Handle negatives, a single element, and no equilibrium at all.

Before this lesson: One-Dimensional Arrays · Loops: for, while, do-while, break and continue

The idea

An **equilibrium index** is a position where the sum of everything to its left equals the sum of everything to its right (the element itself is on neither side). For `A = {−7, 1, 5, 2, −4, 3, 0}`: index 3 works because −7 + 1 + 5 = −1 and −4 + 3 + 0 = −1; index 6 works because the left side sums to 0 and the right side is empty (sum 0).

The **maximum equilibrium sum** is the largest of those equal side-sums. For `{1,2,3,4,5,4,3,2,1}` the only equilibrium is the middle 5, with both sides summing to **10**.

```
total = 25 left right = total − left − a[i]
i=0 a=1 left=0 right=24 0 24 no
i=1 a=2 left=1 right=22 1 22 no
i=2 a=3 left=3 right=19
i=3 a=4 left=6 right=15
i=4 a=5 left=10 right=10 ← equal! equilibrium, sum 10
```

WHY THIS WORKS

Instead of re-adding the right side for every i, note that `right = total - left - a[i]`. Keep `left` as a running sum and update it after testing each index. The total is computed once. This is the same idea as a prefix-sum array, but needs only one variable.

WATCH OUT

Two definitions are in circulation. The slides (and this lesson's main answer) report the side-sum *excluding* a[i]. Many online versions ask for the prefix sum *including* a[i] that equals the suffix sum including a[i]; the equilibrium indices are the same, but the reported sum is larger by a[i] (15 instead of 10 for the example). The program prints both.

Syntax

```
long[] prefix = new long[n + 1]; // prefix[i] = a[0] + … + a[i-1]
for (int i = 0; i < n; i++) prefix[i + 1] = prefix[i] + a[i];
long left = prefix[i]; // sum of a[0 .. i-1]
long right = prefix[n] - prefix[i + 1]; // sum of a[i+1 .. n-1]
long rangeSum = prefix[r + 1] - prefix[l]; // any range a[l..r] in O(1)
```

A tiny example

```
long left = 0;
for (int i = 0; i < n; i++) {
 long right = total - left - a[i];
 if (left == right) { /* equilibrium at i */ }
 left += a[i];
}
```

Guided example

The slide animates the algorithm on `{1, 3, 5, 2, 2}`: total = 13; at index 2 (value 5) the left sum is 4 and the right sum 13 − 4 − 5 = 4 → equilibrium, position 3 in 1-based numbering (`pos = i + 1`).

EquilibriumBrute.java — Brute force (simple solution)

```
1  import java.util.Scanner;
2
3  public class EquilibriumBrute {
4   // For every index i, add up everything to its left and everything to its right. O(n^2).
5   public static void main(String[] args) {
6   Scanner sc = new Scanner(System.in);
7   int n = sc.nextInt();
8   long[] a = new long[n];
9   for (int i = 0; i < n; i++) a[i] = sc.nextLong();
10  boolean found = false;
11  for (int i = 0; i < n; i++) {
12  long left = 0, right = 0;
13  for (int j = 0; j < i; j++) left += a[j];                                    (1)
14  for (int j = i + 1; j < n; j++) right += a[j];                               (2)
15  if (left == right) {                                                         (3)
16  System.out.println("equilibrium index " + i + " (each side sums to " + left + ")");
17  found = true;
18  }
19  }
20  if (!found) System.out.println("no equilibrium index");
21  }
22 }
```

For each index add the left part and the right part separately. O(n²).

Sample input

```
7
-7 1 5 2 -4 3 0
```

Line-by-line walkthrough

#	Where	What happens
1	line 13	Sum of everything strictly before i.
2	line 14	Sum of everything strictly after i.
3	line 15	Equilibrium: report the index and the side sum.

MaxEquilibriumSum.java — One pass (optimised solution)

```
1  import java.util.Scanner;
2
3  public class MaxEquilibriumSum {
4   // One pass with a running left sum. right = total - left - a[i]. O(n) time, O(1) space.
5   public static void main(String[] args) {
6   Scanner sc = new Scanner(System.in);
7   int n = sc.nextInt();
8   long[] a = new long[n];
9   long total = 0;
10  for (int i = 0; i < n; i++) {
11  a[i] = sc.nextLong();
12  total += a[i];                                                    (1)
13  }
14
15  long left = 0;
16  boolean found = false;
17  long bestExcluding = Long.MIN_VALUE; // the sum of ONE side (the slide's definition)
18  long bestIncluding = Long.MIN_VALUE; // prefix sum that also includes a[i] (a common
        textbook variant)
19  for (int i = 0; i < n; i++) {
20  long right = total - left - a[i];                                 (2)
21  if (left == right) {
22  found = true;
23  bestExcluding = Math.max(bestExcluding, left);                    (3)
24  bestIncluding = Math.max(bestIncluding, left + a[i]);             (4)
25  System.out.println("equilibrium index " + i + ": side sum = " + left + ", including
        a[i] = " + (left + a[i]));
26  }
27  left += a[i];                                                     (5)
28  }
29  if (!found) {
30  System.out.println("no equilibrium index");
31  } else {
32  System.out.println("max equilibrium sum (slide definition) = " + bestExcluding);
33  System.out.println("max prefix-including-element variant = " + bestIncluding);
34  }
35  }
36 }
```

Computes total once, then maintains left; right is derived. O(n) time, O(1) extra space.

Sample input

```
9
1 2 3 4 5 4 3 2 1
```

Line-by-line walkthrough

#	Where	What happens
1	line 12	First pass: the total of the whole array.
2	line 20	Right sum without looping: everything except the left part and a[i] itself.
3	line 23	The slide's definition: the side sum (left == right).
4	line 24	The alternative definition (prefix including a[i]).
5	line 27	Update AFTER the test, so that `left` means "sum before i" at the top of the loop.

Dry run: watch it execute

The slide's array: left sum and right sum at every index.

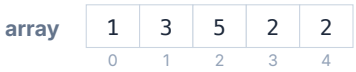
Algorithm: Equilibrium index. **Input:** arr = `1,3,5,2,2`.

Pseudocode

```
total = sum(a); left = 0
for i in 0..n-1:
 right = total - left - a[i]
 if left == right: equilibrium at i
 left += a[i]
```

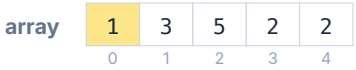
Step 1 of 6. total = 13.

total = 13 · left = 0



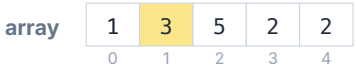
Step 2 of 6. left = 0, right = 12: not equal.

i = 0 · left = 0 · right = 12 · a[i] = 1



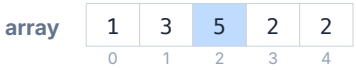
Step 3 of 6. left = 1, right = 9: not equal.

i = 1 · left = 1 · right = 9 · a[i] = 3



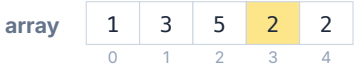
Step 4 of 6. left (4) == right (4): equilibrium index 2, side sum 4.

i = 2 · left = 4 · right = 4 · a[i] = 5



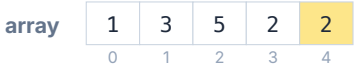
Step 5 of 6. left = 9, right = 2: not equal.

i = 3 · left = 9 · right = 2 · a[i] = 2



Step 6 of 6. left = 11, right = 0: not equal.

i = 4 · left = 11 · right = 0 · a[i] = 2



> **RESULT**
> maximum equilibrium sum = 4

Complexity

Brute force: for each of n indices two sums of up to n elements → **O(n²)** time, O(1) space.

One pass: two linear scans → **O(n) time, O(1) space** (apart from the input).

Use `long` for the sums: 10^5 values of 10^9 overflow `int`.

Common mistakes

Updating left before comparing

Problem

```
left += a[i];
if (left == total - left) ...
```

Why: Then `left` already includes a[i] and the comparison tests the "including" variant with a different right side. Mixing the two definitions produces off-by-one-element results.

Fix: Compare first, then add a[i] to left.

Returning Integer.MIN_VALUE when nothing is found (slide)

Problem

```
return maxSum; // MIN_VALUE if no equilibrium
```

Why: The caller prints a huge negative number as if it were a real answer. Report "none" explicitly.

Fix: Track a `found` flag.

int overflow

Problem

```
int total = 0; for (...) total += a[i];
```

Why: Large inputs wrap around silently and create fake equilibria.

Fix: long total

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · First equilibrium index — *Beginner*

Read n and an array. Print the smallest equilibrium index (0-based) or -1.

Example input	Expected output
7 ⏎ −7 1 5 2 −4 3 0	3

Hint 1: right = total − left − a[i]. **Hint 2:** Stop at the first match.

Exercise 2 · Maximum equilibrium sum — *Intermediate*

Print the largest side-sum over all equilibrium indices (left sum excluding a[i]), or the word `none`.

Example input	Expected output
9 ⏎ 1 2 3 4 5 4 3 2 1	10

Hint 1: Collect a maximum over the equilibria. **Hint 2:** Use a boolean for "found" instead of a sentinel.

Exercise 3 · Balanced split points — *Challenge*

Count the split positions k (1 ≤ k ≤ n−1) such that the sum of a[0..k−1] equals the sum of a[k..n−1].

- 2 ≤ n ≤ 10^5

Example input	Expected output
6 ⏎ 1 −1 1 −1 1 −1	2

Hint 1: Both parts together sum to total, so the prefix must equal total/2. **Hint 2:** One pass with a running prefix.

Quick check

Answers are in the answer key at the end of this part.

Q1. For {1, 3, 5, 2, 2}, the equilibrium index (0-based) is...

- **A.** 1
- **B.** 2
- **C.** 3
- **D.** none

Q2. What is `right` at index 1 of {4, 1, 2, 1}?

```
long total = 8, left = 4;
long right = total - left - 1;
System.out.println(right);
```

Your answer: _____

Q3. How do you get the right sum in O(1)?

- **A.** Loop again
- **B.** total − left − a[i]
- **C.** left × 2
- **D.** sort first

Q4. Time complexity of the single-pass solution?

- **A.** O(1)
- **B.** O(n)
- **C.** O(n log n)
- **D.** O(n^2)

Interview / exam questions

- Find the equilibrium index in O(n) time and O(1) space.
- What if the array has negative numbers and zeros — can there be several equilibrium indices?
- How do you handle overflow?

Watch & read

Video: Equilibrium Index of Array | O(n) Prefix-Sum Solution | An easy detailed solution — dsa-made-easy · 12:38. *Prefix sums and equilibrium*

Video: Prefix Sum Array and Range Sum Queries — Profound Academy · 7:30. *The prefix-sum technique*

REVISION CARD (30–60 SECONDS)

- Equilibrium index: sum(left) == sum(right), a[i] excluded.
- right = total − left − a[i]; update left AFTER the test.
- Max equilibrium sum = max of left over equilibria (none → report none).
- O(n) time, O(1) space; use long.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The Java program on the last slide returns Integer.MIN_VALUE when there is no equilibrium index; the lesson uses an explicit "none". • The deck explains the algorithm with a sequence of animated screenshots using the 1-based position `pos = i + 1`; Java indices here are 0-based.

ASSUMPTIONS MADE

• "Maximum equilibrium sum" is taken as the maximum of the equal side-sum (the deck's code). The prefix-including-a[i] reading is shown as a labelled variant.

5.18 Maximum Sum of an Hourglass in a Matrix

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 30 min · ★ on the minimum-viable path

Slide a fixed 7-cell hourglass pattern over a 2-D array and keep the best sum.

YOU WILL BE ABLE TO

- Describe the hourglass shape and index its seven cells from a top-left corner.
- Count the hourglasses in an R×C matrix: (R−2)(C−2).
- Write two nested loops with correct bounds and a correct initial maximum.
- Generalise with 2-D prefix sums to any k×k window.

Before this lesson: Two-Dimensional Arrays (Matrices) · Loops: for, while, do-while, break and continue

The idea

An **hourglass** in a matrix is the 7-cell shape: a row of three cells, the single cell below its middle, and another row of three cells.

The hourglass is anchored by its top-left corner (i, j).

```
a b c (i, j) (i, j+1) (i, j+2)
d (i+1, j+1)
e f g (i+2, j) (i+2, j+1)(i+2, j+2)
```

For `4×4` matrix `1 2 3 4 / 5 6 7 8 / 9 1 2 3 / 4 5 6 7` there are (4−2)·(4−2) = 4 hourglasses with sums 24, 22, 34 and **41** — the slide's answer.

WHY THIS WORKS

The top-left corner can be at any row 0 … R−3 and any column 0 … C−3 (the hourglass needs 2 extra rows and columns). That gives (R−2) × (C−2) positions, and the problem becomes: evaluate a 7-term sum at each position and keep the largest.

Syntax

```
for (int r = 0; r + 2 < rows; r++) { // top-left corner of a 3 x 3 window
 for (int c = 0; c + 2 < cols; c++) {
 int sum = m[r][c] + m[r][c+1] + m[r][c+2]
 + m[r+1][c+1]
 + m[r+2][c] + m[r+2][c+1] + m[r+2][c+2];
 best = Math.max(best, sum);
 }
}
int best = Integer.MIN_VALUE; // NOT 0: all sums may be negative
```

A tiny example

```
int s = m[i][j] + m[i][j+1] + m[i][j+2]
 + m[i+1][j+1]
 + m[i+2][j] + m[i+2][j+1] + m[i+2][j+2];
```

Guided example

HourglassMax.java — Maximum hourglass sum (optimised solution)

```
1  import java.util.Scanner;
2
3  public class HourglassMax {
4   // An hourglass has its top-left corner at (i, j):
5   // a b c (i, j) (i, j+1) (i, j+2)
6   // d (i+1, j+1)
7   // e f g (i+2, j) (i+2, j+1) (i+2, j+2)
8   static int hourglassSum(int[][] m, int i, int j) {                    (1)
9   return m[i][j] + m[i][j + 1] + m[i][j + 2]
10  + m[i + 1][j + 1]
11  + m[i + 2][j] + m[i + 2][j + 1] + m[i + 2][j + 2];
12  }
13
14  public static void main(String[] args) {
15  Scanner sc = new Scanner(System.in);
16  int rows = sc.nextInt();
17  int cols = sc.nextInt();
18  int[][] m = new int[rows][cols];
19  for (int i = 0; i < rows; i++) {
20  for (int j = 0; j < cols; j++) {
21  m[i][j] = sc.nextInt();
22  }
23  }
24  if (rows < 3 || cols < 3) {                                           (2)
25  System.out.println("Not possible");
26  return;
27  }
28  int best = Integer.MIN_VALUE; // NOT 0: every hourglass sum may be negative   (3)
29  int bestRow = 0, bestCol = 0;
30  for (int i = 0; i + 2 < rows; i++) { // (rows - 2) possible top rows   (4)
31  for (int j = 0; j + 2 < cols; j++) { // (cols - 2) possible left columns   (5)
32  int s = hourglassSum(m, i, j);
33  if (s > best) {
34  best = s;
35  bestRow = i;
36  bestCol = j;
37  }
38  }
39  }
40  System.out.println("hourglasses: " + (rows - 2) * (cols - 2));
41  System.out.println("maximum sum: " + best + " (top-left corner at row " + bestRow + ",
        column " + bestCol + ")");
42  }
43 }
```

Two nested loops over the valid top-left corners; O(R·C) time.

Sample input

```
4 4
1 2 3 4
5 6 7 8
9 1 2 3
4 5 6 7
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	Extracting the 7-term sum into a method makes the loops easy to read and the pattern easy to change.
2	line 24	No hourglass fits. Report it and stop (the slide calls System.exit(0) inside the method — returning is cleaner).
3	line 28	NOT 0: if every hourglass sum is negative, starting at 0 would report 0, which is not the sum of any hourglass.
4	line 30	Top row of the hourglass: it needs rows i, i+1, i+2.
5	line 31	Left column: it needs columns j, j+1, j+2.

Dry run: watch it execute

The four hourglasses of the slide's 4×4 example.

Algorithm: Hourglass sums. **Input:** rows = `4`, cols = `4`, data = `1 2 3 4 5 6 7 8 9 1 2 3 4 5 6 7`.

Pseudocode

```
best = -infinity
for i in 0..R-3:
 for j in 0..C-3:
 s = a[i][j..j+2] + a[i+1][j+1] + a[i+2][j..j+2]
 best = max(best, s)
```

Step 1 of 4. Hourglass with top-left corner (0, 0): 1 + 2 + 3 + 6 + 9 + 1 + 2 = 24. Best so far 24.

i = 0 · j = 0 · sum = 24 · best = 24

1	2	3	4
5	6	7	8
9	1	2	3
4	5	6	7

Step 2 of 4. Hourglass with top-left corner (0, 1): 2 + 3 + 4 + 7 + 1 + 2 + 3 = 22. Best so far 24.

i = 0 · j = 1 · sum = 22 · best = 24

1	2	3	4
5	6	7	8
9	1	2	3
4	5	6	7

Step 3 of 4. Hourglass with top-left corner (1, 0): 5 + 6 + 7 + 1 + 4 + 5 + 6 = 34. Best so far 34.

i = 1 · j = 0 · sum = 34 · best = 34

1	2	3	4
5	6	7	8
9	1	2	3
4	5	6	7

Step 4 of 4. Hourglass with top-left corner (1, 1): 6 + 7 + 8 + 2 + 5 + 6 + 7 = 41. Best so far 41.

i = 1 · j = 1 · sum = 41 · best = 41

1	2	3	4
5	6	7	8
9	1	2	3
4	5	6	7

> **RESULT**
> maximum hourglass sum = 41

Complexity

(R−2)(C−2) hourglasses × 7 additions = **O(R·C) time**, O(1) extra space. For a fixed shape the constant 7 disappears into the O.

For a general k×k window the naive cost is O(R·C·k²); with a 2-D prefix-sum table it drops to O(R·C) (challenge exercise).

Common mistakes

Initial maximum of 0 (slide)

Problem

```
int max_sum = 0;
```

Why: With all-negative hourglass sums the program prints 0, which is wrong.

Fix: int max = Integer.MIN_VALUE;

Loop bounds off by two

Problem

```
for (int i = 0; i < rows; i++) // then m[i + 2][...] → exception
```

Why: The pattern reaches two rows and two columns beyond the corner.

Fix: for (int i = 0; i + 2 < rows; i++)

Using the centre cell twice or the wrong middle row

Problem

```
m[i + 1][j] + m[i + 1][j + 1] + m[i + 1][j + 2]
```

Why: That is a full middle row (a 3×3 block), not an hourglass: only the CENTRE of the middle row belongs to it.

Fix: m[i + 1][j + 1]

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · One hourglass — *Beginner*

Read a 3×3 matrix (nine integers) and print its hourglass sum.

Example input	Expected output
`1 1 1 ⏎ 0 1 0 ⏎ 1 1 1`	7

Hint 1: Skip m[1][0] and m[1][2].

Exercise 2 · Maximum hourglass with negatives — *Intermediate*

Read R, C and the matrix (R, C ≥ 3). Print the maximum hourglass sum.

- 3 ≤ R, C ≤ 100
- −100 ≤ value ≤ 100

Example input	Expected output
`3 3 ⏎ −1 −1 −1 ⏎ −1 −9 −1 ⏎ −1 −1 −1`	−15

Hint 1: Start the maximum at Integer.MIN_VALUE.

Exercise 3 · Largest k×k block — *Challenge*

Read R, C, k and a matrix. Print the maximum sum of any k×k sub-matrix in O(R·C) time using 2-D prefix sums.

- 1 ≤ k ≤ min(R, C) ≤ 1000

Example input	Expected output
`3 3 2 ⏎ 1 2 3 ⏎ 4 5 6 ⏎ 7 8 9`	28

Hint 1: p[i][j] = sum of the rectangle (0,0)…(i−1,j−1). **Hint 2:** Block sum = p[i][j] − p[i−k][j] − p[i][j−k] + p[i−k][j−k].

Quick check

Answers are in the answer key at the end of this part.

Q1. How many hourglasses fit in a 5×6 matrix?

- **A.** 12
- **B.** 15
- **C.** 20
- **D.** 30

Q2. Which cells belong to the hourglass anchored at (i, j)?

- **A.** the full 3×3 block
- **B.** two full rows of 3 and the centre cell
- **C.** the main diagonals
- **D.** a plus sign

Q3. What is the hourglass sum of this 3×3? 1 2 3 / 4 5 6 / 7 8 9

```
// a b c / d / e f g
```

Your answer: _____

Q4. Why initialise the maximum with Integer.MIN_VALUE?

- **A.** It is faster
- **B.** All sums may be negative
- **C.** Arrays start at 0
- **D.** Required by Java

Interview / exam questions

- How many hourglasses are in an R×C matrix and why?
- How would you generalise to a k×k window in O(R·C)?
- What if the matrix is stored row by row in a 1-D array?

Watch & read

Video: 2D Array- DS - Hourglass |ARRAYS |HACKERRANK Solution| PYTHON | Interview |Programming for Beginners — Atharv Mudur · 6:53. *Scanning a 2D array for hourglasses (Python — the idea transfers to Java)*

Video: C_56 2D Arrays Program 1| Program to print Matrix and calculate Sum | C Programming — Jenny's Lectures CS IT · 17:33. *Window sums in a matrix (C — the idea transfers to Java)*

REVISION CARD (30–60 SECONDS)

- Hourglass at (i, j): m[i][j..j+2], m[i+1][j+1], m[i+2][j..j+2].
- Positions: i in 0..R−3, j in 0..C−3 → (R−2)(C−2).
- Initial max = Integer.MIN_VALUE.
- O(R·C) time, O(1) space.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide initialises `max_sum = 0` (wrong for negative data) and exits the JVM with `System.exit(0)` inside the helper. The lesson program avoids both. • The slide's worked sums (24, 22, 34, 41) are correct.

5.19 Toggle the Switch (100 Doors)

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 40 min · ★ on the minimum-viable path

Monkeys flip doors by multiples. Why do exactly the perfect squares stay open?

YOU WILL BE ABLE TO

- Simulate the doors puzzle with nested loops and a boolean array.
- Explain why a door is toggled once per divisor of its number.
- Prove that only perfect squares have an odd number of divisors and write an O(1) solution.
- Compute integer square roots safely for 64-bit inputs.

Before this lesson: Loops: for, while, do-while, break and continue · One-Dimensional Arrays · Divisibility, GCD/LCM, Primes and Modular Arithmetic

The idea

There are 100 closed doors and 100 monkeys. Monkey 1 toggles every door; monkey 2 toggles doors 2, 4, 6, ...; monkey 3 toggles 3, 6, 9, ...; monkey k toggles every k-th door. How many doors are open at the end?

Door number **n** is touched by monkey k exactly when **k divides n**. So door 12 is toggled by monkeys 1, 2, 3, 4, 6, 12 — six times → closed again. Door 16 by monkeys 1, 2, 4, 8, 16 — five times → **open**.

WHY THIS WORKS

Divisors come in pairs (d, n/d). Door 12: (1,12), (2,6), (3,4) — an even count, so it ends closed. The count is odd only if one pair is "d with itself", i.e. d = n/d, i.e. $n = d^2$ is a perfect square. Hence the open doors are exactly 1, 4, 9, ..., 100: **10 doors**, and for n doors the answer is $\lfloor\sqrt{n}\rfloor$.

```
door: 1 2 3 4 5 6 7 8 9 10 ...
toggles: 1 2 2 3 2 4 2 4 3 4
final: O c c O c c c c O c (O = open: 1, 4, 9)
```

Syntax

```
boolean[] open = new boolean[n + 1]; // index 1..n, all closed
for (int pass = 1; pass <= n; pass++)
 for (int door = pass; door <= n; door += pass) // every pass-th door
 open[door] = !open[door]; // toggle
int count = (int) Math.sqrt(n); // the closed form: perfect squares <= n
```

A tiny example

```
for (int monkey = 1; monkey <= n; monkey++)
 for (int door = monkey; door <= n; door += monkey)
 open[door] = !open[door];
```

Guided example

DoorsSimulation.java — Doors by simulation (simple solution)

```
1  import java.util.Scanner;
2
3  public class DoorsSimulation {
4   // Monkey i visits doors i, 2i, 3i, ... and toggles each one.
5   public static void main(String[] args) {
6   Scanner sc = new Scanner(System.in);
7   int n = sc.nextInt();
8   boolean[] open = new boolean[n + 1]; // all doors start closed
9   for (int monkey = 1; monkey <= n; monkey++) {
10  for (int door = monkey; door <= n; door += monkey) {
11  open[door] = !open[door];
12  }
13  }
14  StringBuilder sb = new StringBuilder();
15  int count = 0;
16  for (int door = 1; door <= n; door++) {
17  if (open[door]) {
18  count++;
19  sb.append(door).append(' ');
20  }
21  }
22  System.out.println("open doors: " + count);
23  System.out.println(sb.toString().trim());
24  }
25 }
```

1 2 3

A boolean array and the two nested loops of the puzzle.

Sample input

```
100
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	Index = door number (1-based), false = closed.
2	line 10	Monkey k visits k, 2k, 3k, ...: step by k.
3	line 11	Toggle: open ↔ closed.

DoorsSquares.java — Doors by mathematics (optimised solution)

```
1  import java.util.Scanner;
2
3  public class DoorsSquares {
4   // A door is toggled once for every divisor of its number. Divisors come in pairs (d,
        n/d), so the
5   // count is odd only when d == n/d, i.e. the number is a perfect square. Open doors =
        perfect squares.
6   static long isqrt(long n) {
7   long r = (long) Math.sqrt((double) n);                                    (1)
8   while (r * r > n) r--;                                                    (2)
9   while ((r + 1) * (r + 1) <= n) r++;                                       (3)
10  return r;
11  }
12
13  public static void main(String[] args) {
14  Scanner sc = new Scanner(System.in);
15  long n = sc.nextLong();
16  System.out.println("open doors: " + isqrt(n));
17  }
18 }
```

The number of perfect squares ≤ n is $\lfloor\sqrt{n}\rfloor$. Works for n up to 10^{18}.

Sample input

```
100
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	Floating-point sqrt may be off by one for large n (double has 53 bits of precision).
2	line 8	Correct an overestimate...
3	line 9	...and an underestimate. Now r is exactly $\lfloor\sqrt{n}\rfloor$.

Dry run: watch it execute

After each monkey, which doors are open.

Algorithm: Toggle the doors. **Input:** n = `20`.

Pseudocode

```
for monkey = 1..n:
 for door = monkey; door <= n; door += monkey:
 open[door] = !open[door]
```

The full trace has 21 steps; 14 evenly spaced ones are shown. Run the rest in the website's Dry-run lab.

Step 1 of 21. All doors start closed.

monkey = 0 · open = 0

doors (O = open, c = closed)

c	c	c	c	c	c	c	c	c	c	c	c	c	c	c	c	c	c	c	c
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

Step 3 of 21. Monkey 2 toggles doors 2, 4, 6, … ≤ 20.

monkey = 2 · open = 10

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	0	c	0	c	0	c	0	c	0	c	0	c	0	c	0	c	0	c

Step 4 of 21. Monkey 3 toggles doors 3, 6, 9, … ≤ 20.

monkey = 3 · open = 10

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	c	c	0	0	0	c	c	c	0	0	0	c	c	c	0	0	0	c

Step 6 of 21. Monkey 5 toggles doors 5, 10, 15, … ≤ 20.

monkey = 5 · open = 13

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	c	0	c	0	0	0	c	0	0	c	0	c	0	0	0	0	0	c

Step 7 of 21. Monkey 6 toggles doors 6, 12, 18, … ≤ 20.

monkey = 6 · open = 12

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	c	0	c	c	0	0	c	0	0	0	0	c	0	0	0	c	0	c

Step 9 of 21. Monkey 8 toggles doors 8, 16, 24, … ≤ 20.

monkey = 8 · open = 10

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	c	0	c	c	c	c	c	0	0	0	0	0	0	c	0	c	0	c

Step 10 of 21. Monkey 9 toggles doors 9, 18, 27, … ≤ 20.

monkey = 9 · open = 12

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	c	0	c	c	c	c	0	0	0	0	0	0	0	c	0	0	0	c

Step 12 of 21. Monkey 11 toggles doors 11, 22, 33, … ≤ 20.

monkey = 11 · open = 11

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	c	0	c	c	c	c	0	c	c	0	0	0	0	c	0	0	0	0

Step 13 of 21. Monkey 12 toggles doors 12, 24, 36, … ≤ 20.

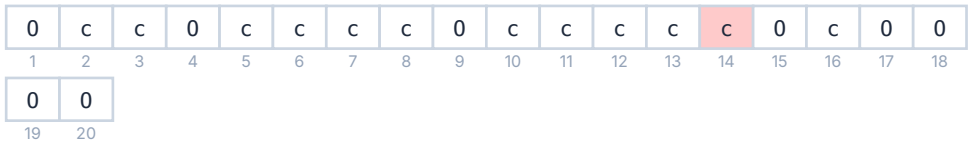
monkey = 12 · open = 10

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	c	0	c	c	c	c	0	c	c	c	0	0	0	c	0	0	0	0

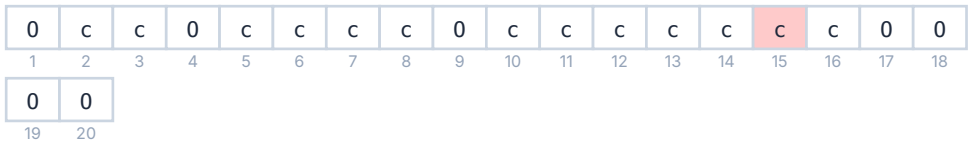
Step 15 of 21. Monkey 14 toggles doors 14, 28, 42, ... ≤ 20.

monkey = 14 · open = 8



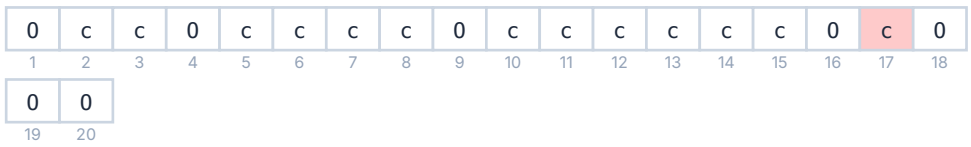
Step 16 of 21. Monkey 15 toggles doors 15, 30, 45, ... ≤ 20.

monkey = 15 · open = 7



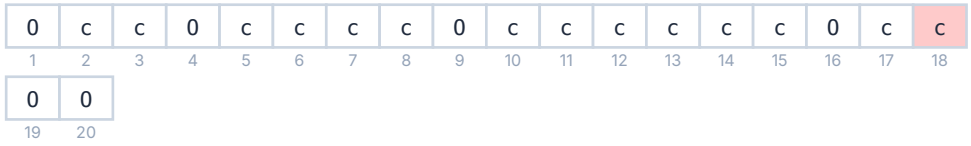
Step 18 of 21. Monkey 17 toggles doors 17, 34, 51, ... ≤ 20.

monkey = 17 · open = 7



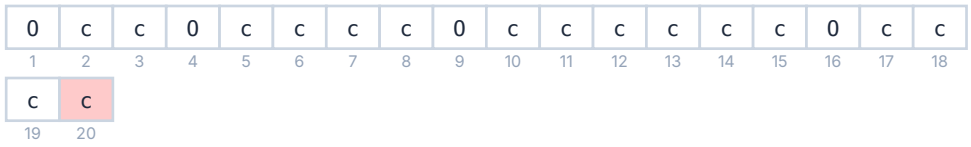
Step 19 of 21. Monkey 18 toggles doors 18, 36, 54, ... ≤ 20.

monkey = 18 · open = 6



Step 21 of 21. Monkey 20 toggles doors 20, 40, 60, ... ≤ 20.

monkey = 20 · open = 4



> **RESULT**
> 4 open doors: 1 4 9 16

Complexity

Simulation: monkey k does $\lfloor n/k \rfloor$ toggles; the total is $n \cdot (1 + 1/2 + \ldots + 1/n)$ = **O(n log n)** (harmonic series). Space O(n).

Mathematical: O(1) time and space (or O(log n) with a binary-search square root).

Common mistakes

Toggling door j instead of door j·i (the slide code)

Problem

```
for (i = 1; i <= n; i++)
 for (j = i; j * i <= n; j++)
 b[j] = !b[j]; // toggles door j, not door i*j
```

Why: The slide's loop never visits "every i-th door"; it toggles consecutive doors j with j·i ≤ n. For n = 100 it reports far more than 10 open doors.

Fix: for (j = i; j <= n; j += i) b[j] = !b[j];

Counting open doors with floating-point sqrt

Problem

```
int open = (int) Math.sqrt(n); // n up to 10^18
```

Why: For large n the double rounding can give 999999999 or 1000000001 instead of 10^9.

Fix: Adjust with the two while-loops after Math.sqrt.

Using int for n

Problem

```
int n = sc.nextInt(); // n = 10^18
```

Why: Does not fit in an int; Scanner throws InputMismatchException.

Fix: long n = sc.nextLong();

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Count the open doors — *Beginner*

Read n (≤ 1000) and simulate the monkeys. Print the number of doors that are open at the end.

Example input	Expected output
10	3

Hint 1: boolean[] and two loops.

Exercise 2 · A trillion doors — *Intermediate*

Read n up to 10^{18}. Print how many doors are open.

Example input	Expected output
1000000000000000000	1000000000

Hint 1: Open doors are perfect squares. **Hint 2:** Use an exact integer square root.

Exercise 3 · Only the first m monkeys — *Challenge*

Only monkeys 1..m are released (m ≤ n). Read n and m and print how many doors are open. (n ≤ 10^6)

Example input	Expected output
10 3	4

Hint 1: A door is toggled once per divisor ≤ m. **Hint 2:** Simulation over m monkeys is O(n log m).

Quick check

Answers are in the answer key at the end of this part.

Q1. After all 100 monkeys, how many doors are open?

- **A.** 5
- **B.** 10
- **C.** 25
- **D.** 50

Q2. Door 36 is toggled how many times?

- **A.** 8
- **B.** 9
- **C.** 10
- **D.** 6

Q3. Why is the number of divisors odd only for perfect squares?

- **A.** Primes have 2 divisors
- **B.** Divisors pair up as (d, n/d) except when d = n/d
- **C.** Squares are even
- **D.** Because of overflow

Q4. The simulation's time complexity is...

- **A.** O(n)
- **B.** O(n log n)
- **C.** $O(n^2)$
- **D.** $O(\sqrt{n})$

Interview / exam questions

- Why do only perfect squares stay open?
- How would you compute $\lfloor\sqrt{n}\rfloor$ exactly for n up to 10^{18}?
- What changes if the monkeys toggle only doors up to some limit?

Watch & read

Video: 100 Doors in a Row | Perfect Square Numbers Magic | Train your Brain | Puzzle — PRiyanka Singh · 9:49. *Why only perfect squares remain open*

Video: 100 Doors Interview Puzzle || Microsoft Interview Puzzle — Simply Logical · 5:26. *The 100-doors puzzle*

REVISION CARD (30–60 SECONDS)

- Monkey k toggles multiples of k; door n flips once per divisor.
- Odd number of divisors ⇔ perfect square.
- Open doors = $\lfloor\sqrt{n}\rfloor$.
- Simulation O(n log n); formula O(1).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide's `Toggle` program toggles b[j] for j from i while j·i ≤ n. That is not "every i-th door"; running it for n = 100 reports many more than 10 open doors. The lesson uses the correct inner loop `door += monkey`. • The deck combines two problems (this one and "Alice's Apple Tree"); each has its own lesson.

5.20 Alice's Apple Tree (Minimum Garden Perimeter)

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 45 min · complete path

Apples grow at every grid point; find the smallest square plot that contains enough — with a closed form and binary search.

YOU WILL BE ABLE TO

- Model the garden: tree (i, j) has |i| + |j| apples.
- Derive the apples in a ring and in a whole square: $12r^2$ and $2r(r+1)(2r+1)$.
- Find the smallest r with a loop, then with binary search on a monotone function.
- Use `long` everywhere and avoid overflow.

Before this lesson: Loops: for, while, do-while, break and continue · Toggle the Switch (100 Doors) · Searching: Linear and Binary Search

The idea

Alice stands at (0, 0) on an infinite grid. The tree at integer point (i, j) bears |i| + |j| apples. She wants a **square plot, centred at the origin and aligned with the axes**, that contains at least `neededApples` apples inside or on its border. What is the smallest possible **perimeter**?

A square with half-side r has corners (±r, ±r) and perimeter 8r. Going from half-side r−1 to r adds a "ring". Counting the apples in that ring gives **$12 \cdot r^2$** (the slide's `sum += 12*cnt*cnt`). Adding the rings 1..r:

$$\text{apples}(r) = \Sigma\ 12k^2\ (k = 1..r) = 2r(r+1)(2r+1) = 4r^3 + 6r^2 + 2r$$

That matches the slide's text: "The total apples is r·r·r·4 + r·r·6 + r·2." The answer is 8·r for the smallest r with apples(r) ≥ needed.

WHY THIS WORKS

apples(r) strictly increases with r, so the set of "good" values of r is an interval [r, ∞). *That monotone property is exactly what binary search** needs: test the middle, discard half. The loop version adds ring by ring and is just as correct but takes O(r) steps, and r can be 500 000 for needed = 10^{15}.

Syntax

```
long lo = 1, hi = 1_000_000; // search the answer (the radius) itself
while (lo < hi) {
 long mid = lo + (hi - lo) / 2;
 if (apples(mid) >= needed) hi = mid; else lo = mid + 1; // monotone yes/no test
}
return 8 * lo; // perimeter of the square
static long apples(long r) { return 2 * r * (r + 1) * (2 * r + 1); }
```

A tiny example

```
long apples = 0, r = 0;
while (apples < needed) {
 r++;
 apples += 12 * r * r; // the ring at distance r
}
return 8 * r;
```

Guided example

Check: needed = 13 → r = 2 (apples(1) = 12 < 13, apples(2) = 60) → perimeter 16. needed = 1 → r = 1 → 8. needed = 10^9 → r = 630 → perimeter **5040**.

AppleTreeLoop.java — Ring by ring (simple solution)

```
1  import java.util.Scanner;
2
3  public class AppleTreeLoop {
4   // Alice stands at the origin of an infinite grid; the tree at (i, j) has |i| + |j|
        apples.
5   // She buys a square plot [-r, r] x [-r, r] (perimeter 8r) and keeps every apple inside.
6   // The apples in the NEW ring when the plot grows from r-1 to r are exactly 12 * r * r.
7   public static void main(String[] args) {
8   Scanner sc = new Scanner(System.in);
9   long needed = sc.nextLong();                                                    (1)
10  long r = 0;
11  long apples = 0;
12  while (apples < needed) {
13  r++;
14  apples += 12 * r * r; // add the ring at distance r                             (2)
15  }
16  System.out.println("half-side r = " + r + ", total apples = " + apples + ", minimum   (3)
        perimeter = " + 8 * r);
17  }
18 }
```

Adds $12r^2$ for r = 1, 2, 3... until the total reaches the target. O(r).

Sample input

```
1000000000
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	Targets up to 10^{15} do not fit in int.
2	line 14	All apples in the new ring: the slide uses exactly this term. `r` is long so r*r cannot overflow int.
3	line 16	A square of half-side r has perimeter 8r.

AppleTreeFormula.java — Binary search (optimised solution)

```
1  import java.util.Scanner;
2
3  public class AppleTreeFormula {
4   // Apples inside the square of half-side r: 2 r (r + 1) (2 r + 1) (= 4r^3 + 6r^2 + 2r)
5   static long applesInside(long r) {                                                    (1)
6   return 2 * r * (r + 1) * (2 * r + 1);
7   }
8
9   public static void main(String[] args) {
10  Scanner sc = new Scanner(System.in);
11  long needed = sc.nextLong();
12  long lo = 1;
13  long hi = 200_000; // 2 * 200000 * 200001 * 400001 > 3 * 10^16, enough for needed <=   (2)
        10^15
14  while (lo < hi) { // smallest r with applesInside(r) >= needed
15  long mid = (lo + hi) / 2;
16  if (applesInside(mid) >= needed) {                                                    (3)
17  hi = mid;
18  } else {
19  lo = mid + 1;                                                                         (4)
20  }
21  }
22  System.out.println(8 * lo);
23  }
24 }
```

Uses apples(r) = 2r(r+1)(2r+1). O(log R).

Sample input

```
1000000000000000
```

Line-by-line walkthrough

#	Where	What happens
1	line 5	The closed form of the ring sums.
2	line 13	An upper bound for r: apples(200000) ≈ 3.2 × 10^{16}, which exceeds the largest allowed target of 10^{15}, and the product still fits in a long.
3	line 16	mid is enough: the answer is mid or smaller.
4	line 19	mid is too small: the answer is larger.

Dry run: watch it execute

Rings add 12, 48, 108, ... apples.

Algorithm: Alice's apple tree. **Input:** needed = `13`.

Pseudocode

```
r = 0; apples = 0
while (apples < needed) {
 r++
 apples += 12 * r * r
}
return 8 * r
```

Step 1 of 3. Need at least 13 apples.

r = 0 · apples = 0 · perimeter = 0

Step 2 of 3. Ring 1 adds $12 \cdot 1^2 = 12$ apples. Total 12.

r = 1 · apples = 12 · perimeter = 8

Step 3 of 3. Ring 2 adds $12 \cdot 2^2 = 48$ apples. Total 60.

r = 2 · apples = 60 · perimeter = 16

> **RESULT**
> r = 2, perimeter = 16

Complexity

Loop: r iterations where r ≈ (needed/4)^(1/3) → **O(needed^(1/3))**. For needed = 10^{15} that is about 63 000... (r = 503 968 at 10^{15} — roughly half a million iterations, still instant).

Binary search: $\log_2(200\,000) \approx 18$ iterations → **O(log R)**.

> **WATCH OUT**
> Intermediate products: $2 \cdot r \cdot (r+1) \cdot (2r+1)$ for r = 200 000 is about $3.2 \times 10^{16} << 9.2 \times 10^{18}$, so `long` is safe. With `int` it overflows already for r ≈ 800.

Common mistakes

Using int (the slide)

Problem

```
int apple = sc.nextInt();
int cnt = 0, sum = 0;
sum += 12 * cnt * cnt;
```

Why: The slide's code reads an int. $12 \cdot cnt^2$ overflows int for cnt > 13 000 and the target itself may be up to 10^{15}.

Fix: long needed; long r; long apples;

Returning r instead of the perimeter

Problem

```
System.out.println(r);
```

Why: The question asks for the perimeter 8r.

Fix: System.out.println(8 * r);

Binary search with an unsafe upper bound

Problem

```
long hi = 1000; // for needed up to 10^15
```

Why: apples(1000) ≈ $4 \cdot 10^9 < 10^{15}$ so the search cannot find the answer.

Fix: long hi = 200_000; or double hi until applesInside(hi) >= needed

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Apples in a square — *Beginner*

Read r. Print the number of apples inside the square with corners (±r, ±r).

Example input	Expected output
3	168

Hint 1: Use the closed form 2r(r+1)(2r+1). **Hint 2:** Check r = 1: 12.

Exercise 2 · Minimum perimeter (small) — *Intermediate*

Read neededApples (≤ 10^9) and print the minimum perimeter.

Example input	Expected output
13	16

Hint 1: Add the rings $12r^2$ until the total is ≥ needed.

Exercise 3 · Minimum perimeter (large) — *Challenge*

Read neededApples (≤ 10^{15}) and print the minimum perimeter, using binary search.

Example input	Expected output
1000000000000000	503968

Hint 1: apples(r) = 2r(r+1)(2r+1) is increasing. **Hint 2:** hi = 200000 is a valid upper bound.

Quick check

Answers are in the answer key at the end of this part.

Q1. How many apples are in the ring at distance r = 2?

- **A.** 24
- **B.** 48
- **C.** 60
- **D.** 12

Q2. What does applesInside(2) return?

```
static long applesInside(long r) {
 return 2 * r * (r + 1) * (2 * r + 1);
}
```

Your answer: _____

Q3. Why can binary search be used here?

- **A.** The array is sorted
- **B.** apples(r) is monotonically increasing
- **C.** r is always even
- **D.** The answer is a prime

Q4. What is the perimeter of the square with half-side r?

- **A.** 4r
- **B.** 8r
- **C.** r^2

- **D.** 2r + 2

Interview / exam questions

- Derive the number of apples in the r-th ring.
- Why is binary search valid and what is its upper bound?
- How do you avoid overflow?

Watch & read

Video: Minimum Garden Perimeter To Collect Enough Apples leetcode weekly contest 252 Leetcode 1954. — CodeinDepth · 9:51. *Counting apples by layers; binary search on the answer*

Video: Binary Search | Monotonic Function | L-29 — Programming Pathshala · 11:57. *Binary search on the answer*

REVISION CARD (30–60 SECONDS)

- Ring r holds $12r^2$ apples; square of half-side r holds $2r(r+1)(2r+1)$.
- Perimeter = 8r.
- Smallest r with apples(r) ≥ needed → loop or binary search.
- Use long: 10^{15} needs r ≈ 503 968.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide's code uses int and a linear loop although the text on the previous slide says "then we apply binary search". Both appear here; the long type is required for the full constraint range (this is LeetCode 1954, "Minimum Garden Perimeter to Collect Enough Apples").

ASSUMPTIONS MADE

- The problem statement is only given in diagrams on the slides; the lesson uses the standard statement (square centred at the origin, tree at (i, j) with |i| + |j| apples).

5.21 Weighted Substring (Smallest String of a Given Weight)

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 50 min · complete path

Letters have factorial-like weights; build the shortest, then alphabetically smallest, string with a given total weight.

YOU WILL BE ABLE TO

- Generate the letter weights A = 1, B = 3, C = 12, D = 60 … from the rule weight(n) = n·weight(n−1) + weight(n−1).
- Explain why the greedy "biggest letter first" gives the fewest letters.
- Sort the letters to get the lexicographically smallest string of that length.
- Use long (weights reach 10^{16}) and stop generating before overflow.

Before this lesson: Strings, StringBuilder and Characters · Searching: Linear and Binary Search · Sorting with Comparable and Comparator

The idea

The weight of a letter depends on the previous one: A = 1, B = 2·A + A = 3, C = 3·B + B = 12, D = 4·C + C = 60 … so each weight is **(position + 1) × previous weight** (B = 3·1, C = 4·3, D = 5·12). These are the numbers 2!/2, 3!/2, 4!/2, 5!/2 … = 1, 3, 12, 60, 360, 2520 …

The weight of a string is the sum of its letters' weights. Given a target weight W, find the **shortest** string with exactly that weight and, among shortest strings, the **lexicographically smallest**. For W = 25 the strings `ABBBBC`, `ACC`, `AAAAAAABBBBBB` all weigh 25; the shortest is `ACC`.

WHY THIS WORKS

Each weight divides the next one (1 | 3 | 12 | 60 | …), like coins in a currency where each coin is a multiple of the previous. For such "canonical" coin systems the **greedy** strategy — always take the largest coin that fits — uses the fewest coins. Which letters appear is therefore fixed; to make the string alphabetically smallest, simply put them in ascending order.

```
W = 20 largest weight ≤ 20 is C = 12 → C, remaining 8
 largest ≤ 8 is B = 3 → B, remaining 5
 B again → B, remaining 2
 A, A → remaining 0
letters C B B A A sorted → A A B B C
```

Syntax

```
char letter = (char) ('A' + index); // 0 -> 'A', 1 -> 'B', …
List<Character> letters = new ArrayList<>();
letters.add(letter);
Collections.sort(letters); // smallest letters first
StringBuilder sb = new StringBuilder();
for (char c : letters) sb.append(c);
String result = sb.toString();
long next = weight * (n + 1); // use long: weights grow factorially
```

A tiny example

```
long[] w = {1, 3, 12, 60, 360};
long remaining = 20;
for (int i = w.length - 1; i >= 0; i--)
 while (remaining >= w[i]) { /* take letter i */ remaining -= w[i]; }
```

Guided example

WeightedString.java — Smallest string of a given weight (optimised solution)

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Scanner;

public class WeightedString {
 // weight(A) = 1, weight(B) = 2*A + A, weight(C) = 3*B + B, ... i.e. weight(letter n) = (n + 1) * weight(letter n - 1)
 static long[] buildWeights(long limit) {
 List<Long> w = new ArrayList<>();
 long cur = 1; // weight of 'A'                                                    (1)
 w.add(cur);
 for (int n = 2; n <= 26; n++) {
 long next = cur * (n + 1); // (n+1) * previous weight, from "n*prev + prev"      (2)
 if (next > limit || next / (n + 1) != cur) break; // stop before exceeding the limit / overflowing   (3)
 w.add(next);
 cur = next;
 }
 long[] arr = new long[w.size()];
 for (int i = 0; i < arr.length; i++) arr[i] = w.get(i);
 return arr;
 }

 // Shortest string with the given weight, lexicographically smallest among those of that length.
 static String smallestString(long weight) {
 long[] w = buildWeights(weight);
 List<Character> letters = new ArrayList<>();
 long remaining = weight;
 for (int i = w.length - 1; i >= 0; i--) { // greedy: take the biggest letters first
 while (remaining >= w[i]) {                                                     (4)
 letters.add((char) ('A' + i));
 remaining -= w[i];
 }
 }
 Collections.sort(letters); // smallest letters first = lexicographically smallest   (5)
 StringBuilder sb = new StringBuilder();
 for (char c : letters) sb.append(c);
 return sb.toString();
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long weight = sc.nextLong();
 System.out.println(smallestString(weight));
 }
}
```

Builds weights up to the limit, takes letters greedily from the largest, then sorts.

Sample input

```
20
```

Line-by-line walkthrough

#	Where	What happens
1	line 10	weight(A) = 1.
2	line 13	weight(n) = (n+1) · weight(n−1): this is "n·prev + prev".
3	line 14	No letter heavier than the target is ever needed; stopping early also prevents overflow (26 letters would overflow long).
4	line 29	Greedy: use the current letter as many times as it fits.
5	line 34	Ascending order is the lexicographically smallest arrangement of this multiset.

> **NOTE**
> The slide uses a binary search (`findFloor`) to find the largest weight ≤ k each time. With at most 17 usable weights a simple backwards loop is just as fast; the binary search matters only if the alphabet were huge.

Dry run: watch it execute

Greedy from the heaviest letter: 100 = 60 + 3×12 + 3 + 1.

Algorithm: Smallest string of a given weight. **Input:** w = `100`.

Pseudocode

```
weights: A=1, B=3, C=12, D=60, E=360, …
remaining = weight
for letter from largest to smallest:
 while remaining >= weight(letter): take it
sort the letters (smallest first)
```

Step 1 of 6. Weights not exceeding 100: A=1, B=3, C=12, D=60.

weight = 100

letters	A	B	C	D
	0	1	2	3

weights	1	3	12	60
	0	1	2	3

Step 2 of 6. Letter D (weight 60) fits 1 time: take D. Remaining 40.

letter = D · taken = 1 · remaining = 40

letters	A	B	C	D
	0	1	2	3

letters taken so far	D
	0

Step 3 of 6. Letter C (weight 12) fits 3 times: take CCC. Remaining 4.

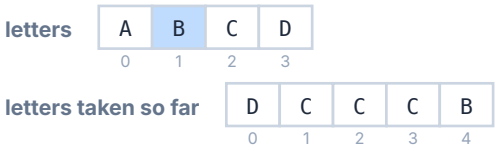
letter = C · taken = 3 · remaining = 4

letters	A	B	C	D
	0	1	2	3

letters taken so far	D	C	C	C
	0	1	2	3

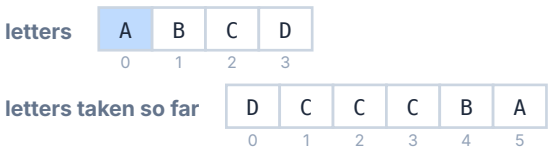
Step 4 of 6. Letter B (weight 3) fits 1 time: take B. Remaining 1.

letter = B · taken = 1 · remaining = 1



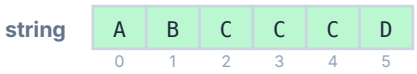
Step 5 of 6. Letter A (weight 1) fits 1 time: take A. Remaining 0.

letter = A · taken = 1 · remaining = 0



Step 6 of 6. Sort the letters so the string is lexicographically smallest: ABCCCD.

length = 6



> **RESULT**
> ABCCCD (6 letters)

Complexity

Weights grow factorially, so only about 17 letters are ever below 10^{16}. The greedy loop takes at most the sum over letters of (weight(next)/weight(this)) − 1 ≤ i + 1 copies each — at most ~160 letters. Time **O(L²)** or O(L log L) with the sort where L = answer length (tiny); space O(L).

Common mistakes

int weights (the slide)

Problem

```
static int[] values = new int[26];
```

Why: weight(M) = 14!/2 ≈ 4.4 × 10^{10} already overflows int and the constraint is up to 10^{16}.

Fix: long[] and a stopping rule

Computing all 26 weights

Problem

```
for (int i = 1; i < 26; i++) values[i] = (i + 1) * prev + prev;
```

Why: weight(Z) = 27!/2 ≈ 5 × 10^{27} overflows even long, giving negative nonsense that confuses the search.

Fix: Stop when the next weight exceeds the target.

Printing without sorting

Problem

```
print letters in the order they were chosen (largest first): "CBBAA"
```

Why: The task asks for the lexicographically smallest string, "AABBC".

Fix: sort ascending (the slide reverses its list, which gives the same effect)

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Weight of a string — *Beginner*

Read a string of uppercase letters and print its total weight (A=1, B=3, C=12, ..., with weight(n) = (n+1)·weight(n−1)).

- length ≤ 10
- only letters A–J

Example input	Expected output
ACC	25

Hint 1: Build the weight table once. **Hint 2:** Sum table[c − 'A'].

Exercise 2 · Smallest string — *Intermediate*

Read a weight W ($1 \le W \le 10^{16}$) and print the shortest string of weight W, choosing the lexicographically smallest among the shortest.

Example input	Expected output
25	ACC
20	AABBC

Hint 1: Greedy from the heaviest letter that fits. **Hint 2:** Sort at the end.

Exercise 3 · Normalise a string — *Challenge*

Read a string of uppercase letters (up to 10^5 characters). Print the shortest, lexicographically smallest string with the same total weight — without ever computing the (possibly huge) total.

Example input	Expected output
AAAAB	ABB

Hint 1: (i + 2) copies of letter i weigh the same as one copy of letter i + 1 (A: 3 copies = B; B: 4 copies = C). **Hint 2:** Count letters, then carry from A upwards like long addition.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is the weight of D?

- **A.** 12
- **B.** 36
- **C.** 60
- **D.** 120

Q2. Shortest string of weight 25?

- **A.** ABBBBC
- **B.** ACC
- **C.** AAAAAAABBBBBB
- **D.** BBBBBBBBA

Q3. What string does the lesson's method produce for weight 7?

```
// weights A=1 B=3 C=12
```

Your answer: _____

Q4. Why does greedy work for these weights?

- **A.** They are prime
- **B.** Each weight divides the next
- **C.** They are powers of two
- **D.** Because letters are sorted

Interview / exam questions

- Why is greedy optimal when each coin divides the next?
- Give a coin system where greedy fails.
- How would you handle weights up to 10^{18} or more?

Watch & read

Video: Smallest String With A Given Numeric Value | Leetcode 1663 | Maths | Live coding session — Coding Decoded · 6:44. *Greedy construction of the smallest string*

Video: Introduction to Greedy Algorithms | GeeksforGeeks — GeeksforGeeks · 5:32. *Greedy reasoning*

REVISION CARD (30–60 SECONDS)

• weight(n) = (n+1)·weight(n−1): 1, 3, 12, 60, 360 ...
• Greedy largest-first = fewest letters; sort for lexicographic order.
• Use long; stop generating weights at the target.
• Answer length is tiny (≤ ~160).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide says the output "should be in the form of integer" but its sample output is the string AABBC for input 20 (a string, not an integer). • The slide stores weights in int[26] (overflows) and reads the weight with nextInt although the constraint is up to 10^{16}.

5.22 Move Hyphen to the Beginning

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 30 min · ★ on the minimum-viable path

Rearrange a string with `indexOf`, `substring` and StringBuilder — then do it for every hyphen in one pass.

YOU WILL BE ABLE TO

- Use `indexOf`, `substring`, `contains` and string concatenation correctly.
- Explain that Java Strings are immutable and what that means for repeated edits.
- Move all hyphens to the front, preserving the order of the other characters.
- Do it in one pass over a `char[]`, scanning from the right.

Before this lesson: Strings, StringBuilder and Characters · Methods: Parameters, Return Values, Scope and Overloading

The idea

The slide transforms `"face-prep"` into `"-faceprep"`: find the hyphen, take everything before it and everything after it, and put the hyphen first.

```
face-prep indexOf('-') = 4
face | - | prep
"-" + "face" + "prep" → -faceprep
```

The slide solves only the **first** hyphen. A more interesting problem moves **all** hyphens to the front without disturbing the other characters: `a-b-c-d` → `---abcd`. This is a *stable partition*: hyphens first, everything else in its original order.

WHY THIS WORKS

Scanning from the right, write every non-hyphen character into the rightmost free slot; whatever slots remain on the left must be hyphens (their count equals the number of hyphens). Because each character is read at or before the position where it is written, nothing is overwritten before being read — so one array and one pass suffice.

Syntax

method	returns	example (s = "face-prep")
s.indexOf('-')	index of first match, or −1	4
s.contains("-")	boolean	true
s.substring(a, b)	chars a … b−1	substring(0, 4) = "face"
s.substring(a)	chars a … end	substring(5) = "prep"
s.toCharArray()	a char[] copy	[f,a,c,e,-,p,r,e,p]

WATCH OUT

Strings are **immutable**: `s.substring(...)` returns a NEW string; `s` itself never changes. Repeated `+=` in a loop creates many temporary strings (O(n²)); use `StringBuilder`.

A tiny example

```
String s = "face-prep";
int i = s.indexOf('-');
String result = "-" + s.substring(0, i) + s.substring(i + 1); // -faceprep
```

Guided example

MoveFirstHyphen.java — Move the first hyphen (from the slides)

```
1  public class MoveFirstHyphen {
2   // The lecture version: only the FIRST hyphen moves to the front.
3   static String moveHyphenToBeginning(String s) {
4   int hyphenIndex = s.indexOf('-');                                    (1)
5   if (hyphenIndex < 0) {                                               (2)
6   return s; // no hyphen: nothing to do
7   }
8   return "-" + s.substring(0, hyphenIndex) + s.substring(hyphenIndex + 1);   (4)
9   }
10
11  public static void main(String[] args) {
12  System.out.println(moveHyphenToBeginning("face-prep"));
13  System.out.println(moveHyphenToBeginning("a-b-c"));
14  System.out.println(moveHyphenToBeginning("nohyphen"));
15  }
16 }
```

indexOf + substring. O(n) time (copying) and O(n) space for the new string.

Line-by-line walkthrough

#	Where	What happens
1	line 4	Position of the first hyphen, or −1 if there is none.
2	line 5	No hyphen: return the original (the slide uses `contains` then `indexOf`, i.e. searches twice).
3	line 8	Everything before the hyphen.
4	line 8	Everything after the hyphen (skip it with +1).

MoveAllHyphens.java — Move every hyphen (optimised solution)

```
1  import java.util.Scanner;
2
3  public class MoveAllHyphens {
4   // Move EVERY hyphen to the front; the other characters keep their original order.
5   static String moveAll(String s) {
6   StringBuilder hyphens = new StringBuilder();                    (1)
7   StringBuilder others = new StringBuilder();
8   for (int i = 0; i < s.length(); i++) {
9   char c = s.charAt(i);
10  if (c == '-') {
11  hyphens.append(c);
12  } else {
13  others.append(c);
14  }
15  }
16  return hyphens.append(others).toString();                       (2)
17  }
18
19  // The same result in ONE array, scanning from the right, no second builder.
20  static String moveAllInPlace(String s) {
21  char[] a = s.toCharArray();
22  int write = a.length - 1; // next free slot from the right       (3)
23  for (int read = a.length - 1; read >= 0; read--) {
24  if (a[read] != '-') {                                           (4)
25  a[write--] = a[read]; // push non-hyphens to the right end
26  }
27  }
28  while (write >= 0) {
29  a[write--] = '-'; // fill the rest with hyphens                 (5)
30  }
31  return new String(a);
32  }
33
34  public static void main(String[] args) {
35  Scanner sc = new Scanner(System.in);
36  String s = sc.nextLine();
37  System.out.println(moveAll(s));
38  System.out.println(moveAllInPlace(s));
39  }
40 }
```

Two versions of the same stable partition.

Sample input

```
a-b-c-d
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	Two builders: hyphens and everything else. Appending to a StringBuilder is amortised O(1).
2	line 16	Hyphens first, then the others in their original order.
3	line 22	The in-place version: `write` is the next free slot from the right.
4	line 24	Copy non-hyphen characters towards the right end, preserving their order (read ≤ write always).
5	line 29	The remaining slots on the left are exactly the hyphens.

Dry run: watch it execute

Right-to-left copy, then fill the front with hyphens.

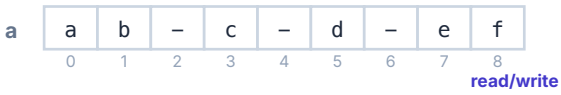
Algorithm: Move every hyphen to the front (in place). **Input:** s = `ab-c-d-ef`.

Pseudocode

```
write = n - 1
for read = n-1 down to 0:
 if a[read] != "-": a[write--] = a[read]
while write >= 0: a[write--] = "-"
```

Step 1 of 11. read and write both start at the last cell.

read = 8 · write = 8



Step 2 of 11. a[8] = "f" is not a hyphen: copy it to a[8].

read = 8 · write = 8



Step 3 of 11. a[7] = "e" is not a hyphen: copy it to a[7].

read = 7 · write = 7



Step 4 of 11. a[6] is a hyphen: skip it (only read moves left, so write falls behind).

read = 6 · write = 6



Step 5 of 11. a[5] = "d" is not a hyphen: copy it to a[6].

read = 5 · write = 6



Step 6 of 11. a[4] is a hyphen: skip it (only read moves left, so write falls behind).

read = 4 · write = 5



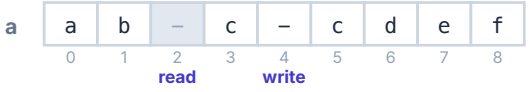
Step 7 of 11. a[3] = "c" is not a hyphen: copy it to a[5].

read = 3 · write = 5



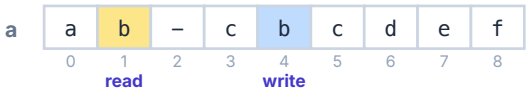
Step 8 of 11. a[2] is a hyphen: skip it (only read moves left, so write falls behind).

read = 2 · write = 4



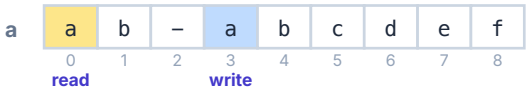
Step 9 of 11. a[1] = "b" is not a hyphen: copy it to a[4].

read = 1 · write = 4



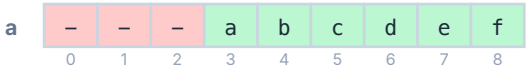
Step 10 of 11. a[0] = "a" is not a hyphen: copy it to a[3].

read = 0 · write = 3



Step 11 of 11. Fill the first 3 cells with hyphens.

hyphens = 3



> **RESULT**
> ---abcdef

Complexity

All versions are **O(n) time**. Memory: the substring/builder versions use O(n) extra characters; the `char[]` right-scan uses O(1) extra beyond the array itself.

The naive "remove each hyphen and insert it at index 0" costs O(n) per hyphen → O(n·h), up to $O(n^2)$.

Common mistakes

Expecting the string to change

Problem

```
s.substring(0, 3);
System.out.println(s); // unchanged
```

Why: Strings are immutable. The result of substring must be stored.

Fix: s = s.substring(0, 3);

Off-by-one on the second substring

Problem

```
s.substring(0, i) + s.substring(i)
```

Why: substring(i) still contains the hyphen: the output has the hyphen twice.

Fix: s.substring(i + 1)

Building the result with += in a loop

Problem

```
String out = "";
for (char c : s.toCharArray()) out += c;
```

Why: Each += copies the whole string so far: $O(n^2)$.

Fix: StringBuilder sb = new StringBuilder(); sb.append(c);

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · First hyphen to the front — *Beginner*

Read a line. If it contains a hyphen, move its FIRST hyphen to the beginning; otherwise print the line unchanged.

Example input	Expected output
face-prep	-faceprep

Hint 1: indexOf, then substring twice.

Exercise 2 · All hyphens to the front — *Intermediate*

Move every hyphen to the beginning of the line, keeping the other characters in order.

- length ≤ 10^6

Example input	Expected output
a-b-c	--abc

Hint 1: Two StringBuilders, or one char array scanned from the right.

Exercise 3 · Minimum adjacent swaps — *Challenge*

Print the minimum number of swaps of ADJACENT characters needed to bring all hyphens to the front.

- length ≤ 10^6 (answer may need long)

Example input	Expected output
a-b-c	3

Hint 1: Each hyphen must pass every non-hyphen character that stands before it. **Hint 2:** Sum, over hyphens, the number of non-hyphen characters seen so far.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
String s = "ab-cd";
int i = s.indexOf('-');
System.out.println("-" + s.substring(0, i) + s.substring(i + 1));
```

Your answer: _____

Q2. What does "face-prep".indexOf('x') return?

- **A.** 0
- **B.** -1
- **C.** null
- **D.** an exception

Q3. Why is building a long string with += in a loop slow?

- **A.** += is not allowed
- **B.** Strings are immutable so every += copies the string
- **C.** Because of garbage collection
- **D.** It is not slow

Q4. Time complexity of the one-pass right scan?

- **A.** O(1)
- **B.** O(n)
- **C.** O(n log n)
- **D.** O(n^2)

Interview / exam questions

- Why are Java Strings immutable?
- Move all zeros to the end of an array — how is it the same problem?
- How do you do a stable partition in place?

Watch & read

Video: Move special characters in String | String Assignment 1 | Programming Using Java — DeCode with Shreyas · 3:08. *Partitioning a string with two pointers*

Video: Lec-10: Two 2 Pointer Technique | Two 2 Sum Problem in Data Structure — Gate Smashers · 10:33. *The two-pointer technique*

REVISION CARD (30–60 SECONDS)

- indexOf, substring(a, b) = a..b−1, substring(a) = a..end.
- Strings are immutable; use StringBuilder for building.
- All hyphens: stable partition, right-to-left write pointer.
- O(n).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide moves only the first hyphen (by name it suggests "the hyphen"); the generalisation to all hyphens is added here and labelled as such.

5.23 Strobogrammatic Number

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 40 min · complete path

Numbers that look the same when rotated 180°: check one with two pointers and a map, generate them all with recursion.

YOU WILL BE ABLE TO

- State exactly which digits rotate to which: 0→0, 1→1, 8→8, 6→9, 9→6.
- Check a number with two pointers and a HashMap.
- Generate all n-digit strobogrammatic numbers by wrapping smaller ones.
- Count them with a formula and spot errors in informal definitions.

Before this lesson: Strings, StringBuilder and Characters · Sets and Maps: HashSet, TreeSet, HashMap, TreeMap · Recursion: Base Case, Recursive Case and the Call Stack

The idea

Turn a number **upside down** (rotate it 180°). Some digits look like a digit again: 0 → 0, 1 → 1, 8 → 8, 6 → 9, 9 → 6. A number is **strobogrammatic** if the rotated picture is the *same number*. Rotating reverses the order of digits as well as turning each one: `69` → rotate 9→6 and 6→9 and read from the other side → `69`. `818` → 818. `906` → 906. `25` → no (2 and 5 do not rotate into a digit in this definition).

```
number 6 9 rotate each digit 9 6
read backwards ← (6→9, 9→6) ↓ reverse → 6 9 ✓ same as the original
```

WHY THIS WORKS

Position i from the left must rotate into the digit at position i from the right. So walk two pointers from the two ends: for each pair (left, right) require `rotate(left) == right`. When the pointers meet in the middle (odd length) the digit must rotate into **itself** — only 0, 1 and 8 can sit in the middle, never 6 or 9. `while (left <= right)` with the same test handles that case automatically because left == right.

Syntax

```
static char rotate(char c) { // what a digit looks like upside down
 switch (c) {
 case '0': case '1': case '8': return c;
 case '6': return '9';
 case '9': return '6';
 default: return '?'; // not strobogrammatic
 }
}
int l = 0, r = s.length() - 1; // two pointers moving inwards
while (l <= r) { if (rotate(s.charAt(l)) != s.charAt(r)) return false; l++; r--; }
```

A tiny example

```
String from = "01689", to = "01986";
int i = from.indexOf(c); // c rotated = to.charAt(i)
```

Guided example

The slide solution builds a `HashMap<Character,Character>` and runs the two-pointer loop. The lesson adds the generator (LeetCode 247) so you can see all numbers of a given length.

StrobogrammaticCheck.java — Strobogrammatic check (optimised solution)

```
1  import java.util.HashMap;
2  import java.util.Map;
3  import java.util.Scanner;
4
5  public class StrobogrammaticCheck {
6   // Rotating a number by 180 degrees reverses the digit order AND rotates each digit:
7   // 0->0, 1->1, 8->8, 6->9, 9->6. Any other digit has no valid rotated form.
8   static boolean isStrobogrammatic(String num) {
9   Map<Character, Character> rotated = new HashMap<>();
10  rotated.put('0', '0');
11  rotated.put('1', '1');
12  rotated.put('8', '8');
13  rotated.put('6', '9');                                                    (1)
14  rotated.put('9', '6');
15
16  int left = 0;
17  int right = num.length() - 1;
18  while (left <= right) {                                                   (2)
19  Character r = rotated.get(num.charAt(left));
20  if (r == null || r != num.charAt(right)) { // left digit must rotate into the right   (3)
        digit
21  return false;
22  }
23  left++;
24  right--;
25  }
26  return true;
27  }
28
29  public static void main(String[] args) {
30  Scanner sc = new Scanner(System.in);
31  String n = sc.next();
32  System.out.println(n + " -> " + isStrobogrammatic(n));
33  }
34 }
```

A map of rotations and two pointers. O(n) time, O(1) space (the map has 5 entries).

Sample input

```
906
```

Line-by-line walkthrough

#	Where	What happens
1	line 13	6 rotates into 9 and 9 into 6; 0, 1, 8 map to themselves. Any other digit is simply absent.
2	line 18	`<=` (not `<`) so that the middle digit of an odd-length number is also tested against itself.
3	line 20	Either the digit cannot rotate, or it rotates into something other than its mirrored partner.

StrobogrammaticGenerate.java — Generate all of length n (demonstration)

```
1  import java.util.ArrayList;
2  import java.util.List;
3  import java.util.Scanner;
4
5  public class StrobogrammaticGenerate {
6   // All strobogrammatic numbers with exactly n digits (no leading zero except the single
        digit "0").
7   // Build from the middle outward: wrap every (n-2)-digit answer with a matching pair.
8   static List<String> build(int n, int total) {
9   if (n == 0) {                                                                    (1)
10  return new ArrayList<>(List.of(""));
11  }
12  if (n == 1) {                                                                    (2)
13  return new ArrayList<>(List.of("0", "1", "8"));
14  }
15  List<String> result = new ArrayList<>();
16  for (String inner : build(n - 2, total)) {
17  if (n != total) {                                                                (3)
18  result.add("0" + inner + "0"); // zeros are fine on the inside, never on the outside
19  }
20  result.add("1" + inner + "1");
21  result.add("6" + inner + "9");                                                   (4)
22  result.add("8" + inner + "8");
23  result.add("9" + inner + "6");
24  }
25  return result;
26  }
27
28  public static void main(String[] args) {
29  Scanner sc = new Scanner(System.in);
30  int n = sc.nextInt();
31  List<String> all = build(n, n);
32  System.out.println(all.size() + " numbers: " + all);
33  }
34 }
```

build(n) wraps every build(n−2) result with 1…1, 6…9, 8…8, 9…6 (and 0…0 on the inside).

Sample input

```
3
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	Base case for even lengths: the empty string.
2	line 12	Base case for odd lengths: the self-rotating digits 0, 1, 8.
3	line 17	A leading zero is not allowed: 0…0 is added only for the inner layers.
4	line 21	Wrapping with a rotating pair keeps the property.

Dry run: watch it execute

Compare rotate(left) with right while the pointers close in.

Algorithm: Strobogrammatic check. **Input:** s = `906`.

Pseudocode

```
left = 0; right = n - 1
while (left <= right) {
 if rotate(s[left]) != s[right]: return false
 left++; right--
}
return true
```

Step 1 of 3. Rotation map: 0→0, 1→1, 8→8, 6→9, 9→6.

left = 0 · right = 2

digits	9	0	6
	0	1	2

Step 2 of 3. rotate(9) = 6; s[2] = 6 → match.

left = 0 · right = 2 · rotated = 6

digits	9	0	6
	0 L	1	2 R

Step 3 of 3. rotate(0) = 0; s[1] = 0 → match.

left = 1 · right = 1 · rotated = 0

digits	9	0	6
	0	1 R	2

> **RESULT**
> 906 IS strobogrammatic

Complexity

Check: one pass over half the digits → O(n) time, O(1) space.

Generate: for length n there are 4·5^(n/2−1) numbers (n even) or 4·5^((n−1)/2−1)·3 (n odd, n ≥ 3); each has n characters, so the output size alone is Θ(n·5^(n/2)). Exponential — but optimal, since you must print them.

Counts of strobogrammatic numbers (no leading zero; the single digit 0 counts for n = 1).

n	1	2	3	4	5	6
count	3	4	12	20	60	100

Common mistakes

Only mapping, not mirroring

Problem

```
// treating "88" and "69" as the only valid ones because "digits look the same"
```

Why: The slide says 6 and 9 are "not strobogrammatic" digits; yet 69 is. The correct statement: each digit must rotate to a digit AND appear mirrored.

Fix: Use the pair map {6→9, 9→6, 0→0, 1→1, 8→8} with two pointers.

Using < instead of <= in the loop

Problem

```
while (left < right)
```

Why: "6" would be accepted: the middle digit is never tested. A single 6 is not strobogrammatic.

Fix: while (left <= right)

Claiming 2 and 5 work

Problem

```
// slide: "The digits 2 and 5 can also be considered strobogrammatic"
```

Why: On a seven-segment display 2 and 5 look alike when rotated, but in the standard definition (digits that turn into valid digits on a printed page) they do not. The standard set is {0, 1, 8, 6, 9}.

Fix: Stick to the stated definition and state it in the problem.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Is it strobogrammatic? — *Beginner*

Read a number as a string. Print YES if it looks the same rotated by 180°, else NO.

Example input	Expected output
69	YES
25	NO

Hint 1: Two pointers from both ends. **Hint 2:** Allowed rotations: 0-0, 1-1, 8-8, 6-9, 9-6.

Exercise 2 · How many of length n? — *Intermediate*

Read n (1 ≤ n ≤ 18). Print how many n-digit strobogrammatic numbers have no leading zero (for n = 1 the digit 0 counts).

Example input	Expected output
4	20

Hint 1: The outermost pair has 4 choices (11, 69, 88, 96). **Hint 2:** Each inner pair has 5 choices; an odd middle digit has 3.

Exercise 3 · Strobogrammatic in a range — *Challenge*

Read low and high (as strings of up to 15 digits, low ≤ high). Print how many strobogrammatic numbers lie in [low, high].

Example input	Expected output
50 100	3

Hint 1: Generate by length from len(low) to len(high). **Hint 2:** For equal lengths, string comparison equals numeric comparison.

Quick check

Answers are in the answer key at the end of this part.

Q1. Which number is strobogrammatic?

- **A.** 25
- **B.** 962
- **C.** 69
- **D.** 1962

Q2. Which digits can be the MIDDLE digit of an odd-length strobogrammatic number?

- **A.** 0, 1, 8
- **B.** 6, 9
- **C.** 2, 5
- **D.** any digit

Q3. What does the check print for "88"?

```
// pairs: (8,8) → rotate(8) = 8 ✓
```

Your answer: _____

Q4. How many 2-digit strobogrammatic numbers (no leading zero) exist?

- **A.** 3
- **B.** 4
- **C.** 5
- **D.** 9

Interview / exam questions

- Check a strobogrammatic number in O(n).
- Generate all strobogrammatic numbers of length n (LeetCode 247).
- Why must the middle digit be 0, 1 or 8?

Watch & read

Video: STROBOGRAMMATIC NUMBER | LEETCODE # 246 | PYTHON SOLUTION — Cracking FAANG · 9:50.
Strobogrammatic number check (Python — the idea transfers to Java)

Video: Valid Palindrome Strings | Two Pointer Approach Explained | Python, C++ & Java — RisingBrain · 14:03.
Two-pointer symmetric checks (Python, C++ — the idea transfers to Java)

REVISION CARD (30–60 SECONDS)

- Rotations: 0↔0, 1↔1, 8↔8, 6↔9.
- Two pointers: rotate(s[l]) == s[r]; use l <= r.
- Middle digit ∈ {0, 1, 8}.
- Count for n digits: 4·5^(⌈n/2⌉−1)... (×3/÷5 for the odd middle).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide says digits 2 and 5 "can also be considered strobogrammatic" and that 6 and 9 "are not"; the standard definition (and the slide's own Java code) uses {0, 1, 8} as self-rotating and {6 ↔ 9} as a pair. • The slide explains 906 as "looks like 609". 906 is strobogrammatic because the rotated number equals 906 itself. • The slide program is split across two slides with numbered lines (22–30) that continue the class; the lesson program is the whole class.

5.24 Lexicographically First Palindromic Rearrangement

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 40 min · ★ on the minimum-viable path

Rearrange the letters of a string into the alphabetically smallest palindrome — using character counts, not permutations.

YOU WILL BE ABLE TO

- State when a multiset of letters can form a palindrome (at most one odd count).
- Build the smallest palindrome: half of each letter in alphabetical order, the odd letter in the middle, then the mirror.
- Explain why the brute-force permutation search is hopeless for long strings.
- Handle case and non-letter characters safely.

Before this lesson: Strings, StringBuilder and Characters · One-Dimensional Arrays · Sets and Maps: HashSet, TreeSet, HashMap, TreeMap

The idea

Given `malayalam` rearrange its letters so the result is a **palindrome** and, among all palindromes, the **alphabetically first**. Answer: `aalmymlaa`.

When is it possible? A palindrome mirrors itself, so every letter must appear an even number of times — except possibly one letter that sits alone in the middle. `malayalam` has a:4, l:2, m:2, y:1 → only `y` is odd ✓. `apple` has a:1, p:2, l:1, e:1 → three odd letters ✗ → "no palindromic string".

```
counts a:4 l:2 m:2 y:1
half a:2 l:1 m:1 front = a a l m (alphabetical)
middle y
back m l a a (front reversed)
result aalm + y + mlaa = aalmymlaa
```

WHY THIS WORKS

To make the string alphabetically small, the **left half** should be the smallest possible arrangement of the letters available for it: sort them. The right half is forced — it must mirror the left. Putting the single odd letter in the middle uses it up without affecting the order of the halves. Trying permutations one by one (the slide's "simple approach") finds the same answer but needs up to n! steps.

Syntax

```
int[] freq = new int[26];
for (char c : s.toCharArray()) freq[c - 'a']++; // count letters
StringBuilder half = new StringBuilder();
for (int i = 0; i < 26; i++)
 half.append(String.valueOf((char) ('a' + i)).repeat(freq[i] / 2)); // half of each letter, in
     order
String palindrome = half + middle + new StringBuilder(half).reverse();
```

A tiny example

```
int[] freq = new int[26];
for (char c : s.toCharArray()) freq[c - 'a']++;
```

Guided example

LexPalindromeBrute.java — Try permutations in order (simple solution)

```
1  import java.util.Arrays;
2  import java.util.Scanner;
3
4  public class LexPalindromeBrute {
5   // Sorted string, then walk through successive permutations until one reads the same
        backwards.
6   // Correct but factorial time: only usable for very short strings.
7   static boolean nextPermutation(char[] a) {                                    (1)
8   int i = a.length - 2;
9   while (i >= 0 && a[i] >= a[i + 1]) i--;
10  if (i < 0) return false;
11  int j = a.length - 1;
12  while (a[j] <= a[i]) j--;
13  char t = a[i]; a[i] = a[j]; a[j] = t;
14  for (int l = i + 1, r = a.length - 1; l < r; l++, r--) {
15  t = a[l]; a[l] = a[r]; a[r] = t;
16  }
17  return true;
18  }
19
20  static boolean isPalindrome(char[] a) {
21  for (int l = 0, r = a.length - 1; l < r; l++, r--) {
22  if (a[l] != a[r]) return false;
23  }
24  return true;
25  }
26
27  public static void main(String[] args) {
28  Scanner sc = new Scanner(System.in);
29  char[] a = sc.next().toLowerCase().toCharArray();
30  Arrays.sort(a);                                                               (2)
31  do {
32  if (isPalindrome(a)) {
33  System.out.println(new String(a));
34  return;
35  }
36  } while (nextPermutation(a));                                                 (3)
37  System.out.println("no palindromic string");
38  }
39 }
```

Sort the characters (the smallest permutation) and iterate next-permutation until one reads the same both ways. Correct, O(n·n!).

Sample input

```
Malayalam
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	The standard algorithm for the next lexicographic permutation (see the Sorted Unique Permutation lesson).
2	line 30	The very first permutation is the sorted one.
3	line 36	Stops after the last permutation → "no palindromic string".

LexPalindrome.java — Lexicographically first palindrome by counting (optimised solution)

```
1  import java.util.Scanner;
2  import java.util.TreeMap;
3
4  public class LexPalindrome {
5   // A string can be rearranged into a palindrome iff at most one character has an odd
        count
6   // (and exactly one when the length is odd). The smallest palindrome puts the smallest
        letters
7   // on the outside: front half in increasing order, middle = the odd character, back
        half mirrored.
8   static String firstPalindrome(String input) {
9   TreeMap<Character, Integer> freq = new TreeMap<>(); // TreeMap keeps characters sorted   (1)
10  for (char c : input.toLowerCase().toCharArray()) {
11  freq.merge(c, 1, Integer::sum);
12  }
13
14  String middle = "";
15  int oddCount = 0;
16  for (var e : freq.entrySet()) {
17  if (e.getValue() % 2 == 1) {                                                            (2)
18  oddCount++;
19  middle = String.valueOf(e.getKey());
20  }
21  }
22  if (oddCount > 1) {                                                                     (3)
23  return "no palindromic string";
24  }
25
26  StringBuilder front = new StringBuilder();
27  for (var e : freq.entrySet()) {
28  front.append(String.valueOf(e.getKey()).repeat(e.getValue() / 2)); // half of each count   (4)
29  }
30  String back = new StringBuilder(front).reverse().toString();                            (5)
31  return front + middle + back;
32  }
33
34  public static void main(String[] args) {
35  Scanner sc = new Scanner(System.in);
36  System.out.println(firstPalindrome(sc.next()));
37  }
38 }
```

O(n + σ log σ): count letters, test the odd-count rule, build half + middle + mirror.

Sample input

```
Malayalam
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	A TreeMap stores keys in sorted order — iteration is alphabetical for free.
2	line 17	Counting letters with an odd frequency.
3	line 22	More than one odd letter cannot be placed: no palindrome exists.
4	line 28	Each letter contributes half of its (even part of its) count to the front.
5	line 30	The right half is the mirror of the left.

Dry run: watch it execute

Counts → half → mirrored string.

Algorithm: Lexicographically first palindrome. **Input:** s = `malayalam`.

Pseudocode

```
count letters
odd = letters with odd count
if odd > 1: no palindromic string
front = (count/2 copies of each letter, a..z)
return front + oddLetter + reverse(front)
```

Step 1 of 4. Letter counts: a:4 l:2 m:2 y:1

a = 4 · l = 2 · m = 2 · y = 1

Step 2 of 4. Letters with an odd count: y.

odd letters = y

Step 3 of 4. Front half (sorted): "aalm".

front = aalm

Step 4 of 4. Result = front + "y" + mirror(front).

result = aalmymlaa

> **RESULT**
> aalmymlaa

Complexity

Permutation search: up to $n!/(c_1!c_2!\ldots)$ candidates × O(n) test → factorial time: n = 12 is already ~10^8–10^9 operations.

Counting: one pass to count (O(n)), σ distinct letters to build (O(σ log σ) with the TreeMap, O(σ) with a 26-array), → **O(n) time, O(σ) extra space** apart from the output.

Common mistakes

Crash on uppercase letters (slide)

Problem

```
freq[str.charAt(i) - 'a']++; // "Malayalam" → 'M' - 'a' = -20
```

Why: The slide's intro shows the input "Malayalam" but the code only works for lowercase: index −20 throws ArrayIndexOutOfBoundsException.

Fix: Normalise with toLowerCase() first, or use a Map / int[256].

A stray space in rear_str (slide)

Problem

```
String front_str = "", rear_str = " ";
```

Why: The back half starts with a space character, so the printed palindrome has a space in it ("aalmymlaa " ...). It is not a palindrome any more.

Fix: String rear = "";

Putting the odd letter at the front

Problem

```
return oddLetter + front + back;
```

Why: The odd letter has no partner, so it must stand alone in the exact middle; anywhere else the string is not symmetric.

Fix: return front + oddLetter + back;

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Can it be a palindrome? — *Beginner*

Read a word (letters only, case-insensitive). Print YES if its letters can be rearranged into a palindrome, else NO.

Example input	Expected output
aabb	YES

Hint 1: Count letters; how many have an odd count?

Exercise 2 · Build the smallest palindrome — *Intermediate*

Read a word. Print the lexicographically first palindrome using ALL its letters (case-insensitive, print in lowercase), or `no palindromic string`.

Example input	Expected output
Malayalam	aalmymlaa
apple	no palindromic string

Hint 1: TreeMap<Character,Integer> keeps letters sorted. **Hint 2:** front = each letter repeated count/2 times.

Exercise 3 · How many palindromes? — *Challenge*

Read a word of up to 20 letters. Print the number of DISTINCT palindromes that can be formed from all its letters (0 if impossible).

Example input	Expected output
aabbcc	6

Hint 1: Only the left half can be chosen freely. **Hint 2:** Number of arrangements of a multiset: m! / (c_1! c_2! ...) where counts are halved.

Quick check

Answers are in the answer key at the end of this part.

Q1. Which word can be rearranged into a palindrome?

- **A.** apple
- **B.** level
- **C.** abcd
- **D.** aabbcd

Q2. Smallest palindrome from "bbaac"?

```
// counts a:2 b:2 c:1
```

Your answer: _____

Q3. Time complexity of the counting solution?

- **A.** O(n!)
- **B.** $O(n^2)$
- **C.** O(n)
- **D.** $O(2^n)$

Q4. In an even-length string, how many letters may have an odd count?

- **A.** 0
- **B.** 1
- **C.** 2
- **D.** any

Interview / exam questions

- How do you know if a palindrome permutation exists?
- Build the lexicographically smallest one in O(n).
- What about the lexicographically LARGEST?
- Count the number of distinct palindromic permutations.

Watch & read

Video: Lexicographically Smallest Palindrome | Leetcode 2697 | Approach + Code with explanation. — Pirai's Academy · 8:28. *Building the smallest palindrome from a multiset*

Video: LeetCode 266. Palindrome Permutation (Algorithm Explained) — Nick White · 5:53. *Frequency counting for palindromes*

REVISION CARD (30–60 SECONDS)

- Palindrome possible ⇔ ≤ 1 letter with odd count.
- Result = sorted half + odd letter + mirror.
- Normalise case; use TreeMap or int[26].
- O(n).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide program starts `rear_str = " "` (a space) and subtracts `'a'` from characters although its first example input is "Malayalam" with a capital M; the lesson program lowercases and fixes the initial value.
- The slide (slide 4) states the expected output for "Malayalam" as "aalmymlaa", which is correct for the lowercased word.

5.25 Manacher's Algorithm (Longest Palindromic Substring)

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 65 min · complete path

Find the longest palindromic substring in linear time by reusing the symmetry of palindromes inside palindromes.

YOU WILL BE ABLE TO

- Define substring vs subsequence and find the longest palindromic substring by expanding around centres.
- Explain why inserting separators makes all palindromes odd-length.
- Describe the mirror idea and trace the centre/right-edge bookkeeping.
- Argue why the total work is O(n) and verify the code with a brute-force test.

Before this lesson: Strings, StringBuilder and Characters · Lexicographically First Palindromic Rearrangement · Analysing Loops: Counting Iterations

The idea

A **palindromic substring** is a block of consecutive characters that reads the same both ways. The longest in `forgeeksskeegfor` is `geeksskeeg` (length 10). The straightforward method is to treat every position as the **centre** of a palindrome and grow outwards while the two ends match — O(n²) in the worst case (try `aaaa…a`).

Manacher's observation: **a palindrome inside a bigger palindrome is mirrored**. If we already know a big palindrome centred at C that reaches to R, then for a position i inside it, the palindrome around i is at least as large as the palindrome around its mirror position 2C − i (up to the border R). So we never need to start from radius 0 again.

```
C−radius ... mirror C i ... R
[ . . . . m . . . C . . i . . . ]
what is known on the left of C also holds on the right of C
→ radius[i] = min(radius[mirror], R − i), then try to extend only beyond R
```

WHY THIS WORKS

To treat even and odd palindromes uniformly, insert a separator between all characters: `abba` → `#a#b#b#a#`. Every palindrome of the original is now an odd-length palindrome centred on a real character or a `#`. The radius in the transformed string equals the length of the palindrome in the original. Because R only ever moves right, and every successful extension moves R right by one, the `while` loops run at most 2n times in total — linear.

Syntax

```
StringBuilder t = new StringBuilder("#"); // "abba" -> "#a#b#b#a#"
for (char c : s.toCharArray()) t.append(c).append('#');
int[] p = new int[t.length()]; // p[i] = palindrome radius around i
int centre = 0, right = 0;
for (int i = 0; i < t.length(); i++) {
 if (i < right) p[i] = Math.min(right - i, p[2 * centre - i]); // reuse the mirror
 while (i - p[i] - 1 >= 0 && i + p[i] + 1 < t.length()
 && t.charAt(i - p[i] - 1) == t.charAt(i + p[i] + 1)) p[i]++; // expand
 if (i + p[i] > right) { centre = i; right = i + p[i]; }
}
```

A tiny example

```
// expand around centre i
int l = i, r = i;
while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { l--; r++; }
// palindrome is s.substring(l + 1, r)
```

Guided example

ExpandAroundCenter.java — Expand around centres (simple solution)

```
1  import java.util.Scanner;
2
3  public class ExpandAroundCenter {
4   // Every palindrome has a centre (a character, or a gap between two characters). From
        each of the
5   // 2n-1 centres grow outwards while the ends match. O(n^2) worst case, O(1) space.
6   static String longestPalindrome(String s) {
7   if (s.isEmpty()) {
8   return "";
9   }
10  int bestStart = 0;
11  int bestLen = 1;
12  for (int centre = 0; centre < s.length(); centre++) {
13  for (int type = 0; type < 2; type++) { // type 0: odd length, type 1: even length      (1)
14  int l = centre;
15  int r = centre + type;                                                               (2)
16  while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) {
17  l--;
18  r++;
19  }
20  int len = r - l - 1;                                                                 (3)
21  if (len > bestLen) {
22  bestLen = len;
23  bestStart = l + 1;
24  }
25  }
26  }
27  return s.substring(bestStart, bestStart + bestLen);
28  }
29
30  public static void main(String[] args) {
31  Scanner sc = new Scanner(System.in);
32  String s = sc.next();
33  String p = longestPalindrome(s);
34  System.out.println("longest palindromic substring: " + p + " (length " + p.length() + ")
        ");
35  }
36 }
```

For each centre (a character or a gap) grow while the characters match. O(n²) time, O(1) space.

Sample input

```
babad
```

Line-by-line walkthrough

#	Where	What happens
1	line 13	type 0 = odd palindrome centred on a character; type 1 = even palindrome centred between two characters.
2	line 15	For even palindromes the right pointer starts one step to the right.
3	line 20	After the loop l and r are one step past the palindrome on both sides.

Manacher.java — Manacher's algorithm (optimised solution)

```
import java.util.Scanner;

public class Manacher {
 // Manacher's algorithm: longest palindromic substring in O(n).
 // 1. Insert '#' between characters so every palindrome has odd length: "abba" ->
     "#a#b#b#a#".
 // 2. radius[i] = how far the palindrome centred at i extends (in the transformed
     string).
 // 3. Reuse the mirror image of i inside the biggest palindrome seen so far instead of
     starting from 0.
 static int[] radii(String s) {
 StringBuilder sb = new StringBuilder("#");
 for (char c : s.toCharArray()) {
 sb.append(c).append('#');                                                    (1)
 }
 String t = sb.toString();
 int n = t.length();
 int[] radius = new int[n];
 int center = 0; // centre of the palindrome that reaches furthest right      (2)
 int right = 0; // its right edge (exclusive)
 for (int i = 0; i < n; i++) {
 if (i < right) {                                                             (3)
 int mirror = 2 * center - i;
 radius[i] = Math.min(right - i, radius[mirror]); // free information from symmetry (4)
 }
 while (i - radius[i] - 1 >= 0 && i + radius[i] + 1 < n                       (5)
 && t.charAt(i - radius[i] - 1) == t.charAt(i + radius[i] + 1)) {
 radius[i]++; // extend only as far as symmetry could not tell us
 }
 if (i + radius[i] > right) {                                                 (6)
 center = i;
 right = i + radius[i];
 }
 }
 return radius;
 }

 static String longestPalindrome(String s) {
 if (s.isEmpty()) {
 return "";
 }
 int[] r = radii(s);
 int bestCentre = 0;
 for (int i = 1; i < r.length; i++) {
 if (r[i] > r[bestCentre]) {
 bestCentre = i;
 }
 }
 int start = (bestCentre - r[bestCentre]) / 2; // map back to an index in the original (7)
     string
 return s.substring(start, start + r[bestCentre]);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 String p = longestPalindrome(s);
```

(code continued)

```
    System.out.println("longest palindromic substring: " + p + " (length " + p.length() + ")
        ");
  }
}
```

Radius array over the "#"-padded string with centre/right-edge reuse. O(n) time and space.

Sample input

```
forgeeksskeegfor
```

Line-by-line walkthrough

#	Where	What happens
1	line 11	Pad: "#a#b#b#a#". Every palindrome is odd-length now.
2	line 16	Centre of the palindrome whose right edge `right` is the furthest seen so far.
3	line 19	i lies inside the known palindrome, so its mirror gives free information.
4	line 21	The mirrored radius, limited by the border (beyond `right` nothing is known).
5	line 23	Extend only while characters match — each success pushes the border right.
6	line 27	New furthest palindrome: update centre and right edge.
7	line 46	Map back: a radius r around transformed index c covers original indices (c − r)/2 ... (c − r)/2 + r − 1.

ManacherCheck.java — Brute-force verification (demonstration)

```
import java.util.Random;

public class ManacherCheck {
 static int[] radii(String s) {
 StringBuilder sb = new StringBuilder("#");
 for (char c : s.toCharArray()) sb.append(c).append('#');
 String t = sb.toString();
 int n = t.length();
 int[] radius = new int[n];
 int center = 0, right = 0;
 for (int i = 0; i < n; i++) {
 if (i < right) radius[i] = Math.min(right - i, radius[2 * center - i]);
 while (i - radius[i] - 1 >= 0 && i + radius[i] + 1 < n && t.charAt(i - radius[i] - 1)
     == t.charAt(i + radius[i] + 1)) radius[i]++;
 if (i + radius[i] > right) { center = i; right = i + radius[i]; }
 }
 return radius;
 }

 static int longestFast(String s) {
 int best = 0;
 for (int r : radii(s)) best = Math.max(best, r);
 return best;
 }

 static int longestBrute(String s) {
 int best = 0;
 for (int i = 0; i < s.length(); i++) {
 for (int j = i; j < s.length(); j++) {
 boolean pal = true;
 for (int l = i, r = j; l < r; l++, r--) if (s.charAt(l) != s.charAt(r)) { pal = false;
     break; }
 if (pal) best = Math.max(best, j - i + 1);
 }
 }
 return best;
 }

 // 20000 random strings over a tiny alphabet (many palindromes) compared with a
     triple-loop brute force.
 public static void main(String[] args) {
 Random rnd = new Random(11);
 for (int t = 0; t < 20000; t++) {
 int n = 1 + rnd.nextInt(14);
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append((char) ('a' + rnd.nextInt(2)));
 String s = sb.toString();
 if (longestFast(s) != longestBrute(s)) {
 System.out.println("MISMATCH on " + s);
 return;
 }
 }
 System.out.println("Manacher matches brute force on 20000 random strings");
 }
}
```

The O(n) result must equal the longest palindrome found by checking every substring.

Line-by-line walkthrough

#	Where	What happens
1	line 43	A two-letter alphabet creates many palindromes and repeated structure — exactly where bugs in the mirror logic show up.

Dry run: watch it execute

Radii in the transformed string; the mirror trick saves re-expansion.

Algorithm: Manacher's algorithm. **Input:** s = `abaxabaxabb`.

Pseudocode

```
t = "#a#b#..#"
for each i:
 if i < right: radius[i] = min(right − i, radius[2*center − i])
 expand while t[i−r−1] == t[i+r+1]
 if i + radius[i] > right: center = i; right = i + radius[i]
```

The full trace has 24 steps; 14 evenly spaced ones are shown. Run the rest in the website's Dry-run lab.

Step 1 of 24. Padded string t = "#a#b#a#x#a#b#a#x#a#b#b#" (length 23).

center = 0 · right = 0

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
t	#	a	#	b	#	a	#	x	#	a	#	b	#	a	#	x	#	a	#	b	#	b	#
radius	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

22
0

Step 3 of 24. i = 1 is outside any known palindrome: start at 0. Extended to 1. New center 1, right edge 2.

i = 1 · radius[i] = 1 · center = 1 · right = 2

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
t	#	a	#	b	#	a	#	x	#	a	#	b	#	a	#	x	#	a	#	b	#	b	#
radius	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

22
0

Step 5 of 24. i = 3 is outside any known palindrome: start at 0. Extended to 3. New center 3, right edge 6.

i = 3 · radius[i] = 3 · center = 3 · right = 6

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
t	#	a	#	b	#	a	#	x	#	a	#	b	#	a	#	x	#	a	#	b	#	b	#
radius	0	1	0	3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

22
0

Step 6 of 24. i = 4 is inside the palindrome (center 3, right 6); mirror = 2 → start with radius 0. No extension possible.

i = 4 · radius[i] = 0 · center = 3 · right = 6



Step 8 of 24. i = 6 is outside any known palindrome: start at 0. No extension possible.

i = 6 · radius[i] = 0 · center = 3 · right = 6



Step 10 of 24. i = 8 is inside the palindrome (center 7, right 14); mirror = 6 → start with radius 0. No extension possible.

i = 8 · radius[i] = 0 · center = 7 · right = 14



Step 12 of 24. i = 10 is inside the palindrome (center 7, right 14); mirror = 4 → start with radius 0. No extension possible.

i = 10 · radius[i] = 0 · center = 7 · right = 14



Step 13 of 24. i = 11 is inside the palindrome (center 7, right 14); mirror = 3 → start with radius 3. Extended to 9. New center 11, right edge 20.

i = 11 · radius[i] = 9 · center = 11 · right = 20



Step 15 of 24. i = 13 is inside the palindrome (center 11, right 20); mirror = 9 → start with radius 1. No extension possible.

i = 13 · radius[i] = 1 · center = 11 · right = 20

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
t	#	a	#	b	#	a	#	x	#	a	#	b	#	a	#	x	#	a	#	b	#	b	#
radius	0	1	0	3	0	1	0	7	0	1	0	9	0	1	0	0	0	0	0	0	0	0	0

Step 17 of 24. i = 15 is inside the palindrome (center 11, right 20); mirror = 7 → start with radius 5. No extension possible.

i = 15 · radius[i] = 5 · center = 11 · right = 20

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
t	#	a	#	b	#	a	#	x	#	a	#	b	#	a	#	x	#	a	#	b	#	b	#
radius	0	1	0	3	0	1	0	7	0	1	0	9	0	1	0	5	0	0	0	0	0	0	0

Step 19 of 24. i = 17 is inside the palindrome (center 11, right 20); mirror = 5 → start with radius 1. No extension possible.

i = 17 · radius[i] = 1 · center = 11 · right = 20

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
t	#	a	#	b	#	a	#	x	#	a	#	b	#	a	#	x	#	a	#	b	#	b	#
radius	0	1	0	3	0	1	0	7	0	1	0	9	0	1	0	5	0	1	0	0	0	0	0

Step 20 of 24. i = 18 is inside the palindrome (center 11, right 20); mirror = 4 → start with radius 0. No extension possible.

i = 18 · radius[i] = 0 · center = 11 · right = 20

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
t	#	a	#	b	#	a	#	x	#	a	#	b	#	a	#	x	#	a	#	b	#	b	#
radius	0	1	0	3	0	1	0	7	0	1	0	9	0	1	0	5	0	1	0	0	0	0	0

Step 22 of 24. i = 20 is outside any known palindrome: start at 0. Extended to 2. New center 20, right edge 22.

i = 20 · radius[i] = 2 · center = 20 · right = 22

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
t	#	a	#	b	#	a	#	x	#	a	#	b	#	a	#	x	#	a	#	b	#	b	#
radius	0	1	0	3	0	1	0	7	0	1	0	9	0	1	0	5	0	1	0	1	2	0	0

Step 24 of 24. i = 22 is outside any known palindrome: start at 0. No extension possible.

i = 22 · radius[i] = 0 · center = 20 · right = 22

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
t	#	a	#	b	#	a	#	x	#	a	#	b	#	a	#	x	#	a	#	b	#	b	#
radius	0	1	0	3	0	1	0	7	0	1	0	9	0	1	0	5	0	1	0	1	2	1	0

> **RESULT**
> longest palindromic substring: "baxabaxab" (length 9)

Complexity

Centre expansion: each of the 2n−1 centres may expand up to n/2 steps → **O(n²)** worst case (`aaaa…`), O(1) space.

Manacher: `right` never decreases and increases at most 2n+1 times in total; each outer iteration does O(1) work apart from successful extensions → **O(n) time, O(n) space** for the radius array and padded string.

Common mistakes

Forgetting even-length palindromes

Problem

```
// expand only from characters: misses "bb" in "cbbd"
```

Why: Palindromes of even length have a centre between two characters.

Fix: Try both (i, i) and (i, i + 1) — or insert separators.

Reading beyond the border

Problem

```
radius[i] = radius[mirror]; // without min(right − i, …)
```

Why: The mirror's palindrome may extend past the known palindrome; nothing is known beyond `right`, so the copied radius can be too large.

Fix: radius[i] = Math.min(right - i, radius[mirror]);

Wrong mapping back to the original string

Problem

```
s.substring(centre − radius, centre + radius)
```

Why: In the padded string indices are doubled. A radius r around transformed index c corresponds to original start (c − r) / 2 and length r.

Fix: int start = (c - r) / 2; s.substring(start, start + r)

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Is it a palindrome? — *Beginner*

Read a word. Print YES if it is a palindrome, else NO.

Example input	Expected output
racecar	YES

Hint 1: Two pointers moving inwards.

Exercise 2 · Longest palindromic substring (expand) — *Intermediate*

Read a word (length ≤ 2000). Print its longest palindromic substring; if several have the same length, print the one that starts first.

Example input	Expected output
cbbd	bb

Hint 1: Two kinds of centres. **Hint 2:** Update only when strictly longer to keep the earliest.

Exercise 3 · Count palindromic substrings — *Challenge*

Read a word of up to 10^5 letters. Print how many substrings (counted by position) are palindromes, in O(n).

Example input	Expected output
abba	6

Hint 1: In the padded string a radius r around a centre contributes $\lceil r/2 \rceil$ palindromes. **Hint 2:** Use long.

Quick check

Answers are in the answer key at the end of this part.

Q1. Why does Manacher's algorithm insert "#" between characters?

- **A.** To speed up comparison
- **B.** To make every palindrome odd-length
- **C.** To sort the string
- **D.** To avoid overflow

Q2. The time complexity of Manacher's algorithm is...

- **A.** $O(n^2)$
- **B.** O(n log n)
- **C.** O(n)
- **D.** $O(n^3)$

Q3. The padded form of "abba" is...

```
// insert # at the start, end, and between characters
```

Your answer: _____

Q4. The worst case input for centre expansion is...

- **A.** abcdef
- **B.** aaaaaa
- **C.** abababa

- **D.** a

Interview / exam questions

- Explain Manacher's algorithm in two minutes.
- Why is it linear although it has nested loops?
- Substring vs subsequence?
- Compare with a suffix-array / hashing approach.

Watch & read

Video: Longest Palindromic Substring Manacher's Algorithm — Tushar Roy - Coding Made Simple · 16:46. *Manacher in linear time*

Video: 5. Longest Palindromic Substring | Part-1: Expand around centers #leetcode — CodeWonka : Get started with programming · 7:55. *Expand-around-center for comparison*

REVISION CARD (30–60 SECONDS)

- Pad with # → all palindromes odd. radius[i] = palindrome radius at i.
- Keep (center, right). If i < right: radius[i] = min(right − i, radius[2·center − i]).
- Extend while chars match; update center/right.
- O(n) time & space; original start = (c − r)/2, length r.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide listing of the Java program stops in the middle of the main loop (slide 5 ends after the first while loop and slide 6 is "Thank you"), so the code is incomplete. The lesson provides a complete, tested implementation. • The slide text (steps 1–12) describes the version with sentinels "@" and "$" and array `p`; the Java snippet uses a different (GeeksforGeeks-style) layout with array `L`. The lesson implements the classical radius-array formulation that both describe.

ASSUMPTIONS MADE

• The unfinished Java fragment on the slide is replaced by the standard algorithm; the explanation follows the 12 steps on slides 3–4.

5.26 Natural Sort Order

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 40 min · complete path

Sort "file2" before "file10" by splitting names into text and number chunks and writing a Comparator.

YOU WILL BE ABLE TO

- Explain why plain string comparison puts "file10" before "file2".
- Split a name into alternating digit and non-digit chunks.
- Write a `Comparator<String>` that compares number chunks numerically.
- Handle ties (leading zeros, case) deterministically.

Before this lesson: Sorting with Comparable and Comparator · Strings, StringBuilder and Characters · Generics and Lists: ArrayList and LinkedList

The idea

Java's default String order compares characters one at a time using their codes. `"file10"` vs `"file2"`: the first difference is `1` vs `2`; `1` < `2`, so `file10` sorts before `file2`. Humans expect the opposite, because 10 is bigger than 2.

```
plain sort : file1.txt file10.txt file2.txt file20.txt file3.txt
natural sort : file1.txt file2.txt file3.txt file10.txt file20.txt
```

WHY THIS WORKS

Natural order cuts each name into chunks where digits and non-digits alternate: `file10.txt` → `file`, `10`, `.txt`. Two names are compared chunk by chunk: if both chunks are numbers, compare them as numbers (`2 < 10`); otherwise compare them as text. The first difference decides. A name that is a prefix of the other (fewer chunks) comes first.

Syntax

A `Comparator<T>` is an object with `int compare(T a, T b)`: negative if a goes first, 0 if equal, positive if b goes first. In Java 8+ you can write it as a lambda: `(a, b) -> …`. `list.sort(comparator)` sorts in place and is **stable**.

```
Comparator<String> byLength = (a, b) -> Integer.compare(a.length(), b.length());
names.sort(byLength);
```

A tiny example

```
boolean isD = Character.isDigit('7'); // true
String num = "file10".substring(4); // "10"
int c = new java.math.BigInteger("0010").compareTo(new java.math.BigInteger("10")); // 0
```

Guided example

NaturalSort.java — Natural-order comparator (optimised solution)

```
import java.math.BigInteger;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Scanner;

public class NaturalSort {
 // Split a name into alternating chunks of digits and non-digits: "file10.txt" ->
     "file", "10", ".txt".
 static List<String> chunks(String s) {                                          (1)
 List<String> out = new ArrayList<>();
 int i = 0;
 while (i < s.length()) {
 int j = i;
 boolean digit = Character.isDigit(s.charAt(i));
 while (j < s.length() && Character.isDigit(s.charAt(j)) == digit) {
 j++;
 }
 out.add(s.substring(i, j));
 i = j;
 }
 return out;
 }

 // Compare chunk by chunk. Two number chunks are compared as numbers; anything else as
     text.
 static final Comparator<String> NATURAL = (a, b) -> {
 List<String> x = chunks(a);
 List<String> y = chunks(b);
 for (int i = 0; i < Math.min(x.size(), y.size()); i++) {
 String p = x.get(i);
 String q = y.get(i);
 int cmp;
 if (Character.isDigit(p.charAt(0)) && Character.isDigit(q.charAt(0))) {
 cmp = new BigInteger(p).compareTo(new BigInteger(q)); // numeric value, any length  (2)
 } else {
 cmp = p.compareToIgnoreCase(q);                                                 (3)
 }
 if (cmp != 0) {
 return cmp;
 }
 }
 return Integer.compare(x.size(), y.size()); // shorter list of chunks first      (4)
 };

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 List<String> names = new ArrayList<>();
 while (sc.hasNext()) {
 names.add(sc.next());
 }
 List<String> lexicographic = new ArrayList<>(names);
 Collections.sort(lexicographic);
 names.sort(NATURAL);                                                            (5)
```

(code continued)

```
    System.out.println("plain sort : " + lexicographic);
    System.out.println("natural sort: " + names);
    }
  }
```

Splits both names into chunks and compares them pairwise; O(L) per comparison for names of length L.

Sample input

```
file1.txt file10.txt file2.txt file20.txt file3.txt
```

Line-by-line walkthrough

#	Where	What happens
1	line 10	Splits "file10.txt" into ["file", "10", ".txt"] by walking while the character class (digit / non-digit) stays the same.
2	line 34	Numeric comparison of arbitrarily long digit strings (leading zeros ignored).
3	line 36	Text chunks compared case-insensitively so that "File2" and "file2" are neighbours.
4	line 42	All shared chunks are equal: the name with fewer chunks goes first.
5	line 53	List.sort is a stable merge sort (TimSort): equal names keep their input order.

> **NOTE**
> The slide code compares `List<String>` objects returned by `extractNumericParts` with `Comparator.comparing(...)`. A `List` is not `Comparable`, so that line **does not compile**, and even if it did it would compare "10" before "2" as text. The lesson compares chunk by chunk and uses `BigInteger` for number chunks of any length.

Dry run: watch it execute

Numbers inside names are compared by value.

Algorithm: Natural-order comparison. **Input:** a = `file10.txt`, b = `file9.txt`.

Pseudocode

```
chunks("file10.txt") = file | 10 | .txt
for i in 0..min(len)−1:
 both numbers → compare as numbers
 otherwise → compare text (ignore case)
 first difference decides
shorter chunk list first
```

Step 1 of 4. Split each name into alternating chunks of non-digits and digits.

chunks of "file10.txt"	file	10	.txt
	0	1	2
chunks of "file9.txt"	file	9	.txt
	0	1	2

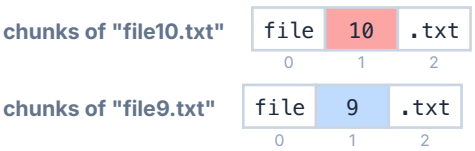
Step 2 of 4. Chunk 0: text "file" vs "file" compared ignoring case → equal.

chunk = 0 · comparison = 0



Step 3 of 4. Chunk 1: numbers 10 vs 9 compared by VALUE (10 vs 9) → greater.

chunk = 1 · comparison = 1



Step 4 of 4. For comparison, plain compareTo gives "file10.txt" first because it compares character by character ('1' < '9').

plain compareTo = -1 · natural = 1

> **RESULT**
> "file9.txt" comes before "file10.txt" (natural order)

Complexity

Each comparison costs O(L) (splitting and comparing). Sorting n names: O(n log n) comparisons → **O(n · L · log n)** time, O(n·L) space for the chunk lists. Pre-computing each name's chunk list once (a "Schwartzian transform") avoids re-splitting at every comparison.

Common mistakes

Comparing numeric chunks as text

Problem

```
"10".compareTo("2") // negative: "10" < "2"
```

Why: Strings are ordered by character codes, so a longer number can come first.

Fix: Parse as number (BigInteger for unlimited length) before comparing.

Using Integer.parseInt on arbitrary chunks

Problem

```
Integer.parseInt("99999999999")
```

Why: Throws NumberFormatException for numbers above $2^{31}-1$.

Fix: new BigInteger(chunk) or compare by (length after stripping zeros, then text)

A comparator that is not a total order

Problem

```
return 0; // for everything that is not obviously different
```

Why: If equal-looking names (file01 vs file1) compare equal while others are ordered inconsistently, sort may throw "Comparison method violates its general contract".

Fix: Add tie-breakers: length, then compareTo (see exercise 3).

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Sort file names naturally — *Beginner*

Read n and then n file names. Print them in natural order, one per line (case-insensitive text, numbers compared numerically).

Example input	Expected output
`4 ⏎ img12.png ⏎ img10.png ⏎ img2.png ⏎ IMG1.png`	`IMG1.png ⏎ img2.png ⏎ img10.png ⏎ img12.png`

Hint 1: Chunk, compare chunk by chunk. **Hint 2:** List.sort(comparator).

Exercise 2 · Sort version numbers — *Intermediate*

Read n and n version strings like 1.2.10. Print them in increasing version order on one line (a missing component counts as 0).

Example input	Expected output
`4 ⏎ 1.10 ⏎ 1.2.10 ⏎ 1.2.9 ⏎ 1.9`	`1.2.9 1.2.10 1.9 1.10`

Hint 1: split("\\.") and compare component by component as numbers.

Exercise 3 · Natural order with deterministic ties — *Challenge*

Sort names naturally, ignoring case in text chunks. If two names are numerically equal chunk by chunk (e.g. file01 and file1), the SHORTER name comes first, then plain compareTo. Print on one line separated by spaces.

Example input	Expected output
`4 ⏎ file01 ⏎ file1 ⏎ File2 ⏎ file10`	`file1 file01 File2 file10`

Hint 1: Compare chunk by chunk, then the number of chunks, then total length, then compareTo.

Quick check

Answers are in the answer key at the end of this part.

Q1. Plain String ordering puts which first?

- **A.** file2
- **B.** file10
- **C.** they are equal
- **D.** depends on locale

Q2. How should two digit chunks "07" and "7" compare in natural order?

- **A.** "07" < "7"
- **B.** equal numerically (then tie-break)
- **C.** "07" > "7"
- **D.** error

Q3. What does a Comparator return when a should come before b?

- **A.** a positive number
- **B.** zero
- **C.** a negative number
- **D.** true

Q4. List.sort in Java is...

- **A.** stable
- **B.** unstable
- **C.** randomised
- **D.** not allowed for Strings

Interview / exam questions

- How would you implement natural sort for file names like "a10b2" vs "a10b10"?
- What makes a comparator valid?
- Why use BigInteger for the number chunks?

Watch & read

Video: Java Comparator Sorting in Java | Practical Tutorial with Eclipse Examples | Hands on | Red9SysTech — Red9SysTech · 3:52. *Comparing strings with embedded numbers*

Video: Sort Strings in Descending Order in Java | Custom Comparator Example | Java Programming — The Corporate Villager · 5:30. *Writing a custom comparator*

REVISION CARD (30–60 SECONDS)

- Split into digit / non-digit chunks.
- Number chunks → numeric compare (BigInteger); text → compareToIgnoreCase.
- First difference decides; fewer chunks first.
- Add tie-breakers so the order is total.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide's `Comparator.comparing(NaturalSort::extractNumericParts)` does not compile (List is not Comparable) and compares digits as text. The lesson replaces it with a chunk comparator. • The slide text describes sorting by the numeric part "in ascending order, otherwise regular string comparison" — which ignores that prefix text must be compared first.

5.27 Maneuvering (Counting Grid Paths)

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 45 min · ★ on the minimum-viable path

Count the right/down paths across a grid — recursion, then dynamic programming, then a combinatorics formula.

YOU WILL BE ABLE TO

- Set up the recurrence paths(m, n) = paths(m−1, n) + paths(m, n−1).
- Show why plain recursion is exponential (it recomputes the same cells).
- Fill a table bottom-up (dynamic programming) in O(m·n).
- Obtain the closed form C(m+n−2, m−1) and handle obstacles.

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Two-Dimensional Arrays (Matrices) · Combinations (nCr)

The idea

You stand in the top-left cell of an m × n grid and want to reach the bottom-right cell moving only **right** or **down**. How many different paths exist? For a 2 × 2 grid there are 2; for 3 × 3 there are 6.

Think about the **last** move. You entered the destination from the cell above or from the cell to the left. So `paths(m, n) = paths(m−1, n) + paths(m, n−1)`. A single row or column has exactly one path.

```
3×3 grid: paths into each cell
1 1 1
1 2 3
1 3 6 → 6 paths to the bottom-right
```

WHY THIS WORKS

The recursive code asks for the same sub-grids again and again: paths(2, 2) is needed by both paths(2, 3) and paths(3, 2). The number of calls roughly doubles with every step → exponential. **Dynamic programming** just stores each answer in a table the first time it is computed. The closed form comes from counting arrangements: every path is a sequence of (m−1) downs and (n−1) rights, so choose which of the m+n−2 moves are downs: C(m+n−2, m−1).

Syntax

```
long[][] dp = new long[m][n]; // dp[i][j] = number of paths to cell (i, j)
for (int i = 0; i < m; i++) dp[i][0] = 1;
for (int j = 0; j < n; j++) dp[0][j] = 1;
for (int i = 1; i < m; i++)
 for (int j = 1; j < n; j++) dp[i][j] = dp[i - 1][j] + dp[i][j - 1];

static long paths(int r, int c) { // plain recursion (exponential): same recurrence
 if (r == 0 || c == 0) return 1;
 return paths(r - 1, c) + paths(r, c - 1);
}
```

A tiny example

```
static int paths(int m, int n) {
 if (m == 1 || n == 1) return 1;
 return paths(m - 1, n) + paths(m, n - 1);
}
```

Guided example

The lecture trace for `numberOfPaths(3, 3)` is exactly the call tree; the recursive program below also counts the calls (11 for a 3 × 3 grid) so you can see the growth.

> **NOTE**
> The commented line `// + numberOfPaths(m-1, n-1)` on the slide adds the diagonal move; the numbers that result are the **Delannoy numbers**.

PathsRecursive.java — Recursive path counting (from the slides)

```
1  import java.util.Scanner;
2
3  public class PathsRecursive {
4   static long calls = 0;                                                    (1)
5
6   // Number of paths from the top-left to the bottom-right of an m x n grid using only
        right/down moves.
7   static long numberOfPaths(int m, int n) {
8   calls++;
9   if (m == 1 || n == 1) {                                                   (2)
10  return 1; // a single row or column: only one way
11  }
12  return numberOfPaths(m - 1, n) + numberOfPaths(m, n - 1); // arrive from above or from   (3)
        the left
13  }
14
15  public static void main(String[] args) {
16  Scanner sc = new Scanner(System.in);
17  int m = sc.nextInt();
18  int n = sc.nextInt();
19  System.out.println("paths = " + numberOfPaths(m, n));
20  System.out.println("recursive calls = " + calls);
21  }
22 }
```

Direct translation of the recurrence. Exponential time.

Sample input

```
3 3
```

Line-by-line walkthrough

#	Where	What happens
1	line 4	A counter to expose the amount of repeated work.
2	line 9	Base case: one row or one column — a single straight path.
3	line 12	Arrive from above, or from the left. Add the two counts.

PathsDP.java — Bottom-up table (optimised solution)

```
1  import java.util.Scanner;
2
3  public class PathsDP {
4   // dp[i][j] = number of ways to reach cell (i, j). Each cell is computed once: O(m * n).
5   static long numberOfPaths(int m, int n) {
6   long[][] dp = new long[m][n];
7   for (int i = 0; i < m; i++) {
8   for (int j = 0; j < n; j++) {
9   if (i == 0 || j == 0) {                                    (1)
10  dp[i][j] = 1; // first row / first column: one way
11  } else {
12  dp[i][j] = dp[i - 1][j] + dp[i][j - 1];                    (2)
13  }
14  }
15  }
16  return dp[m - 1][n - 1];                                   (3)
17  }
18
19  public static void main(String[] args) {
20  Scanner sc = new Scanner(System.in);
21  int m = sc.nextInt();
22  int n = sc.nextInt();
23  System.out.println(numberOfPaths(m, n));
24  }
25 }
```

dp[i][j] = dp[i−1][j] + dp[i][j−1]. O(m·n) time and space (O(n) with a single row).

Sample input

```
18 18
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	First row and first column: exactly one way to reach each.
2	line 12	The same recurrence, but both terms are already computed.
3	line 16	The bottom-right cell is the answer.

PathsFormula.java — Closed form (optimised solution)

```
1  import java.util.Scanner;
2
3  public class PathsFormula {
4   // Every path has (m-1) downs and (n-1) rights in some order: choose where the downs go.
5   // paths = C(m + n - 2, m - 1)
6   static long paths(int m, int n) {
7   int total = m + n - 2;                                                    (1)
8   int k = Math.min(m - 1, n - 1);                                           (2)
9   long result = 1;
10  for (int i = 1; i <= k; i++) {
11  result = result * (total - k + i) / i; // exact at every step             (3)
12  }
13  return result;
14  }
15
16  public static void main(String[] args) {
17  Scanner sc = new Scanner(System.in);
18  int m = sc.nextInt();
19  int n = sc.nextInt();
20  System.out.println(paths(m, n));
21  }
22 }
```

C(m+n−2, m−1) with the multiply-then-divide loop from the Combination lesson. O(min(m, n)) time, O(1) space.

Sample input

```
18 18
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	Total number of moves in any path.
2	line 8	C(total, k) = C(total, total − k): use the smaller side.
3	line 11	Exact at every step (see the Combination lesson).

Dry run: watch it execute

Each cell holds the number of paths that reach it.

Algorithm: Grid paths DP. **Input:** m = `4`, n = `5`.

Pseudocode

```
for i in 0..m-1:
 for j in 0..n-1:
 if i == 0 or j == 0: dp[i][j] = 1
 else dp[i][j] = dp[i-1][j] + dp[i][j-1]
return dp[m-1][n-1]
```

The full trace has 20 steps; 14 evenly spaced ones are shown. Run the rest in the website's Dry-run lab.

Step 1 of 20. Cell (0, 0) is on the first row/column: only one way.

i = 0 · j = 0 · dp[i][j] = 1

1	·	·	·	·
·	·	·	·	·
·	·	·	·	·
·	·	·	·	·

Step 2 of 20. Cell (0, 1) is on the first row/column: only one way.

i = 0 · j = 1 · dp[i][j] = 1

1	1	·	·	·
·	·	·	·	·
·	·	·	·	·
·	·	·	·	·

Step 4 of 20. Cell (0, 3) is on the first row/column: only one way.

i = 0 · j = 3 · dp[i][j] = 1

1	1	1	1	·
·	·	·	·	·
·	·	·	·	·
·	·	·	·	·

Step 5 of 20. Cell (0, 4) is on the first row/column: only one way.

i = 0 · j = 4 · dp[i][j] = 1

1	1	1	1	1
·	·	·	·	·
·	·	·	·	·
·	·	·	·	·

Step 7 of 20. Cell (1, 1) = 1 (from above) + 1 (from the left) = 2.

i = 1 · j = 1 · dp[i][j] = 2

1	1	1	1	1
1	2	·	·	·
·	·	·	·	·
·	·	·	·	·

Step 8 of 20. Cell (1, 2) = 1 (from above) + 2 (from the left) = 3.

i = 1 · j = 2 · dp[i][j] = 3

1	1	1	1	1
1	2	3	·	·
·	·	·	·	·
·	·	·	·	·

Step 10 of 20. Cell (1, 4) = 1 (from above) + 4 (from the left) = 5.

i = 1 · j = 4 · dp[i][j] = 5

1	1	1	1	1
1	2	3	4	5
·	·	·	·	·
·	·	·	·	·

Step 11 of 20. Cell (2, 0) is on the first row/column: only one way.

i = 2 · j = 0 · dp[i][j] = 1

1	1	1	1	1
1	2	3	4	5
1	·	·	·	·
·	·	·	·	·

Step 13 of 20. Cell (2, 2) = 3 (from above) + 3 (from the left) = 6.

i = 2 · j = 2 · dp[i][j] = 6

1	1	1	1	1
1	2	3	4	5
1	3	6	·	·
·	·	·	·	·

Step 14 of 20. Cell (2, 3) = 4 (from above) + 6 (from the left) = 10.

i = 2 · j = 3 · dp[i][j] = 10

1	1	1	1	1
1	2	3	4	5
1	3	6	10	·
·	·	·	·	·

Step 16 of 20. Cell (3, 0) is on the first row/column: only one way.

i = 3 · j = 0 · dp[i][j] = 1

1	1	1	1	1
1	2	3	4	5
1	3	6	10	15
1	·	·	·	·

Step 17 of 20. Cell (3, 1) = 3 (from above) + 1 (from the left) = 4.

i = 3 · j = 1 · dp[i][j] = 4

1	1	1	1	1
1	2	3	4	5
1	3	6	10	15
1	4	·	·	·

Step 19 of 20. Cell (3, 3) = 10 (from above) + 10 (from the left) = 20.

i = 3 · j = 3 · dp[i][j] = 20

1	1	1	1	1
1	2	3	4	5
1	3	6	10	15
1	4	10	20	·

Step 20 of 20. Cell (3, 4) = 15 (from above) + 20 (from the left) = 35.

i = 3 · j = 4 · dp[i][j] = 35

1	1	1	1	1
1	2	3	4	5
1	3	6	10	15
1	4	10	20	35

> **RESULT**
> 35 paths

Complexity

Recursion: T(m, n) = T(m−1, n) + T(m, n−1) + 1 → about C(m+n−2, m−1) leaf calls — **exponential** (for 18 × 18 it is over 2 billion calls).

DP: every cell once → **O(m·n)** time, O(m·n) space (reducible to O(n)).

Formula: O(min(m, n)).

grid	paths	recursive calls
3 × 3	6	11
10 × 10	48 620	97 239
18 × 18	2 333 606 220	≈ 4.7 billion

Common mistakes

int overflow of the count

Problem

```
int paths = ...; // 18 x 18 grid
```

Why: 2 333 606 220 > 2 147 483 647 — the result wraps to a negative number.
Fix: long (or BigInteger / modulus when the problem asks for it)

Confusing grid size with number of moves

Problem

```
C(m + n, m)
```

Why: An m × n grid of CELLS needs m−1 downs and n−1 rights: C(m+n−2, m−1).
Fix: total moves = m + n − 2

Blocked cells treated as 1

Problem

```
dp[i][j] = (i == 0 || j == 0) ? 1 : ... // with obstacles
```

Why: In the first row, a blocked cell cuts off every cell to its right; setting them all to 1 overcounts.
Fix: dp[i][j] = blocked ? 0 : (from above + from left), with dp[0][0] = 1

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Paths in a rectangle — *Beginner*

Read m and n and print the number of right/down paths from the top-left to the bottom-right of an m × n grid.

- 1 ≤ m, n ≤ 18

Example input	Expected output
3 7	28

Hint 1: dp table with first row and column set to 1.

Exercise 2 · Paths with obstacles — *Intermediate*

Read R, C and a grid of 0 (free) and 1 (blocked). Print the number of right/down paths from (0,0) to (R−1,C−1) that avoid blocked cells (0 if the start or end is blocked).

- 1 ≤ R, C ≤ 100
- the answer fits in a long

Example input	Expected output
3 3 ⏎ 0 0 0 ⏎ 0 1 0 ⏎ 0 0 0	2

Hint 1: A blocked cell contributes 0. **Hint 2:** dp[0][0] = 1 only if the start is free.

Exercise 3 · Huge grid modulo a prime — *Challenge*

Read m and n (≤ 100 000). Print the number of paths modulo 1 000 000 007.

Example input	Expected output
3 7	28

Hint 1: A DP table of 10^{10} cells is impossible — use C(m+n−2, m−1). **Hint 2:** Precompute factorials mod p and use a modular inverse (Fermat: a^(p−2)).

Quick check

Answers are in the answer key at the end of this part.

Q1. Number of right/down paths in a 4 × 4 grid of cells?

- **A.** 12
- **B.** 20
- **C.** 24
- **D.** 70

Q2. How many paths does this return?

```
int[][] dp = new int[3][3];
for (int i = 0; i < 3; i++)
 for (int j = 0; j < 3; j++)
 dp[i][j] = (i == 0 || j == 0) ? 1 : dp[i-1][j] + dp[i][j-1];
System.out.println(dp[2][2]);
```

Your answer: _____

Q3. Why is the recursive solution slow?

- **A.** Java recursion is slow
- **B.** It recomputes the same sub-grids many times
- **C.** The base case is wrong
- **D.** Arrays are slow

Q4. Time complexity of the DP table solution?

- **A.** O(m + n)
- **B.** O(m·n)
- **C.** O(2^(m+n))
- **D.** O(log n)

Interview / exam questions

- Explain overlapping sub-problems using this example.
- Reduce the DP space to O(n).
- Count paths with obstacles; with diagonal moves.
- Why is the answer a binomial coefficient?

Watch & read

Video: Unique Paths - Dynamic Programming - Leetcode 62 — NeetCode · 10:48. *Counting grid paths*

Video: Ways to Reach Origin | GFG Problem of the Day | Grid Paths DP & nCr Explained — Gopo workspace · 10:43. *The combinatorial formula*

REVISION CARD (30–60 SECONDS)

- paths(m, n) = paths(m−1, n) + paths(m, n−1); first row/col = 1.
- Recursion is exponential; DP is O(m·n).
- Closed form: C(m+n−2, m−1).
- Obstacle ⇒ 0; use long / modulus.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide says the problem is "Maneuvering a Cave"; the deck contains only the grid-path recurrence (a classic problem). Its trace of numberOfPaths(3, 3) is correct and ends at 6. • The slide mentions "already calculated" for numberOfPaths(2, 2) — the plain recursive code does NOT cache; the second call is recomputed (that is the inefficiency this lesson addresses).

5.28 Sorted Distinct Permutations

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 50 min · complete path

Print every distinct permutation of a string with duplicates, in alphabetical order.

YOU WILL BE ABLE TO

- Count distinct permutations of a multiset: n! / (c_1!·c_2!·...).
- Generate them in sorted order with the next-permutation algorithm.
- Generate them in sorted order with count-based backtracking.
- Recognise that the slide's swap-based generator does not produce sorted output.

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Strings, StringBuilder and Characters · Combinations (nCr)

The idea

The string `AAB` has 3! = 6 arrangements, but only 3 are different: `AAB`, `ABA`, `BAA`. In general the number of **distinct** permutations is `n! / (c₁! · c₂! · …)` where c_i is how often each letter occurs (the slide: "geeks" has 5!/2! = 60).

We want them **sorted** (dictionary order). Two reliable ways:

1. **Next permutation.** Start from the sorted string and repeatedly turn the current arrangement into the next larger one: find the rightmost position that can still grow, swap in the smallest larger letter to its right, and reverse the tail.
2. **Counting backtracking.** Keep how many copies of each letter are left. At every position try the letters in alphabetical order (each distinct letter once, if copies remain) and recurse. Duplicates are never generated and the order is automatically sorted.

```
ABC → ACB → BAC → BCA → CAB → CBA (next-permutation order)
```

WHY THIS WORKS

Next-permutation works because the suffix to the right of the pivot is in descending order — the largest arrangement of those letters. To get the next larger permutation you must raise the pivot by the smallest possible amount and then make the suffix as small as possible (ascending, i.e. reversed).

Syntax

```
char[] c = s.toCharArray();
Arrays.sort(c); // start from the smallest arrangement
String again = new String(c); // char[] back to String
boolean[] used = new boolean[c.length]; // backtracking: which letters are taken
static void swap(char[] c, int i, int j) { char t = c[i]; c[i] = c[j]; c[j] = t; }
Set<Character> seen = new HashSet<>(); // skip repeated letters at one position
if (!seen.add(c[i])) continue; // add() returns false when already present
```

A tiny example

```
int i = a.length - 2;
while (i >= 0 && a[i] >= a[i + 1]) i--; // pivot: first place from the right that is smaller
    than its successor
```

Guided example

NextPermutation.java — Next lexicographic permutation (optimised solution)

```
1   import java.util.Arrays;
2   import java.util.Scanner;
3
4   public class NextPermutation {
5    // Rearrange a into the next lexicographically larger permutation; return false if a is
         the largest.
6    static boolean next(char[] a) {
7    int i = a.length - 2;
8    while (i >= 0 && a[i] >= a[i + 1]) {                                                    1
9    i--; // 1. find the rightmost position that can still grow
10   }
11   if (i < 0) {
12   return false;
13   }
14   int j = a.length - 1;
15   while (a[j] <= a[i]) {                                                                  2
16   j--; // 2. smallest value to its right that is bigger
17   }
18   char t = a[i];
19   a[i] = a[j];                                                                            3
20   a[j] = t; // 3. swap them
21   for (int l = i + 1, r = a.length - 1; l < r; l++, r--) {                                4
22   t = a[l];
23   a[l] = a[r];
24   a[r] = t; // 4. the tail was decreasing: reverse it to make it smallest
25   }
26   return true;
27   }
28
29   public static void main(String[] args) {
30   Scanner sc = new Scanner(System.in);
31   char[] a = sc.next().toCharArray();
32   Arrays.sort(a); // start from the smallest arrangement                                  5
33   StringBuilder sb = new StringBuilder();
34   do {
35   sb.append(new String(a)).append(' ');
36   } while (next(a));
37   System.out.println(sb.toString().trim());
38   }
39  }
```

The classic 4-step algorithm; O(n) per permutation, O(1) space.

Sample input

```
AAB
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	1. Find the pivot: the rightmost i with a[i] < a[i+1]. Everything right of it is non-increasing.
2	line 15	2. Find the smallest letter in the suffix that is larger than the pivot (the suffix is descending, so scan from the right).
3	line 19	3. Swap pivot and that letter.
4	line 21	4. The suffix is still descending: reverse it to ascending to make it the smallest tail.
5	line 32	Starting from the sorted string guarantees we enumerate ALL arrangements, in order.

PermutationsBacktrack.java — Counting backtracking (optimised solution)

```
1  import java.util.Arrays;
2  import java.util.Scanner;
3
4  public class PermutationsBacktrack {
5   static StringBuilder out = new StringBuilder();
6
7   // counts[c] = how many copies of letter c are still unused. Trying letters in
        alphabetical order
8   // produces the permutations in sorted order, and each DISTINCT letter is tried once
        per position.
9   static void build(int[] counts, char[] current, int pos) {
10  if (pos == current.length) {
11  out.append(new String(current)).append(' ');
12  return;
13  }
14  for (int c = 0; c < counts.length; c++) {
15  if (counts[c] == 0) {                                                    1
16  continue;
17  }
18  counts[c]--; // choose                                                   2
19  current[pos] = (char) ('A' + c);
20  build(counts, current, pos + 1); // explore                              3
21  counts[c]++; // un-choose                                                4
22  }
23  }
24
25  public static void main(String[] args) {
26  Scanner sc = new Scanner(System.in);
27  String s = sc.next();
28  int[] counts = new int[26];
29  for (char ch : s.toCharArray()) {
30  counts[ch - 'A']++;
31  }
32  build(counts, new char[s.length()], 0);
33  System.out.println(out.toString().trim());
34  }
35 }
```

Recursion over a counts array: sorted order and no duplicates by construction.

Sample input

```
BAC
```

Line-by-line walkthrough

#	Where	What happens
1	line 15	Skip letters with no copies left.
2	line 18	Choose: use one copy of letter c.
3	line 20	Explore the remaining positions.
4	line 21	Un-choose: put the letter back so another branch can use it.

SwapBasedPermutations.java — Lecture program (swap-based) (from the slides)

```
1  import java.util.Arrays;
2  import java.util.HashSet;
3  import java.util.Scanner;
4  import java.util.Set;
5
6  public class SwapBasedPermutations {
7   // The lecture's approach (sort the input, then generate by swapping, skipping repeated
       letters).
8   // It produces every DISTINCT permutation exactly once, but NOT in sorted order.
9   static StringBuilder out = new StringBuilder();
10
11  static void permute(char[] chars, int index) {
12  if (index == chars.length - 1) {
13  out.append(String.valueOf(chars)).append(' ');
14  return;
15  }
16  Set<Character> used = new HashSet<>();                                    1
17  for (int i = index; i < chars.length; i++) {
18  if (used.contains(chars[i])) {
19  continue;
20  }
21  used.add(chars[i]);
22  swap(chars, index, i);                                                    2
23  permute(chars, index + 1);
24  swap(chars, index, i);                                                    3
25  }
26  }
27
28  static void swap(char[] a, int i, int j) {
29  char t = a[i];
30  a[i] = a[j];
31  a[j] = t;
32  }
33
34  public static void main(String[] args) {
35  Scanner sc = new Scanner(System.in);
36  char[] chars = sc.next().toCharArray();
37  Arrays.sort(chars);
38  permute(chars, 0);
39  System.out.println(out.toString().trim());
40  }
41 }
```

HashSet per level avoids duplicate letters; swapping makes the order non-lexicographic.

Sample input

```
BAC
```

Line-by-line walkthrough

#	Where	What happens
1	line 16	Letters already tried at this position — prevents duplicate permutations.
2	line 22	Bring chars[i] to the current position.
3	line 24	Swap back to restore the array — but the restored order is not the same as the sorted order for the next branch, which is why output is not sorted.

THE SLIDE CODE DOES NOT SORT ITS OUTPUT

Run `SwapBasedPermutations` with input `BAC`: it prints `ABC ACB BAC BCA CBA CAB` — the last two are out of order (CAB must precede CBA). The swap-and-restore technique generates every permutation exactly once, but not in dictionary order, even if the input was sorted first.

Dry run: watch it execute

Successive next-permutation steps.

Algorithm: Next permutation. **Input:** s = `AAB`.

Pseudocode

```
sort(a)
repeat:
 print a
 i = rightmost index with a[i] < a[i+1]
 j = rightmost index with a[j] > a[i]
 swap(a[i], a[j]); reverse(a[i+1..])
```

Step 1 of 3. Permutation #1: AAB. Pivot index 1 (A).

= 1 · permutation = AAB · pivot = 1

current	A	A	B
	0	1	2

Step 2 of 3. Permutation #2: ABA. Pivot index 0 (A).

= 2 · permutation = ABA · pivot = 0

current	A	B	A
	0	1	2

Step 3 of 3. Permutation #3: BAA — the last one (no pivot).

= 3 · permutation = BAA · pivot = —

current	B	A	A
	0	1	2

RESULT

3 distinct permutations

Complexity

There are up to n! permutations, each taking O(n) to print, so any algorithm is $\Omega(n \cdot n!/(\Pi c_i!))$ — factorial in the worst case. Next-permutation uses O(1) extra space; backtracking uses O(n) for the recursion stack and the counts.

For n = 10 distinct letters: 3 628 800 permutations — fine. For n = 13 it is 6.2×10^9 — too many to print.

Common mistakes

Assuming swap-based recursion is sorted

Problem

```
Arrays.sort(chars); permute(chars, 0); // slide
```

Why: After swap/restore operations the remaining letters are no longer in sorted order. For "ABC" it prints CBA before CAB.

Fix: Use next-permutation or count-based backtracking, or sort the final list.

Duplicates from repeated letters

Problem

```
for (int i = index; i < n; i++) { swap; recurse; swap; } // no used-set
```

Why: "AAB" produces 6 outputs with repeats.

Fix: Skip a letter if it was already tried at this position (set/counts).

Not resetting the counts

Problem

```
counts[c]--; build(...); // missing counts[c]++
```

Why: Later branches see too few letters and never complete.

Fix: Always undo the choice after the recursive call.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · How many distinct permutations? — *Beginner*

Read a word (≤ 20 letters). Print the number of distinct permutations.

Example input	Expected output
AAB	3

Hint 1: n! divided by the factorial of every letter count. **Hint 2:** long is enough for n ≤ 20.

Exercise 2 · Print them sorted — *Intermediate*

Read a word (≤ 8 letters). Print every distinct permutation, one per line, in alphabetical order.

Example input	Expected output
ABA	AAB ⏎ ABA ⏎ BAA

Hint 1: Counts array over the 256 characters. **Hint 2:** Try letters in increasing order.

Exercise 3 · The k-th permutation — *Challenge*

Read a word (≤ 15 letters) and k. Print the k-th distinct permutation in sorted order (1-based) WITHOUT generating the others, or `NONE` if k is too large.

Example input	Expected output
DBCA 10	BCDA

Hint 1: At each position, for each letter, count how many permutations start with it: arrangements of the remaining multiset. **Hint 2:** If k ≤ that count, take the letter; otherwise subtract and continue.

Quick check

Answers are in the answer key at the end of this part.

Q1. How many distinct permutations does "AABB" have?

- **A.** 4
- **B.** 6
- **C.** 12
- **D.** 24

Q2. What follows "ACB" in lexicographic order of "ABC"?

```
// permutations of A, B, C
```

Your answer: _____

Q3. Which statement about the slide's swap-based generator is true?

- **A.** It prints duplicates
- **B.** It prints every distinct permutation once, but not in sorted order
- **C.** It prints sorted output
- **D.** It never terminates

Q4. Time to print all permutations of 8 distinct letters is...

- **A.** O(8)
- **B.** O(8^2)
- **C.** O(8·8!)
- **D.** O(2^8)

Interview / exam questions

- Generate all distinct permutations with duplicates.
- Explain next-permutation in 4 steps.
- Find the k-th permutation without enumerating.
- Why is the lecture's "sort first" insufficient?

Watch & read

Video: Permutations of an Array/String | Recursion & Backtracking — Apna College · 22:55. *Distinct permutations with duplicates*

Video: Next Permutation - Intuition in Detail | Brute to Optimal — take U forward · 28:15. *The next-permutation algorithm*

REVISION CARD (30–60 SECONDS)

- Distinct permutations = n! / Π(c_i!).
- Next permutation: pivot, swap with next larger on the right, reverse suffix.
- Count-based backtracking: letters in alphabetical order, choose/explore/un-choose.
- Swap-based generation is NOT lexicographic.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide claims its program prints distinct permutations "in sorted order". Running it on "BAC" prints ABC ACB BAC BCA CBA CAB, which is not sorted. The slide's own "Steps" (sort, count permutations, then find the next greater string repeatedly) describe the NEXT-PERMUTATION method, which IS sorted — this is what `NextPermutation.java` implements. • The slide's example output for "DBCA" lists the 24 sorted permutations (correct).

5.29 N-Queens (Backtracking)

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 60 min · ★ on the minimum-viable path

Place N queens on an N × N board so that none attack each other — the textbook backtracking problem.

YOU WILL BE ABLE TO

- Explain what a queen attacks and why at most one queen fits in each row, column and diagonal.
- Describe backtracking as "choose, explore, un-choose" and trace it on the 4-queens board.
- Write isSafe and the recursive row-by-row solver, then speed up the safety test with three boolean arrays.
- Count solutions for N up to 10 and state the complexity honestly.

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Two-Dimensional Arrays (Matrices) · Algorithm Design Paradigms

The idea

A chess **queen** attacks along its row, its column and both diagonals. N-Queens asks for N queens on an N × N board with **no two attacking each other**. For N = 4 there are exactly 2 solutions; for N = 8 there are 92.

Because two queens in the same row attack each other, a solution has **exactly one queen per row**. So we place queens row by row: for row 0 try column 0; if that is safe go to row 1 and try its columns; if some row has **no** safe column, we made a bad choice earlier — **backtrack**: undo the previous queen and try its next column.

```
N = 4 (row by row)
row 0: Q . . . row 1: cols 0,1 attacked → Q at col 2: . . Q .
row 2: every column attacked → dead end → BACKTRACK to row 1 → try col 3 ...
...eventually: . Q . .
 . . . Q
 Q . . .
 . . Q .
```

WHY THIS WORKS

Backtracking explores a tree of partial placements but **prunes** every branch as soon as it breaks a rule, instead of building all N^N full boards and checking them. The key to correctness is **un-choosing**: after the recursive call returns without success, the queen must be removed so the board is exactly as before and the next column can be tried. The key to speed is the early rejection.

Syntax

```java
int[] col = new int[n]; // col[row] = column of the queen in that row
boolean[] cols = new boolean[n]; // column used?
boolean[] diag1 = new boolean[2 * n - 1]; // "/" diagonals: row + col
boolean[] diag2 = new boolean[2 * n - 1]; // "" diagonals: row - col + n - 1
if (cols[c] || diag1[r + c] || diag2[r - c + n - 1]) continue; // attacked: prune
cols[c] = diag1[r + c] = diag2[r - c + n - 1] = true; // place
solve(r + 1);
cols[c] = diag1[r + c] = diag2[r - c + n - 1] = false; // remove (backtrack)
```

A tiny example

Two queens on the same diagonal satisfy |Δrow| = |Δcol|.

```
// safe if no earlier row has a queen in the same column or on a diagonal
for (int r = 0; r < row; r++) {
 if (queenCol[r] == col || Math.abs(queenCol[r] - col) == row - r) return false;
}
return true;
```

Guided example

NQueensFirst.java — First solution by backtracking (simple solution)

```
1  import java.util.Scanner;
2
3  public class NQueensFirst {
4   static int n;
5   static int[][] board;
6
7   // Can a queen stand at (row, col)? Check the column and both diagonals ABOVE the row
8   // (rows below are still empty because we fill the board one row at a time).
9   static boolean isSafe(int row, int col) {                                    (1)
10  for (int r = 0; r < row; r++) {
11  if (board[r][col] == 1) {
12  return false; // same column
13  }
14  }
15  for (int r = row - 1, c = col - 1; r >= 0 && c >= 0; r--, c--) {             (2)
16  if (board[r][c] == 1) {
17  return false; // upper-left diagonal
18  }
19  }
20  for (int r = row - 1, c = col + 1; r >= 0 && c < n; r--, c++) {              (3)
21  if (board[r][c] == 1) {
22  return false; // upper-right diagonal
23  }
24  }
25  return true;
26  }
27
28  // Place one queen per row. Returns true as soon as every row has a queen.
29  static boolean solve(int row) {
30  if (row == n) {                                                              (4)
31  return true; // all N queens placed
32  }
33  for (int col = 0; col < n; col++) {
34  if (isSafe(row, col)) {
35  board[row][col] = 1; // choose                                               (5)
36  if (solve(row + 1)) { // explore the next row                               (6)
37  return true;
38  }
39  board[row][col] = 0; // un-choose: BACKTRACK and try the next column         (7)
40  }
41  }
42  return false; // no column works in this row -> previous row must move
43  }
44
45  public static void main(String[] args) {
```

(code continued)

```
    Scanner sc = new Scanner(System.in);
    n = sc.nextInt();
    board = new int[n][n];
    if (!solve(0)) {
    System.out.println("Solution does not exist");
    return;
    }
    for (int[] row : board) {
    StringBuilder sb = new StringBuilder();
    for (int cell : row) {
    sb.append(sb.length() == 0 ? "" : " ").append(cell);
    }
    System.out.println(sb);
    }
    }
  }
```

Row-by-row search with a board scan for the safety test. Stops at the first solution.

Sample input

```
4
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	Only rows ABOVE `row` can already hold queens, so only those need checking.
2	line 15	Walk up-left along the diagonal.
3	line 20	Walk up-right along the other diagonal.
4	line 30	Base case: every row has a queen — success.
5	line 35	CHOOSE: place a queen.
6	line 36	EXPLORE: try to place the rest; propagate success upwards.
7	line 39	UN-CHOOSE (backtrack): remove the queen and try the next column.

NQueensCount.java — Counting all solutions (optimised solution)

```java
import java.util.Scanner;

public class NQueensCount {
 static int n;
 static boolean[] column; // column c already has a queen
 static boolean[] diagDown; // diagonal r - c + (n - 1)
 static boolean[] diagUp; // anti-diagonal r + c
 static int solutions;

 // O(1) safety test: three boolean arrays instead of scanning the board.
 static void place(int row) {
 if (row == n) {
 solutions++;
 return;
 }
 for (int col = 0; col < n; col++) {
 int d1 = row - col + n - 1;
 int d2 = row + col;
 if (column[col] || diagDown[d1] || diagUp[d2]) {
 continue;
 }
 column[col] = diagDown[d1] = diagUp[d2] = true; // choose
 place(row + 1); // explore
 column[col] = diagDown[d1] = diagUp[d2] = false; // un-choose
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 n = sc.nextInt();
 column = new boolean[n];
 diagDown = new boolean[2 * n - 1];
 diagUp = new boolean[2 * n - 1];
 place(0);
 System.out.println("N = " + n + ": " + solutions + " solutions");
 }
}
```

1 2 3 4 5

Three boolean arrays make every safety test O(1). 92 solutions for N = 8.

Sample input

```
8
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	Cells on the same "\" diagonal share row − col; shifted by n−1 to be a valid index.
2	line 18	Cells on the same "/" diagonal share row + col.
3	line 22	Mark the three lines as occupied.
4	line 24	Free them again on the way back.
5	line 13	Do not stop at the first solution — keep exploring to count all of them.

The slide fills the board column by column and prints `0 0 1 0 / 1 0 0 0 / 0 0 0 1 / 0 1 0 0`. Filling row by row finds the mirror-image solution `0 1 0 0 / 0 0 0 1 / 1 0 0 0 / 0 0 1 0`; both are valid

because the 4-queens solutions are mirror images of each other.

Dry run: watch it execute

Placing, rejecting and backtracking row by row for N = 4.

Algorithm: N-Queens backtracking. **Input:** n = `4`.

Pseudocode

```
solve(row):
 if row == n: success
 for col in 0..n-1:
 if safe(row, col):
 place; if solve(row+1): return true
 remove (backtrack)
 return false
```

The full trace has 30 steps; 14 evenly spaced ones are shown. Run the rest in the website's Dry-run lab.

Step 1 of 30. Row 0: column 0 is safe → place a queen.

row = 0 · col = 0 · queens = 1

♛	·	·	·
·	·	·	·
·	·	·	·
·	·	·	·

Step 3 of 30. Row 1: column 1 is attacked — skip.

row = 1 · col = 1 · queens = 1

♛	·	·	·
·	·	·	·
·	·	·	·
·	·	·	·

Step 5 of 30. Row 2: column 0 is attacked — skip.

row = 2 · col = 0 · queens = 2

♛	·	·	·
·	·	♛	·
·	·	·	·
·	·	·	·

Step 8 of 30. Row 2: column 3 is attacked — skip.

row = 2 · col = 3 · queens = 2

♛	·	·	·
·	·	♛	·
·	·	·	·
·	·	·	·

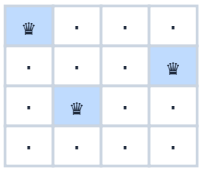
Step 10 of 30. Row 1: column 3 is safe → place a queen.

row = 1 · col = 3 · queens = 2

♛	·	·	·
·	·	·	♛
·	·	·	·
·	·	·	·

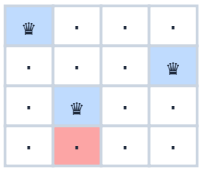
Step 12 of 30. Row 2: column 1 is safe → place a queen.

row = 2 · col = 1 · queens = 3



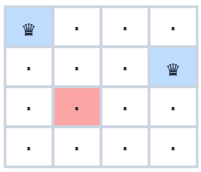
Step 14 of 30. Row 3: column 1 is attacked — skip.

row = 3 · col = 1 · queens = 3



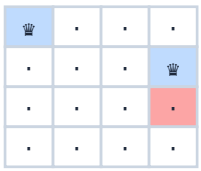
Step 17 of 30. Row 3 had no safe column → remove the queen from row 2, column 1 (backtrack).

row = 2 · col = 1 · queens = 2



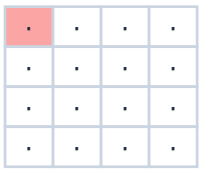
Step 19 of 30. Row 2: column 3 is attacked — skip.

row = 2 · col = 3 · queens = 2



Step 21 of 30. Row 1 had no safe column → remove the queen from row 0, column 0 (backtrack).

row = 0 · col = 0 · queens = 0



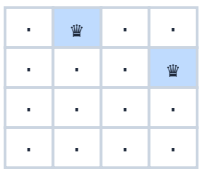
Step 23 of 30. Row 1: column 0 is attacked — skip.

row = 1 · col = 0 · queens = 1



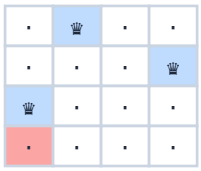
Step 26 of 30. Row 1: column 3 is safe → place a queen.

row = 1 · col = 3 · queens = 2



Step 28 of 30. Row 3: column 0 is attacked — skip.

row = 3 · col = 0 · queens = 3



Step 30 of 30. Row 3: column 2 is safe → place a queen.

row = 3 · col = 2 · queens = 4



> **RESULT**
> solution (column per row): 2 4 1 3

Complexity

There are N^N ways to put one queen in each row; pruning shrinks that dramatically but the worst case is still exponential. The number of nodes visited grows roughly like N! (fewer in practice): N = 8 takes about 15 000 recursive calls; N = 12 about 10 million; N = 15 several billion.

Time: **O(N!)** upper bound (honest worst case). Space: O(N) for the recursion and the three arrays (the slide's N×N board is $O(N^2)$).

N	1	2	3	4	5	6	7	8	9	10
solutions	1	0	0	2	10	4	40	92	352	724

Common mistakes

Forgetting to un-place the queen

Problem

```
board[row][col] = 1;
solve(row + 1); // no reset
```

Why: Dead-end branches leave queens on the board, which then block valid placements, so correct solutions are missed.

Fix: if (!solve(row + 1)) board[row][col] = 0;

Checking the wrong diagonal index

Problem

```
diagDown[row − col] // negative index for col > row
```

Why: row − col ranges from −(n−1) to n−1: negative indices crash.

Fix: diagDown[row - col + n - 1]

Stopping at the first solution when counting

Problem

```
if (place(row + 1)) return true; // in a counting program
```

Why: The function returns as soon as one solution is found; the count is 1.

Fix: Count in the base case and return nothing (void).

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Validate a placement — *Beginner*

Read n and then n integers: the column (1-based) of the queen in each row. Print VALID if no two queens attack each other, else INVALID.

Example input	Expected output
4 ⏎ 2 4 1 3	VALID

Hint 1: Same column, or |Δrow| == |Δcol|, means attack.

Exercise 2 · Count solutions — *Intermediate*

Read N (1 ≤ N ≤ 10) and print the number of distinct ways to place N non-attacking queens.

Example input	Expected output
6	4

Hint 1: Row by row; three boolean arrays for column and diagonals.

Exercise 3 · Lexicographically first solution — *Challenge*

Read N (1 ≤ N ≤ 12). Print the solution that is smallest when read as the column of the queen in row 1, row 2, ... (columns 1-based, separated by spaces), or NONE.

Example input	Expected output
8	1 5 8 6 3 7 2 4

Hint 1: Trying columns in increasing order and stopping at the first success gives the lexicographically first.

Quick check

Answers are in the answer key at the end of this part.

Q1. How many solutions does the 4-queens problem have?

- **A.** 1
- **B.** 2
- **C.** 4
- **D.** 24

Q2. Two queens at (1, 2) and (3, 4) attack each other because...

- **A.** same row
- **B.** same column
- **C.** same diagonal
- **D.** they do not

Q3. Complete the backtracking step.

```
board[row][col] = 1;
if (solve(row + 1)) return true;
board[row][col] = ______;
```

Q4. What does "backtracking" mean here?

- **A.** Sorting the board
- **B.** Undoing the last choice and trying the next option
- **C.** Starting the whole search again
- **D.** Reading the board backwards

Q5. Which strategy fits "find all arrangements that satisfy constraints"?

- **A.** Greedy
- **B.** Binary search
- **C.** Backtracking
- **D.** Sorting

Interview / exam questions

- Why can you place exactly one queen per row?
- How do you check diagonals in O(1)?
- What is the complexity? Can you do better with bitmasks?
- How many solutions are there for N = 8 and how many are distinct up to symmetry (12)?

Watch & read

Video: 6.1 N Queens Problem using Backtracking — Abdul Bari · 13:41. *N-Queens by backtracking*

Video: Visualization of N Queens (5×5 board) — G Bhat · 2:39. *N-Queens implementation*

REVISION CARD (30–60 SECONDS)

- One queen per row; try columns left→right.
- choose → explore → un-choose.
- Conflict: same column, or row − col equal, or row + col equal.
- Counts: 1,0,0,2,10,4,40,92,352,724 for N = 1…10. O(N!) worst case.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide fills the board column by column (`col`), so its printed solution is the transpose/mirror of the one a row-by-row search prints. Both are valid. • The slide says "queen can move any number of steps in any direction … can't change direction" and then lists only rows and columns; diagonals are an essential part of the attack rule.

5.30 Maze Solving (Rat in a Maze)

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 55 min · ★ on the minimum-viable path

Find a path from the entry to the exit of a grid maze by depth-first backtracking, then the shortest path with BFS.

YOU WILL BE ABLE TO

- Model a maze as a 2-D array and neighbours as direction offsets.
- Write a recursive depth-first search that marks the solution path and backtracks from dead ends.
- Explain why a `visited` array is required when moves may go back.
- Use a queue (BFS) to find the shortest route.

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Two-Dimensional Arrays (Matrices) · N-Queens (Backtracking)

The idea

A maze is a grid of **walls** and **open cells** with one entry and one exit. The slide encodes it as: `0` road, `1` wall, `2` entry, `3` exit, `4` a cell on the path that the algorithm found. A rat enters at the entry and tries to reach the exit.

Depth-first search with backtracking is how you would explore a maze with chalk: walk forward, mark where you have been, and if you reach a dead end, walk back and try the next turn.

```
2 0 0 1 0 2 4 4 1 0 (4 = the path found)
1 1 0 1 0 → 1 1 4 1 0
0 0 0 0 0 4 4 4 0 0
0 1 1 1 1 4 1 1 1 1
0 0 0 0 3 4 4 4 4 3
```

WHY THIS WORKS

A cell is on a path to the exit if it is the exit, or if at least one of its neighbours leads to the exit. The recursion asks exactly that: `dfs(cell)` returns true if the exit is reachable from here without revisiting cells. The `visited` array turns an endless walk in circles into a finite search; marking the successful route only on the way **back** (when `dfs` returns true) means dead-end explorations leave no trace in the final path.

Depth-first search finds *a* path, not necessarily the shortest one. **Breadth-first search** (BFS) explores the maze in rings of increasing distance using a queue, so the first time it reaches the exit it has found a shortest route.

Syntax

Direction arrays replace four copies of nearly identical code.

```
int[] dr = {1, 0, -1, 0}; // down, right, up, left
int[] dc = {0, 1, 0, -1};
for (int d = 0; d < 4; d++) {
 int nr = r + dr[d], nc = c + dc[d];
}
```

A tiny example

```
static boolean isSafe(int[][] maze, int x, int y) {
 return x >= 0 && x < N && y >= 0 && y < N && maze[x][y] == 1;
}
```

Guided example

RatMazeDownRight.java — Rat in a maze (down/right only) (from the slides)

```
1  public class RatMazeDownRight {
2   static final int N = 4;
3
4   static boolean isSafe(int[][] maze, int x, int y) {
5   return x >= 0 && x < N && y >= 0 && y < N && maze[x][y] == 1; // inside the grid and an (1)
        open cell
6   }
7
8   // The lecture's version: the rat may only move DOWN or RIGHT.
9   static boolean solve(int[][] maze, int x, int y, int[][] sol) {
10  if (x == N - 1 && y == N - 1) {                                                          (2)
11  sol[x][y] = 1;                                                                           (3)
12  return true; // reached the exit
13  }
14  if (isSafe(maze, x, y)) {
15  sol[x][y] = 1; // mark this cell as part of the path
16  if (solve(maze, x + 1, y, sol)) return true; // try down                                 (4)
17  if (solve(maze, x, y + 1, sol)) return true; // try right                               (5)
18  sol[x][y] = 0; // dead end: un-mark (backtrack)                                          (6)
19  }
20  return false;
21  }
22
23  public static void main(String[] args) {
24  int[][] maze = {
25  {1, 0, 0, 0},
26  {1, 1, 0, 1},
27  {0, 1, 0, 0},
28  {1, 1, 1, 1}
29  };
30  int[][] sol = new int[N][N];
31  if (!solve(maze, 0, 0, sol)) {
32  System.out.println("Solution doesn't exist");
33  return;
34  }
35  for (int[] row : sol) {
36  StringBuilder sb = new StringBuilder();
37  for (int v : row) sb.append(' ').append(v).append(' ');
38  System.out.println(sb);
39  }
40  }
41 }
```

The lecture's recursive backtracker. `sol` marks the path; dead ends are unmarked.

Line-by-line walkthrough

#	Where	What happens
1	line 5	Safe = inside the grid AND an open cell. Bounds are checked first so the array access cannot fail.
2	line 10	Reached the bottom-right exit: mark it and succeed.
3	line 11	Tentatively mark this cell as part of the path.
4	line 16	Try moving down first...
5	line 17	...then right.
6	line 18	Both failed: this cell is a dead end. Un-mark it and return false so the caller tries its next option.

MazeSolver.java — General maze solver (4 directions) (optimised solution)

```
import java.util.Scanner;

public class MazeSolver {
 // Encoding from the lecture slide:
 // 0 road, 1 wall, 2 entry, 3 exit, 4 cell on the path found
 static int rows, cols;
 static int[][] g;
 static boolean[][] visited;
 static final int[] DR = {1, 0, -1, 0}; // down, right, up, left                    (1)
 static final int[] DC = {0, 1, 0, -1};

 static boolean dfs(int r, int c) {
 if (r < 0 || r >= rows || c < 0 || c >= cols || g[r][c] == 1 || visited[r][c]) {
 return false; // outside, wall, or already explored
 }
 visited[r][c] = true; // remember: never walk in circles                          (2)
 if (g[r][c] == 3) {                                                              (3)
 return true; // exit found
 }
 for (int d = 0; d < 4; d++) {
 if (dfs(r + DR[d], c + DC[d])) {                                                 (4)
 if (g[r][c] == 0) g[r][c] = 4; // this cell lies on the successful route         (5)
 return true;
 }
 }
 return false; // all four directions failed: dead end
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 rows = sc.nextInt();
 cols = sc.nextInt();
 g = new int[rows][cols];
 visited = new boolean[rows][cols];
 int sr = 0, sc0 = 0;
 for (int i = 0; i < rows; i++) {
 for (int j = 0; j < cols; j++) {
 g[i][j] = sc.nextInt();
 if (g[i][j] == 2) {
 sr = i;
 sc0 = j;
 }
 }
 }
 if (!dfs(sr, sc0)) {
 System.out.println("No path from entry to exit");
 return;
 }
 for (int[] row : g) {
 StringBuilder sb = new StringBuilder();
 for (int v : row) sb.append(sb.length() == 0 ? "" : " ").append(v);
 System.out.println(sb);
 }
 }
}
```

Depth-first search with a visited array; reads the maze from standard input; marks the route with 4.

Sample input

```
5 5
2 0 0 1 0
1 1 0 1 0
0 0 0 0 0
0 1 1 1 1
0 0 0 0 3
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	Four neighbours as offset arrays.
2	line 16	Never enter a cell twice — otherwise moving up/left would loop forever.
3	line 17	Exit cell: the base case.
4	line 21	Explore each neighbour; the first one that succeeds finishes the search.
5	line 22	Mark the route only on the way back, and keep the entry (2) and exit (3) labels untouched.

> **NOTE**
> The slide shows the 4×4 maze with 1 = path in the code but describes the model 0 = road, 1 = wall in the text. The two programs above keep each convention explicit; mixing them is the most common bug when adapting this code.

Dry run: watch it execute

The rat explores down first, then right, marking and un-marking cells.

Algorithm: Rat in a maze (down/right). **Input:** grid = `1000,1101,0100,1111`.

Pseudocode

```
solve(x, y):
 if (x, y) is the exit: mark; return true
 if safe(x, y):
 mark sol[x][y]
 if solve(x+1, y): return true (down)
 if solve(x, y+1): return true (right)
 unmark sol[x][y] (dead end)
 return false
```

Step 1 of 7. Mark (0, 0) as part of the path.

x = 0 · y = 0

→	▓	▓	▓
1	1	▓	1
▓	1	▓	▓
1	1	1	1

Step 2 of 7. Mark (1, 0) as part of the path.

x = 1 · y = 0

→	▓	▓	▓
→	1	▓	1
▓	1	▓	▓
1	1	1	1

Step 3 of 7. Mark (1, 1) as part of the path.

x = 1 · y = 1

→	▓	▓	▓
→	→	▓	1
▓	1	▓	▓
1	1	1	1

Step 4 of 7. Mark (2, 1) as part of the path.

x = 2 · y = 1

→	▓	▓	▓
→	→	▓	1
▓	→	▓	▓
1	1	1	1

Step 5 of 7. Mark (3, 1) as part of the path.

x = 3 · y = 1

→	▓	▓	▓
→	→	▓	1
▓	→	▓	▓
1	→	1	1

Step 6 of 7. Mark (3, 2) as part of the path.

x = 3 · y = 2

→	▓	▓	▓
→	→	▓	1
▓	→	▓	▓
1	→	→	1

Step 7 of 7. (3, 3) is the exit — success!

x = 3 · y = 3

→	▓	▓	▓
→	→	▓	1
▓	→	▓	▓
1	→	→	→

> **RESULT**
> path found

Complexity

DFS with a visited array: each cell is entered at most once and examines 4 neighbours → **O(R·C) time**, O(R·C) space for `visited` plus the recursion stack (up to R·C deep).

Without a visited array (down/right only) each cell is on a monotone path, so the search is still finite but can be exponential: O(2^(R+C)) in the worst case.

BFS: O(R·C) time and space and it returns the shortest path length.

> **WATCH OUT**
> Recursion depth can reach R·C. For big mazes (1000 × 1000) use an explicit stack/queue to avoid StackOverflowError.

Common mistakes

No visited array when moving in 4 directions

Problem

```
solve(x + 1, y) || solve(x, y + 1) || solve(x - 1, y) || solve(x, y - 1)
```

Why: The rat steps right, then left, then right again ... infinite recursion → StackOverflowError.

Fix: Mark cells as visited before exploring neighbours.

Checking maze[x][y] before the bounds

Problem

```
maze[x][y] == 1 && x >= 0 && x < N
```

Why: ArrayIndexOutOfBounds for x = −1 or x = N: the operands of && are evaluated left to right.

Fix: Test the bounds first.

Marking the path on the way in

Problem

```
g[r][c] = 4; // before exploring neighbours
```

Why: Dead-end cells stay marked as "path". Mark when dfs returns true, or un-mark on failure.

Fix: mark after the successful recursive call (or reset on failure)

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · How many cells can the rat reach? — *Beginner*

Read R, C and an R × C maze (0 road, 1 wall, 2 entry, 3 exit). Print the number of non-wall cells reachable from the entry by moving up, down, left or right (the entry itself counts).

Example input	Expected output
3 3 ⏎ 2 0 1 ⏎ 1 0 1 ⏎ 1 0 3	5

Hint 1: Flood fill with recursion or a stack. **Hint 2:** Mark cells visited.

Exercise 2 · Is there a way out? — *Intermediate*

Same input. Print YES if the exit can be reached from the entry, else NO.

Example input	Expected output
3 3 ⏎ 2 1 3 ⏎ 0 1 0 ⏎ 0 0 0	YES

Hint 1: DFS returning boolean.

Exercise 3 · Shortest path — *Challenge*

Same input. Print the minimum number of moves from the entry to the exit, or -1.

- 1 ≤ R, C ≤ 500

Example input	Expected output
3 3 ⏎ 2 0 0 ⏎ 1 1 0 ⏎ 3 0 0	6

Hint 1: Breadth-first search with a queue and a distance array.

Quick check

Answers are in the answer key at the end of this part.

Q1. Why does a 4-direction maze solver need a visited array?

- **A.** To save memory
- **B.** To avoid walking in circles forever
- **C.** To count steps
- **D.** To find the shortest path

Q2. Which algorithm guarantees the SHORTEST path in an unweighted maze?

- **A.** DFS
- **B.** BFS
- **C.** Binary search
- **D.** Quick sort

Q3. What does `sol[x][y] = 0;` after both recursive calls do?

- **A.** Mark success
- **B.** Un-mark a dead-end cell (backtrack)
- **C.** Reset the maze
- **D.** Nothing

Q4. DFS on an R × C maze with a visited array runs in...

- **A.** O(R + C)
- **B.** O(R·C)
- **C.** O(2^(R·C))
- **D.** O($R^2 \cdot C^2$)

Interview / exam questions

- DFS vs BFS for maze solving — when would you choose each?
- How do you print the actual shortest path with BFS?
- How would you avoid recursion depth problems?

Watch & read

Video: L19. Rat in A Maze | Backtracking — take U forward · 25:10. *Rat in a maze*

Video: Rat in a Maze Problem | Backtracking — Apna College · 32:45. *Backtracking on a grid*

REVISION CARD (30–60 SECONDS)

• Maze = grid; neighbours via dr/dc arrays; bounds before wall test.
• DFS: visited, recurse, mark route when success returns true, backtrack on failure.
• BFS (queue) = shortest path.
• O(R·C).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide models the maze as 0 road / 1 wall / 2 entry / 3 exit / 4 path, but its Java program uses 1 for an open cell and only moves down/right. The lesson provides one program for each convention and states which is which. • The slide's printed solution `1 0 0 0 / 1 1 0 0 / 0 1 0 0 / 0 1 1 1` matches `RatMazeDownRight`. • The 5×5 matrix at the end of the slide has no stated solution; it needs up/left moves, which is why a visited-array solver is provided.

5.31 Hamiltonian Cycle

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 55 min · complete path

Does a graph contain a cycle that visits every vertex exactly once? Backtracking over an adjacency matrix.

YOU WILL BE ABLE TO

- Define a graph, adjacency matrix and Hamiltonian cycle.
- Write the backtracking search with a path array and an `isSafe` test.
- Explain why the start vertex can be fixed and why the problem is exponential.
- Run it on both slide graphs and predict the outcome.

Before this lesson: Trees, Graphs and Heaps · N-Queens (Backtracking) · Two-Dimensional Arrays (Matrices)

The idea

A **graph** is a set of vertices connected by edges. A **Hamiltonian cycle** is a closed walk that visits **every vertex exactly once** and returns to the start. (Compare an Eulerian cycle, which uses every *edge* once — a much easier problem.)

```
(0)--(1)--(2) cycle: 0 → 1 → 2 → 4 → 3 → 0
| / | \ |
| / | \ |
(3)---+--(4) edges 0–1 1–2 2–4 4–3 3–0 are all present
```

The graph is stored as an **adjacency matrix**: `graph[i][j] == 1` if there is an edge between i and j. The search builds a `path` array: place vertex 0 first (any cycle can be rotated to start there), then at each position try every unused vertex that is adjacent to the previous one. When all V vertices are placed, the last must also be adjacent to the first.

WHY THIS WORKS

There is no known fast algorithm for this problem — it is NP-complete. Backtracking works because most partial paths fail quickly (no edge, or a vertex reused); the `path[pos] = -1` reset is the un-choose step that lets the loop try the next candidate. The cost is bounded by (V−1)! complete orderings.

Syntax

```
int[][] graph = {{0,1,0,1,0}, {1,0,1,1,1}, {0,1,0,0,1}, {1,1,0,0,1}, {0,1,1,1,0}}; // adjacency
    matrix
int[] path = new int[n]; Arrays.fill(path, -1); // path[pos] = vertex placed there
path[0] = 0; // fix the start: a cycle can start anywhere
boolean ok = graph[path[pos - 1]][v] == 1; // an edge from the previous vertex?
boolean fresh = true; // v not already on the path
for (int i = 0; i < pos; i++) if (path[i] == v) fresh = false;
// the cycle closes only if the LAST vertex is adjacent to path[0]
```

A tiny example

```
// is there an edge from the previous vertex, and is the candidate unused?
if (graph[path[pos - 1]][v] == 0) return false;
for (int i = 0; i < pos; i++) if (path[i] == v) return false;
return true;
```

Guided example

The two slide graphs (5 vertices) are `graph1` — which has the cycle 0 1 2 4 3 0 — and `graph2`, where vertices 3 and 4 both have only one neighbour besides vertex 1, which makes a cycle impossible.

HamiltonianCycle.java — Hamiltonian cycle by backtracking (optimised solution)

```
import java.util.Arrays;
import java.util.Scanner;

public class HamiltonianCycle {
 static int v;
 static int[][] graph;
 static int[] path;

 // Can vertex `candidate` be placed at position `pos` of the path?
 static boolean isSafe(int candidate, int pos) {                                  (1)
 if (graph[path[pos - 1]][candidate] == 0) {                                      (2)
 return false; // there is no edge from the previous vertex
 }
 for (int i = 0; i < pos; i++) {
 if (path[i] == candidate) {                                                      (3)
 return false; // each vertex may be used only once
 }
 }
 return true;
 }

 static boolean extend(int pos) {
 if (pos == v) {                                                                  (4)
 return graph[path[pos - 1]][path[0]] == 1; // all placed: the last must connect back to
     the start
 }
 for (int candidate = 1; candidate < v; candidate++) {
 if (isSafe(candidate, pos)) {
 path[pos] = candidate; // choose                                                 (5)
 if (extend(pos + 1)) {
 return true;
 }
 path[pos] = -1; // un-choose                                                     (6)
 }
 }
 return false;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 v = sc.nextInt();
 graph = new int[v][v];
 for (int i = 0; i < v; i++) {
 for (int j = 0; j < v; j++) {
```

(code continued)

```
    graph[i][j] = sc.nextInt();
    }
    }
    path = new int[v];
    Arrays.fill(path, -1);
    path[0] = 0; // a cycle can start anywhere: fix vertex 0
    if (!extend(1)) {
    System.out.println("Solution does not exist");
    return;
    }
    StringBuilder sb = new StringBuilder();
    for (int x : path) sb.append(x).append(' ');
    sb.append(path[0]);
    System.out.println("Solution exists: " + sb);
    }
   }
```

Complete version of the (fragmented) slide program, reading its graph from standard input.

Sample input

```
5
0 1 0 1 0
1 0 1 1 1
0 1 0 0 1
1 1 0 0 1
0 1 1 1 0
```

Line-by-line walkthrough

#	Where	What happens
1	line 10	Can `candidate` be the vertex at position `pos` of the path?
2	line 11	There must be an edge from the previously placed vertex.
3	line 15	And the vertex must not already be on the path.
4	line 23	All vertices placed: the cycle must close — the last vertex needs an edge back to vertex 0.
5	line 28	CHOOSE.
6	line 32	UN-CHOOSE: free the slot for the next candidate.
7	line 49	Fix the start. A cycle has no natural start, so every cycle appears as one that begins at vertex 0.

Dry run: watch it execute

Extending and abandoning partial paths.

Algorithm: Hamiltonian cycle. **Input:** matrix = `01010,10111,01001,11001,01110`.

Pseudocode

```
path[0] = 0
extend(pos):
 if pos == V: return edge(path[V-1], path[0])
 for v in 1..V-1:
 if edge(path[pos-1], v) and v not in path:
 path[pos] = v; if extend(pos+1): return true
 path[pos] = -1
 return false
```

Step 1 of 6. Fix the start at vertex 0.

pos = 1

path

0
0

Step 2 of 6. Position 1: vertex 1 is adjacent to 0 and unused → place it.

pos = 1 · v = 1

path

0	1
0	1

Step 3 of 6. Position 2: vertex 2 is adjacent to 1 and unused → place it.

pos = 2 · v = 2

path

0	1	2
0	1	2

Step 4 of 6. Position 3: vertex 4 is adjacent to 2 and unused → place it.

pos = 3 · v = 4

path

0	1	2	4
0	1	2	3

Step 5 of 6. Position 4: vertex 3 is adjacent to 4 and unused → place it.

pos = 4 · v = 3

path

0	1	2	4	3
0	1	2	3	4

Step 6 of 6. All 5 vertices placed and 3 → 0 is an edge: cycle closed!

pos = 5

path

0	1	2	4	3
0	1	2	3	4

> **RESULT**
> Hamiltonian cycle: 0 → 1 → 2 → 4 → 3 → 0

Complexity

Worst case the search examines every permutation of the remaining V−1 vertices: **O(V!)** time. Space: O(V) for `path` and the recursion stack (O(V²) for the matrix).

Smarter algorithms exist (Held–Karp dynamic programming: O(2^V · V²)), but nothing polynomial is known.

Common mistakes

Skipping the closing edge

Problem

```
if (pos == v) return true;
```

Why: A path through all vertices is a Hamiltonian PATH. A cycle also needs an edge from the last vertex back to the start.

Fix: return graph[path[pos - 1]][path[0]] == 1;

Not resetting path[pos]

Problem

```
path[pos] = v; if (extend(pos + 1)) return true; // no reset
```

Why: A stale value remains in the array; isSafe will think that vertex is already used.

Fix: path[pos] = -1;

Expecting the slide output for the second graph

Problem

```
// slide shows "0 1 2 4 3 0" after graph2
```

Why: That is the answer for graph1. graph2 has no Hamiltonian cycle: the program prints "Solution does not exist".

Fix: Run both and compare.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Verify a cycle — *Beginner*

Read V, the V × V adjacency matrix, and then V+1 vertex numbers. Print YES if they form a Hamiltonian cycle (start = end, every vertex once, consecutive vertices adjacent).

Example input	Expected output
4 ⏎ 0 1 0 1 ⏎ 1 0 1 0 ⏎ 0 1 0 1 ⏎ 1 0 1 0 ⏎ 0 1 2 3 0	YES

Hint 1: Check first = last, no repeats among the first V, and an edge between consecutive vertices.

Exercise 2 · Find a cycle — *Intermediate*

Read V and the matrix. Print `YES` followed by the first Hamiltonian cycle found starting at 0 (trying vertices in increasing order) — e.g. `YES 0 1 2 4 3 0` — or `NO`.

- 3 ≤ V ≤ 10

Example input	Expected output
5 ⏎ 0 1 0 1 0 ⏎ 1 0 1 1 1 ⏎ 0 1 0 0 1 ⏎ 1 1 0 0 0 ⏎ 0 1 1 0 0	NO

Hint 1: Backtracking with path[] and the closing check.

Exercise 3 · Count Hamiltonian cycles — *Challenge*

Read V (≤ 10) and the matrix. Count the Hamiltonian cycles that start and end at vertex 0, counting a cycle and its reverse as two different cycles.

Example input	Expected output
4 ⏎ 0 1 1 1 ⏎ 1 0 1 1 ⏎ 1 1 0 1 ⏎ 1 1 1 0	6

Hint 1: Same search but do not stop at the first success. **Hint 2:** K_4 has 3! = 6 orderings of the other vertices.

Quick check

Answers are in the answer key at the end of this part.

Q1. A Hamiltonian cycle visits...

- **A.** every edge once
- **B.** every vertex exactly once and returns to the start
- **C.** every vertex at least once
- **D.** the shortest route

Q2. Why can the search always start at vertex 0?

- **A.** Vertex 0 has most edges
- **B.** A cycle can be rotated to start anywhere
- **C.** Because arrays start at 0
- **D.** It cannot

Q3. Time complexity of the backtracking search?

- **A.** O(V)
- **B.** $O(V^2)$
- **C.** O(V!)
- **D.** O(log V)

Q4. A vertex with only one neighbour makes a Hamiltonian cycle...

- **A.** easier
- **B.** impossible
- **C.** unchanged
- **D.** a path

Interview / exam questions

- Hamiltonian vs Eulerian cycle?
- Is Hamiltonian cycle NP-complete? What does that imply?
- Bitmask DP for Hamiltonian cycle: complexity?
- How does this relate to the travelling salesman problem?

Watch & read

Video: 6.4 Hamiltonian Cycle - Backtracking — Abdul Bari · 18:35. *Hamiltonian cycle by backtracking*

Video: Euler Graph in Graph Theory | Euler Path & Euler Circuit with examples — Gate Smashers · 6:45. *Hamiltonian vs Eulerian paths*

REVISION CARD (30–60 SECONDS)

• Hamiltonian cycle: every vertex once, back to start.
• path[0] = 0; place vertices with edge from previous & unused; close with edge to path[0].
• choose / explore / reset to −1.
• O(V!) — NP-complete.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide listing of the Java class is scrambled over two slides (the `hamCycle` method header is missing and part of `main` is duplicated), so it does not compile as printed. The lesson program is the complete, working version. • The slide's final output line "0 1 2 4 3 0" is printed after `graph2`, but that graph has NO Hamiltonian cycle; the output belongs to `graph1`.

5.32 Knight's Tour & Warnsdorff's Rule

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 55 min · complete path

Move a knight so that it visits every square once — plain backtracking explodes, a one-line heuristic solves 8×8 instantly.

YOU WILL BE ABLE TO

- List the eight knight moves and check them against the board edges.
- Describe Warnsdorff's rule: always move to the square with the fewest onward moves.
- Implement the heuristic with a `degree` function and a deterministic tie-break.
- Compare it with exhaustive backtracking and verify a tour.

Before this lesson: N-Queens (Backtracking) · Maze Solving (Rat in a Maze) · Two-Dimensional Arrays (Matrices)

The idea

A **knight** moves in an "L": two squares in one direction and one square sideways — up to eight possible moves. A **knight's tour** visits every square of the board exactly once. It is a Hamiltonian path on the "knight graph".

Exhaustive backtracking (try a move, undo if stuck) works for 5 × 5 but explodes for 8 × 8. **Warnsdorff's rule** (1823) is a heuristic: from the current square move to the unvisited square that has the **fewest onward moves**. The idea: squares with few exits are the most likely to be stranded later, so visit them first.

```
knight at (x, y) can jump to
(x±1, y±2) and (x±2, y±1)
degree(square) = number of unvisited squares the knight could jump to from it
```

WHY THIS WORKS

Squares with few remaining exits (corners start with only 2) are dangerous: if you leave them for last you may never be able to enter them. Always visiting the *most constrained* square first is a classic "fail-first" principle from constraint solving. It is only a **heuristic** — it usually works for standard boards, but there is no proof it never fails; that is why the slide's program restarts with a random tie-break until it succeeds.

Syntax

```java
final int[] DX = { 1, 2, 2, 1, -1, -2, -2, -1}; // the eight knight moves
final int[] DY = { 2, 1, -1, -2, -2, -1, 1, 2};
int nx = x + DX[k], ny = y + DY[k];
boolean inside = nx >= 0 && ny >= 0 && nx < N && ny < N;
boolean free = inside && board[nx][ny] == -1; // -1 = not visited yet
// Warnsdorff: among free moves choose the one with the fewest onward free moves
```

A tiny example

```
int[] dx = {1, 1, 2, 2, -1, -1, -2, -2};
int[] dy = {2, -2, 1, -1, 2, -2, 1, -1};
```

Guided example

WarnsdorffTour.java — Warnsdorff's heuristic (optimised solution)

```
1   import java.util.Scanner;
2
3   public class WarnsdorffTour {
4    static final int[] DX = {1, 1, 2, 2, -1, -1, -2, -2}; // the eight knight moves          (1)
5    static final int[] DY = {2, -2, 1, -1, 2, -2, 1, -1};
6
7    static int n;
8    static int[][] board; // board[y][x] = move number (1..n*n), 0 = not visited yet
9
10   static boolean free(int x, int y) {
11   return x >= 0 && y >= 0 && x < n && y < n && board[y][x] == 0;
12   }
13
14   // degree = how many unvisited squares a knight standing on (x, y) could still jump to
15   static int degree(int x, int y) {                                                        (2)
16   int count = 0;
17   for (int m = 0; m < 8; m++) {
18   if (free(x + DX[m], y + DY[m])) {
19   count++;
20   }
21   }
22   return count;
23   }
24
25   // Warnsdorff's rule: always jump to the neighbour with the FEWEST onward moves.
26   // Ties are broken by the first move in the DX/DY order (so the result is deterministic)
         .
27   static boolean tour(int startX, int startY) {
28   int x = startX;
29   int y = startY;
30   board[y][x] = 1;
31   for (int step = 2; step <= n * n; step++) {
32   int bestMove = -1;
33   int bestDegree = 9;
34   for (int m = 0; m < 8; m++) {
35   int nx = x + DX[m];
36   int ny = y + DY[m];
37   if (free(nx, ny)) {
38   int d = degree(nx, ny);
39   if (d < bestDegree) {                                                                    (3)
40   bestDegree = d;
41   bestMove = m;
42   }
43   }
44   }
45   if (bestMove == -1) {                                                                    (5)
46   return false; // stuck: the heuristic failed from this start
47   }
48   x += DX[bestMove];
```

(code continued)

```
    y += DY[bestMove];
    board[y][x] = step;
    }
    return true;
    }

    public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    n = sc.nextInt();
    int sx = sc.nextInt();
    int sy = sc.nextInt();
    board = new int[n][n];
    if (!tour(sx, sy)) {
    System.out.println("Warnsdorff's rule got stuck");
    return;
    }
    for (int y = 0; y < n; y++) {
    StringBuilder sb = new StringBuilder();
    for (int x = 0; x < n; x++) {
    sb.append(String.format("%3d", board[y][x]));
    }
    System.out.println(sb);
    }
    }
   }
```

Greedy choice by minimum degree. $O(N^2 \cdot 8 \cdot 8) \approx O(N^2)$ steps.

Sample input

```
8 0 0
```

Line-by-line walkthrough

#	Where	What happens
1	line 4	The eight knight moves; the order also decides which move wins a tie.
2	line 15	Counts unvisited squares reachable from (x, y) — the number of "exits".
3	line 39	Strictly smaller degree wins, so ties go to the first move in the list (deterministic).
4	line 50	Number the square with its move number.
5	line 45	No unvisited neighbour: the heuristic got stuck before covering the board.

KnightBacktracking.java — Backtracking knight's tour (simple solution)

```
import java.util.Scanner;

public class KnightBacktracking {
 static final int[] DX = {2, 1, -1, -2, -2, -1, 1, 2};
 static final int[] DY = {1, 2, 2, 1, -1, -2, -2, -1};
 static int n;
 static int[][] board;
 static long calls = 0;

 // Plain backtracking: try every move in a fixed order, undo when stuck. Exponential —
     only fine for tiny boards.
 static boolean solve(int x, int y, int step) {
 calls++;
 board[y][x] = step;                                                    (1)
 if (step == n * n) {                                                   (2)
 return true;
 }
 for (int m = 0; m < 8; m++) {
 int nx = x + DX[m], ny = y + DY[m];
 if (nx >= 0 && ny >= 0 && nx < n && ny < n && board[ny][nx] == 0) {
 if (solve(nx, ny, step + 1)) {
 return true;
 }
 }
 }
 board[y][x] = 0; // backtrack                                          (3)
 return false;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 n = sc.nextInt();
 board = new int[n][n];
 boolean ok = solve(0, 0, 1);
 System.out.println(ok ? "tour found after " + calls + " calls" : "no tour");
 if (ok) {
 for (int y = 0; y < n; y++) {
 StringBuilder sb = new StringBuilder();
 for (int x = 0; x < n; x++) sb.append(String.format("%3d", board[y][x]));
 System.out.println(sb);
 }
 }
 }
}
```

Tries moves in a fixed order and undoes them when stuck; counts the recursive calls.

Sample input

```
5
```

Line-by-line walkthrough

#	Where	What happens
1	line 13	Choose: put the move number on the square.
2	line 14	Every square visited: success.
3	line 25	Un-choose and return false.

> **NOTE**
> The slide's program searches for a **closed** tour (the last square is a knight move from the first) with a random starting move index, repeating `while (!findClosedTour())`. That randomness is what makes it eventually succeed; the lesson program is deterministic and finds an open tour.

Dry run: watch it execute

Warnsdorff on a 5×5 board: each step shows the chosen square.

Algorithm: Warnsdorff's knight's tour. **Input:** n = `5`, x = `0`, y = `0`.

Pseudocode

```
board[start] = 1
for step = 2..n*n:
 among unvisited knight moves choose
 the one with the smallest degree
 (first move wins ties)
 move there; board = step
```

The full trace has 25 steps; 14 evenly spaced ones are shown. Run the rest in the website's Dry-run lab.

Step 1 of 25. Start on (0, 0).

x = 0 · y = 0 · step = 1

1	·	·	·	·
·	·	·	·	·
·	·	·	·	·
·	·	·	·	·
·	·	·	·	·

Step 3 of 25. Move 3: go to (0, 4), the square with the fewest onward moves (1).

x = 0 · y = 4 · step = 3 · degree chosen = 1

1	·	·	·	·
·	·	·	·	·
·	2	·	·	·
·	·	·	·	·
3	·	·	·	·

Step 5 of 25. Move 5: go to (4, 4), the square with the fewest onward moves (1).

x = 4 · y = 4 · step = 5 · degree chosen = 1

1	·	·	·	·
·	·	·	·	·
·	2	·	·	·
·	·	4	·	·
3	·	·	·	5

Step 7 of 25. Move 7: go to (4, 0), the square with the fewest onward moves (1).

x = 4 · y = 0 · step = 7 · degree chosen = 1

1	·	·	·	7
·	·	·	·	·
·	2	·	6	·
·	·	4	·	·
3	·	·	·	5

Step 8 of 25. Move 8: go to (2, 1), the square with the fewest onward moves (4).

x = 2 · y = 1 · step = 8 · degree chosen = 4

1	·	·	·	7
·	·	8	·	·
·	2	·	6	·
·	·	4	·	·
3	·	·	·	5

Step 10 of 25. Move 10: go to (4, 1), the square with the fewest onward moves (2).

x = 4 · y = 1 · step = 10 · degree chosen = 2

1	·	·	·	7
·	·	8	·	10
·	2	·	6	·
·	·	4	9	·
3	·	·	·	5

Step 12 of 25. Move 12: go to (0, 1), the square with the fewest onward moves (2).

x = 0 · y = 1 · step = 12 · degree chosen = 2

1	·	11	·	7
12	·	8	·	10
·	2	·	6	·
·	·	4	9	·
3	·	·	·	5

Step 14 of 25. Move 14: go to (3, 4), the square with the fewest onward moves (2).

x = 3 · y = 4 · step = 14 · degree chosen = 2

1	·	11	·	7
12	·	8	·	10
·	2	·	6	·
·	13	4	9	·
3	·	·	14	5

Step 16 of 25. Move 16: go to (3, 0), the square with the fewest onward moves (2).

x = 3 · y = 0 · step = 16 · degree chosen = 2

1	·	11	16	7
12	·	8	·	10
·	2	·	6	15
·	13	4	9	·
3	·	·	14	5

Step 18 of 25. Move 18: go to (0, 3), the square with the fewest onward moves (2).

x = 0 · y = 3 · step = 18 · degree chosen = 2

1	·	11	16	7
12	17	8	·	10
·	2	·	6	15
18	13	4	9	·
3	·	·	14	5

Step 19 of 25. Move 19: go to (2, 4), the square with the fewest onward moves (1).

x = 2 · y = 4 · step = 19 · degree chosen = 1

1	·	11	16	7
12	17	8	·	10
·	2	·	6	15
18	13	4	9	·
3	·	19	14	5

Step 21 of 25. Move 21: go to (3, 1), the square with the fewest onward moves (1).

x = 3 · y = 1 · step = 21 · degree chosen = 1

1	·	11	16	7
12	17	8	21	10
·	2	·	6	15
18	13	4	9	20
3	·	19	14	5

Step 23 of 25. Move 23: go to (2, 2), the square with the fewest onward moves (1).

x = 2 · y = 2 · step = 23 · degree chosen = 1

1	22	11	16	7
12	17	8	21	10
·	2	23	6	15
18	13	4	9	20
3	·	19	14	5

Step 25 of 25. Move 25: go to (0, 2), the square with the fewest onward moves (0).

x = 0 · y = 2 · step = 25 · degree chosen = 0

1	22	11	16	7
12	17	8	21	10
25	2	23	6	15
18	13	4	9	20
3	24	19	14	5

> **RESULT**
> tour completed: all 25 squares visited

Complexity

Warnsdorff: N^2 steps; each step looks at ≤ 8 neighbours and computes a degree (≤ 8 more) → **$O(N^2)$** with a constant of about 64. Space $O(N^2)$ for the board.

Backtracking: worst case $O(8^{(N^2)})$; the 5 × 5 board from corner (0,0) already needs thousands of calls, 8 × 8 is infeasible.

Common mistakes

Counting the current square in the degree

Problem

```
degree includes squares already visited
```

Why: The rule counts only UNVISITED onward squares, as the slide states: "we do not count moves that revisit any square already visited".

Fix: use isEmpty(x, y) in the degree loop

Using the heuristic as a proof

Problem

```
// "Warnsdorff always finds a tour"
```

Why: It is a heuristic: it can get stuck on some boards/starts (there are known failures for large boards). Detect the failure and retry with another tie-break.

Fix: check the return value and handle failure

x and y swapped when printing

Problem

```
a[x * N + y] in one place and a[y * N + x] in another
```

Why: Mixed row/column indexing produces a board that looks random and fails verification.

Fix: Use one convention: board[y][x] (row = y).

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Verify a tour — *Beginner*

Read n and an n × n board of move numbers 1..n^2. Print YES if every number appears once and consecutive numbers are a knight's move apart, else NO.

Example input	Expected output
5 ⏎ 1 6 15 10 21 ⏎ 14 9 20 5 16 ⏎ 19 2 7 22 11 ⏎ 8 13 24 17 4 ⏎ 25 18 3 12 23	YES

Hint 1: Store the position of each number. **Hint 2:** A knight move has |dx|, |dy| = (1, 2) or (2, 1).

Exercise 2 · Degree of a square — *Intermediate*

Read n, x, y (0-based) and print how many squares an unrestricted knight on (x, y) can move to on an n × n board.

Example input	Expected output
8 0 0	2

Hint 1: Test all eight moves against the board limits.

Exercise 3 · Reachable squares — *Challenge*

Read n and k. A knight starts at (0, 0) on an n × n board. Print how many squares it can occupy after EXACTLY k moves (it may revisit squares).

Example input	Expected output
8 2	10

Hint 1: Keep a boolean grid of "possible positions" and advance it k times.

Quick check

Answers are in the answer key at the end of this part.

Q1. How many moves can a knight in the middle of a large board make?

- **A.** 4
- **B.** 6
- **C.** 8
- **D.** 12

Q2. Warnsdorff's rule chooses the next square with...

- **A.** the most onward moves
- **B.** the fewest onward moves
- **C.** the smallest row index
- **D.** a random choice

Q3. Is Warnsdorff's rule guaranteed to find a tour?

- **A.** Yes, always
- **B.** No, it is a heuristic
- **C.** Only on 8×8
- **D.** Only from corners

Q4. The heuristic's running time on an N × N board is...

- **A.** O(N)
- **B.** $O(N^2)$
- **C.** O(N!)
- **D.** O(2^N)

Interview / exam questions

- Why is a knight's tour a Hamiltonian path problem?
- What is the "fail-first" principle?
- How would you detect and recover when the heuristic gets stuck?
- Open vs closed tour?

Watch & read

Video: CSES Knight's Tour | Backtracking + Warnsdorff's Rule Explained | Full Intuition + Code in C++ — Yash Poonia · 16:32. *Knight's tour and Warnsdorff's rule (C++, C — the idea transfers to Java)*

Video: Knight's Tour - Numberphile — Numberphile · 4:51. *Knight's tour*

REVISION CARD (30–60 SECONDS)

- Knight: 8 moves (±1,±2),(±2,±1).
- Warnsdorff: go to the unvisited neighbour with the fewest unvisited neighbours.
- Heuristic — can fail; tie-break order matters.
- Backtracking exponential; heuristic O(N²).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• This deck is named `downloadCourseMaterialFacultyPdf.pptx` and has no topic in its file name; its content is Warnsdorff's algorithm for the knight's tour. • The slide's class is not complete Java (e.g. `int y` without a semicolon, and the method `findClosedTour` is declared without a `return` on all paths in the screenshot). The lesson program is a clean, tested rewrite. • The slide uses `ThreadLocalRandom` to pick the first move to examine and loops until a CLOSED tour is found; its sample output is a closed tour starting at square (3, 2).

ASSUMPTIONS MADE

• The deterministic tie-break (first move in the DX/DY list) is an educational simplification of the slide's randomised version.

5.33 Selection Sort

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 35 min · ★ on the minimum-viable path

Repeatedly select the smallest remaining element and swap it into place — simple, always O(n²), at most n−1 swaps.

YOU WILL BE ABLE TO

- Trace selection sort on an array and show the sorted/unsorted split after every pass.
- Write the nested loops with a `minIndex` variable and a single swap per pass.
- State its time (O(n²) always), space (O(1)), stability (not stable) and swap count.
- Explain when its few swaps matter.

Before this lesson: One-Dimensional Arrays · Loops: for, while, do-while, break and continue · Methods: Parameters, Return Values, Scope and Overloading

The idea

Imagine sorting a hand of cards. Look through the whole hand, pick the **smallest** card and put it at the left end. Then look through the remaining cards, pick the smallest and place it second. Repeat. After each pass the left part of the array is sorted **and final**.

```
64 25 12 22 11 pass 1: minimum is 11 → swap with 64
11 25 12 22 64 pass 2: minimum of [25 12 22 64] is 12 → swap with 25
11 12 25 22 64 pass 3: minimum of [25 22 64] is 22 → swap with 25
11 12 22 25 64 pass 4: minimum of [25 64] is 25 → already in place
11 12 22 25 64 sorted
```

WHY THIS WORKS

After pass i the first i elements are the i smallest values in order — they can never need to move again. Each pass shrinks the unsorted part by one, so n−1 passes sort the array. The swap puts the minimum in its final position in a single move, which is why selection sort performs at most **n−1 swaps** — the fewest of the simple sorts.

Syntax

```
for (int i = 0; i < n - 1; i++) { // i = first index of the unsorted part
 int min = i;
 for (int j = i + 1; j < n; j++) // scan the rest
 if (a[j] < a[min]) min = j;
 if (min != i) { int t = a[i]; a[i] = a[min]; a[min] = t; } // at most n - 1 swaps overall
}
```

A tiny example

```
int minIndex = i;
for (int j = i + 1; j < n; j++)
 if (a[j] < a[minIndex]) minIndex = j;
int t = a[minIndex]; a[minIndex] = a[i]; a[i] = t;
```

Guided example

SelectionSort.java — Selection sort (simple solution)

```
1  import java.util.Arrays;
2  import java.util.Scanner;
3
4  public class SelectionSort {
5   // Repeatedly find the smallest value in the unsorted part and swap it to the front of
        that part.
6   static void sort(int[] a) {
7   int n = a.length;
8   for (int i = 0; i < n - 1; i++) {                                                    (1)
9   int minIndex = i; // position of the smallest value seen so far                      (2)
10  for (int j = i + 1; j < n; j++) {
11  if (a[j] < a[minIndex]) {                                                            (3)
12  minIndex = j;
13  }
14  }
15  int temp = a[minIndex]; // one swap per pass                                         (4)
16  a[minIndex] = a[i];
17  a[i] = temp;
18  System.out.println("after pass " + (i + 1) + ": " + Arrays.toString(a));
19  }
20  }
21
22  public static void main(String[] args) {
23  Scanner sc = new Scanner(System.in);
24  int n = sc.nextInt();
25  int[] a = new int[n];
26  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
27  sort(a);
28  System.out.println("sorted: " + Arrays.toString(a));
29  }
30 }
```

In-place, O(n²) comparisons, at most n−1 swaps.

Sample input

```
5
64 25 12 22 11
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	Pass i fixes position i. The last element is automatically in place after n−1 passes.
2	line 9	Assume the first unsorted element is the smallest…
3	line 11	…and look for a smaller one in the rest. Strict `<` keeps the first of equal minima.
4	line 15	Exactly one swap per pass puts the minimum at index i.

> **NOTE**
> On the slide the intermediate arrays are drawn as "[11 | 64 25 12 22]" — i.e. the minimum moved to the front and the other elements shifted right. Real selection sort **swaps**, so the rest of the array is `[25 12 22 64]` (64 goes to where 11 was). Both end sorted, but the program prints what the code really does.

Dry run: watch it execute

The sorted prefix grows by one element per pass.

Algorithm: Selection sort. **Input:** arr = `64,25,12,22,11`.

Pseudocode

```
for i in 0..n-2:
 minIndex = i
 for j in i+1..n-1:
 if a[j] < a[minIndex]: minIndex = j
 swap(a[i], a[minIndex])
```

Step 1 of 9. Unsorted array.

i = —

array	64	25	12	22	11
	0	1	2	3	4

Step 2 of 9. Pass 1: the minimum of a[0..4] is 11 at index 4.

i = 0 · minIndex = 4 · comparisons = 4

array	64	25	12	22	11
	0	1	2	3	4

Step 3 of 9. Swap a[0] and a[4]. The first 1 elements are final.

i = 0 · swaps = 1

array	11	25	12	22	64
	0	1	2	3	4

Step 4 of 9. Pass 2: the minimum of a[1..4] is 12 at index 2.

i = 1 · minIndex = 2 · comparisons = 7

array	11	25	12	22	64
	0	1	2	3	4

Step 5 of 9. Swap a[1] and a[2]. The first 2 elements are final.

i = 1 · swaps = 2

array	11	12	25	22	64
	0	1	2	3	4

Step 6 of 9. Pass 3: the minimum of a[2..4] is 22 at index 3.

i = 2 · minIndex = 3 · comparisons = 9

array	11	12	25	22	64
	0	1	2	3	4

Step 7 of 9. Swap a[2] and a[3]. The first 3 elements are final.

i = 2 · swaps = 3

array	11	12	22	25	64
	0	1	2	3	4

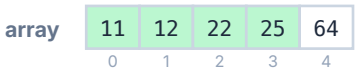
Step 8 of 9. Pass 4: the minimum of a[3..4] is 25 at index 3.

i = 3 · minIndex = 3 · comparisons = 10

array	11	12	22	25	64
	0	1	2	3	4

Step 9 of 9. Already in place. The first 4 elements are final.

i = 3 · swaps = 3



> **RESULT**
> [11, 12, 22, 25, 64] — 10 comparisons, 3 swaps

Complexity

Pass i makes n−1−i comparisons whatever the data looks like → (n−1) + (n−2) + … + 1 = n(n−1)/2 comparisons → **O(n²) in best, average and worst case**. Swaps ≤ n−1. Space O(1) (in place).

n	comparisons	swaps (max)
10	45	9
1 000	499 500	999
100 000	≈ 5 × 10⁹	99 999

Not stable: swapping a far-away minimum over equal elements can reorder them (`[3a, 3b, 1]` → `[1, 3b, 3a]`). Useful when writes are expensive (flash memory) because swaps are minimal.

Common mistakes

Swapping inside the inner loop

Problem

```
if (a[j] < a[i]) swap(a, i, j); // inside the j-loop
```

Why: This "exchange sort" does up to O(n²) swaps and is a different (worse) algorithm, though it also sorts.
Fix: Remember minIndex and swap once after the loop.

Off-by-one in the outer loop

Problem

```
for (int i = 0; i < n; i++) { // inner loop j = i + 1 → fine
```

Why: It works, but the last pass does nothing. Using i ≤ n−1 elsewhere with a[j+1] causes an out-of-bounds read.
Fix: for (int i = 0; i < n - 1; i++)

Using <= to compare

Problem

```
if (a[j] <= a[minIndex]) minIndex = j;
```

Why: Picks the LAST of several equal minima, increasing the number of reorderings of equal keys.
Fix: if (a[j] < a[minIndex])

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Sort and count swaps — *Beginner*

Read n and an array. Sort it with selection sort and print the sorted array on one line, then the number of swaps that actually moved something (skip a swap when minIndex = i).

Example input	Expected output
5 ⏎ 64 25 12 22 11	11 12 22 25 64 ⏎ 3

Hint 1: Count a swap only when minIndex != i.

Exercise 2 · Largest first — *Intermediate*

Sort an array in DESCENDING order using the same idea (select the maximum). Print on one line.

Example input	Expected output
5 ⏎ 3 1 4 1 5	5 4 3 1 1

Hint 1: Only the comparison changes.

Exercise 3 · K smallest, cheaply — *Challenge*

Read n, k and an array. Print the k smallest values in increasing order using only k passes of selection (O(n·k)).

- 1 ≤ k ≤ n ≤ 5000

Example input	Expected output
6 3 ⏎ 7 10 4 3 20 15	3 4 7

Hint 1: You do not have to finish the sort: stop after k passes.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is the array after the 2nd pass?

```
int[] a = {29, 10, 14, 37, 13};
// selection sort
```

Your answer: _____

Q2. How many comparisons does selection sort make on 6 elements, whatever their order?

- **A.** 5
- **B.** 10
- **C.** 15
- **D.** 36

Q3. Which property is TRUE of selection sort?

- **A.** Stable
- **B.** At most n−1 swaps
- **C.** O(n) best case
- **D.** Uses extra O(n) memory

Q4. Best-case time complexity of selection sort?

- **A.** O(n)
- **B.** O(n log n)
- **C.** O(n^2)
- **D.** O(1)

Interview / exam questions

- Why is selection sort not stable? Give an example.
- When would you prefer selection sort?
- How many swaps does it make in the worst case?
- Compare with insertion sort.

Watch & read

Video: 7.5 Selection Sort in Data Structure | Selection Sort Algorithm with C Program — Jenny's Lectures CS IT · 17:49. *How selection sort works*

Video: L-3.8: Selection Sort | Time Complexity(Best, Avg & Worst) Analysis | Stable or Not | Inplace or Not — Gate Smashers · 11:37. *Selection sort in Java*

REVISION CARD (30–60 SECONDS)

- Find minimum of the unsorted part, swap to the front, repeat.
- O(n^2) always; ≤ n−1 swaps; in place; not stable.
- After pass i: first i+1 elements final.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide shows each iteration as "sorted [11], unsorted [64, 25, 12, 22]", which looks like the minimum was inserted at the front and the rest shifted. The slide's code swaps instead, giving [25, 12, 22, 64] after pass 1. The program output matches the code. • The slide describes Quick sort and Selection sort in the same deck; they are split into two lessons here.

5.34 Quick Sort

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 55 min · ★ on the minimum-viable path

Pick a pivot, partition the array around it, and recursively sort the two parts: divide and conquer in place.

YOU WILL BE ABLE TO

- Explain divide and conquer and why one partition places the pivot in its final position.
- Implement Lomuto partitioning and the recursive driver.
- Derive best/average O(n log n) and worst-case O(n²) and say what triggers the worst case.
- Use quickselect to find the k-th smallest element in expected O(n).

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Selection Sort · Algorithm Design Paradigms

The idea

Quick sort is divide and conquer. Choose one element as the **pivot**. Rearrange the array so that everything ≤ pivot is on its left and everything > pivot is on its right — this is called **partitioning**. The pivot is now in its final sorted position! Then sort the left part and the right part the same way.

```
[8 3 1 7 0 10 2] pivot = last element = 2
 partition → [1 0 | 2 | 7 3 10 8]
 ≤2 pivot >2
 sort [1 0] → [0 1] sort [7 3 10 8] → [3 7 8 10]
 result [0 1 2 3 7 8 10]
```

WHY THIS WORKS

Partitioning does O(n) work and fixes one element permanently. If the pivot splits the array roughly in half, the recursion depth is log n and every level does O(n) total work: O(n log n). If the pivot is always the smallest or largest element (e.g. a sorted array with a last-element pivot) one part is empty each time, the depth is n and the total work is O(n²). Randomising the pivot or choosing the median of three makes the bad case very unlikely.

Lomuto partition in words

Keep an index `i` marking the end of the "≤ pivot" region. Scan j from left to right; whenever `a[j] ≤ pivot` grow the region (`i++`) and swap `a[j]` into it. Finally swap the pivot into position `i + 1`.

Syntax

```
static void quickSort(int[] a, int lo, int hi) {
 if (lo >= hi) return; // 0 or 1 element
 int p = partition(a, lo, hi); // pivot lands on its final index p
 quickSort(a, lo, p - 1);
 quickSort(a, p + 1, hi);
}
static int partition(int[] a, int lo, int hi) { // Lomuto: pivot = last element
 int pivot = a[hi], i = lo - 1;
 for (int j = lo; j < hi; j++)
 if (a[j] <= pivot) { i++; int t = a[i]; a[i] = a[j]; a[j] = t; }
 int t = a[i + 1]; a[i + 1] = a[hi]; a[hi] = t;
 return i + 1;
}
// randomised pivot avoids the sorted-input worst case:
int r = java.util.concurrent.ThreadLocalRandom.current().nextInt(lo, hi + 1); // then swap a[r]
    and a[hi]
```

A tiny example

```
int pivot = a[high];
int i = low - 1;
for (int j = low; j < high; j++)
 if (a[j] <= pivot) { i++; swap(a, i, j); }
swap(a, i + 1, high);
```

Guided example

QuickSort.java — Quick sort with Lomuto partition (optimised solution)

```
import java.util.Arrays;
import java.util.Scanner;

public class QuickSort {
 // Lomuto partition: the pivot is the LAST element. Afterwards everything <= pivot is
     on its left,
 // everything > pivot on its right, and the pivot stands at its final sorted position.
 static int partition(int[] a, int low, int high) {
 int pivot = a[high];                                                          (1)
 int i = low - 1; // end of the "<= pivot" region                              (2)
 for (int j = low; j < high; j++) {
 if (a[j] <= pivot) {                                                          (3)
 i++;
 int t = a[i];
 a[i] = a[j];
 a[j] = t;
 }
 }
 int t = a[i + 1]; // drop the pivot between the two regions                   (4)
 a[i + 1] = a[high];
 a[high] = t;
 return i + 1;
 }

 static void quickSort(int[] a, int low, int high) {
 if (low < high) { // 0 or 1 element: already sorted (base case)              (5)
 int p = partition(a, low, high);
 System.out.println("pivot " + a[p] + " placed at index " + p + ": " + Arrays.toString(a)
     );
 quickSort(a, low, p - 1);                                                     (6)
 quickSort(a, p + 1, high);                                                    (7)
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 quickSort(a, 0, n - 1);
 System.out.println("sorted: " + Arrays.toString(a));
 }
}
```

Expected O(n log n), worst O(n²), O(log n) average recursion depth.

Sample input

```
7
8 3 1 7 0 10 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	The last element is the pivot (simple, but bad for already-sorted input).
2	line 9	`i` is the last index of the "≤ pivot" region; it starts empty.
3	line 11	This element belongs in the left region: extend it and swap it in.
4	line 18	Put the pivot between the two regions; it is now final.
5	line 25	Base case: 0 or 1 elements are already sorted.
6	line 28	Sort everything left of the pivot…
7	line 29	…and everything right of it. The pivot itself needs no more work.

SortCheck.java — Randomised verification (demonstration)

```
import java.util.Arrays;
import java.util.Random;

public class SortCheck {
 static void selection(int[] a) {
 for (int i = 0; i < a.length - 1; i++) {
 int m = i;
 for (int j = i + 1; j < a.length; j++) if (a[j] < a[m]) m = j;
 int t = a[m]; a[m] = a[i]; a[i] = t;
 }
 }

 static int partition(int[] a, int lo, int hi) {
 int p = a[hi], i = lo - 1;
 for (int j = lo; j < hi; j++) if (a[j] <= p) { i++; int t = a[i]; a[i] = a[j]; a[j] = t;
     }
 int t = a[i + 1]; a[i + 1] = a[hi]; a[hi] = t;
 return i + 1;
 }

 static void quick(int[] a, int lo, int hi) {
 if (lo < hi) {
 int p = partition(a, lo, hi);
 quick(a, lo, p - 1);
 quick(a, p + 1, hi);
 }
 }

 // 5000 random arrays (with many duplicates) sorted by both algorithms and compared
     with Arrays.sort.
 public static void main(String[] args) {
 Random rnd = new Random(5);
 for (int t = 0; t < 5000; t++) {
 int n = rnd.nextInt(30);
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = rnd.nextInt(10) - 5;
 int[] expected = a.clone();
 Arrays.sort(expected);
 int[] s = a.clone();
 selection(s);
 int[] q = a.clone();
 quick(q, 0, n - 1);
 if (!Arrays.equals(s, expected) || !Arrays.equals(q, expected)) {
 System.out.println("MISMATCH on " + Arrays.toString(a));
 return;
 }
 }
 System.out.println("selection sort and quick sort agree with Arrays.sort on 5000 random
     arrays");
 }
}
```

Sorts 5 000 random arrays with many duplicates and negative numbers and compares with Arrays.sort.

Line-by-line walkthrough

#	Where	What happens
1	line 34	Small value range = many duplicates, which is where partition bugs (`<` vs `<=`) show up.

NOTE

For the slide's array `[8, 3, 1, 7, 0, 10, 2]` the slide says one partition gives `[1, 0, 2, 7, 8, 10, 3]`. The slide's own Lomuto code produces `[1, 0, 2, 7, 3, 10, 8]` — also a valid partition (everything left of 2 is ≤ 2, everything right is > 2) with the pivot at index 2.

Dry run: watch it execute

Pivot placements and the shrinking sub-arrays.

Algorithm: Quick sort (Lomuto). **Input:** arr = `8,3,1,7,0,10,2`.

Pseudocode

```
quickSort(low, high):
 if low < high:
 pivot = a[high]; i = low - 1
 for j in low..high-1:
 if a[j] <= pivot: i++; swap(a[i], a[j])
 swap(a[i+1], a[high]) → p = i+1
 quickSort(low, p-1); quickSort(p+1, high)
```

Step 1 of 5. Unsorted array.

low = 0 · high = 6

	0	1	2	3	4	5	6
array	8	3	1	7	0	10	2

Step 2 of 5. Pivot 2 (last of a[0..6]): 2 element(s) ≤ pivot go left. The pivot lands at index 2 — its final position.

low = 0 · high = 6 · pivot = 2 · p = 2

	0	1	2	3	4	5	6
array	1	0	2	7	3	10	8

Step 3 of 5. Pivot 0 (last of a[0..1]): 0 element(s) ≤ pivot go left. The pivot lands at index 0 — its final position.

low = 0 · high = 1 · pivot = 0 · p = 0

	0	1	2	3	4	5	6
array	0	1	2	7	3	10	8

Step 4 of 5. Pivot 8 (last of a[3..6]): 2 element(s) ≤ pivot go left. The pivot lands at index 5 — its final position.

low = 3 · high = 6 · pivot = 8 · p = 5

	0	1	2	3	4	5	6
array	0	1	2	7	3	8	10

Step 5 of 5. Pivot 3 (last of a[3..4]): 0 element(s) ≤ pivot go left. The pivot lands at index 3 — its final position.

low = 3 · high = 4 · pivot = 3 · p = 3

	0	1	2	3	4	5	6
array	0	1	2	3	7	8	10

RESULT

[0, 1, 2, 3, 7, 8, 10]

Complexity

Let T(n) be the time to sort n elements. A partition costs n. A balanced split gives T(n) = 2T(n/2) + n = **O(n log n)**; an unbalanced split T(n) = T(n−1) + n = **O(n²)**.

case	time	when
best	O(n log n)	pivot near the median every time
average	O(n log n)	random data / random pivot
worst	O(n²)	sorted/reverse-sorted input with first/last pivot, or all equal elements with ≤ partitioning

Space: **O(log n)** average for the recursion stack (O(n) worst case). In place, not stable. Recurse into the smaller part first (and loop on the larger) to guarantee O(log n) stack.

Common mistakes

Quick sort on sorted data with last-element pivot

Problem

```
quickSort(sortedArray) // n = 100 000
```

Why: Every partition peels one element: depth 100 000 → StackOverflowError and O(n²) time.

Fix: Random pivot or median-of-three: swap a random element into a[high] first.

Infinite recursion from the pivot index

Problem

```
quickSort(a, low, p); // should be p - 1
```

Why: The pivot is already in place; including it can leave the sub-problem the same size forever.

Fix: quickSort(a, low, p - 1); quickSort(a, p + 1, high);

Wrong swap temp

Problem

```
a[i] = a[j]; a[j] = a[i];
```

Why: The first assignment destroys the old a[i]; both end up equal.

Fix: int t = a[i]; a[i] = a[j]; a[j] = t;

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Count the partitions — *Beginner*

Quick sort the array with the lesson's Lomuto code. Read n and the array; print the sorted array. (Implement quick sort yourself; the output is simply the sorted array.)

Example input	Expected output
7 ⏎ 8 3 1 7 0 10 2	0 1 2 3 7 8 10

Hint 1: partition returns the pivot's final index. **Hint 2:** Recurse on both sides.

Exercise 2 · Descending quick sort — *Intermediate*

Sort the array in descending order using quick sort.

Example input	Expected output
5 ⏎ 3 1 4 1 5	5 4 3 1 1

Hint 1: Flip the comparison in the partition: move elements ≥ pivot to the left region.

Exercise 3 · k-th smallest with quickselect — *Challenge*

Read n, k and an array. Print the k-th smallest element (1-based) in expected O(n) time without fully sorting.

Example input	Expected output
6 3 ⏎ 7 10 4 3 20 15	7

Hint 1: After a partition the pivot is at index p. If p == k−1 you are done; otherwise only recurse into ONE side.

Quick check

Answers are in the answer key at the end of this part.

Q1. After one partition, the pivot is...

- **A.** at the middle always
- **B.** in its final sorted position
- **C.** at index 0
- **D.** removed

Q2. Worst-case time of quick sort?

- **A.** O(n)
- **B.** O(n log n)
- **C.** O(n^2)
- **D.** O(2^n)

Q3. What does partition return for this call?

```
int[] a = {3, 8, 2, 5, 1, 4, 7, 6};
int p = partition(a, 0, 7); // pivot = a[7] = 6
System.out.println(p);
```

Your answer: _____

Q4. Which strategy is used by quick sort?

- **A.** Greedy
- **B.** Divide and conquer
- **C.** Dynamic programming
- **D.** Brute force

Q5. Is quick sort stable?

- **A.** Yes
- **B.** No

Interview / exam questions

- Why is quick sort usually faster than merge sort in practice?
- How do you avoid the O(n²) worst case?
- What is quickselect and its complexity?
- Hoare vs Lomuto partitioning?

Watch & read

Video: 2.8.1 QuickSort Algorithm — Abdul Bari · 13:43. *Partitioning and recursion*

Video: Quicksort Sort Algorithm in Java - Full Tutorial With Source — Coding with John · 24:58. *Quick sort implementation and pivot choice*

REVISION CARD (30–60 SECONDS)

- Pivot → partition (≤ left, > right) → recurse both sides.
- Lomuto: i = low−1; scan j; if a[j] ≤ pivot {i++; swap}; swap pivot to i+1.
- Average O(n log n), worst O(n²), space O(log n), in place, unstable.
- Quickselect: recurse on one side only.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide's worked example gives the partition result `[1, 0, 2, 7, 8, 10, 3]`; the code produces `[1, 0, 2, 7, 3, 10, 8]`. Then the slide partitions `[7, 8, 10, 3]` with pivot 3 as `[7, 8, 3, 10]`, which is also not what Lomuto does (`[3, 8, 10, 7]`). The final sorted result on the slide is correct.

Part 5 — Solutions to the exercises

Try the exercises before reading these. Each solution is the exact file that was compiled and tested.

5.1 Euclid's Algorithm (GCD)

Exercise 1 · Reduce a fraction

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static int gcd(int a, int b) {
 while (b != 0) {
 int r = a % b;
 a = b;
 b = r;
 }
 return a;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int numerator = sc.nextInt();
 int denominator = sc.nextInt();
 int g = gcd(numerator, denominator);
 System.out.println((numerator / g) + "/" + (denominator / g));
 }
}
```

Why it works: Reduce by the gcd: dividing numerator and denominator by their greatest common divisor leaves no common factor, so the fraction is in lowest terms.

Verified against 3 test cases: `12 18⏎` → `2/3`; `7 5⏎` → `7/5`; `100 25⏎` → `4/1`

Exercise 2 · LCM of a list

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static long gcd(long a, long b) {
 while (b != 0) {
 long r = a % b;
 a = b;
 b = r;
 }
 return a;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long lcm = 1;
 for (int i = 0; i < n; i++) {
 long x = sc.nextLong();
 // divide first, then multiply: keeps the intermediate value small
 lcm = lcm / gcd(lcm, x) * x;
 }
 System.out.println(lcm);
 }
}
```

Why it works: The LCM of a list is built one number at a time with lcm(x, y) = x / gcd(x, y) · y. Dividing first keeps intermediate values as small as possible.

Verified against 3 test cases: `3⏎4 6 8⏎` → `24`; `2⏎7 13⏎` → `91`; `4⏎1 2 3 4⏎` → `12`

Exercise 3 · Modular inverse

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // returns {g, x, y} with a*x + b*y = g = gcd(a, b)
 static long[] extendedGcd(long a, long b) {
 if (b == 0) {
 return new long[] {a, 1, 0};
 }
 long[] next = extendedGcd(b, a % b);
 long g = next[0];
 long x = next[2];
 long y = next[1] - (a / b) * next[2];
 return new long[] {g, x, y};
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long a = sc.nextLong();
 long m = sc.nextLong();
 long[] r = extendedGcd(a, m);
 if (r[0] != 1) {
 System.out.println(-1); // a and m share a factor: no inverse exists
 } else {
 long inverse = ((r[1] % m) + m) % m; // shift a possibly-negative x into [0, m-1]
 System.out.println(inverse);
 }
 }
}
```

Why it works: Extended Euclid carries two extra numbers back up the recursion so that a·x + b·y = gcd(a, b) holds at every level. When the gcd is 1, a·x ≡ 1 (mod m), so x is the inverse.

Verified against 3 test cases: `3 11⏎` → `4`; `10 17⏎` → `12`; `6 9⏎` → `-1`

5.2 Euler's Phi (Totient) Function

Exercise 1 · List the coprime numbers

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static int gcd(int a, int b) {
 while (b != 0) {
 int r = a % b;
 a = b;
 b = r;
 }
 return a;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int k = 1; k <= n; k++) {
 if (gcd(k, n) == 1) {
 sb.append(k).append(' ');
 }
 }
 System.out.println(sb.toString().trim());
 }
}
```

Why it works: For each k test gcd(k, n) == 1 and collect the passes.

Verified against 3 test cases: `24⏎` → `1 5 7 11 13 17 19 23`; `12⏎` → `1 5 7 11`; `7⏎` → `1 2 3 4 5 6`

Exercise 2 · Sum of totients

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] phi = new int[n + 1];
 for (int i = 0; i <= n; i++) {
 phi[i] = i;
 }
 for (int p = 2; p <= n; p++) {
 if (phi[p] == p) { // p is prime
 for (int m = p; m <= n; m += p) {
 phi[m] -= phi[m] / p;
 }
 }
 }
 long sum = 0;
 for (int i = 1; i <= n; i++) {
 sum += phi[i];
 }
 System.out.println(sum);
 }
}
```

Why it works: The sieve-style table computes all φ values in O(n log log n); the sum is a plain loop.

Verified against 3 test cases: `10⏎` → `32`; `1⏎` → `1`; `100⏎` → `3044`

Exercise 3 · Coprime pairs

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Number of ordered pairs (a, b) with 1 <= a, b <= n and gcd(a, b) = 1.
 // For each larger value b = 2..n there are phi(b) smaller partners a; double for order; add (1, 1).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] phi = new int[n + 1];
 for (int i = 0; i <= n; i++) {
 phi[i] = i;
 }
 for (int p = 2; p <= n; p++) {
 if (phi[p] == p) {
 for (int m = p; m <= n; m += p) {
 phi[m] -= phi[m] / p;
 }
 }
 }
 long sum = 0;
 for (int i = 1; i <= n; i++) {
 sum += phi[i];
 }
 System.out.println(2 * sum - 1);
 }
}
```

Why it works: For each b ≥ 2 there are φ(b) values a < b that are coprime to it. Counting both orders gives 2·Σφ(b) for b = 2..n, plus the pair (1,1): 2·(Σ_{i=1..n} φ(i)) − 1.

Verified against 3 test cases: `3⏎` → `7`; `5⏎` → `19`; `1⏎` → `1`

5.3 Chinese Remainder Theorem

Exercise 1 · Two congruences

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int r1 = sc.nextInt();
 int b = sc.nextInt();
 int r2 = sc.nextInt();
 for (int x = 0; x < a * b; x++) {
 if (x % a == r1 && x % b == r2) {
 System.out.println(x);
 return;
 }
 }
 }
}
```

Why it works: CRT guarantees exactly one solution in [0, a·b). A simple loop finds it.

Verified against 3 test cases: `3 2 5 3⏎` → `8`; `4 3 9 5⏎` → `23`; `7 0 11 0⏎` → `0`

Exercise 2 · CRT with the formula

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static long[] extGcd(long a, long b) {
 if (b == 0) {
 return new long[] {a, 1, 0};
 }
 long[] r = extGcd(b, a % b);
 return new long[] {r[0], r[2], r[1] - (a / b) * r[2]};
 }

 static long inverse(long a, long m) {
 long x = extGcd(((a % m) + m) % m, m)[1];
 return ((x % m) + m) % m;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] mod = new long[n];
 long[] rem = new long[n];
 long M = 1;
 for (int i = 0; i < n; i++) {
 mod[i] = sc.nextLong();
 rem[i] = sc.nextLong();
 M *= mod[i];
 }
 long x = 0;
 for (int i = 0; i < n; i++) {
 long Mi = M / mod[i];
 long term = rem[i] % M * Mi % M * inverse(Mi, mod[i]) % M;
 x = (x + term) % M;
 }
 System.out.println(x);
 }
}
```

Why it works: Each term r_i·M_i·inv(M_i) is ≡ r_i for modulus m_i and ≡ 0 for all others. Summing and reducing mod M gives the unique answer.

Verified against 3 test cases: `3⏎3 2⏎5 3⏎7 2⏎` → `23`; `2⏎5 4⏎11 7⏎` → `29`; `1⏎13 6⏎` → `6`

Exercise 3 · Non-coprime moduli

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Moduli need NOT be coprime. Walk through x = r1, r1 + m1, r1 + 2*m1, ... : only these satisfy
 // the first congruence. The remainders mod m2 repeat after m2 steps, so m2 tries are enough.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long r1 = sc.nextLong();
 long m1 = sc.nextLong();
 long r2 = sc.nextLong();
 long m2 = sc.nextLong();
 long x = r1 % m1;
 for (long k = 0; k < m2; k++) {
 if (x % m2 == r2 % m2) {
 System.out.println(x);
 return;
 }
 x += m1;
 }
 System.out.println(-1);
 }
}
```

Why it works: Walk through the candidates r1 + k·m1. Their residues mod m2 form a cycle of length ≤ m2, so after m2 candidates every possibility has been checked.

Verified against 3 test cases: `2 6 4 10⏎` → `14`; `2 6 1 4⏎` → `-1`; `0 5 0 5⏎` → `0`

5.4 Combinations (nCr)

Exercise 1 · nCr as a number

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int r = sc.nextInt();
 long result = 1;
 for (int i = 1; i <= r; i++) {
 result = result * (n - r + i) / i; // always exact: result is C(n-r+i, i)
 }
 System.out.println(result);
 }
}
```

Why it works: The loop keeps the exact value C(n−r+i, i) after step i, so each division is exact.

Verified against 3 test cases: `5 2⏎` → `10`; `30 15⏎` → `155117520`; `7 0⏎` → `1`

Exercise 2 · nCr modulo 1 000 000 007

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 final long MOD = 1_000_000_007L;
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int r = sc.nextInt();
 long[][] c = new long[n + 1][r + 1];
 for (int i = 0; i <= n; i++) {
 c[i][0] = 1;
 for (int j = 1; j <= Math.min(i, r); j++) {
 c[i][j] = (c[i - 1][j - 1] + (j <= i - 1 ? c[i - 1][j] : 0)) % MOD;
 }
 }
 System.out.println(c[n][r]);
 }
}
```

Why it works: Pascal's recurrence uses only additions, so applying `% MOD` at each step keeps every value small.

Verified against 3 test cases: `10 3⏎` → `120`; `1000 500⏎` → `159835829`; `5 5⏎` → `1`

Exercise 3 · List all combinations

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static StringBuilder out = new StringBuilder();

 // choose(start, chosen): extend the current selection with numbers >= start
 static void choose(int n, int r, int start, int[] picked, int count) {
 if (count == r) {
 for (int i = 0; i < r; i++) {
 out.append(i == 0 ? "" : " ").append(picked[i]);
 }
 out.append('\n');
 return;
 }
 for (int v = start; v <= n; v++) {
 picked[count] = v;
 choose(n, r, v + 1, picked, count + 1);
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int r = sc.nextInt();
 choose(n, r, 1, new int[r], 0);
 System.out.print(out);
 }
}
```

Why it works: Recursive loop from `start` upwards: choose a value, recurse with start = value + 1. Because values only increase, each subset appears once and in lexicographic order.

Verified against 3 test cases: `4 2⏎` → `1 2⏎1 3⏎1 4⏎2 3⏎2 4⏎3 4`; `3 3⏎` → `1 2 3`; `3 1⏎` → `1⏎2⏎3`

5.5 Karatsuba's Multiplication

Exercise 1 · School-book multiplication

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // School-book multiplication of two digit strings: O(n * m) digit multiplications.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String x = sc.next();
 String y = sc.next();
 int[] result = new int[x.length() + y.length()];
 for (int i = x.length() - 1; i >= 0; i--) {
 for (int j = y.length() - 1; j >= 0; j--) {
 int product = (x.charAt(i) - '0') * (y.charAt(j) - '0');
 int pos = i + j + 1; // units position of this partial product
 int sum = product + result[pos];
 result[pos] = sum % 10;
 result[pos - 1] += sum / 10; // carry into the next position to the left
 }
 }
 StringBuilder sb = new StringBuilder();
 for (int digit : result) {
 if (!(sb.length() == 0 && digit == 0)) {
 sb.append(digit);
 }
 }
 System.out.println(sb.length() == 0 ? "0" : sb);
 }
}
```

Why it works: The classic method: n·m single-digit products, each added into the right position with carry.

Verified against 3 test cases: `12345 6789⏎` → `83810205`; `0 98765⏎` → `0`; `99999999999 99999999999⏎` → `999999999980000000001`

Exercise 2 · Count the base multiplications

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static int baseCases = 0;

 static long pow10(int k) {
 long p = 1;
 for (int i = 0; i < k; i++) p *= 10;
 return p;
 }

 static long multiply(long x, long y) {
 if (x < 10 || y < 10) {
 baseCases++;
 return x * y;
 }
 int n = Math.max(Long.toString(x).length(), Long.toString(y).length());
 int half = n / 2;
 long p = pow10(half);
 long a = x / p, b = x % p, c = y / p, d = y % p;
 long ac = multiply(a, c);
 long bd = multiply(b, d);
 long abcd = multiply(a + b, c + d);
 return ac * p * p + (abcd - ac - bd) * p + bd;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 multiply(sc.nextLong(), sc.nextLong());
 System.out.println(baseCases);
 }
}
```

Why it works: A static counter in the base case counts leaf calls of the recursion tree.

Verified against 3 test cases: `1234 5678⏎` → `11`; `5 7⏎` → `1`; `12 34⏎` → `3`

Exercise 3 · Big Karatsuba

Ex3.java

```java
import java.math.BigInteger;
import java.util.Scanner;

public class Ex3 {
 static BigInteger multiply(BigInteger x, BigInteger y) {
 int n = Math.max(x.bitLength(), y.bitLength());
 if (n <= 30) { // both factors < 2^30, so the product < 2^60 fits in a long
 return BigInteger.valueOf(x.longValue() * y.longValue());
 }
 int half = n / 2;
 BigInteger a = x.shiftRight(half);
 BigInteger b = x.subtract(a.shiftLeft(half));
 BigInteger c = y.shiftRight(half);
 BigInteger d = y.subtract(c.shiftLeft(half));
 BigInteger ac = multiply(a, c);
 BigInteger bd = multiply(b, d);
 BigInteger mid = multiply(a.add(b), c.add(d)).subtract(ac).subtract(bd);
 return ac.shiftLeft(2 * half).add(mid.shiftLeft(half)).add(bd);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 BigInteger x = new BigInteger(sc.next());
 BigInteger y = new BigInteger(sc.next());
 System.out.println(multiply(x, y));
 }
}
```

Why it works: Same three-product identity as the long version, but with BigInteger and bit shifts; the base case is a plain 30-bit multiplication.

Verified against 3 test cases: `123456789 987654321⏎` → `121932631112635269`; `10000000000000000000 10000000000000000000⏎` → `100000000000000000000000000000000000000000`; `0 5⏎` → `0`

5.6 Booth's Algorithm (signed binary multiplication)

Exercise 1 · n-bit two's complement

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Print x in n-bit two's complement (works for negative x too).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int n = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int bit = n - 1; bit >= 0; bit--) {
 sb.append((x >> bit) & 1); // >> on a negative int keeps the sign, so high bits come out as 1s
 }
 System.out.println(sb);
 }
}
```

Why it works: Java stores int in two's complement, so shifting and masking one bit at a time reads off the pattern directly.

Verified against 3 test cases: `-5 8⏎` → `11111011`; `7 4⏎` → `0111`; `-1 4⏎` → `1111`

Exercise 2 · Booth's operations

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // For an n-bit multiplier given as a bit string, list Booth's operation for each cycle
     (rightmost bit first).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String q = sc.next();
 int previous = 0; // Q-1 starts as 0
 StringBuilder sb = new StringBuilder();
 for (int i = q.length() - 1; i >= 0; i--) {
 int current = q.charAt(i) - '0';
 String op;
 if (current == 1 && previous == 0) {
 op = "-M";
 } else if (current == 0 && previous == 1) {
 op = "+M";
 } else {
 op = "0";
 }
 sb.append(sb.length() == 0 ? "" : " ").append(op);
 previous = current;
 }
 System.out.println(sb);
 }
}
```

Why it works: The current bit with the previously processed (lower) bit forms the pair Q_0Q_{-1}; the table of conditions gives the action.

Verified against 3 test cases: `0110⏎` → `0 -M 0 +M`; `0011⏎` → `-M 0 +M 0`; `1111⏎` → `-M 0 0 0`

Exercise 3 · Booth multiplication

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int m = sc.nextInt();
 int q = sc.nextInt();
 int width = 16; // inputs fit in 16-bit signed numbers
 if (m == -(1 << (width - 1))) {
 width++;
 }
 long mask = (1L << width) - 1;
 long M = m & mask;
 long negM = (-m) & mask;
 long ac = 0;
 long qr = q & mask;
 int qm1 = 0;
 for (int i = 0; i < width; i++) {
 int q0 = (int) (qr & 1);
 if (q0 == 1 && qm1 == 0) {
 ac = (ac + negM) & mask;
 } else if (q0 == 0 && qm1 == 1) {
 ac = (ac + M) & mask;
 }
 qm1 = (int) (qr & 1);
 qr = ((qr >> 1) | ((ac & 1) << (width - 1))) & mask;
 ac = (ac >> 1) | (ac & (1L << (width - 1)));
 }
 long raw = (ac << width) | qr;
 long sign = 1L << (2 * width - 1);
 long product = (raw & sign) != 0 ? raw - (1L << (2 * width)) : raw; // read the 2n-bit pattern
     as signed
 System.out.println(product);
 }
}
```

Why it works: Direct simulation of the hardware loop; the final 2n-bit pattern is converted to a signed number.

Verified against 3 test cases: `-3 4⏎` → `-12`; `7 3⏎` → `21`; `-8 -8⏎` → `64`

5.7 Simple Sieve of Eratosthenes

Exercise 1 · Count primes

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 boolean[] composite = new boolean[n + 1];
 int count = 0;
 for (int i = 2; i <= n; i++) {
 if (!composite[i]) {
 count++;
 for (long m = (long) i * i; m <= n; m += i) {
 composite[(int) m] = true;
 }
 }
 }
 System.out.println(count);
 }
}
```

Why it works: Standard sieve; every i not crossed out when reached is prime and is counted, then its multiples are crossed out.

Verified against 3 test cases: `100⏎` → `25`; `1⏎` → `0`; `1000000⏎` → `78498`

Exercise 2 · The k-th prime

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int k = sc.nextInt();
 int limit = 2_000_000; // the 100 000th prime is 1 299 709, so this is enough
 boolean[] composite = new boolean[limit + 1];
 int seen = 0;
 for (int i = 2; i <= limit; i++) {
 if (!composite[i]) {
 seen++;
 if (seen == k) {
 System.out.println(i);
 return;
 }
 for (long m = (long) i * i; m <= limit; m += i) {
 composite[(int) m] = true;
 }
 }
 }
}
```

Why it works: A single sieve to 2 000 000 contains the 100 000th prime (1 299 709); count survivors until the k-th.

Verified against 3 test cases: `1000⏎` → `7919`; `1⏎` → `2`; `100000⏎` → `1299709`

Exercise 3 · Prime factorisation queries

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 int limit = 1_000_000;
 int[] spf = new int[limit + 1]; // spf[x] = smallest prime factor of x
 for (int i = 2; i <= limit; i++) {
 if (spf[i] == 0) {
 for (int m = i; m <= limit; m += i) {
 if (spf[m] == 0) {
 spf[m] = i;
 }
 }
 }
 }
 Scanner sc = new Scanner(System.in);
 int q = sc.nextInt();
 StringBuilder out = new StringBuilder();
 for (int t = 0; t < q; t++) {
 int x = sc.nextInt();
 int original = x;
 StringBuilder line = new StringBuilder();
 while (x > 1) {
 int p = spf[x];
 int e = 0;
 while (x % p == 0) {
 x /= p;
 e++;
 }
 if (line.length() > 0) {
 line.append(" * ");
 }
 line.append(p);
 if (e > 1) {
 line.append('^').append(e);
 }
 }
 out.append(original).append(" = ").append(original == 1 ? "1" : line).append('\n');
 }
 System.out.print(out);
 }
}
```

Why it works: A "smallest prime factor" sieve stores for each number its smallest prime divisor, so every factorisation takes O(log x) steps.

Verified against 2 test cases: `3⏎12⏎97⏎360⏎` → `12 = 2^2 * 3⏎97 = 97⏎360 = 2^3 * 3^2 * 5`; `2⏎2⏎1000000⏎` → `2 = 2⏎1000000 = 2^6 * 5^6`

5.8 Segmented & Incremental Sieve

Exercise 1 · Primes in a small range

Ex4.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex4 {
 // Count primes in [L, R] where R can be as large as 10^12 but R - L <= 10^6.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long low = sc.nextLong();
 long high = sc.nextLong();
 int root = (int) Math.sqrt((double) high);
 while ((long) root * root > high) root--;
 while ((long) (root + 1) * (root + 1) <= high) root++;

 boolean[] small = new boolean[root + 1];
 List<Integer> base = new ArrayList<>();
 for (int i = 2; i <= root; i++) {
 if (!small[i]) {
 base.add(i);
 for (long m = (long) i * i; m <= root; m += i) {
 small[(int) m] = true;
 }
 }
 }
 boolean[] composite = new boolean[(int) (high - low + 1)];
 for (int p : base) {
 long start = Math.max((long) p * p, (low + p - 1) / p * p);
 for (long m = start; m <= high; m += p) {
 composite[(int) (m - low)] = true;
 }
 }
 int count = 0;
 for (long n = Math.max(low, 2); n <= high; n++) {
 if (!composite[(int) (n - low)]) {
 count++;
 }
 }
 System.out.println(count);
 }
}
```

Why it works: This reference solution already uses the segmented method, which also covers small ranges.

Verified against 3 test cases: `10 30⏎` → `6`; `1 1⏎` → `0`; `100 130⏎` → `6`

Exercise 2 · Primes in a huge range

Ex4.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex4 {
 // Count primes in [L, R] where R can be as large as 10^12 but R - L <= 10^6.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long low = sc.nextLong();
 long high = sc.nextLong();
 int root = (int) Math.sqrt((double) high);
 while ((long) root * root > high) root--;
 while ((long) (root + 1) * (root + 1) <= high) root++;

 boolean[] small = new boolean[root + 1];
 List<Integer> base = new ArrayList<>();
 for (int i = 2; i <= root; i++) {
 if (!small[i]) {
 base.add(i);
 for (long m = (long) i * i; m <= root; m += i) {
 small[(int) m] = true;
 }
 }
 }
 boolean[] composite = new boolean[(int) (high - low + 1)];
 for (int p : base) {
 long start = Math.max((long) p * p, (low + p - 1) / p * p);
 for (long m = start; m <= high; m += p) {
 composite[(int) (m - low)] = true;
 }
 }
 int count = 0;
 for (long n = Math.max(low, 2); n <= high; n++) {
 if (!composite[(int) (n - low)]) {
 count++;
 }
 }
 System.out.println(count);
 }
}
```

Why it works: Segmented sieve: base primes up to √R (≤ 10⁶) and a window array of at most 10⁶ + 1 cells.

Verified against 3 test cases: `1000000000000 1000000000100⏎` → `4`; `2 2⏎` → `1`; `999999999900 1000000000000⏎` → `4`

Exercise 3 · Largest prime gap in a window

Ex5.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex5 {
 // Largest gap between two consecutive primes inside [L, R]; print -1 if fewer than two primes.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long low = sc.nextLong();
 long high = sc.nextLong();
 int root = (int) Math.sqrt((double) high);
 while ((long) root * root > high) root--;
 while ((long) (root + 1) * (root + 1) <= high) root++;
 boolean[] small = new boolean[root + 1];
 List<Integer> base = new ArrayList<>();
 for (int i = 2; i <= root; i++) {
 if (!small[i]) {
 base.add(i);
 for (long m = (long) i * i; m <= root; m += i) {
 small[(int) m] = true;
 }
 }
 }
 boolean[] composite = new boolean[(int) (high - low + 1)];
 for (int p : base) {
 long start = Math.max((long) p * p, (low + p - 1) / p * p);
 for (long m = start; m <= high; m += p) {
 composite[(int) (m - low)] = true;
 }
 }
 long prev = -1;
 long best = -1;
 for (long n = Math.max(low, 2); n <= high; n++) {
 if (!composite[(int) (n - low)]) {
 if (prev != -1) {
 best = Math.max(best, n - prev);
 }
 prev = n;
 }
 }
 System.out.println(best);
 }
}
```

Why it works: After the segmented sieve, one pass over the window tracks the last prime seen and the maximum gap.

Verified against 3 test cases: `1 100⏎` → `8`; `90 96⏎` → `-1`; `2 3⏎` → `1`

5.9 Binary Palindrome

Exercise 1 · Is it a binary palindrome?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int rev = 0, rest = x;
 while (rest > 0) {
 rev = (rev << 1) | (rest & 1);
 rest >>= 1;
 }
 System.out.println(rev == x ? "YES" : "NO");
 }
}
```

Why it works: Reverse the bits into a second variable with shifts and compare. 0 never enters the loop, reversed stays 0 and equals n.

Verified against 4 test cases: `5⏎` → `YES`; `10⏎` → `NO`; `0⏎` → `YES`

Exercise 2 · Count binary palindromes up to n

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static boolean pal(int x) {
 int rev = 0, rest = x;
 while (rest > 0) {
 rev = (rev << 1) | (rest & 1);
 rest >>= 1;
 }
 return rev == x;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int count = 0;
 for (int i = 1; i <= n; i++) if (pal(i)) count++;
 System.out.println(count);
 }
}
```

Why it works: Test every i in 1..n with the O(log i) check. Total O(n log n).

Verified against 3 test cases: `10⏎` → `5`; `1⏎` → `1`; `100⏎` → `19`

Exercise 3 · Next binary palindrome

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static boolean pal(long x) {
 long rev = 0, rest = x;
 while (rest > 0) {
 rev = (rev << 1) | (rest & 1);
 rest >>= 1;
 }
 return rev == x;
 }

 // Smallest binary palindrome strictly greater than n.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 long x = n + 1;
 while (!pal(x)) x++;
 System.out.println(x);
 }
}
```

Why it works: Simple forward search is fast enough because the next palindrome is always within about 2^(len/2) steps (for a 30-bit number, at most ~33 000 steps).

Verified against 3 test cases: `7⏎` → `9`; `0⏎` → `1`; `51⏎` → `63`

5.10 Swap Two Nibbles in a Byte

Exercise 1 · Swap the nibbles

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt() & 0xFF;
 System.out.println(((x & 0xF0) >>> 4) | ((x & 0x0F) << 4));
 }
}
```

Why it works: Mask each nibble, shift into the other's place, OR together.

Verified against 4 test cases: `171⏎` → `186`; `0⏎` → `0`; `255⏎` → `255`

Exercise 2 · Swap adjacent bits

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Swap every pair of adjacent bits (bit0<->bit1, bit2<->bit3, ...) of a 32-bit value; print as unsigned.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long input = sc.nextLong();
 int x = (int) input;
 int even = x & 0x55555555; // bits 0, 2, 4, ...
 int odd = x & 0xAAAAAAAA; // bits 1, 3, 5, ...
 int swapped = (even << 1) | (odd >>> 1);
 System.out.println(swapped & 0xFFFFFFFFL);
 }
}
```

Why it works: The two masks split the word into non-overlapping bit sets; each set moves by one position into the other set's slots.

Verified against 3 test cases: `23⏎` → `43`; `0⏎` → `0`; `4294967295⏎` → `4294967295`

Exercise 3 · Reverse 32 bits

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Reverse the 32 bits of an unsigned 32-bit integer.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long input = sc.nextLong();
 int x = (int) input;
 int result = 0;
 for (int i = 0; i < 32; i++) {
 result = (result << 1) | (x & 1);
 x >>>= 1;
 }
 System.out.println(result & 0xFFFFFFFFL);
 }
}
```

Why it works: Same peel-and-push technique as the binary-palindrome lesson, but for exactly 32 iterations so leading zeros are included.

Verified against 3 test cases: `43261596⏎` → `964176192`; `1⏎` → `2147483648`; `0⏎` → `0`

5.11 Longest Sequence of 1s (and after flipping a bit)

Exercise 1 · Longest run of 1s

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int cur = 0, best = 0;
 for (int i = 0; i < 32; i++) {
 if ((x & 1) == 1) best = Math.max(best, ++cur);
 else cur = 0;
 x >>>= 1;
 }
 System.out.println(best);
 }
}
```

Why it works: Count while bits are 1, reset on 0, remember the maximum.

Verified against 4 test cases: `56825⏎` → `6`; `0⏎` → `0`; `-1⏎` → `32`

Exercise 2 · Flip one bit

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int best = 0;
 for (int bit = 0; bit < 32; bit++) {
 int y = x | (1 << bit);
 int run = 0;
 for (int i = 0; i < 32; i++) {
 if (((y >>> i) & 1) == 1) best = Math.max(best, ++run);
 else run = 0;
 }
 }
 System.out.println(best);
 }
}
```

Why it works: Set each bit in turn (OR with 1 << bit) and measure the longest run; the maximum over 32 trials is the answer. The one-pass method from the lesson gives the same result.

Verified against 4 test cases: `478⏎` → `8`; `0⏎` → `1`; `-1⏎` → `32`

Exercise 3 · Flip up to k zeros

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Longest run of 1s after flipping at most k zeros: sliding window over the 32 bits containing
     <= k zeros.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int k = sc.nextInt();
 int left = 0, zeros = 0, best = 0;
 for (int right = 0; right < 32; right++) {
 if (((x >>> right) & 1) == 0) zeros++;
 while (zeros > k) {
 if (((x >>> left) & 1) == 0) zeros--;
 left++;
 }
 best = Math.max(best, right - left + 1);
 }
 System.out.println(best);
 }
}
```

Why it works: Classic two-pointer window: the right end moves one step per bit; the left end only moves forward, so total work is O(32).

Verified against 3 test cases: `478 2⏎` → `9`; `0 5⏎` → `5`; `-1 0⏎` → `32`

5.12 Josephus Problem

Exercise 1 · Elimination order

Ex1.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 List<Integer> circle = new ArrayList<>();
 for (int i = 1; i <= n; i++) circle.add(i);
 StringBuilder sb = new StringBuilder();
 int idx = 0;
 while (!circle.isEmpty()) {
 idx = (idx + k - 1) % circle.size();
 sb.append(sb.length() == 0 ? "" : " ").append(circle.remove(idx));
 }
 System.out.println(sb);
 }
}
```

Why it works: Direct simulation; the last remaining person is removed at the end of the loop so the output ends with the survivor.

Verified against 3 test cases: `5 2⏎` → `2 4 1 5 3`; `7 3⏎` → `3 6 2 7 5 1 4`; `1 5⏎` → `1`

Exercise 2 · Survivor for large N

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 long k = sc.nextLong();
 long ans = 0;
 for (long people = 2; people <= n; people++) {
 ans = (ans + k) % people;
 }
 System.out.println(ans + 1);
 }
}
```

Why it works: The loop ans = (ans + k) % people for people = 2..N takes O(N) time and O(1) space.

Verified against 3 test cases: `10 3⏎` → `4`; `14 2⏎` → `13`; `1 7⏎` → `1`

Exercise 3 · Josephus with k = 2 and N up to 10^{18}

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 System.out.println(2 * (n - Long.highestOneBit(n)) + 1);
 }
}
```

Why it works: After r eliminations 2^m people remain and the counting position is person 2r+1; with k = 2 and a power-of-two circle that person survives.

Verified against 4 test cases: `100⏎` → `73`; `1⏎` → `1`; `64⏎` → `1`

5.13 Block Swap Algorithm (Array Rotation)

Exercise 1 · Left rotation

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int d = sc.nextInt() % n;
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) {
 sb.append(i == 0 ? "" : " ").append(a[(i + d) % n]);
 }
 System.out.println(sb);
 }
}
```

Why it works: The index formula (i + d) % n gives the result directly, without moving anything in place.

Verified against 3 test cases: `5 2⏎1 2 3 4 5⏎` → `3 4 5 1 2`; `4 0⏎9 8 7 6⏎` → `9 8 7 6`; `3 7⏎1 2 3⏎` → `2 3 1`

Exercise 2 · Right rotation in place

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static void reverse(int[] a, int lo, int hi) {
 while (lo < hi) {
 int t = a[lo];
 a[lo++] = a[hi];
 a[hi--] = t;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int d = sc.nextInt() % n;
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 // right rotation by d == left rotation by n - d
 int left = (n - d) % n;
 reverse(a, 0, left - 1);
 reverse(a, left, n - 1);
 reverse(a, 0, n - 1);
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

Why it works: Convert to a left rotation by (n − d) mod n and apply the three-reversal trick.

Verified against 3 test cases: `5 2⏎1 2 3 4 5⏎` → `4 5 1 2 3`; `6 6⏎1 2 3 4 5 6⏎` → `1 2 3 4 5 6`; `2 1⏎1 2⏎` → `2 1`

Exercise 3 · How many rotations?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // The smallest element sits right after the "break" where a[i] > a[i+1].
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int lo = 0, hi = n - 1;
 while (lo < hi) {
 int mid = (lo + hi) / 2;
 if (a[mid] > a[hi]) {
 lo = mid + 1; // the minimum is to the right of mid
 } else {
 hi = mid; // the minimum is at mid or to its left
 }
 }
 System.out.println(lo); // index of the minimum == number of left rotations
 }
}
```

Why it works: If a[mid] > a[hi] the minimum is right of mid; otherwise it is at mid or left of it. The loop narrows to the minimum's index in O(log n).

Verified against 3 test cases: `6⏎15 18 2 3 6 12⏎` → `2`; `5⏎1 2 3 4 5⏎` → `0`; `4⏎2 3 4 1⏎` → `3`

5.14 Leaders in an Array

Exercise 1 · Print the leaders

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int[] out = new int[n];
 int c = 0;
 int max = Integer.MIN_VALUE;
 for (int i = n - 1; i >= 0; i--) {
 if (a[i] > max) {
 out[c++] = a[i];
 max = a[i];
 }
 }
 StringBuilder sb = new StringBuilder();
 for (int i = c - 1; i >= 0; i--) sb.append(sb.length() == 0 ? "" : " ").append(out[i]);
 System.out.println(sb);
 }
}
```

Why it works: The running maximum of the suffix decides each element in O(1).

Verified against 4 test cases: `6⏎16 17 4 3 5 2⏎` → `17 5 2`; `5⏎14 20 3 6 5⏎` → `20 6 5`; `3⏎5 4 3⏎` → `5 4 3`

Exercise 2 · Non-strict leaders

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Non-strict leaders: an element counts if it is >= every element to its right.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int[] out = new int[n];
 int c = 0;
 int max = Integer.MIN_VALUE;
 for (int i = n - 1; i >= 0; i--) {
 if (a[i] >= max) {
 out[c++] = a[i];
 max = a[i];
 }
 }
 StringBuilder sb = new StringBuilder();
 for (int i = c - 1; i >= 0; i--) sb.append(sb.length() == 0 ? "" : " ").append(out[i]);
 System.out.println(sb);
 }
}
```

Why it works: Replace `>` by `>=` in the running-maximum test.

Verified against 3 test cases: `5⏎7 10 4 10 6⏎` → `10 10 6`; `3⏎2 2 2⏎` → `2 2 2`; `1⏎9⏎` → `9`

Exercise 3 · Next greater element

Ex3.java

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Scanner;

public class Ex3 {
 // Next greater element to the right (or -1), in O(n) with a stack of candidates scanned from the right.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int[] ans = new int[n];
 Deque<Integer> stack = new ArrayDeque<>(); // values, decreasing from top... top is nearest candidate
 for (int i = n - 1; i >= 0; i--) {
 while (!stack.isEmpty() && stack.peek() <= a[i]) {
 stack.pop(); // can never be the next-greater for anything to the left
 }
 ans[i] = stack.isEmpty() ? -1 : stack.peek();
 stack.push(a[i]);
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(ans[i]);
 System.out.println(sb);
 }
}
```

Why it works: The stack holds the elements that could still be "next greater" for something on the left, in decreasing order. Each element is pushed and popped at most once: O(n).

Verified against 3 test cases: `4⏎4 5 2 25⏎` → `5 25 25 -1`; `4⏎13 7 6 12⏎` → `-1 12 12 -1`; `3⏎3 2 1⏎` → `-1 -1 -1`

5.15 Majority Element (Boyer–Moore Voting)

Exercise 1 · Majority with a HashMap

Ex1.java

```java
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 Map<Integer, Integer> freq = new HashMap<>();
 Integer winner = null;
 for (int i = 0; i < n; i++) {
 int x = sc.nextInt();
 int c = freq.merge(x, 1, Integer::sum);
 if (c > n / 2) winner = x;
 }
 System.out.println(winner == null ? "No majority element" : winner);
 }
}
```

Why it works: merge(x, 1, Integer::sum) increments the counter; any value whose count passes n/2 is the answer.

Verified against 3 test cases: `5⏎2 2 1 1 1⏎` → `1`; `4⏎1 2 1 2⏎` → `No majority element`; `1⏎7⏎` → `7`

Exercise 2 · Boyer–Moore

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int candidate = 0, count = 0;
 for (int x : a) {
 if (count == 0) {
 candidate = x;
 count = 1;
 } else if (x == candidate) count++;
 else count--;
 }
 int occ = 0;
 for (int x : a) if (x == candidate) occ++;
 System.out.println(occ > n / 2 ? String.valueOf(candidate) : "No majority element");
 }
}
```

Why it works: Phase 1 keeps a candidate and a vote counter; phase 2 counts the real occurrences.

Verified against 3 test cases: `5⏎2 2 1 1 1⏎` → `1`; `7⏎3 3 4 2 4 4 2⏎` → `No majority element`; `9⏎2 2 2 6 2 2 8 2 1⏎` → `2`

Exercise 3 · Elements appearing more than n/3 times

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Elements appearing more than n/3 times: at most TWO of them exist, so keep two candidates.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int c1 = 0, c2 = 1, n1 = 0, n2 = 0;
 for (int x : a) {
 if (x == c1) n1++;
 else if (x == c2) n2++;
 else if (n1 == 0) { c1 = x; n1 = 1; }
 else if (n2 == 0) { c2 = x; n2 = 1; }
 else { n1--; n2--; }
 }
 int o1 = 0, o2 = 0;
 for (int x : a) {
 if (x == c1) o1++;
 else if (x == c2) o2++;
 }
 StringBuilder sb = new StringBuilder();
 if (o1 > n / 3) sb.append(c1);
 if (o2 > n / 3) sb.append(sb.length() == 0 ? "" : " ").append(c2);
 if (sb.length() > 0 && o1 > n / 3 && o2 > n / 3 && c1 > c2) {
 sb = new StringBuilder().append(c2).append(' ').append(c1); // print in increasing order
 }
 System.out.println(sb.length() == 0 ? "None" : sb);
 }
}
```

Why it works: Generalised Boyer–Moore: removing three different values at a time cannot eliminate a value that has more than n/3 occurrences.

Verified against 3 test cases: `8⏎1 1 1 3 3 2 2 2⏎` → `1 2`; `3⏎3 2 3⏎` → `3`; `4⏎1 2 3 4⏎` → `None`

5.16 Maximum Product Subarray

Exercise 1 · Count subarrays with positive product

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Count subarrays whose product is strictly positive.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int count = 0;
 for (int i = 0; i < n; i++) {
 int negatives = 0;
 for (int j = i; j < n; j++) {
 if (a[j] == 0) break; // a zero makes every longer subarray's product 0
 if (a[j] < 0) negatives++;
 if (negatives % 2 == 0) count++;
 }
 }
 System.out.println(count);
 }
}
```

Why it works: Track the parity of negatives while extending; stop at a zero.

Verified against 3 test cases: `5⏎2 3 -2 4 0⏎` → `4`; `3⏎-1 -2 -3⏎` → `2`; `3⏎0 0 0⏎` → `0`

Exercise 2 · Maximum product subarray

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long mx = 0, mn = 0, best = 0;
 for (int i = 0; i < n; i++) {
 long x = sc.nextInt();
 if (i == 0) {
 mx = mn = best = x;
 continue;
 }
 if (x < 0) {
 long t = mx;
 mx = mn;
 mn = t;
 }
 mx = Math.max(x, mx * x);
 mn = Math.min(x, mn * x);
 best = Math.max(best, mx);
 }
 System.out.println(best);
 }
}
```

Why it works: The O(n) max/min recurrence from the lesson.

Verified against 4 test cases: `4⏎2 3 -2 4⏎` → `6`; `3⏎-2 0 -1⏎` → `0`; `4⏎-5 -4 -1 -2⏎` → `40`

Exercise 3 · Which subarray?

Ex3.java

```
import java.util.Scanner;

public class Ex3 {
 // Print "product start end" for the maximum-product subarray (0-based, earliest start, then
     earliest end).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] a = new long[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextLong();
 long best = Long.MIN_VALUE;
 int bs = 0, be = 0;
 for (int i = 0; i < n; i++) {
 long p = 1;
 for (int j = i; j < n; j++) {
 p *= a[j];
 if (p > best) {
 best = p;
 bs = i;
 be = j;
 }
 }
 }
 System.out.println(best + " " + bs + " " + be);
 }
}
```

Why it works: Enumerate (i, j) in order and update the record only when the product is strictly greater.

Verified against 3 test cases: `4⏎2 3 -2 4⏎` → `6 0 1`; `1⏎5⏎` → `5 0 0`; `3⏎-1 -1 -1⏎` → `1 0 1`

5.17 Maximum Equilibrium Sum

Exercise 1 · First equilibrium index

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] a = new long[n];
 long total = 0;
 for (int i = 0; i < n; i++) {
 a[i] = sc.nextLong();
 total += a[i];
 }
 long left = 0;
 for (int i = 0; i < n; i++) {
 if (left == total - left - a[i]) {
 System.out.println(i);
 return;
 }
 left += a[i];
 }
 System.out.println(-1);
 }
}
```

Why it works: Running left sum and a precomputed total give the right sum in O(1).

Verified against 3 test cases: `7⏎-7 1 5 2 -4 3 0⏎` → `3`; `3⏎1 2 3⏎` → `-1`; `1⏎5⏎` → `0`

Exercise 2 · Maximum equilibrium sum

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Print the maximum equilibrium sum (one side's sum), or "none".
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] a = new long[n];
 long total = 0;
 for (int i = 0; i < n; i++) {
 a[i] = sc.nextLong();
 total += a[i];
 }
 long left = 0;
 boolean found = false;
 long best = Long.MIN_VALUE;
 for (int i = 0; i < n; i++) {
 if (left == total - left - a[i]) {
 found = true;
 best = Math.max(best, left);
 }
 left += a[i];
 }
 System.out.println(found ? String.valueOf(best) : "none");
 }
}
```

Why it works: Same scan; update the maximum whenever left == right.

Verified against 3 test cases: `9⏎1 2 3 4 5 4 3 2 1⏎` → `10`; `7⏎-7 1 5 2 -4 3 0⏎` → `0`; `2⏎1 2⏎` → `none`

Exercise 3 · Balanced split points

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Count split points k (1 <= k <= n-1) where sum(a[0..k-1]) == sum(a[k..n-1]).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] a = new long[n];
 long total = 0;
 for (int i = 0; i < n; i++) {
 a[i] = sc.nextLong();
 total += a[i];
 }
 long prefix = 0;
 int count = 0;
 for (int k = 1; k < n; k++) {
 prefix += a[k - 1];
 if (prefix == total - prefix) count++;
 }
 System.out.println(count);
 }
}
```

Why it works: prefix == total − prefix at each split point; count them.

Verified against 3 test cases: `6⏎1 -1 1 -1 1 -1⏎` → `2`; `4⏎2 2 2 2⏎` → `1`; `3⏎1 2 3⏎` → `1`

5.18 Maximum Sum of an Hourglass in a Matrix

Exercise 1 · One hourglass

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Read a 3x3 matrix and print the single hourglass sum.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int[][] m = new int[3][3];
 for (int i = 0; i < 3; i++) for (int j = 0; j < 3; j++) m[i][j] = sc.nextInt();
 System.out.println(m[0][0] + m[0][1] + m[0][2] + m[1][1] + m[2][0] + m[2][1] + m[2][2]);
 }
}
```

Why it works: Add the two full outer rows and the centre cell.

Verified against 3 test cases: `1 1 1⏎0 1 0⏎1 1 1⏎` → `7`; `0 0 0⏎5 5 5⏎0 0 0⏎` → `5`; `-1 -1 -1⏎-1 -1 -1⏎-1 -1 -1⏎` → `-7`

Exercise 2 · Maximum hourglass with negatives

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 int[][] m = new int[r][c];
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) m[i][j] = sc.nextInt();
 int best = Integer.MIN_VALUE;
 for (int i = 0; i + 2 < r; i++) {
 for (int j = 0; j + 2 < c; j++) {
 int s = m[i][j] + m[i][j + 1] + m[i][j + 2] + m[i + 1][j + 1] + m[i + 2][j] + m[i + 2][j + 1] +
 m[i + 2][j + 2];
 best = Math.max(best, s);
 }
 }
 System.out.println(best);
 }
}
```

Why it works: All (R−2)(C−2) hourglass sums are compared; the initial value must be below any possible sum.

Verified against 3 test cases: `3 3⏎-1 -1 -1⏎-1 -9 -1⏎-1 -1 -1⏎` → `-15`; `4 4⏎1 2 3 4⏎5 6 7 8⏎9 1 2 3⏎4 5 6 7⏎` → `41`; `3 4⏎1 1 1 1⏎1 9 1 1⏎1 1 1 1⏎` → `15`

Exercise 3 · Largest k×k block

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Largest sum of any k x k block, using a 2-D prefix-sum table so each block costs O(1).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 int k = sc.nextInt();
 long[][] p = new long[r + 1][c + 1]; // p[i][j] = sum of the rectangle rows 0..i-1, cols 0..j-1
 for (int i = 1; i <= r; i++) {
 for (int j = 1; j <= c; j++) {
 p[i][j] = sc.nextLong() + p[i - 1][j] + p[i][j - 1] - p[i - 1][j - 1];
 }
 }
 long best = Long.MIN_VALUE;
 for (int i = k; i <= r; i++) {
 for (int j = k; j <= c; j++) {
 long s = p[i][j] - p[i - k][j] - p[i][j - k] + p[i - k][j - k];
 best = Math.max(best, s);
 }
 }
 System.out.println(best);
 }
}
```

Why it works: The prefix table lets any rectangle sum be evaluated in four lookups, independent of its size.

Verified against 3 test cases: `3 3 2⏎1 2 3⏎4 5 6⏎7 8 9⏎` → `28`; `2 2 2⏎-1 -2⏎-3 -4⏎` → `-10`; `2 3 1⏎5 -1 3⏎0 2 -4⏎` → `5`

5.19 Toggle the Switch (100 Doors)

Exercise 1 · Count the open doors

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 boolean[] open = new boolean[n + 1];
 for (int m = 1; m <= n; m++) for (int d = m; d <= n; d += m) open[d] = !open[d];
 int c = 0;
 for (int d = 1; d <= n; d++) if (open[d]) c++;
 System.out.println(c);
 }
}
```

Why it works: Simulate every monkey and count the true cells.

Verified against 3 test cases: `10⏎` → `3`; `100⏎` → `10`; `1⏎` → `1`

Exercise 2 · A trillion doors

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 long r = (long) Math.sqrt((double) n);
 while (r * r > n) r--;
 while ((r + 1) * (r + 1) <= n) r++;
 System.out.println(r);
 }
}
```

Why it works: $\lfloor\sqrt{n}\rfloor$ with a floating-point estimate corrected by two while-loops.

Verified against 3 test cases: `1000000000000000000⏎` → `1000000000`; `99⏎` → `9`; `999999999999999999⏎` → `999999999`

Exercise 3 · Only the first m monkeys

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Only the first m monkeys are released. Count doors that are open afterwards.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int m = sc.nextInt();
 boolean[] open = new boolean[n + 1];
 for (int monkey = 1; monkey <= Math.min(m, n); monkey++) {
 for (int d = monkey; d <= n; d += monkey) open[d] = !open[d];
 }
 int c = 0;
 for (int d = 1; d <= n; d++) if (open[d]) c++;
 System.out.println(c);
 }
}
```

Why it works: The shortcut no longer applies (a door's toggle count is the number of divisors ≤ m), so simulate only the m monkeys.

Verified against 3 test cases: `10 3⏎` → `4`; `10 10⏎` → `3`; `6 1⏎` → `6`

5.20 Alice's Apple Tree (Minimum Garden Perimeter)

Exercise 1 · Apples in a square

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long r = sc.nextLong();
 System.out.println(2 * r * (r + 1) * (2 * r + 1));
 }
}
```

Why it works: Direct evaluation of the closed form in long arithmetic.

Verified against 3 test cases: `3⏎` → `168`; `1⏎` → `12`; `2⏎` → `60`

Exercise 2 · Minimum perimeter (small)

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long need = sc.nextLong();
 long r = 0, sum = 0;
 while (sum < need) {
 r++;
 sum += 12 * r * r;
 }
 System.out.println(8 * r);
 }
}
```

Why it works: Ring-by-ring accumulation in long.

Verified against 3 test cases: `13⏎` → `16`; `1⏎` → `8`; `1000000000⏎` → `5040`

Exercise 3 · Minimum perimeter (large)

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static long inside(long r) {
 return 2 * r * (r + 1) * (2 * r + 1);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long need = sc.nextLong();
 long lo = 1, hi = 200_000;
 while (lo < hi) {
 long mid = (lo + hi) / 2;
 if (inside(mid) >= need) hi = mid;
 else lo = mid + 1;
 }
 System.out.println(8 * lo);
 }
}
```

Why it works: Binary search for the smallest r with apples(r) ≥ needed; answer 8r.

Verified against 3 test cases: `100000000000000⏎` → `503968`; `13⏎` → `16`; `1⏎` → `8`

5.21 Weighted Substring (Smallest String of a Given Weight)

Exercise 1 · Weight of a string

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Total weight of a string made of A..Z, with weight(A)=1 and weight(next) = (index+1) *
     weight(previous).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 long[] w = new long[26];
 w[0] = 1;
 for (int i = 1; i < 26; i++) w[i] = w[i - 1] * (i + 2);
 long sum = 0;
 for (char c : s.toCharArray()) sum += w[c - 'A'];
 System.out.println(sum);
 }
}
```

Why it works: Table lookup per character; sum in long.

Verified against 3 test cases: `ACC⏎` → `25`; `AABBC⏎` → `20`; `D⏎` → `60`

Exercise 2 · Smallest string

Ex2.java

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long weight = sc.nextLong();
 List<Long> w = new ArrayList<>();
 long cur = 1;
 w.add(cur);
 for (int n = 2; n <= 26; n++) {
 if (cur > weight / (n + 1)) break;
 cur *= (n + 1);
 w.add(cur);
 }
 List<Character> out = new ArrayList<>();
 for (int i = w.size() - 1; i >= 0; i--) {
 while (weight >= w.get(i)) {
 out.add((char) ('A' + i));
 weight -= w.get(i);
 }
 }
 Collections.sort(out);
 StringBuilder sb = new StringBuilder();
 for (char c : out) sb.append(c);
 System.out.println(sb);
 }
}
```

Why it works: Each weight divides the next, so greedy gives the minimum count; ascending order gives the smallest string.

Verified against 4 test cases: `25⏎` → `ACC`; `20⏎` → `AABBC`; `1⏎` → `A`

Exercise 3 · Normalise a string

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Normalise: print the shortest, lexicographically smallest string with the same weight as s.
 // Carry like a number system: (letter i+1 has weight (i+2) times letter i), so (i+2) copies of
     letter i
 // become one copy of letter i+1.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 long[] count = new long[27];
 for (char c : s.toCharArray()) count[c - 'A']++;
 for (int i = 0; i < 25; i++) {
 long base = i + 3; // weight(letter i+1) / weight(letter i): 3 for A->B, 4 for B->C, ...
 count[i + 1] += count[i] / base;
 count[i] %= base;
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < 26; i++) {
 for (long k = 0; k < count[i]; k++) sb.append((char) ('A' + i));
 }
 System.out.println(sb);
 }
}
```

Why it works: A mixed-radix number system: carry count[i] / (i + 2) into the next letter and keep the remainder.

Verified against 4 test cases: `AAAAB⏎` → `ABB`; `AAA⏎` → `B`; `BBBB⏎` → `C`

5.22 Move Hyphen to the Beginning

Exercise 1 · First hyphen to the front

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.nextLine();
 int i = s.indexOf('-');
 System.out.println(i < 0 ? s : "-" + s.substring(0, i) + s.substring(i + 1));
 }
}
```

Why it works: Rebuild the string as "-" + before + after.

Verified against 3 test cases: `face-prep⏎` → `-faceprep`; `abc⏎` → `abc`; `a-b-c⏎` → `-ab-c`

Exercise 2 · All hyphens to the front

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.nextLine();
 StringBuilder h = new StringBuilder();
 StringBuilder o = new StringBuilder();
 for (char c : s.toCharArray()) (c == '-' ? h : o).append(c);
 System.out.println(h.append(o));
 }
}
```

Why it works: A stable partition into hyphens and non-hyphens, concatenated.

Verified against 3 test cases: `a-b-c⏎` → `--abc`; `---⏎` → `---`; `abc⏎` → `abc`

Exercise 3 · Minimum adjacent swaps

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Minimum number of ADJACENT swaps to bring all hyphens to the front:
 // each hyphen must jump over every non-hyphen character that stands before it.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.nextLine();
 long nonHyphensSeen = 0;
 long swaps = 0;
 for (char c : s.toCharArray()) {
 if (c == '-') swaps += nonHyphensSeen;
 else nonHyphensSeen++;
 }
 System.out.println(swaps);
 }
}
```

Why it works: Hyphens never need to cross each other (that would be a wasted swap), so the cost is the number of (non-hyphen, hyphen) inversions.

Verified against 3 test cases: `a-b-c⏎` → `3`; `--ab⏎` → `0`; `ab--⏎` → `4`

5.23 Strobogrammatic Number

Exercise 1 · Is it strobogrammatic?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 String from = "01689";
 String to = "01986";
 int l = 0, r = s.length() - 1;
 boolean ok = true;
 while (l <= r) {
 int i = from.indexOf(s.charAt(l));
 if (i < 0 || to.charAt(i) != s.charAt(r)) {
 ok = false;
 break;
 }
 l++;
 r--;
 }
 System.out.println(ok ? "YES" : "NO");
 }
}
```

Why it works: The digit at the left pointer must rotate into the digit at the right pointer; the middle digit must be 0, 1 or 8.

Verified against 5 test cases: `69⏎` → `YES`; `818⏎` → `YES`; `962⏎` → `NO`

Exercise 2 · How many of length n?

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Count of n-digit strobogrammatic numbers (n >= 1, no leading zero; the single digit 0 counts
     for n = 1).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long count;
 if (n == 1) {
 count = 3; // 0, 1, 8
 } else {
 count = 4; // outer pair: 11, 69, 88, 96
 for (int i = 1; i < n / 2; i++) count *= 5; // each inner pair: 00, 11, 69, 88, 96
 if (n % 2 == 1) count *= 3; // middle digit: 0, 1, 8
 }
 System.out.println(count);
 }
}
```

Why it works: Multiplication principle: 4 · 5^(pairs−1) · (3 if n is odd).

Verified against 4 test cases: `4⏎` → `20`; `1⏎` → `3`; `3⏎` → `12`

Exercise 3 · Strobogrammatic in a range

Ex3.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex3 {
 static List<String> build(int n, int total) {
 if (n == 0) return new ArrayList<>(List.of(""));
 if (n == 1) return new ArrayList<>(List.of("0", "1", "8"));
 List<String> res = new ArrayList<>();
 for (String inner : build(n - 2, total)) {
 if (n != total) res.add("0" + inner + "0");
 res.add("1" + inner + "1");
 res.add("6" + inner + "9");
 res.add("8" + inner + "8");
 res.add("9" + inner + "6");
 }
 return res;
 }

 // Count strobogrammatic numbers in [low, high] given as strings of up to 15 digits.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String low = sc.next();
 String high = sc.next();
 int count = 0;
 for (int len = low.length(); len <= high.length(); len++) {
 for (String s : build(len, len)) {
 if (len == low.length() && s.compareTo(low) < 0) continue; // same length: string order =
     numeric order
 if (len == high.length() && s.compareTo(high) > 0) continue;
 count++;
 }
 }
 System.out.println(count);
 }
}
```

Why it works: Generate all candidates for each length and filter the ones outside the bounds.

Verified against 3 test cases: `50 100⏎` → `3`; `0 0⏎` → `1`; `1 1000⏎` → `18`

5.24 Lexicographically First Palindromic Rearrangement

Exercise 1 · Can it be a palindrome?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next().toLowerCase();
 int[] freq = new int[256];
 for (char c : s.toCharArray()) freq[c]++;
 int odd = 0;
 for (int f : freq) if (f % 2 == 1) odd++;
 System.out.println(odd <= 1 ? "YES" : "NO");
 }
}
```

Why it works: At most one letter may have an odd frequency.

Verified against 3 test cases: `aabb⏎` → `YES`; `apple⏎` → `NO`; `Malayalam⏎` → `YES`

Exercise 2 · Build the smallest palindrome

Ex2.java

```java
import java.util.Scanner;
import java.util.TreeMap;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 TreeMap<Character, Integer> f = new TreeMap<>();
 for (char c : sc.next().toLowerCase().toCharArray()) f.merge(c, 1, Integer::sum);
 String mid = "";
 int odd = 0;
 for (var e : f.entrySet()) {
 if (e.getValue() % 2 == 1) {
 odd++;
 mid = String.valueOf(e.getKey());
 }
 }
 if (odd > 1) {
 System.out.println("no palindromic string");
 return;
 }
 StringBuilder front = new StringBuilder();
 for (var e : f.entrySet()) front.append(String.valueOf(e.getKey()).repeat(e.getValue() / 2));
 System.out.println(front + mid + new StringBuilder(front).reverse());
 }
}
```

Why it works: Front half sorted, optional odd letter in the middle, mirrored back half.

Verified against 4 test cases: `Malayalam⏎` → `aalmymlaa`; `aabbcc⏎` → `abccba`; `apple⏎` → `no palindromic string`

Exercise 3 · How many palindromes?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Number of DISTINCT palindromic permutations: arrange the first half freely.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next().toLowerCase();
 int[] freq = new int[256];
 for (char c : s.toCharArray()) freq[c]++;
 int odd = 0;
 for (int f : freq) if (f % 2 == 1) odd++;
 if (odd > 1) {
 System.out.println(0);
 return;
 }
 long[] fact = new long[21];
 fact[0] = 1;
 for (int i = 1; i <= 20; i++) fact[i] = fact[i - 1] * i;
 int half = s.length() / 2;
 long ways = fact[half];
 for (int f : freq) ways /= fact[f / 2];
 System.out.println(ways);
 }
}
```

Why it works: The palindrome is determined by its left half; count the distinct arrangements of the half-multiset.

Verified against 4 test cases: `aabbcc⏎` → `6`; `aabb⏎` → `2`; `abc⏎` → `0`

5.25 Manacher's Algorithm (Longest Palindromic Substring)

Exercise 1 · Is it a palindrome?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 boolean ok = true;
 for (int l = 0, r = s.length() - 1; l < r; l++, r--) {
 if (s.charAt(l) != s.charAt(r)) {
 ok = false;
 break;
 }
 }
 System.out.println(ok ? "YES" : "NO");
 }
}
```

Why it works: Compare the characters at both ends and move inwards; any mismatch decides NO.

Verified against 3 test cases: `racecar⏎` → `YES`; `abca⏎` → `NO`; `a⏎` → `YES`

Exercise 2 · Longest palindromic substring (expand)

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 int bestStart = 0, bestLen = 1;
 for (int c = 0; c < s.length(); c++) {
 for (int t = 0; t < 2; t++) {
 int l = c, r = c + t;
 while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) {
 l--;
 r++;
 }
 if (r - l - 1 > bestLen) {
 bestLen = r - l - 1;
 bestStart = l + 1;
 }
 }
 }
 System.out.println(s.substring(bestStart, bestStart + bestLen));
 }
}
```

Why it works: Centre expansion with odd and even centres; O(n²).

Verified against 4 test cases: `cbbd⏎` → `bb`; `babad⏎` → `bab`; `a⏎` → `a`

Exercise 3 · Count palindromic substrings

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Count ALL palindromic substrings (by position) in O(n) using Manacher's radii.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 StringBuilder sb = new StringBuilder("#");
 for (char c : s.toCharArray()) sb.append(c).append('#');
 String t = sb.toString();
 int n = t.length();
 int[] rad = new int[n];
 int center = 0, right = 0;
 long total = 0;
 for (int i = 0; i < n; i++) {
 if (i < right) rad[i] = Math.min(right - i, rad[2 * center - i]);
 while (i - rad[i] - 1 >= 0 && i + rad[i] + 1 < n && t.charAt(i - rad[i] - 1) == t.charAt(i +
     rad[i] + 1)) rad[i]++;
 if (i + rad[i] > right) {
 center = i;
 right = i + rad[i];
 }
 total += (rad[i] + 1) / 2; // a radius of r in the transformed string gives ceil(r/2) palindromes
 }
 System.out.println(total);
 }
}
```

Why it works: Manacher's radii give, for every centre, the number of palindromes centred there; summing them is O(n).

Verified against 3 test cases: `abba⏎` → `6`; `aaa⏎` → `6`; `abc⏎` → `3`

5.26 Natural Sort Order

Exercise 1 · Sort file names naturally

Ex1.java

```java
import java.math.BigInteger;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex1 {
 static List<String> chunks(String s) {
 List<String> out = new ArrayList<>();
 int i = 0;
 while (i < s.length()) {
 int j = i;
 boolean d = Character.isDigit(s.charAt(i));
 while (j < s.length() && Character.isDigit(s.charAt(j)) == d) j++;
 out.add(s.substring(i, j));
 i = j;
 }
 return out;
 }

 static int cmp(String a, String b) {
 List<String> x = chunks(a), y = chunks(b);
 for (int i = 0; i < Math.min(x.size(), y.size()); i++) {
 String p = x.get(i), q = y.get(i);
 int c = (Character.isDigit(p.charAt(0)) && Character.isDigit(q.charAt(0))) ? new BigInteger(p)
 .compareTo(new BigInteger(q)) : p.compareToIgnoreCase(q);
 if (c != 0) return c;
 }
 return Integer.compare(x.size(), y.size());
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<String> names = new ArrayList<>();
 for (int i = 0; i < n; i++) names.add(sc.next());
 names.sort(Ex1::cmp);
 for (String s : names) System.out.println(s);
 }
}
```

Why it works: The comparator from the lesson, applied with List.sort.

Verified against 3 test cases: `4⏎img12.png⏎img10.png⏎img2.png⏎IMG1.png⏎` → `IMG1.png⏎img2.png⏎img10.png⏎img12.png`; `2⏎b2⏎a10⏎` → `a10⏎b2`; `3⏎x9⏎x10⏎x1⏎` → `x1⏎x9⏎x10`

Exercise 2 · Sort version numbers

Ex2.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex2 {
 // Sort version numbers such as 1.2.10 and 1.10 numerically, field by field.
 static int cmp(String a, String b) {
 String[] x = a.split("\\.");
 String[] y = b.split("\\.");
 for (int i = 0; i < Math.max(x.length, y.length); i++) {
 long p = i < x.length ? Long.parseLong(x[i]) : 0;
 long q = i < y.length ? Long.parseLong(y[i]) : 0;
 if (p != q) return Long.compare(p, q);
 }
 return 0;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<String> v = new ArrayList<>();
 for (int i = 0; i < n; i++) v.add(sc.next());
 v.sort(Ex2::cmp);
 System.out.println(String.join(" ", v));
 }
}
```

Why it works: Numeric comparison per dotted component; pad the shorter version with zeros.

Verified against 3 test cases: `4⏎1.10⏎1.2.10⏎1.2.9⏎1.9⏎` → `1.2.9 1.2.10 1.9 1.10`; `2⏎2.0⏎1.9.9⏎` → `1.9.9 2.0`; `2⏎1.0⏎1.0.0⏎` → `1.0 1.0.0`

Exercise 3 · Natural order with deterministic ties

Ex3.java

```java
import java.math.BigInteger;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex3 {
 static List<String> chunks(String s) {
 List<String> out = new ArrayList<>();
 int i = 0;
 while (i < s.length()) {
 int j = i;
 boolean d = Character.isDigit(s.charAt(i));
 while (j < s.length() && Character.isDigit(s.charAt(j)) == d) j++;
 out.add(s.substring(i, j));
 i = j;
 }
 return out;
 }

 // Natural order, ignoring case. When two names are equal numerically (file01 vs file1),
 // the one with FEWER leading zeros (the shorter text) comes first, then plain compareTo.
 static int cmp(String a, String b) {
 List<String> x = chunks(a), y = chunks(b);
 for (int i = 0; i < Math.min(x.size(), y.size()); i++) {
 String p = x.get(i), q = y.get(i);
 int c = (Character.isDigit(p.charAt(0)) && Character.isDigit(q.charAt(0))) ? new BigInteger(p)
 .compareTo(new BigInteger(q)) : p.compareToIgnoreCase(q);
 if (c != 0) return c;
 }
 if (x.size() != y.size()) return Integer.compare(x.size(), y.size());
 if (a.length() != b.length()) return Integer.compare(a.length(), b.length());
 return a.compareTo(b);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<String> names = new ArrayList<>();
 for (int i = 0; i < n; i++) names.add(sc.next());
 names.sort(Ex3::cmp);
 System.out.println(String.join(" ", names));
 }
}
```

Why it works: Adding tie-breakers makes the comparator a strict total order, so the output is deterministic.

Verified against 3 test cases: `4⏎file01⏎file1⏎File2⏎file10⏎` → `file1 file01 File2 file10`; `3⏎a1⏎A1⏎a01⏎` → `A1 a1 a01`; `1⏎zzz⏎` → `zzz`

5.27 Maneuvering (Counting Grid Paths)

Exercise 1 · Paths in a rectangle

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int m = sc.nextInt();
 int n = sc.nextInt();
 long[][] dp = new long[m][n];
 for (int i = 0; i < m; i++)
 for (int j = 0; j < n; j++)
 dp[i][j] = (i == 0 || j == 0) ? 1 : dp[i - 1][j] + dp[i][j - 1];
 System.out.println(dp[m - 1][n - 1]);
 }
}
```

Why it works: Bottom-up DP with long values.

Verified against 3 test cases: `3 7⏎` → `28`; `1 5⏎` → `1`; `18 18⏎` → `2333606220`

Exercise 2 · Paths with obstacles

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Grid of 0 (free) and 1 (blocked). Count right/down paths from top-left to bottom-right.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 int[][] g = new int[r][c];
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) g[i][j] = sc.nextInt();
 long[][] dp = new long[r][c];
 for (int i = 0; i < r; i++) {
 for (int j = 0; j < c; j++) {
 if (g[i][j] == 1) {
 dp[i][j] = 0;
 } else if (i == 0 && j == 0) {
 dp[i][j] = 1;
 } else {
 dp[i][j] = (i > 0 ? dp[i - 1][j] : 0) + (j > 0 ? dp[i][j - 1] : 0);
 }
 }
 }
 System.out.println(dp[r - 1][c - 1]);
 }
}
```

Why it works: Same recurrence; blocked cells are forced to 0 so no path goes through them.

Verified against 3 test cases: `3 3⏎0 0 0⏎0 1 0⏎0 0 0⏎` → `2`; `2 2⏎0 1⏎1 0⏎` → `0`; `1 1⏎0⏎` → `1`

Exercise 3 · Huge grid modulo a prime

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static final long MOD = 1_000_000_007L;

 static long power(long b, long e) {
 long r = 1;
 b %= MOD;
 while (e > 0) {
 if ((e & 1) == 1) r = r * b % MOD;
 b = b * b % MOD;
 e >>= 1;
 }
 return r;
 }

 // C(m+n-2, m-1) modulo 1e9+7 for m, n up to 100000: factorials and a modular inverse (Fermat).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int m = sc.nextInt();
 int n = sc.nextInt();
 int total = m + n - 2;
 long[] fact = new long[total + 1];
 fact[0] = 1;
 for (int i = 1; i <= total; i++) fact[i] = fact[i - 1] * i % MOD;
 long denom = fact[m - 1] * fact[n - 1] % MOD;
 System.out.println(fact[total] * power(denom, MOD - 2) % MOD);
 }
}
```

Why it works: nCr mod p = n! · inverse(r!) · inverse((n−r)!) mod p, with the inverse by fast exponentiation.

Verified against 3 test cases: `3 7⏎` → `28`; `1 1⏎` → `1`; `100000 100000⏎` → `691090292`

5.28 Sorted Distinct Permutations

Exercise 1 · How many distinct permutations?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Number of distinct permutations = n! / (c1! * c2! * ...)
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 int[] freq = new int[256];
 for (char c : s.toCharArray()) freq[c]++;
 long[] f = new long[21];
 f[0] = 1;
 for (int i = 1; i <= 20; i++) f[i] = f[i - 1] * i;
 long ways = f[s.length()];
 for (int x : freq) ways /= f[x];
 System.out.println(ways);
 }
}
```

Why it works: Multinomial coefficient from letter counts.

Verified against 4 test cases: `AAB⏎` → `3`; `ABCD⏎` → `24`; `AAAA⏎` → `1`

Exercise 2 · Print them sorted

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static StringBuilder out = new StringBuilder();

 static void build(int[] counts, char[] cur, int pos) {
 if (pos == cur.length) {
 out.append(new String(cur)).append('\n');
 return;
 }
 for (int c = 0; c < counts.length; c++) {
 if (counts[c] == 0) continue;
 counts[c]--;
 cur[pos] = (char) c;
 build(counts, cur, pos + 1);
 counts[c]++;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 int[] counts = new int[256];
 for (char c : s.toCharArray()) counts[c]++;
 build(counts, new char[s.length()], 0);
 System.out.print(out);
 }
}
```

Why it works: Count-based backtracking enumerates in sorted order without duplicates.

Verified against 3 test cases: `ABA⏎` → `AAB⏎ABA⏎BAA`; `BAC⏎` → `ABC⏎ACB⏎BAC⏎BCA⏎CAB⏎CBA`; `X⏎` → `X`

Exercise 3 · The k-th permutation

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static long[] fact = new long[21];

 // number of distinct arrangements of the remaining letters
 static long arrangements(int[] counts, int remaining) {
 long r = fact[remaining];
 for (int c : counts) r /= fact[c];
 return r;
 }

 // k-th (1-based) distinct permutation in sorted order, without generating the others.
 public static void main(String[] args) {
 fact[0] = 1;
 for (int i = 1; i <= 20; i++) fact[i] = fact[i - 1] * i;
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 long k = sc.nextLong();
 int[] counts = new int[256];
 for (char c : s.toCharArray()) counts[c]++;
 int n = s.length();
 if (k > arrangements(counts, n)) {
 System.out.println("NONE");
 return;
 }
 StringBuilder sb = new StringBuilder();
 for (int pos = 0; pos < n; pos++) {
 for (int c = 0; c < 256; c++) {
 if (counts[c] == 0) continue;
 counts[c]--;
 long block = arrangements(counts, n - pos - 1); // permutations that start with this letter here
 if (k <= block) {
 sb.append((char) c);
 break;
 }
 k -= block;
 counts[c]++;
 }
 }
 System.out.println(sb);
 }
}
```

Why it works: Ranking via multinomial counts: O(n · σ) instead of enumerating up to n! permutations.

Verified against 3 test cases: `DBCA 10⏎` → `BCDA`; `AAB 3⏎` → `BAA`; `AB 3⏎` → `NONE`

5.29 N-Queens (Backtracking)

Exercise 1 · Validate a placement

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Read n and, for each row, the column of its queen. Print VALID if no two queens attack each other.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] col = new int[n];
 for (int i = 0; i < n; i++) col[i] = sc.nextInt();
 boolean ok = true;
 for (int i = 0; i < n && ok; i++) {
 for (int j = i + 1; j < n; j++) {
 if (col[i] == col[j] || Math.abs(col[i] - col[j]) == j - i) {
 ok = false;
 break;
 }
 }
 }
 System.out.println(ok ? "VALID" : "INVALID");
 }
}
```

Why it works: Check every pair of rows: equal columns or equal row and column distance mean attack.

Verified against 3 test cases: `4⏎2 4 1 3⏎` → `VALID`; `4⏎1 2 3 4⏎` → `INVALID`; `1⏎1⏎` → `VALID`

Exercise 2 · Count solutions

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static int n, solutions;
 static boolean[] c, d1, d2;

 static void place(int r) {
 if (r == n) {
 solutions++;
 return;
 }
 for (int col = 0; col < n; col++) {
 int a = r - col + n - 1, b = r + col;
 if (c[col] || d1[a] || d2[b]) continue;
 c[col] = d1[a] = d2[b] = true;
 place(r + 1);
 c[col] = d1[a] = d2[b] = false;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 n = sc.nextInt();
 c = new boolean[n];
 d1 = new boolean[2 * n];
 d2 = new boolean[2 * n];
 place(0);
 System.out.println(solutions);
 }
}
```

Why it works: Exhaustive backtracking with O(1) conflict checks.

Verified against 4 test cases: `6⏎` → `4`; `8⏎` → `92`; `3⏎` → `0`

Exercise 3 · Lexicographically first solution

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static int n;
 static int[] pos;
 static boolean[] c, d1, d2;

 static boolean place(int r) {
 if (r == n) return true;
 for (int col = 0; col < n; col++) {
 int a = r - col + n - 1, b = r + col;
 if (c[col] || d1[a] || d2[b]) continue;
 c[col] = d1[a] = d2[b] = true;
 pos[r] = col;
 if (place(r + 1)) return true;
 c[col] = d1[a] = d2[b] = false;
 }
 return false;
 }

 // The lexicographically first solution as "column of the queen in each row" (1-based), or NONE.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 n = sc.nextInt();
 pos = new int[n];
 c = new boolean[n];
 d1 = new boolean[2 * n];
 d2 = new boolean[2 * n];
 if (!place(0)) {
 System.out.println("NONE");
 return;
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(pos[i] + 1);
 System.out.println(sb);
 }
}
```

Why it works: Depth-first search trying columns left to right returns the lexicographically smallest solution first.

Verified against 3 test cases: `8⏎` → `1 5 8 6 3 7 2 4`; `4⏎` → `2 4 1 3`; `3⏎` → `NONE`

5.30 Maze Solving (Rat in a Maze)

Exercise 1 · How many cells can the rat reach?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Count the cells reachable from the entry (2) moving up/down/left/right through non-walls
     (flood fill).
 static int r, c;
 static int[][] g;
 static boolean[][] seen;

 static int fill(int i, int j) {
 if (i < 0 || j < 0 || i >= r || j >= c || g[i][j] == 1 || seen[i][j]) return 0;
 seen[i][j] = true;
 return 1 + fill(i + 1, j) + fill(i - 1, j) + fill(i, j + 1) + fill(i, j - 1);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 r = sc.nextInt();
 c = sc.nextInt();
 g = new int[r][c];
 seen = new boolean[r][c];
 int si = 0, sj = 0;
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) {
 g[i][j] = sc.nextInt();
 if (g[i][j] == 2) { si = i; sj = j; }
 }
 System.out.println(fill(si, sj));
 }
}
```

Why it works: A flood fill counts every cell entered by the DFS exactly once.

Verified against 3 test cases: `3 3⏎2 0 1⏎1 0 1⏎1 0 3⏎` → `5`; `1 3⏎2 1 3⏎` → `1`; `2 2⏎2 0⏎0 3⏎` → `4`

Exercise 2 · Is there a way out?

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static int r, c;
 static int[][] g;
 static boolean[][] seen;

 static boolean reach(int i, int j) {
 if (i < 0 || j < 0 || i >= r || j >= c || g[i][j] == 1 || seen[i][j]) return false;
 if (g[i][j] == 3) return true;
 seen[i][j] = true;
 return reach(i + 1, j) || reach(i - 1, j) || reach(i, j + 1) || reach(i, j - 1);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 r = sc.nextInt();
 c = sc.nextInt();
 g = new int[r][c];
 seen = new boolean[r][c];
 int si = 0, sj = 0;
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) {
 g[i][j] = sc.nextInt();
 if (g[i][j] == 2) { si = i; sj = j; }
 }
 System.out.println(reach(si, sj) ? "YES" : "NO");
 }
}
```

Why it works: Depth-first search with a visited array; true when the exit cell is reached.

Verified against 3 test cases: `3 3⏎2 1 3⏎0 1 0⏎0 0 0⏎` → YES; `1 3⏎2 1 3⏎` → NO; `1 2⏎2 3⏎` → YES

Exercise 3 · Shortest path

Ex3.java

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Scanner;

public class Ex3 {
 // Length (number of moves) of the SHORTEST route from entry (2) to exit (3), or -1.
     Breadth-first search.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 int[][] g = new int[r][c];
 int si = 0, sj = 0;
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) {
 g[i][j] = sc.nextInt();
 if (g[i][j] == 2) { si = i; sj = j; }
 }
 int[][] dist = new int[r][c];
 for (int[] row : dist) java.util.Arrays.fill(row, -1);
 Deque<int[]> q = new ArrayDeque<>();
 q.add(new int[] {si, sj});
 dist[si][sj] = 0;
 int[] dr = {1, -1, 0, 0};
 int[] dc = {0, 0, 1, -1};
 while (!q.isEmpty()) {
 int[] cur = q.poll();
 if (g[cur[0]][cur[1]] == 3) {
 System.out.println(dist[cur[0]][cur[1]]);
 return;
 }
 for (int d = 0; d < 4; d++) {
 int ni = cur[0] + dr[d], nj = cur[1] + dc[d];
 if (ni < 0 || nj < 0 || ni >= r || nj >= c || g[ni][nj] == 1 || dist[ni][nj] != -1) continue;
 dist[ni][nj] = dist[cur[0]][cur[1]] + 1;
 q.add(new int[] {ni, nj});
 }
 }
 System.out.println(-1);
 }
}
```

Why it works: BFS visits cells in order of distance; the first time the exit is dequeued its distance is minimal.

Verified against 3 test cases: `3 3⏎2 0 0⏎1 1 0⏎3 0 0⏎` → `6`; `1 3⏎2 1 3⏎` → `-1`; `1 2⏎2 3⏎` → `1`

5.31 Hamiltonian Cycle

Exercise 1 · Verify a cycle

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Input: v, adjacency matrix, then a vertex sequence of length v+1. Is it a Hamiltonian cycle?
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int v = sc.nextInt();
 int[][] g = new int[v][v];
 for (int i = 0; i < v; i++) for (int j = 0; j < v; j++) g[i][j] = sc.nextInt();
 int[] p = new int[v + 1];
 for (int i = 0; i <= v; i++) p[i] = sc.nextInt();
 boolean ok = p[0] == p[v];
 boolean[] seen = new boolean[v];
 for (int i = 0; i < v && ok; i++) {
 if (seen[p[i]]) ok = false;
 seen[p[i]] = true;
 if (g[p[i]][p[i + 1]] == 0) ok = false;
 }
 System.out.println(ok ? "YES" : "NO");
 }
}
```

Why it works: Linear scan with a seen array.

Verified against 3 test cases: `4⏎0 1 0 1⏎1 0 1 0⏎0 1 0 1⏎1 0 1 0⏎0 1 2 3 0⏎` → `YES`; `4⏎0 1 0 1⏎1 0 1 0⏎0 1 0 1⏎1 0 1 0⏎0 2 1 3 0⏎` → `NO`; `3⏎0 1 1⏎1 0 1⏎1 1 0⏎0 1 2 0⏎` → `YES`

Exercise 2 · Find a cycle

Ex2.java

```java
import java.util.Arrays;
import java.util.Scanner;

public class Ex2 {
 static int v;
 static int[][] g;
 static int[] path;

 static boolean extend(int pos) {
 if (pos == v) return g[path[v - 1]][path[0]] == 1;
 for (int c = 1; c < v; c++) {
 boolean ok = g[path[pos - 1]][c] == 1;
 for (int i = 0; i < pos && ok; i++) if (path[i] == c) ok = false;
 if (!ok) continue;
 path[pos] = c;
 if (extend(pos + 1)) return true;
 path[pos] = -1;
 }
 return false;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 v = sc.nextInt();
 g = new int[v][v];
 for (int i = 0; i < v; i++) for (int j = 0; j < v; j++) g[i][j] = sc.nextInt();
 path = new int[v];
 Arrays.fill(path, -1);
 path[0] = 0;
 if (!extend(1)) {
 System.out.println("NO");
 return;
 }
 StringBuilder sb = new StringBuilder();
 for (int x : path) sb.append(x).append(' ');
 sb.append(0);
 System.out.println("YES " + sb);
 }
}
```

Why it works: Depth-first backtracking, candidates in increasing order, close the cycle at the end.

Verified against 3 test cases: `5⏎0 1 0 1 0⏎1 0 1 1 1⏎0 1 0 0 1⏎1 1 0 0 0⏎0 1 1 0 0⏎` → `NO`; `5⏎0 1 0 1 0⏎1 0 1 1 1⏎0 1 0 0 1⏎1 1 0 0 1⏎0 1 1 1 0⏎` → `YES 0 1 2 4 3 0`; `3⏎0 1 1⏎1 0 1⏎1 1 0⏎` → `YES 0 1 2 0`

Exercise 3 · Count Hamiltonian cycles

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static int v, count;
 static int[][] g;
 static boolean[] used;

 static void dfs(int cur, int placed) {
 if (placed == v) {
 if (g[cur][0] == 1) count++;
 return;
 }
 for (int nxt = 1; nxt < v; nxt++) {
 if (!used[nxt] && g[cur][nxt] == 1) {
 used[nxt] = true;
 dfs(nxt, placed + 1);
 used[nxt] = false;
 }
 }
 }

 // Count Hamiltonian cycles that start and end at vertex 0, counting a cycle and its reverse separately.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 v = sc.nextInt();
 g = new int[v][v];
 for (int i = 0; i < v; i++) for (int j = 0; j < v; j++) g[i][j] = sc.nextInt();
 used = new boolean[v];
 used[0] = true;
 dfs(0, 1);
 System.out.println(count);
 }
}
```

Why it works: Count every complete path that closes back to vertex 0.

Verified against 3 test cases: `4⏎0 1 1 1⏎1 0 1 1⏎1 1 0 1⏎1 1 1 0⏎` → `6`; `3⏎0 1 1⏎1 0 1⏎1 1 0⏎` → `2`; `4⏎0 1 0 0⏎1 0 1 0⏎0 1 0 1⏎0 0 1 0⏎` → `0`

5.32 Knight's Tour & Warnsdorff's Rule

Exercise 1 · Verify a tour

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Input: n, then an n x n board of move numbers. Is it a valid knight's tour (1..n*n, each step
     a knight move)?
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] px = new int[n * n + 1];
 int[] py = new int[n * n + 1];
 boolean ok = true;
 for (int y = 0; y < n; y++) {
 for (int x = 0; x < n; x++) {
 int v = sc.nextInt();
 if (v < 1 || v > n * n || px[v] != 0) { // out of range or a repeated number
 ok = false;
 } else {
 px[v] = x + 1;
 py[v] = y + 1;
 }
 }
 }
 for (int v = 1; v < n * n && ok; v++) {
 if (px[v] == 0 || px[v + 1] == 0) {
 ok = false;
 break;
 }
 int dx = Math.abs(px[v] - px[v + 1]);
 int dy = Math.abs(py[v] - py[v + 1]);
 if (!((dx == 1 && dy == 2) || (dx == 2 && dy == 1))) ok = false;
 }
 System.out.println(ok ? "YES" : "NO");
 }
}
```

Why it works: Positions per number, then check each consecutive pair.

Verified against 3 test cases: `5⏎1 6 15 10 21⏎14 9 20 5 16⏎19 2 7 22 11⏎8 13 24 17 4⏎25 18 3 12 23⏎` → YES; `3⏎1 2 3⏎4 5 6⏎7 8 9⏎` → NO; `1⏎1⏎` → YES

Exercise 2 · Degree of a square

Ex2.java

```
import java.util.Scanner;

public class Ex2 {
 // How many squares can a knight on (x, y) reach on an empty n x n board? (the "degree"
     Warnsdorff uses)
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int x = sc.nextInt();
 int y = sc.nextInt();
 int[] dx = {1, 1, 2, 2, -1, -1, -2, -2};
 int[] dy = {2, -2, 1, -1, 2, -2, 1, -1};
 int c = 0;
 for (int m = 0; m < 8; m++) {
 int nx = x + dx[m], ny = y + dy[m];
 if (nx >= 0 && ny >= 0 && nx < n && ny < n) c++;
 }
 System.out.println(c);
 }
}
```

Why it works: Count the legal moves; corners have 2, the centre 8.

Verified against 4 test cases: `8 0 0⏎` → `2`; `8 3 3⏎` → `8`; `5 2 2⏎` → `8`

Exercise 3 · Reachable squares

Ex3.java

```
import java.util.Scanner;

public class Ex3 {
 // Count the squares of an n x n board that a knight can reach from (0,0) in exactly k moves
 // (squares may be visited again). Plain breadth-first layers.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 int[] dx = {1, 1, 2, 2, -1, -1, -2, -2};
 int[] dy = {2, -2, 1, -1, 2, -2, 1, -1};
 boolean[][] cur = new boolean[n][n];
 cur[0][0] = true;
 for (int step = 0; step < k; step++) {
 boolean[][] next = new boolean[n][n];
 for (int y = 0; y < n; y++) {
 for (int x = 0; x < n; x++) {
 if (!cur[y][x]) continue;
 for (int m = 0; m < 8; m++) {
 int nx = x + dx[m], ny = y + dy[m];
 if (nx >= 0 && ny >= 0 && nx < n && ny < n) next[ny][nx] = true;
 }
 }
 }
 cur = next;
 }
 int c = 0;
 for (boolean[] row : cur) for (boolean b : row) if (b) c++;
 System.out.println(c);
 }
}
```

Why it works: Layer-by-layer breadth-first expansion: the set of squares reachable in exactly i moves.

Verified against 3 test cases: `8 2⏎` → `10`; `8 1⏎` → `2`; `8 0⏎` → `1`

5.33 Selection Sort

Exercise 1 · Sort and count swaps

Ex1.java

```
import java.util.Scanner;

public class Ex1 {
 // Selection sort; print how many swaps that actually move something (minIndex != i) were
     performed.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int swaps = 0;
 for (int i = 0; i < n - 1; i++) {
 int m = i;
 for (int j = i + 1; j < n; j++) if (a[j] < a[m]) m = j;
 if (m != i) {
 int t = a[m];
 a[m] = a[i];
 a[i] = t;
 swaps++;
 }
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 System.out.println(swaps);
 }
}
```

Why it works: Standard selection sort with a counter around the swap.

Verified against 3 test cases: `5⏎64 25 12 22 11⏎` → `11 12 22 25 64⏎3`; `3⏎1 2 3⏎` → `1 2 3⏎0`; `4⏎4 3 2 1⏎` → `1 2 3 4⏎2`

Exercise 2 · Largest first

SelEx2.java

```
import java.util.Scanner;

public class SelEx2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 for (int i = 0; i < n - 1; i++) {
 int maxIndex = i;
 for (int j = i + 1; j < n; j++) {
 if (a[j] > a[maxIndex]) maxIndex = j;
 }
 int t = a[maxIndex];
 a[maxIndex] = a[i];
 a[i] = t;
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

Why it works: Select the largest remaining value in each pass.

Verified against 3 test cases: `5⏎3 1 4 1 5⏎` → `5 4 3 1 1`; `1⏎7⏎` → `7`; `3⏎-1 -5 -3⏎` → `-1 -3 -5`

Exercise 3 · K smallest, cheaply

SelEx3.java

```java
import java.util.Scanner;

public class SelEx3 {
 // The k smallest values in increasing order using only k selection passes: O(n * k).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 for (int i = 0; i < k; i++) {
 int m = i;
 for (int j = i + 1; j < n; j++) {
 if (a[j] < a[m]) m = j;
 }
 int t = a[m];
 a[m] = a[i];
 a[i] = t;
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < k; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

Why it works: Reuse selection: after k passes the first k positions hold the k smallest in order.

Verified against 3 test cases: `6 3⏎7 10 4 3 20 15⏎` → `3 4 7`; `3 3⏎3 2 1⏎` → `1 2 3`; `4 1⏎9 8 7 6⏎` → `6`

5.34 Quick Sort

Exercise 1 · Count the partitions

QuickSortEx1.java

```java
import java.util.Scanner;

public class QuickSortEx1 {
 static int partition(int[] a, int low, int high) {
 int pivot = a[high];
 int i = low - 1;
 for (int j = low; j < high; j++) {
 if (a[j] <= pivot) {
 i++;
 int t = a[i];
 a[i] = a[j];
 a[j] = t;
 }
 }
 int t = a[i + 1];
 a[i + 1] = a[high];
 a[high] = t;
 return i + 1;
 }

 static void quickSort(int[] a, int low, int high) {
 if (low < high) {
 int p = partition(a, low, high);
 quickSort(a, low, p - 1);
 quickSort(a, p + 1, high);
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 quickSort(a, 0, n - 1);
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

Why it works: The lesson's algorithm with the output formatted as one line.

Verified against 3 test cases: `7⏎8 3 1 7 0 10 2⏎` → `0 1 2 3 7 8 10`; `1⏎5⏎` → `5`; `5⏎2 2 2 2 2⏎` → `2 2 2 2 2`

Exercise 2 · Descending quick sort

QuickSortEx2.java

```java
import java.util.Scanner;

public class QuickSortEx2 {
 static void quick(int[] a, int lo, int hi) {
 if (lo >= hi) return;
 int pivot = a[hi], i = lo - 1;
 for (int j = lo; j < hi; j++) {
 if (a[j] >= pivot) { // descending order: larger values go to the left part
 i++;
 int t = a[i];
 a[i] = a[j];
 a[j] = t;
 }
 }
 int t = a[i + 1];
 a[i + 1] = a[hi];
 a[hi] = t;
 quick(a, lo, i);
 quick(a, i + 2, hi);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 quick(a, 0, n - 1);
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

Why it works: Partition by "≥ pivot goes left" and recurse as before.

Verified against 3 test cases: `5⏎3 1 4 1 5⏎` → `5 4 3 1 1`; `2⏎1 2⏎` → `2 1`; `1⏎0⏎` → `0`

Exercise 3 · k-th smallest with quickselect

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Quickselect: k-th smallest (1-based) without sorting everything. Average O(n).
 static int select(int[] a, int k) {
 int lo = 0, hi = a.length - 1;
 while (true) {
 int pivot = a[hi], i = lo - 1;
 for (int j = lo; j < hi; j++) {
 if (a[j] <= pivot) {
 i++;
 int t = a[i];
 a[i] = a[j];
 a[j] = t;
 }
 }
 int t = a[i + 1];
 a[i + 1] = a[hi];
 a[hi] = t;
 int p = i + 1;
 if (p == k - 1) return a[p];
 if (p > k - 1) hi = p - 1;
 else lo = p + 1;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 System.out.println(select(a, k));
 }
}
```

Why it works: Quickselect discards the side that cannot contain the k-th element, so the work forms a geometric series ≈ 2n.

Verified against 3 test cases: `6 3⏎7 10 4 3 20 15⏎` → `7`; `3 1⏎9 8 7⏎` → `7`; `4 4⏎1 2 3 4⏎` → `4`

Part 5 — Quiz answer key

5.1 Euclid's Algorithm (GCD)

	Answer	Why
Q1	B. 6	48 = 2·18 + 12, 18 = 1·12 + 6, 12 = 2·6 + 0 → gcd = 6.
Q2	12	84 mod 36 = 12 → (36, 12); 36 mod 12 = 0 → (12, 0). a = 12.
Q3	line 4	Line 4 runs after a was overwritten, so `a % b` is `b % b` = 0. Save the remainder first.
Q4	0 | a | b	gcd(a, 0) = a, otherwise gcd(a, b) = gcd(b, a mod b).
Q5	B. O(log min(a, b))	The remainder is more than halved every two steps, so only about $2 \cdot \log_2(\min)$ steps are needed.
Q6	a after iteration 1 = 12; b after iteration 1 = 2; b after iteration 2 = 0; final a = 2	(50, 12) → (12, 2) → (2, 0). The answer is the last a.

5.2 Euler's Phi (Totient) Function

	Answer	Why
Q1	C. 16	17 is prime: every k from 1 to 16 is coprime to it. φ(p) = p − 1.
Q2	A. 16	$60 = 2^2 \cdot 3 \cdot 5$ → 60 × 1/2 × 2/3 × 4/5 = 16. (The lecture slide says 32 — that is a mistake.)
Q3	12	$36 = 2^2 \cdot 3^2$ → 36 − 18 = 18 (p = 2); then 18 − 6 = 12 (p = 3).
Q4	-=	result − result/p = result · (1 − 1/p).
Q5	B. A prime factor larger than √n can remain	The loop only searches p ≤ √n. A single bigger prime factor can survive as the remaining value of n.
Q6	φ(1) → 1; φ(9) → 6; φ(10) → 4; φ(13) → 12	φ(9) = 9 − 3 = 6; φ(10) = 4 (1,3,7,9); φ(13) = 12 (prime).

5.3 Chinese Remainder Theorem

	Answer	Why
Q1	A. 23	233 mod 105 = 23 and 23 checks out for all three.
Q2	C. Pairwise coprime	Pairwise coprime moduli make each M_i invertible modulo m_i.
Q3	17	Numbers ≡ 1 (mod 4): 1, 5, 9, 13, 17 → 17 % 5 = 2.
Q4	B. M / m_i	M_i = M / m_i is the product of all the other moduli.
Q5	C. no solution	x ≡ 1 (mod 6) forces x odd; x ≡ 2 (mod 4) forces x even.

5.4 Combinations (nCr)

	Answer	Why
Q1	B. 15	6!/(2!·4!) = 720/(2·24) = 15.
Q2	C. 30	6 · 5 = 30. Dividing by 2! gives the 15 combinations.
Q3	15	i=1: 1·5/1 = 5; i=2: 5·6/2 = 15.
Q4	B. fact(15) overflows int	$15! \approx 1.3 \times 10^{12} > 2.1 \times 10^9$.
Q5	A. C(n, n−r)	Choosing r to include is the same as choosing n−r to leave out.

5.5 Karatsuba's Multiplication

	Answer	Why
Q1	B. 3	ac, bd and (a+b)(c+d).
Q2	A. (a+b)(c+d) − ac − bd	Expanding (a+b)(c+d) = ac + ad + bc + bd.
Q3	C. O(n^1.585)	T(n) = 3T(n/2) + O(n) gives n^($\log_2$ 3).
Q4	2840	46 × 134 = 6164; 6164 − 672 − 2652 = 2840 = ad + bc.
Q5	B. Karatsuba has overhead that dominates for small n	Recursion and extra additions cost more than 1-2 machine multiplications.

5.6 Booth's Algorithm (signed binary multiplication)

	Answer	Why
Q1	C. 10	"10" marks the start of a run of 1s.
Q2	1001	1000 + 0001 = 1001 = −7.
Q3	B. 1110	The sign bit (1) is copied into the vacated left position.
Q4	A. 00010101	0001 0101 = 21 = 7 × 3.
Q5	B. 11111111	A single long run of 1s needs just one subtract (and the result is just a shift).

5.7 Simple Sieve of Eratosthenes

	Answer	Why
Q1	B. Because every composite ≤ n has a prime factor ≤ √n	A composite m = a·b with a ≤ b has a ≤ √m ≤ √n.
Q2	9 12 15 18 21 24 27 30	Multiples of 3 starting from 9.
Q3	B. O(n log log n)	Σ n/p over primes ≈ n log log n.
Q4	line 4	Starting at m = p crosses out p itself, so every prime is removed. Start at 2·p (or p·p).
Q5	B. 15	2 3 5 7 11 13 17 19 23 29 31 37 41 43 47.

5.8 Segmented & Incremental Sieve

	Answer	Why
Q1	C. √R	Every composite ≤ R has a prime factor ≤ √R.
Q2	105	(100 + 6) / 7 = 15; 15 · 7 = 105.
Q3	B. To avoid marking p itself and values below p·p	Smaller multiples were handled by smaller primes; p itself must stay unmarked.
Q4	B. Produces primes one after another, remembering upcoming composites	It stores for each upcoming composite the primes that will cross it out.

5.9 Binary Palindrome

	Answer	Why
Q1	B. 9 (1001)	1001 reads the same both ways.
Q2	3	6 = 110 → reversed 011 = 3.
Q3	B. multiplies by 2	A left shift moves every bit one place to the left: multiplication by 2.

	Answer	Why
Q4	&	`rest & 1` isolates the lowest bit.
Q5	B. O(log n)	One iteration per bit; the number of bits is $\log_2 n$.

5.10 Swap Two Nibbles in a Byte

	Answer	Why
Q1	B. 4	A nibble is half a byte: 4 bits.
Q2	195	0x3C → 0xC3 = 195.
Q3	B. 0xF0	0xF0 = 1111 0000.
Q4	B. To remove sign extension and get 0–255	Bytes are signed; & 0xFF keeps the low 8 bits as an unsigned value.

5.11 Longest Sequence of 1s (and after flipping a bit)

	Answer	Why
Q1	3	01101110: runs of 1s are 3 (bits 1–3) and 2 (bits 5–6).
Q2	C. previous + current + 1	Left run + flipped bit + right run.
Q3	B. 32	All 32 bits are already 1; a 32-bit int cannot hold more.
Q4	B. O(log n) (32 steps)	One step per bit; the number of bits is logarithmic in the value.

5.12 Josephus Problem

	Answer	Why
Q1	C. 5	Order: 2, 4, 6, 3, 1 → survivor 5.
Q2	3	ans: 0 → (0+2)%2=0 → (0+2)%3=2 → (2+2)%4=0 → (0+2)%5=2; +1 = 3.
Q3	C. O(n)	One loop iteration per person.
Q4	+	Shift by k places and wrap around the circle of `people`.
Q5	A. 9	2·4 + 1 = 9.

5.13 Block Swap Algorithm (Array Rotation)

	Answer	Why
Q1	3 4 5 1 2	Element i of the output is element (i + 2) mod 5 of the input.
Q2	C. Three reversals	Reverse(A), reverse(B), reverse(all) uses only swaps.
Q3	B. 5	n − d = 5.
Q4	C. O(n·d)	d passes of n element moves.
Q5	3	Final array: 3 4 5 6 1 2.

5.14 Leaders in an Array

	Answer	Why
Q1	A. 17 5 2	17 > all to its right; 5 > 2; 2 is last. 16 is smaller than 17.
Q2	line 1	Initialising max to 0 hides negative leaders; use Integer.MIN_VALUE.
Q3	B. O(n)	One pass.

	Answer	Why
Q4	A. Yes	Nothing to its right can be larger.

5.15 Majority Element (Boyer–Moore Voting)

	Answer	Why
Q1	B. at least 5	More than 9/2 = 4.5 → at least 5.
Q2	count after 2nd element (1) = 0; candidate after 3rd element (2) = 2; count after 4th element (4) = 0; final candidate = 7	(2,1) → (2,0) → (2,1) → (2,0) → (7,1). 7 is only a candidate; verification fails.
Q3	B. The candidate may not be a majority	If no majority exists, phase 1 still returns some value.
Q4	C. Boyer–Moore	Voting needs only a candidate and a counter.
Q5	B. O(n log n)	The sort dominates.

5.16 Maximum Product Subarray

	Answer	Why
Q1	B. 15	n(n+1)/2 = 15.
Q2	24	(−2)·3·(−4) = 24.
Q3	B. A negative number can turn it into the maximum	Most negative × negative = largest positive.
Q4	A. O(n)	One pass with O(1) work per element.
Q5	2	The subarray {2} has product 2.

5.17 Maximum Equilibrium Sum

	Answer	Why
Q1	B. 2	Left 1+3 = 4, right 2+2 = 4 at index 2.
Q2	3	8 − 4 − 1 = 3 (the elements 2 and 1).
Q3	B. total − left − a[i]	Everything not on the left and not a[i] is on the right.
Q4	B. O(n)	Two linear passes.

5.18 Maximum Sum of an Hourglass in a Matrix

	Answer	Why
Q1	A. 12	(5−2)(6−2) = 12.
Q2	B. two full rows of 3 and the centre cell	Top row, centre of the middle row, bottom row.
Q3	35	1+2+3+5+7+8+9 = 35 (the slide's own example).
Q4	B. All sums may be negative	Otherwise the result 0 would not correspond to any hourglass.

5.19 Toggle the Switch (100 Doors)

	Answer	Why
Q1	B. 10	The perfect squares 1, 4, 9, ..., 100.
Q2	B. 9	Divisors of 36: 1, 2, 3, 4, 6, 9, 12, 18, 36 — nine (odd), so it is open.
Q3	B. Divisors pair up as (d, n/d) except when d = n/d	The unpaired divisor √n makes the count odd.
Q4	B. O(n log n)	Σ n/k = n H(n).

5.20 Alice's Apple Tree (Minimum Garden Perimeter)

	Answer	Why
Q1	B. 48	$12 \cdot 2^2 = 48$ (and 12 + 48 = 60 inside the r = 2 square).
Q2	60	2·2·3·5 = 60.
Q3	B. apples(r) is monotonically increasing	More rings always mean more apples.
Q4	B. 8r	Side length 2r, four sides: 8r.

5.21 Weighted Substring (Smallest String of a Given Weight)

	Answer	Why
Q1	C. 60	D = 4·C + C = 5·12 = 60.
Q2	B. ACC	12 + 12 + 1 = 25 with just 3 letters.
Q3	ABB	Greedy takes B (3), B (3), then A (1): 3 + 3 + 1 = 7. Sorted: ABB.
Q4	B. Each weight divides the next	Divisibility makes the coin system canonical.

5.22 Move Hyphen to the Beginning

	Answer	Why
Q1	-abcd	Before: "ab", after: "cd".
Q2	B. -1	indexOf returns −1 when there is no match.
Q3	B. Strings are immutable so every += copies the string	Each step creates a new String of increasing length.
Q4	B. O(n)	Every character is read once and written once.

5.23 Strobogrammatic Number

	Answer	Why
Q1	C. 69	69 rotated by 180° is 69. 25 and 1962 contain 2 (no valid rotation); 962 contains 2 as well.
Q2	A. 0, 1, 8	The middle digit must rotate into itself.
Q3	true	8 rotates to 8 and 88 reads the same.
Q4	B. 4	11, 69, 88, 96.

5.24 Lexicographically First Palindromic Rearrangement

	Answer	Why
Q1	B. level	level: l:2, e:2, v:1 — one odd count.

	Answer	Why
Q2	abcba	front "ab", middle "c", back "ba".
Q3	C. O(n)	One pass to count plus a pass over the alphabet.
Q4	A. 0	The counts add up to an even number, so the number of odd counts is even; for a palindrome it must be ≤ 1, hence 0.

5.25 Manacher's Algorithm (Longest Palindromic Substring)

	Answer	Why
Q1	B. To make every palindrome odd-length	Even palindromes get a "#" centre.
Q2	C. O(n)	The right edge only moves forward.
Q3	#a#b#b#a#	n characters → 2n+1 characters.
Q4	B. aaaaaa	Every centre expands to the borders.

5.26 Natural Sort Order

	Answer	Why
Q1	B. file10	Compare "1" with "2": 1 < 2, so file10 comes first.
Q2	B. equal numerically (then tie-break)	Numerically they are both 7; add a tie-breaker for a deterministic order.
Q3	C. a negative number	Negative, zero, positive: less, equal, greater.
Q4	A. stable	It uses TimSort, a stable sort.

5.27 Maneuvering (Counting Grid Paths)

	Answer	Why
Q1	B. 20	C(6, 3) = 20.
Q2	6	Rows: 1 1 1 / 1 2 3 / 1 3 6.
Q3	B. It recomputes the same sub-grids many times	Overlapping sub-problems → exponential calls.
Q4	B. O(m·n)	Each cell is filled once.

5.28 Sorted Distinct Permutations

	Answer	Why
Q1	B. 6	4!/(2!·2!) = 6.
Q2	BAC	Pivot is A (A < C); smallest larger letter to its right is B; swap → BCA, reverse the tail "CA" → "AC" → BAC.
Q3	B. It prints every distinct permutation once, but not in sorted order	The HashSet prevents duplicates; swapping disturbs the order.
Q4	C. O(8·8!)	8! permutations × O(8) to build/print each.

5.29 N-Queens (Backtracking)

	Answer	Why
Q1	B. 2	Two mirror-image solutions.
Q2	C. same diagonal	|Δrow| = |Δcol| = 2.

	Answer	Why
Q3	0	Remove the queen again (un-choose).
Q4	B. Undoing the last choice and trying the next option	Return to the previous decision and change it.
Q5	C. Backtracking	Systematic search with pruning.

5.30 Maze Solving (Rat in a Maze)

	Answer	Why
Q1	B. To avoid walking in circles forever	Without it the search can bounce between two cells indefinitely.
Q2	B. BFS	BFS explores in rings of increasing distance.
Q3	B. Un-mark a dead-end cell (backtrack)	The cell is not on any path to the exit.
Q4	B. O(R·C)	Each cell is expanded once.

5.31 Hamiltonian Cycle

	Answer	Why
Q1	B. every vertex exactly once and returns to the start	Edges once = Eulerian; vertices once = Hamiltonian.
Q2	B. A cycle can be rotated to start anywhere	Any cycle through all vertices contains vertex 0.
Q3	C. O(V!)	Up to (V−1)! orderings.
Q4	B. impossible	A cycle needs two edges at every vertex.

5.32 Knight's Tour & Warnsdorff's Rule

	Answer	Why
Q1	C. 8	Eight L-shaped jumps.
Q2	B. the fewest onward moves	Visit the most constrained square first.
Q3	B. No, it is a heuristic	It can get stuck; check the result.
Q4	B. O(N^2)	N^2 moves with constant work each.

5.33 Selection Sort

	Answer	Why
Q1	10 13 14 37 29	Pass 1: swap 10 with 29 → 10 29 14 37 13. Pass 2: min of rest is 13 → swap with 29 → 10 13 14 37 29.
Q2	C. 15	5 + 4 + 3 + 2 + 1 = 15.
Q3	B. At most n−1 swaps	One swap per pass at most.
Q4	C. O(n^2)	It always scans the entire unsorted part.

5.34 Quick Sort

	Answer	Why
Q1	B. in its final sorted position	Everything left is ≤ and everything right is >.
Q2	C. O(n^2)	Highly unbalanced partitions.

	Answer	Why
Q3	5	Five elements (3, 2, 5, 1, 4) are ≤ 6 so the pivot goes to index 5.
Q4	B. Divide and conquer	Partition, then solve the two halves independently.
Q5	B. No	Partitioning swaps distant elements, so equal keys may be reordered.

Appendices

Reference material for quick lookup and revision.

Appendix A — Cheat sheets

Java syntax quick sheet

Program skeleton	Meaning
`public class Main { … }`	A class named like the file (Main.java).
`public static void main(String[] args)`	Where execution starts.
`System.out.println(x);`	Print x and a newline. print = no newline; printf = formatted.
`// comment /* block */`	Ignored by the compiler.

Primitive types	Meaning
`byte / short / int / long`	8 / 16 / 32 / 64-bit integers. int max = 2 147 483 647.
`float / double`	32 / 64-bit decimals. Use double by default.
`char`	One UTF-16 character, written 'A'. Behaves like a small int.
`boolean`	true or false. if() needs a boolean — no int.
`long x = 5_000_000_000L;`	Suffix L for long literals; f for float.

Input / output	Meaning
`Scanner sc = new Scanner(System.in);`	import java.util.*;
`sc.nextInt() / nextLong() / nextDouble()`	Read one token of that type.
`sc.next() / sc.nextLine()`	One word / the rest of the line.
`System.out.printf("%d %5.2f %s%n", a, b, c);`	%d int, %f decimal, %s string, %n newline.

Control flow	Meaning
`if (c) {…} else if (c2) {…} else {…}`	Condition must be boolean.
`for (int i = 0; i < n; i++) {…}`	Counting loop.
`for (int x : arr) {…}`	for-each over arrays/collections.
`while (c) {…} do {…} while (c);`	do-while runs at least once.
`switch (v) { case 1: …; break; default: … }`	Without break execution falls through.

Operators & precedence

From highest to lowest precedence	Meaning
`() [] .`	grouping, array index, member access
`x++ x-- ++x --x +x -x ! ~`	unary
`* / %`	multiplicative (left to right)
`+ -`	additive (+ also joins strings)

From highest to lowest precedence	Meaning
`<< >> >>>`	shifts
`< <= > >= instanceof`	relational
`== !=`	equality
`& ^ |`	bitwise AND, XOR, OR (in that order)
`&& ||`	logical AND, OR (short-circuit)
`?:`	ternary (right to left)
`= += -= *= /= %= <<= >>= &= |= ^=`	assignment (right to left)

Gotchas	Meaning
`7 / 2 → 3 7 / 2.0 → 3.5`	int ÷ int is integer division.
`-7 % 3 → -1`	Remainder keeps the sign of the dividend.
`"" + 3 + 4 → "34" 3 + 4 + "" → "7"`	Left to right: numbers add until a String appears.
`a == b on Strings`	Compares references. Use a.equals(b).

Bit-manipulation toolkit

Operators	Meaning
`x & y`	AND — 1 only where both are 1
`x | y`	OR — 1 where either is 1
`x ^ y`	XOR — 1 where they differ
`~x`	NOT — flips every bit (−x−1)
`x << k`	multiply by 2^k
`x >> k`	arithmetic shift: divide by 2^k, keeps sign
`x >>> k`	logical shift: shifts in zeros

Patterns	Meaning
`(x >> i) & 1`	read bit i
`x | (1 << i)`	set bit i
`x & ~(1 << i)`	clear bit i
`x ^ (1 << i)`	toggle bit i
`x & 1`	is x odd?
`x & (x - 1)`	clear the lowest set bit
`x & -x`	isolate the lowest set bit
`(1 << n) - 1`	mask of n ones
`((b & 0xF0) >>> 4) | ((b & 0x0F) << 4)`	swap the nibbles of a byte

Patterns	Meaning
`Integer.toBinaryString(x)` `Integer.bitCount(x)`	built-in helpers

Complexity cheat sheet

Growth classes (best → worst)	Meaning
`O(1)`	array access, odd/even
`O(log n)`	binary search, Euclid, halving loops
`O(√n)`	trial division, loops while i·i ≤ n
`O(n)`	one pass, linear search
`O(n log n)`	merge sort, quick sort (average), sorting + scan
$O(n^2)$	two nested loops, bubble/selection/insertion sort
$O(n^3)$	three nested loops, naive matrix multiply
$O(2^n)$	all subsets, naive Fibonacci
`O(n!)`	all permutations

Counting rules	Meaning
`sequential blocks`	add the costs → keep the largest
`nested loops`	multiply the iteration counts
`i *= 2 or n /= 2`	$\log_2 n$ iterations
`j += i inside i-loop`	harmonic series → n log n
`T(n) = T(n-1) + O(1)`	O(n)
`T(n) = 2T(n/2) + O(n)`	O(n log n)
`T(n) = 3T(n/2) + O(n)`	O(n^1.585) — Karatsuba
`recursion space`	depth of the call stack

Java Collections quick sheet

Which one?	Meaning
`ArrayList<T>`	resizable array. get(i) O(1), add at end O(1) amortized, insert middle O(n)
`LinkedList<T>`	doubly linked list; also a Deque. No fast get(i).
`HashSet<T>`	unique items, O(1) contains/add average, no order
`HashMap<K,V>`	key → value, O(1) average. getOrDefault, merge, entrySet
`TreeMap / TreeSet`	sorted, O(log n)
`ArrayDeque<T>`	stack AND queue: push/pop/offer/poll
`PriorityQueue<T>`	min-heap by default: poll() returns the smallest

Which one?	Meaning
`Arrays.sort(a) Collections.sort(list)`	sort; comparator for custom order

String & StringBuilder

Methods you will use constantly	Meaning
`s.length() s.charAt(i)`	size / character at index i
`s.substring(a, b)`	chars a … b−1
`s.indexOf(x) s.contains(x)`	position (−1 if absent) / test
`s.equals(t) s.compareTo(t)`	content equality / ordering
`s.toCharArray() new String(arr)`	to and from char[]
`s.split(" ") String.join(",", list)`	split / join
`new StringBuilder(s).reverse().toString()`	reverse a string
`sb.append(x) sb.insert(0, x) sb.setCharAt(i, c)`	mutable edits
`c - 'a'`	index 0..25 of a lowercase letter

Formula sheet (archive topics)

Number theory	Meaning
`gcd(a, b) = gcd(b, a mod b)`	Euclid; gcd(a, 0) = a
`lcm(a, b) = a / gcd(a, b) · b`	divide first to avoid overflow
`φ(n) = n · Π(1 − 1/p)`	p over distinct primes dividing n
`C(n, r) = n! / (r!(n−r)!)`	C(n, r) = C(n−1, r−1) + C(n−1, r)
`x = Σ r_i·M_i·inv(M_i) mod M`	Chinese remainder theorem
`Josephus: J(1) = 0; J(n) = (J(n−1) + k) mod n`	0-based; add 1 for 1-based
`Equilibrium: leftSum == total − leftSum − a[i]`	single pass
`Alice's apples: 2n(n+1)(2n+1); perimeter 8n`	garden of half-side n

Appendix B — Core facts to remember

Core facts to remember

Question	Answer
Which operator gives the remainder in Java?	`%` — e.g. 17 % 5 = 2. The result takes the sign of the left operand: -7 % 3 = -1.
Why is `7 / 2` equal to 3 in Java?	Both operands are int, so Java performs integer division and drops the fraction. Use `7 / 2.0` for 3.5.
How do you compare two Strings by content?	`a.equals(b)` — `==` compares references, not characters.
What does `i++` vs `++i` do inside an expression?	`i++` yields the OLD value then increments; `++i` increments first and yields the new value.
int overflow: what is Integer.MAX_VALUE + 1?	Integer.MIN_VALUE (−2 147 483 648). Java integers wrap around silently.
What must every recursive method have?	A base case that stops the recursion, and a recursive step that moves toward it.
Pass-by-value for objects: what is copied?	The reference (the address). The called method can change the object, but cannot make the caller's variable point to a different object.
Difference between == and equals() for Integer objects?	`==` compares references (cached only for −128..127); `equals` compares values. Prefer equals.
What does a `break` do inside a switch?	Leaves the switch. Without it, execution falls through into the next case.
How do you get the last digit and drop the last digit of n?	Last digit: `n % 10`. Drop it: `n / 10`.
Stable vs unstable sorting?	A stable sort keeps equal elements in their original relative order (merge sort, insertion sort). Quick sort and selection sort are not stable.
What does "in-place" mean?	The algorithm uses O(1) extra memory beyond the input (it rearranges elements within the array).
Big-O vs Big-Omega vs Big-Theta?	O = upper bound (at most), Ω = lower bound (at least), Θ = tight bound (both).
Auxiliary space vs space complexity?	Auxiliary = extra temporary memory. Space complexity = auxiliary + the input itself.
Backtracking template?	choose → explore (recurse) → un-choose. Undo the choice so the next option starts from a clean state.
x & (x − 1) does what?	Clears the lowest set bit. x is a power of two exactly when x > 0 and x & (x−1) == 0.
Arithmetic vs logical right shift?	`>>` copies the sign bit (divides negatives correctly); `>>>` always shifts in 0.
Two's complement: how do you negate?	Flip every bit and add 1. −x = ~x + 1.

Appendix C — Video and reading library

Entries marked **verified video** were retrieved from YouTube (title, channel and length are YouTube's own). Entries marked **search link** open a targeted YouTube search — they are not endorsements of a specific video.

Lesson	Resource	Channel	Length	Type
0.1 How Java Works: Source, Bytecode, JVM, JRE, JDK	JVM, JRE, and JDK - Fully Explained in 5 Minutes	Coding with John	5:28	verified video
0.1 How Java Works: Source, Bytecode, JVM, JRE, JDK	# 03-Difference between JDK JRE JVM | BYTECODE | |JIT COMPILER |JAVA COMPILATION PROCESS | IN HINDI	PortHub Academy	27:20	verified video
0.2 The Java Landscape: History, Features, Editions, JVM Memory	#26 Stack And Heap in Java	Telusko	12:37	verified video
0.2 The Java Landscape: History, Features, Editions, JVM Memory	Stack vs Heap Memory - Simple Explanation	Alex Hyett	5:28	verified video
0.3 Installing Java on macOS and Choosing an Editor	How to Install Java on Mac | Install Java JDK on macOS	ProgrammingKnowledge	16:48	verified video
0.3 Installing Java on macOS and Choosing an Editor	IntelliJ IDEA Setup for Java Beginners | Install IDE + Run Hello World (Step-by-Step)	The Study Room Journal	5:45	verified video
0.4 Your First Program: class, main, statements, comments	Java Hello World Program | Meaning of public static void main (String args)| System.out.println()	TechnonTechTV	8:22	verified video
0.4 Your First Program: class, main, statements, comments	Java - Basic Syntax	TutorialsPoint	8:50	verified video
0.5 Compile-time, Runtime and Logic Errors	Syntax Errors, Runtime Errors & Logical Errors in Java (Demo)	CodeWithHarry	17:51	verified video
0.5 Compile-time, Runtime and Logic Errors	Java Tutorial - Debug syntax errors vs runtime exceptions	LinkedIn Learning	5:34	verified video
0.6 What Is an Algorithm?	Flow of Program - Flowcharts & Pseudocode	Kunal Kushwaha	27:02	verified video
0.6 What Is an Algorithm?	Data Structures & Algorithms (Episode 1) – Introduction & Why They Matter	Smart Tutorials	4:49	verified video
1.1 Variables, Constants and Primitive Types	Java Tutorial: Variables and Data Types in Java Programming	CodeWithHarry	24:48	verified video
1.1 Variables, Constants and Primitive Types	Data Types in Java (float,double,char, boolean) | part 2 | JAVA Placement Course	Jenny's Lectures CS IT	1:05:58	verified video
1.2 Type Conversion, Casting and Overflow	Java Type Casting Explained | Widening & Narrowing Casting with Examples | KTU B.Tech, BCA, MCA	TECH MARIA	14:13	verified video
1.2 Type Conversion, Casting and Overflow	The Integer Overflow Error	Uma Abu	4:47	verified video
1.3 Arithmetic, Assignment and Increment Operators	Increment and Decrement Operators in Java	Neso Academy	5:14	verified video
1.3 Arithmetic, Assignment and Increment Operators	The Division Operator in Java	Neso Academy	5:21	verified video
1.4 Relational, Logical and Ternary Operators; Precedence	Java Tutorial: Relational and Logical Operators in Java	CodeWithHarry	12:25	verified video

Lesson	Resource	Channel	Length	Type
1.4 Relational, Logical and Ternary Operators; Precedence	Logical Operators in Java	Neso Academy	6:19	verified video
1.5 Bitwise Operators and Shifts	Bitwise operators in Java | and, or, not, xor, left shift, right shift & unsigned right shift	Engineering Digest	17:40	verified video
1.5 Bitwise Operators and Shifts	How do I Set, Clear, and Toggle a Single Bit?	Jacob Sorber	9:26	verified video
1.6 Output: print, println, printf and String Concatenation	Java System.out,System.err,append,println,print, printf,format and static import	Brainy Ghosts	23:10	verified video
1.6 Output: print, println, printf and String Concatenation	JAVA || How to format output in java using System.out.printf(...) & String x = String.format(...);	Computer Engineering life	11:06	verified video
1.7 Input: Scanner, BufferedReader and Command-Line Arguments	#83 User Input using BufferedReader and Scanner in Java	Telusko	11:49	verified video
1.7 Input: Scanner, BufferedReader and Command-Line Arguments	Fast Input/Output template for Competitive Programming in JAVA (Reduce TLE)	Tech Tips	5:15	verified video
1.8 Decision Making: if, else, switch	Conditional Statements | If-else, Switch Break | Complete Java Placement Course | Lecture 3	Apna College	25:08	verified video
1.8 Decision Making: if, else, switch	Java Switch Statement Explained | break, default, fall-through | Interview Essential!	ConstantPath	16:08	verified video
1.9 Loops: for, while, do-while, break and continue	Java For Beginners: While, Do While & For Loops (7/10)	Codecourse	10:59	verified video
1.9 Loops: for, while, do-while, break and continue	15. Break, Continue and Nested Loops in Java	Abhimanyu Kumar Vatsa	6:59	verified video
1.10 Methods: Parameters, Return Values, Scope and Overloading	Java Methods for Beginners: Arguments, Parameters, Returning Values,	Professor Hank Stalica	19:57	verified video
1.10 Methods: Parameters, Return Values, Scope and Overloading	#25 Method Overloading in Java	Telusko	6:03	verified video
1.11 Tracing and Debugging Programs	Debugging Java code in IntelliJ IDEA	IntelliJ IDEA, a JetBrains IDE	7:26	verified video
1.11 Tracing and Debugging Programs	how to run the Trace Table | Step-by-Step Dry Run, Variable Tracking, Common Errors & Programming	Computing Knowledge	23:31	verified video
2.1 One-Dimensional Arrays	Single-Dimensional Arrays in Java (Part 1)	Neso Academy	9:15	verified video
2.1 One-Dimensional Arrays	Operations on Arrays in Data Structures: Traversal, Insertion, Deletion and Searching	CodeWithHarry	28:45	verified video
2.2 Two-Dimensional Arrays (Matrices)	Learn Java 2D arrays in 9 minutes! □	Bro Code	9:38	verified video
2.2 Two-Dimensional Arrays (Matrices)	2D Arrays | Java Complete Placement Course | Lecture 11	Apna College	19:53	verified video
2.3 Strings, StringBuilder and Characters	Useful string methods in Java!	Bro Code	8:36	verified video

Lesson	Resource	Channel	Length	Type
2.3 Strings, StringBuilder and Characters	#35 Mutable vs Immutable String in Java	Telusko	6:23	verified video
2.4 Number Systems, Binary and Two's Complement	Number Systems Introduction - Decimal, Binary, Octal & Hexadecimal	The Organic Chemistry Tutor	10:57	verified video
2.4 Number Systems, Binary and Two's Complement	Twos complement: Negative numbers in binary	Ben Eater	13:49	verified video
2.5 Divisibility, GCD/LCM, Primes and Modular Arithmetic	Prime Factorization to find GCD and LCM of two numbers! (So easy!)	Quoc Dat Phung	5:21	verified video
2.5 Divisibility, GCD/LCM, Primes and Modular Arithmetic	L3. Check if a Number if Prime or not | Maths Playlist	take U forward	7:10	verified video
2.6 Recursion: Base Case, Recursive Case and the Call Stack	Recursion in Java Full Tutorial - How to Create Recursive Methods	Coding with John	11:11	verified video
2.6 Recursion: Base Case, Recursive Case and the Call Stack	Fibonacci Sequence - Anatomy of recursion and space complexity analysis	mycodeschool	9:39	verified video
2.7 References, Pass-by-Value and Memory	Java is ALWAYS Pass By Value. Here's Why	Coding with John	5:22	verified video
2.7 References, Pass-by-Value and Memory	Null Pointer Exceptions In Java - What EXACTLY They Are and How to Fix Them	Coding with John	14:20	verified video
2.8 Boundary Conditions, Validation and Common Mistakes	Error in Arrays:Array index out of bounds exception in java with example	Bhargava Teja Maddula	2:57	verified video
2.8 Boundary Conditions, Validation and Common Mistakes	Boundary Value Testing | Black Testing | Software Engineering with Real Life examples	Gate Smashers	6:13	verified video
3.1 Classes, Objects, Fields, Constructors and this	This Keyword in Java Full Tutorial - How to Use "this"	Coding with John	7:39	verified video
3.1 Classes, Objects, Fields, Constructors and this	Java OOPs in One Shot | Object Oriented Programming | Java Language | Placement Course	Apna College	1:06:27	verified video
3.2 Encapsulation and Access Modifiers	Java Tutorial: Access modifiers, getters & setters in Java	CodeWithHarry	12:56	verified video
3.2 Encapsulation and Access Modifiers	Access Specifiers in Java | Public Private Protected Default | Explained with Examples | JAVA Course	Jenny's Lectures CS IT	38:36	verified video
3.3 Inheritance, Overriding and Polymorphism	#6.4 Java Tutorial | Method Overriding | Super Keyword	Telusko	4:59	verified video
3.3 Inheritance, Overriding and Polymorphism	#56 Dynamic Method Dispatch in Java	Telusko	8:04	verified video
3.4 Abstract Classes and Interfaces	Java Tutorial: Abstract Classes Vs Interfaces	CodeWithHarry	14:50	verified video
3.4 Abstract Classes and Interfaces	8.22 Interface in Java 8 Default , Static Methods | New features	Telusko	18:20	verified video
3.5 static, final and Initialisation Order	#57 Final keyword in java	Telusko	6:15	verified video

Lesson	Resource	Channel	Length	Type
3.5 static, final and Initialisation Order	#39 Static Block in java	Telusko	7:18	verified video
3.6 Packages and Imports	Java Packages and Import Statements - Java programming	Will Tollefson	10:43	verified video
3.6 Packages and Imports	#53 Packages in Java	Telusko	12:20	verified video
3.7 Exceptions: try, catch, finally, throw and throws	Java Exception Handling in One Shot | try-catch, finally & throw vs throws	The Curious Coder	35:54	verified video
3.7 Exceptions: try, catch, finally, throw and throws	Checked vs. Unchecked Exceptions in Java Tutorial - What's The Difference?	Coding with John	10:14	verified video
3.8 Generics and Lists: ArrayList and LinkedList	LinkedList vs ArrayList in Java Tutorial - Which Should You Use?	Coding with John	11:43	verified video
3.8 Generics and Lists: ArrayList and LinkedList	14.5 LinkedList vs ArrayList in Java	Telusko	9:16	verified video
3.9 Sets and Maps: HashSet, TreeSet, HashMap, TreeMap	Map and HashMap in Java - Full Tutorial	Coding with John	10:10	verified video
3.9 Sets and Maps: HashSet, TreeSet, HashMap, TreeMap	Hashmap in Java | Internal Working of Hashmap in Java | Hashmap Implementation | DSA-One Course #30	Anuj Kumar Sharma	16:39	verified video
3.10 Queues, Deques, Stacks and PriorityQueue	#13 Queue Implementation using Java Part 1 | EnQueue	Telusko	9:38	verified video
3.10 Queues, Deques, Stacks and PriorityQueue	Learn Priority Queue data structures in 5 minutes	Bro Code	5:01	verified video
3.11 Sorting with Comparable and Comparator	#95 Comparator vs Comparable in Java	Telusko	15:43	verified video
3.11 Sorting with Comparable and Comparator	Java Custom Lambda Expression In Arrays.sort and Collections.sort?	Pepcoding	34:46	verified video
3.12 equals(), hashCode() and Immutability	02. Equals and HashCode Contract & Different Variations	Ankit Wasankar	11:19	verified video
3.12 equals(), hashCode() and Immutability	Immutable Classes and Objects in Java	Neso Academy	5:06	verified video
4.1 Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables	Data Structures Made Easy:Animated Explanations of Arrays,Linked Lists,Stacks,Queues,Heaps, HashMaps	Coding Society	4:35	verified video
4.1 Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables	#5 Linked List Implementation in Java Part 1 | Data Structures	Telusko	23:05	verified video
4.2 Trees, Graphs and Heaps	Lec-53: Binary Search Tree in Data Structure | Insertion and Traversal in BST	Gate Smashers	11:02	verified video
4.2 Trees, Graphs and Heaps	Graphs: Edge List, Adjacency Matrix, Adjacency List, DFS, BFS - DSA Course in Python Lecture 11	Greg Hogg	32:11	verified video
4.3 Searching: Linear and Binary Search	Binary Search in Java - Full Simple Coding Tutorial	Coding with John	17:48	verified video
4.3 Searching: Linear and Binary Search	Time Complexity of Linear Search and Binary Search || Lesson 33 || Algorithms || Learning Monkey ||	Wisdomers - Computer Science and Engineering	7:53	verified video

Lesson	Resource	Channel	Length	Type
4.4 Elementary Sorting: Bubble, Insertion, Selection; Stability	Sorting Algorithms | Bubble Sort, Selection Sort & Insertion Sort | DSA Series by Shradha Ma'am	Apna College	34:33	verified video
4.4 Elementary Sorting: Bubble, Insertion, Selection; Stability	What are Adaptive/ In-Place/ Stable Algorithms? | All Sorting algo Explained | Algorithm | Gate CSE	GATE for Geeks	8:10	verified video
4.5 Merge Sort and Divide and Conquer	2.7.2. Merge Sort Algorithm	Abdul Bari	20:23	verified video
4.5 Merge Sort and Divide and Conquer	Learn Merge Sort in 13 minutes	Bro Code	13:45	verified video
4.6 Iteration versus Recursion	Recursion vs Iteration : Understand with Printing Fibonacci Series Example	PrepBytes	12:52	verified video
4.6 Iteration versus Recursion	Fibonacci Sequence - Recursion with memoization	mycodeschool	9:12	verified video
4.7 Time Complexity and Big-O Notation	Big O Notation, Time Complexity | DSA	Telusko	21:17	verified video
4.7 Time Complexity and Big-O Notation	Iterative Time Complexity Analysis | Examples | O(1) - O(n) - O(logn) - O(√n) - O(m+n) - O(n^2)	Badhon Nandi	13:13	verified video
4.8 Algorithm Design Paradigms	L-5.1: Introduction to Dynamic Programming | Greedy Vs Dynamic Programming | Algorithm(DAA)	Gate Smashers	9:08	verified video
4.8 Algorithm Design Paradigms	DP#2: Coin Change Problem Number of ways to get total | Dynamic Programming | Algorithms	Jenny's Lectures CS IT	22:32	verified video
4.9 Analysing Loops: Counting Iterations	Time Complexity of Nested Loops (Independent Loops)	Neso Academy	15:44	verified video
4.9 Analysing Loops: Counting Iterations	Time Complexity of Nested Loops (Dependent Loops)	Neso Academy	13:14	verified video
4.10 Analysing Recursion: Recurrence Relations	L-2.9: Recurrence Relation T(n)= 2T(n/2) +cn | Recursive Tree method | Algorithm	Gate Smashers	7:14	verified video
4.10 Analysing Recursion: Recurrence Relations	L-2.6: Recurrence Relation T(n)= 8T(n/2) + n^2 | Master Theorem | Example#1 | Algorithm	Gate Smashers	6:34	verified video
4.11 Space Complexity and the Call Stack	Space Complexity of Recursive Algorithms (The Call Stack)	Neso Academy	13:55	verified video
4.11 Space Complexity and the Call Stack	Time and Space Complexity - Strivers A2Z DSA Course	take U forward	35:16	verified video
4.12 Best, Average and Worst Case	1.11 Best Worst and Average Case Analysis	Abdul Bari	18:56	verified video
4.12 Best, Average and Worst Case	L-1.3: Asymptotic Notations | Big O | Big Omega | Theta Notations | Most Imp Topic Of Algorithm	Gate Smashers	14:25	verified video
5.1 Euclid's Algorithm (GCD)	GCD - Euclidean Algorithm (Method 1)	Neso Academy	14:38	verified video
5.1 Euclid's Algorithm (GCD)	Euclidean Algorithm (GCD). Java	Aleksandr Azimut	8:08	verified video
5.2 Euler's Phi (Totient) Function	Euler's Totient Function (Phi Function)	Neso Academy	8:40	verified video

Lesson	Resource	Channel	Length	Type
5.2 Euler's Phi (Totient) Function	Euler's Totient Function (Solved Examples)	Neso Academy	12:53	verified video
5.3 Chinese Remainder Theorem	The Chinese Remainder Theorem (Solved Example 1)	Neso Academy	14:22	verified video
5.3 Chinese Remainder Theorem	The Chinese Remainder Theorem (Solved Example 2)	Neso Academy	12:06	verified video
5.4 Combinations (nCr)	The Combination Formula and Why it Works | Combinatorics, Combinations and Permutations	Wrath of Math	8:14	verified video
5.4 Combinations (nCr)	Pascal Triangle | Finding nCr in minimal time	take U forward	26:45	verified video
5.5 Karatsuba's Multiplication	1 3 Karatsuba Multiplication 13 min	Stanford Algorithms	12:40	verified video
5.5 Karatsuba's Multiplication	Understanding Karatsuba Algorithm based on Divide and Conquer Implementation	MathPhy	20:36	verified video
5.6 Booth's Algorithm (signed binary multiplication)	Booth's Algorithm for Signed Multiplication	TutorialsPoint	14:32	verified video
5.6 Booth's Algorithm (signed binary multiplication)	Booth's algorithm - Binary multiplication example | Computer Organization	Scholarly Things	5:39	verified video
5.7 Simple Sieve of Eratosthenes	Sieve of Eratosthenes | Journey into cryptography | Computer Science | Khan Academy	Khan Academy Labs	4:12	verified video
5.7 Simple Sieve of Eratosthenes	L6. Sieve of Eratosthenes | Maths Playlist	take U forward	18:27	verified video
5.8 Segmented & Incremental Sieve	Segmented Sieve | Sample Video II for Essential Maths for CP | GeeksforGeeks	GeeksforGeeks	11:42	verified video
5.8 Segmented & Incremental Sieve	15.1 Sieve of Eratosthenes | Challenge | C++ Placement Course	Apna College	10:47	verified video
5.9 Binary Palindrome	C++ Challenge: Check If a Number's Binary Form Is a Palindrome	Atshk.h programing	6:09	verified video
5.9 Binary Palindrome	Reverse Bits - Binary - Leetcode 190 - Python	NeetCode	10:13	verified video
5.10 Swap Two Nibbles in a Byte	Swap two nibbles in a byte | GFG POTD | Bit Manipulation | CPP | AlgoBazaar	AlgoBazaar	9:06	verified video
5.10 Swap Two Nibbles in a Byte	Bitwise Operators in C (Part 2)	Neso Academy	4:59	verified video
5.11 Longest Sequence of 1s (and after flipping a bit)	Maximizing Contiguous Ones: Find the Longest Sequence in a Binary Array with a Single Flip | Medium	Tutorial Horizon	12:00	verified video
5.11 Longest Sequence of 1s (and after flipping a bit)	L4. Max Consecutive Ones III | 2 Pointers and Sliding Window Playlist	take U forward	29:58	verified video
5.12 Josephus Problem	The Josephus Problem - Numberphile	Numberphile	13:58	verified video
5.12 Josephus Problem	Josephus Problem | GeeksforGeeks	GeeksforGeeks	12:13	verified video
5.13 Block Swap Algorithm (Array Rotation)	Block swap algorithm for Array rotation || easy explanation	Code with Med	13:42	verified video
5.13 Block Swap Algorithm (Array Rotation)	Rotate Array - Leetcode 189 - Python	NeetCode	8:59	verified video

Lesson	Resource	Channel	Length	Type
5.14 Leaders in an Array	Leaders in an Array | Brute - Optimal | Strivers A2Z DSA Course	take U forward	11:53	verified video
5.14 Leaders in an Array	Leaders in an Array | Right-to-Left Scan	Beyond the Function	4:47	verified video
5.15 Majority Element (Boyer–Moore Voting)	Majority Element I | Brute-Better-Optimal | Moore's Voting Algorithm | Intuition |Brute to Optimal	take U forward	18:13	verified video
5.15 Majority Element (Boyer–Moore Voting)	Majority Element - Leetcode 169 - Python	NeetCode	14:39	verified video
5.16 Maximum Product Subarray	Maximum Product Subarray - Best Intuitive Approach Discussed	take U forward	20:27	verified video
5.16 Maximum Product Subarray	Maximum Product Subarray - Dynamic Programming - Leetcode 152	NeetCode	15:31	verified video
5.17 Maximum Equilibrium Sum	Equilibrium Index of Array | O(n) Prefix-Sum Solution | An easy detailed solution	dsa-made-easy	12:38	verified video
5.17 Maximum Equilibrium Sum	Prefix Sum Array and Range Sum Queries	Profound Academy	7:30	verified video
5.18 Maximum Sum of an Hourglass in a Matrix	2D Array- DS - Hourglass |ARRAYS |HACKERRANK Solution| PYTHON | Interview |Programming for Beginners	Atharv Mudur	6:53	verified video
5.18 Maximum Sum of an Hourglass in a Matrix	C_56 2D Arrays Program 1| Program to print Matrix and calculate Sum | C Programming	Jenny's Lectures CS IT	17:33	verified video
5.19 Toggle the Switch (100 Doors)	100 Doors in a Row | Perfect Square Numbers Magic | Train your Brain | Puzzle	PRiyanka Singh	9:49	verified video
5.19 Toggle the Switch (100 Doors)	100 Doors Interview Puzzle || Microsoft Interview Puzzle	Simply Logical	5:26	verified video
5.20 Alice's Apple Tree (Minimum Garden Perimeter)	Minimum Garden Perimeter To Collect Enough Apples leetcode weekly contest 252 Leetcode 1954.	CodeinDepth	9:51	verified video
5.20 Alice's Apple Tree (Minimum Garden Perimeter)	Binary Search | Monotonic Function | L-29	Programming Pathshala	11:57	verified video
5.21 Weighted Substring (Smallest String of a Given Weight)	Smallest String With A Given Numeric Value | Leetcode 1663 | Maths | Live coding session	Coding Decoded	6:44	verified video
5.21 Weighted Substring (Smallest String of a Given Weight)	Introduction to Greedy Algorithms | GeeksforGeeks	GeeksforGeeks	5:32	verified video
5.22 Move Hyphen to the Beginning	Move special characters in String | String Assignment 1 | Programming Using Java	DeCode with Shreyas	3:08	verified video
5.22 Move Hyphen to the Beginning	Lec-10: Two 2 Pointer Technique | Two 2 Sum Problem in Data Structure	Gate Smashers	10:33	verified video
5.23 Strobogrammatic Number	STROBOGRAMMATIC NUMBER | LEETCODE # 246 | PYTHON SOLUTION	Cracking FAANG	9:50	verified video
5.23 Strobogrammatic Number	Valid Palindrome Strings | Two Pointer Approach Explained | Python, C++ & Java	RisingBrain	14:03	verified video
5.24 Lexicographically First Palindromic Rearrangement	Lexicographically Smallest Palindrome | Leetcode 2697 | Approach + Code with explanation.	Pirai's Academy	8:28	verified video

Lesson	Resource	Channel	Length	Type
5.24 Lexicographically First Palindromic Rearrangement	LeetCode 266. Palindrome Permutation (Algorithm Explained)	Nick White	5:53	verified video
5.25 Manacher's Algorithm (Longest Palindromic Substring)	Longest Palindromic Substring Manacher's Algorithm	Tushar Roy - Coding Made Simple	16:46	verified video
5.25 Manacher's Algorithm (Longest Palindromic Substring)	5. Longest Palindromic Substring | Part-1: Expand around centers #leetcode	CodeWonka : Get started with programming	7:55	verified video
5.26 Natural Sort Order	Java Comparator Sorting in Java | Practical Tutorial with Eclipse Examples | Hands on | Red9SysTech	Red9SysTech	3:52	verified video
5.26 Natural Sort Order	Sort Strings in Descending Order in Java | Custom Comparator Example | Java Programming	The Corporate Villager	5:30	verified video
5.27 Maneuvering (Counting Grid Paths)	Unique Paths - Dynamic Programming - Leetcode 62	NeetCode	10:48	verified video
5.27 Maneuvering (Counting Grid Paths)	Ways to Reach Origin | GFG Problem of the Day | Grid Paths DP & nCr Explained	Gopo workspace	10:43	verified video
5.28 Sorted Distinct Permutations	Permutations of an Array/String | Recursion & Backtracking	Apna College	22:55	verified video
5.28 Sorted Distinct Permutations	Next Permutation - Intuition in Detail | Brute to Optimal	take U forward	28:15	verified video
5.29 N-Queens (Backtracking)	6.1 N Queens Problem using Backtracking	Abdul Bari	13:41	verified video
5.29 N-Queens (Backtracking)	Visualization of N Queens (5×5 board)	G Bhat	2:39	verified video
5.30 Maze Solving (Rat in a Maze)	L19. Rat in A Maze | Backtracking	take U forward	25:10	verified video
5.30 Maze Solving (Rat in a Maze)	Rat in a Maze Problem | Backtracking	Apna College	32:45	verified video
5.31 Hamiltonian Cycle	6.4 Hamiltonian Cycle - Backtracking	Abdul Bari	18:35	verified video
5.31 Hamiltonian Cycle	Euler Graph in Graph Theory | Euler Path & Euler Circuit with examples	Gate Smashers	6:45	verified video
5.32 Knight's Tour & Warnsdorff's Rule	CSES Knight's Tour | Backtracking + Warnsdorff's Rule Explained | Full Intuition + Code in C++	Yash Poonia	16:32	verified video
5.32 Knight's Tour & Warnsdorff's Rule	Knight's Tour - Numberphile	Numberphile	4:51	verified video
5.33 Selection Sort	7.5 Selection Sort in Data Structure | Selection Sort Algorithm with C Program	Jenny's Lectures CS IT	17:49	verified video
5.33 Selection Sort	L-3.8: Selection Sort | Time Complexity(Best, Avg & Worst) Analysis | Stable or Not | Inplace or Not	Gate Smashers	11:37	verified video
5.34 Quick Sort	2.8.1 QuickSort Algorithm	Abdul Bari	13:43	verified video
5.34 Quick Sort	Quicksort Sort Algorithm in Java - Full Tutorial With Source	Coding with John	24:58	verified video

Appendix D — Archive coverage checklist

Tick a deck when you can solve its problems without looking.

	Slide deck	Slides	Taught in lesson(s)
☐	Java Introduction: Features, Structure, Data Types	17	0.1, 0.2, 0.3, 0.4, 1.1
☐	Basic Input/Output and Operators	27	0.4, 1.2, 1.3, 1.5, 1.6, 1.7
☐	Introduction to Algorithms	4	0.6
☐	Decision Making and Control Structures	29	1.8, 1.9
☐	Time and Space Complexity (part 1)	34	1.9, 4.7, 4.9
☐	Time and Space Complexity (part 2)	11	2.6, 4.7, 4.10, 4.11
☐	Euclid's Algorithm	10	2.5, 5.1
☐	Euler's Phi (Totient) Function	7	2.5, 5.2
☐	Chinese Remainder Theorem	7	2.5, 5.3
☐	Combinations (nCr)	7	3.8, 5.4
☐	Karatsuba Multiplication	8	4.5, 4.8, 4.10, 5.5
☐	Booth's Multiplication	14	2.4, 5.6
☐	Sieve of Eratosthenes	6	2.5, 5.7
☐	Segmented and Incremental Sieve	4	5.8
☐	Binary Palindrome	7	2.4, 5.9
☐	Swap Two Nibbles in a Byte	7	2.4, 5.10
☐	Longest Sequence of 1s after Flipping a Bit	10	5.11
☐	Josephus Problem	13	2.6, 3.10, 4.6, 5.12
☐	Block Swap (Array Rotation)	12	2.1, 5.13
☐	Leaders in an Array	10	5.14
☐	Majority Element	19	5.15
☐	Maximum Product Subarray	8	5.16
☐	Maximum Equilibrium Sum	14	5.17
☐	Maximum Hourglass Sum in a Matrix	8	2.2, 5.18
☐	Toggle the Switch and Alice's Apple Tree	8	4.3, 5.19, 5.20
☐	Weighted Substring	7	3.11, 4.3, 4.8, 5.21
☐	Move Hyphen to the Beginning	3	2.3, 5.22
☐	Strobogrammatic Number	6	3.9, 5.23
☐	Lexicographically First Palindromic String	14	3.9, 5.24
☐	Manacher's Algorithm	6	2.3, 5.25
☐	Natural Sort Order	5	2.3, 3.8, 3.11, 5.26

	Slide deck	Slides	Taught in lesson(s)
☐	Maneuvering (Grid Paths)	7	2.6, 4.6, 4.8, 5.27
☐	Sorted Unique Permutations	6	3.9, 5.28
☐	N-Queens	11	4.8, 5.29
☐	Maze Solving	7	2.2, 3.10, 4.2, 5.30
☐	Hamiltonian Cycle	7	4.2, 5.31
☐	Quick Sort and Selection Sort	8	4.4, 4.5, 5.33, 5.34
☐	Knight's Tour (Warnsdorff's Rule)	8	5.32

Appendix E — Where the slides were corrected

Each algorithm was re-derived and checked by running code (and, for small inputs, a brute-force comparison). Differences from the slides:

0.1 How Java Works: Source, Bytecode, JVM, JRE, JDK

- The slide "Why is JAVA slow?" says Java is slow because linking happens at run time; modern JVMs with JIT compilation are competitive with C++ for long-running programs. Start-up time is where Java is slower.

0.6 What Is an Algorithm?

- The Introduction deck is a single definition and one MCQ ("What is an algorithm?" answer: b). The properties listed here are the standard textbook definition.

1.3 Arithmetic, Assignment and Increment Operators

- The "Basic I/O & Operators" deck introduces printing, input and arithmetic together; operators are covered here and I/O in the next two lessons.

1.7 Input: Scanner, BufferedReader and Command-Line Arguments

- The slide example `avg = (args[0]+args[1]+…)/5` does not compile (String / int); the slide then fixes it with Float.valueOf — as the lesson shows, convert BEFORE adding.
- On the Console slide the code `System.console().readLine()` returns null in IDEs and when input is piped; use Scanner for programs that must work everywhere.

2.1 One-Dimensional Arrays

- The Block Swap slide introduces arrays using C-like syntax (`Int Arr[5]= {…}`, `Char Arr[7]={…}` with only 6 initialisers). Java declares `int[] arr = {…};` and the size is taken from the initialiser.

2.4 Number Systems, Binary and Two's Complement

- The Booth deck revises binary addition and two's complement only as screenshots (21 → 10101 by repeated division); this lesson makes those prerequisites explicit.

4.5 Merge Sort and Divide and Conquer

- Merge sort is not itself a slide topic; it is added because divide and conquer (Karatsuba, quick sort) needs a clean worked example and "merge sort" appears in the lecture's Big-O cheat sheet as the O(n log n) example.

4.7 Time Complexity and Big-O Notation

- The slide on the time-complexity chart says quadratic time complexity "increases exponentially with the input size"; quadratic is polynomial.
- Slide 10 attributes $O(n^2 \cdot 2^n)$ to brute-force travelling salesman and slide 11 O(n!); both describe brute-force enumeration — the Held–Karp DP achieves $O(n^2 \cdot 2^n)$.

4.9 Analysing Loops: Counting Iterations

- Slide 22: the written derivation of $O(\log^2 n)$ is incorrect; the total is O(n log n) (the multiple-choice option C is the right one).
- Slide 26: option D (n^4) is correct but the explanation concludes n^3.
- Slide 20 claims that with `break` the loop is O(n) — correct — but its prose says "the inner loop always runs only once", which is also correct; the worry is only the suggested answer list that omits O(n) in that choice set.
- Slide 33 correctly derives $O(\sqrt{n})$ for `s = s + i`.

4.11 Space Complexity and the Call Stack

- Slide 9 answers O(n) for recursive fibonacci space — correct (depth n), but the printed reason talks about "recursive calls n times"; precisely it is the depth of the call tree, n, not the number of calls.

5.1 Euclid's Algorithm (GCD)

- The slide pseudocode states the prerequisite "a > b". It is not needed: if a < b the first step swaps them (gcd(12, 50) → gcd(50, 12)).

5.2 Euler's Phi (Totient) Function

- The slide computes φ(60) = 32. The correct value is 16: 60 × (1/2) × (2/3) × (4/5) = 16. The slide repeated factors for the primes 2 and 5. The Java code on the slide is correct and returns 16.
- The slide's product formula φ(n) = (p₁−1)(p₂−1)…·p₁^(k₁−1)… is right, but its intermediate text for φ(60) lists the factors of 5 twice.

5.3 Chinese Remainder Theorem

- The slide gives the answer as "x ≡ 233 (mod 105)". That is a correct congruence but the canonical answer is the reduced value 23.
- The slide's Java program has a compile error (`for(int i=0; i<n; i++)` uses n, which is never declared — it should be `size`) and its brute-force loop never terminates when no solution exists. The versions in this lesson fix both.

5.4 Combinations (nCr)

- The slide places an ArrayList of 5 numbers into the program, but only list.size() is ever used, so the "9, 12, 19, 61, 19" values have no effect on the result (10).
- The slide's factorial uses int and would overflow for n ≥ 13 — fine for the sample, wrong in general.

5.5 Karatsuba's Multiplication

- The slide code contains `long ad+bc = …` and later uses `adbc`; neither compiles. The slide also uses Math.pow (floating point) to build powers of ten.
- The slide's explanation labels z0 = ac in one paragraph and z0 = bd in the next. In this lesson ac is the high product and bd the low product, consistently.
- The slide says the three-product count "reduces overall complexity" but never states it: it is O(n^1.585) versus O(n²).

5.6 Booth's Algorithm (signed binary multiplication)

- The Booth deck is entirely screenshots from a video lecture; it has no Java code. The program here is original and verified exhaustively.

5.7 Simple Sieve of Eratosthenes

- The slide walk-through for n = 20 is correct, but its last numbered step is cut off ("11. The rem…"); the final answer is 2 3 5 7 11 13 17 19.

5.8 Segmented & Incremental Sieve

- The slide's SegSieve does not compile (`False`, a commented Arrays.fill) and, if repaired naively, marks small primes in the window as composite and ignores the p·p optimisation.
- The slide's array has size h + 1 (memory proportional to R), which defeats the purpose of segmenting.

5.9 Binary Palindrome

- The slides list 16 (10000) as a non-palindrome and 17 (10001) as a palindrome — both correct. The explanation text calls the reversed value "00001"; mathematically the reversed integer is 1.

5.10 Swap Two Nibbles in a Byte

- The slide's English steps are written as Python (`swap_nibbles(byte)`, `bin()`) but the program on the next slide is Java — the Java is what this lesson explains. In the example, the slide moves from `1010 1100` (0xAC) to `1100 1010` (0xCA), while its code demo uses 0xAB → 0xBA; both are consistent.

5.11 Longest Sequence of 1s (and after flipping a bit)

- The slide states that 0b1101110111111001 equals 87769 and that its longest run is 5. It equals 56825 and the longest run is 6.
- The slide's Longafterflip code uses Integer.toBinaryString and char loops; its last line `maxLength = max(maxLength, currentLength + previousLength + 1)` yields 33 for -1. This lesson caps the answer at 32.

5.12 Josephus Problem

- The slide's explanatory text says "recursively delete the Kth position until only one person is left", but its code does not delete anything: it uses the arithmetic recurrence. The slide diagram (steps 01–06) uses start = (start + k) % size with k = 1 in 0-based form, which is the same idea as the recurrence run forwards.

5.13 Block Swap Algorithm (Array Rotation)

- Example 2 on the slide prints {5,1,2,3,4,6,8,7,9,0}; the correct result is {5,1,3,2,4,6,8,7,9,0}.
- The slide says "O(log n)" auxiliary space for the recursive version. The depth can be O(n); the iterative version is O(1).
- The slide's `leftRotate` and the helper `swap` appear in a confusing order across two slides; they belong to the same class.

5.14 Leaders in an Array

- The Leaders deck is a screen-recorded video presentation (slides are images). Its right-to-left code prints leaders in reverse order (2 5 17) — the lesson prints them left-to-right to match the definition.

5.15 Majority Element (Boyer–Moore Voting)

- The slide cites https://www.youtube.com/watch?v=fCGLjlsEsNA as the video for this topic. YouTube's own metadata shows that video is titled "Maximum average subarray of size k" (IDeserve) — it is not about majority elements. It is NOT used in this library.

5.16 Maximum Product Subarray

- The slide prints `{4, 0, −2, 6}` products with a stray "x1 = ..." notation, but its conclusion (6) is correct.
- The slide code loops `i < 4` regardless of array length and uses a three-way branch with a temp variable; the swap form in this lesson is equivalent and shorter.

5.17 Maximum Equilibrium Sum

- The Java program on the last slide returns Integer.MIN_VALUE when there is no equilibrium index; the lesson uses an explicit "none".
- The deck explains the algorithm with a sequence of animated screenshots using the 1-based position `pos = i + 1`; Java indices here are 0-based.

5.18 Maximum Sum of an Hourglass in a Matrix

- The slide initialises `max_sum = 0` (wrong for negative data) and exits the JVM with `System.exit(0)` inside the helper. The lesson program avoids both.
- The slide's worked sums (24, 22, 34, 41) are correct.

5.19 Toggle the Switch (100 Doors)

- The slide's `Toggle` program toggles b[j] for j from i while j·i ≤ n. That is not "every i-th door"; running it for n = 100 reports many more than 10 open doors. The lesson uses the correct inner loop `door += monkey`.
- The deck combines two problems (this one and "Alice's Apple Tree"); each has its own lesson.

5.20 Alice's Apple Tree (Minimum Garden Perimeter)

- The slide's code uses int and a linear loop although the text on the previous slide says "then we apply binary search". Both appear here; the long type is required for the full constraint range (this is LeetCode 1954, "Minimum Garden Perimeter to Collect Enough Apples").

5.21 Weighted Substring (Smallest String of a Given Weight)

- The slide says the output "should be in the form of integer" but its sample output is the string AABBC for input 20 (a string, not an integer).
- The slide stores weights in int[26] (overflows) and reads the weight with nextInt although the constraint is up to 10^{16}.

5.22 Move Hyphen to the Beginning

- The slide moves only the first hyphen (by name it suggests "the hyphen"); the generalisation to all hyphens is added here and labelled as such.

5.23 Strobogrammatic Number

- The slide says digits 2 and 5 "can also be considered strobogrammatic" and that 6 and 9 "are not"; the standard definition (and the slide's own Java code) uses {0, 1, 8} as self-rotating and {6 ↔ 9} as a pair.
- The slide explains 906 as "looks like 609". 906 is strobogrammatic because the rotated number equals 906 itself.

- The slide program is split across two slides with numbered lines (22–30) that continue the class; the lesson program is the whole class.

5.24 Lexicographically First Palindromic Rearrangement

- The slide program starts `rear_str = " "` (a space) and subtracts `'a'` from characters although its first example input is "Malayalam" with a capital M; the lesson program lowercases and fixes the initial value.
- The slide (slide 4) states the expected output for "Malayalam" as "aalmymlaa", which is correct for the lowercased word.

5.25 Manacher's Algorithm (Longest Palindromic Substring)

- The slide listing of the Java program stops in the middle of the main loop (slide 5 ends after the first while loop and slide 6 is "Thank you"), so the code is incomplete. The lesson provides a complete, tested implementation.
- The slide text (steps 1–12) describes the version with sentinels "@" and "$" and array `p`; the Java snippet uses a different (GeeksforGeeks-style) layout with array `L`. The lesson implements the classical radius-array formulation that both describe.

5.26 Natural Sort Order

- The slide's `Comparator.comparing(NaturalSort::extractNumericParts)` does not compile (List is not Comparable) and compares digits as text. The lesson replaces it with a chunk comparator.
- The slide text describes sorting by the numeric part "in ascending order, otherwise regular string comparison" — which ignores that prefix text must be compared first.

5.27 Maneuvering (Counting Grid Paths)

- The slide says the problem is "Maneuvering a Cave"; the deck contains only the grid-path recurrence (a classic problem). Its trace of numberOfPaths(3, 3) is correct and ends at 6.
- The slide mentions "already calculated" for numberOfPaths(2, 2) — the plain recursive code does NOT cache; the second call is recomputed (that is the inefficiency this lesson addresses).

5.28 Sorted Distinct Permutations

- The slide claims its program prints distinct permutations "in sorted order". Running it on "BAC" prints ABC ACB BAC BCA CBA CAB, which is not sorted. The slide's own "Steps" (sort, count permutations, then find the next greater string repeatedly) describe the NEXT-PERMUTATION method, which IS sorted — this is what `NextPermutation.java` implements.
- The slide's example output for "DBCA" lists the 24 sorted permutations (correct).

5.29 N-Queens (Backtracking)

- The slide fills the board column by column (`col`), so its printed solution is the transpose/mirror of the one a row-by-row search prints. Both are valid.
- The slide says "queen can move any number of steps in any direction … can't change direction" and then lists only rows and columns; diagonals are an essential part of the attack rule.

5.30 Maze Solving (Rat in a Maze)

- The slide models the maze as 0 road / 1 wall / 2 entry / 3 exit / 4 path, but its Java program uses 1 for an open cell and only moves down/right. The lesson provides one program for each convention and states which is which.
- The slide's printed solution `1 0 0 0 / 1 1 0 0 / 0 1 0 0 / 0 1 1 1` matches `RatMazeDownRight`.
- The 5×5 matrix at the end of the slide has no stated solution; it needs up/left moves, which is why a visited-array solver is provided.

5.31 Hamiltonian Cycle

- The slide listing of the Java class is scrambled over two slides (the `hamCycle` method header is missing and part of `main` is duplicated), so it does not compile as printed. The lesson program is the complete, working version.
- The slide's final output line "0 1 2 4 3 0" is printed after `graph2`, but that graph has NO Hamiltonian cycle; the output belongs to `graph1`.

5.32 Knight's Tour & Warnsdorff's Rule

- This deck is named `downloadCourseMaterialFacultyPdf.pptx` and has no topic in its file name; its content is Warnsdorff's algorithm for the knight's tour.
- The slide's class is not complete Java (e.g. `int y` without a semicolon, and the method `findClosedTour` is declared without a `return` on all paths in the screenshot). The lesson program is a clean, tested rewrite.
- The slide uses `ThreadLocalRandom` to pick the first move to examine and loops until a CLOSED tour is found; its sample output is a closed tour starting at square (3, 2).

5.33 Selection Sort

- The slide shows each iteration as "sorted [11], unsorted [64, 25, 12, 22]", which looks like the minimum was inserted at the front and the rest shifted. The slide's code swaps instead, giving [25, 12, 22, 64] after pass 1. The program output matches the code.
- The slide describes Quick sort and Selection sort in the same deck; they are split into two lessons here.

5.34 Quick Sort

- The slide's worked example gives the partition result `[1, 0, 2, 7, 8, 10, 3]`; the code produces `[1, 0, 2, 7, 3, 10, 8]`. Then the slide partitions `[7, 8, 10, 3]` with pivot 3 as `[7, 8, 3, 10]`, which is also not what Lomuto does (`[3, 8, 10, 7]`). The final sorted result on the slide is correct.